

DriveU

Complete Project Workflow & File-by-File Code Explanation

From the moment the server starts to the very last file: what runs first, what every file is for, what every line of code does, and where control goes next.

Frontend	Angular 16 (non-standalone modules, reactive forms, RxJS) - port 8081
Backend	Spring Boot 3.0.1, Java 17, Spring Security + JWT, Spring Data JPA / Hibernate - port 8080
Database	H2 in-memory (MySQL driver present for the SRS database appdb)
AI features	Google Gemini (optional) with a built-in local fallback
Source files explained	153 files, 8,395 lines - every non-blank line is shown and explained
Team	Aleena, Vanshika, Arjun, Prajan, Saumya, Aarti, Vaishalini

Owner names in square brackets, e.g. **[Arjun - Driver & Booking]**, follow the Sprint Team Allocation document.

Table of Contents

How to read this document	6
DriveU at a glance	8
The big picture	9
Complete file index	12
Part 1 - Starting the backend: from mvn spring-boot:run to a listening server	16
Step 1: pom.xml - the Maven build file [Vaishalini]	16
Step 2: application.properties - runtime settings [Vaishalini]	18
Step 3: SpringappApplication - the main() entry point [Vaishalini]	20
Step 4: User - the people who can log in (table users) [Aleena]	21
Step 5: LoginDTO - the answer to a successful login [Aleena]	23
Step 6: Driver - the drivers that can be hired (table driver) [Arjun]	25
Step 7: DriverRequest - one booking of a driver (table driver_request) [Arjun]	27
Step 8: Feedback - reviews written by customers (table feedback) [Saumya]	30
Step 9: ErrorLog - a stored record of every handled error (table ErrorLogs) [Vaishalini]	33
Step 10: UserRepo - database access for users [Aleena]	35
Step 11: DriverRepo - database access for drivers [Arjun]	36
Step 12: DriverRequestRepo - queries over booking requests [Arjun]	36
Step 13: FeedbackRepo - queries over feedback [Saumya]	37
Step 14: ErrorLogRepo - saving error records [Vaishalini]	38
Step 15: SecurityConfig - the rulebook for every URL [Vanshika]	39
Step 16: JwtUtils - creating and checking tokens [Vanshika]	43
Step 17: JwtAuthenticationFilter - reads the token on every request [Vanshika]	45
Step 18: MyUserDetailsService - finds a user for Spring Security [Vanshika]	47
Step 19: UserPrinciple - our User as Spring Security sees it [Vanshika]	48
Step 20: JwtAuthenticationEntryPoint - the 401 answer [Vanshika]	50
Step 21: JwtAccessDeniedHandler - the 403 answer [Vanshika]	51
Step 22: CrosConfig - allowing the Angular app to call the API (CORS) [Vanshika]	52
Step 23: UserService - the user business-rule contract [Aleena]	53
Step 24: UserServiceImpl - registering a user [Aleena]	54
Step 25: DuplicateDriverException - license number already used [Vaishalini]	55
Step 26: DriverDeletionException - driver could not be deleted [Vaishalini]	56
Step 27: DriverService - driver business-rule contract [Arjun]	57
Step 28: DriverServiceImpl - add, read, update, delete drivers [Arjun]	58
Step 29: DriverRequestDeletionException - request could not be deleted [Vaishalini]	60
Step 30: DriverRequestService - booking request contract [Arjun]	60
Step 31: DriverRequestServiceImpl - booking logic and partial update [Arjun]	61
Step 32: GeminiService - talking to Google Gemini [Saumya]	63
Step 33: AiService - AI search, sentiment and review summary [Saumya]	66

Step 34: FeedbackService - feedback contract [Saumya]	77
Step 35: FeedbackServiceImpl - saving feedback with AI analysis [Saumya]	77
Step 36: AuthController - register and login endpoints [Aleena]	79
Step 37: DriverController - /api/driver endpoints [Arjun]	81
Step 38: DriverRequestController - /api/driverRequest endpoints [Arjun]	83
Step 39: FeedbackController - /api/feedback endpoints [Saumya]	85
Step 40: AiController - /api/ai endpoints [Saumya]	87
Step 41: GlobalExceptionHandler - one place for all errors [Vaishalini]	88

Part 2 - Starting the frontend: from ng serve to the app shell 91

Step 42: package.json - the npm project file [Vaishalini]	91
Step 43: angular.json - the Angular workspace and build settings [Vaishalini]	93
Step 44: tsconfig.json - base TypeScript compiler options [Vaishalini]	96
Step 45: tsconfig.app.json - TypeScript settings for the app build [Vaishalini]	97
Step 46: index.html - the single HTML page [Vaishalini]	98
Step 47: main.ts - starts the Angular application [Vaishalini]	99
Step 48: polyfills.ts - legacy polyfill file (not used by the build) [Vaishalini]	100
Step 49: environment.ts - environment settings [Vaishalini]	101
Step 50: apiconfig.ts - the API base URL constant [Vaishalini]	102
Step 51: AppModule - the root module [Vaishalini]	103
Step 52: AppRoutingModuleModule - URLs, guards and roles [Vanshika]	105
Step 53: AppComponent - logic of the app shell [Vaishalini]	107
Step 54: app.component.html - navbar choice and router outlet [Vaishalini]	108
Step 55: styles.css - global styles for the whole app [Vaishalini]	109
Step 56: constants.ts - fixed lists and regular expressions [Vaishalini]	117
Step 57: format.ts - date, time and duration helpers [Prajan]	119
Step 58: driver-image.ts - driver photos and the default avatar [Arjun]	121
Step 59: Login model - the credentials object [Aleena]	123
Step 60: User model - a registered user [Aleena]	123
Step 61: Driver model - a driver [Arjun]	124
Step 62: DriverRequest model - a booking request [Arjun]	125
Step 63: Feedback model - customer feedback with AI fields [Saumya]	126
Step 64: DriverFeedbackSummary - the AI summary result [Saumya]	127
Step 65: HttpErrorInterceptor - global HTTP error handling [Vanshika]	127
Step 66: AuthService - register, login, logout and login state [Aleena]	129
Step 67: DriverService - REST calls for drivers [Arjun]	132
Step 68: DriverRequestService - REST calls for trip requests [Arjun]	134
Step 69: FeedbackService - REST calls for feedback [Saumya]	135
Step 70: AiService - AI driver search and feedback summary [Saumya]	136
Step 71: AuthGuard - protecting routes by login and role [Vanshika]	138

Part 3 - The user journey: page by page 140

Step 72: LoginComponent (TypeScript) - the login page logic [Aleena]	140
Step 73: LoginComponent (template) - what the user sees on /login [Aleena]	144

Step 74: LoginComponent (CSS) - component-level style [Aleena]	146
Step 75: SignupComponent (TypeScript) - registration page logic [Aleena]	147
Step 76: SignupComponent (template) - the registration form [Aleena]	150
Step 77: SignupComponent (CSS) - component-level style [Aleena]	152
Step 78: HomePageComponent (TypeScript) - the landing page after login [Vaishalini]	153
Step 79: HomePageComponent (template) - hero, welcome text, contact card [Vaishalini]	154
Step 80: HomePageComponent (CSS) - layout of the landing page [Vaishalini]	155
Step 81: AdminnavComponent (TypeScript) - admin navigation bar logic [Vaishalini]	157
Step 82: AdminnavComponent (template) - admin menu and logout popup [Vaishalini]	159
Step 83: AdminnavComponent (CSS) - component-level style [Vaishalini]	160
Step 84: CustomernavComponent (TypeScript) - customer navigation bar logic [Vaishalini]	161
Step 85: CustomernavComponent (template) - customer menu and logout popup [Vaishalini]	162
Step 86: CustomernavComponent (CSS) - component-level style [Vaishalini]	163
Step 87: DriverManagementComponent (TypeScript) - the Add / Edit Driver form logic [Arjun]	164
Step 88: DriverManagementComponent (HTML) - the driver form template [Arjun]	170
Step 89: DriverManagementComponent (CSS) - file chooser styling [Arjun]	173
Step 90: AdminViewDriversComponent (TypeScript) - driver list, filters, edit, delete, status change [Arjun]	173
Step 91: AdminViewDriversComponent (HTML) - driver cards, toolbar and delete popup [Arjun]	177
Step 92: AdminViewDriversComponent (CSS) - card grid and action menu [Arjun]	179
Step 93: CustomerviewdriverComponent (TypeScript) - Available Drivers, plain search and AI search [Arjun]	181
Step 94: CustomerviewdriverComponent (HTML) - search boxes and driver cards [Arjun]	185
Step 95: CustomerviewdriverComponent (CSS) - search rows and three-column cards [Arjun]	186
Step 96: CustomerRequestComponent (TypeScript) - booking a driver (create) and editing a request [Arjun]	188
Step 97: CustomerRequestComponent (HTML) - the booking form template [Arjun]	192
Step 98: CustomerRequestComponent (CSS) - date and time input height [Arjun]	193
Step 99: AdminviewrequestsComponent - admin list, approve / reject, stages [Aarti]	194
Step 100: adminviewrequests.component.html - admin requests table and modals [Aarti]	198
Step 101: adminviewrequests.component.css - table actions, modals, stage bar [Aarti]	201
Step 102: CustomerviewrequestedComponent - My Requests, trip end and fare [Vaishalini]	202
Step 103: customerviewrequested.component.html - request cards and modals [Vaishalini]	208
Step 104: customerviewrequested.component.css - cards and status colours [Vaishalini]	211
Step 105: CustomerpostfeedbackComponent - the "Submit Feedback" form logic [Saumya]	212
Step 106: customerpostfeedback.component.html - the feedback form page [Saumya]	215
Step 107: customerpostfeedback.component.css - star styles [Saumya]	218
Step 108: CustomerviewfeedbackComponent - "My Feedback" list logic [Saumya]	218
Step 109: customerviewfeedback.component.html - the My Feedback cards [Saumya]	221
Step 110: customerviewfeedback.component.css - card layout and badges [Saumya]	223
Step 111: AdminviewfeedbackComponent - all feedback with filters and AI summary [Saumya]	224
Step 112: adminviewfeedback.component.html - feedback table, filters and modals [Saumya]	227
Step 113: adminviewfeedback.component.css - toolbar, badges, modals, AI panel [Saumya]	230
Step 114: ErrorComponent - the "Something Went Wrong" page [Vaishalini]	231
Step 115: error.component.html - error message markup [Vaishalini]	232
Step 116: error.component.css - error page look [Vaishalini]	232

Part 4 - Appendix: tests, harness and project files

233

Step 117: app.component.spec.ts - tests for the root component [Vaishalini]	233
Step 118: login.component.spec.ts - tests for the login page [Aleena]	235
Step 119: signup.component.spec.ts - tests for the registration page [Aleena]	237
Step 120: home-page.component.spec.ts - tests for the home page [Vaishalini]	239
Step 121: adminnav.component.spec.ts - tests for the admin navigation bar [Vaishalini]	240
Step 122: customernav.component.spec.ts - tests for the customer navigation bar [Vaishalini]	241
Step 123: driver-management.component.spec.ts - tests for the Add / Edit Driver form [Arjun]	243
Step 124: admin-view-drivers.component.spec.ts - tests for the admin driver list [Arjun]	245
Step 125: customerviewdriver.component.spec.ts - tests for the customer driver list [Arjun]	247
Step 126: customer-request.component.spec.ts - tests for the trip request form [Arjun]	249
Step 127: adminviewrequests.component.spec.ts - tests for the admin request table [Aarti]	252
Step 128: customerviewrequested.component.spec.ts - tests for "My Requests" [Vaishalini]	254
Step 129: customerpostfeedback.component.spec.ts - tests for the Submit Feedback form [Saumya]	257
Step 130: customerviewfeedback.component.spec.ts - tests for "My Feedback" [Saumya]	258
Step 131: adminviewfeedback.component.spec.ts - tests for the admin feedback table [Saumya]	260
Step 132: error.component.spec.ts - tests for the error page [Vaishalini]	262
Step 133: authguard.guard.spec.ts - tests for the role-based route guard [Vanshika]	263
Step 134: user.model.spec.ts - official test for the User model [Aleena]	264
Step 135: driver.model.spec.ts - official test for the Driver model [Arjun]	265
Step 136: driver-request.model.spec.ts - official test for the DriverRequest model [Arjun]	266
Step 137: feedback.model.spec.ts - official test for the Feedback model [Saumya]	267
Step 138: auth.service.spec.ts - official test for AuthService [Aleena]	268
Step 139: driver.service.spec.ts - official test for DriverService [Arjun]	269
Step 140: driver-request.service.spec.ts - official test for DriverRequestService [Arjun]	270
Step 141: feedback.service.spec.ts - official test for FeedbackService [Saumya]	270
Step 142: ai.service.spec.ts - our own test for AiService [Saumya]	271
Step 143: test.ts - old Angular test entry point (unused) [Vaishalini]	272
Step 144: karma.conf.js (angularapp) - Karma test-runner settings [Shared]	273
Step 145: tsconfig.spec.json - TypeScript settings for tests [Shared]	275
Step 146: karma.sh - the official frontend grading script [Shared]	276
Step 147: karma/karma.conf.js - the official Karma configuration [Shared]	278
Step 148: SpringappApplicationTests - the official backend integration tests [Shared]	280
Step 149: junit.sh - the official backend grading script [Shared]	290
Step 150: build.yml - GitHub Actions workflow from the template [Shared]	291
Step 151: workflows/pom.xml - tiny Maven file for the Sonar run [Shared]	292
Step 152: README.md - platform stub, not a real README [Shared]	293
Step 153: db.json - leftover json-server file (not used by DriveU) [Shared]	294

Part 5 - Observations and presentation tips

296

How to read this document

This document walks through the project **in the same order the computer runs it**. You start the backend first, then the frontend, then a user opens the browser. We follow exactly that path, one file at a time.

- **Part 1 - Starting the backend** (`mvn spring-boot:run`): `pom.xml`, `application.properties`, `SpringappApplication`, then the entities, repositories, security classes, services, AI classes, controllers and the exception handler.
- **Part 2 - Starting the frontend** (`ng serve --port 8081`): `package.json`, `angular.json`, the TypeScript config, `index.html`, `main.ts`, `AppModule`, routing, `AppComponent`, global styles, utilities, models, services, interceptor and guard.
- **Part 3 - The user journey**: every page in the order a user meets it (login, signup, home, navbars, then the driver, request and feedback pages). Each page file is followed into the backend call it makes.
- **Part 4 - Appendix: tests and project/CI files**: the files that check the project but are not part of the request flow.
- **Part 5 - Observations**: honest notes about limitations and leftover code, useful when you present the project.

How each file section is laid out

- **Dark header**: the step number, the file title, the owner in square brackets, the exact path, and a coloured tag (green BACKEND, blue FRONTEND, orange CONFIG, purple TEST).
- **Info table**: what the file is, who or what triggers it (who calls it), and what it uses.
- **Code blocks**: the real source lines with their real line numbers, taken straight from the file. A dark bar above each block tells you which lines it covers.
- **Blue explanation box** under every code block: what that code does, line by line, and why it is there.
- **Green "Workflow, next" box** at the end: how execution moves on to the next file.

Words you will meet

Term	Plain-English meaning
Annotation (<code>@Something</code>)	A label on a Java class/method/field that tells Spring what it is or how to treat it (e.g. <code>@Entity</code> = "this is a database table").
Bean	An object created and managed by Spring. You never write <code>new</code> for services/controllers; Spring makes one and hands it out.
Dependency injection	Spring automatically puts the objects a class needs into it (<code>@Autowired</code> in Java, constructor parameters in Angular).
Entity / Repository	An entity is a Java class mapped to a table. A repository is an interface with ready-made database operations (save, findAll, findById ...).
Controller / Service	A controller receives the HTTP request and returns the response. A service holds the business rules and is called by the controller.
JWT	JSON Web Token: a signed text string given to the browser after login. The browser sends it back on every request to prove who the user is.
Filter	Code that sees every request before the controller does. <code>JwtAuthenticationFilter</code> is the one that reads the token.
CORS	A browser safety rule: a page from port 8081 may call port 8080 only if the server says so. <code>CorsConfig</code> says yes.
Component (Angular)	One piece of screen: a TypeScript class (.ts), a template (.html) and styles (.css).
Module	<code>AppModule</code> is the list of everything the Angular app consists of.
Service (Angular)	A reusable class that talks to the backend or keeps shared state. Components get it through their constructor.
Observable / subscribe	An HTTP call in Angular returns an Observable. Nothing happens until you <code>subscribe</code> , which delivers the result later.
Reactive form	A form defined in TypeScript (<code>FormGroup</code> , <code>Validators</code>) and connected to the HTML with <code>[formGroup]</code> and <code>formControlName</code> .
Guard / Interceptor	A guard decides whether a page may be opened. An interceptor sees every HTTP call and can react to errors.

Term	Plain-English meaning
Modal / popup	A box that appears over the page (confirm delete, details, success message).

DriveU at a glance

DriveU is a platform where a **Customer** can hire a professional driver and an **Admin** manages drivers and approves trips.

What each role does

- **Admin:** adds, edits, deletes and changes the status of drivers; sees every request and approves, rejects or closes it; reads all feedback and an AI summary of each driver's reviews.
- **Customer:** browses available drivers (plain search and AI search), requests a driver, edits or deletes a pending request, ends the trip, checks the fare, writes feedback and reads own feedback.

The life of a booking

Pending (customer submitted) -> **Approved** or **Rejected** (admin; Approved and Rejected can be switched by the admin) -> **Trip End** (customer presses Trip End; only possible when Approved; the fare is calculated) -> **Closed** (admin closes it; only possible when Trip End).

A customer may Edit or Delete a request only while it is **Pending**.

Technology map

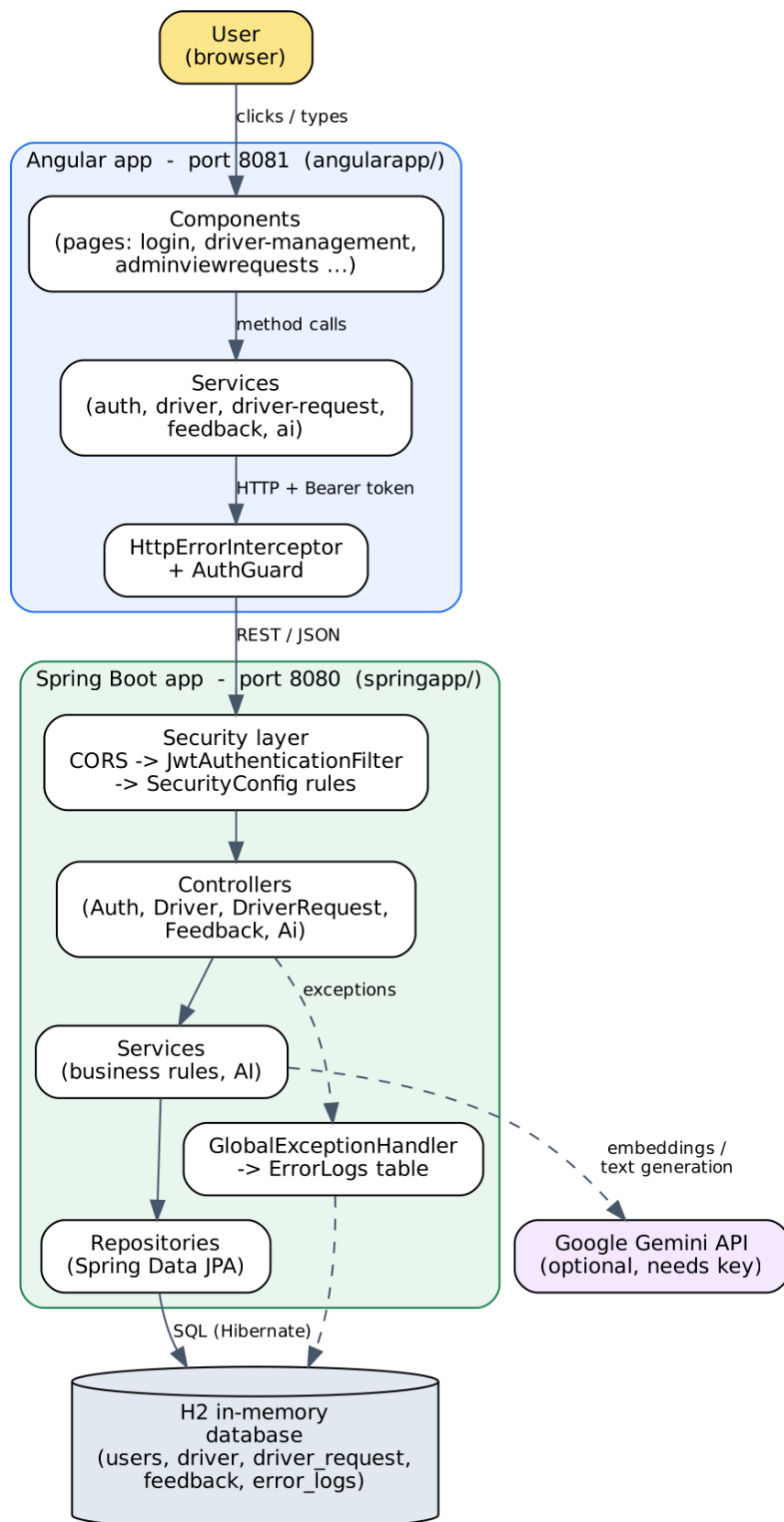
Layer	Technology	Where in the project
Browser UI	Angular 16, TypeScript, RxJS, reactive forms, CSS	angularapp/
REST API	Spring MVC @RestController, JSON via Jackson	springapp/.../controller
Security	Spring Security, JWT (jjwt 0.9.1), BCrypt, CORS	springapp/.../config
Business logic	Spring @Service classes, AI helpers	springapp/.../service
Persistence	Spring Data JPA, Hibernate, H2 (MySQL ready)	model, repository, application.properties
Errors	@RestControllerAdvice, ErrorLogs table	springapp/.../exceptions, ErrorLog
AI (optional)	Google Gemini REST API + local fallback	GeminiService, AiService
Tests	JUnit 5 + MockMvc, Karma + Jasmine	springapp/src/test, *.spec.ts

Folder map

```
driverapp-main/
|-- springapp/                                BACKEND (Spring Boot, port 8080)
|   |-- pom.xml                               Maven build + libraries
|   |-- src/
|       |-- main/
|           |-- resources/application.properties  settings (port, H2, Gemini key)
|           |-- java/com/examly/springapp/
|               |-- SpringappApplication.java    main() - starts everything
|               |-- model/                       entities (database tables) + LoginDTO
|               |-- repository/                  Spring Data JPA interfaces
|               |-- config/                     JWT, filter, SecurityConfig, CORS
|               |-- service/                    business logic + AI (Gemini)
|               |-- controller/                 REST endpoints (/api/...)
|               |-- exceptions/                 custom exceptions + GlobalExceptionHandler
|           |-- test/.../SpringappApplicationTests.java  JUnit tests
|-- angularapp/                                FRONTEND (Angular 16, port 8081)
|   |-- package.json, angular.json, tsconfig*.json  build + compiler settings
|   |-- src/
|       |-- index.html, main.ts, styles.css        page shell, bootstrap, global CSS
|       |-- apiconfig.ts, environments/            backend URL (http://localhost:8080)
|       |-- app/
|           |-- app.module.ts, app-routing.module.ts, app.component.*
|           |-- components/                       16 pages/widgets (each: .ts .html .css)
|           |-- models/                           TypeScript shapes of Driver, DriverRequest ...
|           |-- services/                         HTTP services + interceptor
|           |-- utils/                            constants, date helpers, driver image helper
|-- karma/   junit/                             official test harness files
|-- .github/workflows/                          CI (build.yml, pom.xml)
```

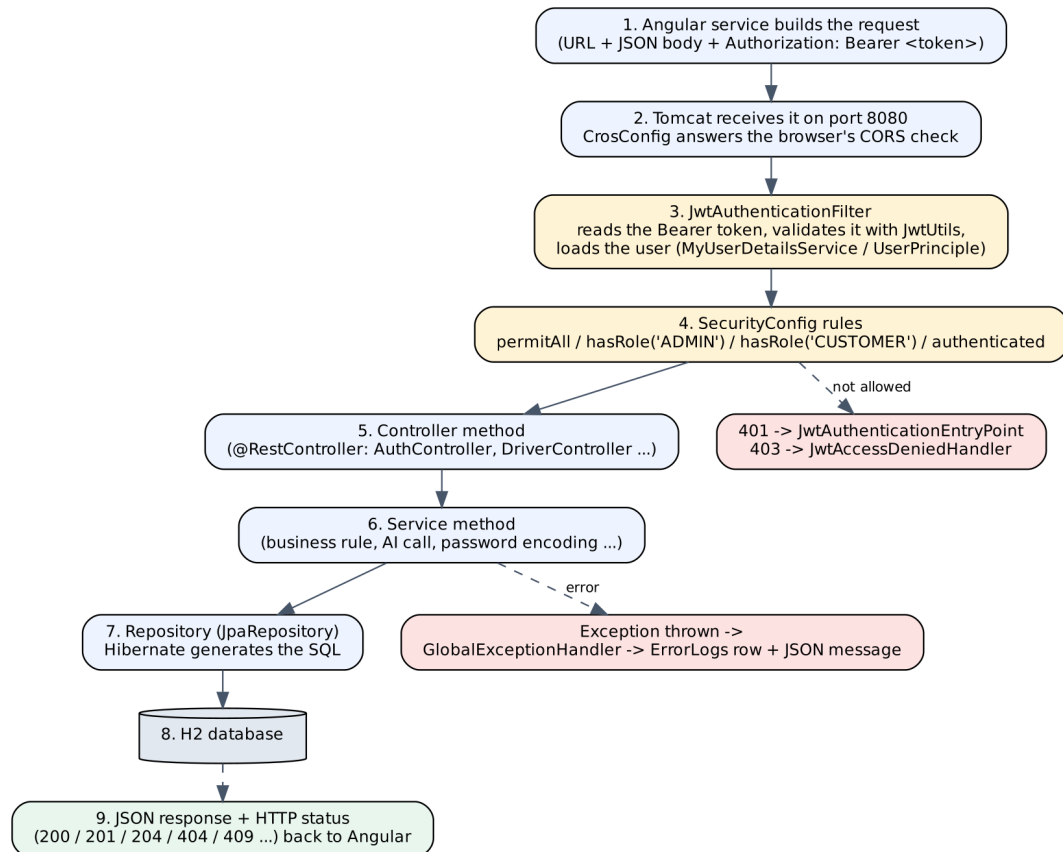
The big picture

1. How the two programs talk to each other



The browser only ever talks to the **Angular app** (port 8081). Angular's services send REST/JSON calls to the **Spring Boot app** (port 8080), which reads and writes the **H2 database**. The Gemini API is optional.

2. What happens to every single request inside the backend



Keep this picture in mind: almost every backend file in Part 1 is one of these boxes. When Part 3 follows a button click from a page into the backend, it walks through exactly this chain.

3. How to start the project (the order used in this document)

- **Step 1 - backend:** open a terminal in `springapp` and run `mvn spring-boot:run`. Wait for "Started SpringappApplication". The API now answers on `http://localhost:8080`.
- **Step 2 - frontend:** open a second terminal in `angularapp` and run `npm install` (first time only) and `npm start` (it runs `ng serve --port 8081 --disable-host-check --host=0.0.0.0`). Wait for "Compiled successfully".
- **Step 3 - browser:** open `http://localhost:8081`. The empty path redirects to `/login`.
- **Optional - AI:** set the environment variable `GEMINI_API_KEY` before step 1 to switch the AI features from the local fallback to Gemini.
- **Optional - database viewer:** while the backend runs, `http://localhost:8080/h2-console` shows the tables (JDBC URL `jdbc:h2:mem:testdb`, user `sa`, password `password`).

Who owns what (Sprint Team Allocation)

Member	Module	Main files
Aleena	Authentication & User Onboarding	login, signup, AuthService, AuthController, User, LoginDTO, UserRepo, UserService/Impl
Vanshika	Security & Authorization	SecurityConfig, JwtUtils, JwtAuthenticationFilter, MyUserDetailsService, UserPrincipal, entry-point/denied handlers, CrosConfig, AuthGuard, HttpErrorInterceptor, routing guards
Arjun	Driver Management & Booking Engine	DriverController, DriverRequestController, Driver, DriverRequest, repos, services, driver-management, admin-view-drivers, customerviewdriver, customer-request
Prajan	Payment & Trip Completion	Trip End + Fetch Pay Amount parts of customerviewrequested, fare/duration helpers in utils/format.ts, payment fields in DriverRequestServiceImpl
Saumya	Feedback & AI Services	FeedbackController/Service, Feedback, AiController, AiService, GeminiService, customerpostfeedback, customerviewfeedback, adminviewfeedback, AI search and summary UI
Aarti	Admin Operations & Request Management	adminviewrequests (approve, reject, close, stage modal, filters, driver popup)
Vaishalini	Customer Journey & Shared Infrastructure	customerviewrequested, home-page, adminnav, customernav, error component, GlobalExceptionHandler, ErrorLog(+Repo), exceptions, app shell, constants, styles, config files

Where one file is shared (for example ``customerviewrequested`` or ``DriverRequestServiceImpl``), both owners are shown in the brackets.

Complete file index

Every file explained in this document, in reading order. The number is the step number used in the file headers.

#	File	Owner	Lines
1	springapp/pom.xml	Vaishalini	84
2	springapp/src/main/resources/application.properties	Vaishalini	16
3	.../springapp/SpringappApplication.java	Vaishalini	14
4	.../springapp/model/User.java	Aleena	79
5	.../springapp/model/LoginDTO.java	Aleena	50
6	.../springapp/model/Driver.java	Arjun	106
7	.../springapp/model/DriverRequest.java	Arjun	156
8	.../springapp/model/Feedback.java	Saumya	112
9	.../springapp/model/ErrorLog.java	Vaishalini	81
10	.../springapp/repository/UserRepo.java	Aleena	12
11	.../springapp/repository/DriverRepo.java	Arjun	11
12	.../springapp/repository/DriverRequestRepo.java	Arjun	13
13	.../springapp/repository/FeedbackRepo.java	Saumya	13
14	.../springapp/repository/ErrorLogRepo.java	Vaishalini	9
15	.../springapp/config/SecurityConfig.java	Vanshika	107
16	.../springapp/config/JwtUtils.java	Vanshika	46
17	.../springapp/config/JwtAuthenticationFilter.java	Vanshika	64
18	.../springapp/config/MyUserDetailsService.java	Vanshika	26
19	.../springapp/config/UserPrinciple.java	Vanshika	80
20	.../springapp/config/JwtAuthenticationEntryPoint.java	Vanshika	20
21	.../springapp/config/JwtAccessDeniedHandler.java	Vanshika	20
22	.../springapp/config/CrosConfig.java	Vanshika	18
23	.../springapp/service/UserService.java	Aleena	8
24	.../springapp/service/UserServiceImpl.java	Aleena	35
25	.../springapp/exceptions/DuplicateDriverException.java	Vaishalini	7
26	.../springapp/exceptions/DriverDeletionException.java	Vaishalini	7
27	.../springapp/service/DriverService.java	Arjun	13
28	.../springapp/service/DriverServiceImpl.java	Arjun	70
29	.../springapp/exceptions/DriverRequestDeletionException.java	Vaishalini	7
30	.../springapp/service/DriverRequestService.java	Arjun	15
31	.../springapp/service/DriverRequestServiceImpl.java	Arjun	82
32	.../springapp/service/GeminiService.java	Saumya	114
33	.../springapp/service/AIService.java	Saumya	368
34	.../springapp/service/FeedbackService.java	Saumya	12
35	.../springapp/service/FeedbackServiceImpl.java	Saumya	59
36	.../springapp/controller/AuthController.java	Aleena	58
37	.../springapp/controller/DriverController.java	Arjun	65
38	.../springapp/controller/DriverRequestController.java	Arjun	83
39	.../springapp/controller/FeedbackController.java	Saumya	64
40	.../springapp/controller/AiController.java	Saumya	36

#	File	Owner	Lines
41	.../springapp/exceptions/GlobalExceptionHandler.java	Vaishalini	60
42	angularapp/package.json	Vaishalini	40
43	angularapp/angular.json	Vaishalini	107
44	angularapp/tsconfig.json	Vaishalini	20
45	angularapp/tsconfig.app.json	Vaishalini	14
46	src/index.html	Vaishalini	13
47	src/main.ts	Vaishalini	7
48	src/polyfills.ts	Vaishalini	63
49	src/environments/environment.ts	Vaishalini	5
50	src/apiconfig.ts	Vaishalini	4
51	app/app.module.ts	Vaishalini	56
52	app/app-routing.module.ts	Vanshika	52
53	app/app.component.ts	Vaishalini	21
54	app/app.component.html	Vaishalini	3
55	src/styles.css	Vaishalini	384
56	app/utils/constants.ts	Vaishalini	21
57	app/utils/format.ts	Prajan	86
58	app/utils/driver-image.ts	Arjun	29
59	app/models/login.model.ts	Aleena	4
60	app/models/user.model.ts	Aleena	8
61	app/models/driver.model.ts	Arjun	12
62	app/models/driver-request.model.ts	Arjun	24
63	app/models/feedback.model.ts	Saumya	21
64	app/models/ai.model.ts	Saumya	8
65	app/services/http-error.interceptor.ts	Vanshika	35
66	app/services/auth.service.ts	Aleena	107
67	app/services/driver.service.ts	Arjun	39
68	app/services/driver-request.service.ts	Arjun	47
69	app/services/feedback.service.ts	Saumya	35
70	app/services/ai.service.ts	Saumya	33
71	app/components/authguard/authguard.guard.ts	Vanshika	28
72	app/components/login/login.component.ts	Aleena	60
73	app/components/login/login.component.html	Aleena	38
74	app/components/login/login.component.css	Aleena	2
75	app/components/signup/signup.component.ts	Aleena	82
76	app/components/signup/signup.component.html	Aleena	75
77	app/components/signup/signup.component.css	Aleena	2
78	app/components/home-page/home-page.component.ts	Vaishalini	11
79	app/components/home-page/home-page.component.html	Vaishalini	25
80	app/components/home-page/home-page.component.css	Vaishalini	53
81	app/components/adminnav/adminnav.component.ts	Vaishalini	42
82	app/components/adminnav/adminnav.component.html	Vaishalini	45
83	app/components/adminnav/adminnav.component.css	Vaishalini	3

#	File	Owner	Lines
84	app/components/customernav/customernav.component.ts	Vaishalini	42
85	app/components/customernav/customernav.component.html	Vaishalini	26
86	app/components/customernav/customernav.component.css	Vaishalini	3
87	app/components/driver-management/driver-management.component.ts	Arjun	183
88	app/components/driver-management/driver-management.component.html	Arjun	70
89	app/components/driver-management/driver-management.component.css	Arjun	13
90	app/components/admin-view-drivers/admin-view-drivers.component.ts	Arjun	129
91	app/components/admin-view-drivers/admin-view-drivers.component.html	Arjun	63
92	app/components/admin-view-drivers/admin-view-drivers.component.css	Arjun	59
93	app/components/customerviewdriver/customerviewdriver.component.ts	Arjun	126
94	app/components/customerviewdriver/customerviewdriver.component.html	Arjun	45
95	app/components/customerviewdriver/customerviewdriver.component.css	Arjun	24
96	app/components/customer-request/customer-request.component.ts	Arjun	138
97	app/components/customer-request/customer-request.component.html	Arjun	45
98	app/components/customer-request/customer-request.component.css	Arjun	2
99	app/components/adminviewrequests/adminviewrequests.component.ts	Aarti	135
100	app/components/adminviewrequests/adminviewrequests.component.html	Aarti	83
101	app/components/adminviewrequests/adminviewrequests.component.css	Aarti	21
102	app/components/customerviewrequested/customerviewrequested.component.ts	Vaishalini	184
103	app/components/customerviewrequested/customerviewrequested.component.html	Vaishalini	86
104	app/components/customerviewrequested/customerviewrequested.component.css	Vaishalini	17
105	app/components/customerpostfeedback/customerpostfeedback.component.ts	Saumya	91
106	app/components/customerpostfeedback/customerpostfeedback.component.html	Saumya	36
107	app/components/customerpostfeedback/customerpostfeedback.component.css	Saumya	4
108	app/components/customerviewfeedback/customerviewfeedback.component.ts	Saumya	94
109	app/components/customerviewfeedback/customerviewfeedback.component.html	Saumya	56
110	app/components/customerviewfeedback/customerviewfeedback.component.css	Saumya	13
111	app/components/adminviewfeedback/adminviewfeedback.component.ts	Saumya	121
112	app/components/adminviewfeedback/adminviewfeedback.component.html	Saumya	102
113	app/components/adminviewfeedback/adminviewfeedback.component.css	Saumya	23
114	app/components/error/error.component.ts	Vaishalini	11
115	app/components/error/error.component.html	Vaishalini	4
116	app/components/error/error.component.css	Vaishalini	3
117	app/app.component.spec.ts	Vaishalini	50
118	app/components/login/login.component.spec.ts	Aleena	72
119	app/components/signup/signup.component.spec.ts	Aleena	71
120	app/components/home-page/home-page.component.spec.ts	Vaishalini	28
121	app/components/adminnav/adminnav.component.spec.ts	Vaishalini	49
122	app/components/customernav/customernav.component.spec.ts	Vaishalini	49
123	app/components/driver-management/driver-management.component.spec.ts	Arjun	78

#	File	Owner	Lines
124	app/components/admin-view-drivers/admin-view-drivers.component.spec.ts	Arjun	86
125	app/components/customerviewdriver/customerviewdriver.component.spec.ts	Arjun	101
126	app/components/customer-request/customer-request.component.spec.ts	Arjun	91
127	app/components/adminviewrequests/adminviewrequests.component.spec.ts	Aarti	100
128	app/components/customerviewrequested/customerviewrequested.component.spec.ts	Vaishalini	103
129	app/components/customerpostfeedback/customerpostfeedback.component.spec.ts	Saumya	58
130	app/components/customerviewfeedback/customerviewfeedback.component.spec.ts	Saumya	68
131	app/components/adminviewfeedback/adminviewfeedback.component.spec.ts	Saumya	83
132	app/components/error/error.component.spec.ts	Vaishalini	25
133	app/components/authguard/authguard.guard.spec.ts	Vanshika	48
134	app/models/user.model.spec.ts	Aleena	25
135	app/models/driver.model.spec.ts	Arjun	33
136	app/models/driver-request.model.spec.ts	Arjun	43
137	app/models/feedback.model.spec.ts	Saumya	27
138	app/services/auth.service.spec.ts	Aleena	18
139	app/services/driver.service.spec.ts	Arjun	19
140	app/services/driver-request.service.spec.ts	Arjun	19
141	app/services/feedback.service.spec.ts	Saumya	19
142	app/services/ai.service.spec.ts	Saumya	37
143	src/test.ts	Vaishalini	25
144	angularapp/karma.conf.js	Shared	63
145	angularapp/tsconfig.spec.json	Shared	14
146	karma/karma.sh	Shared	79
147	karma/karma.conf.js	Shared	63
148	springapp/src/test/java/com/examly/springapp/SpringappApplicationTests.java	Shared	488
149	junit/junit.sh	Shared	20
150	.github/workflows/build.yml	Shared	53
151	.github/workflows/pom.xml	Shared	34
152	README.md	Shared	2
153	angularapp/db.json	Shared	38

Part 1 - Starting the backend: from mvn spring-boot:run to a listening server

This part follows what happens when you run `mvn spring-boot:run` inside the `springapp` folder.

1. **Maven** reads `pom.xml`, downloads libraries and compiles the code.
2. Spring Boot loads `application.properties` (port, database, Gemini settings).
3. The JVM calls `main()` in `SpringappApplication`.
4. `@SpringBootApplication` scans the package and creates every bean. Hibernate reads the **entities** and creates the tables in H2.
5. Spring Data creates the **repositories**; the **security** beans build the filter chain; **services** and **controllers** are created and wired together; the **exception handler** registers itself.
6. Tomcat starts on port 8080 and waits.

To make this easy to follow, the files are explained in layers: configuration -> entities -> repositories -> security -> services (with AI) -> controllers -> exception handling. Each layer only depends on the ones before it.

Step 1: pom.xml - the Maven build file [Vaishalini - Customer Journey & Shared Infra]

springapp/pom.xml

CONFIG

What this file is	The Project Object Model . Maven reads this file first when you run <code>mvn spring-boot:run</code> (or <code>mvn test</code>). It tells Maven what this project is called, which Java version to compile with, and which libraries (dependencies) to download. Nothing in this file is "executed" by the application itself; it only decides what code and libraries are on the classpath before <code>main()</code> is ever called.
Who / what triggers it	You, in a terminal inside the <code>springapp</code> folder: <code>mvn spring-boot:run</code> . Maven is the very first program that touches the backend.
What it uses / calls	Maven Central (to download libraries) and the <code>spring-boot-starter-parent</code> parent POM (which supplies default versions and plugin settings).
Size	84 lines

Lines 1-3 | XML header and project namespace

```
1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3 |     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Line 1 declares that the file is XML. Lines 2-3 open the `<project>` element and point to the Maven XSD schema so editors can validate the file. Every Maven `pom.xml` starts like this.

Line 4 | Model version

```
4 |     <modelVersion>4.0.0</modelVersion>
```

`<modelVersion>4.0.0</modelVersion>` is the version of the POM format itself. It is always `4.0.0` for modern Maven.

Lines 5-10 | Parent: Spring Boot 3.0.1

```
5 |     <parent>
6 |         <groupId>org.springframework.boot</groupId>
7 |         <artifactId>spring-boot-starter-parent</artifactId>
8 |         <version>3.0.1</version>
9 |         <relativePath/> <!-- lookup parent from repository -->
10 |     </parent>
```

The `<parent>` block makes this project inherit from **spring-boot-starter-parent 3.0.1**. That parent pins compatible versions of Spring, Hibernate, Jackson, Tomcat and so on, so the dependencies below do not need version numbers (except two that Spring does not manage). Spring Boot 3 requires Java 17 and uses the `jakarta.*` package names (which is why the Java files import `jakarta.persistence` and `jakarta.servlet`, not `javax.*`). `<relativePath/>` tells Maven to download the parent from the repository instead of looking on disk.

Lines 11-15 | Project identity

```
11 | <groupId>com.examly</groupId>
12 | <artifactId>springapp</artifactId>
13 | <version>0.0.1-SNAPSHOT</version>
14 | <name>springapp</name>
15 | <description>Demo project for Spring Boot</description>
```

These lines give the project its coordinates: group `com.examly`, artifact `springapp`, version `0.0.1-SNAPSHOT`, plus a display name and description. The group id matches the Java package `com.examly.springapp` that all backend source files live in.

Lines 16-18 | Java version property

```
16 | <properties>
17 |   <java.version>17</java.version>
18 | </properties>
```

`<java.version>17</java.version>` makes the compiler use Java 17. Spring Boot 3.0.1 will not run on anything older.

Lines 19-35 | Core starters: JPA, Security, Web, Web Services

```
19 | <dependencies>
20 |   <dependency>
21 |     <groupId>org.springframework.boot</groupId>
22 |     <artifactId>spring-boot-starter-data-jpa</artifactId>
23 |   </dependency>
24 |   <dependency>
25 |     <groupId>org.springframework.boot</groupId>
26 |     <artifactId>spring-boot-starter-security</artifactId>
27 |   </dependency>
28 |   <dependency>
29 |     <groupId>org.springframework.boot</groupId>
30 |     <artifactId>spring-boot-starter-web</artifactId>
31 |   </dependency>
32 |   <dependency>
33 |     <groupId>org.springframework.boot</groupId>
34 |     <artifactId>spring-boot-starter-web-services</artifactId>
35 |   </dependency>
```

A **starter** is a bundle of related libraries.

- `spring-boot-starter-data-jpa` (lines 20-23): Hibernate + Spring Data JPA. This is what lets the `@Entity` classes become database tables and lets `JpaRepository` interfaces work with no SQL.
- `spring-boot-starter-security` (24-27): Spring Security, the filter chain that protects every URL. `SecurityConfig` configures it.
- `spring-boot-starter-web` (28-31): Spring MVC and the embedded Tomcat server. It provides `@RestController`, `@RequestMapping` and the JSON converter (Jackson).
- `spring-boot-starter-web-services` (32-35): the SOAP web-services starter that came with the project template; the app does not actually use SOAP.

Lines 37-52 | Runtime-only libraries: devtools, H2, MySQL

```
37 | <dependency>
38 |   <groupId>org.springframework.boot</groupId>
39 |   <artifactId>spring-boot-devtools</artifactId>
40 |   <scope>runtime</scope>
41 |   <optional>true</optional>
42 | </dependency>
43 | <dependency>
44 |   <groupId>com.h2database</groupId>
45 |   <artifactId>h2</artifactId>
46 |   <scope>runtime</scope>
47 | </dependency>
48 | <dependency>
49 |   <groupId>com.mysql</groupId>
50 |   <artifactId>mysql-connector-j</artifactId>
51 |   <scope>runtime</scope>
52 | </dependency>
```

- `spring-boot-devtools` (37-42): automatic restart when code changes. `runtime` + `optional` means it is not packaged into the final jar for others.
- `h2` (43-47): the in-memory database engine. This is what actually stores data in the current configuration (see `application.properties`).
- `mysql-connector-j` (48-52): the MySQL driver. The SRS names a MySQL database called `appdb`; the driver is on the classpath so you can switch to MySQL by changing the datasource properties. Scope `runtime` means it is only needed when the app runs, not to compile.

Lines 53-57 | Test starter

```
53 |         <dependency>
54 |             <groupId>org.springframework.boot</groupId>
55 |             <artifactId>spring-boot-starter-test</artifactId>
56 |             <scope>test</scope>
57 |         </dependency>
```

`spring-boot-starter-test` (scope `test`) brings JUnit 5, Mockito and `MockMvc`. It is only used by `SpringappApplicationTests.java` and is not part of the running application.

Lines 58-67 | JWT and JAXB libraries

```
58 |         <dependency>
59 |             <groupId>io.jsonwebtoken</groupId>
60 |             <artifactId>jjwt</artifactId>
61 |             <version>0.9.1</version>
62 |         </dependency>
63 |         <dependency>
64 |             <groupId>javax.xml.bind</groupId>
65 |             <artifactId>jaxb-api</artifactId>
66 |             <version>2.3.1</version>
67 |         </dependency>
```

- `jjwt 0.9.1` (58-62): the library that builds and verifies **JSON Web Tokens**. `JwtUtils` uses it. The version is written explicitly because Spring Boot does not manage it.
- `jaxb-api 2.3.1` (63-67): JWT 0.9.1 uses `javax.xml.bind.DataConverter` internally, which was removed from Java 11+. This dependency puts it back so token signing does not crash.

Lines 68-73 | Security test helper and end of dependencies

```
68 |         <dependency>
69 |             <groupId>org.springframework.security</groupId>
70 |             <artifactId>spring-security-test</artifactId>
71 |             <scope>test</scope>
72 |         </dependency>
73 |     </dependencies>
```

`spring-security-test` (scope `test`) lets tests send requests as a logged-in user. Line 73 closes `<dependencies>`.

Lines 75-84 | Build plugin

```
75 |     <build>
76 |         <plugins>
77 |             <plugin>
78 |                 <groupId>org.springframework.boot</groupId>
79 |                 <artifactId>spring-boot-maven-plugin</artifactId>
80 |             </plugin>
81 |         </plugins>
82 |     </build>
83 |
84 | </project>
```

The `spring-boot-maven-plugin` (lines 77-80) is what makes `mvn spring-boot:run` start the app and what packages an executable jar. Lines 81-84 close `<plugins>`, `<build>` and `<project>`.

Workflow, next: Maven has resolved the libraries and compiled all Java sources under `src/main/java`. Because of the `spring-boot-maven-plugin` it now launches the class annotated with `@SpringBootApplication`, but first it loads the settings file, so control goes to `application.properties`.

Step 2: application.properties - runtime settings

[Vaishalini - Customer Journey & Shared Infra]

springapp/src/main/resources/application.properties

CONFIG

What this file is	The central configuration file read by Spring Boot at startup. Each <code>key=value</code> line either configures Spring Boot itself (port, database, JPA) or defines a custom value that application code reads with <code>@Value</code> .
Who / what triggers it	Spring Boot loads it automatically from the classpath (<code>src/main/resources</code>) the moment <code>SpringApplication.run</code> begins, before any bean is created.
What it uses / calls	H2 database driver (datasource lines), Hibernate (JPA lines), Tomcat (port), and <code>GeminiService</code> (the three <code>gemini.*</code> keys).
Size	16 lines

Line 1 | Hibernate schema mode

```
1 | spring.jpa.hibernate.ddl-auto=update
```

`spring.jpa.hibernate.ddl-auto=update` tells Hibernate to inspect the entities (`User`, `Driver`, `DriverRequest`, `Feedback`, `ErrorLog`) at startup and create or alter the database tables to match them. With the in-memory H2 database the tables start empty each run, so Hibernate creates all of them from scratch.

Lines 2-5 | Datasource (database connection)

```
2 | spring.datasource.url=jdbc:h2:mem:testdb
3 | spring.datasource.driverClassName=org.h2.Driver
4 | spring.datasource.username=sa
5 | spring.datasource.password=password
```

These four lines are the JDBC connection: URL `jdbc:h2:mem:testdb` (an in-memory H2 database named `testdb`, which disappears when the server stops), driver class `org.h2.Driver`, user `sa`, password `password`. To use MySQL (`appdb` in the SRS) you would change these lines to a `jdbc:mysql://localhost:3306/appdb` URL and the MySQL driver.

Lines 6-8 | Dialect, H2 console and SQL logging

```
6 | spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
7 | spring.h2.console.enabled=true
8 | spring.jpa.show-sql=true
```

`H2Dialect` makes Hibernate generate SQL that H2 understands. `spring.h2.console.enabled=true` turns on the browser database viewer at `/h2-console` (which `SecurityConfig` explicitly allows). `spring.jpa.show-sql=true` prints every SQL statement in the server console, which is very useful when learning what the repositories do.

Lines 9-10 | Swagger flag and path matching

```
9 | enable.swagger.plugin=true
10 | spring.mvc.pathmatch.matching-strategy=ant-path-matcher
```

`enable.swagger.plugin=true` is a custom flag from the project template.

`spring.mvc.pathmatch.matching-strategy=ant-path-matcher` switches Spring MVC to the older Ant-style URL matcher, a compatibility setting that Spring Boot 2.6+/3 projects with Springfox-style tools need.

Line 11 | Server port

```
11 | server.port=8080
```

`server.port=8080` makes the embedded Tomcat listen on port 8080. The Angular app is configured to call `http://localhost:8080` (see `environment.ts`).

Lines 13-16 | Gemini AI settings

```
13 | # ---- Gemini AI features (optional). Leave the key empty to use the built-in fallback logic. ----
14 | gemini.api.key=${GEMINI_API_KEY:}
15 | gemini.embedding.model=text-embedding-004
16 | gemini.generation.model=gemini-2.0-flash
```

Line 13 is a comment. `gemini.api.key=${GEMINI_API_KEY:}` reads the Google Gemini key from the environment variable `GEMINI_API_KEY`; the `:` with nothing after it means the default is an **empty string**. If no key is set, `GeminiService.isEnabled()` is false and `AIService` quietly uses its local keyword logic instead, so the app and its tests work with no internet or key. The two model lines choose the embedding model (`text-embedding-004`) and the text-generation model (`gemini-2.0-flash`).

Workflow, next: Settings are loaded (port 8080, H2 database, Gemini placeholders). Now Spring needs the class that starts everything: `SpringappApplication.java`.

Step 3: SpringappApplication - the main() entry point

[Vaishalini - Customer Journey & Shared Infra]

BACKEND

springapp/src/main/java/com/examly/springapp/SpringappApplication.java

What this file is	The first Java class that runs . It contains the standard Java <code>main</code> method and the <code>@SpringBootApplication</code> annotation, which turns on component scanning, auto-configuration and bean creation for the whole backend.
Who / what triggers it	<code>mvn spring-boot:run</code> calls <code>SpringappApplication.main(...)</code> . The JVM executes this class before any other class in the project.
What it uses / calls	Spring Boot (<code>SpringApplication.run</code>). Component scanning then discovers every class in the <code>com.examly.springapp</code> package and its sub-packages: <code>config</code> , <code>controller</code> , <code>exceptions</code> , <code>model</code> , <code>repository</code> , <code>service</code> .
Size	14 lines

Line 1 | Package declaration

```
1 | package com.examly.springapp;
```

Places the class in `com.examly.springapp`. Because `@SpringBootApplication` scans **this package and everything below it**, all the other folders (`config`, `controller`, ...) are automatically found without any manual registration.

Lines 3-5 | Imports

```
3 | import org.springframework.boot.SpringApplication;
4 | import org.springframework.boot.autoconfigure.SpringBootApplication;
5 | import org.springframework.context.annotation.ComponentScan;
```

`SpringApplication` starts the framework. `SpringBootApplication` is the all-in-one annotation. `ComponentScan` is imported but not used (a leftover from the template; it is harmless).

Lines 7-14 | The application class and main method

```
7 | @SpringBootApplication
8 | public class SpringappApplication {
9 |
10 |     public static void main(String[] args) {
11 |         SpringApplication.run(SpringappApplication.class, args);
12 |     }
13 |
14 | }
```

`@SpringBootApplication` combines three things: `@Configuration` (this class can define beans), `@EnableAutoConfiguration` (Spring Boot guesses and sets up Tomcat, Jackson, JPA, Security from the libraries on the classpath) and `@ComponentScan` (find all `@Component`, `@Service`, `@Repository`, `@RestController`, `@Configuration` classes).

`main` calls `SpringApplication.run(SpringappApplication.class, args)`, which in order: reads `application.properties`, creates the application context, scans for beans, connects to H2, lets Hibernate create the tables for the five entities, builds the Spring Security filter chain, starts Tomcat on port 8080 and prints "Started SpringappApplication". After this the server just waits for HTTP requests.

Workflow, next: The context now starts creating beans. Spring finds the `@Entity` classes first (the database model), then the repositories that talk to them, then security, services, controllers and finally the global exception handler. We follow exactly that layered order, beginning with the entities: `User.java`.

Step 4: User - the people who can log in (table users)

[Aleena - Authentication]

BACKEND

springapp/src/main/java/com/examly/springapp/model/User.java

What this file is	The first entity of the project. An entity is an ordinary Java class that is mapped to one table in the database: one object of the class = one row, one field = one column. The mapping is done by annotations (the words starting with @); an annotation is a label you put on a class, field or method that tells a framework (here Hibernate/JPA) how to treat it. User describes every person who can use DriveU, whether they are an Admin or a Customer. It stores the e-mail used to log in, the (encrypted) password, a display name, a phone number and the role text.
Who / what triggers it	Hibernate, the library behind Spring Data JPA, finds this class during start-up because of @Entity . With spring.jpa.hibernate.ddl-auto=update (see application.properties) it executes a CREATE TABLE statement for it before the server accepts the first request. Later the class is used at run time whenever a user registers or logs in.
What it uses / calls	jakarta.persistence.* (the JPA annotations: @Entity , @Table , @Id , @GeneratedValue , GenerationType) and Jackson's @JsonProperty (the JSON converter that Spring MVC uses). UserRepo reads and writes it, DriverRequest and Feedback point to it with a foreign key, and UserPrinciple copies data out of it for Spring Security.
Size	79 lines

Lines 1-4 | Package and imports

```
1 | package com.examly.springapp.model;
2 |
3 | import com.fasterxml.jackson.annotation.JsonProperty;
4 | import jakarta.persistence.*;
```

Line 1 puts the class in the package **com.examly.springapp.model**, the folder that holds all database classes. Line 3 imports **JsonProperty**, which controls how Jackson converts this object to and from JSON. Line 4 imports every JPA annotation with one star; the ones used below are **@Entity**, **@Table**, **@Id** and **@GeneratedValue**. Note **jakarta.persistence** and not **javax.persistence**: Spring Boot 3 moved to the Jakarta names.

Lines 6-8 | Entity and table name

```
6 | @Entity
7 | @Table(name = "users")
8 | public class User {
```

@Entity tells Hibernate: "this class is a database table". **@Table(name = "users")** chooses the table name explicitly. The default would have been **user**, but **USER** is a reserved word in H2 and in many other databases, so the table is called **users** to avoid a SQL error.

Line 8 is the normal Java class declaration.

Lines 9-11 | Primary key with IDENTITY

```
9 |     @Id
10 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
11 |     private Long userId;
```

@Id marks **userId** as the **primary key**, the column that uniquely identifies a row. **@GeneratedValue(strategy = GenerationType.IDENTITY)** means that the database itself creates the number: the column is an auto-increment (identity) column, so the first user gets 1, the next 2, and so on. The application never has to choose an id; after **save(...)** Hibernate reads the generated value back into this field.

Spring Boot's default naming rule turns the Java name **userId** into the column name **user_id** (camelCase becomes snake_case). Other tables point to this column with their **userId** foreign key.

Line 13 | email column

```
13 |     private String email;
```

A plain field with no annotation becomes an ordinary column (**email**, type VARCHAR). The e-mail is the **login name** of the application: **MyUserDetailsService** looks users up with **findByEmail**. There is no **unique** constraint on the column; uniqueness is checked in the service layer (a second registration with the same e-mail gets HTTP 409).

Lines 14-15 | password column, never sent back to the browser

```
14 |     @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
15 |     private String password;
```

`@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)` is the security detail of this class. **WRITE_ONLY** means: Jackson may **read** the password from incoming JSON (the register form posts `{"password": "..."}`), but it must never **write** it into outgoing JSON. So even when a `User` is returned from `/api/register`, or is nested inside a `DriverRequest` or `Feedback` response, the `password` key is simply missing from the JSON. What is stored in the column is not the typed password but its BCrypt hash (the service encodes it before saving).

Lines 16-18 | username, mobileNumber, userRole

```
16 |     private String username;
17 |     private String mobileNumber;
18 |     private String userRole;
```

Three more normal columns: `username` (display name, shown in the navigation bar), `mobile_number` and `user_role`. The role is stored as plain text such as `Admin` or `Customer` (the sign-up form's drop-down). `UserPrinciple.build` later converts that text into the Spring Security authority `ROLE_ADMIN` or `ROLE_CUSTOMER`.

Lines 20-30 | Constructors

```
20 |     public User() {
21 |     }
22 |
23 |     public User(Long userId, String email, String password, String username, String mobileNumber, String userRole) {
24 |         this.userId = userId;
25 |         this.email = email;
26 |         this.password = password;
27 |         this.username = username;
28 |         this.mobileNumber = mobileNumber;
29 |         this.userRole = userRole;
30 |     }
```

JPA requires a **no-argument constructor** (lines 20-21), because Hibernate and Jackson first create an empty object and then fill the fields. The second constructor (lines 23-30) takes all six values and copies each into the matching field with `this.x = x`; it is a convenience for code and tests that want to build a complete user in one line.

Lines 32-79 | Getters and setters

```
32 |     public Long getUserId() {
33 |         return userId;
34 |     }
35 |
36 |     public void setUserId(Long userId) {
37 |         this.userId = userId;
38 |     }
39 |
40 |     public String getEmail() {
41 |         return email;
42 |     }
43 |
44 |     public void setEmail(String email) {
45 |         this.email = email;
46 |     }
47 |
48 |     public String getPassword() {
49 |         return password;
50 |     }
51 |
52 |     public void setPassword(String password) {
53 |         this.password = password;
54 |     }
55 |
56 |     public String getUsername() {
57 |         return username;
58 |     }
59 |
60 |     public void setUsername(String username) {
61 |         this.username = username;
62 |     }
63 |
64 |     public String getMobileNumber() {
65 |         return mobileNumber;
66 |     }
67 |
68 |     public void setMobileNumber(String mobileNumber) {
69 |         this.mobileNumber = mobileNumber;
70 |     }
71 |
72 |     public String getUserRole() {
73 |         return userRole;
74 |     }
75 |
76 |     public void setUserRole(String userRole) {
77 |         this.userRole = userRole;
78 |     }
79 | }
```

For each of the six fields there is a **getter** (`getEmail()` returns the value) and a **setter** (`setEmail(x)` stores a value). They are not decoration: Jackson uses them to turn JSON into a `User` and back, Hibernate can use them, and the service layer calls them. The pattern is identical for every field, and nothing special happens inside them. Because Jackson sees `getPassword()` together with the `WRITE_ONLY` annotation on the field, the password is skipped when writing JSON but `setPassword()` is still used when reading it.

Workflow, next: The `users` table is defined: `user_id` (identity key), `email`, `password`, `username`, `mobile_number`, `user_role`. The next class is not a table at all but a small helper object that the login endpoint returns to the browser: `LoginDTO.java`.

Step 5: LoginDTO - the answer to a successful login

[Aleena - Authentication]

BACKEND

springapp/src/main/java/com/examly/springapp/model/LoginDTO.java

What this file is

A **DTO** (Data Transfer Object) is a plain class whose only job is to carry data between two parts of a system. `LoginDTO` carries the result of a successful login from the server to the Angular app: the JWT token, the user's display name, the role and the id. It has **no** `@Entity` annotation, so Hibernate ignores it and no table is created for it. It exists only in memory for the moment between building the answer and converting it to JSON.

Who / what triggers it

`AuthController.authenticateUser` (the method behind `POST /api/login`) creates one with `new LoginDTO(jwt, user.getUsername(), user.getUserRole(), user.getUserId())` after the password has been checked.

What it uses / calls	Nothing. It is a pure data holder. Spring MVC converts it to JSON, and the Angular login page stores the four values in <code>localStorage</code> under the keys <code>token</code> , <code>username</code> , <code>userRole</code> and <code>userId</code> .
Size	50 lines

Lines 1-7 | Package and the four fields

```

1 | package com.examly.springapp.model;
2 |
3 | public class LoginDTO {
4 |     private String token;
5 |     private String username;
6 |     private String userRole;
7 |     private Long userId;

```

The class lives in the `model` package. It has four private fields: `token` (the signed JWT string that the browser must send in the `Authorization: Bearer ...` header from now on), `username` (name to display), `userRole` (`Admin` or `Customer`, used by Angular to decide which menu and pages to show) and `userId` (the number that Angular later puts into URLs such as `/api/driverRequest/user/{userId}`). Note that the password is deliberately **not** part of this class.

Lines 9-17 | Constructors

```

9 |     public LoginDTO() {
10 |     }
11 |
12 |     public LoginDTO(String token, String username, String userRole, Long userId) {
13 |         this.token = token;
14 |         this.username = username;
15 |         this.userRole = userRole;
16 |         this.userId = userId;
17 |     }

```

The empty constructor lets Jackson or tests create a blank object and fill it with setters. The four-argument constructor is the one actually used by `AuthController`: it stores every parameter in the matching field.

Lines 19-50 | Getters and setters

```

19 |     public String getToken() {
20 |         return token;
21 |     }
22 |
23 |     public void setToken(String token) {
24 |         this.token = token;
25 |     }
26 |
27 |     public String getUsername() {
28 |         return username;
29 |     }
30 |
31 |     public void setUsername(String username) {
32 |         this.username = username;
33 |     }
34 |
35 |     public String getUserRole() {
36 |         return userRole;
37 |     }
38 |
39 |     public void setUserRole(String userRole) {
40 |         this.userRole = userRole;
41 |     }
42 |
43 |     public Long getUserId() {
44 |         return userId;
45 |     }
46 |
47 |     public void setUserId(Long userId) {
48 |         this.userId = userId;
49 |     }
50 | }

```

Standard accessor pairs for the four fields. They matter because Jackson builds the JSON answer by calling the getters, so the browser receives exactly `{"token": "...", "username": "...", "userRole": "...", "userId": 1}`. The setters are there for completeness and for tests that deserialize the answer.

Workflow, next: With the user data and its login answer defined, the next entity is the thing customers actually book: `Driver.java` (table `driver`).

Step 6: Driver - the drivers that can be hired (table driver) [Arjun - Driver & Booking]

BACKEND

springapp/src/main/java/com/examly/springapp/model/Driver.java

What this file is	Entity for the chauffeurs offered by DriveU. Each row stores the name, licence number, experience, contact number, availability text, address, vehicle type, hourly rate and a photo. The admin creates and edits drivers; customers browse them and pick one when they raise a driver request.
Who / what triggers it	Hibernate creates the driver table from this class at start-up. At run time it is used by DriverRepo , DriverServiceImpl , DriverController , and by the DriverRequest and Feedback entities that reference a driver.
What it uses / calls	jakarta.persistence.* annotations only. There are no relations declared inside Driver itself; the <i>*other*</i> entities hold the foreign key to it.
Size	106 lines

Lines 1-3 | Package and import

```
1 | package com.examly.springapp.model;
2 |
3 | import jakarta.persistence.*;
```

com.examly.springapp.model is the package of all entities; **jakarta.persistence.*** brings in **@Entity**, **@Id**, **@GeneratedValue**, **@Lob**, **@Column** and **GenerationType**.

Lines 5-6 | Entity

```
5 | @Entity
6 | public class Driver {
```

@Entity makes this a table. There is no **@Table**, so the table name follows Spring Boot's default naming: class **Driver** becomes the table **driver**.

Lines 7-9 | Identity primary key

```
7 |     @Id
8 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
9 |     private Long driverId;
```

driverId is the primary key (**@Id**) and its value is created by the database as an auto-increment number (**GenerationType.IDENTITY**). The column is named **driver_id**. This is the number that appears in **DriverRequest.driverId**, in **Feedback.driverId** and in URLs like **/api/driver/5**.

Lines 11-18 | Plain data columns

```
11 |     private String driverName;
12 |     private String licenseNumber;
13 |     private Integer experienceYears;
14 |     private String contactNumber;
15 |     private String availabilityStatus;
16 |     private String address;
17 |     private String vehicleType;
18 |     private Double hourlyRate;
```

Eight ordinary fields, each becoming one column with the camelCase name converted to snake_case: **driver_name**, **license_number**, **experience_years** (a whole number, **Integer**), **contact_number**, **availability_status** (text such as Available), **address**, **vehicle_type** and **hourly_rate** (a decimal **Double**). The hourly rate is important: when a trip ends, the Angular app multiplies this rate by the trip duration to compute the fare. The licence number is checked for duplicates in the service layer through **DriverRepo.findByLicenseNumber**.

Lines 20-22 | Photo column (@Lob)

```
20 |     @Lob
21 |     @Column(columnDefinition = "LONGBLOB")
22 |     private String image;
```

@Lob means **Large Object**: the value may be much longer than the 255 characters of a normal text column. **@Column(columnDefinition = "LONGBLOB")** forces the exact database type of the column to **LONGBLOB** (MySQL's biggest binary type; H2, the database used here, accepts that name as an alias for its own BLOB type). The Java field itself is a **String**, so the driver's picture is kept as one very long piece of text, for example a Base64-encoded image sent from the admin form. Because it is stored inside the table, every response of **GET /api/driver** carries

all pictures; that is simple but makes the list larger.

Lines 24-25 | Constructor

```
24 | public Driver() {  
25 | }
```

The empty constructor is required by Hibernate (it creates a blank `Driver` and fills the fields afterwards) and by Jackson (which does the same when the admin form's JSON is read). There is no all-arguments constructor in this class.

Lines 27-106 | Getters and setters

```
27 | public Long getDriverId() {  
28 |     return driverId;  
29 | }  
30 |  
31 | public void setDriverId(Long driverId) {  
32 |     this.driverId = driverId;  
33 | }  
34 |  
35 | public String getDriverName() {  
36 |     return driverName;  
37 | }  
38 |  
39 | public void setDriverName(String driverName) {  
40 |     this.driverName = driverName;  
41 | }  
42 |  
43 | public String getLicenseNumber() {  
44 |     return licenseNumber;  
45 | }  
46 |  
47 | public void setLicenseNumber(String licenseNumber) {  
48 |     this.licenseNumber = licenseNumber;  
49 | }  
50 |  
51 | public Integer getExperienceYears() {  
52 |     return experienceYears;  
53 | }  
54 |  
55 | public void setExperienceYears(Integer experienceYears) {  
56 |     this.experienceYears = experienceYears;  
57 | }  
58 |  
59 | public String getContactNumber() {  
60 |     return contactNumber;  
61 | }  
62 |  
63 | public void setContactNumber(String contactNumber) {  
64 |     this.contactNumber = contactNumber;  
65 | }  
66 |  
67 | public String getAvailabilityStatus() {  
68 |     return availabilityStatus;  
69 | }  
70 |  
71 | public void setAvailabilityStatus(String availabilityStatus) {  
72 |     this.availabilityStatus = availabilityStatus;  
73 | }  
74 |  
75 | public String getAddress() {  
76 |     return address;  
77 | }  
78 |  
79 | public void setAddress(String address) {  
80 |     this.address = address;  
81 | }  
82 |  
83 | public String getVehicleType() {  
84 |     return vehicleType;  
85 | }  
86 |  
87 | public void setVehicleType(String vehicleType) {  
88 |     this.vehicleType = vehicleType;  
89 | }  
90 |  
91 | public Double getHourlyRate() {  
92 |     return hourlyRate;  
93 | }  
94 |  
95 | public void setHourlyRate(Double hourlyRate) {  
96 |     this.hourlyRate = hourlyRate;  
97 | }  
98 |  
99 | public String getImage() {  
100 |     return image;
```

```

101 |     }
102 |
103 |     public void setImage(String image) {
104 |         this.image = image;
105 |     }
106 | }

```

A getter and a setter for each of the ten fields, with no logic inside. They are called by Jackson (JSON to object and back), by Hibernate, and by `DriverServiceImpl.updateDriver` when it copies the edited values onto the stored driver.

Workflow, next: Next comes the entity that links a customer to a driver for one trip: `DriverRequest.java`. It is the first class that uses a relationship (foreign key) to both `User` and `Driver`.

Step 7: DriverRequest - one booking of a driver (table `driver_request`) [Arjun - Driver & Booking]

BACKEND

`springapp/src/main/java/com/examly/springapp/model/DriverRequest.java`

What this file is	The central business entity. One row is one trip request made by a customer. It records who asked (<code>user</code>), which driver was chosen (<code>driver</code>), when and where the trip is, its status (Pending, Approved, Rejected, Trip End, Closed), and, after the trip, the actual drop-off time, duration and the payment amount. The status chain is Pending, then Approved or Rejected (the admin may switch between those two), then Trip End (customer, only after Approved), then Closed (admin, only after Trip End). The chain itself is enforced in the Angular pages; this class only stores the text.
Who / what triggers it	Hibernate creates the table <code>driver_request</code> at start-up. At run time <code>DriverRequestRepo</code> , <code>DriverRequestServiceImpl</code> and <code>DriverRequestController</code> read and write it.
What it uses / calls	<code>User</code> and <code>Driver</code> (the two relationships), the JPA annotations, and the Java date classes <code>LocalDate</code> and <code>LocalTime</code> for dates and times.
Size	156 lines

Lines 1-5 | Package and imports

```

1 | package com.examly.springapp.model;
2 |
3 | import jakarta.persistence.*;
4 | import java.time.LocalDate;
5 | import java.time.LocalTime;

```

`jakarta.persistence.*` provides the annotations (`@Entity`, `@Id`, `@ManyToOne`, `@JoinColumn`...). `LocalDate` (a date without time) and `LocalTime` (a time without date) from `java.time` are the types of the trip date and time fields. Jackson turns them into ISO text such as `2025-03-14` and `09:30:00` in JSON, and Hibernate stores them in DATE and TIME columns.

Lines 7-8 | Entity

```

7 | @Entity
8 | public class DriverRequest {

```

`@Entity` with no `@Table` gives the table name `driver_request` (CamelCase class name converted to snake_case).

Lines 9-11 | Primary key

```

9 |     @Id
10 |     @GeneratedValue(strategy = GenerationType.IDENTITY)
11 |     private Long driverRequestId;

```

`driverRequestId` is the auto-increment (`IDENTITY`) primary key, column `driver_request_id`. The id appears in URLs such as `PUT /api/driverRequest/7`.

Lines 13-15 | Foreign key to the customer

```

13 |     @ManyToOne
14 |     @JoinColumn(name = "userId", nullable = false)
15 |     private User user;

```

`@ManyToOne` states the relationship: **many** requests can belong to **one** user. Hibernate therefore adds a column that points to the `users` table. `@JoinColumn(name = "userId", nullable = false)` names that foreign-key column (stored as `user_id` after Spring's naming rule) and makes it **NOT NULL**: a request cannot exist without the customer who made it. In Java the field is a whole `User` object, not just a number, so a loaded request carries the

full user (without the password because of WRITE_ONLY). By default a `@ManyToOne` is loaded eagerly, meaning the user row is fetched together with the request.

Lines 17-19 | Foreign key to the driver (optional)

```
17 | @ManyToOne
18 | @JoinColumn(name = "driverId", nullable = true)
19 | private Driver driver;
```

The same idea for the driver: many requests, one driver, column `driver_id` in the table. The difference is `nullable = true`: the database allows the column to be empty, so a request can exist while no driver is attached to it. When a driver is chosen, the whole `Driver` object is nested inside the request JSON. There is no cascade setting, so deleting a driver that is still referenced by requests is refused by the database; `DriverServiceImpl.deleteDriver` catches that failure and raises its own `DriverDeletionException`.

Lines 21-32 | Trip data columns

```
21 | private LocalDate requestDate;
22 | private String status;
23 | private LocalDate tripDate;
24 | private LocalTime timeSlot;
25 | private String pickupLocation;
26 | private String dropLocation;
27 | private String estimatedDuration;
28 | private Double paymentAmount;
29 | private String comments;
30 | private LocalTime actualDropTime;
31 | private LocalDate actualDropDate;
32 | private String actualDuration;
```

Twelve plain columns, written here in the order of the business process:

- `requestDate`: the day the request was created (the service fills it in if the browser did not send it).
- `status`: text such as Pending, Approved, Rejected, Trip End, Closed.
- `tripDate` and `timeSlot`: the day and start time the customer wants.
- `pickupLocation`, `dropLocation`: where the trip starts and ends.
- `estimatedDuration`: the customer's estimate, kept as text.
- `paymentAmount`: the fare, a `Double`. Angular computes it as hourly rate x duration when the trip ends and sends it back with a PUT.
- `comments`: free text from the customer.
- `actualDropTime`, `actualDropDate`, `actualDuration`: filled in only when the customer ends the trip; they stay empty (NULL) before that.

Every name becomes snake_case in the database (`pickup_location`, `actual_drop_time`, and so on).

Lines 34-35 | Constructor

```
34 | public DriverRequest() {
35 | }
```

Only the empty constructor exists. Hibernate and Jackson create a blank object and use the setters to fill it, which is why no other constructor is needed.

Lines 37-156 | Getters and setters

```
37 | public Long getDriverRequestId() {
38 |     return driverRequestId;
39 | }
40 |
41 | public void setDriverRequestId(Long driverRequestId) {
42 |     this.driverRequestId = driverRequestId;
43 | }
44 |
45 | public User getUser() {
46 |     return user;
47 | }
48 |
49 | public void setUser(User user) {
50 |     this.user = user;
51 | }
52 |
53 | public Driver getDriver() {
54 |     return driver;
55 | }
56 |
57 | public void setDriver(Driver driver) {
58 |     this.driver = driver;
59 | }
60 |
61 | public LocalDate getRequestDate() {
62 |     return requestDate;
63 | }
64 |
65 | public void setRequestDate(LocalDate requestDate) {
66 |     this.requestDate = requestDate;
67 | }
68 |
69 | public String getStatus() {
70 |     return status;
71 | }
72 |
73 | public void setStatus(String status) {
74 |     this.status = status;
75 | }
76 |
77 | public LocalDate getTripDate() {
78 |     return tripDate;
79 | }
80 |
81 | public void setTripDate(LocalDate tripDate) {
82 |     this.tripDate = tripDate;
83 | }
84 |
85 | public LocalTime getTimeSlot() {
86 |     return timeSlot;
87 | }
88 |
89 | public void setTimeSlot(LocalTime timeSlot) {
90 |     this.timeSlot = timeSlot;
91 | }
92 |
93 | public String getPickupLocation() {
94 |     return pickupLocation;
95 | }
96 |
97 | public void setPickupLocation(String pickupLocation) {
98 |     this.pickupLocation = pickupLocation;
99 | }
100 |
101 | public String getDropLocation() {
102 |     return dropLocation;
103 | }
104 |
105 | public void setDropLocation(String dropLocation) {
106 |     this.dropLocation = dropLocation;
107 | }
108 |
109 | public String getEstimatedDuration() {
110 |     return estimatedDuration;
111 | }
112 |
113 | public void setEstimatedDuration(String estimatedDuration) {
114 |     this.estimatedDuration = estimatedDuration;
115 | }
116 |
117 | public Double getPaymentAmount() {
118 |     return paymentAmount;
119 | }
120 |
121 | public void setPaymentAmount(Double paymentAmount) {
122 |     this.paymentAmount = paymentAmount;
```

```

123 |     }
124 |
125 |     public String getComments() {
126 |         return comments;
127 |     }
128 |
129 |     public void setComments(String comments) {
130 |         this.comments = comments;
131 |     }
132 |
133 |     public LocalDateTime getActualDropTime() {
134 |         return actualDropTime;
135 |     }
136 |
137 |     public void setActualDropTime(LocalDateTime actualDropTime) {
138 |         this.actualDropTime = actualDropTime;
139 |     }
140 |
141 |     public LocalDate getActualDropDate() {
142 |         return actualDropDate;
143 |     }
144 |
145 |     public void setActualDropDate(LocalDate actualDropDate) {
146 |         this.actualDropDate = actualDropDate;
147 |     }
148 |
149 |     public String getActualDuration() {
150 |         return actualDuration;
151 |     }
152 |
153 |     public void setActualDuration(String actualDuration) {
154 |         this.actualDuration = actualDuration;
155 |     }
156 | }

```

A getter/setter pair for each of the fourteen fields (id, user, driver and the twelve data fields). Jackson uses them to read the JSON of `POST /api/driverRequest` and `PUT /api/driverRequest/{id}`, and `DriverRequestServiceImpl.updateDriverRequest` calls the setters to merge only the fields that the browser actually sent.

Workflow, next: Another entity that links a customer and a driver follows the same pattern: `Feedback.java`, the review a customer writes about a driver.

Step 8: Feedback - reviews written by customers (table feedback) [Saumya - Feedback & AI]

BACKEND

springapp/src/main/java/com/examly/springapp/model/Feedback.java

What this file is	Entity for a customer's review. It stores the text, the date, a category, a star rating, who wrote it and which driver it is about. Three extra columns (<code>sentiment</code> , <code>sentimentScore</code> , <code>aiTags</code>) hold the result of the AI analysis that the backend runs when the feedback is saved.
Who / what triggers it	Hibernate creates the table <code>feedback</code> at start-up. At run time <code>FeedbackRepo</code> , <code>FeedbackServiceImpl</code> , <code>AIService</code> and <code>FeedbackController</code> use it.
What it uses / calls	<code>User</code> and <code>Driver</code> (many-to-one relationships) and the JPA annotations; <code>LocalDate</code> for the date.
Size	112 lines

Lines 1-4 | Package and imports

```

1 | package com.examly.springapp.model;
2 |
3 | import jakarta.persistence.*;
4 | import java.time.LocalDate;

```

JPA annotations from `jakarta.persistence.*` and `LocalDate` from `java.time` for the feedback date.

Lines 6-7 | Entity

```

6 | @Entity
7 | public class Feedback {

```

`@Entity` on class `Feedback`: the table is called `feedback` by the default naming rule.

Lines 8-10 | Primary key

```
8 | @Id
9 | @GeneratedValue(strategy = GenerationType.IDENTITY)
10 | private Long feedbackId;
```

`feedbackId` is the auto-increment (`IDENTITY`) key, column `feedback_id`, used in `DELETE /api/feedback/{id}`.

Lines 12-13 | Text and date

```
12 | private String feedbackText;
13 | private LocalDate date;
```

`feedbackText` (column `feedback_text`) is what the customer wrote; `date` is the day it was written (a `LocalDate`, so only year-month-day).

Lines 15-17 | Foreign key to the author

```
15 | @ManyToOne
16 | @JoinColumn(name = "userId", nullable = false)
17 | private User user;
```

Many feedback rows can belong to one user. The column `userId` (stored as `user_id`) is a foreign key to `users` and is `nullable = false`: every feedback must have an author. `FeedbackRepo.findByIdUserUserId` uses exactly this relationship to list a customer's own reviews.

Lines 19-21 | Foreign key to the driver (optional)

```
19 | @ManyToOne
20 | @JoinColumn(name = "driverId", nullable = true)
21 | private Driver driver;
```

Many feedback rows can be about one driver, via the column `driverId` (stored as `driver_id`). `nullable = true` allows general feedback that is not about a particular driver, although the normal screens always select a driver. `AiService` uses `findByDriverDriverId` to gather all feedback of one driver for its summary.

Lines 23-24 | Category and rating

```
23 | private String category;
24 | private Integer rating;
```

`category` is a text label chosen by the customer (for example the topic of the feedback) and `rating` is a whole-number score stored in an `Integer` column.

Lines 26-28 | AI analysis columns

```
26 | private String sentiment;
27 | private Double sentimentScore;
28 | private String aiTags;
```

Added for the AI feature: `sentiment` (a word such as Positive, Neutral or Negative), `sentimentScore` (a decimal number measuring how positive the text is) and `aiTags` (a text list of keywords). The browser does not fill them: `FeedbackServiceImpl.createFeedback` calls `AiService.applyAnalysis`, which sets them before saving. Column names are `sentiment`, `sentiment_score` and `ai_tags`.

Lines 30-31 | Constructor

```
30 | public Feedback() {
31 | }
```

Only the empty constructor, for Hibernate and Jackson.

Lines 33-112 | Getters and setters

```
33 | public Long getFeedbackId() {
34 |     return feedbackId;
35 | }
36 |
37 | public void setFeedbackId(Long feedbackId) {
38 |     this.feedbackId = feedbackId;
39 | }
40 |
41 | public String getFeedbackText() {
42 |     return feedbackText;
43 | }
44 |
45 | public void setFeedbackText(String feedbackText) {
46 |     this.feedbackText = feedbackText;
47 | }
48 |
49 | public LocalDate getDate() {
50 |     return date;
51 | }
52 |
53 | public void setDate(LocalDate date) {
54 |     this.date = date;
55 | }
56 |
57 | public User getUser() {
58 |     return user;
59 | }
60 |
61 | public void setUser(User user) {
62 |     this.user = user;
63 | }
64 |
65 | public Driver getDriver() {
66 |     return driver;
67 | }
68 |
69 | public void setDriver(Driver driver) {
70 |     this.driver = driver;
71 | }
72 |
73 | public String getCategory() {
74 |     return category;
75 | }
76 |
77 | public void setCategory(String category) {
78 |     this.category = category;
79 | }
80 |
81 | public Integer getRating() {
82 |     return rating;
83 | }
84 |
85 | public void setRating(Integer rating) {
86 |     this.rating = rating;
87 | }
88 |
89 | public String getSentiment() {
90 |     return sentiment;
91 | }
92 |
93 | public void setSentiment(String sentiment) {
94 |     this.sentiment = sentiment;
95 | }
96 |
97 | public Double getSentimentScore() {
98 |     return sentimentScore;
99 | }
100 |
101 | public void setSentimentScore(Double sentimentScore) {
102 |     this.sentimentScore = sentimentScore;
103 | }
104 |
105 | public String getAiTags() {
106 |     return aiTags;
107 | }
108 |
109 | public void setAiTags(String aiTags) {
110 |     this.aiTags = aiTags;
111 | }
112 | }
```

A getter and a setter for each of the ten fields. Nothing special happens inside; Jackson uses them when reading `POST /api/feedback` and writing the list responses, and the service uses them to set the AI fields.

Workflow, next: The last entity is not business data but a record of problems: `ErrorLog.java`, the table where the global exception handler writes every handled error.

Step 9: ErrorLog - a stored record of every handled error (table ErrorLogs) [Vaishalini - Customer Journey & Shared Infra]

BACKEND

springapp/src/main/java/com/examly/springapp/model/ErrorLog.java

What this file is	Entity for an error journal. Each time the backend catches an exception and turns it into an HTTP error answer, one <code>ErrorLog</code> row is saved with the time, the HTTP status, the Java exception type, the message and the URL that was being called. This gives the team a permanent list of what went wrong, even after the console output has scrolled away.
Who / what triggers it	Hibernate creates the table at start-up. At run time <code>GlobalExceptionHandler.respond</code> builds one with <code>new ErrorLog(status.value(), ex.getClass().getName(), ex.getMessage(), request.getRequestURI())</code> and saves it through <code>ErrorLogRepo</code> .
What it uses / calls	<code>jakarta.persistence.*</code> , <code>LocalDateTime</code> (date plus time) from <code>java.time</code> . It has no relationship to the other entities.
Size	81 lines

Lines 1-4 | Package and imports

```
1 | package com.examly.springapp.model;
2 |
3 | import jakarta.persistence.*;
4 | import java.time.LocalDateTime;
```

JPA annotations and `LocalDateTime`, the type used to remember the exact moment of the error.

Lines 6-9 | Entity and table name

```
6 | /** One row per handled exception / error, stored in the "ErrorLogs" table. */
7 | @Entity
8 | @Table(name = "ErrorLogs")
9 | public class ErrorLog {
```

The comment on line 6 is a Javadoc comment saying one row is stored per handled exception. `@Entity` makes the class a table and `@Table(name = "ErrorLogs")` names it **ErrorLogs**. In practice Spring Boot's naming rule and H2's habit of upper-casing unquoted names turn this into the physical table `ERROR_LOGS`, which is the name you will see in the H2 console.

Lines 10-12 | Primary key

```
10 | @Id
11 | @GeneratedValue(strategy = GenerationType.IDENTITY)
12 | private Long errorLogId;
```

`errorLogId` is an auto-increment (`IDENTITY`) key, column `error_log_id`.

Lines 14-16 | When, which status, which exception

```
14 | private LocalDateTime loggedAt;
15 | private int status;
16 | private String exceptionType;
```

`loggedAt` (column `logged_at`) is the moment the error was recorded; `status` is the HTTP code that was returned (404, 409, 500...; it is a primitive `int`, so it can never be empty); `exceptionType` is the full Java class name of the exception, for example `java.lang.RuntimeException`.

Lines 18-19 | Message column with length 1000

```
18 | @Column(length = 1000)
19 | private String message;
```

Normally a text field becomes `VARCHAR(255)`. `@Column(length = 1000)` widens this column to 1000 characters because exception messages are often long. The constructor below makes sure the text is never longer than that.

Line 21 | Request path

```
21 | private String path;
```

`path` stores the URL of the request that failed, such as `/api/driver`, so you can see where the problem happened.

Lines 23-24 | Empty constructor

```
23 |     public ErrorLog() {  
24 |     }
```

Required by Hibernate, which creates a blank object when it reads a row.

Lines 26-32 | Convenience constructor

```
26 |     public ErrorLog(int status, String exceptionType, String message, String path) {  
27 |         this.loggedAt = LocalDateTime.now();  
28 |         this.status = status;  
29 |         this.exceptionType = exceptionType;  
30 |         this.message = message != null && message.length() > 1000 ? message.substring(0, 1000) : message;  
31 |         this.path = path;  
32 |     }
```

This constructor is what the exception handler calls. It copies the four values it receives and sets `loggedAt` to `LocalDateTime.now()`, so the caller never has to supply the time. Line 30 protects the 1000-character column: if the message is not null **and** longer than 1000 characters (`message != null && message.length() > 1000`) it is cut to the first 1000 with `substring(0, 1000)`; otherwise it is stored unchanged (including null). Without this, a very long message would make the INSERT fail.

Lines 34-81 | Getters and setters

```
34 |     public Long getErrorLogId() {  
35 |         return errorLogId;  
36 |     }  
37 |  
38 |     public void setErrorLogId(Long errorLogId) {  
39 |         this.errorLogId = errorLogId;  
40 |     }  
41 |  
42 |     public LocalDateTime getLoggedAt() {  
43 |         return loggedAt;  
44 |     }  
45 |  
46 |     public void setLoggedAt(LocalDateTime loggedAt) {  
47 |         this.loggedAt = loggedAt;  
48 |     }  
49 |  
50 |     public int getStatus() {  
51 |         return status;  
52 |     }  
53 |  
54 |     public void setStatus(int status) {  
55 |         this.status = status;  
56 |     }  
57 |  
58 |     public String getExceptionType() {  
59 |         return exceptionType;  
60 |     }  
61 |  
62 |     public void setExceptionType(String exceptionType) {  
63 |         this.exceptionType = exceptionType;  
64 |     }  
65 |  
66 |     public String getMessage() {  
67 |         return message;  
68 |     }  
69 |  
70 |     public void setMessage(String message) {  
71 |         this.message = message;  
72 |     }  
73 |  
74 |     public String getPath() {  
75 |         return path;  
76 |     }  
77 |  
78 |     public void setPath(String path) {  
79 |         this.path = path;  
80 |     }  
81 | }
```

A getter and a setter for the six fields. They are used by Hibernate and available for any code that wants to read the log.

Workflow, next: All five tables are now defined (`users`, `driver`, `driver_request`, `feedback`, `error_logs`) and Hibernate creates them while the context starts. Next the classes that read and write those tables are built. The first repository is `UserRepo.java`.

Step 10: UserRepo - database access for users [Aleena - Authentication]

BACKEND

springapp/src/main/java/com/examly/springapp/repository/UserRepo.java

What this file is	A repository is the layer that talks to the database. With Spring Data JPA you do not write the class at all: you write an <i>*interface*</i> that extends JpaRepository , and Spring creates the implementation (a proxy object) at start-up. That generated object already knows save , findById , findAll , deleteById , count , and so on, and it runs the SQL for you. UserRepo is the repository for the users table.
Who / what triggers it	Spring's component scan finds it because of @Repository and the fact that it extends JpaRepository ; it creates the implementation bean while the application context starts. It is then injected (handed in automatically) into MyUserDetailsService and UserServiceImpl .
What it uses / calls	User (the entity) and Spring Data JPA's JpaRepository . Behind the scenes Hibernate sends SQL to the H2 database.
Size	12 lines

Lines 1-7 | Package and imports

```
1 | package com.examly.springapp.repository;
2 |
3 | import com.examly.springapp.model.User;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 | import org.springframework.stereotype.Repository;
6 |
7 | import java.util.Optional;
```

The **repository** package. Imports: the **User** entity, **JpaRepository** (the base interface providing ready-made database methods), **@Repository** (marks the type as a data-access component) and **Optional** (a container that holds either one value or nothing, used instead of returning **null**).

Lines 9-12 | The interface and findByIdByEmail

```
9 | @Repository
10 | public interface UserRepo extends JpaRepository<User, Long> {
11 |     Optional<User> findByIdByEmail(String email);
12 | }
```

@Repository marks the interface as a bean and enables translation of database exceptions into Spring's own exception types.

extends JpaRepository<User, Long> says: "give me the standard database methods for the entity **User**, whose primary key is of type **Long**".

Optional<User> findByIdByEmail(String email); is a **derived query method**: there is no body and no SQL, yet it works. Spring Data reads the **method name**: the prefix **findBy** means "select", the next word **Email** is matched to the field **email** of **User**, so it builds **select * from users where email = ?** and binds the argument. If a row is found the **Optional** holds the **User**; if not, it is empty. Callers: **MyUserDetailsService.loadUserByUsername** (to find the person who is logging in or whose token arrived) and **UserServiceImpl** (to see whether an e-mail is already registered).

Workflow, next: Users can now be read and saved. The next table's repository is **DriverRepo.java**.

Step 11: DriverRepo - database access for drivers [Arjun - Driver & Booking]

BACKEND

springapp/src/main/java/com/examly/springapp/repository/DriverRepo.java

What this file is	The Spring Data repository for the driver table. Like all repositories in this project it is only an interface; Spring generates the working class when the application starts.
Who / what triggers it	Created by Spring at start-up and injected into DriverServiceImpl (and AiService , which reads the drivers when searching).
What it uses / calls	Driver entity, JpaRepository .
Size	11 lines

Lines 1-6 | Package and imports

```
1 | package com.examly.springapp.repository;
2 |
3 | import com.examly.springapp.model.Driver;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 | import org.springframework.stereotype.Repository;
6 | import java.util.Optional;
```

Imports the **Driver** entity, **JpaRepository**, **@Repository** and **Optional**.

Lines 8-11 | Interface and findByLicenseNumber

```
8 | @Repository
9 | public interface DriverRepo extends JpaRepository<Driver, Long> {
10 |     Optional<Driver> findByLicenseNumber(String licenseNumber);
11 | }
```

@Repository registers it as a bean. **JpaRepository<Driver, Long>** supplies **findAll()** (list every driver), **findById(id)**, **save(driver)** (insert or update), **deleteById(id)** and more, with **Long** as the key type. **findByLicenseNumber(String)** is derived from its name: **findBy** + the field **licenseNumber** gives **select * from driver where license_number = ?**. It returns an **Optional** because there may be no such driver. **DriverServiceImpl.addDriver** calls it before inserting: if a driver with that licence number is already there, it raises **DuplicateDriverException**, which ends up as HTTP **409 Conflict**.

Workflow, next: Next is the repository for the busiest table: **DriverRequestRepo.java**, which has the first queries that follow a relationship.

Step 12: DriverRequestRepo - queries over booking requests [Arjun - Driver & Booking]

BACKEND

springapp/src/main/java/com/examly/springapp/repository/DriverRequestRepo.java

What this file is	The Spring Data repository for driver_request . Besides the standard methods it declares two queries that select requests by the owner of the foreign key, one by customer and one by driver.
Who / what triggers it	Spring generates it at start-up; DriverRequestServiceImpl receives it by injection and calls it for the pages 'my requests' (customer) and for the admin's view of requests per driver.
What it uses / calls	DriverRequest entity, JpaRepository , java.util.List .
Size	13 lines

Lines 1-7 | Package and imports

```
1 | package com.examly.springapp.repository;
2 |
3 | import com.examly.springapp.model.DriverRequest;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 | import org.springframework.stereotype.Repository;
6 |
7 | import java.util.List;
```

Imports the entity, `JpaRepository`, `@Repository` and `List` (these queries can return many rows, so the return type is a `List`, which is empty when nothing matches).

Lines 9-13 | Interface and the two relationship queries

```
9 | @Repository
10 | public interface DriverRequestRepo extends JpaRepository<DriverRequest, Long> {
11 |     List<DriverRequest> findByUserUserId(Long userId);
12 |     List<DriverRequest> findByDriverDriverId(Long driverId);
13 | }
```

`JpaRepository<DriverRequest, Long>` provides `save`, `findAll`, `findById` and `deleteById`, used for create, list-all, get-one, update and delete.

How `findByUserUserId(Long userId)` is parsed. Spring Data splits the name at capital letters into the words `find`, `By`, `User`, `User`, `Id`.

1. `find` and `By` are the keywords: "select where".
2. It then tries to match the longest possible property path from the entity `DriverRequest`. `User` is a field of `DriverRequest` (the `User user` relationship), so it walks into it.
3. In the `User` class it looks for the next property: `UserId` becomes `userId`, the primary key field of `User`.
4. The path is therefore `user.userId`, and the generated SQL is roughly `select ... from driver_request where user_id = ?`. Because the id is stored in the foreign-key column, Hibernate does not even need to join the `users` table.

So the doubled word is not a typo: the first `User` is the field in `DriverRequest`, the second part `UserId` is the field inside `User`.

`findByDriverDriverId(Long driverId)` follows the same rule with the path `driver.driverId` and gives `where driver_id = ?`. The first query serves `GET /api/driverRequest/user/{userId}` (customer), the second serves `GET /api/driverRequest/driver/{driverId}` (admin).

Workflow, next: Feedback has the same two relationships, so its repository is almost a copy:
`FeedbackRepo.java`.

Step 13: FeedbackRepo - queries over feedback [Saumya - Feedback & AI]

BACKEND

`springapp/src/main/java/com/examly/springapp/repository/FeedbackRepo.java`

What this file is	The Spring Data repository for the <code>feedback</code> table, with two queries based on the foreign keys: all feedback of a customer and all feedback about a driver.
Who / what triggers it	Generated by Spring at start-up and injected into <code>FeedbackServiceImpl</code> and <code>AiService</code> .
What it uses / calls	<code>Feedback</code> entity, <code>JpaRepository</code> , <code>List</code> .
Size	13 lines

Lines 1-7 | Package and imports

```
1 | package com.examly.springapp.repository;
2 |
3 | import com.examly.springapp.model.Feedback;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 | import org.springframework.stereotype.Repository;
6 |
7 | import java.util.List;
```

The entity, `JpaRepository`, `@Repository` and `java.util.List`.

Lines 9-13 | Interface and queries

```
9 | @Repository
10 | public interface FeedbackRepo extends JpaRepository<Feedback, Long> {
11 |     List<Feedback> findByUserUserId(Long userId);
12 |     List<Feedback> findByDriverDriverId(Long driverId);
13 | }
```

`JpaRepository<Feedback, Long>` supplies `save`, `findAll`, `findById`, `deleteById`.

`findByUserUserId(Long userId)` is parsed exactly as in `DriverRequestRepo`: the property path `user.userId`, producing where `user_id = ?`. `FeedbackServiceImpl` uses it for `GET /api/feedback/user/{userId}` (the customer's own reviews).

`findByDriverDriverId(Long driverId)` uses the path `driver.driverId` (where `driver_id = ?`).

`AIService.summarizeDriverFeedback` calls it to collect everything written about one driver before building the summary.

Workflow, next: The last repository is the simplest one: `ErrorLogRepo.java`.

Step 14: ErrorLogRepo - saving error records [Vaishalini - Customer Journey & Shared Infra]

BACKEND

springapp/src/main/java/com/examly/springapp/repository/ErrorLogRepo.java

What this file is	The Spring Data repository for the <code>ErrorLogs</code> table. It has no custom queries at all: the inherited <code>save(...)</code> is all the project needs.
Who / what triggers it	Generated by Spring at start-up and injected into <code>GlobalExceptionHandler</code> , which calls <code>errorLogRepo.save(new ErrorLog(...))</code> for each handled error.
What it uses / calls	<code>ErrorLog</code> entity, <code>JpaRepository</code> .
Size	9 lines

Lines 1-5 | Package and imports

```
1 | package com.examly.springapp.repository;
2 |
3 | import com.examly.springapp.model.ErrorLog;
4 | import org.springframework.data.jpa.repository.JpaRepository;
5 | import org.springframework.stereotype.Repository;
```

The entity, `JpaRepository` and `@Repository`.

Lines 7-9 | Empty interface

```
7 | @Repository
8 | public interface ErrorLogRepo extends JpaRepository<ErrorLog, Long> {
9 | }
```

`@Repository` plus `extends JpaRepository<ErrorLog, Long>` is the whole definition. The empty body is fine: Spring still generates a full implementation with `save`, `findAll`, and the other inherited methods. Only `save` is used by the application.

Workflow, next: All five repositories are ready. Now Spring builds the security layer, the part that decides who may call which URL. Its main file, where all the rules are written, is `SecurityConfig.java`.

Step 15: SecurityConfig - the rulebook for every URL

[Vanshika - Security]

BACKEND

springapp/src/main/java/com/examly/springapp/config/SecurityConfig.java

What this file is	The heart of the security layer. It builds the security filter chain: an ordered list of small programs (filters) that every HTTP request passes through before it reaches a controller. A filter can look at the request, change it, or stop it and send back an error. This class defines (1) which URL needs which role, (2) that the server keeps no login session and trusts a JWT token instead, (3) how passwords are checked (BCrypt), (4) which handlers answer 401 and 403, and (5) where our own <code>JwtAuthenticationFilter</code> sits in the chain. Words used here: a bean is an object that Spring creates and manages; a method marked <code>@Bean</code> tells Spring "call me once and keep what I return as a bean". Dependency injection means a class does not create its helpers with <code>new</code>; Spring finds the right beans and hands them in (<code>@Autowired</code>).
Who / what triggers it	Spring Boot finds it during component scanning because of <code>@Configuration</code>, then calls the <code>@Bean</code> methods while building the context. Security is therefore in place before Tomcat takes its first request. After that, the chain built in <code>filterChain</code> runs for every single request.
What it uses / calls	<code>MyUserDetailsService</code>, <code>JwtAuthenticationEntryPoint</code>, <code>JwtAccessDeniedHandler</code>, <code>JwtUtils</code> (all injected), <code>JwtAuthenticationFilter</code> (created here), Spring Security classes (<code>HttpSecurity</code>, <code>DaoAuthenticationProvider</code>, <code>BCryptPasswordEncoder</code>, ...) and the CORS rules from <code>CorsConfig</code>.
Size	107 lines

Lines 1-16 | Package and imports

```
1 | package com.examly.springapp.config;
2 |
3 | import org.springframework.beans.factory.annotation.Autowired;
4 | import org.springframework.context.annotation.Bean;
5 | import org.springframework.context.annotation.Configuration;
6 | import org.springframework.http.HttpMethod;
7 | import org.springframework.security.authentication.AuthenticationManager;
8 | import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
9 | import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
10 | import org.springframework.security.config.annotation.web.builders.HttpSecurity;
11 | import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
12 | import org.springframework.security.config.http.SessionCreationPolicy;
13 | import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
14 | import org.springframework.security.crypto.password.PasswordEncoder;
15 | import org.springframework.security.web.SecurityFilterChain;
16 | import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
```

Each import is used below: `Autowired` (dependency injection), `Bean` and `Configuration` (bean definitions), `HttpMethod` (GET, POST... constants for the rules), `AuthenticationManager` and `DaoAuthenticationProvider` (login checking), `AuthenticationConfiguration` (source of the default manager), `HttpSecurity` (the builder for the filter chain), `EnableWebSecurity` (switches Spring Security's web support on), `SessionCreationPolicy` (stateless setting), `BCryptPasswordEncoder` and `PasswordEncoder` (password hashing), `SecurityFilterChain` (the finished chain) and `UsernamePasswordAuthenticationFilter` (the built-in filter that we use as an anchor to position ours).

Lines 18-20 | Class annotations

```
18 | @Configuration
19 | @EnableWebSecurity
20 | public class SecurityConfig {
```

`@Configuration` says "this class contains bean definitions", so Spring runs its `@Bean` methods. `@EnableWebSecurity` activates the Spring Security web infrastructure and lets this class replace the default security setup. Without it the Spring Boot default would protect every URL with a generated password and a login form.

Lines 22-32 | Injected helpers

```
22 | @Autowired
23 | private MyUserDetailsService userDetailsService;
24 |
25 | @Autowired
26 | private JwtAuthenticationEntryPoint unauthorizedHandler;
27 |
28 | @Autowired
29 | private JwtAccessDeniedHandler accessDeniedHandler;
30 |
31 | @Autowired
32 | private JwtUtils jwtUtils;
```

Four collaborators are injected by `@Autowired` field injection: `userDetailsService` (loads a user from the database), `unauthorizedHandler` (sends 401), `accessDeniedHandler` (sends 403) and `jwtUtils` (creates and checks tokens). Spring finds them because each is itself a bean (`@Service` or `@Component`).

Lines 34-37 | The JWT filter as a bean

```
34 | @Bean
35 | public JwtAuthenticationFilter authenticationJwtTokenFilter() {
36 |     return new JwtAuthenticationFilter(jwtUtils, userDetailsService);
37 | }
```

This method creates our `JwtAuthenticationFilter` and passes it `jwtUtils` and `userDetailsService`. The filter is built here with `new` instead of being annotated `@Component` so that this class controls its constructor arguments. Because the result is a `Filter` bean, Spring Boot would also register it in Tomcat's own filter list; that does not make it run twice, because the filter extends `OncePerRequestFilter`, which skips itself if it already ran for the same request.

Lines 39-45 | DaoAuthenticationProvider

```
39 | @Bean
40 | public DaoAuthenticationProvider authenticationProvider() {
41 |     DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
42 |     authProvider.setUserDetailsService(userDetailsService);
43 |     authProvider.setPasswordEncoder(passwordEncoder());
44 |     return authProvider;
45 | }
```

An **authentication provider** is the object that actually checks credentials. `DaoAuthenticationProvider` is the standard one that checks them against a database: given an e-mail and a typed password it (1) asks `userDetailsService` for the stored user (line 42), then (2) uses the `PasswordEncoder` (line 43) to compare the typed password with the stored BCrypt hash. Returns `authProvider` as a bean. The login endpoint reaches this provider through the `AuthenticationManager` below.

Lines 47-50 | AuthenticationManager bean

```
47 | @Bean
48 | public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig) throws Exception {
49 |     return authConfig.getAuthenticationManager();
50 | }
```

The `AuthenticationManager` is the front door of the authentication system: `AuthController` calls `authenticationManager.authenticate(...)` with the e-mail and password, and the manager hands the work to the provider. `authConfig.getAuthenticationManager()` asks Spring Security for the manager it has already assembled from the provider bean above. Exposing it as a bean is needed so that `AuthController` can inject it, because Spring does not publish it as a bean by default.

Lines 52-55 | Password encoder

```
52 | @Bean
53 | public PasswordEncoder passwordEncoder() {
54 |     return new BCryptPasswordEncoder();
55 | }
```

`BCryptPasswordEncoder` hashes passwords with **BCrypt**, a one-way algorithm that adds a random **salt** and is deliberately slow, so stored passwords cannot be read back and are costly to guess. `UserServiceImpl.createUser` calls `encode(rawPassword)` when registering; `DaoAuthenticationProvider` calls `matches(raw, hash)` when logging in. Making it a bean lets both places share the same instance.

Lines 57-60 | The filter chain method, and CORS

```
57 | @Bean
58 | public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
59 |     http
60 |         .cors(cors -> cors.configure(http))
```

`filterChain` is the most important method of the file: Spring Security passes an `HttpSecurity` builder, we configure it, and `http.build()` at the end returns the finished chain.

`.cors(cors -> cors.configure(http))` turns on **CORS** support, so Spring Security adds a filter at the very front that answers the browser's pre-flight question and adds the `Access-Control-Allow-...` headers. The actual rules (which origins, methods and headers) come from `CorsConfig`. This matters because the Angular page runs on a different port (8081) than the API (8080). Writing `cors.configure(http)` is an unusual style (the common one is `Customizer.withDefaults()`), but the result is that the MVC CORS configuration is used.

Lines 61-62 | CSRF off and stateless sessions

```
61 | .csrf(csrf -> csrf.disable())
62 | .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

`.csrf(csrf -> csrf.disable())` switches off **CSRF** protection. CSRF is an attack where another website makes your browser send a request together with your login *cookie*. Here there is no cookie login: the token is sent manually in the `Authorization` header, which another site cannot do, so the protection is unnecessary. (It would also block the POST requests of the H2 console.)

`SessionCreationPolicy.STATELESS` tells Spring Security never to create an HTTP session and never to remember a user between requests. The server keeps **no** record of who is logged in; every request must prove itself again with its JWT. This is why `JwtAuthenticationFilter` has to authenticate on every request.

Lines 63-66 | Error handlers: 401 vs 403

```
63 | .exceptionHandling(ex -> ex
64 |     .authenticationEntryPoint(unauthorizedHandler)
65 |     .accessDeniedHandler(accessDeniedHandler)
66 | )
```

`exceptionHandling` registers who answers when security says no:

- `authenticationEntryPoint(unauthorizedHandler)`: used when the caller is **not authenticated** at all (no token, bad token, expired token) and wants a protected URL. Result: HTTP **401 Unauthorized**.
- `accessDeniedHandler(accessDeniedHandler)`: used when the caller **is** logged in but their role is not allowed (a Customer calling an admin URL). Result: HTTP **403 Forbidden**.

401 means "who are you?", 403 means "I know who you are, and you may not do this".

Lines 67-69 | Authorization rules, part 1: register and login

```
67 | .authorizeHttpRequests(auth -> auth
68 |     // Public
69 |     .requestMatchers("/api/register", "/api/login").permitAll())
```

`authorizeHttpRequests` starts the list of URL rules. **Important: rules are checked from top to bottom and the first rule that matches wins**; later rules are never looked at for that request. That is why specific rules must come before general ones.

Line 69: `/api/register` and `/api/login` (any HTTP method) are `permitAll()`: anyone, even without a token, may call them. They have to be open, otherwise nobody could ever create an account or get the first token.

Lines 70-72 | Authorization rules, part 2: AI endpoints

```
70 | .requestMatchers(HttpMethod.POST, "/api/ai/driver-search").permitAll()
71 | .requestMatchers(HttpMethod.GET, "/api/ai/driver/**").hasAnyRole("ADMIN", "CUSTOMER")
72 | .requestMatchers(HttpMethod.POST, "/api/ai/feedback/analyze-existing").hasRole("ADMIN")
```

- Line 70: `POST /api/ai/driver-search` is public (`permitAll`), so even visitors who are not logged in can use the smart driver search on the home page.
- Line 71: `GET /api/ai/driver/...` (in practice `GET /api/ai/driver/{id}/feedback-summary`) needs `hasAnyRole("ADMIN", "CUSTOMER")`: any logged-in Admin or Customer. The comment above says "Public", but this line is not public; only the lines up to 70 and 73 are.
- Line 72: `POST /api/ai/feedback/analyze-existing` (re-analyse stored feedback) is for **ADMIN** only.

Spring adds the `ROLE_` prefix itself, so `hasRole("ADMIN")` checks for the authority `ROLE_ADMIN` (see `UserPrinciple`). A trailing `/...` in these explanations stands for the pattern's two-star wildcard (written as slash and two stars in the Java source), which means "this path and anything below it".

Line 73 | Authorization rules, part 3: H2 console

```
73 | .requestMatchers("/h2-console/**").permitAll()
```

`/h2-console/...` is open to everyone so that developers can browse the in-memory database in the browser (`spring.h2.console.enabled=true`). This is convenient for a student project and would be removed in a real deployment.

Lines 75-76 | Driver rules: public list

```
75 | // ===== Driver =====
76 | .requestMatchers(HttpMethod.GET, "/api/driver").permitAll()
```

Line 75 is a comment. Line 76: `GET /api/driver` (exact URL, no id) is `permitAll`: the list of drivers is public, so customers (and the home page for visitors) can see all drivers.

Lines 77-79 | Driver rules: admin writes

```
77 | .requestMatchers(HttpMethod.POST, "/api/driver").hasRole("ADMIN")
78 | .requestMatchers(HttpMethod.PUT, "/api/driver/**").hasRole("ADMIN")
79 | .requestMatchers(HttpMethod.DELETE, "/api/driver/**").hasRole("ADMIN")
```

Lines 77-79: `POST /api/driver` (add), `PUT /api/driver/...` (edit one) and `DELETE /api/driver/...` (remove one) need the role `ADMIN`. Customers can never change the driver catalogue.

Line 80 | Driver rules: single driver is admin-only, and why the order matters

```
80 | .requestMatchers(HttpMethod.GET, "/api/driver/**").hasRole("ADMIN")
```

`GET /api/driver/...` (for example `GET /api/driver/5`) is restricted to `ADMIN`. **Order matters here:** the pattern `/api/driver/...` also matches the bare `/api/driver`. If this line stood *above* line 76, the public driver list would require an admin. Because the exact permit rule sits earlier, it wins for `GET /api/driver`, and only the ones with an id fall through to this stricter rule.

Lines 82-86 | Driver request rules: specific URLs first

```
82 | // ===== DriverRequest =====
83 | .requestMatchers(HttpMethod.POST, "/api/driverRequest").hasRole("CUSTOMER")
84 | .requestMatchers(HttpMethod.GET, "/api/driverRequest").hasRole("ADMIN")
85 | .requestMatchers(HttpMethod.GET, "/api/driverRequest/user/**").hasRole("CUSTOMER")
86 | .requestMatchers(HttpMethod.GET, "/api/driverRequest/driver/**").hasRole("ADMIN")
```

- Line 83: `POST /api/driverRequest` (create a booking) is for `CUSTOMER` only.
- Line 84: `GET /api/driverRequest` (every request of everybody) is for `ADMIN` only.
- Line 85: `GET /api/driverRequest/user/...` (a customer's own requests) is for `CUSTOMER`.
- Line 86: `GET /api/driverRequest/driver/...` (requests of one driver) is for `ADMIN`.

These two narrow GET rules are deliberately listed **before** the wide GET rule on line 87. Otherwise `/api/driverRequest/user/3` would be caught by the wide rule and any logged-in user could read it.

Lines 87-89 | Driver request rules: general and delete

```
87 | .requestMatchers(HttpMethod.GET, "/api/driverRequest/**").authenticated()
88 | .requestMatchers(HttpMethod.PUT, "/api/driverRequest/**").authenticated()
89 | .requestMatchers(HttpMethod.DELETE, "/api/driverRequest/**").hasRole("CUSTOMER")
```

- Line 87: any other `GET /api/driverRequest/...`, which in practice is `GET /api/driverRequest/{id}`, only needs `authenticated()`: any logged-in user (admin or customer), because both need to open one request.
- Line 88: `PUT /api/driverRequest/...` is also `authenticated()`: the customer edits a Pending request and ends the trip, and the admin approves, rejects and closes; all of them use PUT. The finer rules (who may move which status) live in the Angular app.
- Line 89: `DELETE /api/driverRequest/...` is for `CUSTOMER` only (only when the request is still Pending, a rule checked by the app).

Lines 91-93 | Feedback rules: create and list

```
91 | // ===== Feedback =====
92 | .requestMatchers(HttpMethod.POST, "/api/feedback").hasRole("CUSTOMER")
93 | .requestMatchers(HttpMethod.GET, "/api/feedback").permitAll()
```

- Line 92: `POST /api/feedback` (write a review) is for `CUSTOMER`.
- Line 93: `GET /api/feedback` (all reviews) is `permitAll`, so reviews are public and can be shown to anybody.

Line 91 is only a comment.

Lines 94-96 | Feedback rules: own reviews, one review, delete

```
94 | .requestMatchers(HttpMethod.GET, "/api/feedback/user/**").hasRole("CUSTOMER")
95 | .requestMatchers(HttpMethod.GET, "/api/feedback/**").authenticated()
96 | .requestMatchers(HttpMethod.DELETE, "/api/feedback/**").hasRole("CUSTOMER")
```

- Line 94: `GET /api/feedback/user/...` (a customer's own reviews) is for `CUSTOMER`. It sits above line 95 for the same reason as the driver request rules.
- Line 95: any other `GET /api/feedback/...` (`GET /api/feedback/{id}`) needs `authenticated()`.
- Line 96: `DELETE /api/feedback/...` is for `CUSTOMER`.

Lines 98-99 | Catch-all rule

```
98 | .anyRequest().authenticated()
99 | );
```

`anyRequest().authenticated()` is the **default** for anything not mentioned above: the caller must be logged in, but any role will do. It must be last, because it matches everything. Line 99 closes the `authorizeHttpRequests(...)`

call and ends the statement started on line 59 with a semicolon.

Lines 101-103 | Provider, JWT filter position, frames

```
101 | http.authenticationProvider(authenticationProvider());
102 | http.addFilterBefore(authenticationJwtTokenFilter(), UsernamePasswordAuthenticationFilter.class);
103 | http.headers(headers -> headers.frameOptions(frameOptions -> frameOptions.disable()));
```

Line 101 registers our `DaoAuthenticationProvider` with the chain.

Line 102 is where our JWT check is plugged in: `addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)` puts `JwtAuthenticationFilter` **just before** the standard login filter. The form-login filter itself is not enabled in this app; its class is only used as a fixed place in the order. What matters is that the JWT filter runs **before** the authorization decision is made (the `AuthorizationFilter` at the end of the chain), so by the time the URL rules above are tested, the user is already known.

Line 103 disables the `X-Frame-Options` header. That header normally forbids showing the site inside a `<frame>`; the H2 console is built from frames and would show a blank page otherwise.

Lines 105-107 | Build and return

```
105 | return http.build();
106 | }
107 | }
```

`http.build()` turns all the settings into the finished `SecurityFilterChain`, which Spring registers as the bean that runs for every request. Lines 106-107 close the method and the class.

Workflow, next: The rules are set. Now we look at the pieces they use, starting with the class that builds and verifies the tokens: `JwtUtils.java`.

Step 16: JwtUtils - creating and checking tokens [Vanshika - Security]

BACKEND

springapp/src/main/java/com/examly/springapp/config/JwtUtils.java

What this file is	A JWT (JSON Web Token) is a short text that proves who you are. It has three parts joined by dots: <code>header.payload.signature</code> . The header says which algorithm signed it (here HS512), the payload contains the facts (who, when issued, when it expires) and the signature is a cryptographic stamp made with a secret key that only the server knows. The payload is only Base64-encoded, <i>*not*</i> encrypted, so anyone can read it; but nobody can change it without breaking the signature. <code>JwtUtils</code> has three jobs: create a token after login, check that a token is genuine and not expired, and read the user name (the e-mail) out of it.
Who / what triggers it	<code>AuthController.authenticateUser</code> calls <code>generateJwtToken</code> after a successful login. <code>JwtAuthenticationFilter</code> calls <code>validateJwtToken</code> and <code>getUserNameFromJwtToken</code> on every request that has a Bearer token.
What it uses / calls	The <code>jjwt</code> 0.9.1 library (<code>io.jsonwebtoken</code> , see <code>pom.xml</code>) and <code>UserPrincipal</code> (to read the e-mail from the logged-in user).
Size	46 lines

Lines 1-8 | Package and imports

```
1 | package com.examly.springapp.config;
2 |
3 | import io.jsonwebtoken.*;
4 | import org.springframework.beans.factory.annotation.Value;
5 | import org.springframework.security.core.Authentication;
6 | import org.springframework.stereotype.Component;
7 |
8 | import java.util.Date;
```

`io.jsonwebtoken.*` brings the JWT library (`Jwts`, `SignatureAlgorithm` and the exception classes). `Value` is imported but not used anywhere (a leftover). `Authentication` represents the logged-in user passed in at login, `Component` makes the class a bean and `Date` provides the timestamps.

Lines 10-11 | Bean declaration

```
10 | @Component
11 | public class JwtUtils {
```

@Component lets Spring create one `JwtUtils` and inject it into `SecurityConfig`, `AuthController` and (through the config) the filter.

Lines 13-15 | Secret key and lifetime

```
13 | private String jwtSecret = "driveuSecretKeyForJwtAuthenticationOnlyNotForProd";
14 |
15 | private int jwtExpirationMs = 86400000;
```

`jwtSecret` is the **secret key** used to sign and verify tokens. It is written directly in the code (the text itself warns "NotForProd"); in a real system it would come from an environment variable, the way the Gemini key does in `application.properties`. If anyone learns the secret, they can forge tokens, and if the secret changes, all old tokens become invalid.

`jwtExpirationMs = 86400000` is the lifetime in milliseconds: $24 \times 60 \times 60 \times 1000 = \text{one day}$. After 24 hours a token is rejected and the user has to log in again.

Lines 17-27 | generateJwtToken

```
17 | public String generateJwtToken(Authentication authentication) {
18 |
19 |     UserPrincipal userPrincipal = (UserPrincipal) authentication.getPrincipal();
20 |
21 |     return Jwts.builder()
22 |         .setSubject((userPrincipal.getUsername()))
23 |         .setIssuedAt(new Date())
24 |         .setExpiration(new Date((new Date()).getTime() + jwtExpirationMs))
25 |         .signWith(SignatureAlgorithm.HS512, jwtSecret)
26 |         .compact();
27 | }
```

Called with the `Authentication` object produced by a successful login.

- Line 19: `authentication.getPrincipal()` is the logged-in user object; it was created by `UserPrincipal.build`, so we cast it back to `UserPrincipal`.
- Line 21: `Jwts.builder()` starts building a token.
- Line 22: `setSubject(...)`: the **subject** (sub claim) is `userPrincipal.getUsername()`. In `UserPrincipal` that method returns the **e-mail**, so the token says "this token belongs to ravi@example.com".
- Line 23: `setIssuedAt(new Date())`: the **iat** claim, the moment of creation.
- Line 24: `setExpiration(...)`: the **exp** claim, now plus 86 400 000 ms (24 hours).
- Line 25: `signWith(SignatureAlgorithm.HS512, jwtSecret)` signs with the **HMAC-SHA512** algorithm and the secret. (This old library version treats the secret text as Base64 input; the text is not a strong 64-byte key, another reason not to use it in production.)
- Line 26: `compact()` produces the final single string `xxxxx.yyyyy.zzzzz`.

Note that the role is **not** inside the token. Only the e-mail is; the role is loaded from the database on each request.

Lines 29-31 | getUsernameFromJwtToken

```
29 | public String getUsernameFromJwtToken(String token) {
30 |     return Jwts.parser().setSigningKey(jwtSecret).parseClaimsJws(token).getBody().getSubject();
31 | }
```

`Jwts.parser().setSigningKey(jwtSecret)` creates a parser that knows the secret, `parseClaimsJws(token)` checks the signature and expiry (throwing an exception if anything is wrong), `getBody()` returns the payload claims and `getSubject()` returns the **sub** value: the e-mail. The filter uses this e-mail to load the user from the database.

Lines 33-46 | validateJwtToken and the silent catches

```
33 | public boolean validateJwtToken(String authToken) {
34 |     try {
35 |         Jwts.parser().setSigningKey(jwtSecret).parseClaimsJws(authToken);
36 |         return true;
37 |     } catch (SignatureException e) {
38 |     } catch (MalformedJwtException e) {
39 |     } catch (ExpiredJwtException e) {
40 |     } catch (UnsupportedJwtException e) {
41 |     } catch (IllegalArgumentException e) {
42 |     }
43 |
44 |     return false;
45 | }
46 | }
```

Tries to parse the token with the same parser. If `parseClaimsJws` finishes without an exception, the signature matches the secret and the token is not expired, so the method returns `true` (line 36).

Otherwise one of five exceptions is thrown, and each `catch` block is **empty** (they do nothing and swallow the exception):

- `SignatureException`: the signature does not match: the token was changed or signed with another secret.
- `MalformedJwtException`: the text is not a valid token at all.
- `ExpiredJwtException`: the `exp` time has passed (older than 24 hours).
- `UnsupportedJwtException`: the token has a format this parser does not accept (for example an unsigned token).
- `IllegalArgumentException`: the token is null, empty or only spaces.

All of them fall through to `return false` (line 44). Hiding the reason is simple and safe for the caller (it just sees "not valid"), but it makes debugging harder, because no log line tells you whether a token expired or was forged. Line 46 closes the class.

Workflow, next: Tokens can be made and checked. The class that uses these methods on every request is next: `JwtAuthenticationFilter.java`.

Step 17: JwtAuthenticationFilter - reads the token on every request [Vanshika - Security]

BACKEND

`springapp/src/main/java/com/examly/springapp/config/JwtAuthenticationFilter.java`

What this file is	A custom filter that sits in the Spring Security chain. It runs at the start of every request, looks for Authorization: Bearer <jwt> , and if the token is genuine it tells Spring Security "this request comes from user X with these roles". It never rejects a request itself: it only <i>*identifies*</i> the caller. The decision to allow or deny is made later by the URL rules in <code>SecurityConfig</code> . It extends <code>OncePerRequestFilter</code> , a Spring base class that guarantees <code>doFilterInternal</code> runs only once per request, even if the filter is registered twice.
Who / what triggers it	Every HTTP request to the backend (placed in the chain by <code>SecurityConfig.filterChain</code> before <code>UsernamePasswordAuthenticationFilter</code>). Angular sends the header from each service, so this filter fires for each API call.
What it uses / calls	<code>JwtUtils</code> (validate the token and read the e-mail), <code>MyUserDetailsService</code> (load the user and authorities), <code>SecurityContextHolder</code> (where Spring Security keeps the current user) and <code>RequestAttributeSecurityContextRepository</code> (stores the context for this request).
Size	64 lines

Lines 1-17 | Imports

```
1 | package com.examly.springapp.config;
2 |
3 | import jakarta.servlet.FilterChain;
4 | import jakarta.servlet.ServletException;
5 | import jakarta.servlet.http.HttpServletRequest;
6 | import jakarta.servlet.http.HttpServletResponse;
7 | import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
8 | import org.springframework.security.core.context.SecurityContext;
9 | import org.springframework.security.core.context.SecurityContextHolder;
10 | import org.springframework.security.core.userdetails.UserDetails;
11 | import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
12 | import org.springframework.security.web.context.RequestAttributeSecurityContextRepository;
13 | import org.springframework.security.web.context.SecurityContextRepository;
14 | import org.springframework.util.StringUtils;
15 | import org.springframework.web.filter.OncePerRequestFilter;
16 |
17 | import java.io.IOException;
```

`FilterChain`, `ServletException`, `HttpServletRequest`, `HttpServletResponse` (the servlet types for a request, a response and the rest of the chain), `UsernamePasswordAuthenticationToken` (an Authentication object holding user and roles), `SecurityContext`, `SecurityContextHolder`, `SecurityContextRepository`, `RequestAttributeSecurityContextRepository` (the storage of the current login), `UserDetails` (the user interface), `WebAuthenticationDetailsSource` (adds request details such as the IP address), `StringUtils` (null-safe text checks), `OncePerRequestFilter` (the base class) and `IOException`.

Lines 19-25 | Class and fields

```
19 | public class JwtAuthenticationFilter extends OncePerRequestFilter {
20 |
21 |     private final JwtUtils jwtUtils;
22 |     private final MyUserDetailsService userDetailsService;
23 |     // Spring Security 6: must persist SecurityContext through the SecurityContextHolderFilter's deferred loader
24 |     private final SecurityContextRepository securityContextRepository =
25 |         new RequestAttributeSecurityContextRepository();
```

Two final fields hold the helpers passed through the constructor. The third field is a `SecurityContextRepository`. The comment explains why: since Spring Security 6 the framework no longer saves the login for you at the end of a filter, it reads it from this repository through a deferred loader, so the filter must save the context itself or the authentication would be lost before the authorization step.

Lines 27-30 | Constructor

```
27 |     public JwtAuthenticationFilter(JwtUtils jwtUtils, MyUserDetailsService userDetailsService) {
28 |         this.jwtUtils = jwtUtils;
29 |         this.userDetailsService = userDetailsService;
30 |     }
```

Receives `JwtUtils` and `MyUserDetailsService` from `SecurityConfig.authenticationJwtTokenFilter()` and keeps them in the fields.

Lines 32-37 | doFilterInternal: start and token check

```
32 |     @Override
33 |     protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
34 |         throws ServletException, IOException {
35 |         try {
36 |             String jwt = parseJwt(request);
37 |             if (jwt != null && jwtUtils.validateJwtToken(jwt)) {
```

This method runs for every request. `request` is the incoming call, `response` the outgoing answer and `filterChain` the rest of the chain.

The whole body is inside `try` so that no problem in token handling can crash the request. Line 36 asks `parseJwt` (below) for the token; it returns `null` if there is no usable header. Line 37 continues only if a token exists **and** `validateJwtToken` says it is genuine and not expired. If either is false, nothing inside the `if` is executed.

Lines 38-39 | Who is it?

```
38 |         String username = jwtUtils.getUserNameFromJwtToken(jwt);
39 |         UserDetails userDetails = userDetailsService.loadUserByUsername(username);
```

`getUserNameFromJwtToken` reads the `sub` value of the token: the e-mail. `userDetailsService.loadUserByUsername(username)` then reads the user from the database (via `UserRepo.findByEmail`) and builds a `UserPrincipal`, which contains the authority `ROLE_ADMIN` or `ROLE_CUSTOMER`. So the role always comes from the database, not from the token.

Lines 41-43 | Build the authentication object

```
41 |         UsernamePasswordAuthenticationToken authentication =
42 |             new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());
43 |         authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
```

`new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities())` creates the object that stands for "this logged-in user". The three arguments are: the principal (the user), the credentials (`null`, the password is not needed again) and the authorities (the roles). Because authorities are supplied, the object counts as **authenticated**. Line 43 attaches details of the request (remote address, session id) for auditing.

Lines 45-50 | Put it where Spring Security looks

```
45 |         SecurityContext context = SecurityContextHolder.createEmptyContext();
46 |         context.setAuthentication(authentication);
47 |         SecurityContextHolder.setContext(context);
48 |         // CRITICAL: save to repository so SecurityContextHolderFilter's deferred loader picks it up
49 |         securityContextRepository.saveContext(context, request, response);
50 |     }
```

Line 45 creates a fresh empty `SecurityContext` (the box that holds the current login), line 46 puts the authentication into it and line 47 makes it the current context of this thread with `SecurityContextHolder.setContext`. Line 49 saves it into the request through the repository, which the comment marks as critical: without that, Spring Security 6 would ignore the context and treat the request as anonymous. Line 50 closes the `if`.

From this moment, the later `AuthorizationFilter` sees an authenticated user with a role, and rules such as `hasRole("ADMIN")` can be evaluated.

Lines 51-55 | Error handling and continuing the chain

```
51 |         } catch (Exception e) {  
52 |             logger.error("Cannot set user authentication: " + e.getMessage());  
53 |         }  
54 |         filterChain.doFilter(request, response);  
55 |     }
```

If anything throws (for instance the e-mail in a valid token no longer exists in the database, which happens after a restart because H2 is in memory and starts empty), the `catch` writes one error line to the log and **does not** authenticate the request. Line 54 `filterChain.doFilter(request, response)` then always hands the request to the next filter, whether authentication succeeded or not.

So the three cases are:

- **Valid token:** the request continues as the logged-in user. Rules decide whether the role is enough: allowed, or 403.
- **No header or invalid/expired token:** the request continues as **anonymous**. A `permitAll` URL still works; any protected URL is refused with 401 by the entry point.
- **Valid token but user gone:** logged, then anonymous, so a protected URL gives 401. Angular's interceptor reacts to 401 by sending the user back to the login page.

Lines 57-64 | parseJwt

```
57 |     private String parseJwt(HttpServletRequest request) {  
58 |         String headerAuth = request.getHeader("Authorization");  
59 |         if (StringUtils.hasText(headerAuth) && headerAuth.startsWith("Bearer ")) {  
60 |             return headerAuth.substring(7);  
61 |         }  
62 |         return null;  
63 |     }  
64 | }
```

Reads the `Authorization` header. If it has text (`StringUtils.hasText`) and starts with `Bearer` (with a space), `substring(7)` cuts off those seven characters, leaving only the token. Otherwise (header missing, or another scheme such as `Basic`) it returns `null`, which means "no token". Line 64 closes the class.

Workflow, next: The filter calls `loadUserByUsername` on the user details service. That class is next: `MyUserDetailsService.java`.

Step 18: MyUserDetailsService - finds a user for Spring Security [Vanshika - Security]

BACKEND

springapp/src/main/java/com/examly/springapp/config/MyUserDetailsService.java

What this file is	Spring Security does not know our <code>UserRepo</code> or <code>User</code> class. It only knows an interface called <code>UserDetailsService</code> with a single method, <code>loadUserByUsername</code> . This class is the adapter: it implements that interface by looking in our database and wrapping the result in a <code>UserPrinciple</code> .
Who / what triggers it	Two situations: (1) at login, <code>DaoAuthenticationProvider</code> calls it to fetch the stored user and password hash; (2) on every request with a valid token, <code>JwtAuthenticationFilter</code> calls it to rebuild the user and the roles.
What it uses / calls	<code>UserRepo.findByEmail</code> , <code>UserPrinciple.build</code> .
Size	26 lines

Lines 1-10 | Imports

```
1 | package com.examly.springapp.config;  
2 |  
3 | import com.examly.springapp.model.User;  
4 | import com.examly.springapp.repository.UserRepo;  
5 | import org.springframework.beans.factory.annotation.Autowired;  
6 | import org.springframework.security.core.userdetails.UserDetails;  
7 | import org.springframework.security.core.userdetails.UserDetailsService;  
8 | import org.springframework.security.core.userdetails.UsernameNotFoundException;  
9 | import org.springframework.stereotype.Service;  
10 | import org.springframework.transaction.annotation.Transactional;
```

`User` and `UserRepo` (our data), `Autowired` (injection), `UserDetails`, `UserDetailsService`, `UsernameNotFoundException` (Spring Security types), `Service` (marks a business-logic bean) and `Transactional` (run in a database transaction).

Lines 12-16 | Class and repository

```
12 | @Service
13 | public class MyUserDetailsService implements UserDetailsService {
14 |
15 |     @Autowired
16 |     private UserRepo userRepo;
```

`@Service` makes it a bean. `implements UserDetailsService` is the contract Spring Security needs. `@Autowired` injects the `UserRepo` generated by Spring Data.

Lines 18-26 | loadUserByUsername

```
18 |     @Override
19 |     @Transactional
20 |     public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
21 |         User user = userRepo.findByEmail(email)
22 |             .orElseThrow(() -> new UsernameNotFoundException("User Not Found with email: " + email));
23 |
24 |         return UserPrincipal.build(user);
25 |     }
26 | }
```

The parameter is called `email` because in DriveU the "user name" **is the e-mail**.

`@Override` confirms we are implementing the interface method. `@Transactional` runs the method in one database transaction.

Line 21 calls `userRepo.findByEmail(email)`, which returns an `Optional<User>`. `.orElseThrow(...)` returns the user if present, otherwise throws `UsernameNotFoundException("User Not Found with email: ...")`. During login that exception makes `AuthController` answer 401 "Invalid credentials"; inside the filter it is caught and logged.

Line 24 converts our `User` to a `UserPrincipal` (a `UserDetails`) with `UserPrincipal.build(user)` and returns it.

Workflow, next: The returned object type is our own class: `UserPrincipal.java`.

Step 19: UserPrincipal - our User as Spring Security sees it [Vanshika - Security]

BACKEND

springapp/src/main/java/com/examly/springapp/config/UserPrincipal.java

What this file is	Spring Security describes a logged-in person with the interface <code>UserDetails</code> (name, password, roles, account flags). <code>UserPrincipal</code> implements it and wraps our <code>User</code> entity, so the security system gets what it needs while our controllers can still reach the original <code>User</code> through <code>getUser()</code> .
Who / what triggers it	Created by <code>MyUserDetailsService</code> through <code>UserPrincipal.build(user)</code> at login and on every authenticated request. <code>AuthController</code> reads <code>getUser()</code> from it to fill the <code>LoginDTO</code> , and <code>JwtUtils</code> reads <code>getUsername()</code> for the token subject.
What it uses / calls	<code>User</code> entity, <code>GrantedAuthority</code> , <code>SimpleGrantedAuthority</code> , <code>UserDetails</code> .
Size	80 lines

Lines 1-9 | Imports

```
1 | package com.examly.springapp.config;
2 |
3 | import com.examly.springapp.model.User;
4 | import org.springframework.security.core.GrantedAuthority;
5 | import org.springframework.security.core.authority.SimpleGrantedAuthority;
6 | import org.springframework.security.core.userdetails.UserDetails;
7 |
8 | import java.util.Collection;
9 | import java.util.Collections;
```

`User` (our entity), `GrantedAuthority` and `SimpleGrantedAuthority` (a role/permission), `UserDetails` (the interface), `Collection` and `Collections` (to hold the single authority).

Lines 11-17 | Fields

```
11 | public class UserPrinciple implements UserDetails {
12 |
13 |     private Long id;
14 |     private String email;
15 |     private String password;
16 |     private Collection<? extends GrantedAuthority> authorities;
17 |     private User user;
```

`implements UserDetails` makes the class acceptable to Spring Security. The fields hold the id, the e-mail, the password hash, the list of authorities (roles) and the original `User` object.

Lines 19-25 | Constructor

```
19 |     public UserPrinciple(Long id, String email, String password, Collection<? extends GrantedAuthority> authorities, U
20 |         ser user) {
21 |         this.id = id;
22 |         this.email = email;
23 |         this.password = password;
24 |         this.authorities = authorities;
25 |         this.user = user;
26 |     }
```

Stores the five values in the fields. It is called only from `build` below.

Lines 27-36 | `build`: from `User` to `UserPrinciple`, and the `ROLE_` rule

```
27 |     public static UserPrinciple build(User user) {
28 |         GrantedAuthority authority = new SimpleGrantedAuthority("ROLE_" + user.getUserRole().toUpperCase());
29 |         return new UserPrinciple(
30 |             user.getId(),
31 |             user.getEmail(),
32 |             user.getPassword(),
33 |             Collections.singletonList(authority),
34 |             user
35 |         );
36 |     }
```

A static factory used by `MyUserDetailsService`.

Line 28 is the key line: `"ROLE_" + user.getUserRole().toUpperCase()`. The database stores the role as text like `Admin` or `Customer`. Spring Security's `hasRole("ADMIN")` does not compare with the text `ADMIN`: it automatically adds the prefix and looks for the authority `ROLE_ADMIN`. So `Admin` is converted to upper case and given the `ROLE_` prefix: `ROLE_ADMIN` (or `ROLE_CUSTOMER`), which is what the rules in `SecurityConfig` match. Without the prefix and the upper-casing, `hasRole("ADMIN")` would never be true for a user whose role is `Admin`. (If `userRole` were null this line would throw a `NullPointerException`; the sign-up form always sends a role.)

Lines 29-35 create the object with the user's id, e-mail, password hash, a one-element list holding the authority (`Collections.singletonList`) and the `User` itself.

Lines 38-44 | `getId` and `getUser`

```
38 |     public Long getId() {
39 |         return id;
40 |     }
41 |
42 |     public User getUser() {
43 |         return user;
44 |     }
```

Our own extra getters (not from the interface). `getUser()` is used by `AuthController` after login to read the display name, role and id for the `LoginDTO`.

Lines 46-49 | `getAuthorities`

```
46 |     @Override
47 |     public Collection<? extends GrantedAuthority> getAuthorities() {
48 |         return authorities;
49 |     }
```

Required by `UserDetails`: returns the list of roles. This is what `hasRole(...)` and `hasAnyRole(...)` check.

Lines 51-59 | `getPassword` and `getUsername`

```
51 |     @Override
52 |     public String getPassword() {
53 |         return password;
54 |     }
55 |
56 |     @Override
57 |     public String getUsername() {
58 |         return email;
59 |     }
```

`getPassword()` returns the BCrypt hash; `DaoAuthenticationProvider` compares the typed password against it. `getUsername()` returns the **e-mail**, because the e-mail is DriveU's login name; this value becomes the **sub** of the JWT.

Lines 61-80 | Account status flags

```
61 | @Override
62 | public boolean isAccountNonExpired() {
63 |     return true;
64 | }
65 |
66 | @Override
67 | public boolean isAccountNonLocked() {
68 |     return true;
69 | }
70 |
71 | @Override
72 | public boolean isCredentialsNonExpired() {
73 |     return true;
74 | }
75 |
76 | @Override
77 | public boolean isEnabled() {
78 |     return true;
79 | }
80 | }
```

Four methods required by the interface: account not expired, not locked, credentials not expired and enabled. DriveU has no such features, so all four always return **true**, meaning "everything is fine".

Workflow, next: Authentication data is complete. Two small classes remain that decide what the browser sees when security says no, starting with the 401 handler: `JwtAuthenticationEntryPoint.java`.

Step 20: JwtAuthenticationEntryPoint - the 401 answer

[Vanshika - Security]

BACKEND

`springapp/src/main/java/com/examly/springapp/config/JwtAuthenticationEntryPoint.java`

What this file is	The entry point is called when someone who is not authenticated requests a protected URL: no token, a bad token, or an expired one. It decides the answer: here an HTTP 401 Unauthorized with the message Error: Unauthorized .
Who / what triggers it	Spring Security's <code>ExceptionTranslationFilter</code> calls <code>commence</code> when a request without a valid login reaches a rule that needs authentication (for example <code>GET /api/driverRequest</code> with no token). It was registered in <code>SecurityConfig</code> line 64.
What it uses / calls	<code>HttpServletResponse.sendError</code> .
Size	20 lines

Lines 1-10 | Imports

```
1 | package com.examly.springapp.config;
2 |
3 | import jakarta.servlet.ServletException;
4 | import jakarta.servlet.http.HttpServletRequest;
5 | import jakarta.servlet.http.HttpServletResponse;
6 | import org.springframework.security.core.AuthenticationException;
7 | import org.springframework.security.web.AuthenticationEntryPoint;
8 | import org.springframework.stereotype.Component;
9 |
10 | import java.io.IOException;
```

The servlet request/response types, `AuthenticationException` (the reason for the failure), `AuthenticationEntryPoint` (the interface), `Component` (to make it a bean for injection) and `IOException`.

Lines 12-13 | Class

```
12 | @Component
13 | public class JwtAuthenticationEntryPoint implements AuthenticationEntryPoint {
```

`@Component` makes Spring create it so `SecurityConfig` can inject it. `implements AuthenticationEntryPoint` is the contract.

Lines 15-20 | commence

```
15 | @Override
16 | public void commence(HttpServletRequest request, HttpServletResponse response,
17 |                     AuthenticationException authException) throws IOException, ServletException {
18 |     response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Error: Unauthorized");
19 | }
20 | }
```

`commence` is the interface's only method. `response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Error: Unauthorized")` sets the status 401 and a short message. No login page redirect is used because this is an API, not a website. In the browser, Angular's `HttpErrorInterceptor` sees status 401 and, if a token was stored, sends the user back to `/login`.

Workflow, next: Its partner for logged-in users with the wrong role is `JwtAccessDeniedHandler.java`.

Step 21: JwtAccessDeniedHandler - the 403 answer

[Vanshika - Security]

springapp/src/main/java/com/examly/springapp/config/JwtAccessDeniedHandler.java

BACKEND

What this file is	Called when a user is authenticated but their role does not allow the URL, for example a Customer trying <code>GET /api/driverRequest</code> (admin only). It answers with HTTP 403 Forbidden and the message <code>Error: Forbidden</code> .
Who / what triggers it	Spring Security's <code>ExceptionHandlerFilter</code> calls <code>handle</code> when the <code>AuthorizationFilter</code> refuses an already-logged-in user. Registered in <code>SecurityConfig</code> line 65.
What it uses / calls	<code>HttpServletResponse.sendError</code> .
Size	20 lines

Lines 1-10 | Imports

```
1 | package com.examly.springapp.config;
2 |
3 | import jakarta.servlet.ServletException;
4 | import jakarta.servlet.http.HttpServletRequest;
5 | import jakarta.servlet.http.HttpServletResponse;
6 | import org.springframework.security.access.AccessDeniedException;
7 | import org.springframework.security.web.access.AccessDeniedHandler;
8 | import org.springframework.stereotype.Component;
9 |
10 | import java.io.IOException;
```

Servlet types, `AccessDeniedException` (the reason), `AccessDeniedHandler` (the interface), `Component` and `IOException`.

Lines 12-13 | Class

```
12 | @Component
13 | public class JwtAccessDeniedHandler implements AccessDeniedHandler {
```

A `@Component` bean implementing `AccessDeniedHandler`, injected into `SecurityConfig`.

Lines 15-20 | handle

```
15 | @Override
16 | public void handle(HttpServletRequest request, HttpServletResponse response,
17 |                 AccessDeniedException accessDeniedException) throws IOException, ServletException {
18 |     response.sendError(HttpServletResponse.SC_FORBIDDEN, "Error: Forbidden");
19 | }
20 | }
```

`response.sendError(HttpServletResponse.SC_FORBIDDEN, "Error: Forbidden")` returns status 403. The difference to the previous class: 401 means "we do not know who you are, log in", 403 means "we know who you are, but your role may not do this". Logging in again would not help with a 403.

Workflow, next: The last file of the security layer deals with the browser rather than the user: `CrosConfig.java`.

Step 22: CorsConfig - allowing the Angular app to call the API (CORS) [Vanshika - Security]

BACKEND

springapp/src/main/java/com/examly/springapp/config/CorsConfig.java

What this file is	A browser treats two addresses as the same origin only if scheme, host and port are identical. The Angular app is served from <code>http://localhost:8081</code> and the API lives at <code>http://localhost:8080</code>: different ports, so different origins. By the browser's same-origin policy, JavaScript on 8081 may not read answers from 8080 unless the server says it is allowed. That permission mechanism is CORS (Cross-Origin Resource Sharing): the server adds headers such as <code>Access-Control-Allow-Origin</code> to its answers. The class name is a typo for "Cors"; it works the same. The pre-flight request: before sending a request that is "not simple" (a <code>PUT</code> or <code>DELETE</code>, a JSON body, or a custom header such as <code>Authorization</code>, which this app uses on every call), the browser first sends an <code>OPTIONS</code> request to the same URL, asking "may I send this method with these headers from this origin?". Only if the answer carries the right <code>Access-Control-Allow-...</code> headers does the browser send the real request. This pre-flight carries no token, so it must be answered before the security rules refuse it.
Who / what triggers it	Spring MVC reads it during start-up (it is a <code>@Configuration</code> implementing <code>WebMvcConfigurer</code>). <code>SecurityConfig</code> turns CORS support on with <code>.cors(...)</code>, so the CORS filter in the security chain uses these mappings and answers pre-flight requests before authorization.
What it uses / calls	Spring MVC's <code>CorsRegistry</code>.
Size	18 lines

Lines 1-5 | Package and imports

```
1 | package com.examly.springapp.config;
2 |
3 | import org.springframework.context.annotation.Configuration;
4 | import org.springframework.web.servlet.config.annotation.CorsRegistry;
5 | import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
```

`Configuration` (bean definitions), `CorsRegistry` (where CORS rules are registered) and `WebMvcConfigurer` (the interface for adjusting Spring MVC).

Lines 7-8 | Class

```
7 | @Configuration
8 | public class CorsConfig implements WebMvcConfigurer {
```

`@Configuration` makes Spring read the class; `implements WebMvcConfigurer` allows overriding `addCorsMappings`.

Lines 10-12 | Apply to all URLs

```
10 |     @Override
11 |     public void addCorsMappings(CorsRegistry registry) {
12 |         registry.addMapping("/**")
```

`addCorsMappings` is called once at start-up. `registry.addMapping(...)` with the pattern slash plus two stars (the same wildcard as in `SecurityConfig`) says that the following rules apply to **every** URL of the API.

Lines 13-14 | Origins and methods

```
13 |             .allowedOriginPatterns("*")
14 |             .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
```

`allowedOriginPatterns("*")` allows requests from any origin, so `http://localhost:8081` (Angular) is allowed, and so is every other site. (`allowedOrigins("*")` is not allowed together with credentials, which is why the `Patterns` variant is used.) `allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")` lists the HTTP methods the browser may use; `OPTIONS` is the pre-flight method.

Lines 15-16 | Headers and credentials

```
15 |             .allowedHeaders("*")
16 |             .allowCredentials(true);
```

`allowedHeaders("*")` accepts any request header, including `Authorization` and `Content-Type`. `allowCredentials(true)` lets the browser send credentials such as cookies with cross-origin calls. The app's JWT goes in a header, so it would work even without this, but it is harmless. By Spring's default the browser may cache the pre-flight answer for 30 minutes, so it does not ask before every call. Being this open is convenient for

development; a production system would allow only the real front-end address.

Lines 17-18 | End of the method and class

```
17 |     }  
18 | }
```

Closing braces of the method and the class.

Workflow, next: The **security layer is now complete**: the chain accepts or refuses every request (CORS answers pre-flight, `JwtAuthenticationFilter` identifies the caller, `SecurityConfig` rules allow or deny, 401/403 handlers answer when needed). Requests that pass it will reach controllers, which hand the real work to the service layer. We continue with that layer, whose first file is `UserService.java`.

Step 23: UserService - the user business-rule contract

[Aleena - Authentication]

BACKEND

springapp/src/main/java/com/examly/springapp/service/UserService.java

What this file is	The first file of the service layer . The backend is built in three horizontal layers: the controller receives the HTTP request, the service applies the business rules, and the repository talks to the database. <code>UserService</code> is an interface : a list of method signatures with no bodies. It only says what the user service can do (create a user, log a user in). The real code is in <code>UserServiceImpl</code> , the next file. Why split a service into an interface and an <code>Impl</code> class? The controller only knows the interface type, so it does not care how the work is done. Spring finds the one class that implements the interface and hands it over. This keeps layers loosely coupled and makes the service easy to replace by a fake in a unit test. Every service in this project follows this pattern (<code>UserService/UserServiceImpl</code> , <code>DriverService/DriverServiceImpl</code> , and so on).
Who / what triggers it	Nothing runs in this file, because an interface has no code. It is used at compile time (the controller and the implementation both refer to it) and at startup, when Spring looks for a bean that implements it.
What it uses / calls	<code>User</code> (the entity from the model package, used as parameter and return type). It is implemented by <code>UserServiceImpl</code> and injected into <code>AuthController</code> .
Size	8 lines

Lines 1-3 | Package and import

```
1 | package com.examly.springapp.service;  
2 |  
3 | import com.examly.springapp.model.User;
```

`package com.examly.springapp.service;` puts the file in the service package, which sits next to `config`, `controller`, `model` and `repository` below the root package that `@SpringBootApplication` scans. The single import brings in the `User` entity so the method signatures can mention it.

Lines 5-6 | The interface and createUser

```
5 | public interface UserService {  
6 |     User createUser(User user);
```

`public interface UserService` declares a contract. `User createUser(User user);` means: give me a `User` object (filled from the registration form) and I hand back the saved `User`, or `null` if the e-mail is already registered. Because it is an interface, the line ends with `;` and has no `{ }` body. The `public` modifier on interface methods is implicit.

Lines 7-8 | loginUser and the closing brace

```
7 |         User loginUser(User user);  
8 | }
```

`User loginUser(User user);` is declared for looking a user up at login. Honest note: nothing in the project calls it. The real login is done by `AuthController` through Spring Security's `AuthenticationManager` (see the comment in `UserServiceImpl`), so this method is a leftover of the first design. Line 8 closes the interface.

Workflow, next: Spring now needs a class that implements this contract, so we continue with `UserServiceImpl.java`, where the registration rule (reject duplicate e-mail, encrypt the password, save) really lives.

Step 24: UserServiceImpl - registering a user [Aleena - Authentication]

BACKEND

springapp/src/main/java/com/examly/springapp/service/UserServiceImpl.java

What this file is	The implementation of <code>UserService</code> . It is a Spring bean (an object that Spring creates once at startup and keeps in its container, the application context). Its job is the business rule of registration: an e-mail address may only be used once, and the password must never be stored as plain text.
Who / what triggers it	Spring creates the single instance of this class while the application context is built (component scanning finds the <code>@Service</code> annotation). The method <code>createUser</code> runs every time the browser sends <code>POST /api/register</code> , via <code>AuthController.registerUser</code> .
What it uses / calls	<code>UserRepo</code> (Spring Data repository, <code>findByEmail</code> and <code>save</code>), <code>PasswordEncoder</code> (the <code>BCryptPasswordEncoder</code> bean defined in <code>SecurityConfig</code>), and the <code>User</code> entity.
Size	35 lines

Lines 1-9 | Package and imports

```
1 | package com.examly.springapp.service;
2 |
3 | import com.examly.springapp.model.User;
4 | import com.examly.springapp.repository.UserRepo;
5 | import org.springframework.beans.factory.annotation.Autowired;
6 | import org.springframework.security.crypto.password.PasswordEncoder;
7 | import org.springframework.stereotype.Service;
8 |
9 | import java.util.Optional;
```

- `User` and `UserRepo`: the entity and the repository used below.
- `Autowired`: the annotation that asks Spring to fill a field with a bean.
- `PasswordEncoder`: the Spring Security interface for hashing passwords. The actual object is the `BCryptPasswordEncoder` created by the `passwordEncoder()` method in `SecurityConfig`.
- `Service`: marks the class as a service bean.
- `Optional`: Java's container for a value that may be missing (explained at line 22).

Lines 11-12 | @Service and the class declaration

```
11 | @Service
12 | public class UserServiceImpl implements UserService {
```

`@Service` is a **stereotype annotation**: it tells component scanning "create one object of this class and register it as a bean". It behaves like `@Component` but also documents that this class belongs to the service layer. `implements UserService` means the class promises to provide both methods of the interface; the compiler checks that. Because `UserServiceImpl` is the only implementation, Spring will inject it wherever a `UserService` is requested (in `AuthController`).

Lines 14-18 | Dependencies injected with @Autowired

```
14 |     @Autowired
15 |     private UserRepo userRepo;
16 |
17 |     @Autowired
18 |     private PasswordEncoder encoder;
```

`@Autowired` on a field is **dependency injection**: the class does not write `new UserRepo()` or `new BCryptPasswordEncoder()`. When Spring creates the `UserServiceImpl` object it looks in the container for a bean of the declared type and assigns it to the field (through reflection, which is why the field is `private` and has no setter). The `UserRepo` bean is an object that Spring Data generated at startup from the interface; the `PasswordEncoder` bean comes from `SecurityConfig`. Field injection is the shortest style; constructor injection is the modern alternative, but this project uses fields everywhere.

Lines 20-28 | createUser - the registration rule

```
20 | @Override
21 | public User createUser(User user) {
22 |     Optional<User> existingUser = userRepo.findByEmail(user.getEmail());
23 |     if (existingUser.isPresent()) {
24 |         return null; // Signals duplicate email
25 |     }
26 |     user.setPassword(encoder.encode(user.getPassword()));
27 |     return userRepo.save(user);
28 | }
```

`@Override` tells the compiler this method implements the interface method (a typo in the name would then be an error).

What happens, in order:

- Line 22: `userRepo.findByEmail(user.getEmail())` runs `SELECT ... FROM users WHERE email = ?` (a **derived query**: Spring Data builds it from the method name). It returns an `Optional<User>`. An **Optional** is a box that is either empty or holds one user; it forces the caller to deal with the "not found" case instead of risking a `NullPointerException`.
- Lines 23-25: `existingUser.isPresent()` is true when someone already owns that e-mail. The method then returns `null`. This `null` is the signal for a duplicate; `AuthController` turns it into HTTP **409 Conflict**.
- Line 26: otherwise the plain password that the form sent is replaced by `encoder.encode(...)`. **BCrypt** is a one-way hash with a random salt, so the stored value looks like `$2a$10$...` (60 characters) and the same password gives a different hash each time. Nobody, not even the database administrator, can read the original. At login Spring Security calls `matches(raw, hash)` to compare.
- Line 27: `userRepo.save(user)` executes the `INSERT`. Because `userId` is `GenerationType.IDENTITY`, H2 assigns the id and the returned object contains it. That object goes back to the controller and is serialised to JSON (the `password` field is `WRITE_ONLY`, so the hash is not sent back).

Honest notes: the method does not check that `userRole` is one of the expected values, so whatever role text the client posts is saved (any capitalisation works because `UserPrincipal` upper-cases it) (the signup page offers a drop-down with Admin and Customer; the server trusts whatever arrives, so anyone calling the API directly could register as an admin). A user saved with no role would later crash in `UserPrincipal.build`, which calls `getUserRole().toUpperCase()`.

Lines 30-35 | loginUser - unused leftover

```
30 | @Override
31 | public User loginUser(User user) {
32 |     // Will be handled via Spring Security and JwtUtils in AuthController
33 |     return userRepo.findByEmail(user.getEmail()).orElse(null);
34 | }
35 | }
```

It just looks the user up by e-mail and returns it, or `null` (`Optional.orElse(null)`). The comment says login is handled in `AuthController` with Spring Security and `JwtUtils`, and indeed this method is never called. It stays because the interface declares it.

Workflow, next: A user can now be created. The next services throw their own custom errors when something goes wrong, so before we read `DriverServiceImpl` we first look at the exception classes it uses. Next file: `DuplicateDriverException.java`.

Step 25: DuplicateDriverException - license number already used [Vaishalini - Customer Journey & Shared Infra]

BACKEND

springapp/src/main/java/com/examly/springapp/exceptions/DuplicateDriverException.java

What this file is	A tiny custom exception . An exception is Java's way of stopping normal execution when something is wrong and handing a description of the problem to whoever can deal with it. This class gives a business problem its own name: "a driver with this license number already exists". Naming it lets the code (and <code>GlobalExceptionHandler</code>) react to this exact situation instead of to a generic error.
Who / what triggers it	Created and thrown (<code>throw new DuplicateDriverException(...)</code>) by <code>DriverServiceImpl.addDriver</code> when the license number is already in the database.
What it uses / calls	<code>RuntimeException</code> from the Java standard library.

Size	7 lines
------	---------

Line 1 | Package

```
1 | package com.examly.springapp.exceptions;
```

The `exceptions` package holds the three custom exception classes and the global handler. It is below `com.examly.springapp`, so it is component-scanned (needed for `GlobalExceptionHandler`).

Line 3 | Class declaration - extends RuntimeException

```
3 | public class DuplicateDriverException extends RuntimeException {
```

`extends RuntimeException` makes this an **unchecked exception**: Java does not force every method in the call chain to write `throws` or `try/catch`. That is why `addDriver` can throw it without declaring it in the `DriverService` interface. (A checked exception, such as `IOException`, must be declared or caught.)

Lines 4-7 | Constructor passing the message to the parent

```
4 |     public DuplicateDriverException(String message) {
5 |         super(message);
6 |     }
7 | }
```

The constructor accepts a text message and calls `super(message)`, which stores it in the parent `RuntimeException`. Later `ex.getMessage()` returns it, for example "Driver with license number KA01 already exists." No other code is needed: the class only exists to be a distinct type.

Workflow, next: Next is another custom exception of the same shape, used when a driver cannot be deleted: `DriverDeletionException.java`.

Step 26: DriverDeletionException - driver could not be deleted [Vaishalini - Customer Journey & Shared Infra]

BACKEND

springapp/src/main/java/com/examly/springapp/exceptions/DriverDeletionException.java

What this file is	A second custom unchecked exception with exactly the same structure as <code>DuplicateDriverException</code> . It means: "the database refused to delete this driver". The usual reason is that the driver is referenced by existing <code>DriverRequest</code> or <code>Feedback</code> rows (a foreign key), so the database blocks the <code>DELETE</code> .
Who / what triggers it	Thrown by <code>DriverServiceImpl.deleteDriver</code> inside its <code>catch</code> block. Handled later by <code>GlobalExceptionHandler.handleConflict</code> , which answers with HTTP 409.
What it uses / calls	<code>RuntimeException</code> .
Size	7 lines

Line 1 | Package

```
1 | package com.examly.springapp.exceptions;
```

Same `exceptions` package as the other exception classes.

Line 3 | Class declaration

```
3 | public class DriverDeletionException extends RuntimeException {
```

Extends `RuntimeException`, so it is unchecked (see `DuplicateDriverException` for the explanation of checked vs unchecked).

Lines 4-7 | Constructor with message

```
4 |     public DriverDeletionException(String message) {
5 |         super(message);
6 |     }
7 | }
```

Passes the message (for example "Failed to delete driver with ID: 3") up to `RuntimeException` through `super(message)`. The text will later be shown to the user in the JSON `message` field.

Workflow, next: With both driver exceptions known, we can read the driver service contract:
`DriverService.java`.

Step 27: DriverService - driver business-rule contract

[Arjun - Driver & Booking]

BACKEND

`springapp/src/main/java/com/examly/springapp/service/DriverService.java`

What this file is	The interface of the driver service: five operations for the admin's driver management (add, read one, read all, update, delete). As with <code>UserService</code> , it only lists the signatures; <code>DriverServiceImpl</code> supplies the code and <code>DriverController</code> is the caller.
Who / what triggers it	Used at compile time by <code>DriverController</code> and <code>DriverServiceImpl</code> ; at startup Spring injects the implementation into the controller.
What it uses / calls	The <code>Driver</code> entity, <code>java.util.List</code> and <code>java.util.Optional</code> .
Size	13 lines

Lines 1-5 | Package and imports

```
1 | package com.examly.springapp.service;
2 |
3 | import com.examly.springapp.model.Driver;
4 | import java.util.List;
5 | import java.util.Optional;
```

`Driver` is the entity. `List` is Java's ordered collection (used for "all drivers"). `Optional` is the "maybe a value" box that `getDriverById` returns.

Lines 7-10 | Create and read methods

```
7 | public interface DriverService {
8 |     Driver addDriver(Driver driver);
9 |     Optional<Driver> getDriverById(Long driverId);
10 |    List<Driver> getAllDrivers();
```

- `Driver addDriver(Driver driver)`: saves a new driver and returns the stored one (with its generated id). Throws `DuplicateDriverException` if the license number exists.
- `Optional<Driver> getDriverById(Long driverId)`: the return type is an **Optional** because the id may not exist. The controller checks `isPresent()` and answers 404 if empty.
- `List<Driver> getAllDrivers()`: every driver; an empty list is possible.

Lines 11-13 | Update and delete methods

```
11 |     Driver updateDriver(Long driverId, Driver driver);
12 |     Driver deleteDriver(Long driverId);
13 | }
```

- `Driver updateDriver(Long driverId, Driver driver)`: overwrite the stored driver that has this id with the data of the second argument; returns `null` if the id is unknown.
- `Driver deleteDriver(Long driverId)`: removes the driver and returns the removed object, or `null` if the id is unknown. May throw `DriverDeletionException`.

Note the convention: "not found" is expressed as `null` (or an empty `Optional`) and the controller turns it into a status code.

Workflow, next: Now the class that does the work: `DriverServiceImpl.java`.

Step 28: DriverServiceImpl - add, read, update, delete drivers [Arjun - Driver & Booking]

BACKEND

springapp/src/main/java/com/examly/springapp/service/DriverServiceImpl.java

What this file is	The implementation of <code>DriverService</code> . It holds the rules for managing drivers: license numbers must be unique, an update copies every editable field, and a delete that fails in the database is reported with a meaningful exception. It is a <code>@Service</code> bean.
Who / what triggers it	Called from <code>DriverController</code> for <code>POST /api/driver</code> , <code>GET /api/driver</code> , <code>GET /api/driver/{id}</code> , <code>PUT /api/driver/{id}</code> and <code>DELETE /api/driver/{id}</code> . Spring creates the single instance at startup.
What it uses / calls	<code>DriverRepo</code> (<code>findByLicenseNumber</code> , <code>findById</code> , <code>findAll</code> , <code>save</code> , <code>delete</code>), the <code>Driver</code> entity and the two exceptions <code>DuplicateDriverException</code> and <code>DriverDeletionException</code> .
Size	70 lines

Lines 1-11 | Package and imports

```
1 | package com.examly.springapp.service;
2 |
3 | import com.examly.springapp.exceptions.DriverDeletionException;
4 | import com.examly.springapp.exceptions.DuplicateDriverException;
5 | import com.examly.springapp.model.Driver;
6 | import com.examly.springapp.repository.DriverRepo;
7 | import org.springframework.beans.factory.annotation.Autowired;
8 | import org.springframework.stereotype.Service;
9 |
10 | import java.util.List;
11 | import java.util.Optional;
```

The two exception classes are imported because this service throws them. `DriverRepo` is the data access object, `Autowired` and `Service` are the Spring annotations explained in `UserServiceImpl`, `List` and `Optional` are used in the method signatures.

Lines 13-17 | Class, @Service and the injected repository

```
13 | @Service
14 | public class DriverServiceImpl implements DriverService {
15 |
16 |     @Autowired
17 |     private DriverRepo driverRepo;
```

`@Service` registers the class as a bean and `implements DriverService` binds it to the interface. `@Autowired private DriverRepo driverRepo;` is injected by Spring: the `DriverRepo` object was generated at startup from the `JpaRepository<Driver, Long>` interface, so `findAll`, `findById`, `save` and `delete` are available without any SQL written by hand.

Lines 19-26 | addDriver - unique license number rule

```
19 |     @Override
20 |     public Driver addDriver(Driver driver) {
21 |         Optional<Driver> existingDriver = driverRepo.findByLicenseNumber(driver.getLicenseNumber());
22 |         if (existingDriver.isPresent()) {
23 |             throw new DuplicateDriverException("Driver with license number " + driver.getLicenseNumber() + " already e
24 | xists.");
25 |         }
26 |         return driverRepo.save(driver);
27 |     }
```

- Line 21: `findByLicenseNumber` (derived query on the `licenseNumber` column) returns an `Optional<Driver>`.
- Lines 22-24: if one is already there, the method **throws** `DuplicateDriverException` with a message that contains the license number. Execution leaves the method at once; nothing is saved.
- Line 25: otherwise `save(driver)` inserts the row and returns the managed entity with the new `driverId`.

Who sees the exception? It travels up to `DriverController.addDriver`, which has a `try/catch (Exception)` and converts it into HTTP 409 (see that file).

Lines 28-31 | getDriverById

```
28 |     @Override
29 |     public Optional<Driver> getDriverById(Long driverId) {
30 |         return driverRepo.findById(driverId);
31 |     }
```

A one-line delegation: `findById` already returns an `Optional<Driver>`, so the service simply passes it on. The controller decides between 200 and 404.

Lines 33-36 | getAllDrivers

```
33 | @Override
34 | public List<Driver> getAllDrivers() {
35 |     return driverRepo.findAll();
36 | }
```

`findAll()` selects every row of the driver table and returns a `List<Driver>` (empty if the table is empty). This is the method behind `GET /api/driver`, which `SecurityConfig` makes public (`permitAll`). Note that every `Driver` carries its `image` text (a base64 picture), so this list can be large.

Lines 38-55 | updateDriver - copy every field

```
38 | @Override
39 | public Driver updateDriver(Long driverId, Driver driver) {
40 |     Optional<Driver> existingDriverOpt = driverRepo.findById(driverId);
41 |     if (existingDriverOpt.isPresent()) {
42 |         Driver existingDriver = existingDriverOpt.get();
43 |         existingDriver.setDriverName(driver.getDriverName());
44 |         existingDriver.setLicenseNumber(driver.getLicenseNumber());
45 |         existingDriver.setExperienceYears(driver.getExperienceYears());
46 |         existingDriver.setContactNumber(driver.getContactNumber());
47 |         existingDriver.setAvailabilityStatus(driver.getAvailabilityStatus());
48 |         existingDriver.setAddress(driver.getAddress());
49 |         existingDriver.setVehicleType(driver.getVehicleType());
50 |         existingDriver.setHourlyRate(driver.getHourlyRate());
51 |         existingDriver.setImage(driver.getImage());
52 |         return driverRepo.save(existingDriver);
53 |     }
54 |     return null;
55 | }
```

- Line 40: find the stored driver. If it does not exist the `if` is skipped and line 54 returns `null` (the controller answers 404).
- Line 42: `existingDriverOpt.get()` takes the entity out of the `Optional` (safe here because `isPresent()` was just checked).
- Lines 43-51: nine setters copy **all** editable fields from the incoming object into the stored one: name, license number, experience, contact, availability, address, vehicle type, hourly rate and image. The id is not copied, it stays the same.
- Line 52: `save(existingDriver)` runs an **UPDATE** (the entity already has an id, so JPA updates instead of inserting) and returns it.

Honest notes: unlike `updateDriverRequest`, this method does not skip null fields, so if the client sends a body without `image`, the stored picture is erased. It also does not re-check that the new license number is unique.

Lines 57-70 | deleteDriver - catch and translate database errors

```
57 | @Override
58 | public Driver deleteDriver(Long driverId) {
59 |     Optional<Driver> existingDriver = driverRepo.findById(driverId);
60 |     if (existingDriver.isPresent()) {
61 |         try {
62 |             driverRepo.delete(existingDriver.get());
63 |             return existingDriver.get();
64 |         } catch (Exception e) {
65 |             throw new DriverDeletionException("Failed to delete driver with ID: " + driverId);
66 |         }
67 |     }
68 |     return null;
69 | }
70 | }
```

- Line 59: find the driver; if missing, line 68 returns `null` (404).
- Line 62: `driverRepo.delete(...)` issues `DELETE FROM driver WHERE driver_id = ?`. The repository method has its own transaction, so the database checks its **foreign key** constraints immediately. If some `DriverRequest` or `Feedback` row still points to this driver, H2 refuses and a persistence exception (for example `DataIntegrityViolationException`) is thrown.
- Lines 64-66: `catch (Exception e)` catches that and throws the custom `DriverDeletionException("Failed to delete driver with ID: ...")` instead, so the user sees a readable message. This one is **not** caught by the controller, so it reaches `GlobalExceptionHandler`, which answers 409 with that message.
- Line 63: on success the deleted object is returned (the controller sends it back with 200).

Workflow, next: The driver request service also has a delete-failure exception, so we read that small class first: `DriverRequestDeletionException.java`.

Step 29: DriverRequestDeletionException - request could not be deleted [Vaishalini - Customer Journey & Shared Infra]

BACKEND

`springapp/src/main/java/com/examly/springapp/exceptions/DriverRequestDeletionException.java`

What this file is	The third custom unchecked exception. It is used when deleting a driver request fails in the database. It has the same shape as <code>DriverDeletionException</code> and <code>DuplicateDriverException</code> : the class body only forwards a message to <code>RuntimeException</code> .
Who / what triggers it	Thrown by <code>DriverRequestServiceImpl.deleteDriverRequest</code> and handled by <code>GlobalExceptionHandler.handleConflict</code> (409).
What it uses / calls	<code>RuntimeException</code> .
Size	7 lines

Line 1 | Package

```
1 | package com.examly.springapp.exceptions;  
Same exceptions package.
```

Line 3 | Class declaration

```
3 | public class DriverRequestDeletionException extends RuntimeException {  
extends RuntimeException (unchecked, so no throws clause is needed in the service interface).
```

Lines 4-7 | Constructor

```
4 |     public DriverRequestDeletionException(String message) {  
5 |         super(message);  
6 |     }  
7 | }
```

Takes the message and passes it on with `super(message)`. The text, for example "Failed to delete driver request with ID: 5", will be returned to the client.

Workflow, next: Now the interface for booking requests: `DriverRequestService.java`.

Step 30: DriverRequestService - booking request contract [Arjun - Driver & Booking]

BACKEND

`springapp/src/main/java/com/examly/springapp/service/DriverRequestService.java`

What this file is	The interface of the driver request service. A driver request is a customer's booking of a driver for a trip; it moves through the status chain Pending, Approved or Rejected, Trip End and Closed. The interface lists seven operations used by the customer pages and the admin request page.
Who / what triggers it	Used by <code>DriverRequestController</code> (compile time) and implemented by <code>DriverRequestServiceImpl</code> ; Spring injects the implementation at startup.
What it uses / calls	The <code>DriverRequest</code> entity, <code>List</code> and <code>Optional</code> .
Size	15 lines

Lines 1-5 | Package and imports

```
1 | package com.examly.springapp.service;  
2 |  
3 | import com.examly.springapp.model.DriverRequest;  
4 | import java.util.List;  
5 | import java.util.Optional;
```

Same imports as `DriverService`, with `DriverRequest` as the entity.

Lines 7-10 | Create, read, update, delete

```
7 | public interface DriverRequestService {
8 |     DriverRequest addDriverRequest(DriverRequest driverRequest);
9 |     Optional<DriverRequest> getDriverRequestById(Long driverRequestId);
10 |     List<DriverRequest> getAllDriverRequests();
```

- `addDriverRequest`: save a new booking.
- `getDriverRequestById`: returns an `Optional` because the id might not exist.
- `getAllDriverRequests`: used by the admin's request list.
- `updateDriverRequest(id, request)`: used for edits, approve/reject, Trip End (which also saves the fare) and Close. Returns `null` for an unknown id.

Lines 11-15 | Delete and the two finder methods

```
11 |     DriverRequest updateDriverRequest(Long driverRequestId, DriverRequest driverRequest);
12 |     DriverRequest deleteDriverRequest(Long driverRequestId);
13 |     List<DriverRequest> findDriverRequestsByUserId(Long userId);
14 |     List<DriverRequest> findDriverRequestsByDriverId(Long driverId);
15 | }
```

- `deleteDriverRequest`: remove a request (customer only, allowed while Pending according to the pages).
- `findDriverRequestsByUserId(Long userId)`: all requests made by one customer ("My requests").
- `findDriverRequestsByDriverId(Long driverId)`: all requests for one driver (admin side).

The last two are the only methods whose names are not simple CRUD (create, read, update, delete).

Workflow, next: The code for these seven methods is in `DriverRequestServiceImpl.java`.

Step 31: DriverRequestServiceImpl - booking logic and partial update [Arjun - Driver & Booking + Prajan (payment fields)]

BACKEND

springapp/src/main/java/com/examly/springapp/service/DriverRequestServiceImpl.java

What this file is	Implements <code>DriverRequestService</code> . Two things are special compared with the driver service: a new request automatically gets today's date, and the update method is a partial update that only changes the fields the client actually sent (non-null). That second behaviour is what lets the same <code>PUT</code> serve every step of the status chain: approve (only <code>status</code>), Trip End (<code>status</code> , <code>actualDropTime</code> , <code>actualDropDate</code> , <code>actualDuration</code> , <code>paymentAmount</code>), edit (trip fields) and so on.
Who / what triggers it	Called by <code>DriverRequestController</code> for every <code>/api/driverRequest</code> endpoint.
What it uses / calls	<code>DriverRequestRepo</code> (including the derived queries <code>findByUserUserId</code> and <code>findByDriverDriverId</code>), <code>DriverRequest</code> , <code>DriverRequestDeletionException</code> and <code>java.time.LocalDate</code> .
Size	82 lines

Lines 1-11 | Package and imports

```
1 | package com.examly.springapp.service;
2 |
3 | import com.examly.springapp.exceptions.DriverRequestDeletionException;
4 | import com.examly.springapp.model.DriverRequest;
5 | import com.examly.springapp.repository.DriverRequestRepo;
6 | import org.springframework.beans.factory.annotation.Autowired;
7 | import org.springframework.stereotype.Service;
8 |
9 | import java.time.LocalDate;
10 | import java.util.List;
11 | import java.util.Optional;
```

`LocalDate` (a date without time, java.time API) is imported for the request date. The rest repeats the service pattern: repository, `@Autowired`, `@Service`, `List`, `Optional` and the deletion exception.

Lines 13-17 | Class and repository injection

```
13 | @Service
14 | public class DriverRequestServiceImpl implements DriverRequestService {
15 |
16 |     @Autowired
17 |     private DriverRequestRepo driverRequestRepo;
```

`@Service` makes it a bean, and Spring injects the generated `DriverRequestRepo`.

Lines 19-25 | addDriverRequest - default request date

```
19 | @Override
20 | public DriverRequest addDriverRequest(DriverRequest driverRequest) {
21 |     if (driverRequest.getRequestDate() == null) {
22 |         driverRequest.setRequestDate(LocalDate.now());
23 |     }
24 |     return driverRequestRepo.save(driverRequest);
25 | }
```

If the browser did not send `requestDate`, line 22 sets it to `LocalDate.now()` (today on the server). Then `save` inserts the row. The request keeps the `user` and `driver` objects from the JSON body; only their ids matter to the database (the `userId` column cannot be null, otherwise the insert fails and the controller answers 409).

Lines 27-35 | Read one and read all

```
27 | @Override
28 | public Optional<DriverRequest> getDriverRequestById(Long driverRequestId) {
29 |     return driverRequestRepo.findById(driverRequestId);
30 | }
31 |
32 | @Override
33 | public List<DriverRequest> getAllDriverRequests() {
34 |     return driverRequestRepo.findAll();
35 | }
```

`getDriverRequestById` forwards the `Optional` from `findById`. `getAllDriverRequests` returns `findAll()` (used by the admin).

Lines 37-41 | updateDriverRequest - load the stored request

```
37 | @Override
38 | public DriverRequest updateDriverRequest(Long driverRequestId, DriverRequest driverRequest) {
39 |     Optional<DriverRequest> existingReqOpt = driverRequestRepo.findById(driverRequestId);
40 |     if (existingReqOpt.isPresent()) {
41 |         DriverRequest existingReq = existingReqOpt.get();
```

Loads the stored entity by id. If it is absent the method returns `null` at line 56 and the controller answers 404. From here on `existingReq` is the entity that will be modified and saved.

Lines 42-53 | Copy only the fields that are not null

```
42 |         if (driverRequest.getStatus() != null) existingReq.setStatus(driverRequest.getStatus());
43 |         if (driverRequest.getTripDate() != null) existingReq.setTripDate(driverRequest.getTripDate());
44 |         if (driverRequest.getTimeSlot() != null) existingReq.setTimeSlot(driverRequest.getTimeSlot());
45 |         if (driverRequest.getPickupLocation() != null) existingReq.setPickupLocation(driverRequest.getPickupLocation());
46 |         if (driverRequest.getDropLocation() != null) existingReq.setDropLocation(driverRequest.getDropLocation());
47 |         if (driverRequest.getEstimatedDuration() != null) existingReq.setEstimatedDuration(driverRequest.getEstimatedDuration());
48 |         if (driverRequest.getPaymentAmount() != null) existingReq.setPaymentAmount(driverRequest.getPaymentAmount());
49 |         if (driverRequest.getComments() != null) existingReq.setComments(driverRequest.getComments());
50 |         if (driverRequest.getActualDropTime() != null) existingReq.setActualDropTime(driverRequest.getActualDropTime());
51 |         if (driverRequest.getActualDropDate() != null) existingReq.setActualDropDate(driverRequest.getActualDropDate());
52 |         if (driverRequest.getActualDuration() != null) existingReq.setActualDuration(driverRequest.getActualDuration());
53 |         if (driverRequest.getDriver() != null) existingReq.setDriver(driverRequest.getDriver());
```

Each line has the shape `if (incoming.getX() != null) existing.setX(incoming.getX());`. So a field that is missing from the JSON (it arrives as `null`) is left unchanged. Twelve fields are covered: `status`, `tripDate`, `timeSlot`, `pickupLocation`, `dropLocation`, `estimatedDuration`, `paymentAmount`, `comments`, `actualDropTime`, `actualDropDate`, `actualDuration` and `driver`.

Which page uses what: the admin approve/reject button only sends `status`; the customer edit sends trip date, slot, locations and duration; the Trip End action sends `status` plus the actual drop time/date/duration and `paymentAmount` (the Angular app computes the fare as hourly rate multiplied by duration before sending). The `user` and `requestDate` are intentionally not copyable, so a request can never be handed to another customer. Note that the service does **not** check the status chain (for example that Close only follows Trip End); that order is enforced by the Angular pages.

Lines 54-57 | Save and not-found result

```
54 |         return driverRequestRepo.save(existingReq);
55 |     }
56 |     return null;
57 | }
```

`save(existingReq)` runs an `UPDATE` and returns the updated entity; line 56 returns `null` when the id was not found.

Lines 59-71 | deleteDriverRequest

```
59 | @Override
60 | public DriverRequest deleteDriverRequest(Long driverRequestId) {
61 |     Optional<DriverRequest> existingReqOpt = driverRequestRepo.findById(driverRequestId);
62 |     if (existingReqOpt.isPresent()) {
63 |         try {
64 |             driverRequestRepo.delete(existingReqOpt.get());
65 |             return existingReqOpt.get();
66 |         } catch (Exception e) {
67 |             throw new DriverRequestDeletionException("Failed to delete driver request with ID: " + driverRequestId);
68 |         }
69 |     }
70 |     return null;
71 | }
```

Same pattern as `deleteDriver`: find; if present, delete inside a `try`; if the database throws, convert it into `DriverRequestDeletionException`; return the deleted request; return `null` when it did not exist. The controller answers 200 with the removed request, 404 if `null`, and the exception handler answers 409 on the failure.

Lines 73-82 | Finders by user and by driver

```
73 | @Override
74 | public List<DriverRequest> findDriverRequestsByUserId(Long userId) {
75 |     return driverRequestRepo.findByUserUserId(userId);
76 | }
77 |
78 | @Override
79 | public List<DriverRequest> findDriverRequestsByDriverId(Long driverId) {
80 |     return driverRequestRepo.findByDriverDriverId(driverId);
81 | }
82 | }
```

These two delegate to derived queries in `DriverRequestRepo`. `findByUserUserId` means "the `user` property, then its `userId` property": it runs `WHERE user_id = ?`. `findByDriverDriverId` does the same through the `driver` property. They return empty lists (never `null`) when nothing matches; the controller turns an empty list into 404. Line 82 closes the class.

Workflow, next: Next the AI side of the backend. `AIService` and `FeedbackServiceImpl` depend on Google Gemini, so we start with the HTTP wrapper around it: `GeminiService.java`.

Step 32: GeminiService - talking to Google Gemini

[Saumya - Feedback & AI]

springapp/src/main/java/com/examly/springapp/service/GeminiService.java

BACKEND

What this file is	A thin wrapper around the Google Gemini REST API . Gemini is a large language model service on the internet. This class offers two abilities and nothing else: embed (turn a text into a list of numbers, an embedding vector, that captures its meaning) and generate (send a prompt and get text back). It contains no business logic; <code>AIService</code> decides what to ask. The most important design idea is graceful degradation . If no API key is configured (<code>GEMINI_API_KEY</code> is empty, which is the default), or the network call fails, <code>embed</code> and <code>generate</code> simply return <code>null</code> , and the caller switches to its local, non-AI logic. That is why the whole application and its tests work without internet or a key.
Who / what triggers it	Spring creates one instance at startup (<code>@Service</code>). Its methods are called by <code>AIService</code> whenever the customer uses the AI driver search, a feedback is saved, or a feedback summary is requested.
What it uses / calls	The three <code>gemini.*</code> keys of <code>application.properties</code> (read with <code>@Value</code>), the JDK's built-in <code>java.net.http.HttpClient</code> (no extra library), Jackson (<code>ObjectMapper</code> , <code>JsonNode</code>) for JSON, and SLF4J for log messages. Over the internet it calls https://generativelanguage.googleapis.com/v1beta/models/...
Size	114 lines

Lines 1-17 | Package and imports

```
1 | package com.examly.springapp.service;
2 |
3 | import com.fasterxml.jackson.databind.JsonNode;
4 | import com.fasterxml.jackson.databind.ObjectMapper;
5 | import org.slf4j.Logger;
6 | import org.slf4j.LoggerFactory;
7 | import org.springframework.beans.factory.annotation.Value;
8 | import org.springframework.stereotype.Service;
9 |
10 | import java.net.URI;
11 | import java.net.http.HttpClient;
12 | import java.net.http.HttpRequest;
13 | import java.net.http.HttpResponse;
14 | import java.time.Duration;
15 | import java.util.LinkedHashMap;
16 | import java.util.List;
17 | import java.util.Map;
```

- Jackson **JsonNode** / **ObjectMapper**: build and read JSON text. **JsonNode** is a read-only tree of a JSON document.
- SLF4J **Logger** / **LoggerFactory**: write log lines to the console (Spring Boot's default logging facade).
- **Value**: reads a property into a field. **Service**: bean annotation.
- **URI**, **HttpClient**, **HttpRequest**, **HttpResponse**: the Java 11+ HTTP client classes used to call the Gemini server.
- **Duration**: timeouts. **LinkedHashMap**, **List**, **Map**: building the JSON request bodies.

Lines 19-25 | Class comment, @Service and class declaration

```
19 | /**
20 |  * Thin wrapper around the Google Gemini REST API (embeddings + text generation).
21 |  * Every method degrades gracefully: when no API key is configured, or a call fails,
22 |  * embed() / generate() return null and the callers fall back to non-AI logic.
23 |  */
24 | @Service
25 | public class GeminiService {
```

The Javadoc comment states the contract: every method degrades gracefully and returns **null** on failure. **@Service** registers one shared bean, so the HTTP client and the key are created only once.

Lines 27-28 | Logger and base URL

```
27 |     private static final Logger log = LoggerFactory.getLogger(GeminiService.class);
28 |     private static final String BASE_URL = "https://generativelanguage.googleapis.com/v1beta/models/";
```

log is a static logger named after the class; **log.warn(...)** below writes a warning line when a Gemini call fails. **BASE_URL** is the common start of every Gemini endpoint; the model name and the method (**embedContent** or **generateContent**) are appended later in **post**.

Lines 30-37 | Settings read with @Value

```
30 |     @Value("${gemini.api.key}")
31 |     private String apiKey;
32 |
33 |     @Value("${gemini.embedding.model:text-embedding-004}")
34 |     private String embeddingModel;
35 |
36 |     @Value("${gemini.generation.model:gemini-2.0-flash}")
37 |     private String generationModel;
```

@Value("\${gemini.api.key}") fills the field from the property of the same name. The text after **:** is the default, here an empty string. In **application.properties** the key is **\${GEMINI_API_KEY:}**, i.e. the value of the operating-system environment variable **GEMINI_API_KEY**, or empty. The other two fields hold the model names: **text-embedding-004** for embeddings and **gemini-2.0-flash** for text generation (each also has a default in case the property is missing). Spring writes the values into the fields after it creates the object.

Lines 39-40 | JSON mapper and HTTP client

```
39 |     private final ObjectMapper mapper = new ObjectMapper();
40 |     private final HttpClient client = HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(10)).build();
```

mapper converts Java objects to JSON text and JSON text to a tree. **client** is built once with a 10-second connection timeout and reused for every request (building one per call would be wasteful). Both are **final** fields created directly because they do not need Spring.

Lines 42-44 | isEnabled

```
42 |     public boolean isEnabled() {
43 |         return apiKey != null && !apiKey.trim().isEmpty();
44 |     }
```

The on/off switch for all AI features: true only when **apiKey** is not null and not blank after trimming. **aiService** calls it before every Gemini attempt.

Lines 46-50 | embed - guard clause

```
46 |     /** Returns the embedding vector for the text, or null when AI is disabled / the call fails. */
47 |     public float[] embed(String text) {
48 |         if (!isEnabled() || text == null || text.isBlank()) {
49 |             return null;
50 |         }
```

`embed(String text)` returns a `float[]`, the embedding vector. The first `if` returns `null` straight away when AI is switched off or the text is null or blank, so no HTTP call is made.

Lines 51-61 | embed - build request, call, read vector

```
51 |         try {
52 |             Map<String, Object> body = Map.of("content", Map.of("parts", List.of(Map.of("text", text))));
53 |             JsonNode values = post(embeddingModel, "embedContent", body).path("embedding").path("values");
54 |             if (!values.isArray() || values.size() == 0) {
55 |                 return null;
56 |             }
57 |             float[] vector = new float[values.size()];
58 |             for (int i = 0; i < vector.length; i++) {
59 |                 vector[i] = (float) values.get(i).asDouble();
60 |             }
61 |             return vector;
```

- Line 52 builds the JSON body `{"content":{"parts":[{"text":"..."}]}}` out of nested `Map.of` and `List.of` (immutable helper collections; Jackson turns them into JSON).
- Line 53 calls `post(embeddingModel, "embedContent", body)`, which returns the parsed answer, then walks down `embedding` then `values`. `JsonNode.path(name)` never returns `null`: a missing field gives a "missing node", so the chain cannot crash.
- Lines 54-56: if `values` is not a non-empty array the answer is unusable and `null` is returned.
- Lines 57-61: copy the JSON numbers into a `float[]`. For the model `text-embedding-004` this array has 768 numbers. Similar meanings produce vectors that point in similar directions (used by `AIService.cosine`).

Lines 62-66 | embed - failure handling

```
62 |         } catch (Exception e) {
63 |             log.warn("Gemini embedding failed: {}", e.getMessage());
64 |             return null;
65 |         }
66 |     }
```

Any exception (no network, HTTP error, bad JSON) is caught; a warning is logged and `null` is returned. The caller treats `null` as "AI not available" and uses the fallback.

Lines 68-76 | generate and generateJson - public entry points

```
68 |     /** Returns the generated text, or null when AI is disabled / the call fails. */
69 |     public String generate(String prompt) {
70 |         return generate(prompt, false);
71 |     }
72 |
73 |     /** Same as generate(prompt) but asks Gemini to answer with raw JSON. */
74 |     public String generateJson(String prompt) {
75 |         return generate(prompt, true);
76 |     }
```

Two public convenience methods that both call the private `generate(prompt, json)`: `generate(prompt)` asks for normal text (`json = false`), `generateJson(prompt)` asks Gemini to answer with raw JSON (`json = true`). This is called **method overloading**: two methods with the same name `generate` but different parameters (one public with one argument, one private with two).

Lines 78-81 | generate - guard clause

```
78 |     private String generate(String prompt, boolean json) {
79 |         if (!isEnabled() || prompt == null || prompt.isBlank()) {
80 |             return null;
81 |         }
```

Same safety check as `embed`: no key or an empty prompt gives `null` without any network traffic.

Lines 82-90 | generate - build the request body

```
82 |         try {
83 |             Map<String, Object> body = new LinkedHashMap<>();
84 |             body.put("contents", List.of(Map.of("parts", List.of(Map.of("text", prompt)))));
85 |             Map<String, Object> config = new LinkedHashMap<>();
86 |             config.put("temperature", 0.2);
87 |             if (json) {
88 |                 config.put("responseMimeType", "application/json");
89 |             }
90 |             body.put("generationConfig", config);
```

Builds `{"contents":[{"parts":[{"text":prompt}]}], "generationConfig":{"temperature":0.2, "responseMimeType":"application/json"}}`. **Temperature** controls randomness: 0.2 is low, so the answers are stable and factual, which suits classification. When `json` is true the config adds `responseMimeType = application/json`, which makes Gemini return pure JSON instead of free text. `LinkedHashMap` is used so the keys keep the insertion order.

Lines 91-98 | generate - read the answer text

```
91 |         JsonNode text = post(generationModel, "generateContent", body)
92 |             .path("candidates").path(0).path("content").path("parts").path(0).path("text");
93 |         return text.isMissingNode() ? null : text.asText();
94 |     } catch (Exception e) {
95 |         log.warn("Gemini generation failed: {}", e.getMessage());
96 |         return null;
97 |     }
98 | }
```

Posts to the generation model with the method `generateContent` and digs into the answer: `candidates` then the first candidate `content`, `parts`, first part `text`. If that node is missing the method returns `null`; otherwise `asText()` returns the generated string. As in `embed`, any exception is logged and becomes `null`.

Lines 100-107 | post - build the HTTP request

```
100 |     private JsonNode post(String model, String method, Object body) throws Exception {
101 |         HttpRequest request = HttpRequest.newBuilder()
102 |             .uri(URI.create(BASE_URL + model + ":" + method))
103 |             .timeout(Duration.ofSeconds(25))
104 |             .header("Content-Type", "application/json")
105 |             .header("x-goog-api-key", apiKey.trim())
106 |             .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(body)))
107 |             .build();
```

A private helper shared by `embed` and `generate`. It builds a `POST` request to `BASE_URL + model + ":" + method`, for example `.../models/text-embedding-004:embedContent`, with a 25-second timeout, header `Content-Type: application/json` and the API key in the header `x-goog-api-key` (so the key is not put in the URL). The body is the Java map converted to a JSON string by `mapper.writeValueAsString`. The method declares `throws Exception` because the HTTP client and the JSON writer can throw checked exceptions; the callers catch them.

Lines 108-114 | post - send and check the status

```
108 |         HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
109 |         if (response.statusCode() / 100 != 2) {
110 |             throw new IllegalStateException("Gemini API returned HTTP " + response.statusCode());
111 |         }
112 |         return mapper.readTree(response.body());
113 |     }
114 | }
```

`client.send(...)` blocks until Gemini answers and returns the body as a `String`. `statusCode() / 100 != 2` is true for anything outside 200-299 (integer division gives the "class" of the status, 2 for success), and then an `IllegalStateException` is thrown, which the callers log and turn into `null`. For a successful answer `mapper.readTree` parses the JSON text into a `JsonNode` tree and returns it.

Workflow, next: `GeminiService` is ready (and, without an API key, quietly returns `null`). The class that uses it for every AI feature is `AIService.java`.

Step 33: AIService - AI search, sentiment and review summary [Saumya - Feedback & AI]

BACKEND

springapp/src/main/java/com/examly/springapp/service/AIService.java

What this file is

The biggest service. It provides three AI features, each with a Gemini version and a local fallback so that everything keeps working with no key:

- **Semantic driver search:** the customer types a sentence like "experienced sedan driver in Dallas" and gets drivers ranked by meaning (Gemini embeddings plus cosine similarity), or by simple keyword scoring when Gemini is off.
- **Feedback analysis:** every new feedback receives a sentiment (Positive, Neutral, Negative), a score between 0 and 1 and a list of tags such as punctuality or safety.
- **Per-driver feedback summary:** average rating, an AI score out of 100, a short summary, strengths and improvements.

Who / what triggers it	AiController (endpoints <code>/api/ai/...</code>) calls <code>searchDrivers</code>, <code>summarizeDriverFeedback</code> and <code>analyzeExistingFeedback</code>. FeedbackServiceImpl.createFeedback calls <code>applyAnalysis</code> every time a customer posts feedback.
What it uses / calls	GeminiService (<code>embed / generateJson</code>), DriverRepo (<code>findAll</code>), FeedbackRepo (<code>findAll</code> , <code>save</code> , <code>findByDriverDriverId</code>), the Driver and Feedback entities, Jackson for parsing Gemini's JSON, and Java streams.
Size	368 lines

Lines 1-14 | Package and imports

```

1 | package com.examly.springapp.service;
2 |
3 | import com.examly.springapp.model.Driver;
4 | import com.examly.springapp.model.Feedback;
5 | import com.examly.springapp.repository.DriverRepo;
6 | import com.examly.springapp.repository.FeedbackRepo;
7 | import com.fasterxml.jackson.databind.JsonNode;
8 | import com.fasterxml.jackson.databind.ObjectMapper;
9 | import org.springframework.beans.factory.annotation.Autowired;
10 | import org.springframework.stereotype.Service;
11 |
12 | import java.util.*;
13 | import java.util.concurrent.ConcurrentHashMap;
14 | import java.util.stream.Collectors;

```

- Driver, Feedback, DriverRepo, FeedbackRepo**: the data this service reads and writes.
- JsonNode, ObjectMapper**: parse the JSON text that Gemini returns.
- Autowired, Service**: Spring annotations.
- java.util.***: `List`, `Map`, `Set`, `HashMap`, `HashSet`, `LinkedHashMap`, `ArrayList`, `Arrays`.
- ConcurrentHashMap**: a thread-safe map, used for the embedding cache (several web requests can run at the same time).
- Collectors**: ends a **stream** pipeline (a stream is a chain of operations such as `filter`, `map`, `sorted`, `limit` applied to a collection).

Lines 16-22 | Class comment, @Service and class

```

16 | /**
17 |  * AI features: semantic driver search, feedback sentiment / tagging and per-driver review summary.
18 |  * Gemini is used when an API key is configured; otherwise (or when a call fails) a local
19 |  * keyword based fallback keeps every feature - and the existing test cases - working.
20 |  */
21 | @Service
22 | public class AiService {

```

The comment says what the class offers and the fallback rule. `@Service` makes it a bean, injected into **AiController** and **FeedbackServiceImpl**.

Lines 24-26 | STOP_WORDS - words ignored in a search

```

24 |     private static final Set<String> STOP_WORDS = new HashSet<>(Arrays.asList(
25 |         "a", "an", "and", "the", "for", "to", "of", "in", "on", "at", "with", "my", "me", "i", "is", "are", "need"
26 |         ,
27 |         "want", "looking", "find", "who", "that", "can", "please", "driver", "drivers", "trip", "good", "best"));

```

A **static final Set<String>** built from an array. **Stop words** are very common words that carry no search meaning: "a", "the", "for", "need", "want", "looking", "find", "driver", "trip", "good", "best", and so on. They are removed from the query before keyword scoring, because every driver profile contains the word "driver" and matching it would not help rank anyone. A **Set** is used because `contains` is a very fast check.

Lines 28-30 | POSITIVE_WORDS

```

28 |     private static final Set<String> POSITIVE_WORDS = new HashSet<>(Arrays.asList(
29 |         "good", "great", "excellent", "amazing", "awesome", "polite", "friendly", "safe", "smooth", "comfortable",
30 |         "clean", "punctual", "professional", "helpful", "perfect", "best", "happy", "recommend", "careful", "promp
31 |         t"));

```

Twenty words that count as praise (good, great, excellent, polite, friendly, safe, smooth, clean, punctual, professional, recommend, prompt, ...). The local sentiment scoring counts how many of these appear in the feedback text. Note that "good" and "best" are in this set and also in **STOP_WORDS**; the two sets are used in different places (feedback analysis vs driver search).

Lines 32-34 | NEGATIVE_WORDS

```
32 | private static final Set<String> NEGATIVE_WORDS = new HashSet<>(Arrays.asList(
33 |     "bad", "poor", "terrible", "rude", "late", "dirty", "unsafe", "rash", "worst", "horrible", "delay",
34 |     "delayed", "unprofessional", "slow", "angry", "uncomfortable", "overcharged", "cancelled", "reckless"));
```

Nineteen words that count as complaints (bad, poor, terrible, rude, late, dirty, unsafe, rash, worst, delay, slow, angry, overcharged, cancelled, reckless, ...). Each hit lowers the local sentiment score.

Lines 36-45 | TAG_KEYWORDS - which words create which tag

```
36 | private static final Map<String, String[]> TAG_KEYWORDS = new LinkedHashMap<>();
37 | static {
38 |     TAG_KEYWORDS.put("punctuality", new String[]{"punctual", "on time", "late", "delay", "prompt", "waiting"});
39 |     TAG_KEYWORDS.put("safety", new String[]{"safe", "unsafe", "rash", "careful", "reckless", "speed", "accident"});
40 |     TAG_KEYWORDS.put("cleanliness", new String[]{"clean", "dirty", "hygiene", "smell"});
41 |     TAG_KEYWORDS.put("behaviour", new String[]{"polite", "rude", "friendly", "behaviour", "behavior", "courteous",
42 |         "attitude"});
43 |     TAG_KEYWORDS.put("driving skill", new String[]{"smooth", "driving", "route", "navigation", "skilled", "experie
44 |         nced"});
45 |     TAG_KEYWORDS.put("pricing", new String[]{"price", "charge", "overcharged", "fare", "cost", "cheap", "expensive"});
46 |     TAG_KEYWORDS.put("vehicle condition", new String[]{"vehicle", "car", "comfortable", "condition", "seat"});
47 | }
```

A **LinkedHashMap** (keeps insertion order) from a tag name to the keywords that trigger it. The **static { ... }** block is a **static initializer**: Java runs it once when the class is first loaded and it fills the map:

- punctuality: punctual, on time, late, delay, prompt, waiting
- safety: safe, unsafe, rash, careful, reckless, speed, accident
- cleanliness: clean, dirty, hygiene, smell
- behaviour: polite, rude, friendly, behaviour, behavior, courteous, attitude
- driving skill: smooth, driving, route, navigation, skilled, experienced
- pricing: price, charge, overcharged, fare, cost, cheap, expensive
- vehicle condition: vehicle, car, comfortable, condition, seat

analyzeLocally later adds a tag if the feedback text contains any of its keywords. Because the matching is a plain substring check, "car" also matches "careful", and "late" matches "plate"; this makes tagging approximate.

Lines 47-48 | JSON mapper and embedding cache

```
47 | private final ObjectMapper mapper = new ObjectMapper();
48 | private final Map<String, float[]> embeddingCache = new ConcurrentHashMap<>();
```

mapper reads Gemini's JSON answers. **embeddingCache** stores **driver profile text to embedding vector**. Asking Gemini for an embedding is a slow, paid network call, so once a driver's profile has been embedded the vector is kept in memory and reused by every later search (until the server restarts). The key is the profile text, so if the admin edits a driver the profile text changes and a new embedding is fetched automatically.

Lines 50-57 | Injected beans

```
50 | @Autowired
51 | private GeminiService geminiService;
52 |
53 | @Autowired
54 | private DriverRepo driverRepo;
55 |
56 | @Autowired
57 | private FeedbackRepo feedbackRepo;
```

Three **@Autowired** fields: **GeminiService** (the AI calls), **DriverRepo** (all drivers for the search) and **FeedbackRepo** (feedback for analysis and summaries).

Lines 59-68 | searchDrivers - entry point of the AI search

```
59 | // ----- semantic driver search
60 |
61 | public List<Driver> searchDrivers(String query) {
62 |     List<Driver> drivers = driverRepo.findAll();
63 |     if (query == null || query.isBlank() || drivers.isEmpty()) {
64 |         return drivers;
65 |     }
66 |     List<Driver> semantic = semanticSearch(query, drivers);
67 |     return semantic != null ? semantic : keywordSearch(query, drivers);
68 | }
```

POST /api/ai/driver-search arrives here through **AiController.searchDrivers**; **query** is the plain text the customer typed.

- Line 62 loads **all** drivers from the database.
- Lines 63-65: if the query is empty or there are no drivers, just return the full list (nothing to rank).

- Line 66 tries the semantic (Gemini) path. It returns a ranked list, or `null` when Gemini is off or fails.
- Line 67: `semantic != null ? semantic : keywordSearch(...)` is the ternary operator: use the semantic result when it exists, else the local keyword result. So the caller always receives a list.

Lines 70-78 | semanticSearch - embed the query

```
70 | /** Embedding + cosine similarity ranking; null when Gemini is unavailable. */
71 | private List<Driver> semanticSearch(String query, List<Driver> drivers) {
72 |     if (!geminiService.isEnabled()) {
73 |         return null;
74 |     }
75 |     float[] queryVector = geminiService.embed(query);
76 |     if (queryVector == null) {
77 |         return null;
78 |     }
```

- Line 72: when `isEnabled()` is false the method returns `null` immediately, which triggers the fallback.
- Line 75: `geminiService.embed(query)` returns the **query vector**, the list of 768 numbers that represents the meaning of the customer's sentence. If Gemini failed, `null` again means fallback.

Lines 79-86 | semanticSearch - score every driver

```
79 | Map<Driver, Double> scores = new LinkedHashMap<>();
80 | for (Driver driver : drivers) {
81 |     float[] vector = driverVector(driver);
82 |     if (vector == null) {
83 |         return null;
84 |     }
85 |     scores.put(driver, cosine(queryVector, vector));
86 | }
```

A `LinkedHashMap<Driver, Double>` collects one score per driver. For each driver, `driverVector(driver)` returns the vector of its profile text (from the cache or from Gemini). If **any** driver vector cannot be obtained, the whole semantic search gives up with `null`, so the ranking is never half-AI. Otherwise `cosine(queryVector, vector)` gives the similarity and is stored. The first search after a restart makes one Gemini call per driver (slow); later searches only embed the new query.

Lines 87-92 | semanticSearch - sort, limit 10, return

```
87 | return scores.entrySet().stream()
88 |     .sorted(Map.Entry.<Driver, Double>comparingByValue().reversed())
89 |     .limit(10)
90 |     .map(Map.Entry::getKey)
91 |     .collect(Collectors.toList());
92 | }
```

A stream over the map entries: `sorted(...comparingByValue().reversed())` orders by score from highest to lowest (the explicit `<Driver, Double>` is needed so Java can infer the generic types), `limit(10)` keeps only the **top 10**, `map(Map.Entry::getKey)` keeps just the `Driver` objects and `collect(Collectors.toList())` makes the final list. So the AI search returns at most 10 drivers, best match first.

Lines 94-105 | driverVector - cache lookup

```
94 | private float[] driverVector(Driver driver) {
95 |     String profile = driverProfile(driver);
96 |     float[] cached = embeddingCache.get(profile);
97 |     if (cached != null) {
98 |         return cached;
99 |     }
100 |     float[] vector = geminiService.embed(profile);
101 |     if (vector != null) {
102 |         embeddingCache.put(profile, vector);
103 |     }
104 |     return vector;
105 | }
```

Builds the profile text, then checks `embeddingCache`. A cache hit returns at once. On a miss it calls Gemini; if a vector comes back it is stored with `put` for next time and returned. If Gemini returned `null`, nothing is cached and `null` is returned (which makes `semanticSearch` give up).

Lines 107-113 | driverProfile - the text that represents a driver

```
107 | private String driverProfile(Driver driver) {
108 |     return "Driver " + driver.getDriverName() + ", vehicle: " + driver.getVehicleType()
109 |         + ", experience: " + driver.getExperienceYears() + " years"
110 |         + ", hourly rate: " + driver.getHourlyRate()
111 |         + ", status: " + driver.getAvailabilityStatus()
112 |         + ", address: " + driver.getAddress();
113 | }
```

Concatenates the searchable facts of a driver into one sentence, for example: `Driver Ravi, vehicle: Sedan, experience: 8 years, hourly rate: 30.0, status: Active, address: Dallas`. This is exactly the text that is

embedded for the semantic search, and (lower-cased) the text searched by the keyword fallback. The license number, contact number and image are left out on purpose.

Lines 115-124 | cosine - cosine similarity

```
115 | static double cosine(float[] a, float[] b) {
116 |     int n = Math.min(a.length, b.length);
117 |     double dot = 0, na = 0, nb = 0;
118 |     for (int i = 0; i < n; i++) {
119 |         dot += a[i] * b[i];
120 |         na += a[i] * a[i];
121 |         nb += b[i] * b[i];
122 |     }
123 |     return (na == 0 || nb == 0) ? 0 : dot / (Math.sqrt(na) * Math.sqrt(nb));
124 | }
```

Cosine similarity measures how similar two vectors are by the angle between them: 1 means the same direction (same meaning), 0 means unrelated, negative means opposite. Formula: dot product divided by (length of a times length of b).

- The loop (lines 118-122) accumulates three sums: **dot** = sum of $a[i] \cdot b[i]$, **na** = sum of $a[i]$ squared, **nb** = sum of $b[i]$ squared. **n** is the shorter length as a safety measure.
- Line 123: if either vector has length 0 the result is 0 (avoids dividing by zero); otherwise **dot / (sqrt(na) * sqrt(nb))**. The method is **static** and package-private so tests can call it directly.

Worked example with 2-number vectors (real ones have 768 numbers). Query $q = [1, 0]$.

- Driver A = [1, 0]: dot = 1, $|q| = 1$, $|A| = 1$, so cosine = $1/(1 \cdot 1) = \mathbf{1.00}$ (same direction).
- Driver B = [1, 1]: dot = 1, $|B| = \text{sqrt}(2) = 1.414$, so cosine = $1/1.414 = \mathbf{0.71}$.
- Driver C = [0, 1]: dot = 0, so cosine = **0.00**.

After sorting by score: A, B, C. So A is the best match, which is what the sort and **limit(10)** in **semanticSearch** produce.

Lines 126-130 | keywordSearch - tokens and intent flags

```
126 | /** Local fallback: scores each driver by the query terms found in its profile. */
127 | private List<Driver> keywordSearch(String query, List<Driver> drivers) {
128 |     List<String> tokens = tokenize(query).stream().filter(t -> !STOP_WORDS.contains(t)).collect(Collectors.toList(
129 | ));
130 |     boolean wantsExperience = tokens.stream().anyMatch(t -> t.startsWith("experie") || t.equals("senior") || t.e
131 | quals("expert"));
132 |     boolean wantsCheap = tokens.stream().anyMatch(t -> t.equals("cheap") || t.equals("affordable") || t.equals("bu
133 | dget"));
```

The local fallback used when Gemini is not available.

- Line 128: **tokenize(query)** splits the text into lower-case words, then **filter** removes the stop words.
- Line 129: **wantsExperience** is true if any token starts with "experie" (experience, experienced) or equals "senior" or "expert".
- Line 130: **wantsCheap** is true if a token is "cheap", "affordable" or "budget".

Lines 132-140 | keywordSearch - +2 per token found

```
132 | Map<Driver, Double> scores = new LinkedHashMap<>();
133 | for (Driver driver : drivers) {
134 |     String text = driverProfile(driver).toLowerCase();
135 |     double score = 0;
136 |     for (String token : tokens) {
137 |         if (text.contains(token)) {
138 |             score += 2;
139 |         }
140 |     }
```

For each driver: take its profile text in lower case and start with score 0. For every query token, if the profile **contains** the token (**String.contains**, a substring test), add **2 points**. So "sedan" and "dallas" found in a profile give 4.

Lines 141-153 | keywordSearch - bonuses and who is kept

```
141 |         if (score > 0 || tokens.isEmpty()) {
142 |             if (wantsExperience && driver.getExperienceYears() != null) {
143 |                 score += Math.min(driver.getExperienceYears(), 10) / 5.0;
144 |             }
145 |             if (wantsCheap && driver.getHourlyRate() != null) {
146 |                 score += 1.0 / (1 + driver.getHourlyRate() / 100.0);
147 |             }
148 |             if ("Active".equalsIgnoreCase(driver.getAvailabilityStatus())) {
149 |                 score += 0.5;
150 |             }
151 |             scores.put(driver, score);
152 |         }
153 |     }
```

The `if (score > 0 || tokens.isEmpty())` line decides which drivers are listed: a driver is kept only if at least one token matched (or if the query had no useful tokens at all, e.g. "best driver", in which case everyone is kept). Drivers with a match then get bonuses:

- **Experience bonus** (only if `wantsExperience`): `min(years, 10) / 5.0`, so 0 to 2 points (5 years gives 1, 10 or more years gives 2).
- **Cheap bonus** (only if `wantsCheap`): `1 / (1 + rate / 100)`, between 0 and 1; a rate of 20 gives 0.83, a rate of 100 gives 0.5, so cheaper drivers get more.
- **Active bonus**: +0.5 when `availabilityStatus` equals "Active" (ignoring case).

Then `scores.put(driver, score)`.

Worked example. Query "experienced sedan driver in Dallas". Tokens: experienced, sedan, driver, in, dallas; after removing the stop words "driver" and "in": experienced, sedan, dallas. `wantsExperience` = true.

- Driver A (Sedan, 8 years, Dallas, Active): sedan +2, dallas +2 (the word "experienced" is not a substring of "experience: 8", so no points there) = 4; experience bonus $8/5 = 1.6$; active +0.5. Total **6.1**.
- Driver C (Sedan, 10 years, Austin, Active): sedan +2, bonus 2.0, active 0.5. Total **4.5**.
- Driver B (SUV, 2 years, Dallas, Inactive): dallas +2, bonus 0.4. Total **2.4**.
- Driver D (SUV, Austin): no token matches, score 0, so D is left out.

Result order: A, C, B.

Lines 154-158 | keywordSearch - sort

```
154 |     return scores.entrySet().stream()
155 |         .sorted(Map.Entry.<Driver, Double>comparingByValue().reversed())
156 |         .map(Map.Entry::getKey)
157 |         .collect(Collectors.toList());
158 | }
```

Sorts by score descending and returns the driver list. There is no `limit` here, so every matching driver is returned. Drivers with equal scores keep the database order (the sort is stable).

Lines 160-162 | tokenize

```
160 |     private List<String> tokenize(String text) {
161 |         return Arrays.stream(text.toLowerCase().split("[^a-z0-9]+")).filter(t -> t.length() > 1).collect(Collectors.to
162 |             List());
163 |     }
```

Turns text into a list of words: `toLowerCase()`, then `split("[^a-z0-9]+")` cuts at every run of characters that are not a-z or 0-9 (spaces, commas, quotes, punctuation), and `filter(t -> t.length() > 1)` throws away empty pieces and one-letter words. Example: "Driver was on time!" gives driver, was, on, time. This also explains why the raw request body of `driver-search`, which may arrive with surrounding quotation marks, still works. Accented letters are treated as separators.

Lines 164-170 | analyzeFeedback - Gemini first, local second

```
164 | // ----- sentiment + tags
165 |
166 | /** Returns sentiment (Positive/Neutral/Negative), sentimentScore (0..1) and aiTags (comma separated). */
167 | public Map<String, Object> analyzeFeedback(String text, String category, Integer rating) {
168 |     Map<String, Object> result = analyzeWithGemini(text, category, rating);
169 |     return result != null ? result : analyzeLocally(text, rating);
170 | }
```

The public entry point for sentiment analysis. It returns a map with three keys: `sentiment` (Positive, Neutral or Negative), `sentimentScore` (0 to 1) and `aiTags` (comma separated). It first tries `analyzeWithGemini`; if that returns `null` (AI off or failed), it uses `analyzeLocally`. So the result is never `null`.

Lines 172-175 | analyzeWithGemini - guard

```
172 |     private Map<String, Object> analyzeWithGemini(String text, String category, Integer rating) {
173 |         if (!geminiService.isEnabled() || text == null || text.isBlank()) {
174 |             return null;
175 |         }
```

Skips Gemini (returns `null`) if AI is disabled or the feedback text is empty.

Lines 176-179 | analyzeWithGemini - the prompt

```
176 |         String prompt = "Classify this customer feedback about a hired driver. Respond ONLY with JSON of the form "
177 |             + "{ \"sentiment\": \"Positive|Neutral|Negative\", \"sentimentScore\": 0.0-1.0, \"tags\": [\"short theme\", .
178 |             + "(sentimentScore 0 = very negative, 1 = very positive, at most 4 tags).\\n"
179 |             + "Category: " + category + "\\nRating (1-5): " + rating + "\\nFeedback: " + text;
```

The prompt tells Gemini: classify this customer feedback, respond **only** with JSON of the form `{"sentiment": "Positive|Neutral|Negative", "sentimentScore": 0.0-1.0, "tags": ["short theme", ...]}`, where 0 means very negative, 1 very positive and at most 4 tags. It then appends the category, the star rating and the feedback text. The `\` in the Java source is an escaped quote inside the string.

Lines 180-183 | analyzeWithGemini - call

```
180 |         String raw = geminiService.generateJson(prompt);
181 |         if (raw == null) {
182 |             return null;
183 |         }
```

`generateJson(prompt)` returns Gemini's answer as text, or `null` on failure (then this method also returns `null`).

Lines 184-196 | analyzeWithGemini - parse and validate

```
184 |         try {
185 |             JsonNode node = mapper.readTree(stripFences(raw));
186 |             String sentiment = normalizeSentiment(node.path("sentiment").asText(""));
187 |             if (sentiment == null) {
188 |                 return null;
189 |             }
190 |             List<String> tags = new ArrayList<>();
191 |             node.path("tags").forEach(t -> tags.add(t.asText().trim().toLowerCase()));
192 |             return result(sentiment, clamp(node.path("sentimentScore").asDouble(0.5)), String.join(",", tags));
193 |         } catch (Exception e) {
194 |             return null;
195 |         }
196 |     }
```

- `stripFences` removes a markdown code fence (three backtick characters, optionally followed by a label such as `json`) in case the model wrapped its answer; `mapper.readTree` then parses the clean text.
- `normalizeSentiment` accepts values starting with `pos/neg/neu` in any case and maps them to `Positive/Negative/Neutral`; anything else gives `null` and the method gives up (fallback).
- Line 191: every element of `tags` is lower-cased and trimmed into a list.
- Line 192: `clamp(...)` forces the score into 0..1 (default 0.5 if missing), the tags are joined with commas ("`punctuality,safety`") and `result(...)` builds the map.
- Any parsing exception returns `null`, i.e. fallback to the local analysis.

Lines 198-205 | analyzeLocally - the scoring formula

```
198 |     private Map<String, Object> analyzeLocally(String text, Integer rating) {
199 |         String lower = text == null ? "" : text.toLowerCase();
200 |         List<String> words = tokenize(lower);
201 |         long positive = words.stream().filter(POSITIVE_WORDS::contains).count();
202 |         long negative = words.stream().filter(NEGATIVE_WORDS::contains).count();
203 |         double textScore = clamp(0.5 + 0.15 * (positive - negative));
204 |         double score = rating != null && rating > 0 ? (textScore + (rating - 1) / 4.0) / 2.0 : textScore;
205 |         String sentiment = score >= 0.6 ? "Positive" : score <= 0.4 ? "Negative" : "Neutral";
```

The local sentiment engine.

- Lines 199-200: lower-case the text and split it into words (`null` text is treated as empty).
- Lines 201-202: `positive` = how many words are in `POSITIVE_WORDS`, `negative` = how many are in `NEGATIVE_WORDS`.
- Line 203: `textScore = clamp(0.5 + 0.15 * (positive - negative))`. It starts at neutral 0.5; every net positive word adds 0.15, every net negative word removes 0.15, and the result is kept between 0 and 1.
- Line 204: if the customer gave a rating above 0, the rating is converted to a 0..1 number with `(rating - 1) / 4.0` (1 star gives 0, 3 stars 0.5, 5 stars 1) and the final `score` is the **average** of the text score and the rating score. Without a rating, `score = textScore`.
- Line 205: `score >= 0.6` is Positive, `score <= 0.4` is Negative, otherwise Neutral.

Worked example. Text: "Very polite and punctual driver, but the car was a bit dirty", rating 4. Words found: polite and punctual are positive (2), dirty is negative (1). $\text{textScore} = 0.5 + 0.15 * (2 - 1) = 0.65$. Rating score = $(4 - 1)/4 = 0.75$. $\text{score} = (0.65 + 0.75)/2 = \mathbf{0.70}$, which is at least 0.6, so the sentiment is **Positive**.

Another: "Rude and late, terrible", no rating: 3 negative words, $\text{textScore} = 0.5 - 0.45 = \mathbf{0.05}$, so **Negative**. And a feedback with no known words but 5 stars: $(0.5 + 1.0)/2 = 0.75$, so Positive.

Lines 207-217 | analyzeLocally - tags and the result

```
207 |         List<String> tags = new ArrayList<>();
208 |         for (Map.Entry<String, String[]> entry : TAG_KEYWORDS.entrySet()) {
209 |             for (String keyword : entry.getValue()) {
210 |                 if (lower.contains(keyword)) {
211 |                     tags.add(entry.getKey());
212 |                     break;
213 |                 }
214 |             }
215 |         }
216 |         return result(sentiment, Math.round(score * 100.0) / 100.0, String.join(",", tags));
217 |     }
```

- Lines 207-215: for each tag in `TAG_KEYWORDS`, check whether the lower-case text contains any of its keywords (a substring check, so the two-word keyword "on time" also works). On the first hit the tag name is added and `break` stops looking at further keywords of that tag, so each tag appears at most once.
- Line 216: the score is rounded to two decimals (`Math.round(score * 100.0) / 100.0`) and `result(...)` builds the map with the tags joined by commas.

Continuing the example: "punctual" gives punctuality, "dirty" gives cleanliness, "polite" gives behaviour, "car" gives vehicle condition. The tags are added in `TAG_KEYWORDS` order, so `aiTags = punctuality,cleanliness,behaviour,vehicle condition`, sentiment Positive, score 0.7.

Lines 219-225 | result - small map builder

```
219 |     private Map<String, Object> result(String sentiment, double score, String tags) {
220 |         Map<String, Object> result = new HashMap<>();
221 |         result.put("sentiment", sentiment);
222 |         result.put("sentimentScore", score);
223 |         result.put("aiTags", tags);
224 |         return result;
225 |     }
```

Creates the `HashMap` with the keys `sentiment`, `sentimentScore` and `aiTags` that every analysis returns. The score (a primitive `double`) is auto-boxed into a `Double`, which `applyAnalysis` later casts back.

Lines 227-238 | analyzeExistingFeedback - back-fill old feedback

```
227 |     /** Back-fills sentiment and tags for feedback created before the AI feature existed. */
228 |     public int analyzeExistingFeedback() {
229 |         int updated = 0;
230 |         for (Feedback feedback : feedbackRepo.findAll()) {
231 |             if (feedback.getSentiment() == null || feedback.getSentiment().isBlank()) {
232 |                 applyAnalysis(feedback);
233 |                 feedbackRepo.save(feedback);
234 |                 updated++;
235 |             }
236 |         }
237 |         return updated;
238 |     }
```

Called by `POST /api/ai/feedback/analyze-existing` (admin only). Feedback rows created before the AI feature existed have no sentiment. The method loops over all feedback; for each one whose `sentiment` is `null` or blank it runs `applyAnalysis` (fills sentiment, score and tags), saves the row and counts it. It returns the number of rows updated; the controller wraps it as `{"updated": 3}`. With Gemini on, this makes one API call per old feedback.

Lines 240-245 | applyAnalysis - write the analysis into the entity

```
240 |     public void applyAnalysis(Feedback feedback) {
241 |         Map<String, Object> analysis = analyzeFeedback(feedback.getFeedbackText(), feedback.getCategory(), feedback.getRating());
242 |         feedback.setSentiment((String) analysis.get("sentiment"));
243 |         feedback.setSentimentScore((Double) analysis.get("sentimentScore"));
244 |         feedback.setAiTags((String) analysis.get("aiTags"));
245 |     }
```

Public because `FeedbackServiceImpl.createFeedback` also calls it. It calls `analyzeFeedback` with the text, category and rating of the feedback and copies the three results into the entity: `setSentiment`, `setSentimentScore` (cast back from `Object` to `Double`) and `setAiTags`. It does not save; the caller saves.

Lines 247-255 | summarizeDriverFeedback - numbers

```
247 | // ----- per-driver summary
248 |
249 | public Map<String, Object> summarizeDriverFeedback(Long driverId) {
250 |     List<Feedback> feedbacks = feedbackRepo.findByDriverDriverId(driverId);
251 |     int count = feedbacks.size();
252 |     double average = feedbacks.stream().filter(f -> f.getRating() != null).mapToInt(Feedback::getRating).average()
    | .orElse(0);
253 |     average = Math.round(average * 10.0) / 10.0;
254 |     long positive = feedbacks.stream().filter(f -> "Positive".equals(f.getSentiment())).count();
255 |     int aiScore = count == 0 ? 0 : (int) Math.round(average / 5.0 * 70 + (double) positive / count * 30);
```

GET /api/ai/driver/{driverId}/feedback-summary ends here.

- Line 250: `feedbackRepo.findByDriverDriverId(driverId)` gets every feedback for that driver (derived query on `driver.driverId`).
- Line 251: `count` = how many.
- Line 252: `average` = the mean of all non-null ratings (`OptionalDouble.average().orElse(0)` gives 0 when there is none). Line 253 rounds it to one decimal.
- Line 254: `positive` = how many feedbacks have sentiment exactly "Positive".
- Line 255: `aiScore` = `average / 5 * 70 + positive / count * 30`, rounded to a whole number; 0 if there is no feedback. So 70 points come from the star rating and 30 points from the share of positive sentiments. `(double) positive / count` makes the division decimal (otherwise integer division would give 0 or 1).

Example. 4 feedbacks with ratings 5, 4, 4, 3 and 3 of them Positive: $\text{average} = 16/4 = 4.0$; rating part = $4.0/5 * 70 = 56$; sentiment part = $3/4 * 30 = 22.5$; total 78.5, rounded by `Math.round` to **79**. A single 5-star positive feedback gives $70 + 30 = 100$.

Lines 257-263 | summarizeDriverFeedback - the result map

```
257 | Map<String, Object> result = new LinkedHashMap<>();
258 | result.put("summary", "");
259 | result.put("strengths", new ArrayList<String>());
260 | result.put("improvements", new ArrayList<String>());
261 | result.put("aiScore", aiScore);
262 | result.put("averageRating", average);
263 | result.put("feedbackCount", count);
```

A `LinkedHashMap` is created with the final JSON shape and default values: empty `summary`, empty lists `strengths` and `improvements`, plus `aiScore`, `averageRating` and `feedbackCount`. The key order in the JSON follows the insertion order.

Lines 265-273 | summarizeDriverFeedback - empty case and choice of method

```
265 | if (count == 0) {
266 |     result.put("summary", "No feedback has been posted for this driver yet.");
267 |     return result;
268 | }
269 | if (!summarizeWithGemini(feedbacks, result)) {
270 |     summarizeLocally(feedbacks, average, positive, result);
271 | }
272 | return result;
273 | }
```

If the driver has no feedback, the summary is "No feedback has been posted for this driver yet." and the method returns at once (the HTTP answer is still 200; an unknown driver id gives the same message, not 404). Otherwise `summarizeWithGemini` is tried; it returns `true` when it filled the map, and if it returns `false` the `!` makes `summarizeLocally` run. Either way the numbers computed above stay in the map; Gemini only writes the text parts.

Lines 275-282 | summarizeWithGemini - build the review list

```
275 | private boolean summarizeWithGemini(List<Feedback> feedbacks, Map<String, Object> result) {
276 |     if (!geminiService.isEnabled()) {
277 |         return false;
278 |     }
279 |     String reviews = feedbacks.stream()
280 |         .limit(40)
281 |         .map(f -> "- (" + f.getRating() + "/5, " + f.getCategory() + ") " + f.getFeedbackText())
282 |         .collect(Collectors.joining("\n"));
```

Returns `false` when AI is off. Otherwise the first 40 feedbacks (`limit(40)`, to keep the prompt small) are turned into lines like - (4/5, Punctuality) The driver was on time, joined with newlines.

Lines 283-288 | summarizeWithGemini - prompt and call

```
283 | String raw = geminiService.generateJson("Summarise these customer reviews of one driver. Respond ONLY with JSO
    | N "
284 |         + "{\n\"summary\":\n\"2-3 sentences\", \"strengths\": [\n\"...\n\"], \"improvements\": [\n\"...\n\"]} "
285 |         + "(at most 4 short items per list).\nReviews:\n" + reviews);
286 | if (raw == null) {
287 |     return false;
288 | }
```

The prompt asks Gemini to summarise the reviews of one driver and respond only with JSON `{"summary": "2-3 sentences", "strengths": ["..."], "improvements": ["..."]}` with at most 4 short items per list, followed by the review lines. A `null` answer returns `false` (fallback).

Lines 289-302 | summarizeWithGemini - parse into the result

```
289 | try {
290 |     JsonNode node = mapper.readTree(stripFences(raw));
291 |     String summary = node.path("summary").asText("");
292 |     if (summary.isBlank()) {
293 |         return false;
294 |     }
295 |     result.put("summary", summary);
296 |     result.put("strengths", toList(node.path("strengths")));
297 |     result.put("improvements", toList(node.path("improvements")));
298 |     return true;
299 | } catch (Exception e) {
300 |     return false;
301 | }
302 | }
```

Parses the JSON (after `stripFences`), takes `summary` (an empty summary counts as failure), puts the summary and the two lists (via `toList`) into the result map and returns `true`. Any exception returns `false`.

Lines 304-317 | summarizeLocally - count tags of good and bad reviews

```
304 | private void summarizeLocally(List<Feedback> feedbacks, double average, long positive, Map<String, Object> result)
    | {
305 |     Map<String, Integer> good = new HashMap<>();
306 |     Map<String, Integer> bad = new HashMap<>();
307 |     for (Feedback feedback : feedbacks) {
308 |         boolean isGood = "Positive".equals(feedback.getSentiment()) || (feedback.getRating() != null && feedback.ge
    | etRating() >= 4);
309 |         boolean isBad = "Negative".equals(feedback.getSentiment()) || (feedback.getRating() != null && feedback.ge
    | tRating() <= 2);
310 |         String tags = feedback.getAiTags() == null ? "" : feedback.getAiTags();
311 |         for (String tag : tags.split(",")) {
312 |             if (!tag.isBlank()) {
313 |                 if (isGood) good.merge(tag.trim(), 1, Integer::sum);
314 |                 if (isBad) bad.merge(tag.trim(), 1, Integer::sum);
315 |             }
316 |         }
317 |     }
```

The no-AI summary. Two maps count how often each tag occurs. For every feedback: it is **good** if its sentiment is Positive or its rating is 4 or more; **bad** if Negative or rating 2 or less. Its `aiTags` text (stored earlier when it was analysed) is split on commas, and for each non-blank tag the matching counter is increased with `merge(tag, 1, Integer::sum)` (insert 1 if absent, otherwise add 1).

Lines 318-330 | summarizeLocally - build strengths, improvements and summary

```
318 | List<String> strengths = topKeys(good).stream().map(t -> "Customers praise the " + t).collect(Collectors.toLis
    | t());
319 | List<String> improvements = topKeys(bad).stream().map(t -> "Customers raised concerns about " + t).collect(Col
    | lectors.toList());
320 | if (strengths.isEmpty() && average >= 4) {
321 |     strengths.add("Consistently high ratings from customers");
322 | }
323 | if (improvements.isEmpty()) {
324 |     improvements.add("No recurring issues reported");
325 | }
326 | result.put("summary", "Based on " + feedbacks.size() + " review(s) with an average rating of " + average + "/5
    | , "
327 |         + positive + " of them were positive.");
328 | result.put("strengths", strengths);
329 | result.put("improvements", improvements);
330 | }
```

- `topKeys` returns the 3 most frequent tags. Each good tag becomes "Customers praise the punctuality"; each bad tag becomes "Customers raised concerns about pricing".
- If there are no strengths and the average is 4 or more, "Consistently high ratings from customers" is added. If there are no improvements, "No recurring issues reported" is added.
- The summary sentence is built from the numbers: for example "Based on 4 review(s) with an average rating of 4.0/5, 3 of them were positive."

The results are written into the map passed in.

Lines 332-338 | topKeys helper

```
332 | // ----- helpers
333 |
334 | private List<String> topKeys(Map<String, Integer> counts) {
335 |     return counts.entrySet().stream()
336 |         .sorted(Map.Entry.<String, Integer>comparingByValue().reversed().thenComparing(Map.Entry.comparingByKey()))
337 |         .limit(3).map(Map.Entry::getKey).collect(Collectors.toList());
338 | }
```

Sorts a tag-count map by count descending, then by tag name alphabetically (so ties are stable), keeps the first three and returns the tag names.

Lines 340-346 | toList helper

```
340 | private List<String> toList(JsonNode array) {
341 |     List<String> list = new ArrayList<>();
342 |     array.forEach(item -> {
343 |         if (!item.asText().isBlank()) list.add(item.asText().trim());
344 |     });
345 |     return list;
346 | }
```

Converts a JSON array node into a `List<String>`: every element is trimmed and blank ones are skipped.

Lines 348-354 | normalizeSentiment helper

```
348 | private String normalizeSentiment(String value) {
349 |     String v = value == null ? "" : value.trim().toLowerCase();
350 |     if (v.startsWith("pos")) return "Positive";
351 |     if (v.startsWith("neg")) return "Negative";
352 |     if (v.startsWith("neu")) return "Neutral";
353 |     return null;
354 | }
```

Cleans whatever word Gemini used: trim, lower-case, then `startsWith("pos")` gives "Positive", `"neg"` gives "Negative", `"neu"` gives "Neutral", otherwise `null` (meaning "invalid answer").

Lines 356-358 | clamp helper

```
356 | private double clamp(double value) {
357 |     return Math.max(0.0, Math.min(1.0, value));
358 | }
```

`Math.max(0.0, Math.min(1.0, value))` forces a number into the range 0..1.

Lines 360-368 | stripFences helper

```
360 | private String stripFences(String raw) {
361 |     String s = raw.trim();
362 |     if (s.startsWith("```")) {
363 |         s = s.replaceFirst("^```[a-zA-Z]*", "");
364 |         s = s.replaceFirst("```\\s*$", "");
365 |     }
366 |     return s.trim();
367 | }
368 | }
```

Gemini sometimes wraps its JSON in a markdown code fence: three backtick characters plus an optional language label such as `json` before the text, and three backticks after it. This helper removes them. It trims the text; if the result starts with three backticks, the first `replaceFirst` call deletes the opening fence together with the label (the regular expression means: start of text, three backticks, then any letters), and the second `replaceFirst` deletes the closing fence (three backticks followed only by white space up to the end of text). The cleaned text is trimmed and returned, ready for `mapper.readTree`. Line 368 closes the class.

Workflow, next: All AI logic is defined. The service that stores feedback and calls `applyAnalysis` for each new entry follows, starting with its contract: `FeedbackService.java`.

Step 34: FeedbackService - feedback contract [Saumya - Feedback & AI]

BACKEND

springapp/src/main/java/com/examly/springapp/service/FeedbackService.java

What this file is	The interface of the feedback service: customers write feedback about a driver (a rating from 1 to 5, a category and a text); the admin and the public pages read them. Five operations are declared.
Who / what triggers it	Used by <code>FeedbackController</code> and implemented by <code>FeedbackServiceImpl</code> .
What it uses / calls	The <code>Feedback</code> entity and <code>java.util.List</code> .
Size	12 lines

Lines 1-4 | Package and imports

```
1 | package com.examly.springapp.service;
2 |
3 | import com.examly.springapp.model.Feedback;
4 | import java.util.List;
```

`Feedback` is the entity, `List` the collection type.

Lines 6-12 | The five operations

```
6 | public interface FeedbackService {
7 |     Feedback createFeedback(Feedback feedback);
8 |     Feedback getFeedbackById(Long feedbackId);
9 |     List<Feedback> getAllFeedbacks();
10 |    Feedback deleteFeedback(Long feedbackId);
11 |    List<Feedback> getFeedbacksByUserId(Long userId);
12 | }
```

- `createFeedback(Feedback)`: saves a feedback (and, in the implementation, enriches it with AI sentiment and tags).
- `getFeedbackById(Long)`: returns the feedback or `null`. Unlike the driver services, this one returns the entity directly instead of an `Optional`.
- `getAllFeedbacks()`: all feedback (public list).
- `deleteFeedback(Long)`: delete and return the removed feedback, or `null` if not found.
- `getFeedbacksByUserId(Long)`: the feedback written by one customer.

Workflow, next: The implementation is `FeedbackServiceImpl.java`.

Step 35: FeedbackServiceImpl - saving feedback with AI analysis [Saumya - Feedback & AI]

BACKEND

springapp/src/main/java/com/examly/springapp/service/FeedbackServiceImpl.java

What this file is	Implements <code>FeedbackService</code> . Its special feature is in <code>createFeedback</code> : before the feedback is saved it is sent through <code>AIService.applyAnalysis</code> , which attaches sentiment, score and tags. The analysis is optional: if anything goes wrong the feedback is still saved without those fields.
Who / what triggers it	<code>FeedbackController</code> for the <code>/api/feedback</code> endpoints. For <code>POST /api/feedback</code> it is the customer pressing Submit on the feedback page.
What it uses / calls	<code>FeedbackRepo</code> , <code>AIService</code> (which may call <code>GeminiService</code>), the <code>Feedback</code> entity and <code>LocalDate</code> .
Size	59 lines

Lines 1-10 | Package and imports

```
1 | package com.examly.springapp.service;
2 |
3 | import com.examly.springapp.model.Feedback;
4 | import com.examly.springapp.repository.FeedbackRepo;
5 | import org.springframework.beans.factory.annotation.Autowired;
6 | import org.springframework.stereotype.Service;
7 |
8 | import java.time.LocalDate;
9 | import java.util.List;
10 | import java.util.Optional;
```

The usual service imports plus `LocalDate` for the default date.

Lines 12-19 | Class and two injected beans

```
12 | @Service
13 | public class FeedbackServiceImpl implements FeedbackService {
14 |
15 |     @Autowired
16 |     private FeedbackRepo feedbackRepo;
17 |
18 |     @Autowired
19 |     private AiService aiService;
```

`@Service` bean with two `@Autowired` collaborators: the repository for persistence and `AiService` for the sentiment analysis. Spring works out the creation order itself: `AiService` is built first because `FeedbackServiceImpl` needs it.

Lines 21-33 | createFeedback - date, AI analysis, save

```
21 |     @Override
22 |     public Feedback createFeedback(Feedback feedback) {
23 |         if (feedback.getDate() == null) {
24 |             feedback.setDate(LocalDate.now());
25 |         }
26 |         try {
27 |             // AI sentiment analysis + auto-tagging; feedback is still saved if the analysis fails.
28 |             aiService.applyAnalysis(feedback);
29 |         } catch (Exception ignored) {
30 |             // keep the feedback without AI attributes
31 |         }
32 |         return feedbackRepo.save(feedback);
33 |     }
```

- Lines 23-25: if the form did not provide a date, set it to today.
- Lines 26-31: `aiService.applyAnalysis(feedback)` fills `sentiment`, `sentimentScore` and `aiTags`. It is wrapped in `try/catch (Exception ignored)`, so even an unexpected failure only means those three fields stay empty. (`analyzeFeedback` never returns `null`, so this catch is a second safety net.)
- Line 32: `feedbackRepo.save(feedback)` inserts the row and returns it with its `feedbackId` and AI fields. The controller sends that object back with status 201.

Lines 35-43 | getFeedbackById and getAllFeedbacks

```
35 |     @Override
36 |     public Feedback getFeedbackById(Long feedbackId) {
37 |         return feedbackRepo.findById(feedbackId).orElse(null);
38 |     }
39 |
40 |     @Override
41 |     public List<Feedback> getAllFeedbacks() {
42 |         return feedbackRepo.findAll();
43 |     }
```

`findById(id).orElse(null)` unwraps the `Optional`: the entity, or `null` when absent (the controller then answers 404). `getAllFeedbacks` returns `findAll()`.

Lines 45-53 | deleteFeedback

```
45 |     @Override
46 |     public Feedback deleteFeedback(Long feedbackId) {
47 |         Optional<Feedback> feedbackOpt = feedbackRepo.findById(feedbackId);
48 |         if (feedbackOpt.isPresent()) {
49 |             feedbackRepo.delete(feedbackOpt.get());
50 |             return feedbackOpt.get();
51 |         }
52 |         return null;
53 |     }
```

Finds the feedback, deletes it and returns the removed object, or `null` when it does not exist. Unlike driver deletion there is no `try/catch`, because nothing else refers to a feedback row, so a foreign-key failure cannot happen.

Lines 55-59 | getFeedbacksByUserId

```
55 | @Override
56 | public List<Feedback> getFeedbacksByUserId(Long userId) {
57 |     return feedbackRepo.findByUserUserId(userId);
58 | }
59 | }
```

Calls the derived query `findByUserUserId` (feedback whose `user.userId` equals the id). Line 59 closes the class.

Workflow, next: All services are done. The layer above them is the controllers, which receive the HTTP requests. The first, and the one the Angular app calls first (register and login), is `AuthController.java`.

Step 36: AuthController - register and login endpoints

[Aleena - Authentication]

springapp/src/main/java/com/examly/springapp/controller/AuthController.java

BACKEND

What this file is

The first **controller**. A controller is the front door of the backend: it maps an HTTP method plus URL to a Java method, converts the JSON body into Java objects, calls a service and converts the answer back into JSON with a status code. `AuthController` serves the two public endpoints `POST /api/register` and `POST /api/login`.

The path of every request, from the browser to the controller and back (this applies to all controllers that follow):

- The Angular app (port 8081) calls the backend on port 8080. Because the origin differs, the browser may first send an `OPTIONS` pre-flight request; the CORS settings (`CorsConfig`, enabled by `.cors(...)` in `SecurityConfig`) answer it.
- Tomcat hands the request to the **Spring Security filter chain** built in `SecurityConfig`. `JwtAuthenticationFilter` runs first: it reads the header `Authorization: Bearer <token>`, validates the token with `JwtUtils`, loads the user with `MyUserDetailsService` and puts the logged-in user and his role (`ROLE_ADMIN` or `ROLE_CUSTOMER`) into the security context. Without a valid token the request simply continues as anonymous.
- The authorization rules of `SecurityConfig` are checked from top to bottom; the **first rule that matches the method and URL wins**. A `permitAll` rule lets anybody through, `hasRole` needs that role, `authenticated` needs any logged-in user. If nobody is logged in and the rule needs it, `JwtAuthenticationEntryPoint` answers **401**; if the user is logged in with the wrong role, `JwtAccessDeniedHandler` answers **403**. In both cases the controller is never reached.
- If the rules pass, Spring MVC's `DispatcherServlet` picks the controller method whose mapping fits, Jackson converts the JSON body to the Java parameter, the method calls the service, the service calls the repository, Hibernate talks to H2.
- The returned `ResponseEntity` is converted to JSON by Jackson and sent back with its status code. If an exception escapes the controller, `GlobalExceptionHandler` turns it into a JSON error (last file of this part).

Who / what triggers it

The browser: the signup page posts to `/api/register`, the login page posts to `/api/login`. Both URLs are `permitAll` in `SecurityConfig`, so no token is needed.

What it uses / calls

`UserService` (to create a user), `AuthenticationManager` and `JwtUtils` (to log in and make a token), `UserPrincipal` (the Spring Security wrapper of a `User`), `LoginDTO` (the login answer) and `User` (used for both request bodies).

Size

58 lines

Lines 1-15 | Package and imports

```
1 | package com.examly.springapp.controller;
2 |
3 | import com.examly.springapp.config.JwtUtils;
4 | import com.examly.springapp.config.UserPrincipal;
5 | import com.examly.springapp.model.LoginDTO;
6 | import com.examly.springapp.model.User;
7 | import com.examly.springapp.service.UserService;
8 | import org.springframework.beans.factory.annotation.Autowired;
9 | import org.springframework.http.HttpStatus;
10 | import org.springframework.http.ResponseEntity;
11 | import org.springframework.security.authentication.AuthenticationManager;
12 | import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
13 | import org.springframework.security.core.Authentication;
14 | import org.springframework.security.core.context.SecurityContextHolder;
15 | import org.springframework.web.bind.annotation.*;
```

- `JwtUtils`, `UserPrinciple`: from the `config` package. `JwtUtils.generateJwtToken` builds the token; `UserPrinciple` wraps the `User` for Spring Security.
- `LoginDTO`, `User`: model classes. A **DTO** (data transfer object) is a plain class that only carries data for one answer.
- `UserService`: the business-rule layer.
- `Autowired`; `HttpStatus` (enum of status codes such as `CREATED`, `CONFLICT`); `ResponseEntity` (a whole HTTP response: status, headers, body).
- `AuthenticationManager`, `UsernamePasswordAuthenticationToken`, `Authentication`, `SecurityContextHolder`: Spring Security classes used to check the e-mail/password.
- `org.springframework.web.bind.annotation.*`: `@RestController`, `@RequestMapping`, `@PostMapping`, `@RequestBody`, and (used in later controllers) `@GetMapping`, `@PutMapping`, `@DeleteMapping`, `@PathVariable`.

Lines 17-19 | `@RestController` and `@RequestMapping("/api")`

```
17 | @RestController
18 | @RequestMapping("/api")
19 | public class AuthController {
```

`@RestController` = `@Controller` + `@ResponseBody`: the class handles web requests and whatever its methods return is written straight into the response body as JSON (not looked up as an HTML page). `@RequestMapping("/api")` on the class adds the prefix `/api` to every method mapping, so `@PostMapping("/register")` is really `POST /api/register`. Spring creates one instance of the class at startup, as a bean.

Lines 21-28 | Injected beans

```
21 |     @Autowired
22 |     private AuthenticationManager authenticationManager;
23 |
24 |     @Autowired
25 |     private UserService userService;
26 |
27 |     @Autowired
28 |     private JwtUtils jwtUtils;
```

- `AuthenticationManager` is the bean from `SecurityConfig.authenticationManager`; it checks credentials using the `DaoAuthenticationProvider` (which uses `MyUserDetailsService` and the Bcrypt encoder).
- `UserService` is injected as the interface type; Spring supplies `UserServiceImpl`.
- `JwtUtils` is the `@Component` that signs tokens.

Lines 30-37 | `POST /api/register` - `registerUser`

```
30 |     @PostMapping("/register")
31 |     public ResponseEntity<?> registerUser(@RequestBody User user) {
32 |         User createdUser = userService.createUser(user);
33 |         if (createdUser == null) {
34 |             return ResponseEntity.status(HttpStatus.CONFLICT).body("A user with this email already exists");
35 |         }
36 |         return ResponseEntity.status(HttpStatus.CREATED).body(createdUser);
37 |     }
```

- `@RequestBody User user`: Jackson reads the JSON from the signup form (`email`, `password`, `username`, `mobileNumber`, `userRole`) and creates a `User` object.
- Line 32 calls `userService.createUser(user)`.
- Lines 33-35: `null` means the e-mail is taken. The controller answers **409 Conflict** with the text "A user with this email already exists". 409 means "your request clashes with the current state of the data".
- Line 36: otherwise **201 Created** with the saved user as JSON body. 201 (not 200) is the correct code when a new resource was created. The JSON has `userId`, `email`, `username`, `mobileNumber` and `userRole`; the password is hidden by `@JsonProperty(WRITE_ONLY)` on the entity.
- The return type `ResponseEntity<?>` is used because the body is a `String` in one branch and a `User` in the other.

In the browser the Angular signup page then navigates to the login page.

Lines 39-43 | `POST /api/login` - `authenticate the credentials`

```
39 |     @PostMapping("/login")
40 |     public ResponseEntity<?> authenticateUser(@RequestBody User loginRequest) {
41 |         try {
42 |             Authentication authentication = authenticationManager.authenticate(
43 |                 new UsernamePasswordAuthenticationToken(loginRequest.getEmail(), loginRequest.getPassword()));
```

The request body is also a `User` object (only `email` and `password` are filled).

`UsernamePasswordAuthenticationToken(email, password)` is an "authentication request" holding the raw values. `authenticationManager.authenticate(...)` hands it to the `DaoAuthenticationProvider`, which calls `MyUserDetailsService.loadUserByUsername(email)`, loads the user, and compares the typed password with the

stored BCrypt hash. If the e-mail is unknown or the password wrong it throws an `AuthenticationException` and the code jumps to the `catch`. On success it returns an `Authentication` object whose principal is a `UserPrincipal`.

Lines 45-46 | Create the JWT

```
45 | SecurityContextHolder.getContext().setAuthentication(authentication);
46 | String jwt = jwtUtils.generateJwtToken(authentication);
```

Line 45 stores the authentication in the security context of the current thread (in this stateless setup it is not really needed, because the request ends right after). Line 46 calls `jwtUtils.generateJwtToken`, which builds a **JWT** (JSON Web Token): a signed text containing the e-mail as subject and an expiry time (24 hours), signed with HS512. The browser keeps it and sends it back on every later request in the `Authorization: Bearer` header.

Lines 48-53 | Build and return the LoginDTO

```
48 | UserPrincipal userDetails = (UserPrincipal) authentication.getPrincipal();
49 | User user = userDetails.getUser();
50 |
51 | LoginDTO loginDTO = new LoginDTO(jwt, user.getUsername(), user.getUserRole(), user.getId());
52 | // Tests expect HTTP 200 OK for login
53 | return ResponseEntity.status(HttpStatus.OK).body(loginDTO);
```

The principal is cast to `UserPrincipal`, and `getUser()` gives the full `User` entity. A `LoginDTO` is built with the token, username, role and `userId`, and returned with **200 OK**. The Angular login page stores these four values in `localStorage` (keys `token`, `username`, `userRole`, `userId`) and then navigates to `/home`; the navbars and guards read `userRole` from `localStorage` to show the admin or the customer menu. The comment says the project's tests expect 200 here.

Lines 54-58 | Failure = 401

```
54 |     } catch (Exception e) {
55 |         return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid credentials");
56 |     }
57 | }
58 | }
```

Any exception (wrong password, unknown e-mail, even an unexpected error) is caught and answered with **401 Unauthorized** and the text "Invalid credentials". The same message for both causes is deliberate: it does not reveal whether the e-mail exists. Honest note: because the catch is for `Exception`, a bug inside the `try` would also show up as "Invalid credentials". Line 58 closes the class.

Workflow, next: Authentication endpoints are done. The next controller manages drivers (admin writes, public reads): `DriverController.java`.

Step 37: DriverController - /api/driver endpoints [Arjun - Driver & Booking]

BACKEND

springapp/src/main/java/com/examly/springapp/controller/DriverController.java

What this file is

The REST endpoints for drivers. All five methods are thin: they call `DriverService` and translate the result into a status code. The security rules (from `SecurityConfig`, first matching rule wins) are:

- `GET /api/driver` is **public** (`permitAll`), so even visitors who are not logged in can see the driver list.
- `POST`, `PUT`, `DELETE /api/driver...` need the role **ADMIN**.
- `GET /api/driver/{id}` also needs **ADMIN** (the exact-URL `permitAll` rule only covers the list).

Who / what triggers it

The admin driver management page (add, edit, delete), the admin and customer driver lists, and the edit form that loads one driver. The request first passes the filter chain described in `AuthController`, then lands here.

What it uses / calls

`DriverService` (implemented by `DriverServiceImpl`), the `Driver` entity, `ResponseEntity` and `HttpStatus`.

Size

65 lines

Lines 1-11 | Package and imports

```
1 | package com.examly.springapp.controller;
2 |
3 | import com.examly.springapp.model.Driver;
4 | import com.examly.springapp.service.DriverService;
5 | import org.springframework.beans.factory.annotation.Autowired;
6 | import org.springframework.http.HttpStatus;
7 | import org.springframework.http.ResponseEntity;
8 | import org.springframework.web.bind.annotation.*;
9 |
10 | import java.util.List;
11 | import java.util.Optional;
```

The `Driver` entity, the `DriverService` interface, `Autowired`, `HttpStatus`, `ResponseEntity`, the web annotations (`org.springframework.web.bind.annotation.*`), plus `List` and `Optional` for the return types.

Lines 13-18 | Class, base path and injected service

```
13 | @RestController
14 | @RequestMapping("/api")
15 | public class DriverController {
16 |
17 |     @Autowired
18 |     private DriverService driverService;
```

`@RestController + @RequestMapping("/api")`: every URL below starts with `/api`. The `DriverService` interface type is injected by `@Autowired`; at runtime it is the `DriverServiceImpl` bean.

Lines 20-28 | POST /api/driver - addDriver (201 or 409)

```
20 |     @PostMapping("/driver")
21 |     public ResponseEntity<Driver> addDriver(@RequestBody Driver driver) {
22 |         try {
23 |             Driver savedDriver = driverService.addDriver(driver);
24 |             return ResponseEntity.status(HttpStatus.CREATED).body(savedDriver);
25 |         } catch (Exception e) {
26 |             return ResponseEntity.status(HttpStatus.CONFLICT).build();
27 |         }
28 |     }
```

`@PostMapping("/driver")` binds `POST /api/driver`. `@RequestBody Driver driver` turns the JSON of the admin form into a `Driver` (the picture arrives as a base64 text in `image`). The service saves it and the controller answers **201 Created** with the saved driver (it now has a `driverId`).

If anything fails, `catch (Exception e)` answers **409 Conflict** with an empty body. The intended case is the duplicate license number (`DuplicateDriverException`), which is a conflict with existing data. Honest note: the catch is **generic**, so any other error (for example a database error) is also reported as 409, and the message text of the exception is thrown away. Because the controller catches it, `DuplicateDriverException` never reaches `GlobalExceptionHandler` on this path.

Lines 30-37 | GET /api/driver/{driverId} - viewDriverById (200 or 404)

```
30 |     @GetMapping("/driver/{driverId}")
31 |     public ResponseEntity<Driver> viewDriverById(@PathVariable Long driverId) {
32 |         Optional<Driver> driverOpt = driverService.getDriverById(driverId);
33 |         if (driverOpt.isPresent()) {
34 |             return ResponseEntity.status(HttpStatus.OK).body(driverOpt.get());
35 |         }
36 |         return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
37 |     }
```

`@GetMapping("/driver/{driverId}")` with `@PathVariable Long driverId`: the part of the URL in braces is copied into the parameter and converted to a `Long` (for `/api/driver/7`, `driverId` is 7). The service returns an `Optional<Driver>`: if present the answer is **200 OK** with the driver; if empty, **404 Not Found** (no such resource) with no body. Admin only.

Lines 39-46 | GET /api/driver - viewAllDrivers (200 or 204)

```
39 |     @GetMapping("/driver")
40 |     public ResponseEntity<List<Driver>> viewAllDrivers() {
41 |         List<Driver> drivers = driverService.getAllDrivers();
42 |         if (drivers.isEmpty()) {
43 |             return ResponseEntity.status(HttpStatus.NO_CONTENT).build();
44 |         }
45 |         return ResponseEntity.status(HttpStatus.OK).body(drivers);
46 |     }
```

Returns every driver. If the list is empty the answer is **204 No Content**: the request succeeded but there is nothing to send, so the body is empty (Angular's `HttpClient` then delivers `null` as the body, which the pages must handle). Otherwise **200 OK** with the JSON array. Public endpoint.

Lines 48-55 | PUT /api/driver/{driverId} - updateDriver (200 or 404)

```
48 | @PutMapping("/driver/{driverId}")
49 | public ResponseEntity<Driver> updateDriver(@PathVariable Long driverId, @RequestBody Driver driver) {
50 |     Driver updatedDriver = driverService.updateDriver(driverId, driver);
51 |     if (updatedDriver != null) {
52 |         return ResponseEntity.status(HttpStatus.OK).body(updatedDriver);
53 |     }
54 |     return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
55 | }
```

Takes the id from the URL and the new data from the body. `driverService.updateDriver` returns the updated entity, or `null` when the id does not exist. **200 OK** with the updated driver or **404 Not Found**. **PUT** means "replace this resource with the data I send". Admin only.

Lines 57-65 | DELETE /api/driver/{driverId} - deleteDriver (200 or 404)

```
57 | @DeleteMapping("/driver/{driverId}")
58 | public ResponseEntity<Driver> deleteDriver(@PathVariable Long driverId) {
59 |     Driver deletedDriver = driverService.deleteDriver(driverId);
60 |     if (deletedDriver != null) {
61 |         return ResponseEntity.status(HttpStatus.OK).body(deletedDriver);
62 |     }
63 |     return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
64 | }
65 | }
```

Deletes by id. `null` from the service means the driver does not exist, so **404**; otherwise **200 OK** with the deleted driver in the body (a delete could also answer 204, but this project returns the removed object). If the database refuses (driver already used by requests or feedback), `DriverServiceImpl` throws `DriverDeletionException`; it is **not** caught here, so `GlobalExceptionHandler` answers 409 with the message "Failed to delete driver with ID: ...". Admin only. Line 65 closes the class.

Workflow, next: Driver endpoints are covered. Booking requests come next: `DriverRequestController.java`.

Step 38: DriverRequestController - /api/driverRequest endpoints [Arjun - Driver & Booking]

BACKEND

springapp/src/main/java/com/examly/springapp/controller/DriverRequestController.java

What this file is

The REST endpoints for booking requests. Eight methods; the roles come from `SecurityConfig` (rules are checked in order, the first match wins, which is why the specific `/user/` and `/driver/` URLs are listed before the general ones):

- **POST /api/driverRequest: CUSTOMER** only (only a customer books).
- **GET /api/driverRequest** (all): **ADMIN** only.
- **GET /api/driverRequest/user/{userId}: CUSTOMER** only ("my requests").
- **GET /api/driverRequest/driver/{driverId}: ADMIN** only.
- **GET /api/driverRequest/{id}** and **PUT /api/driverRequest/{id}: any logged-in user** (customer edits and Trip End, admin approves, rejects and closes).
- **DELETE /api/driverRequest/{id}: CUSTOMER** only.

Honest note: the rules check the role but not ownership, so a logged-in customer who knows another request's id could read or change it. The Angular pages only show a customer his own requests.

Who / what triggers it

Customer pages (book a driver, my requests, edit, delete, Trip End) and the admin request page (list, approve, reject, close). The request passes the security chain first (see `AuthController`).

What it uses / calls

`DriverRequestService` (`DriverRequestServiceImpl`), the `DriverRequest` entity, `ResponseEntity`, `HttpStatus`.

Size

83 lines

Lines 1-11 | Package and imports

```
1 | package com.examly.springapp.controller;
2 |
3 | import com.examly.springapp.model.DriverRequest;
4 | import com.examly.springapp.service.DriverRequestService;
5 | import org.springframework.beans.factory.annotation.Autowired;
6 | import org.springframework.http.HttpStatus;
7 | import org.springframework.http.ResponseEntity;
8 | import org.springframework.web.bind.annotation.*;
9 |
10 | import java.util.List;
11 | import java.util.Optional;
```

Same set as `DriverController`, with the `DriverRequest` entity and `DriverRequestService`.

Lines 13-18 | Class, base path and injected service

```
13 | @RestController
14 | @RequestMapping("/api")
15 | public class DriverRequestController {
16 |
17 |     @Autowired
18 |     private DriverRequestService driverRequestService;
```

`@RestController` with `@RequestMapping("/api")`, and the service injected by `@Autowired`.

Lines 20-28 | POST /api/driverRequest - addDriverRequest (201 or 409)

```
20 |     @PostMapping("/driverRequest")
21 |     public ResponseEntity<DriverRequest> addDriverRequest(@RequestBody DriverRequest driverRequest) {
22 |         try {
23 |             DriverRequest savedRequest = driverRequestService.addDriverRequest(driverRequest);
24 |             return ResponseEntity.status(HttpStatus.CREATED).body(savedRequest);
25 |         } catch (Exception e) {
26 |             return ResponseEntity.status(HttpStatus.CONFLICT).build();
27 |         }
28 |     }
```

The body is JSON with the nested `user` and `driver` (only their ids matter) plus trip fields and `status` "Pending". On success **201 Created** with the saved request; the service has set `requestDate` to today if missing. The generic `catch (Exception e)` gives **409 Conflict** with an empty body for any failure, for example a missing user (the `userId` column cannot be null). As with drivers, this is a broad catch rather than a precise 409 rule.

Lines 30-37 | GET /api/driverRequest/{driverRequestId} - viewDriverRequestById (200 or 404)

```
30 |     @GetMapping("/driverRequest/{driverRequestId}")
31 |     public ResponseEntity<DriverRequest> viewDriverRequestById(@PathVariable Long driverRequestId) {
32 |         Optional<DriverRequest> reqOpt = driverRequestService.getDriverRequestById(driverRequestId);
33 |         if (reqOpt.isPresent()) {
34 |             return ResponseEntity.status(HttpStatus.OK).body(reqOpt.get());
35 |         }
36 |         return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
37 |     }
```

Path variable converted to `Long`; the Optional from the service decides **200 OK** with the request or **404 Not Found**. The edit page and the Trip End screen load a request this way.

Lines 39-46 | GET /api/driverRequest/user/{userId} - viewDriverRequestsByUserId (200 or 404)

```
39 |     @GetMapping("/driverRequest/user/{userId}")
40 |     public ResponseEntity<List<DriverRequest>> viewDriverRequestsByUserId(@PathVariable Long userId) {
41 |         List<DriverRequest> requests = driverRequestService.findDriverRequestsByUserId(userId);
42 |         if (requests == null || requests.isEmpty()) {
43 |             return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
44 |         }
45 |         return ResponseEntity.status(HttpStatus.OK).body(requests);
46 |     }
```

All requests of one customer. The service returns a list; `null` or empty gives **404 Not Found**, otherwise **200 OK** with the array. Notice the difference from the "all requests" endpoint below: a customer with no booking yet gets 404 here, not 204, so the Angular page has to treat 404 as "nothing to show".

Lines 48-55 | GET /api/driverRequest - viewAllDriverRequests (200 or 204)

```
48 |     @GetMapping("/driverRequest")
49 |     public ResponseEntity<List<DriverRequest>> viewAllDriverRequests() {
50 |         List<DriverRequest> requests = driverRequestService.getAllDriverRequests();
51 |         if (requests.isEmpty()) {
52 |             return ResponseEntity.status(HttpStatus.NO_CONTENT).build();
53 |         }
54 |         return ResponseEntity.status(HttpStatus.OK).body(requests);
55 |     }
```

Admin list of all requests. Empty gives **204 No Content**, otherwise **200 OK** with the JSON array. (Both 204 and 404 are used for "empty list" in this class; the project is not consistent.)

Lines 57-64 | PUT /api/driverRequest/{driverRequestId} - updateDriverRequest (200 or 404)

```
57 | @PutMapping("/driverRequest/{driverRequestId}")
58 | public ResponseEntity<DriverRequest> updateDriverRequest(@PathVariable Long driverRequestId, @RequestBody DriverRe
   | quest driverRequest) {
59 |     DriverRequest updated = driverRequestService.updateDriverRequest(driverRequestId, driverRequest);
60 |     if (updated != null) {
61 |         return ResponseEntity.status(HttpStatus.OK).body(updated);
62 |     }
63 |     return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
64 | }
```

Receives the request id in the URL and the changed fields in the body. `updateDriverRequest` in the service copies only non-null fields. **200 OK** with the updated request, or **404 Not Found** when the id does not exist. All status changes (Approved, Rejected, Trip End, Closed) travel through this single endpoint.

Lines 66-73 | GET /api/driverRequest/driver/{driverId} - viewDriverRequestsByDriverId (200 or 404)

```
66 | @GetMapping("/driverRequest/driver/{driverId}")
67 | public ResponseEntity<List<DriverRequest>> viewDriverRequestsByDriverId(@PathVariable Long driverId) {
68 |     List<DriverRequest> requests = driverRequestService.findDriverRequestsByDriverId(driverId);
69 |     if (requests == null || requests.isEmpty()) {
70 |         return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
71 |     }
72 |     return ResponseEntity.status(HttpStatus.OK).body(requests);
73 | }
```

All requests of one driver (admin). Same list rule as the by-user endpoint: `null` or empty gives **404**, otherwise **200**.

Lines 75-83 | DELETE /api/driverRequest/{driverRequestId} - deleteDriverRequest (200 or 404)

```
75 | @DeleteMapping("/driverRequest/{driverRequestId}")
76 | public ResponseEntity<DriverRequest> deleteDriverRequest(@PathVariable Long driverRequestId) {
77 |     DriverRequest deleted = driverRequestService.deleteDriverRequest(driverRequestId);
78 |     if (deleted != null) {
79 |         return ResponseEntity.status(HttpStatus.OK).body(deleted);
80 |     }
81 |     return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
82 | }
83 | }
```

Deletes by id: **200 OK** with the removed request, or **404 Not Found** when `null`. A database failure inside the service throws `DriverRequestDeletionException`, which is not caught here and ends in `GlobalExceptionHandler` as 409. The Angular page only offers delete while the request is Pending. Line 83 closes the class.

Workflow, next: Next the feedback endpoints: `FeedbackController.java`.

Step 39: FeedbackController - /api/feedback endpoints

[Saumya - Feedback & AI]

springapp/src/main/java/com/examly/springapp/controller/FeedbackController.java

BACKEND

What this file is

The REST endpoints for customer feedback about drivers. Roles (from `SecurityConfig`):

- **POST /api/feedback**: **CUSTOMER** only.
- **GET /api/feedback** (all): **public**, so visitors and customers can read the reviews.
- **GET /api/feedback/user/{userId}**: **CUSTOMER** only (my feedback).
- **GET /api/feedback/{id}**: any logged-in user.
- **DELETE /api/feedback/{id}**: **CUSTOMER** only.

Notes: the public list contains each feedback together with its nested `user` and `driver` objects, so the author's e-mail and mobile number are visible to anyone calling **GET /api/feedback** (the password is never sent). As with requests, there is no owner check on delete.

Who / what triggers it

The customer's "post feedback" and "my feedback" pages, the admin feedback page and the public feedback view.

What it uses / calls

`FeedbackService` (`FeedbackServiceImpl`, which calls `AiService`), the `Feedback` entity, `ResponseEntity`, `HttpStatus`.

Size

64 lines

Lines 1-10 | Package and imports

```
1 | package com.examly.springapp.controller;
2 |
3 | import com.examly.springapp.model.Feedback;
4 | import com.examly.springapp.service.FeedbackService;
5 | import org.springframework.beans.factory.annotation.Autowired;
6 | import org.springframework.http.HttpStatus;
7 | import org.springframework.http.ResponseEntity;
8 | import org.springframework.web.bind.annotation.*;
9 |
10 | import java.util.List;
```

The **Feedback** entity, **FeedbackService**, Spring web classes, and **List**.

Lines 12-17 | Class, base path and injected service

```
12 | @RestController
13 | @RequestMapping("/api")
14 | public class FeedbackController {
15 |
16 |     @Autowired
17 |     private FeedbackService feedbackService;
```

@RestController, **@RequestMapping("/api")**, and the injected **FeedbackService**.

Lines 19-27 | POST /api/feedback - createFeedback (201 or 409)

```
19 |     @PostMapping("/feedback")
20 |     public ResponseEntity<Feedback> createFeedback(@RequestBody Feedback feedback) {
21 |         try {
22 |             Feedback created = feedbackService.createFeedback(feedback);
23 |             return ResponseEntity.status(HttpStatus.CREATED).body(created);
24 |         } catch (Exception e) {
25 |             return ResponseEntity.status(HttpStatus.CONFLICT).build();
26 |         }
27 |     }
```

The JSON body has **feedbackText**, **rating**, **category**, **user** and **driver**. **feedbackService.createFeedback** adds the date and the AI sentiment and tags and saves it. The answer is **201 Created** with the stored feedback (now including **sentiment**, **sentimentScore** and **aiTags**). Again a generic **catch (Exception e)** produces **409 Conflict** with an empty body for any failure (for example a missing user).

Lines 29-36 | GET /api/feedback/{feedbackId} - viewFeedbackById (200 or 404)

```
29 |     @GetMapping("/feedback/{feedbackId}")
30 |     public ResponseEntity<Feedback> viewFeedbackById(@PathVariable Long feedbackId) {
31 |         Feedback feedback = feedbackService.getFeedbackById(feedbackId);
32 |         if (feedback != null) {
33 |             return ResponseEntity.status(HttpStatus.OK).body(feedback);
34 |         }
35 |         return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
36 |     }
```

Here the service returns the entity or **null** directly (no Optional). **null** gives **404 Not Found**, otherwise **200 OK**.

Lines 38-45 | GET /api/feedback - viewAllFeedbacks (200 or 204)

```
38 |     @GetMapping("/feedback")
39 |     public ResponseEntity<List<Feedback>> viewAllFeedbacks() {
40 |         List<Feedback> feedbacks = feedbackService.getAllFeedbacks();
41 |         if (feedbacks.isEmpty()) {
42 |             return ResponseEntity.status(HttpStatus.NO_CONTENT).build();
43 |         }
44 |         return ResponseEntity.status(HttpStatus.OK).body(feedbacks);
45 |     }
```

All feedback. **204 No Content** when there is none, otherwise **200 OK** with the array.

Lines 47-54 | GET /api/feedback/user/{userId} - viewFeedbacksByUser (200 or 404)

```
47 |     @GetMapping("/feedback/user/{userId}")
48 |     public ResponseEntity<List<Feedback>> viewFeedbacksByUser(@PathVariable Long userId) {
49 |         List<Feedback> feedbacks = feedbackService.getFeedbacksByUserId(userId);
50 |         if (feedbacks == null || feedbacks.isEmpty()) {
51 |             return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
52 |         }
53 |         return ResponseEntity.status(HttpStatus.OK).body(feedbacks);
54 |     }
```

The feedback written by one customer. **null** or empty gives **404 Not Found**, otherwise **200 OK**. As in the request controller, "none yet" is a 404 here.

Lines 56-64 | DELETE /api/feedback/{feedbackId} - deleteFeedback (200 or 404)

```
56 | @DeleteMapping("/feedback/{feedbackId}")
57 | public ResponseEntity<Feedback> deleteFeedback(@PathVariable Long feedbackId) {
58 |     Feedback deleted = feedbackService.deleteFeedback(feedbackId);
59 |     if (deleted != null) {
60 |         return ResponseEntity.status(HttpStatus.OK).body(deleted);
61 |     }
62 |     return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
63 | }
64 | }
```

200 OK with the deleted feedback, or **404 Not Found**. There is no try/catch because deleting a feedback cannot violate a foreign key. Line 64 closes the class.

Workflow, next: The last controller exposes the AI features: `AiController.java`.

Step 40: AiController - /api/ai endpoints [Saumya - Feedback & AI]

BACKEND

springapp/src/main/java/com/examly/springapp/controller/AiController.java

What this file is	Three endpoints in front of <code>AIService</code> . Its base path is <code>/api/ai</code> (its own <code>@RequestMapping</code> , different from the other controllers). Roles from <code>SecurityConfig</code> : • <code>POST /api/ai/driver-search</code>: public (<code>permitAll</code>), the AI search box for customers. • <code>GET /api/ai/driver/{driverId}/feedback-summary</code>: ADMIN or CUSTOMER. • <code>POST /api/ai/feedback/analyze-existing</code>: ADMIN only. All three always answer 200 OK : the service has a fallback for every case, so there is no 404, 204 or 409 here.
Who / what triggers it	The customer driver page (search by sentence) and the pages that show an AI summary of a driver's feedback. No Angular page calls <code>analyze-existing</code> ; it is an admin maintenance endpoint that can be called with a tool such as Postman.
What it uses / calls	<code>AIService</code> , the <code>Driver</code> entity, <code>ResponseEntity</code> , <code>Map</code> .
Size	36 lines

Lines 1-11 | Package and imports

```
1 | package com.examly.springapp.controller;
2 |
3 | import com.examly.springapp.model.Driver;
4 | import com.examly.springapp.service.AIService;
5 | import org.springframework.beans.factory.annotation.Autowired;
6 | import org.springframework.http.ResponseEntity;
7 | import org.springframework.web.bind.annotation.*;
8 |
9 | import java.util.List;
10 | import java.util.Map;
11 | import java.util.HashMap;
```

`Driver` (return type of the search), `AIService`, the Spring web classes, and `List`, `Map`, `HashMap` for the answers.

Lines 13-18 | Class, base path /api/ai and injected service

```
13 | @RestController
14 | @RequestMapping("/api/ai")
15 | public class AiController {
16 |
17 |     @Autowired
18 |     private AIService aiService;
```

`@RequestMapping("/api/ai")` gives all three methods the prefix `/api/ai`. `AIService` is injected.

Lines 20-23 | POST /api/ai/driver-search - searchDrivers

```
20 | @PostMapping("/driver-search")
21 | public ResponseEntity<List<Driver>> searchDrivers(@RequestBody String query) {
22 |     return ResponseEntity.ok(aiService.searchDrivers(query));
23 | }
```

`@RequestBody String query`: because the parameter type is `String`, Spring does not parse JSON; it passes the **raw body text** (the Angular `AIService` sends the sentence as `text/plain`). The method calls `aiService.searchDrivers(query)` and returns the ranked drivers as a JSON array with **200 OK**. Without a Gemini key this is the keyword fallback.

Lines 25-28 | GET /api/ai/driver/{driverId}/feedback-summary - getFeedbackSummary

```
25 | @GetMapping("/driver/{driverId}/feedback-summary")
26 | public ResponseEntity<Map<String, Object>> getFeedbackSummary(@PathVariable Long driverId) {
27 |     return ResponseEntity.ok(aiService.summarizeDriverFeedback(driverId));
28 | }
```

`@PathVariable Long driverId` takes the id from the URL. The service returns a `Map<String, Object>` (summary, strengths, improvements, aiScore, averageRating, feedbackCount); Jackson turns the map into a JSON object. **200 OK** even if the driver has no feedback (the summary then says so).

Lines 30-36 | POST /api/ai/feedback/analyze-existing - analyzeExistingFeedback

```
30 | @PostMapping("/feedback/analyze-existing")
31 | public ResponseEntity<Map<String, Integer>> analyzeExistingFeedback() {
32 |     Map<String, Integer> response = new HashMap<>();
33 |     response.put("updated", aiService.analyzeExistingFeedback());
34 |     return ResponseEntity.ok(response);
35 | }
36 | }
```

No body. It runs the back-fill and returns `{"updated": n}`, the number of old feedback rows that received sentiment and tags. Line 36 closes the class.

Workflow, next: All controllers are covered. One piece of the backend is left: what happens when an exception escapes a controller, handled by `GlobalExceptionHandler.java`.

Step 41: GlobalExceptionHandler - one place for all errors [Vaishalini - Customer Journey & Shared Infra]

BACKEND

springapp/src/main/java/com/examly/springapp/exceptions/GlobalExceptionHandler.java

What this file is	The last class of the backend. <code>@RestControllerAdvice</code> makes it a global safety net for every controller. Whenever a controller method throws an exception that it does not catch itself, Spring looks here for a method annotated <code>@ExceptionHandler</code> that fits the exception type. The handler does three things: it logs the error, stores one row in the <code>ErrorLogs</code> table, and returns a short JSON message such as <code>{"message": "..."} with a suitable status code, so the browser never receives a Java stack trace.</code> Limits: it only sees exceptions thrown inside controller methods. Security errors (401, 403) are produced earlier in the filter chain by <code>JwtAuthenticationEntryPoint</code> and <code>JwtAccessDeniedHandler</code>. And because the controllers in this project catch generic <code>Exception</code> in their create methods (and answer 409 themselves), those exceptions never reach this class.
Who / what triggers it	Spring calls it automatically when an exception leaves a controller, for example <code>DriverDeletionException</code> from <code>DELETE /api/driver/{id}</code>, a malformed JSON body, or an unexpected <code>NullPointerException</code>.
What it uses / calls	<code>ErrorLogRepo</code> and <code>ErrorLog</code> (to store the error), the three custom exceptions, <code>HttpServletRequest</code> (for method and URL), SLF4J logging, Spring's <code>ResponseEntity</code>.
Size	60 lines

Lines 1-17 | Package and imports

```
1 | package com.examly.springapp.exceptions;
2 |
3 | import com.examly.springapp.model.ErrorLog;
4 | import com.examly.springapp.repository.ErrorLogRepo;
5 | import jakarta.servlet.http.HttpServletRequest;
6 | import org.slf4j.Logger;
7 | import org.slf4j.LoggerFactory;
8 | import org.springframework.beans.factory.annotation.Autowired;
9 | import org.springframework.http.HttpStatus;
10 | import org.springframework.http.HttpStatusCodes;
11 | import org.springframework.http.ResponseEntity;
12 | import org.springframework.http.converter.HttpMessageNotReadableException;
13 | import org.springframework.web.ErrorResponse;
14 | import org.springframework.web.bind.annotation.ExceptionHandler;
15 | import org.springframework.web.bind.annotation.RestControllerAdvice;
16 |
17 | import java.util.Map;
```

- `ErrorLog`, `ErrorLogRepo`: the entity and repository for the `ErrorLogs` table.
- `HttpServletRequest`: lets the handler read the request URL and method.

- `Logger`, `LoggerFactory`: console logging.
- `HttpStatus` (named status codes) and `HttpStatusCode` (the general type that can hold any status number).
- `ResponseEntity`: the response to return.
- `HttpMessageNotReadableException`: thrown by Spring when the JSON body cannot be parsed.
- `org.springframework.web.ErrorResponse`: an interface that Spring 6's own web exceptions implement; it carries the correct status code.
- `ExceptionHandler`, `RestControllerAdvice`: the two annotations that make this class work.
- `Map`: for the small JSON answer.

Lines 19-26 | Class, @RestControllerAdvice and logger

```
19 | /**
20 |  * Central exception handling: every handled exception is logged and stored in the
21 |  * "ErrorLogs" table, and the client gets a short, user-friendly message.
22 |  */
23 | @RestControllerAdvice
24 | public class GlobalExceptionHandler {
25 |
26 |     private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);
```

`@RestControllerAdvice = @ControllerAdvice + @ResponseBody`: a bean that gives advice to all controllers, and whose return values are written as JSON. `log` is the logger used in `respond`.

Lines 28-29 | Injected repository

```
28 |     @Autowired
29 |     private ErrorLogRepo errorLogRepo;
```

`ErrorLogRepo` (a `JpaRepository<ErrorLog, Long>`) is used to save the error rows.

Lines 31-34 | @ExceptionHandler 1 - the three conflict exceptions (409)

```
31 |     @ExceptionHandler({DriverDeletionException.class, DriverRequestDeletionException.class, DuplicateDriverException.class})
32 |     public ResponseEntity<Map<String, String>> handleConflict(RuntimeException ex, HttpServletRequest request) {
33 |         return respond(HttpStatus.CONFLICT, ex, ex.getMessage(), request);
34 |     }
```

`@ExceptionHandler({A.class, B.class, C.class})` registers this method for `DriverDeletionException`, `DriverRequestDeletionException` and `DuplicateDriverException`. Spring passes the exception object (typed as their common parent `RuntimeException`) and the current request. The method answers **409 Conflict** and uses the exception's own message, for example "Failed to delete driver with ID: 3". In practice, only the two deletion exceptions arrive here; `DuplicateDriverException` is caught earlier in `DriverController.addDriver`.

Lines 36-39 | @ExceptionHandler 2 - bad input (400)

```
36 |     @ExceptionHandler({HttpMessageNotReadableException.class, IllegalArgumentException.class})
37 |     public ResponseEntity<Map<String, String>> handleBadRequest(Exception ex, HttpServletRequest request) {
38 |         return respond(HttpStatus.BAD_REQUEST, ex, "The request could not be processed. Please check the data and try
39 |         again.", request);
39 |     }
```

Handles `HttpMessageNotReadableException` (the request body is not valid JSON, or has a wrong type such as text in a number field) and `IllegalArgumentException`. The status is **400 Bad Request** and the message is a fixed friendly text: "The request could not be processed. Please check the data and try again." The technical detail is not shown to the user (it is stored in the log row instead).

Lines 41-49 | @ExceptionHandler 3 - everything else, with the ErrorResponse instanceof pattern

```
41 |     @ExceptionHandler(Exception.class)
42 |     public ResponseEntity<Map<String, String>> handleAny(Exception ex, HttpServletRequest request) {
43 |         // Standard Spring MVC errors (405, 415, ...) keep their own status code.
44 |         HttpStatusCode status = ex instanceof ErrorResponse errorResponse
45 |             ? errorResponse.getStatusCode()
46 |             : HttpStatus.INTERNAL_SERVER_ERROR;
47 |         String message = status.is5xxServerError() ? "Something went wrong. Please try again later." : ex.getMessage();
48 |         ;
49 |         return respond(status, ex, message, request);
49 |     }
```

The catch-all for `Exception`. Spring always uses the most specific handler, so this one only runs when neither of the two above fits.

Line 44 uses **pattern matching for instanceof**: `ex instanceof ErrorResponse errorResponse` tests the type and, if true, also creates the variable `errorResponse` in one step. Spring 6's own web exceptions (wrong HTTP method gives 405, wrong content type gives 415, missing parameter gives 400, unknown URL gives 404) implement the `ErrorResponse` interface and know their status code, so the handler keeps it: `errorResponse.getStatusCode()`. Any other exception (a `NullPointerException`, a database error) gets **500 Internal Server Error**.

Line 47: for a 5xx status the user only sees "Something went wrong. Please try again later." so internal details are not leaked; for a 4xx status the exception's own message is shown, for example `Request method 'PATCH' is not supported`. A non-numeric id such as `/api/driver/abc` is, as far as I can tell from the code, not an `ErrorResponse`, so it would most likely end up here as a 500.

Lines 51-60 | respond - log, store the `ErrorLog` row, return JSON

```
51 |     private ResponseEntity<Map<String, String>> respond(HttpStatusCode status, Exception ex, String message, HttpServl
52 |         etRequest request) {
53 |         log.error("{} {} -> {}: {}", request.getMethod(), request.getRequestURI(), ex.getClass().getSimpleName(), ex.g
54 |         etMessage());
55 |         try {
56 |             errorLogRepo.save(new ErrorLog(status.value(), ex.getClass().getName(), ex.getMessage(), request.getReques
57 |             tURI()));
58 |         } catch (Exception loggingFailure) {
59 |             log.warn("Could not store the error log: {}", loggingFailure.getMessage());
60 |         }
61 |         return ResponseEntity.status(status).body(Map.of("message", message == null ? "Unexpected error" : message));
62 |     }
```

All three handlers end in this private method.

- Line 52: `log.error(...)` prints a line such as `DELETE /api/driver/3 -> DriverDeletionException: Failed to delete driver with ID: 3` in the server console.
- Line 54: `new ErrorLog(status, exceptionClassName, message, uri)` builds the row: the constructor sets `loggedAt` to now and shortens the message to 1000 characters; `errorLogRepo.save(...)` inserts it into the `ErrorLogs` table (columns `errorLogId`, `loggedAt`, `status`, `exceptionType`, `message`, `path`). The full class name such as `com.examply.springapp.exceptions.DriverDeletionException` is stored in `exceptionType`. No endpoint reads this table; you can view the rows in the H2 console at `/h2-console`.
- Lines 55-57: saving the log is wrapped in its own `try/catch`, so a failure to log can never hide the original error from the user.
- Line 58: the answer is `ResponseEntity.status(status).body(Map.of("message", ...))`, which Jackson writes as JSON: `{"message":"Failed to delete driver with ID: 3"}` with status 409. If the message were `null`, the text "Unexpected error" is used instead.

What the Angular app receives, by case: 409 `{"message": "Failed to delete driver with ID: 3"}`, 400 `{"message": "The request could not be processed. Please check the data and try again."}`, 500 `{"message": "Something went wrong. Please try again later."}`. The Angular `HttpErrorInterceptor` sends status 0 and 5xx to the error page. Line 60 closes the class.

Workflow, next: The backend is **completely started** and **Tomcat is waiting for requests on port 8080**. The H2 database is still empty: there are no users, drivers, requests or feedback until people register and the admin adds data through the app. The first program (the Spring Boot server) is finished. Now the **second program is started: the Angular app** in the `angularapp` folder (`ng serve --port 8081`). We begin with its project description file, `angularapp/package.json`, which is the first file of the next part.

Part 2 - Starting the frontend: from ng serve to the app shell

Now the second program. In a new terminal inside `angularapp`, `npm start` runs the Angular CLI dev server on port 8081.

1. **package.json** names the scripts and libraries; **angular.json** tells the CLI how to build; **tsconfig** tells the compiler how to treat TypeScript.
2. The browser loads **index.html**, which only contains `<app-root>`.
3. **main.ts** starts Angular with **AppModule**, which lists all components, imports the Angular modules and registers the HTTP interceptor.
4. The **router** (`app-routing.module.ts`) decides which component to show for each URL; the **AuthGuard** protects the private ones.
5. **AppComponent** is the permanent shell: it shows the right navigation bar and a `<router-outlet>` where the current page appears.
6. Shared helpers (constants, date/time and image helpers), the **models**, and the **services** that talk to the backend are the building blocks the pages use. They are explained here once, before the pages use them.

Step 42: package.json - the npm project file [Vaishalini - Customer Journey & Shared Infra]

CONFIG

angularapp/package.json

What this file is	The package manifest of the Angular front end. npm (Node Package Manager) is the tool that downloads JavaScript libraries; it reads this file to learn the project name, which libraries (called dependencies) the app needs, and which short commands (called scripts) you can run. The Angular side of DriveU starts here, just as the Spring Boot side started with <code>pom.xml</code> : nothing in this file is run by the browser, it only decides which tools and libraries exist in the <code>node_modules</code> folder.
Who / what triggers it	You, in a terminal inside the <code>angularapp</code> folder: <code>npm start</code> (after a one-time <code>npm install</code>). <code>npm</code> looks up the script called <code>start</code> in this file and runs the command written next to it.
What it uses / calls	The npm registry (to download the libraries into <code>node_modules</code>) and, through the scripts, the Angular command-line tool <code>ng</code> that comes from <code>@angular/cli</code> .
Size	40 lines

Lines 1-3 | Opening brace, project name and version

```
1 | {
2 |   "name": "angularapp",
3 |   "version": "0.0.0",
```

The file is **JSON**: a list of `"key": value` pairs inside curly braces. `"name": "angularapp"` is the package name (it also matches the project name in `angular.json`). `"version": "0.0.0"` is just a placeholder because this app is never published as a library.

Lines 4-10 | scripts - the commands you can run

```
4 |   "scripts": {
5 |     "ng": "ng",
6 |     "start": "ng serve --port 8081 --disable-host-check --host=0.0.0.0",
7 |     "build": "ng build",
8 |     "watch": "ng build --watch --configuration development",
9 |     "test": "ng test"
10 |   },
```

Each entry is a nickname for a terminal command. You run them as `npm run <name>` (for `start` and `test` the word `run` can be left out).

- `ng`: lets you type `npm run ng -- <anything>` to reach the Angular CLI.
- `start`: **the command used to launch the front end**. `ng serve` compiles the app and starts a small development web server. `--port 8081` makes it listen on port 8081 (the Spring Boot back end uses 8080). `--disable-host-check` turns off the safety check that rejects requests whose host name is not `localhost` (needed when the app is opened through a cloud workspace URL). `--host=0.0.0.0` makes the server reachable

from other machines and not only from your own computer.

- **build:** `ng build` creates the optimised production files in `dist/angularapp`.
- **watch:** builds with the `development` configuration and rebuilds whenever a file changes, without starting a server.
- **test:** `ng test` runs the unit tests (`*.spec.ts` files) in a Chrome browser using Karma and Jasmine.

Line 11 | private

```
11 | "private": true,
```

`"private": true` is a safety switch that stops anyone from accidentally publishing this project to the public npm registry.

Lines 12-23 | dependencies - libraries the running app needs

```
12 | "dependencies": {
13 |   "@angular/animations": "^16.2.0",
14 |   "@angular/common": "^16.2.0",
15 |   "@angular/compiler": "^16.2.0",
16 |   "@angular/core": "^16.2.0",
17 |   "@angular/forms": "^16.2.0",
18 |   "@angular/platform-browser": "^16.2.0",
19 |   "@angular/platform-browser-dynamic": "^16.2.0",
20 |   "@angular/router": "^16.2.0",
21 |   "rxjs": "~7.8.0",
22 |   "tslib": "^2.3.0",
23 |   "zone.js": "~0.13.0",
```

These libraries are bundled into the JavaScript that the browser downloads. The numbers are **semantic versions** (major.minor.patch). `^16.2.0` means "16.2.0 or any newer 16.x version", while `~7.8.0` means "7.8.x only", which is stricter.

- `@angular/core`: the heart of Angular (components, dependency injection, change detection, the `@Component` and `@NgModule` decorators).
- `@angular/common`: common directives such as `*ngIf` and `*ngFor`, pipes such as `async`, and the `HttpClient` used to call the Spring Boot REST API.
- `@angular/compiler`: turns the HTML templates into JavaScript in the browser when needed (the `platformBrowserDynamic` bootstrap in `main.ts` uses it).
- `@angular/forms`: template-driven (`ngModel`) and reactive (`FormGroup`, `FormControl`) forms, used on login, signup and the request pages.
- `@angular/platform-browser` and `@angular/platform-browser-dynamic`: the code that runs Angular inside a web browser and starts the application (`main.ts`).
- `@angular/router`: the routing system that shows a different page component for each URL (`app-routing.module.ts`).
- `@angular/animations`: Angular animation support. It is installed, but no file in this project imports the animations module, so it is unused.
- `rxjs`: the **Observable** library. Every HTTP call and the login state in this app are Observables.
- `tslib`: small helper functions that the TypeScript compiler shares instead of copying them into every file (works with `importHelpers` in `tsconfig.json`).
- `zone.js`: the library that lets Angular notice when asynchronous work (a click, a timer, an HTTP response) has finished so it can refresh the screen.

Lines 24-25 | puppeteer and end of dependencies

```
24 |   "puppeteer": "^19.7.5"
25 | },
```

`puppeteer` is a library that controls a headless Chrome browser from code. The application code never imports it; only `karma.conf.js` (the test configuration) does `require(puppeteer)`, because Puppeteer downloads a Chrome browser that the Karma tests can run in without opening a window (headless). It really belongs in `devDependencies`, but the project template lists it under `dependencies`. Line 25 closes the `dependencies` object.

Lines 26-29 | devDependencies - build tools

```
26 | "devDependencies": {
27 |   "@angular-devkit/build-angular": "^16.2.16",
28 |   "@angular/cli": "^16.2.16",
29 |   "@angular/compiler-cli": "^16.2.0",
```

The next group is only needed **while developing**, not inside the browser bundle. `@angular-devkit/build-angular` contains the **builders** (webpack based compiler, dev server, Karma test runner) that `angular.json` refers to. `@angular/cli` provides the `ng` command. `@angular/compiler-cli` is the ahead-of-time compiler (`ngc`) that checks

and compiles the templates and TypeScript.

Lines 30-37 | devDependencies - testing tools

```
30 |     "@types/jasmine": "~4.3.0",
31 |     "jasmine-core": "~4.6.0",
32 |     "karma": "~6.4.0",
33 |     "karma-chrome-launcher": "~3.2.0",
34 |     "karma-coverage": "~2.2.0",
35 |     "karma-jasmine": "~5.1.0",
36 |     "karma-spec-reporter": "^0.0.36",
37 |     "karma-jasmine-html-reporter": "~2.1.0",
```

`jasmine-core` and `@types/jasmine` are the test framework (`describe`, `it`, `expect`) and its TypeScript types. `karma` is the test runner that opens a browser and runs the tests; `karma-chrome-launcher` starts Chrome for it; `karma-jasmine` connects Karma to Jasmine; `karma-coverage` measures how much code the tests touch; `karma-spec-reporter` and `karma-jasmine-html-reporter` print the test results in the terminal and in the browser page.

Lines 38-40 | TypeScript and end of file

```
38 |     "typescript": "~5.1.3"
39 |   }
40 | }
```

`typescript ~5.1.3` is the compiler that turns the `.ts` files into plain JavaScript (Angular 16 supports TypeScript 4.9 to 5.1). Lines 39-40 close the `devDependencies` object and the whole JSON file.

Workflow, next: `npm runs ng serve --port 8081 ...` The Angular CLI now needs to know how to build this particular project, so it opens the workspace configuration file `angular.json`.

Step 43: angular.json - the Angular workspace and build settings [Vaishalini - Customer Journey & Shared Infra]

CONFIG

angularapp/angular.json

What this file is	The Angular CLI workspace file . It describes the project <code>angularapp</code> and, for each task the CLI can perform (build, serve, test, ...), which builder program to use and with what options. When <code>ng serve</code> starts, this file tells it which file is the entry point, which index page to use, which global style sheet to add, which assets to copy and which TypeScript config to compile with.
Who / what triggers it	The Angular CLI reads it every time you run <code>ng serve</code> , <code>ng build</code> or <code>ng test</code> , i.e. immediately after <code>package.json</code> launches <code>npm start</code> .
What it uses / calls	The builders in <code>@angular-devkit/build-angular</code> (<code>browser</code> , <code>dev-server</code> , <code>extract-i18n</code> , <code>karma</code>), <code>tsconfig.app.json</code> , <code>src/index.html</code> , <code>src/main.ts</code> , <code>src/styles.css</code> , <code>src/assets</code> , <code>src/favicon.ico</code> , and for tests <code>tsconfig.spec.json</code> and <code>karma.conf.js</code> .
Size	107 lines

Lines 1-4 | Schema, version and workspace root

```
1 | {
2 |   "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
3 |   "version": 1,
4 |   "newProjectRoot": "projects",
```

`$schema` points to the JSON schema that editors use to give auto-complete and error hints. `version: 1` is the workspace file format. `newProjectRoot: "projects"` is only the folder where `ng generate application` would put additional projects; DriveU has just one project, so it is not used.

Lines 5-12 | The project angularapp

```
5 |     "projects": {
6 |       "angularapp": {
7 |         "projectType": "application",
8 |         "schematics": {},
9 |         "root": "",
10 |        "sourceRoot": "src",
11 |        "prefix": "app",
12 |        "architect": {
```

`projects` lists the projects in the workspace; here only `angularapp`. `projectType: "application"` (not a library). `schematics` holds code-generation defaults (empty = use the standard ones). `root: ""` means the project sits at

the workspace root, `sourceRoot: "src"` is where its source code lives, and `prefix: "app"` is the default start of component selectors, which is why the root element is `app-root` and other components are named `app-login`, `app-adminnav` and so on. `architect` (line 12) starts the list of **targets** (tasks) of this project.

Lines 13-18 | build target - builder and entry files

```
13 |         "build": {
14 |             "builder": "@angular-devkit/build-angular:browser",
15 |             "options": {
16 |                 "outputPath": "dist/angularapp",
17 |                 "index": "src/index.html",
18 |                 "main": "src/main.ts",
```

The `build` target uses the `browser` builder, which bundles everything into JavaScript for a normal web browser. Its `options` begin here:

- `outputPath`: where the finished files go when you run `ng build (dist/angularapp)`. `ng serve` keeps them in memory instead.
- `index`: the HTML page the browser opens first, `src/index.html`.
- `main`: **the first TypeScript file to execute**, `src/main.ts`. The compiler starts there and follows every `import` to discover all other files.

Lines 19-21 | polyfills

```
19 |         "polyfills": [
20 |             "zone.js"
21 |         ],
```

`"polyfills": ["zone.js"]` tells the builder to load the `zone.js` package before the application. This is the modern (Angular 15+) way: a list of package names instead of a `polyfills.ts` file. Because of this line the old `src/polyfills.ts` file is **not used** (see that file later).

Line 22 | tsConfig

```
22 |         "tsConfig": "tsconfig.app.json",
```

`tsConfig: "tsconfig.app.json"` is the TypeScript configuration used for the app build (which files to compile and with which rules).

Lines 23-26 | assets

```
23 |         "assets": [
24 |             "src/favicon.ico",
25 |             "src/assets"
26 |         ],
```

Files and folders that are copied unchanged into the output so the browser can request them by URL: `src/favicon.ico` (the browser-tab icon) and everything in `src/assets`. DriveU keeps `images/background.webp` (used by the login background in `styles.css`) and `images/homepage.webp` there.

Lines 27-31 | styles, scripts and end of options

```
27 |         "styles": [
28 |             "src/styles.css"
29 |         ],
30 |         "scripts": []
31 |     },
```

`styles`: global CSS files added to every page. Only `src/styles.css` is listed, so that is the file with the shared look (navbar, buttons, tables, modals, auth pages). `scripts: []` means no extra non-Angular JavaScript files. Line 31 closes `options`.

Lines 32-47 | configurations: production

```
32 |         "configurations": {
33 |             "production": {
34 |                 "budgets": [
35 |                     {
36 |                         "type": "initial",
37 |                         "maximumWarning": "500kb",
38 |                         "maximumError": "1mb"
39 |                     },
40 |                     {
41 |                         "type": "anyComponentStyle",
42 |                         "maximumWarning": "2kb",
43 |                         "maximumError": "4kb"
44 |                     }
45 |                 ],
46 |                 "outputHashing": "all"
47 |             },
```

A **configuration** is a named set of option overrides applied on top of **options**. **production** adds **budgets** (size limits that raise a warning or an error at build time): the first bundle loaded must stay under 500 kB (warning) and 1 MB (error), and the CSS of any single component under 2 kB (warning) and 4 kB (error). **outputHashing: "all"** puts a content hash in every output file name (for example `main.3f9a1c.js`) so browsers never use an outdated cached copy after a new release.

Lines 48-58 | configurations: development and default

```
48 |         "development": {
49 |             "buildOptimizer": false,
50 |             "optimization": false,
51 |             "vendorChunk": true,
52 |             "extractLicenses": false,
53 |             "sourceMap": true,
54 |             "namedChunks": true
55 |         },
56 |     },
57 |     "defaultConfiguration": "production"
58 | },
```

development switches the optimisations off so building is fast and errors are easy to read: **buildOptimizer** and **optimization** false (no minifying), **vendorChunk** true (library code in its own file, rebuilt less often), **extractLicenses** false, **sourceMap** true (lets the browser debugger show the original `.ts` files) and **namedChunks** true (readable file names). **defaultConfiguration: "production"** means a plain `ng build` uses production; `ng serve` explicitly asks for **development** below.

Lines 59-70 | serve target - the development server

```
59 |     "serve": {
60 |         "builder": "@angular-devkit/build-angular:dev-server",
61 |         "configurations": {
62 |             "production": {
63 |                 "browserTarget": "angularapp:build:production"
64 |             },
65 |             "development": {
66 |                 "browserTarget": "angularapp:build:development"
67 |             }
68 |         },
69 |         "defaultConfiguration": "development"
70 |     },
```

The **serve** target uses the **dev-server** builder, which builds the app in memory, serves it over HTTP and refreshes the browser when you save a file. Each configuration just forwards to a build configuration through **browserTarget**: **production** uses `angularapp:build:production`, **development** uses `angularapp:build:development`. **defaultConfiguration: "development"** is why `npm start` gives a fast, debuggable build. The port (8081) and host come from the command line in `package.json`.

Lines 71-76 | extract-i18n target

```
71 |     "extract-i18n": {
72 |         "builder": "@angular-devkit/build-angular:extract-i18n",
73 |         "options": {
74 |             "browserTarget": "angularapp:build"
75 |         }
76 |     },
```

Used for translating text into other languages (`ng extract-i18n`). It simply points to the build target. DriveU is English only, so this target is never used.

Lines 77-95 | test target - Karma unit tests

```
77 |     "test": {
78 |         "builder": "@angular-devkit/build-angular:karma",
79 |         "options": {
80 |             "polyfills": [
81 |                 "zone.js",
82 |                 "zone.js/testing"
83 |             ],
84 |             "tsConfig": "tsconfig.spec.json",
85 |             "karmaConfig": "karma.conf.js",
86 |             "assets": [
87 |                 "src/favicon.ico",
88 |                 "src/assets"
89 |             ],
90 |             "styles": [
91 |                 "src/styles.css"
92 |             ],
93 |             "scripts": []
94 |         }
95 |     }
```

Used by `ng test`. The `karma` builder compiles the `*.spec.ts` files and runs them in Chrome. `polyfills` here has two entries, `zone.js` and `zone.js/testing` (extra helpers for async tests). `tsConfig` is `tsconfig.spec.json` (it includes the spec files and the Jasmine types), `karmaConfig` is `karma.conf.js` (browser and reporter settings), and `assets` and `styles` repeat the same files as the build so tests see the same CSS.

Lines 96-98 | Closing of the architect and project blocks

```
96 |     }  
97 |   }  
98 | },
```

These closing braces end `architect`, the `angularapp` project and the `projects` list.

Line 99 | defaultProject

```
99 |   "defaultProject": "angularapp",
```

`defaultProject` names the project the CLI uses when you do not pass one. (In Angular 16 this key is deprecated and ignored, but it does no harm.)

Lines 100-107 | CLI settings

```
100 |     "cli": {  
101 |       "analytics": "false",  
102 |       "cache": {  
103 |         "enabled": false  
104 |       }  
105 |     }  
106 |   }  
107 | }
```

`analytics: "false"` stops the CLI from asking whether it may collect usage statistics. `cache.enabled: false` disables the CLI disk cache so every build is done from scratch (slower, but it avoids stale-cache surprises in a shared workspace). The last lines close the JSON object.

Workflow, next: The CLI now knows the entry file is `src/main.ts`, the TypeScript settings are in `tsconfig.app.json`, and the base compiler options come from the file it extends. It reads `tsconfig.json` first for the shared compiler rules.

Step 44: tsconfig.json - base TypeScript compiler options [Vaishalini - Customer Journey & Shared Infra]

angularapp/tsconfig.json

CONFIG

What this file is	The base TypeScript configuration for the whole project. Browsers cannot run TypeScript (<code>.ts</code>), only JavaScript, so a compiler converts it. This file says how strictly to check the code and which version of JavaScript to produce. <code>tsconfig.app.json</code> and <code>tsconfig.spec.json</code> both extend it and add their own details.
Who / what triggers it	The Angular builder passes the configuration to the TypeScript compiler on every build; it is loaded because <code>tsconfig.app.json</code> says <code>extends: "../tsconfig.json"</code> .
What it uses / calls	The TypeScript compiler and the helper library <code>tslib</code> . Nothing from the application code is imported here.
Size	20 lines

Lines 1-3 | Comment and compileOnSave

```
1 | /* To learn more about this file see: https://angular.io/config/tsconfig. */  
2 | {  
3 |   "compileOnSave": false,
```

Line 1 is a comment (this file allows comments, unlike plain JSON). `compileOnSave: false` is an old Visual Studio setting that disables compiling when a file is saved; the Angular CLI handles compiling itself.

Lines 4-9 | Paths, output and source maps

```
4 |   "compilerOptions": {  
5 |     "baseUrl": "./",  
6 |     "outDir": "./dist/out-tsc",  
7 |     "sourceMap": true,  
8 |     "declaration": false,  
9 |     "downlevelIteration": true,
```

`baseUrl: "./"` lets imports be written relative to the project folder. `outDir` is where compiled JavaScript would go if the compiler were run alone (the CLI builder ignores it). `sourceMap: true` creates map files so that the browser debugger can show the original TypeScript. `declaration: false` means no `.d.ts` type declaration files are created (only needed when publishing a library). `downlevelIteration: true` makes `for...of` loops and the spread syntax `[...x]` work correctly on arrays, strings and Sets even when the compile target is an older JavaScript version.

Line 10 | experimentalDecorators

```
10 | "experimentalDecorators": true,
```

Angular is built on **decorators**, the `@Component(...)`, `@NgModule(...)`, `@Injectable(...)` lines that put extra information on a class. TypeScript treats them as an experimental feature, so this switch **must be on** or none of the Angular files would compile.

Lines 11-12 | Module resolution and helpers

```
11 | "moduleResolution": "node",
12 | "importHelpers": true,
```

`moduleResolution: "node"` finds imports such as `@angular/core` by searching `node_modules` the same way Node.js does. `importHelpers: true` imports shared helper functions from `tslib` (listed in `package.json`) instead of repeating them in every compiled file, which keeps the bundle smaller.

Lines 13-14 | Target and module format

```
13 | "target": "es2015",
14 | "module": "es2020",
```

`target: "es2015"` sets the JavaScript level of the compiled output (classes and arrow functions are kept). `module: "es2020"` keeps the `import/export` syntax so that the bundler (webpack) can analyse it and remove unused code (tree-shaking).

Lines 15-20 | lib

```
15 | "lib": [
16 |     "es2018",
17 |     "dom"
18 | ]
19 | }
20 | }
```

`lib` lists the built-in type definitions that TypeScript may assume exist: `es2018` (modern JavaScript features such as `Promise` and `Object.entries`) and `dom` (browser objects such as `document`, `localStorage`, `atob`, `HTMLImageElement` used in this project). Lines 19-20 close the file. Note that the file does not enable `strict` mode, which is why the code can contain things like `userId?: number` without the compiler complaining about every possible `undefined`.

Workflow, next: The base rules are known. The app-specific compiler file extends them, so next is `tsconfig.app.json`.

Step 45: tsconfig.app.json - TypeScript settings for the app build [Vaishalini - Customer Journey & Shared Infra]

angularapp/tsconfig.app.json

CONFIG

What this file is	A small file that extends <code>tsconfig.json</code> and adds what is special for building the real application (as opposed to the tests): which file is the starting point and which type definitions are loaded.
Who / what triggers it	<code>angular.json</code> names it in the build option <code>"tsConfig": "tsconfig.app.json"</code> , so the builder reads it for <code>ng serve</code> and <code>ng build</code> .
What it uses / calls	<code>tsconfig.json</code> (the file it extends) and <code>src/main.ts</code> (the start file).
Size	14 lines

Lines 1-3 | Comment and extends

```
1 | /* To learn more about this file see: https://angular.io/config/tsconfig. */
2 | {
3 |   "extends": "../tsconfig.json",
```

`extends: "../tsconfig.json"` copies every option from the base file first, then the settings below override or add to them.

Lines 4-7 | compilerOptions overrides

```
4 | "compilerOptions": {
5 |   "outDir": "./out-tsc/app",
6 |   "types": []
7 | },
```

`outDir` changes the compiler output folder to `./out-tsc/app`. `types: []` means "do not automatically load any global type packages from `node_modules/@types`". This keeps the Jasmine test types (`describe`, `it`) out of the app code; the test config (`tsconfig.spec.json`) loads them instead.

Lines 8-10 | files - the entry point

```
8 | "files": [
9 |   "src/main.ts"
10 | ],
```

`files: ["src/main.ts"]` tells the compiler where to start. It compiles `main.ts` and then automatically every file reachable through `import` statements: `AppModule`, all components, services and models. A file that nobody imports, such as `polyfills.ts`, is therefore not part of the app build.

Lines 11-14 | include

```
11 | "include": [
12 |   "src/**/*.d.ts"
13 | ]
14 | }
```

`include: ["src/**/*.d.ts"]` additionally adds any hand-written type declaration files (`*.d.ts`). `**` means "in any sub-folder". The spec files (`*.spec.ts`) are deliberately left out here.

Workflow, next: TypeScript is configured. The CLI compiles the code starting from `main.ts`, builds the bundles and starts the server on port 8081. When you open `http://localhost:8081` the browser asks the server for the page, and the server answers with `src/index.html`.

Step 46: index.html - the single HTML page [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/index.html

FRONTEND

What this file is	The only real HTML page of the whole application. Angular builds a single-page application (SPA) : the browser loads this one page once, and afterwards JavaScript swaps the content inside it when you move between "pages" (login, home, drivers...). This file is almost empty; it only provides the hosting element <code>app-root</code> .
Who / what triggers it	The browser requests <code>/</code> from the dev server on port 8081 and receives this file. The dev server also answers every unknown path (for example <code>/home</code>) with this same file, so refreshing the browser on any route still works.
What it uses / calls	The compiled bundles, which the Angular CLI adds automatically as <code>script</code> tags at the end of the page while serving (runtime, polyfills, vendor, main) and the <code>styles.css</code> link. It also uses <code>favicon.ico</code> and the custom element name <code>app-root</code> (the selector of <code>AppComponent</code>).
Size	13 lines

Lines 1-2 | Doctype and html element

```
1 | <!doctype html>
2 | <html lang="en">
```

`<!doctype html>` switches the browser into modern standards mode. `<html lang="en">` declares the page language as English (helps screen readers and translation tools).

Lines 3-5 | head start, charset and title

```
3 | <head>
4 |   <meta charset="utf-8">
5 |   <title>DriveU</title>
```

`<meta charset="utf-8">` makes the browser read the text as UTF-8 so all characters display correctly. `<title>DriveU</title>` is the text shown on the browser tab.

Lines 6-9 | Base href, viewport and favicon

```
6 | <base href="/">
7 | <meta name="viewport" content="width=device-width, initial-scale=1">
8 | <link rel="icon" type="image/x-icon" href="favicon.ico">
9 | </head>
```

`<base href="/">` is essential for the Angular router: it says that all relative URLs and all routes start at the site root, so `/login` and `/home` resolve correctly. The `viewport` meta tag makes the page scale to the device width on phones (with the media queries in `styles.css`). The `link rel="icon"` line sets the tab icon to `favicon.ico`, which `angular.json` copies from `src/`.

Lines 10-13 | body and the app-root element

```
10 | <body>
11 |   <app-root></app-root>
12 | </body>
13 | </html>
```

`<app-root></app-root>` is **not a standard HTML tag**; it is the selector declared in `AppComponent` (`selector: 'app-root'`). When Angular starts it finds this element and replaces its (empty) content with the output of `app.component.html`: the navbar and the router outlet. Everything the user ever sees is rendered inside this one element. Lines 12-13 close `body` and `html`.

Workflow, next: The browser has shown an empty `app-root`, downloaded the JavaScript bundles and started running them. The very first line of our own code that runs is `src/main.ts`.

Step 47: main.ts - starts the Angular application

[Vaishalini - Customer Journey & Shared Infra]

angularapp/src/main.ts

FRONTEND

What this file is	The first TypeScript file that executes in the browser (named in <code>angular.json</code> as <code>main</code>). It has just one job: boot (bootstrap) Angular with the root module <code>AppModule</code> . It is the Angular counterpart of <code>SpringappApplication.main</code> on the back end.
Who / what triggers it	The browser runs the <code>main</code> bundle after the polyfills (<code>zone.js</code>) have loaded.
What it uses / calls	<code>platformBrowserDynamic</code> from <code>@angular/platform-browser-dynamic</code> and the <code>AppModule</code> class from <code>src/app/app.module.ts</code> .
Size	7 lines

Line 1 | Import the browser platform

```
1 | import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
```

`platformBrowserDynamic` is a function that creates the **platform**, Angular's object that knows how to run an application inside a web browser. "Dynamic" means the templates are compiled in the browser at start-up when they were not already compiled during the build.

Line 3 | Import the root module

```
3 | import { AppModule } from './app/app.module';
```

`AppModule` is the **root module**: the class (decorated with `@NgModule`) that lists every component, imported module and service provider of the app. Importing it also makes the compiler pull in all the other files.

Lines 6-7 | Bootstrap and error logging

```
6 | platformBrowserDynamic().bootstrapModule(AppModule)
7 |   .catch(err => console.error(err));
```

`platformBrowserDynamic().bootstrapModule(AppModule)` creates the platform, builds the dependency injection system from `AppModule`, creates the component named in `bootstrap: [AppComponent]` and puts it into the `app-root` element of `index.html`. It returns a **Promise** (a value that will arrive later). `.catch(err => console.error(err))` runs only if start-up fails (for example a typo in a template or a missing provider) and prints the error in the browser console. If everything works, nothing is printed.

Workflow, next: Angular has been asked to start `AppModule`. Before that it needs the browser polyfills; the CLI loaded `zone.js` through `angular.json`. The old-style polyfill file is `src/polyfills.ts`, which we check next to see whether it is actually involved, and then we continue to `AppModule`.

Step 48: polyfills.ts - legacy polyfill file (not used by the build) [Vaishalini - Customer Journey & Shared Infra]

FRONTEND

angularapp/src/polyfills.ts

What this file is	In older Angular versions (up to 14) this file loaded <code>zone.js</code> and optional polyfills (small scripts that add modern browser features to old browsers) before the application started. Honest note: in this project the file is a leftover. <code>angular.json</code> does not contain the text <code>polyfills.ts</code> anywhere; the build and test options use the new array form <code>"polyfills": ["zone.js"]</code> instead. <code>tsconfig.app.json</code> lists only <code>main.ts</code> as an entry file and nothing imports <code>polyfills.ts</code> , so nothing in it is compiled or executed. Almost all of the file is comments, with a single active line (line 58).
Who / what triggers it	Nothing. It is not referenced by <code>angular.json</code> , <code>tsconfig.app.json</code> or any <code>import</code> . It stays in the folder because older Angular project templates created it automatically.
What it uses / calls	If it were loaded it would import <code>zone.js/dist/zone</code> . The same job is really done today by the <code>"polyfills": ["zone.js"]</code> entry in <code>angular.json</code> .
Size	63 lines

Lines 1-15 | File description comment

```
1 | /**
2 |  * This file includes polyfills needed by Angular and is loaded before the app.
3 |  * You can add your own extra polyfills to this file.
4 |  *
5 |  * This file is divided into 2 sections:
6 |  *   1. Browser polyfills. These are applied before loading ZoneJS and are sorted by browsers.
7 |  *   2. Application imports. Files imported after ZoneJS that should be loaded before your main
8 |  *      file.
9 |  *
10 |  * The current setup is for so-called "evergreen" browsers; the last versions of browsers that
11 |  * automatically update themselves. This includes Safari >= 10, Chrome >= 55 (including Opera),
12 |  * Edge >= 13 on the desktop, and iOS 10 and Chrome on mobile.
13 |  *
14 |  * Learn more in https://angular.io/guide/browser-support
15 |  */
```

A big comment that explains the idea: polyfills are loaded **before** the app; section 1 is for browser polyfills (before ZoneJS), section 2 for application imports. It mentions that the setup targets "evergreen" browsers (Chrome, Edge, Safari that update themselves), so no polyfills for old browsers such as Internet Explorer are needed. It is only documentation and produces no code.

Lines 17-22 | Section 1 and the classlist polyfill

```
17 | /*****
18 |  * BROWSER POLYFILLS
19 |  */
20 |
21 | /** IE10 and IE11 requires the following for NgClass support on SVG elements */
22 | // import 'classlist.js'; // Run `npm install --save classlist.js`.
```

The banner marks "BROWSER POLYFILLS". The commented-out `import 'classlist.js'` would add support for `ngClass` on SVG elements in Internet Explorer 10/11. It is switched off (starts with `//`) and Angular 16 does not support Internet Explorer anyway.

Lines 24-29 | Web animations polyfill (commented out)

```
24 | /**
25 |  * Web Animations `@angular/platform-browser/animations`
26 |  * Only required if AnimationBuilder is used within the application and using IE/Edge or Safari.
27 |  * Standard animation support in Angular DOES NOT require any polyfills (as of Angular 6.0).
28 |  */
29 | // import 'web-animations-js'; // Run `npm install --save web-animations-js`.
```

Another disabled optional import: `web-animations-js`. It would only matter if the app used Angular's `AnimationBuilder` on old IE, Edge or Safari. The comment itself says that normal Angular animations need no polyfill since Angular 6, and this app does not use Angular animations at all.

Lines 31-53 | Zone flags explanation (comment)

```
31 | /**
32 |  * By default, zone.js will patch all possible macroTask and DomEvents
33 |  * user can disable parts of macroTask/DomEvents patch by setting following flags
34 |  * because those flags need to be set before `zone.js` being loaded, and webpack
35 |  * will put import in the top of bundle, so user need to create a separate file
36 |  * in this directory (for example: zone-flags.ts), and put the following flags
37 |  * into that file, and then add the following code before importing zone.js.
38 |  * import './zone-flags';
39 |  *
40 |  * The flags allowed in zone-flags.ts are listed here.
41 |  *
42 |  * The following flags will work for all browsers.
43 |  *
44 |  * (window as any).__Zone_disable_requestAnimationFrame = true; // disable patch requestAnimationFrame
45 |  * (window as any).__Zone_disable_on_property = true; // disable patch onProperty such as onclick
46 |  * (window as any).__zone_symbol__UNPATCHED_EVENTS = ['scroll', 'mousemove']; // disable patch specified eventNames
47 |  *
48 |  * in IE/Edge developer tools, the addEventListener will also be wrapped by zone.js
49 |  * with the following flag, it will bypass `zone.js` patch for IE/Edge
50 |  *
51 |  * (window as any).__Zone_enable_cross_context_check = true;
52 |  *
53 | */
```

A long comment about advanced **zone.js flags**. By default zone.js wraps (patches) almost all browser asynchronous APIs: timers, `requestAnimationFrame`, and DOM event listeners. A developer can switch parts of this off by setting global variables such as `__Zone_disable_requestAnimationFrame` **before** zone.js loads, which is why the comment recommends putting them in a separate `zone-flags.ts` file imported first. The examples (lines 44-46 and 51) are only text. DriveU sets none of them, so zone.js runs with its default behaviour.

Lines 55-58 | Zone JS import - the only active line

```
55 | /*****
56 |  * Zone JS is required by default for Angular itself.
57 |  */
58 | import 'zone.js/dist/zone'; // Included with Angular CLI.
```

The banner says zone.js is required by Angular. **Line 58**, `import 'zone.js/dist/zone';` is the only executable statement in the file. It loads zone.js, which lets Angular know when asynchronous work (clicks, HTTP responses, timers) has finished so that it can update the screen automatically. Newer zone.js versions (0.13, the one in `package.json`) want the shorter name `zone.js`, which is exactly what `angular.json` uses, so this old path is another sign that the file is a leftover.

Lines 61-63 | Application imports section

```
61 | /*****
62 |  * APPLICATION IMPORTS
63 |  */
```

The last banner "APPLICATION IMPORTS" is the place where, in old projects, you would import your own files that must run before `main.ts`. Nothing is imported here.

Workflow, next: Zone.js really comes in through `angular.json`, so the browser is ready to run Angular. `main.ts` is bootstrapping `AppModule`, and the next piece is the place where the app is told how to reach the server: `src/environments/environment.ts`, which the services read through `apiconfig.ts`.

Step 49: environment.ts - environment settings [Vaishalini]

- Customer Journey & Shared Infra]

angularapp/src/environments/environment.ts

FRONTEND

What this file is	A tiny configuration file that holds settings that may differ between machines or between development and production . Here that is the address of the Spring Boot back end. The app never hard-codes <code>http://localhost:8080</code> in many places; it reads it from here.
Who / what triggers it	Imported by <code>src/apiconfig.ts</code> , which is imported by every service when the first component needs them.
What it uses / calls	Nothing; it only exports a constant object.
Size	5 lines

Lines 1-2 | The environment object and production flag

```
1 | export const environment = {  
2 |   production: false,
```

`export const environment = { ... }` exports an object that other files can `import`. `production: false` tells the code that this is a development setup (a conventional flag; no file in DriveU reads it).

Lines 3-4 | apiUrl

```
3 | // Replace "localhost" with the URL of your workspace port 8080 if needed.  
4 | apiUrl: 'http://localhost:8080'
```

Line 3 is a comment reminding the developer to replace `localhost` if the back end runs behind another host (for example a cloud workspace URL). `apiUrl: 'http://localhost:8080'` is the **base address of the Spring Boot REST API**. Every service builds its URLs as `apiUrl + "/api/..."`, for example `http://localhost:8080/api/login`.

Line 5 | End of object

```
5 | };
```

Closes the object and the statement. Note that this project has only this one environment file: there is no `environment.prod.ts` and `angular.json` has no `fileReplacements`, so even the production build uses the same URL.

Workflow, next: The base URL is defined here. It is passed on by the one-line helper `src/apiconfig.ts`.

Step 50: apiconfig.ts - the API base URL constant

[Vaishalini - Customer Journey & Shared Infra]

angularapp/src/apiconfig.ts

FRONTEND

What this file is	A one-line convenience wrapper that re-exports the back-end address as a simple constant called <code>apiUrl</code> . All five services import <code>apiUrl</code> from this file, so there is one single place to change the address.
Who / what triggers it	Imported at the top of <code>auth.service.ts</code> , <code>driver.service.ts</code> , <code>driver-request.service.ts</code> , <code>feedback.service.ts</code> and <code>ai.service.ts</code> .
What it uses / calls	<code>environment</code> from <code>./environments/environment</code> .
Size	4 lines

Line 1 | Import

```
1 | import { environment } from './environments/environment';
```

Brings in the `environment` object defined in `environment.ts`.

Lines 3-4 | Exported constant

```
3 | /** Base URL of the Spring Boot backend (runs on port 8080). */  
4 | export const apiUrl: string = environment.apiUrl;
```

The `/** ... */` line is a documentation comment (it says the back end runs on port 8080). `export const apiUrl: string = environment.apiUrl;` copies the text `'http://localhost:8080'` into a constant with the type `string`. Services build their URLs by joining it with a path (a template string that places `this.apiUrl` in front of `/api/driver`), which gives `http://localhost:8080/api/driver`.

Workflow, next: The address is ready for the services. Back in the boot sequence, `main.ts` asked Angular to start `AppModule`, so we now read the root module: `src/app/app.module.ts`.

Step 51: AppModule - the root module [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/app.module.ts

FRONTEND

What this file is	The root module of the Angular application. An NgModule is a box that tells Angular which components exist, which other modules they need (forms, HTTP, router) and which services should be provided. AppModule is the one module started by main.ts . It declares all 16 components of DriveU, imports the Angular feature modules, registers the HTTP error interceptor and names AppComponent as the component to start with.
Who / what triggers it	main.ts calls platformBrowserDynamic().bootstrapModule(AppModule) .
What it uses / calls	BrowserModule , FormsModule , ReactiveFormsModule , HttpClientModule , AppRoutingModule , all 16 component classes (15 page/nav components plus AppComponent) and HttpErrorInterceptor .
Size	56 lines

Lines 1-4 | Angular imports

```
1 | import { NgModule } from '@angular/core';
2 | import { BrowserModule } from '@angular/platform-browser';
3 | import { FormsModule, ReactiveFormsModule } from '@angular/forms';
4 | import { HTTP_INTERCEPTORS, HttpClientModule } from '@angular/common/http';
```

These lines import Angular's own building blocks.

- **NgModule**: the decorator that turns the class below into a module.
- **BrowserModule**: services every browser app needs (rendering, and the basic directives ***ngIf** and ***ngFor**). It may be imported only in the root module.
- **FormsModule**: template-driven forms (**[(ngModel)]**), used for example in search boxes.
- **ReactiveFormsModule**: reactive forms (**[formGroup]**, **formControlName**) used on the login, signup and request pages.
- **HttpClientModule**: provides **HttpClient**, the service that calls the REST API. **HTTP_INTERCEPTORS** is the token under which interceptors are registered.

Lines 6-23 | Imports of our own files

```
6 | import { AppRoutingModule } from './app-routing.module';
7 | import { AppComponent } from './app.component';
8 | import { AdminViewDriversComponent } from './components/admin-view-drivers/admin-view-drivers.component';
9 | import { AdminnavComponent } from './components/adminnav/adminnav.component';
10 | import { AdminviewfeedbackComponent } from './components/adminviewfeedback/adminviewfeedback.component';
11 | import { AdminviewrequestsComponent } from './components/adminviewrequests/adminviewrequests.component';
12 | import { CustomerRequestComponent } from './components/customer-request/customer-request.component';
13 | import { CustomernavComponent } from './components/customernav/customernav.component';
14 | import { CustomerpostfeedbackComponent } from './components/customerpostfeedback/customerpostfeedback.component';
15 | import { CustomerviewdriverComponent } from './components/customerviewdriver/customerviewdriver.component';
16 | import { CustomerviewfeedbackComponent } from './components/customerviewfeedback/customerviewfeedback.component';
17 | import { CustomerviewrequestedComponent } from './components/customerviewrequested/customerviewrequested.component';
18 | import { DriverManagementComponent } from './components/driver-management/driver-management.component';
19 | import { ErrorComponent } from './components/error/error.component';
20 | import { HomePageComponent } from './components/home-page/home-page.component';
21 | import { LoginComponent } from './components/login/login.component';
22 | import { SignupComponent } from './components/signup/signup.component';
23 | import { HttpErrorInterceptor } from './services/http-error.interceptor';
```

Line 6 imports **AppRoutingModule** (the route table). Line 7 imports **AppComponent**. Lines 8-22 import the 15 feature components, each from its own folder under **components/** (admin pages, customer pages, the two navigation bars, login, signup, home page and error page). Line 23 imports the **HttpErrorInterceptor** class from **services/**. An **import** only makes a class **available** in this file; the **@NgModule** below is what actually registers it with Angular.

Lines 25-43 | declarations - the components this module owns

```
25 | @NgModule({
26 |   declarations: [
27 |     AppComponent,
28 |     AdminViewDriversComponent,
29 |     AdminnavComponent,
30 |     AdminviewfeedbackComponent,
31 |     AdminviewrequestsComponent,
32 |     CustomerRequestComponent,
33 |     CustomernavComponent,
34 |     CustomerpostfeedbackComponent,
35 |     CustomerviewdriverComponent,
36 |     CustomerviewfeedbackComponent,
37 |     CustomerviewrequestedComponent,
38 |     DriverManagementComponent,
39 |     ErrorComponent,
40 |     HomePageComponent,
41 |     LoginComponent,
42 |     SignupComponent
43 | ],
```

`@NgModule({...})` is a **decorator**: it attaches configuration to the class `AppModule` at line 56. `declarations` lists every component, directive and pipe that belongs to this module. Angular only recognises a tag such as `<app-login>` or a route target if the component is declared in exactly one module. All 16 components of DriveU are declared here: `AppComponent` (the shell), `AdminViewDriversComponent`, `AdminnavComponent`, `AdminviewfeedbackComponent`, `AdminviewrequestsComponent`, `CustomerRequestComponent`, `CustomernavComponent`, `CustomerpostfeedbackComponent`, `CustomerviewdriverComponent`, `CustomerviewfeedbackComponent`, `CustomerviewrequestedComponent`, `DriverManagementComponent`, `ErrorComponent`, `HomePageComponent`, `LoginComponent` and `SignupComponent`. Because they all share this one module, the templates of all of them may use the directives of the modules imported below.

Lines 44-50 | imports - the modules this module uses

```
44 | imports: [
45 |   BrowserModule,
46 |   FormsModule,
47 |   ReactiveFormsModule,
48 |   HttpClientModule,
49 |   AppRoutingModule
50 | ],
```

Five modules are imported: `BrowserModule` (core browser support), `FormsModule` (`ngModel`), `ReactiveFormsModule` (`formGroup`), `HttpClientModule` (HTTP calls, needed by all services) and `AppRoutingModule` (our routes, which in turn exports `RouterModule` so that `<router-outlet>` and `routerLink` work in every template). The order matters little, but `AppRoutingModule` is conventionally last so its wildcard route does not shadow others.

Lines 51-53 | providers - registering the interceptor

```
51 | providers: [
52 |   { provide: HTTP_INTERCEPTORS, useClass: HttpErrorInterceptor, multi: true }
53 | ],
```

The services (`AuthService`, `DriverService`, ...) are marked `providedIn: 'root'` so they do not need to be listed here. The only provider is the interceptor: `{ provide: HTTP_INTERCEPTORS, useClass: HttpErrorInterceptor, multi: true }`. It says "when `HttpClient` builds its chain of interceptors, include an instance of `HttpErrorInterceptor`". `multi: true` means this adds to a list of interceptors rather than replacing one, so more interceptors could be added later. From now on **every** HTTP response passes through that class (see `http-error.interceptor.ts`).

Lines 54-56 | bootstrap and the module class

```
54 | bootstrap: [AppComponent]
55 | })
56 | export class AppModule { }
```

`bootstrap: [AppComponent]` names the **root component**: the one Angular creates and puts into the `app-root` tag of `index.html`. Line 56 is the actual (empty) class `AppModule`. A module class needs no code; all behaviour is in the decorator above it.

Workflow, next: Angular has now registered 16 components, forms, HTTP and the interceptor. To know what to show inside the page, it needs the route table that `AppModule` imported as `AppRoutingModule`: `src/app/app-routing.module.ts`.

Step 52: AppRoutingModuleModule - URLs, guards and roles

[Vanshika - Security (route guards) + shared]

angularapp/src/app/app-routing.module.ts

FRONTEND

What this file is	The routing table . In a single-page app the address bar still changes (<code>/login</code> , <code>/home</code> , <code>/driver-management/3</code>), but the browser never loads a new HTML page. Instead the Angular router compares the URL with this table and puts the matching component into the <code><router-outlet></code> in <code>app.component.html</code> . Here is also where each page is protected by the <code>AuthGuard</code> and tagged with the role (Admin or Customer) that may open it.
Who / what triggers it	Imported by <code>AppModule</code> . The router starts when the app boots, reads the browser URL (empty at first) and later on every <code>router.navigate</code> , <code>routerLink</code> click or manual change of the address bar.
What it uses / calls	<code>RouterModule</code> , <code>AuthGuard</code> , and all 14 page components that appear as route targets (all components except <code>AppComponent</code> and the two navigation bars, which are used as tags, not routes; <code>CustomerRequestComponent</code> and <code>DriverManagementComponent</code> are used by two routes each).
Size	52 lines

Lines 1-2 | Router imports

```
1 | import { NgModule } from '@angular/core';
2 | import { RouterModule, Routes } from '@angular/router';
```

`NgModule` for the decorator at the bottom. `RouterModule` provides the router itself and `Routes` is the type of the route array (a list of route objects).

Lines 4-17 | Component and guard imports

```
4 | import { AuthGuard } from '../components/authguard/authguard.guard';
5 | import { LoginComponent } from '../components/login/login.component';
6 | import { SignupComponent } from '../components/signup/signup.component';
7 | import { HomeComponent } from '../components/home-page/home-page.component';
8 | import { DriverManagementComponent } from '../components/driver-management/driver-management.component';
9 | import { AdminViewDriversComponent } from '../components/admin-view-drivers/admin-view-drivers.component';
10 | import { AdminviewrequestsComponent } from '../components/adminviewrequests/adminviewrequests.component';
11 | import { AdminviewfeedbackComponent } from '../components/adminviewfeedback/adminviewfeedback.component';
12 | import { CustomerviewdriverComponent } from '../components/customerviewdriver/customerviewdriver.component';
13 | import { CustomerRequestComponent } from '../components/customer-request/customer-request.component';
14 | import { CustomerviewrequestedComponent } from '../components/customerviewrequested/customerviewrequested.component';
15 | import { CustomerpostfeedbackComponent } from '../components/customerpostfeedback/customerpostfeedback.component';
16 | import { CustomerviewfeedbackComponent } from '../components/customerviewfeedback/customerviewfeedback.component';
17 | import { ErrorComponent } from '../components/error/error.component';
```

Line 4 imports `AuthGuard` (the class that decides whether a page may be opened). Lines 5-17 import every page component that will be a target of a route: login, signup, home, the five admin pages, the six customer pages and the error page. The two navigation bar components are missing on purpose: they are placed by `AppComponent` as tags, not through routes.

Lines 19-21 | The routes array and the empty-path redirect

```
19 | const routes: Routes = [
20 |   // The login page is the first page rendered when the application loads.
21 |   { path: '', redirectTo: 'login', pathMatch: 'full' },
```

`const routes: Routes = [ ... ]` is the table. The router checks entries **from top to bottom and uses the first match**. The first entry `{ path: '', redirectTo: 'login', pathMatch: 'full' }` handles the plain address `http://localhost:8081/`. `redirectTo: 'login'` sends the user to `/login`. `pathMatch: 'full'` is essential: it means "only when the **whole** URL is empty". Without it the empty path would be a prefix of every URL and everything would be redirected. This redirect is not guarded, which is why the login page is the first screen of the app.

Lines 22-23 | Public routes: login and signup

```
22 | { path: 'login', component: LoginComponent },
23 | { path: 'signup', component: SignupComponent },
```

`{ path: 'login', component: LoginComponent }` shows the login page at `/login`; `signup` shows the registration form. They have **no `canActivate`**, so anyone can open them without a token. These two pages need to be public because a user cannot have a token before logging in.

Lines 25-26 | Common route: home (Admin and Customer)

```
25 | // Common (Admin and Customer)
26 | { path: 'home', component: HomeComponent, canActivate: [AuthGuard] },
```

`home` shows `HomeComponent` and has `canActivate: [AuthGuard]` but no `data`. So the guard only checks "is the user logged in with a valid token?" and lets both roles in.

Lines 28-30 | Admin routes: driver management

```
28 | // Admin
29 | { path: 'driver-management', component: DriverManagementComponent, canActivate: [AuthGuard], data: { role: 'Admin' } },
30 | { path: 'driver-management/:driverId', component: DriverManagementComponent, canActivate: [AuthGuard], data: { role: 'Admin' } },
```

`driver-management` opens `DriverManagementComponent` in **add** mode. `driver-management/:driverId` opens the **same component** in **edit** mode: `:driverId` is a **route parameter** (a variable part of the URL, for example `/driver-management/3`) that the component reads with `ActivatedRoute`. Both routes use `canActivate: [AuthGuard]` and `data: { role: 'Admin' }`. The `data` object is a free place to store route-specific facts; here it holds the role the guard compares with the logged-in user's role.

Lines 31-33 | Admin routes: view drivers, requests, feedback

```
31 | { path: 'admin-view-drivers', component: AdminViewDriversComponent, canActivate: [AuthGuard], data: { role: 'Admin' } },
32 | { path: 'adminviewrequests', component: AdminviewrequestsComponent, canActivate: [AuthGuard], data: { role: 'Admin' } },
33 | { path: 'adminviewfeedback', component: AdminviewfeedbackComponent, canActivate: [AuthGuard], data: { role: 'Admin' } },
```

Three more Admin-only pages, each with the guard and `data: { role: 'Admin' }`: `admin-view-drivers` (the driver table), `adminviewrequests` (approve or reject trip requests, close trips) and `adminviewfeedback` (all feedback with the AI sentiment).

Lines 35-38 | Customer routes: drivers and the request form

```
35 | // Customer
36 | { path: 'customerviewdriver', component: CustomerviewdriverComponent, canActivate: [AuthGuard], data: { role: 'Customer' } },
37 | { path: 'customer-request/:driverId', component: CustomerRequestComponent, canActivate: [AuthGuard], data: { role: 'Customer' } },
38 | { path: 'customer-request/edit/:driverRequestId', component: CustomerRequestComponent, canActivate: [AuthGuard], data: { role: 'Customer' } },
```

Customer-only pages (`data: { role: 'Customer' }`). `customerviewdriver` lists the drivers (with AI search). `customer-request/:driverId` opens `CustomerRequestComponent` to book the driver whose id is in the URL. `customer-request/edit/:driverRequestId` opens the **same component** in edit mode for an existing request; since the router tries routes top to bottom, the more specific `edit/` pattern is a different shape (three segments) from the first (two segments), so there is no clash.

Lines 39-41 | Customer routes: requests and feedback

```
39 | { path: 'customerviewrequested', component: CustomerviewrequestedComponent, canActivate: [AuthGuard], data: { role: 'Customer' } },
40 | { path: 'customerpostfeedback', component: CustomerpostfeedbackComponent, canActivate: [AuthGuard], data: { role: 'Customer' } },
41 | { path: 'customerviewfeedback', component: CustomerviewfeedbackComponent, canActivate: [AuthGuard], data: { role: 'Customer' } },
```

`customerviewrequested` shows the customer's own requests (Trip End, payment), `customerpostfeedback` is the feedback form, and `customerviewfeedback` shows the customer's own feedback. All three use the guard with `role: 'Customer'`.

Lines 43-46 | Error page and wildcard

```
43 | // Error page
44 | { path: 'error', component: ErrorComponent },
45 | { path: '**', component: ErrorComponent }
46 | ];
```

`error` shows `ErrorComponent` without a guard (the interceptor sends the user there after a network or 5xx failure). The last route has a path made of two asterisk characters in a row, the **wildcard** pattern: it matches any URL that no earlier route matched (a mistyped address such as `/abc`), and also shows `ErrorComponent`. It must be last because routes are matched in order. Line 46 closes the array.

Lines 48-52 | The routing module

```
48 | @NgModule({
49 |   imports: [RouterModule.forRoot(routes)],
50 |   exports: [RouterModule]
51 | })
52 | export class AppRoutingModule { }
```

`RouterModule.forRoot(routes)` registers the table as the **root** router configuration (it may be called only once in an app; feature modules would use `forChild`). `exports: [RouterModule]` re-exports the router's directives (`router-outlet`, `routerLink`, `routerLinkActive`) so that every component declared in `AppModule` can use them in its template. The class `AppRoutingModule` itself is empty.

Workflow, next: The route table is ready. When the app starts, the router reads the URL `/`, matches the first entry and redirects to `/login`. But before any route is displayed, the shell component that holds the `<router-outlet>` must be created: `src/app/app.component.ts`.

Step 53: AppComponent - logic of the app shell [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/app.component.ts

FRONTEND

What this file is	The root component : the first component Angular creates (named in <code>bootstrap</code>). It is the permanent frame around every page. Its class decides which navigation bar to show depending on the role of the logged-in user: admin bar, customer bar or none (login and signup pages).
Who / what triggers it	Angular creates it right after <code>bootstrapModule</code> because <code>AppModule</code> has <code>bootstrap: [AppComponent]</code> , and puts it into the <code>app-root</code> tag.
What it uses / calls	<code>AuthService.userRole\$</code> (an Observable of the role), the RxJS <code>map</code> operator, and its template <code>app.component.html</code> .
Size	21 lines

Lines 1-4 | Imports

```
1 | import { Component } from '@angular/core';
2 | import { Observable } from 'rxjs';
3 | import { map } from 'rxjs/operators';
4 | import { AuthService } from '../services/auth.service';
```

`Component` is the decorator that makes a class a component. `Observable` is the RxJS type of a stream of values over time, used here for the two type declarations. `map` is an RxJS **operator**: a small function that transforms each value the stream produces. `AuthService` is the service that stores who is logged in.

Lines 6-10 | The @Component decorator

```
6 | @Component({
7 |   selector: 'app-root',
8 |   templateUrl: './app.component.html',
9 |   styleUrls: ['./app.component.css']
10 | })
```

A **component** is a piece of screen: a class (logic) plus an HTML template (view) plus CSS. The decorator configures it. `selector: 'app-root'` is the HTML tag name this component answers to (matches `index.html`). `templateUrl` points to the HTML file and `styleUrls` to the CSS file; the CSS file `app.component.css` is **empty (0 lines)**, so the shell has no styles of its own (everything comes from `styles.css`).

Lines 11-12 | Class and title

```
11 | export class AppComponent {
12 |   title = 'angularapp';
```

`export class AppComponent` is the class. `title = 'angularapp'` is a leftover from the Angular template (the `ng new` command creates it); it is not shown anywhere.

Lines 14-15 | Observable properties

```
14 | isAdmin$: Observable<boolean>;
15 | isCustomer$: Observable<boolean>;
```

Two properties declare **observables of true/false**. The `$` at the end is a naming convention meaning "this is an Observable". `isAdmin$` will emit `true` when the logged-in user is an admin; `isCustomer$` when a customer. The template reads them with the `async` pipe.

Lines 17-21 | Constructor and the two pipelines

```
17 |   constructor(private authService: AuthService) {
18 |       this.isAdmin$ = this.authService.userRole$.pipe(map((role) => (role || '').toLowerCase() === 'admin'));
19 |       this.isCustomer$ = this.authService.userRole$.pipe(map((role) => (role || '').toLowerCase() === 'customer'));
20 |   }
21 | }
```

`constructor(private authService: AuthService)` uses **dependency injection**: Angular sees the parameter type and hands in the single shared `AuthService` object. `private` also creates a class property automatically. Inside, `this.authService.userRole$.pipe(map(...))` builds a new stream from the role stream: each time the role changes, `(role || '').toLowerCase() === 'admin'` runs. The `|| ''` protects against `null` (when nobody is logged in), `.toLowerCase()` makes the comparison case-insensitive, and the result is a boolean. So:

- logged out: role is `null`, both streams emit `false`, no bar is shown.
- after an admin logs in: `userRole$` emits `'Admin'`, so `isAdmin$` emits `true`.
- after a customer logs in: `isCustomer$` emits `true`.
- after logout: `null` again, both `false`.

Line 21 closes the class. Because `userRole$` comes from a `BehaviorSubject`, the constructor immediately receives the role that was saved in `localStorage` (so a page refresh keeps the right bar).

Workflow, next: The component class is ready and its two streams are connected to the login state. Angular now renders its template: `src/app/app.component.html`.

Step 54: app.component.html - navbar choice and router outlet [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/app.component.html

FRONTEND

What this file is	The template of the root component, only three lines long. It shows at most one of the two navigation bars and always contains the router outlet , the placeholder where the page for the current URL appears. The matching style file <code>app.component.css</code> exists but is an empty 0-line file .
Who / what triggers it	Rendered by Angular as the view of <code>AppComponent</code> , once at start-up, and re-evaluated whenever the observables change.
What it uses / calls	<code>AdminnavComponent</code> (tag <code>app-adminnav</code>), <code>CustomernavComponent</code> (tag <code>app-customernav</code>), <code>RouterOutlet</code> and the <code>async</code> pipe and <code>*ngIf</code> from Angular's common module.
Size	3 lines

Line 1 | Admin navigation bar

```
1 | <app-adminnav *ngIf="isAdmin$ | async"></app-adminnav>
```

`<app-adminnav>` is the tag of the admin navigation bar component. `*ngIf="isAdmin$ | async"` inserts it into the page only when the expression is true. The `async` pipe (`|` is the pipe symbol) **subscribes** to the Observable `isAdmin$`, hands the latest value to `ngIf`, and unsubscribes automatically when the component is destroyed. So the admin bar appears the moment the role becomes Admin, and disappears on logout, without any manual code.

Line 2 | Customer navigation bar

```
2 | <app-customernav *ngIf="isCustomer$ | async"></app-customernav>
```

The same idea for customers: `<app-customernav>` is shown only when `isCustomer$ | async` is true. Because a user has exactly one role, at most one bar is visible. When nobody is logged in neither is rendered, which is why the login and signup pages look clean.

Line 3 | Router outlet

```
3 | <router-outlet></router-outlet>
```

`<router-outlet></router-outlet>` is a directive from the router. Angular puts here the component chosen by the route table: `LoginComponent` for `/login`, `HomePageComponent` for `/home` and so on. When the URL changes, the component in the outlet is replaced while the navigation bar above it stays.

Workflow, next: The shell is displayed: no bar yet (nobody is logged in), plus an empty router outlet. Next come the global style rules that decorate every element, `src/styles.css`. The router, started by the module, will fill the outlet right after the styles are described.

Step 55: styles.css - global styles for the whole app

[Vaishalini - Customer Journey & Shared Infra]

angularapp/src/styles.css

FRONTEND

What this file is	The global style sheet (listed in <code>angular.json</code> under <code>styles</code>). It is loaded once for the whole application and holds the look and feel that is shared by every page: colours (as CSS variables), the navigation bar, page cards, buttons, modal pop-ups, forms, tables, the search toolbar, small animations and the two authentication pages. The individual <code>*.component.css</code> files only add a few page-specific rules on top. It has 384 lines.
Who / what triggers it	The Angular CLI injects it as a <code>link</code> or <code>style</code> tag in <code>index.html</code> (because of <code>angular.json</code>), so the browser applies it before any component is shown.
What it uses / calls	Pure CSS. One image reference: <code>/assets/images/background.webp</code> (copied by the <code>assets</code> option of <code>angular.json</code>). The HTML class names are used in the component templates (for example <code>class="btn btn-primary"</code>).
Size	384 lines

Lines 1-3 | File banner comment

```
1 | /* =====
2 |    DriveU - global styles (shared look & feel for every page)
3 |    ===== */
```

A comment describing the file (CSS comments are written `/* ... */`). It has no visual effect.

Lines 4-20 | CSS variables in :root

```
4 | :root {
5 |   --blue: #0a6fe8;
6 |   --blue-dark: #0958bd;
7 |   --steel: #2f6f9f;
8 |   --table-head: #3b8de0;
9 |   --green: #28a745;
10 |  --red: #d9363e;
11 |  --orange: #f4b00c;
12 |  --grey: #6c757d;
13 |  --teal: #1796a8;
14 |  --text: #222;
15 |  --muted: #666;
16 |  --border: #e4e4e4;
17 |  --nav-from: #2e5d86;
18 |  --nav-mid: #7a8588;
19 |  --nav-to: #c3a25d;
20 | }
```

`:root` is the top element of the page (the `html` element). Names that start with two dashes are **custom properties (CSS variables)**; anywhere in the CSS `var(--blue)` reads the value. Defining colours once gives a consistent theme and lets you change a colour in one place.

- `--blue`, `--blue-dark`: main action colour and its darker hover version.
- `--steel`: steel-blue used for page titles. `--table-head`: lighter blue for table header rows.
- `--green`, `--red`, `--orange`, `--grey`, `--teal`: colours for success, danger, warning, secondary and info buttons.
- `--text`, `--muted`, `--border`: main text, secondary text and light border colours.
- `--nav-from`, `--nav-mid`, `--nav-to`: the three colour stops (blue, grey, gold) of the navigation bar gradient.

Lines 22-32 | Reset and base typography

```
22 | * { box-sizing: border-box; }
23 | html, body { margin: 0; padding: 0; }
24 | body {
25 |   font-family: Arial, Helvetica, sans-serif;
26 |   font-size: 14px;
27 |   color: var(--text);
28 |   background: #fcfcfc;
29 | }
30 | a { text-decoration: none; color: inherit; }
31 | h1, h2, h3, h4 { font-family: Arial, Helvetica, sans-serif; }
32 | button { font-family: inherit; }
```

*** { box-sizing: border-box; }** (the universal selector) changes how widths are counted: padding and border are **included** in an element's width, so a **width: 100%** input with padding does not overflow its container. **html, body** lose the default margin and padding so the page touches the window edges. **body** sets the default font (Arial), size 14 px, text colour **var(--text)** and a very light grey background **#fcfcfc**. **a** removes link underlines and makes links inherit the surrounding text colour. **h1-h4** use the same font family, and **button { font-family: inherit; }** makes buttons use the page font instead of the browser's default button font.

Lines 34-47 | Navbar container and brand

```
34 | /* ----- Navbar (adminnav + customernav) ----- */
35 | .navbar {
36 |   position: relative;
37 |   z-index: 50;
38 |   display: flex;
39 |   align-items: center;
40 |   justify-content: space-between;
41 |   height: 54px;
42 |   padding: 0 26px;
43 |   color: #fff;
44 |   background: linear-gradient(90deg, var(--nav-from) 0%, var(--nav-mid) 50%, var(--nav-to) 100%);
45 |   box-shadow: 0 2px 6px rgba(0, 0, 0, 0.25);
46 | }
47 | .navbar .brand { font-size: 23px; font-weight: 700; color: #fff; letter-spacing: 0.2px; }
```

The comment marks the section shared by **adminnav** and **customernav**. **.navbar** is a 54 px tall flex row (**display: flex**): **align-items: center** centres the items vertically, **justify-content: space-between** pushes the brand to the left and the menu to the right. **position: relative** plus **z-index: 50** keeps the bar (and its dropdowns) above page content. **padding: 0 26px** adds side spacing, **color: #fff** makes the text white. **background: linear-gradient(90deg, ...)** paints the bar with a horizontal gradient from blue (**--nav-from**) through grey to gold (**--nav-to**), and **box-shadow** gives a soft shadow below it. **.navbar .brand** is the "DriveU" logo text: 23 px, bold, white, with slight letter spacing.

Lines 48-56 | Navbar right side and user chip

```
48 | .navbar .nav-right { display: flex; align-items: center; gap: 22px; }
49 | .navbar .user-chip {
50 |   border: 1.5px solid #fff;
51 |   border-radius: 7px;
52 |   padding: 4px 12px;
53 |   font-size: 13px;
54 |   font-weight: 700;
55 |   white-space: nowrap;
56 | }
```

.nav-right is another flex row with a 22 px gap that holds the links, the user chip and the logout button. **.user-chip** is the small rounded box showing the logged-in username and role: a 1.5 px white border, 7 px rounded corners, bold 13 px text, and **white-space: nowrap** so the name never wraps onto two lines.

Lines 57-67 | Navbar links

```
57 | .navbar .nav-links { display: flex; gap: 4px; margin: 0; padding: 0; list-style: none; }
58 | .navbar .nav-links > li { position: relative; }
59 | .navbar .nav-links a,
60 | .navbar .nav-links .nav-label {
61 |   display: block;
62 |   padding: 18px 12px;
63 |   font-size: 13px;
64 |   color: #fff;
65 |   opacity: 0.92;
66 |   cursor: pointer;
67 | }
```

.nav-links is the menu list: a flex row with a small gap, and **list-style: none** plus zero margin and padding remove the bullet points of the **ul**. Each **li** gets **position: relative** so a dropdown can be positioned relative to it (line 58). The rule for **.nav-links a** and **.nav-links .nav-label** (both the real links and the non-link label of a dropdown) makes them blocks with 18 px top and bottom padding (so the clickable area fills the bar height), white 13 px text, a slightly faded **opacity: 0.92** and a pointer cursor.

Lines 68-70 | Navbar hover and active state

```
68 | .navbar .nav-links > li > a:hover,  
69 | .navbar .nav-links > li > a.active,  
70 | .navbar .nav-links .dropdown:hover > .nav-label { background: rgba(255, 255, 255, 0.18); opacity: 1; }
```

For a top-level link on hover, for a link that has the class **active** (the current page) and for the label of a dropdown while the mouse is over the dropdown, the background becomes a translucent white (**rgba(255, 255, 255, 0.18)**) and the opacity returns to 1, so the item lights up.

Lines 71-85 | Dropdown menu

```
71 | .navbar .dropdown-menu {  
72 |   display: none;  
73 |   position: absolute;  
74 |   top: 100%;  
75 |   left: 0;  
76 |   min-width: 140px;  
77 |   background: #fff;  
78 |   border-radius: 4px;  
79 |   box-shadow: 0 6px 18px rgba(0, 0, 0, 0.28);  
80 |   overflow: hidden;  
81 | }  
82 | .navbar .dropdown:hover .dropdown-menu,  
83 | .navbar .dropdown:focus-within .dropdown-menu { display: block; }  
84 | .navbar .dropdown-menu a { color: #222; padding: 11px 16px; font-size: 13px; opacity: 1; white-space: nowrap; }  
85 | .navbar .dropdown-menu a:hover { background: #0e4a8f; color: #fff; }
```

.dropdown-menu is the pop-up list under a menu item. It starts hidden (**display: none**), is placed with **position: absolute** right below the parent (**top: 100%; left: 0**), is at least 140 px wide, has a white background, 4 px rounded corners and a shadow, and **overflow: hidden** clips the hover colour to the rounded corners. The rule at lines 82-83 shows it (**display: block**) when the mouse is over the **.dropdown** or when something inside has keyboard focus (**:focus-within**), so it also works with the Tab key. Lines 84-85 style the links inside: dark text, 11 px by 16 px padding, no wrapping, and a dark-blue background with white text on hover.

Lines 86-96 | Logout button

```
86 | .navbar .logout-btn {  
87 |   background: #e53935;  
88 |   color: #fff;  
89 |   border: none;  
90 |   border-radius: 3px;  
91 |   padding: 8px 16px;  
92 |   font-size: 13px;  
93 |   font-weight: 700;  
94 |   cursor: pointer;  
95 | }  
96 | .navbar .logout-btn:hover { background: #c62828; }
```

.logout-btn is the red button at the right end of the bar: red background (**#e53935**), white bold text, no border, 3 px radius, pointer cursor. On hover (line 96) it becomes a darker red **#c62828**.

Lines 98-110 | Page layout and titles

```
98 | /* ----- Page layout ----- */  
99 | .page-container { max-width: 1040px; margin: 26px auto 44px; padding: 0 16px; }  
100 | .page-card {  
101 |   background: #fff;  
102 |   border-radius: 8px;  
103 |   box-shadow: 0 2px 10px rgba(0, 0, 0, 0.12);  
104 |   padding: 18px 20px 22px;  
105 | }  
106 | .page-title { margin: 6px 0 0; font-size: 26px; font-weight: 700; color: var(--text); }  
107 | .page-title.steel { color: var(--steel); }  
108 | .title-line { height: 2px; margin: 10px auto 18px; background: #2f6fb0; width: 86%; }  
109 | .empty-text { text-align: center; font-size: 15px; font-weight: 700; color: #222; padding: 24px 0 12px; }  
110 | .muted { color: var(--muted); }
```

.page-container centres the page content: maximum width 1040 px, **margin: 26px auto 44px** (the **auto** side margins centre it), plus 16 px side padding. **.page-card** is the white panel: white background, rounded corners, a soft shadow and inner padding. **.page-title** is the big 26 px bold heading and **.page-title.steel** (both classes on one element) changes its colour to steel blue. **.title-line** draws a thin 2 px blue line under the title, 86% wide and centred. **.empty-text** is the bold centred message shown when a list is empty (for example "No Data Found"). **.muted** makes text grey.

Lines 112-124 | Button base style and interaction

```
112 | /* ----- Buttons ----- */
113 | .btn {
114 |   border: none;
115 |   border-radius: 4px;
116 |   padding: 8px 16px;
117 |   font-size: 13px;
118 |   font-weight: 700;
119 |   color: #fff;
120 |   cursor: pointer;
121 |   transition: filter 0.15s, transform 0.1s;
122 | }
123 | .btn:hover:not(:disabled) { filter: brightness(0.92); }
124 | .btn:active:not(:disabled) { transform: translateY(1px); }
```

`.btn` is the shared button shape: no border, 4 px radius, 8 px by 16 px padding, 13 px bold white text and a pointer cursor. `transition: filter 0.15s, transform 0.1s` animates the two changes below smoothly.

`.btn:hover:not(:disabled)` darkens an enabled button with `filter: brightness(0.92);`

`.btn:active:not(:disabled)` moves it 1 px down while pressed so it feels like a real press. The `:not(:disabled)` part keeps disabled buttons still.

Lines 125-131 | Button colour variants

```
125 | .btn-primary { background: var(--blue); }
126 | .btn-success { background: var(--green); }
127 | .btn-danger { background: var(--red); }
128 | .btn-warning { background: var(--orange); color: #fff; }
129 | .btn-secondary { background: var(--grey); }
130 | .btn-info { background: var(--teal); }
131 | .btn-pill { border-radius: 999px; padding: 7px 16px; }
```

Each class only sets the background (the shape comes from `.btn`): `btn-primary` blue, `btn-success` green, `btn-danger` red, `btn-warning` orange, `btn-secondary` grey, `btn-info` teal, all through the CSS variables. Templates combine them, for example `class="btn btn-danger"`. `btn-pill` makes the button fully rounded (`border-radius: 999px`) with slightly different padding.

Lines 132-137 | Disabled buttons

```
132 | .btn:disabled {
133 |   background: #dcdcdc;
134 |   color: #a0a0a0;
135 |   cursor: not-allowed;
136 |   filter: none;
137 | }
```

`.btn:disabled` gives every disabled button a light grey background, grey text, a `not-allowed` cursor and cancels the hover filter. This is what the user sees when a form is invalid (for example a Submit button bound to `[disabled]="form.invalid"`).

Lines 139-151 | Modal backdrop

```
139 | /* ----- Modals ----- */
140 | .modal-backdrop {
141 |   position: fixed;
142 |   inset: 0;
143 |   z-index: 1000;
144 |   display: flex;
145 |   align-items: center;
146 |   justify-content: center;
147 |   padding: 16px;
148 |   background: rgba(0, 0, 0, 0.32);
149 |   -webkit-backdrop-filter: blur(3px);
150 |   backdrop-filter: blur(3px);
151 | }
```

A **modal** is a pop-up window over the page. `.modal-backdrop` is the dark layer behind it: `position: fixed; inset: 0` stretches it over the entire browser window (`inset: 0` is the shortcut for top, right, bottom and left all 0), `z-index: 1000` puts it above everything including the navbar, and `display: flex` with centring properties places the modal box in the middle. `background: rgba(0, 0, 0, 0.32)` dims the page, and `backdrop-filter: blur(3px)` (with the `-webkit-` version for Safari) blurs what is behind it.

Lines 152-165 | Modal box and its text

```
152 | .modal-box {
153 |   position: relative;
154 |   min-width: 300px;
155 |   max-width: 94vw;
156 |   max-height: 90vh;
157 |   overflow-y: auto;
158 |   padding: 26px 32px;
159 |   background: #fff;
160 |   border-radius: 8px;
161 |   text-align: center;
162 |   box-shadow: 0 10px 30px rgba(0, 0, 0, 0.35);
163 | }
164 | .modal-box h3 { margin: 0 0 14px; font-size: 18px; }
165 | .modal-box p { margin: 0 0 16px; font-size: 14px; }
```

`.modal-box` is the white dialog: at least 300 px wide but never wider than 94% of the screen (94vw) or taller than 90% (90vh), with `overflow-y: auto` so long content scrolls inside it. It has padding, a white background, rounded corners, centred text and a stronger shadow. `position: relative` lets the close button be placed inside it. The rules for `h3` (18 px heading) and `p` (14 px paragraph) inside the box set their spacing.

Lines 166-176 | Modal close button

```
166 | .modal-close {
167 |   position: absolute;
168 |   top: 10px;
169 |   right: 14px;
170 |   border: none;
171 |   background: none;
172 |   font-size: 18px;
173 |   font-weight: 700;
174 |   line-height: 1;
175 |   cursor: pointer;
176 | }
```

`.modal-close` is the small "x" in the top-right corner: absolutely positioned 10 px from the top and 14 px from the right, no border or background, bold 18 px, pointer cursor.

Lines 177-181 | Modal buttons and success box

```
177 | .modal-actions { display: flex; flex-direction: column; align-items: center; gap: 8px; }
178 | .modal-actions.row { flex-direction: row; justify-content: center; }
179 | .modal-actions .btn { min-width: 140px; border-radius: 999px; padding: 9px 18px; }
180 | .modal-box.success-box { border: 1px solid #6fa8e8; }
181 | .modal-box.success-box .btn { min-width: 110px; border-radius: 4px; }
```

`.modal-actions` stacks buttons vertically (`flex-direction: column`) and centres them with an 8 px gap; with the extra class `row` they sit side by side. Buttons inside are at least 140 px wide and pill-shaped.

`.modal-box.success-box` gives the "Request Submitted Successfully" style dialog a thin light-blue border, and its buttons are narrower (110 px) with only slightly rounded corners.

Lines 183-194 | Form card

```
183 | /* ----- Forms ----- */
184 | .form-card {
185 |   position: relative;
186 |   width: 430px;
187 |   max-width: 92vw;
188 |   margin: 34px auto;
189 |   padding: 24px 28px 28px;
190 |   background: #fafafa;
191 |   border: 1px solid var(--border);
192 |   border-radius: 6px;
193 |   box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
194 | }
```

`.form-card` is the centred card that holds a form (driver form, request form, feedback form): 430 px wide but never more than 92% of the screen, centred horizontally by `margin: 34px auto`, with padding, a light grey background (#fafafa), a thin border from `--border`, rounded corners and a faint shadow. `position: relative` anchors the back button.

Lines 195-214 | Form title and back button

```
195 | .form-card .form-title {
196 |   margin: 0 0 6px;
197 |   padding-bottom: 12px;
198 |   text-align: center;
199 |   font-size: 20px;
200 |   border-bottom: 1px solid #4a90e2;
201 | }
202 | .form-card .back-btn {
203 |   position: absolute;
204 |   top: -12px;
205 |   right: 16px;
206 |   padding: 5px 22px;
207 |   border: none;
208 |   border-radius: 3px;
209 |   background: #2196f3;
210 |   color: #fff;
211 |   font-size: 12px;
212 |   font-weight: 700;
213 |   cursor: pointer;
214 | }
```

`.form-card .form-title` is the heading at the top of the card: centred, 20 px, with a blue line (`border-bottom`) under it. `.form-card .back-btn` is the little blue "Back" tab: it is positioned absolutely half outside the top edge (`top: -12px`) at the right (`right: 16px`), blue background, white bold 12 px text.

Lines 215-232 | Form labels, inputs and error texts

```
215 | .form-label { display: block; margin: 15px 0 6px; font-size: 14px; color: #333; }
216 | .form-label .req { color: #333; }
217 | .form-control {
218 |   width: 100%;
219 |   padding: 11px 12px;
220 |   border: 1px solid transparent;
221 |   border-radius: 4px;
222 |   background: #efefef;
223 |   font-family: inherit;
224 |   font-size: 14px;
225 |   color: #222;
226 | }
227 | .form-control:focus { outline: none; border-color: #4a90e2; background: #f4f8ff; }
228 | textarea.form-control { min-height: 90px; resize: vertical; }
229 | .form-card .submit-row { margin-top: 18px; text-align: center; }
230 | .form-card .submit-row .btn { padding: 10px 30px; font-size: 14px; }
231 | .error-text { margin-top: 4px; font-size: 12px; color: #c0392b; }
232 | .form-error { margin: 14px 0 0; text-align: left; font-size: 12px; color: #c0392b; }
```

`.form-label` is a block label with spacing above and below; `.form-label .req` styles the span holding the asterisk that marks a required field (its colour is set to the same dark grey as the label, so it is not red). `.form-control` is the common input/select/textarea look: full width, padding, a transparent border (so nothing jumps on focus), light grey background, inherited font. On focus (line 227) the outline is removed and replaced by a blue border and a pale-blue background, which is the focus highlight. `textarea.form-control` makes text areas at least 90 px high and resizable only vertically. `.form-card .submit-row` centres the submit button area, and its `.btn` gets bigger padding and font. `.error-text` (12 px red) is the validation message under a field, and `.form-error` is a similar red message for a general form error, left-aligned.

Lines 234-248 | Tables

```
234 | /* ----- Tables ----- */
235 | .table-wrap { overflow-x: auto; }
236 | .data-table { width: 100%; border-collapse: collapse; font-size: 14px; }
237 | .data-table thead th {
238 |   padding: 11px 10px;
239 |   background: var(--table-head);
240 |   color: #fff;
241 |   font-size: 13px;
242 |   font-weight: 700;
243 |   text-transform: uppercase;
244 |   text-align: center;
245 | }
246 | .data-table tbody td { padding: 12px 10px; text-align: center; border-bottom: 1px solid #ececfc; }
247 | .data-table tbody tr:nth-child(even) { background: #fafafa; }
248 | .data-table .empty-row td { padding: 14px; font-size: 14px; font-style: italic; color: #555; background: #fff; }
```

`.table-wrap` allows horizontal scrolling when a wide table does not fit on a phone. `.data-table` is the shared table: full width, `border-collapse: collapse` (cells share borders), 14 px text. The header cells (`thead th`) get a blue background (`--table-head`), white bold uppercase 13 px centred text. Body cells (`tbody td`) are centred with padding and a light grey bottom border. `tbody tr:nth-child(even)` gives every second row a faint grey background (**zebra stripes**). `.empty-row td` styles the "no records" row: italic, grey and white background.

Lines 250-260 | Toolbar for search and filters

```
250 | /* ----- Toolbar (search / filter) ----- */
251 | .toolbar { display: flex; align-items: center; justify-content: space-between; gap: 12px; flex-wrap: wrap; margin-bottom: 16px; font-size: 13px; font-weight: 700; }
252 | .toolbar input, .toolbar select, .search-input {
253 |   padding: 8px 11px;
254 |   border: 1px solid #cfcfcf;
255 |   border-radius: 4px;
256 |   background: #fff;
257 |   font-family: inherit;
258 |   font-size: 13px;
259 | }
260 | .toolbar select { background: #eee; }
```

`.toolbar` is the row above a table that holds the search box and filters: a flex row with a gap, `justify-content: space-between` and `flex-wrap: wrap` so items drop to a new line on small screens, bold 13 px text. The rule at lines 252-259 gives the toolbar's `input` and `select` elements and the `.search-input` class the same compact look: padding, thin grey border, white background, inherited font. `.toolbar select` then makes dropdowns light grey.

Lines 262-274 | Animations

```
262 | /* ----- Animations ----- */
263 | @keyframes blink { 0%, 100% { opacity: 1; } 50% { opacity: 0.25; } }
264 | @keyframes spin { to { transform: rotate(360deg); } }
265 | .blink { animation: blink 1.1s ease-in-out infinite; }
266 | .spinner {
267 |   width: 30px;
268 |   height: 30px;
269 |   margin: 8px auto;
270 |   border: 3px solid #e3e3e3;
271 |   border-top-color: #3b8de0;
272 |   border-radius: 50%;
273 |   animation: spin 0.9s linear infinite;
274 | }
```

`@keyframes blink` defines an animation that goes from full opacity to 25% and back (0% and 100% are 1, 50% is 0.25); `@keyframes spin` rotates an element a full 360 degrees. `.blink` applies the blink animation (1.1 s, smooth easing, repeating forever); the admin request table uses it on the status cell of a request that is in the "Trip End" stage so that it blinks and draws the admin's attention. `.spinner` (used while a payment or an AI summary is loading) is a 30 px circle with a light grey border whose top segment is blue (`border-top-color`); with `border-radius: 50%` and the spin animation (0.9 s, constant speed, infinite) it becomes the rotating loading wheel.

Lines 276-280 | Media query for small screens (navbar)

```
276 | @media (max-width: 720px) {
277 |   .navbar { height: auto; flex-wrap: wrap; padding: 8px 12px; gap: 6px; }
278 |   .navbar .nav-right { flex-wrap: wrap; gap: 8px; }
279 |   .navbar .nav-links a, .navbar .nav-links .nav-label { padding: 8px 8px; }
280 | }
```

`@media (max-width: 720px)` applies the rules inside only when the window is 720 px wide or narrower, e.g. a phone. The navbar then grows to its content (`height: auto`), may wrap onto several lines (`flex-wrap: wrap`) with smaller padding and gaps, the right-hand menu may wrap too, and menu links get less padding. This makes the menu usable on a narrow screen.

Lines 282-299 | Auth page and card

```
282 | /* ----- Auth pages (login + signup) ----- */
283 | .auth-page {
284 |   min-height: 100vh;
285 |   display: flex;
286 |   align-items: center;
287 |   justify-content: center;
288 |   padding: 20px 12px;
289 |   background: #f0f0f0;
290 | }
291 | .auth-card {
292 |   display: flex;
293 |   width: min(860px, 100%);
294 |   min-height: 500px;
295 |   background: #fff;
296 |   border-radius: 8px;
297 |   overflow: hidden;
298 |   box-shadow: 0 6px 22px rgba(0, 0, 0, 0.16);
299 | }
```

This section is for the login and signup pages. `.auth-page` fills the whole window height (`min-height: 100vh`), centres its card both ways with flexbox and has a light grey background. `.auth-card` is the white rounded card: a flex row (picture on the left, form on the right), width `min(860px, 100%)` (the smaller of 860 px and the screen width), at least 500 px high, `overflow: hidden` to clip the image to the rounded corners and a larger shadow.

Lines 300-315 | Auth image panel

```
300 | .auth-image {
301 |   position: relative;
302 |   flex: 0 0 47%;
303 |   display: flex;
304 |   align-items: center;
305 |   justify-content: center;
306 |   background: #2a9fb5 url('/assets/images/background.webp') center / cover no-repeat;
307 | }
308 | .auth-image .auth-heading {
309 |   margin: 0;
310 |   font-size: 38px;
311 |   font-weight: 700;
312 |   color: #fff;
313 |   text-shadow: 0 2px 6px rgba(0, 0, 0, 0.55);
314 |   letter-spacing: 0.3px;
315 | }
```

`.auth-image` is the left picture panel: `flex: 0 0 47%` fixes it at 47% of the card (it will neither grow nor shrink), centres its content, and sets the background to a teal colour with `/assets/images/background.webp` centred, scaled with `cover` to fill the panel and not repeated. `.auth-heading` is the big white 38 px "DriveU" text over the picture with a dark text shadow so it is readable on any image.

Lines 316-324 | Auth form panel and form box

```
316 | .auth-panel { flex: 1; display: flex; align-items: center; justify-content: center; padding: 24px 20px; background: #f
      |   dfdfd; }
317 | .auth-form {
318 |   width: 100%;
319 |   max-width: 340px;
320 |   padding: 20px 24px 18px;
321 |   background: #fff;
322 |   border-radius: 4px;
323 |   box-shadow: 0 1px 8px rgba(0, 0, 0, 0.15);
324 | }
```

`.auth-panel` fills the remaining width (`flex: 1`) and centres the form. `.auth-form` is the white form box: full width but at most 340 px, with padding, slightly rounded corners and a small shadow.

Lines 325-335 | Auth heading and subtitle

```
325 | .auth-form h2 { margin: 2px 0 12px; text-align: center; font-size: 22px; font-weight: 700; }
326 | .auth-form .subtitle {
327 |   margin: -4px 0 12px;
328 |   padding-bottom: 7px;
329 |   text-align: center;
330 |   font-size: 10px;
331 |   font-weight: 700;
332 |   letter-spacing: 0.7px;
333 |   color: #1f8ad6;
334 |   border-bottom: 1px solid #2f8fd8;
335 | }
```

`.auth-form h2` is the form title (22 px, bold, centred). `.auth-form .subtitle` is the small blue spaced-out line below it (for example "JOIN DRIVEU TODAY" on the signup page; 10 px bold text with `letter-spacing`) with a blue underline. The negative top margin (`-4px`) tucks it closer to the title.

Lines 336-349 | Auth inputs

```
336 | .auth-form .auth-field { margin-bottom: 8px; }
337 | .auth-form .auth-input,
338 | .auth-form .auth-select {
339 |   width: 100%;
340 |   padding: 9px 2px;
341 |   border: none;
342 |   border-bottom: 1px solid #ccc;
343 |   background: transparent;
344 |   font-family: inherit;
345 |   font-size: 14px;
346 |   color: #222;
347 | }
348 | .auth-form .auth-input:focus,
349 | .auth-form .auth-select:focus { outline: none; border-bottom-color: #0a6fe8; }
```

`.auth-field` adds spacing between fields. `.auth-input` and `.auth-select` use a modern **underline-only** look: no border except `border-bottom: 1px solid #ccc`, a transparent background and full width. When focused (lines 348-349) the outline is removed and the underline becomes blue (`#0a6fe8`).

Lines 350-360 | Password field and eye button

```
350 | .auth-form .password-wrap { position: relative; }
351 | .auth-form .eye-btn {
352 |   position: absolute;
353 |   right: 0;
354 |   top: 5px;
355 |   border: none;
356 |   background: none;
357 |   font-size: 16px;
358 |   cursor: pointer;
359 |   color: #444;
360 | }
```

`.password-wrap` is `position: relative` so that `.eye-btn` (the show/hide password icon) can be placed with `position: absolute` at the right edge (`right: 0`) of the input. The button has no border or background, a 16 px icon and a pointer cursor.

Lines 361-378 | Auth messages, submit button and links

```
361 | .auth-form .auth-error { margin: 3px 0 4px; font-size: 11.5px; color: #c00; }
362 | .auth-form .auth-submit {
363 |   display: block;
364 |   width: 80%;
365 |   margin: 16px auto 10px;
366 |   padding: 10px 0;
367 |   border: none;
368 |   border-radius: 4px;
369 |   background: #0a6fe8;
370 |   color: #fff;
371 |   font-size: 14px;
372 |   font-weight: 700;
373 |   cursor: pointer;
374 | }
375 | .auth-form .auth-submit:hover { background: #0958bd; }
376 | .auth-form .auth-link { margin: 6px 0 2px; text-align: center; font-size: 12px; color: #333; }
377 | .auth-form .auth-link a { color: #0a6fe8; }
378 | .auth-form .server-error { margin: 8px 0 -6px; text-align: center; font-size: 11.5px; color: #c00; }
```

`.auth-error` is the small red (11.5 px) validation message under a field. `.auth-submit` is the main button: displayed as a block, 80% wide, centred by `margin: 16px auto 10px`, blue background, white bold text, 4 px radius; on hover (line 375) it turns the darker blue `#0958bd`. `.auth-link` is the line "Don't have an account? Sign Up" (or "Already have an account? Log In"), and `.auth-link a` makes the link blue. `.server-error` is the red centred message shown when the back end rejects the request (for example "Invalid credentials" or "Email already exists"), with a negative bottom margin so it sits close to the button.

Lines 380-384 | Media query for small screens (auth pages)

```
380 | @media (max-width: 640px) {
381 |   .auth-card { flex-direction: column; }
382 |   .auth-image { flex: 0 0 150px; }
383 |   .auth-form { max-width: 320px; }
384 | }
```

`@media (max-width: 640px)`: on phones the auth card stacks vertically (`flex-direction: column`), the image shrinks to a 150 px-high strip (`flex: 0 0 150px` now controls the height because the direction is vertical) and the form box is limited to 320 px wide.

Workflow, next: The global look is in place. Next come the small shared helper files that many components use, starting with the constant lists in `src/app/utils/constants.ts`.

Step 56: constants.ts - fixed lists and regular expressions [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/utils/constants.ts

FRONTEND

What this file is

A shared file of **constants**: lists of allowed values (vehicle types, statuses, categories) and two **regular expressions** for validating e-mail and phone numbers. Keeping them in one file means the drop-down lists and the validation rules are written once and reused by several components, so spelling stays identical everywhere (important because the back end stores and compares these exact strings).

Who / what triggers it	Imported by the components that need them: <code>driver-management</code> (vehicle types), <code>admin-view-drivers</code> (driver statuses), <code>adminviewrequests</code> (request statuses and stages), <code>customerpostfeedback</code> and <code>adminviewfeedback</code> (feedback categories), <code>adminviewfeedback</code> (sentiments), <code>login</code> and <code>signup</code> (e-mail pattern) and <code>signup</code> (mobile pattern).
What it uses / calls	Nothing. It only exports values.
Size	21 lines

Line 1 | VEHICLE_TYPES

```
1 | export const VEHICLE_TYPES: string[] = ['Sedan', 'SUV', 'Hatchback', 'Truck', 'Van', 'Bike'];
```

`export const VEHICLE_TYPES: string[] = [...]` exports a constant array of strings (`string[]` is the TypeScript type "array of text"). The six values fill the vehicle-type drop-down when an admin adds or edits a driver.

Line 3 | DRIVER_STATUSES

```
3 | export const DRIVER_STATUSES: string[] = ['Active', 'Inactive', 'On Leave'];
```

The three allowed values of a driver's `availabilityStatus` (`Active`, `Inactive`, `On Leave`), used for the status filter and the status drop-down in the admin driver list.

Line 5 | REQUEST_STATUSES

```
5 | export const REQUEST_STATUSES: string[] = ['Pending', 'Approved', 'Rejected', 'Trip End', 'Closed'];
```

All five states a trip request can have, in life-cycle order: `Pending` (created by the customer), `Approved` or `Rejected` (decided by the admin), `Trip End` (the customer finished the trip) and `Closed` (the admin completed it). The admin request page uses this list for its status filter.

Lines 7-8 | REQUEST_STAGES

```
7 | /** Stages shown in the "Request Progression" modal (a rejected request stops after "Pending"). */
8 | export const REQUEST_STAGES: string[] = ['Pending', 'Approved', 'Trip End', 'Closed'];
```

A shorter list for the **Request Progression** pop-up that shows how far a request has moved. `Rejected` is left out because a rejected request stops after `Pending`, as the comment on line 7 explains.

Lines 10-16 | FEEDBACK_CATEGORIES

```
10 | export const FEEDBACK_CATEGORIES: string[] = [
11 |   'Driver Performance',
12 |   'Service Experience',
13 |   'Punctuality',
14 |   'Vehicle Condition',
15 |   'Other'
16 | ];
```

The five categories a customer can pick when posting feedback (`Driver Performance`, `Service Experience`, `Punctuality`, `Vehicle Condition`, `Other`). The same list is used by the admin feedback page as a filter. It is written across several lines only for readability.

Line 18 | SENTIMENTS

```
18 | export const SENTIMENTS: string[] = ['Positive', 'Neutral', 'Negative'];
```

The three values the back end's AI analysis assigns to feedback (`Positive`, `Neutral`, `Negative`). The admin feedback page uses them for filtering.

Lines 20-21 | EMAIL_REGEX and MOBILE_REGEX

```
20 | export const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
21 | export const MOBILE_REGEX = /^\d{10}$/;
```

A **regular expression** (regex) is a pattern that describes the shape of a text; `/.../` marks one in JavaScript, `^` means "start of text" and `$` means "end of text".

- `EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/`: one or more characters that are not whitespace and not `@`, then `@`, again such characters, then a dot, and again such characters. `user@example.com` matches; `user@example` (no dot part), `user example@x.com` (space) and `@x.com` do not. It is a simple shape check, not a full e-mail validation.
- `MOBILE_REGEX = /^\d{10}$/`: `\d` is a digit and `{10}` means exactly ten of them, so `9876543210` matches but `98765` or `98765abcde` does not.

The login and signup forms pass these to `Validators.pattern(...)`.

Workflow, next: The shared lists are defined. Next is another helper file, `src/app/utils/format.ts`, which converts dates, times and durations.

Step 57: format.ts - date, time and duration helpers

[Prajan - Payment & Trip End (date/duration helpers)]

angularapp/src/app/utils/format.ts

FRONTEND

What this file is	A collection of small pure functions (they only compute a result from their input, with no side effects) that turn dates, times and duration texts into the exact formats that the back end and the screens need. The back end uses LocalDate (yyyy-MM-dd) and LocalTime (HH:mm:ss), the HTML date/time inputs use similar strings, and the screens show dates as dd/MM/yyyy. These helpers hide the differences so components stay simple. The trip-end payment calculation also relies on the duration helpers.
Who / what triggers it	Called by customer-request , customerviewrequested , adminviewrequests , customerpostfeedback , customerviewfeedback and adminviewfeedback (components and their templates).
What it uses / calls	Only built-in JavaScript objects (Date , Array , String , Math).
Size	86 lines

Line 1 | pad - add a leading zero

```
1 | const pad = (n: number): string => (n < 10 ? '0' + n : String(n));
```

const pad = (n: number): string => (n < 10 ? '0' + n : String(n)); is an **arrow function** that makes every number at least two digits. It is not exported, so it is private to this file. **pad(5)** returns **'05'**, **pad(12)** returns **'12'**. The **condition ? a : b** form is the ternary operator ("if ... then a, else b").

Lines 3-15 | toDateString - anything to "YYYY-MM-DD"

```
3 | /** Accepts "YYYY-MM-DD", [y, m, d], a Date or null and returns "YYYY-MM-DD" ('' when empty). */
4 | export function toDateString(value: any): string {
5 |   if (value === null || value === undefined || value === '') {
6 |     return '';
7 |   }
8 |   if (Array.isArray(value)) {
9 |     return `${value[0]}-${pad(value[1])}-${pad(value[2])}`;
10 |   }
11 |   if (value instanceof Date) {
12 |     return `${value.getFullYear()}-${pad(value.getMonth() + 1)}-${pad(value.getDate())}`;
13 |   }
14 |   return String(value).substring(0, 10);
15 | }
```

Accepts a text, an array [**year**, **month**, **day**], a **Date** object, **null** or **undefined** (**value: any**) and always returns text in **yyyy-MM-dd** form, or an empty string. Step by step:

- If the value is **null**, **undefined** or empty, return **''**.
- If it is an array (some Java JSON settings send dates as **[2024, 7, 2]**), join the parts: **[2024, 7, 2]** gives **'2024-07-02'** (the month and day are padded).
- If it is a **Date**, read the **local** year, month and day. **getMonth()** counts from 0, so 1 is added: **new Date(2024, 6, 2)** gives **'2024-07-02'**. Local getters are used on purpose; **toISOString()** would convert to UTC and could shift the date by one day.
- Otherwise convert to text and keep the first 10 characters: **'2024-07-02'** stays the same and **'2024-07-02T10:15:00'** becomes **'2024-07-02'**.

Examples: **toDateString(null)** is **''**; **toDateString('2024-07-02')** is **'2024-07-02'**. Spring Boot normally sends dates as ISO text, so the array branch is a defensive extra.

Lines 17-40 | toTimeString - anything to "HH:mm"

```
17 | /** Accepts "HH:mm[:ss]", [h, m, s], a Date or null and returns "HH:mm" ('' when empty). */
18 | export function toTimeString(value: any, withSeconds = false): string {
19 |   if (value === null || value === undefined || value === '') {
20 |     return '';
21 |   }
22 |   let h = 0;
23 |   let m = 0;
24 |   let s = 0;
25 |   if (Array.isArray(value)) {
26 |     h = value[0] || 0;
27 |     m = value[1] || 0;
28 |     s = value[2] || 0;
29 |   } else if (value instanceof Date) {
30 |     h = value.getHours();
31 |     m = value.getMinutes();
32 |     s = value.getSeconds();
33 |   } else {
34 |     const parts = String(value).split(':');
35 |     h = Number(parts[0]) || 0;
36 |     m = Number(parts[1]) || 0;
37 |     s = Number(parts[2]) || 0;
38 |   }
39 |   return withSeconds ? `${pad(h)}:${pad(m)}:${pad(s)}` : `${pad(h)}:${pad(m)}`;
40 | }
```

Converts a time into **HH:mm** (or **HH:mm:ss** when the second argument **withSeconds** is **true**; the default = **false** makes the argument optional). The three variables **h**, **m**, **s** start at 0 and are filled depending on the input type:

- Array **[h, m, s]**: each part, with **|| 0** replacing a missing part (**undefined**) by 0. **[9, 5]** gives **'09:05'**.
- **Date**: **getHours()**, **getMinutes()**, **getSeconds()**. **new Date(2024, 0, 1, 17, 45, 10)** gives **'17:45'**, or **'17:45:10'** with seconds.
- Text: split at colons. **'09:30'** gives **'09:30'**; **'09:30:15'** with seconds gives **'09:30:15'**; **'9'** gives **'09:00'**. **Number(x) || 0** turns anything that is not a number (such as **'abc'**) into 0, so **'abc'** gives **'00:00'**.

Empty input returns **''**. The final line chooses the output with a ternary and a **template string** (text between backticks where **\${...}** inserts a value) after padding each part.

Lines 42-50 | toDisplayDate - "YYYY-MM-DD" to "DD/MM/YYYY"

```
42 | /** "YYYY-MM-DD" -> "DD/MM/YYYY" */
43 | export function toDisplayDate(value: any): string {
44 |   const iso = toDateString(value);
45 |   if (!iso) {
46 |     return '';
47 |   }
48 |   const [y, m, d] = iso.split('-');
49 |   return `${d}/${m}/${y}`;
50 | }
```

Used wherever a date is shown to the user. It first normalises the input with **toDateString**, returns **''** for empty input, then uses **array destructuring**: **const [y, m, d] = iso.split('-')** splits **'2024-07-02'** into **y = '2024'**, **m = '07'**, **d = '02'**, and rebuilds them in the other order. Examples: **toDisplayDate('2024-07-02')** is **'02/07/2024'**; **toDisplayDate([2024, 7, 2])** is **'02/07/2024'**; **toDisplayDate(null)** is **''**.

Lines 52-54 | todayString - today as "YYYY-MM-DD"

```
52 | export function todayString(): string {
53 |   return toDateString(new Date());
54 | }
```

Returns the current date by calling **toDateString(new Date())**. **new Date()** is "right now" on the user's computer. If today is 6 October 2026 the result is **'2026-10-06'**. The request form and the feedback form use it to stamp **requestDate** and **date**.

Lines 56-66 | combineDateTime - date plus time to a Date

```
56 | /** Combines a trip date and time slot into a Date (local time). */
57 | export function combineDateTime(date: any, time: any): Date | null {
58 |   const d = toDateString(date);
59 |   const t = toTimeString(time, true);
60 |   if (!d || !t) {
61 |     return null;
62 |   }
63 |   const [y, mo, day] = d.split('-').map(Number);
64 |   const [h, mi, s] = t.split(':').map(Number);
65 |   return new Date(y, mo - 1, day, h, mi, s);
66 | }
```

Used to find when a trip started. It normalises both inputs (**toTimeString(time, true)** always includes seconds), returns **null** if either is missing, and otherwise builds a **Date**. **d.split('-').map(Number)** splits the date text and converts every piece to a number; the same is done for the time. **new Date(y, mo - 1, day, h, mi, s)** creates a

local-time date; the month is `mo - 1` because JavaScript months start at 0. Example: `combineDateTime('2024-07-02', '09:30')` returns 2 July 2024, 09:30:00 local time. `combineDateTime(null, '09:30')` returns `null`. The caller subtracts this start time from "now" to learn how long the trip took.

Lines 68-80 | `parseDurationToMinutes` - text to minutes

```
68 | /** Parses strings such as "3 hours", "1.5 hrs", "45 minutes" into minutes (0 when unknown). */
69 | export function parseDurationToMinutes(text: string | null | undefined): number {
70 |   if (!text) {
71 |     return 0;
72 |   }
73 |   const lower = text.toLowerCase();
74 |   const match = lower.match(/(\d+(\.\d+)?)?/);
75 |   if (!match) {
76 |     return 0;
77 |   }
78 |   const value = parseFloat(match[1]);
79 |   return /min/.test(lower) ? Math.round(value) : Math.round(value * 60);
80 | }
```

Turns the free text a customer typed in "Estimated Duration" (placeholder "e.g. 3 hours") into a number of minutes.

- Empty, `null` or `undefined` text gives 0.
- `text.toLowerCase()` makes the check case-insensitive.
- `lower.match(/(\d+(\.\d+)?)?/)` finds the **first number** in the text: digits, optionally followed by a dot and more digits. If there is none, return 0.
- `parseFloat(match[1])` turns that number text into a number.
- If the text contains `min` anywhere (`/min/.test(lower)`), the number is minutes and is rounded; otherwise it is treated as **hours** and multiplied by 60 before rounding.

Worked examples: `parseDurationToMinutes('3 hours')` is 180; `parseDurationToMinutes('1.5 hours')` is 90; `'1.5 hrs'` is 90; `'45 minutes'` is 45; `'30 MIN'` is 30; `'2'` (no unit) is 120; `'abc'` is 0; `' '` and `null` are 0.

Limitation worth knowing: only the first number is read, so `'2 hours 30 minutes'` contains `min` and is read as just 2 minutes (not 150).

Lines 82-86 | `formatDuration` - minutes to text

```
82 | export function formatDuration(totalMinutes: number): string {
83 |   const hours = Math.floor(totalMinutes / 60);
84 |   const minutes = totalMinutes % 60;
85 |   return `${hours} hours ${minutes} minutes`;
86 | }
```

The reverse conversion, used when a trip ends to store `actualDuration`. `Math.floor(totalMinutes / 60)` is the number of whole hours and `totalMinutes % 60` (the **remainder**) is the minutes left over. The result is always written as `X hours Y minutes`. Examples: `formatDuration(90)` is `'1 hours 30 minutes'`; `formatDuration(180)` is `'3 hours 0 minutes'`; `formatDuration(45)` is `'0 hours 45 minutes'`. The wording never uses the singular ("1 hours"), a small cosmetic quirk.

Workflow, next: The text and time helpers are ready. The last helper handles driver pictures:
`src/app/utils/driver-image.ts`.

Step 58: `driver-image.ts` - driver photos and the default avatar [Arjun - Driver & Booking]

`angularapp/src/app/utils/driver-image.ts`

FRONTEND

What this file is	Drivers have an <code>image</code> field that holds a Base64 text (a way to write picture bytes as plain text) coming from the database. This helper turns whatever is stored into something a browser can use as the <code>src</code> of an <code>img</code> tag, and provides a grey-and-black person icon (an inline SVG) for drivers with no photo or a broken photo.
Who / what triggers it	Used by the templates and classes of <code>admin-view-drivers</code> , <code>customerviewdriver</code> , <code>customerviewrequested</code> , <code>adminviewrequests</code> and <code>customerviewfeedback</code> : <code>getDriverImage(driver.image)</code> for the <code>src</code> and <code>useDefaultAvatar(\$event)</code> for the <code>(error)</code> event.
What it uses / calls	<code>encodeURIComponent</code> and the DOM type <code>HTMLImageElement</code> . No other project file.

Lines 1-10 | DEFAULT_AVATAR - an SVG picture stored as text

```

1 | /** Generic avatar (white disc, dark silhouette) used when a driver has no photo. */
2 | export const DEFAULT_AVATAR =
3 |   'data:image/svg+xml;utf8,' +
4 |   encodeURIComponent(
5 |     '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 100 100">' +
6 |     '<circle cx="50" cy="50" r="49" fill="#fff" stroke="#111" stroke-width="2"/>' +
7 |     '<circle cx="50" cy="38" r="15" fill="#111"/>' +
8 |     '<path d="M22 80c4-17 16-24 28-24s24 7 28 24c-8 8-18 12-28 12s30 88 22 80z" fill="#111"/>' +
9 |     '</svg>'
10 |   );

```

A **data URL** is a way to put a whole file inside a URL: `data:image/svg+xml;utf8`, says "what follows is an SVG image written as text". The text is built by joining (+) pieces: `encodeURIComponent(...)` converts special characters (such as `<`, `#` and spaces) into `%` codes so that the URL is valid. The SVG itself (lines 5-9) is drawn on a 100 by 100 grid: a white circle with a dark outline (the disc), a small dark circle for the head (`cx="50" cy="38" r="15"`) and a dark curved **path** for the shoulders. Because it is embedded, no image file is needed and it can never fail to load.

Lines 12-21 | getDriverImage - make a usable img src

```

12 | /** Returns something usable as an <img src>, accepting full data URLs and raw Base64 strings. */
13 | export function getDriverImage(image?: string | null): string {
14 |   if (!image || !image.trim()) {
15 |     return DEFAULT_AVATAR;
16 |   }
17 |   if (image.startsWith('data:') || image.startsWith('http')) {
18 |     return image;
19 |   }
20 |   return 'data:image/png;base64,' + image;
21 | }

```

The parameter `image?: string | null` means "a text, `null` or missing". The function chooses:

- `!image || !image.trim()`: no image, or only spaces, so return `DEFAULT_AVATAR`.
- The text already starts with `data:` (a full data URL) or `http` (a normal web address): return it unchanged.
- Otherwise assume it is **raw Base64** from the database and add the prefix `data:image/png;base64,`.

Examples: `getDriverImage(null)` gives the default avatar; `getDriverImage('iVBORw0KGgo...')` gives `'data:image/png;base64,iVBORw0KGgo...'`; `getDriverImage('data:image/jpeg;base64,/9j/...')` is returned unchanged.

Lines 23-29 | useDefaultAvatar - the (error) handler

```

23 | /** (error) handler for <img>: fall back to the default avatar. */
24 | export function useDefaultAvatar(event: Event): void {
25 |   const img = event.target as HTMLImageElement;
26 |   if (img && img.src !== DEFAULT_AVATAR) {
27 |     img.src = DEFAULT_AVATAR;
28 |   }
29 | }

```

Used in templates as `<img [src]="getDriverImage(d.image)" (error)="useDefaultAvatar($event)">`. The browser fires an **error** event when an image cannot be loaded or decoded (for example corrupted Base64). `event.target as HTMLImageElement` is a TypeScript **type assertion** ("trust me, this is an image element"). If the image exists and its current `src` is not already the default avatar, it is replaced by `DEFAULT_AVATAR`. The comparison prevents an endless loop if the default image itself failed.

Workflow, next: The helper files are done. Next are the **models**, the TypeScript shapes of the data that travels between Angular and Spring Boot. The first is the one used on the login page: `src/app/models/login.model.ts`.

Step 59: Login model - the credentials object [Aleena - Authentication]

FRONTEND

angularapp/src/app/models/login.model.ts

What this file is	A model describes the shape of a piece of data. TypeScript checks at build time that code uses the right property names and types (the checks are removed in the final JavaScript). There are two ways to write a model: an interface (only a description, produces no JavaScript, you create the object yourself with <code>{ ... }</code>) and a class (a real blueprint that also exists at run time, so you can write <code>new Login()</code> and give properties default values). <code>Login</code> is a class. It carries what the user typed on the login page.
Who / what triggers it	Imported by <code>auth.service.ts</code> (type of the <code>login()</code> argument) and used by <code>login.component.ts</code> when it builds the credentials to send.
What it uses / calls	Nothing.
Size	4 lines

Lines 1-4 | class Login

```
1 | export class Login {  
2 |   email: string = '';  
3 |   password: string = '';  
4 | }
```

`export class Login` makes the class available to other files. `email: string = ''` declares a property named `email` of type text and gives it the **default value** empty text; `password` likewise. So `new Login()` produces `{ email: '', password: '' }`. When posted to the back end this becomes the JSON `{"email": "...", "password": "..."}`, which `AuthController.authenticateUser` reads (it receives it as a `User` object and uses only the e-mail and password fields).

Workflow, next: The login credentials shape is known. The user who registers and who is returned by the server is described in `src/app/models/user.model.ts`.

Step 60: User model - a registered user [Aleena - Authentication]

FRONTEND

angularapp/src/app/models/user.model.ts

What this file is	The data of one user account (customer or admin). It mirrors the Java entity <code>User</code> on the back end, so the JSON the signup form sends can be read directly by <code>AuthController.registerUser</code> . This one is a class with defaults, because the signup page can create an empty <code>User</code> and fill it step by step.
Who / what triggers it	Used by <code>auth.service.ts</code> (<code>register(user: User)</code>), <code>signup.component.ts</code> , and as the nested <code>user?</code> property in the <code>DriverRequest</code> and <code>Feedback</code> models.
What it uses / calls	Nothing.
Size	8 lines

Lines 1-8 | class User and its fields

```
1 | export class User {  
2 |   userId?: number;  
3 |   email: string = '';  
4 |   password: string = '';  
5 |   username: string = '';  
6 |   mobileNumber: string = '';  
7 |   userRole: string = '';  
8 | }
```

A class with six properties.

- `userId?: number`: the database id. The `?` makes it **optional**: a new user has no id yet because the database generates it. The type is `number | undefined`.
- `email`, `password`, `username`, `mobileNumber`, `userRole`: all text, default `''`.

The role is the text `'Admin'` or `'Customer'`. The `password` field is only used when registering or logging in: on the Java side it is marked `WRITE_ONLY`, so it is accepted in incoming JSON (and stored as a **BCrypt hash**) but never included in responses. A `User` nested inside a request or feedback therefore arrives without a password, and the default `''` here simply stays empty.

Workflow, next: Next is the model for the main business object, the driver: `src/app/models/driver.model.ts`.

Step 61: Driver model - a driver [Arjun - Driver & Booking]

`angularapp/src/app/models/driver.model.ts`

FRONTEND

What this file is	The shape of a driver record, matching the Java entity <code>Driver</code> . It is an interface , so it exists only for type checking: a <code>Driver</code> object is written as a plain <code>{ driverName: '...', ... }</code> literal or arrives as parsed JSON from the server.
Who / what triggers it	Used by <code>driver.service.ts</code> , <code>ai.service.ts</code> and the driver components; also referenced as the nested <code>driver?</code> of <code>DriverRequest</code> and <code>Feedback</code> .
What it uses / calls	Nothing.
Size	12 lines

Lines 1-12 | interface Driver

```

1 | export interface Driver {
2 |   driverId?: number;
3 |   driverName: string;
4 |   licenseNumber: string;
5 |   experienceYears: number;
6 |   contactNumber: string;
7 |   availabilityStatus: string; // Initially "Active". Allowed: "Active", "Inactive", "On Leave"
8 |   address: string;
9 |   vehicleType: string; // e.g., "Sedan", "SUV", "Bike"
10 |   hourlyRate: number;
11 |   image: string; // Base64-encoded image of the driver
12 | }
```

Each line is `name: type`. `driverId?: number` is optional because a driver that has not been saved has no id yet (the database assigns it). All other fields are **required**; the compiler gives an error if one is missing in a literal.

- `driverName`, `licenseNumber`, `contactNumber`, `address`: text. The licence number must be unique (the back end answers 409 on a duplicate).
- `experienceYears`: number of years.
- `availabilityStatus`: text; initially `"Active"`, allowed `"Active"`, `"Inactive"`, `"On Leave"` (see `DRIVER_STATUSES`).
- `vehicleType`: text such as Sedan, SUV or Bike (see `VEHICLE_TYPES`).
- `hourlyRate`: a number; the fare of a trip is this rate times the hours driven.
- `image`: the driver photo as a Base64 string (see `driver-image.ts`).

Workflow, next: Drivers are described. Customers book drivers with requests, modelled in `src/app/models/driver-request.model.ts`.

Step 62: DriverRequest model - a booking request [Arjun - Driver & Booking]

FRONTEND

angularapp/src/app/models/driver-request.model.ts

What this file is	The shape of a trip request: who asked, which driver, when, from where to where, the status and the payment. It matches the Java entity <code>DriverRequest</code> and is an interface . It also shows an important detail of this project: the interface has flat <code>userId</code> and <code>driverId</code> fields and nested <code>user</code> and <code>driver</code> objects.
Who / what triggers it	Used by <code>driver-request.service.ts</code> and the request components.
What it uses / calls	The <code>Driver</code> and <code>User</code> models (imported for the nested properties).
Size	24 lines

Lines 1-2 | Imports of the nested types

```
1 | import { Driver } from './driver.model';
2 | import { User } from './user.model';
```

`Driver` and `User` are imported so that the nested properties on lines 22-23 can be typed. Imports used only as types disappear when the code is compiled to JavaScript.

Lines 4-10 | Identity, dates and status

```
4 | export interface DriverRequest {
5 |   driverRequestId?: number;
6 |   userId: number;
7 |   driverId?: number;
8 |   requestDate: Date; // ISO Date format (YYYY-MM-DD)
9 |   status: string; // "Pending", "Approved", "Rejected", "Trip End", "Closed"
10 |   tripDate: Date; // ISO Date format (YYYY-MM-DD)
```

`driverRequestId?` is optional (no id before saving). `userId: number` and `driverId?: number` are the plain ids of the requester and the driver. `requestDate`, `tripDate` and `timeSlot` are typed as `Date` (the comments say they travel as `YYYY-MM-DD` and a time, which the back end maps to `LocalDate` and `LocalTime`; in practice the app keeps them as text and converts them with `format.ts`). `status` is text, one of "Pending", "Approved", "Rejected", "Trip End", "Closed" (see `REQUEST_STATUSES`).

Lines 11-14 | Trip details

```
11 |   timeSlot: Date; // Time as a Date object (compatible with LocalTime)
12 |   pickupLocation: string;
13 |   dropLocation: string;
14 |   estimatedDuration: string; // e.g., "3 hours"
```

`pickupLocation` and `dropLocation` (text), and `estimatedDuration` (text typed by the customer, such as "3 hours", which `parseDurationToMinutes` later reads).

Lines 15-19 | Optional payment and actual-trip fields

```
15 |   paymentAmount?: number;
16 |   comments?: string;
17 |   actualDropTime?: Date;
18 |   actualDuration?: string;
19 |   actualDropDate?: Date;
```

These fields are optional (?) because they are empty until the trip ends: `paymentAmount` (the fare, calculated in the Angular app at Trip End and saved with a PUT), `comments` (an optional note), and `actualDropTime`, `actualDuration`, `actualDropDate` (when and for how long the trip really lasted).

Lines 21-24 | Nested user and driver - why both ids and objects exist

```
21 | // The backend sends / expects the related entities nested ({ user: {...}, driver: {...} }).
22 | user?: User;
23 | driver?: Driver;
24 | }
```

The Java entity has **no** `userId` field: it has `User user` and `Driver driver` relationships, so the JSON that Spring sends and expects has nested objects: `{ "user": { "userId": 1, ... }, "driver": { "driverId": 3, ... } }`. That is why `user?` and `driver?` are here (optional, because they are filled by the server when reading, and only the id inside is needed when writing). The flat `userId` and `driverId` come from the original specification of the exercise and the old tests; they are not sent by the back end. In real use the request form builds `{ user: {`

`userId }`, `driver: { driverId }, ... }` and casts it to `DriverRequest`. Components therefore read `request.driver?.driverName` rather than `request.driverId`. The `?` is the **optional chaining** operator: it returns `undefined` instead of an error if `driver` is missing.

Workflow, next: Requests are described. Customers also give feedback, whose model is `src/app/models/feedback.model.ts`.

Step 63: Feedback model - customer feedback with AI fields [Saumya - Feedback & AI]

`angularapp/src/app/models/feedback.model.ts`

FRONTEND

What this file is	The shape of one feedback entry. Unlike <code>Driver</code> and <code>DriverRequest</code> , this is a class with default values , so the feedback form can start from <code>new Feedback()</code> and fill it. It also holds three fields that the back end computes with AI (sentiment, score, tags) when the feedback is saved.
Who / what triggers it	Used by <code>feedback.service.ts</code> , <code>customerpostfeedback</code> , <code>customerviewfeedback</code> and <code>adminviewfeedback</code> .
What it uses / calls	The <code>Driver</code> and <code>User</code> models.
Size	21 lines

Lines 1-2 | Imports

```
1 | import { Driver } from './driver.model';
2 | import { User } from './user.model';
```

The two models used for the nested properties.

Lines 4-11 | Own fields with defaults

```
4 | export class Feedback {
5 |   feedbackId?: number;
6 |   feedbackText: string = '';
7 |   date: Date = new Date();
8 |   userId: number = 0;
9 |   driverId?: number; // Optional for non-driver feedback
10 |   category: string = '';
11 |   rating: number = 0;
```

`export class Feedback` is a class, so each property can have an initial value: `feedbackText = ''`, `date = new Date()` (today), `userId = 0`, `category = ''`, `rating = 0`. `feedbackId?` is optional (no id before saving). `driverId?: number` is optional "for non-driver feedback" (a general comment about the service rather than about one driver). `rating` is the star score the customer gives.

Lines 13-16 | AI-generated fields

```
13 | // AI generated attributes (populated automatically by the backend on creation)
14 | sentiment?: string; // "Positive" | "Neutral" | "Negative"
15 | sentimentScore?: number; // 0.0 (very negative) - 1.0 (very positive)
16 | aiTags?: string; // comma-separated theme tags
```

These are never typed by the user; they are filled by `aiService.applyAnalysis` on the back end inside `FeedbackServiceImpl.createFeedback`, and they come back in the response. `sentiment` is "Positive", "Neutral" or "Negative"; `sentimentScore` is a number from 0.0 (very negative) to 1.0 (very positive); `aiTags` is one text of comma-separated theme tags such as `punctuality,safety`. They are optional because a feedback object created in the browser does not have them yet.

Lines 18-21 | Nested user and driver

```
18 | // The backend sends / expects the related entities nested ({ user: {...}, driver: {...} }).
19 | user?: User;
20 | driver?: Driver;
21 | }
```

The same pattern as in `DriverRequest`: the Java entity `Feedback` has relationships `User user` and `Driver driver`, so JSON from the server contains `user: {...}` and `driver: {...}`, while the flat `userId` and `driverId` are conveniences of the Angular side. The form therefore sends `{ ..., user: { userId }, driver: { driverId } }`. The admin feedback list displays names through `feedback.user?.username` and `feedback.driver?.driverName`.

Workflow, next: Feedback is described. The last model describes the AI summary of a driver's feedback: `src/app/models/ai.model.ts`.

Step 64: DriverFeedbackSummary - the AI summary result [Saumya - Feedback & AI]

FRONTEND

`angularapp/src/app/models/ai.model.ts`

What this file is	An interface for the JSON that <code>GET /api/ai/driver/{id}/feedback-summary</code> returns. It is a result only: the front end never creates it, it only receives it, so an interface (no run-time code) is enough.
Who / what triggers it	Used as the return type in <code>ai.service.ts</code> and called from <code>adminviewfeedback.component.ts</code> , which shows the AI summary of a driver's feedback.
What it uses / calls	Nothing.
Size	8 lines

Lines 1-8 | interface DriverFeedbackSummary

```
1 | export interface DriverFeedbackSummary {
2 |   summary: string;
3 |   strengths: string[];
4 |   improvements: string[];
5 |   aiScore: number;
6 |   averageRating: number;
7 |   feedbackCount: number;
8 | }
```

The fields match the Java `Map<String, Object>` built by `AiService.summarizeDriverFeedback` (returned by `AiController.getFeedbackSummary`): `summary` is a short paragraph of text; `strengths` and `improvements` are **arrays of text** (`string[]`), shown as two bullet lists; `aiScore` is a number rating from the AI; `averageRating` is the mean of the customer star ratings; `feedbackCount` is how many feedback entries were analysed.

Workflow, next: All data shapes are known. Now come the **services**, the classes that actually send data over HTTP. First the safety net that sees every HTTP result: `src/app/services/http-error.interceptor.ts`.

Step 65: HttpErrorInterceptor - global HTTP error handling [Vanshika - Security]

FRONTEND

`angularapp/src/app/services/http-error.interceptor.ts`

What this file is	An HTTP interceptor is a class that sits between the application and the network. Every request that <code>HttpClient</code> sends, and every response or error that comes back, passes through it, so one class can handle a problem for all services at once instead of each component repeating the same code. This interceptor deals with two situations: the server is unreachable or crashed (the user is sent to the <code>/error</code> page) and the stored token is no longer accepted (the user is logged out and sent to <code>/login</code>). It does not add the <code>Authorization</code> header; each service does that itself.
Who / what triggers it	Registered in <code>AppModule</code> with <code>{ provide: HTTP_INTERCEPTORS, useClass: HttpErrorInterceptor, multi: true }</code> . From then on Angular calls its <code>intercept</code> method for every HTTP call made by <code>AuthService</code> , <code>DriverService</code> , <code>DriverRequestService</code> , <code>FeedbackService</code> and <code>AiService</code> .
What it uses / calls	<code>Router</code> (to navigate), <code>Injector</code> (to fetch <code>AuthService</code> later), <code>AuthService.logout()</code> , <code>RxJS catchError</code> and <code>throwError</code> , <code>HttpErrorResponse</code> and <code>localStorage</code> .
Size	35 lines

Lines 1-6 | Imports

```
1 | import { Injectable, Injector } from '@angular/core';
2 | import { HttpResponse, HttpEvent, HttpHandler, HttpInterceptor, HttpRequest } from '@angular/common/http';
3 | import { Router } from '@angular/router';
4 | import { Observable, throwError } from 'rxjs';
5 | import { catchError } from 'rxjs/operators';
6 | import { AuthService } from './auth.service';
```

Injectable marks the class as something Angular can create with dependency injection; **Injector** is Angular's "container" from which any service can be fetched by hand. From **@angular/common/http**: **HttpInterceptor** is the interface this class must implement, **HttpRequest** and **HttpHandler** are the outgoing request and the next link in the chain, **HttpEvent** is the type of what flows back, and **HttpResponse** is the error object created for a failed call. **Router** is used to change the page. **Observable** and **throwError** come from RxJS, **catchError** is an RxJS operator that reacts to an error in the stream. **AuthService** is imported only as a type/key for **injector.get**.

Lines 8-14 | Description, @Injectable and class

```
8 | /**
9 |  * Sends the user to the error page when the server is unreachable or fails (status 0 / 5xx)
10 |  * and back to the login page when a stored token is no longer accepted (401).
11 |  * Business errors (4xx) are left for the components to handle.
12 |  */
13 | @Injectable()
14 | export class HttpErrorInterceptor implements HttpInterceptor {
```

The comment states the rules: status 0 or 5xx goes to the error page, 401 goes to login, and business errors (other 4xx such as 404 or 409) are left to the components that made the call. **@Injectable()** (without **providedIn**) is enough because **AppModule** provides this class explicitly. **implements HttpInterceptor** forces the class to have an **intercept** method with the right signature.

Lines 15-16 | Constructor and the lazy Injector

```
15 | // AuthService is resolved lazily: it depends on HttpClient, which depends on this interceptor.
16 | constructor(private router: Router, private injector: Injector) {}
```

Angular injects **Router** and **Injector**. The comment explains the key trick: **AuthService is not injected in the constructor on purpose**. **AuthService** needs **HttpClient**; **HttpClient** needs all registered interceptors; this interceptor would need **AuthService**: that is a circular dependency (A needs B, B needs C, C needs A), and Angular would stop with an error. By taking the **Injector** instead and calling **this.injector.get(AuthService)** only when an error actually happens, the lookup is delayed until all objects exist, so the cycle never occurs.

Lines 18-19 | intercept - pass the request on

```
18 | intercept(request: HttpRequest<unknown>, next: HttpHandler): Observable<HttpEvent<unknown>> {
19 |     return next.handle(request).pipe(
```

intercept(request, next) is called for every request. **request** is the outgoing call (URL, method, body, headers); **next** is the rest of the chain, ending at the real network call. **next.handle(request)** sends it and returns an **Observable** of the response. **.pipe(catchError(...))** attaches an error handler to that stream. Successful responses flow through unchanged.

Lines 20-22 | Classifying the call

```
20 |         catchError((error: unknown) => {
21 |             const isAuthCall = request.url.includes('/api/login') || request.url.includes('/api/register');
22 |             const isAiCall = request.url.includes('/api/ai/'); // AI features degrade gracefully
```

Runs only when the call failed. **error** is typed **unknown** (it could be anything). Two flags are computed from the URL: **isAuthCall** is true for **/api/login** and **/api/register**; there a 401 or 409 simply means wrong password or e-mail already used and must be shown on the form, not trigger a redirect. **isAiCall** is true for any **/api/ai/** URL; the comment says AI features "degrade gracefully", so a failing AI service must not throw the user out of the page.

Lines 23-25 | Server down or crashed: go to /error

```
23 |         if (error instanceof HttpResponse && !isAuthCall && !isAiCall) {
24 |             if (error.status === 0 || error.status >= 500) {
25 |                 this.router.navigate(['/error']);
```

error instanceof HttpResponse checks that this really is an HTTP failure, and the two flags must be false. **Status 0** is what the browser reports when the request never reached a server (back end stopped, wrong URL, network down); **status 500 and above** means the server crashed. In both cases **this.router.navigate(['/error'])** shows **ErrorComponent**.

Lines 26-30 | Token rejected: logout and go to /login

```
26 |         } else if (error.status === 401 && localStorage.getItem('token')) {
27 |             this.injector.get(AuthService).logout();
28 |             this.router.navigate(['/login']);
29 |         }
30 |     }
```

`else if (error.status === 401 && localStorage.getItem('token'))`: 401 means "not authenticated". The extra check that a token is stored means the user thought they were logged in, so the token must have expired or become invalid. The code then calls `this.injector.get(AuthService).logout()` (clears `localStorage` and the login state, which also hides the navbar) and navigates to `/login`. A 401 without a stored token (a guest) is ignored. Other codes (400, 403, 404, 409) are not handled here.

Lines 31-35 | Re-throw the error

```
31 |         return throwError(() => error);
32 |     })
33 | );
34 | }
35 | }
```

`throwError(() => error)` returns a new Observable that fails with the same error. This is important: after the interceptor has done its redirect, the original `subscribe({ error: ... })` block of the component still runs, so components can show messages such as "Email already exists" or "Driver not found". Lines 32-35 close the arrow function, the `pipe` call, the method and the class.

Workflow, next: The interceptor is ready to catch failures. The service it relies on for logout, and the one every login goes through, is `src/app/services/auth.service.ts`.

Step 66: AuthService - register, login, logout and login state [Aleena - Authentication]

angularapp/src/app/services/auth.service.ts

FRONTEND

What this file is	A service is a class that holds logic or data shared by many components. <code>AuthService</code> handles everything about who is logged in: it calls the register and login endpoints, saves the login result (token, role, user id, username) in the browser's <code>localStorage</code> (a small key-value store that survives page refresh), and broadcasts the current role and username so that other parts of the UI (the navbar choice in <code>AppComponent</code> , the nav components) update immediately. It also checks whether the stored token has expired.
Who / what triggers it	Created once by Angular's dependency injection the first time something asks for it (the <code>AppComponent</code> constructor does this at start-up). <code>login.component.ts</code> and <code>signup.component.ts</code> call <code>login()</code> and <code>register()</code> ; the navbar components call <code>logout()</code> ; <code>AuthGuard</code> calls <code>isLoggedIn()</code> and <code>getUserRole()</code> ; many pages call <code>getUserId()</code> .
What it uses / calls	<code>HttpClient</code> for the calls to the Spring Boot back end, RxJS <code>BehaviorSubject</code> and the <code>tap</code> operator, <code>localStorage</code> , <code>apiUrl</code> from <code>apiconfig.ts</code> , and the <code>User</code> and <code>Login</code> models.
Size	107 lines

Lines 1-7 | Imports

```
1 | import { Injectable } from '@angular/core';
2 | import { HttpClient } from '@angular/common/http';
3 | import { BehaviorSubject, Observable } from 'rxjs';
4 | import { tap } from 'rxjs/operators';
5 | import { apiUrl } from '../..//apiconfig';
6 | import { User } from '../models/user.model';
7 | import { Login } from '../models/login.model';
```

`Injectable` makes the class a service. `HttpClient` sends HTTP requests. `BehaviorSubject` (a special Observable that remembers its latest value) and `Observable` come from RxJS; `tap` is an RxJS operator that runs a side effect (here: saving data) without changing the value. `apiUrl` is `http://localhost:8080`. `User` and `Login` are the two models for the request bodies.

Lines 9-12 | @Injectable providedIn root and the class

```
9 | @Injectable({
10 |   providedIn: 'root'
11 | })
12 | export class AuthService {
```

`@Injectable({ providedIn: 'root' })` tells Angular to create **exactly one instance** of this class for the whole application (a **singleton**) and to make it available everywhere without listing it in `AppModule.providers`. Every class that asks for `AuthService` in its constructor receives that same object. This is **dependency injection**: components do not create services with `new`; Angular supplies them. The class is `AuthService`.

Line 13 | apiUrl property

```
13 | public apiUrl: string = apiUrl;
```

`public apiUrl: string = apiUrl;` stores the base URL on the object so the URLs below can be written with `${this.apiUrl}`. (The right-hand `apiUrl` is the imported constant; the left one is the property.)

Lines 15-17 | BehaviorSubjects holding the login state

```
15 | private userRoleSubject = new BehaviorSubject<string | null>(this.readStored('userRole'));
16 | private userIdSubject = new BehaviorSubject<number | null>(this.toNumber(this.readStored('userId')));
17 | private usernameSubject = new BehaviorSubject<string | null>(this.readStored('username'));
```

A **BehaviorSubject** is both something you can push values into (`.next(value)`) and an Observable others can listen to. It always keeps the latest value and gives it immediately to every new subscriber. Three are created, holding the role (text or `null`), the user id (number or `null`) and the username. Their **initial value is read from localStorage**: after a browser refresh the user is still logged in, so `userRole` starts as `'Admin'` or `'Customer'` instead of `null` and the right navbar shows at once. `readStored` and `toNumber` (defined below) read a key safely and convert the id text to a number.

Lines 19-21 | Public read-only observables

```
19 | userRole$: Observable<string | null> = this.userRoleSubject.asObservable();
20 | userId$: Observable<number | null> = this.userIdSubject.asObservable();
21 | username$: Observable<string | null> = this.usernameSubject.asObservable();
```

`asObservable()` returns a version that can be listened to but not pushed into. Other classes subscribe to `userRole$`, `userId$` and `username$`, while only `AuthService` itself can call `next`. The trailing `$` marks an Observable. `AppComponent` uses `userRole$` to choose the navbar; the two nav components use `username$` and `userRole$` to show the user chip.

Line 23 | Constructor

```
23 | constructor(private http: HttpClient) {}
```

`constructor(private http: HttpClient) {}` asks Angular to inject the `HttpClient`. `private` automatically creates the property `this.http`. The body is empty because all work is in the methods.

Lines 25-27 | register

```
25 | register(user: User): Observable<any> {
26 |     return this.http.post(`${this.apiUrl}/api/register`, user);
27 | }
```

`register(user: User): Observable<any>` sends `POST http://localhost:8080/api/register` with the user as the JSON body. On the back end `AuthController.registerUser` answers 201 with the saved user, or 409 if the e-mail already exists. An **Observable** is a "plan" for a result that will arrive later: **nothing is sent until someone calls .subscribe()** (the signup component does). The component then handles the success or error in its subscribe callbacks.

Lines 29-43 | login - call, then remember the result

```
29 | login(login: Login): Observable<any> {
30 |     return this.http.post<any>(`${this.apiUrl}/api/login`, login).pipe(
31 |         tap((response) => {
32 |             if (response && response.token) {
33 |                 localStorage.setItem('token', response.token);
34 |                 localStorage.setItem('userRole', response.userRole);
35 |                 localStorage.setItem('userId', String(response.userId));
36 |                 localStorage.setItem('username', response.username);
37 |                 this.userRoleSubject.next(response.userRole);
38 |                 this.userIdSubject.next(Number(response.userId));
39 |                 this.usernameSubject.next(response.username);
40 |             }
41 |         })
42 |     );
43 | }
```

`this.http.post<any>(...)` sends `POST /api/login` with `{ email, password }`; `AuthController.authenticateUser` checks the password and replies with a `LoginDTO { token, username, userRole, userId }` (or 401 "Invalid credentials"). `.pipe(tap(...))` runs the function inside `tap` when the answer arrives, before it is passed on to the component. If the response has a `token`, the code:

- saves `token`, `userRole`, `userId` and `username` with `localStorage.setItem(key, text)` (localStorage can only store text, hence `String(response.userId)`);

- calls `.next(...)` on each `BehaviorSubject`, which immediately notifies every subscriber. This is the moment `AppComponent` sees the new role and the navbar appears, without a page reload.

The response is returned unchanged, so `login.component.ts` can decide where to navigate.

Lines 45-53 | logout

```
45 |   logout(): void {
46 |       localStorage.removeItem('token');
47 |       localStorage.removeItem('userRole');
48 |       localStorage.removeItem('userId');
49 |       localStorage.removeItem('username');
50 |       this.userRoleSubject.next(null);
51 |       this.userIdSubject.next(null);
52 |       this.usernameSubject.next(null);
53 |   }
```

Removes the four keys from `localStorage` and pushes `null` into the three subjects, so the navbar disappears and the next `AuthGuard` check fails. It has no HTTP call: JWT logout is purely client-side, the token is simply forgotten. The return type `void` means it returns nothing. It is called by both nav bars, by the interceptor (on a 401) and by `isLoggedIn` (on expiry).

Lines 55-65 | isLoggedIn - token present and not expired

```
55 |   isLoggedIn(): boolean {
56 |       const token = this.getToken();
57 |       if (!token) {
58 |           return false;
59 |       }
60 |       if (this.isTokenExpired(token)) {
61 |           this.logout();
62 |           return false;
63 |       }
64 |       return true;
65 |   }
```

Used by `AuthGuard` before every protected route. If there is no token, return `false`. If there is one, check `isTokenExpired`; an expired token triggers `this.logout()` (clean up) and returns `false`. Otherwise `true`. This means a user whose 24-hour token has run out is sent to the login page by the guard without waiting for a server 401.

Lines 67-81 | Simple getters

```
67 |   getToken(): string | null {
68 |       return localStorage.getItem('token');
69 |   }
70 |
71 |   getUserRole(): string | null {
72 |       return localStorage.getItem('userRole');
73 |   }
74 |
75 |   getUserId(): number | null {
76 |       return this.toNumber(localStorage.getItem('userId'));
77 |   }
78 |
79 |   getUsername(): string | null {
80 |       return localStorage.getItem('username');
81 |   }
```

Four small methods read the saved values straight from `localStorage`: `getToken()` (the JWT), `getUserRole()`, `getUserId()` (converted to a number or `null` by `toNumber`) and `getUsername()`. They return `null` when nothing is stored. Components use `getUserId()` to ask for "my requests" or to put the user id into a new request; `AuthGuard` uses `getUserRole()`.

Lines 83-91 | isTokenExpired - reading the JWT payload with atob

```
83 |   private isTokenExpired(token: string): boolean {
84 |       try {
85 |           const payload = JSON.parse(atob(token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/')));
86 |           return !payload.exp && payload.exp * 1000 < Date.now();
87 |       } catch {
88 |           // A token we cannot decode is left for the server to judge.
89 |           return false;
90 |       }
91 |   }
```

A **JWT (JSON Web Token)** is three parts separated by dots: `header.payload.signature`. The payload is JSON that has been **Base64URL-encoded**, and it includes `exp`, the expiry moment in seconds since 1970. (The back end builds it in `JwtUtils` with an expiry of 86 400 000 ms = 24 hours.) The code:

- `token.split('.')[1]` takes the middle part, the payload;
- `.replace(/-/g, '+').replace(/_/g, '/')` converts the URL-safe alphabet back to normal Base64 (- becomes +, _ becomes /; the `g` means "all occurrences");

- `atob(...)` (a browser function, "ASCII to binary") decodes Base64 into the JSON text, and `JSON.parse` turns it into an object;
- `!!payload.exp && payload.exp * 1000 < Date.now()` multiplies seconds by 1000 to get milliseconds, compares with the current time, and `!!` makes sure a token without `exp` counts as not expired.

The browser **cannot verify the signature** (it does not have the secret); this check only avoids sending obviously dead tokens. If decoding fails (`catch`), the method returns `false` and leaves the decision to the server, as the comment says.

Lines 93-99 | `readStored` - safe `localStorage` read

```
93 | private readStored(key: string): string | null {
94 |   try {
95 |     return localStorage.getItem(key);
96 |   } catch {
97 |     return null;
98 |   }
99 | }
```

Reads one key. The `try/catch` is a safety net: in some situations (browser privacy settings, storage disabled) touching `localStorage` throws an error, and the app should then behave as "nobody is logged in" instead of crashing at start-up. It is `private`, so only this class can use it.

Lines 101-107 | `toNumber` - text to number or null

```
101 | private toNumber(value: string | null): number | null {
102 |   if (value === null || value === '' || isNaN(Number(value))) {
103 |     return null;
104 |   }
105 |   return Number(value);
106 | }
107 | }
```

The id is stored as text. This helper returns `null` for `null`, empty text or something that is not a number (`isNaN(Number(value))`), and otherwise the number: `toNumber('5')` is 5, `toNumber('abc')` is `null`, `toNumber(null)` is `null`. Line 107 closes the class.

Workflow, next: The login service is understood. The other four services use the stored token to call the protected API, starting with the driver service: `src/app/services/driver.service.ts`.

Step 67: DriverService - REST calls for drivers [Arjun - Driver & Booking]

`angularapp/src/app/services/driver.service.ts`

FRONTEND

What this file is	The class that talks to the back end about drivers. Each method corresponds to one REST endpoint of <code>DriverController</code> . It returns an Observable , so the component decides what to do when the answer arrives (<code>subscribe</code>). All authorisation is done by sending the JWT in an <code>Authorization</code> header.
Who / what triggers it	Injected into <code>driver-management</code> , <code>admin-view-drivers</code> , <code>customerviewdriver</code> , and other components that need drivers.
What it uses / calls	<code>HttpClient</code> , <code>HttpHeaders</code> , <code>localStorage</code> (the token), <code>apiUrl</code> , and the <code>Driver</code> model.
Size	39 lines

Lines 1-5 | Imports

```
1 | import { Injectable } from '@angular/core';
2 | import { HttpClient, HttpHeaders } from '@angular/common/http';
3 | import { Observable } from 'rxjs';
4 | import { apiUrl } from '../..//apiconfig';
5 | import { Driver } from '../models/driver.model';
```

`Injectable` for dependency injection, `HttpClient` to call the API and `HttpHeaders` to build request headers, `Observable` as the return type, `apiUrl` for the base address and the `Driver` model for typing.

Lines 7-13 | Service declaration and constructor

```
7 | @Injectable({
8 |   providedIn: 'root'
9 | })
10 | export class DriverService {
11 |   public apiUrl: string = apiUrl;
12 |
13 |   constructor(private http: HttpClient) {}
```

`@Injectable({ providedIn: 'root' })` makes a single shared instance available application-wide. `public apiUrl` keeps the base URL. The constructor receives the `HttpClient` (dependency injection).

Lines 15-18 | getHeaders - the Authorization header

```
15 | private getHeaders(): HttpHeaders {
16 |   const token = localStorage.getItem('token');
17 |   return new HttpHeaders({ Authorization: `Bearer ${token}` });
18 | }
```

Reads the saved JWT from `localStorage` **every time** a call is made (so a fresh login is used immediately) and builds `HttpHeaders` with `Authorization: Bearer <token>`. `Bearer` is the standard word for "whoever carries this token". On the server `JwtAuthenticationFilter` reads this header, validates the token and tells Spring Security who the user is. If nobody is logged in the header becomes `Bearer null`; the filter then fails to validate it and treats the call as anonymous, which is fine for the public `GET /api/driver`. `private` means only this class uses it.

Lines 20-22 | getAllDrivers

```
20 | getAllDrivers(): Observable<Driver[]> {
21 |   return this.http.get<Driver[]>(`${this.apiUrl}/api/driver`, { headers: this.getHeaders() });
22 | }
```

`GET /api/driver` returns `Driver[]` (an array of drivers); answered by `DriverController.viewAllDrivers`. Public in `SecurityConfig` (customers and admins both see the list). The server answers 204 No Content with an empty body when there are no drivers. The generic `<Driver[]>` tells TypeScript what the JSON contains. `{ headers: this.getHeaders() }` is the options object passed as the last argument.

Lines 24-26 | getDriverById

```
24 | getDriverById(driverId: number): Observable<Driver> {
25 |   return this.http.get<Driver>(`${this.apiUrl}/api/driver/${driverId}`, { headers: this.getHeaders() });
26 | }
```

`GET /api/driver/{id}`; `DriverController.viewDriverById` (404 if missing). ADMIN only. Used by the driver-management page to fill the form in edit mode. The id is placed into the URL with `${driverId}` in a template string.

Lines 28-30 | addDriver

```
28 | addDriver(driver: Driver): Observable<Driver> {
29 |   return this.http.post<Driver>(`${this.apiUrl}/api/driver`, driver, { headers: this.getHeaders() });
30 | }
```

`POST /api/driver` with the driver as JSON body; `DriverController.addDriver` saves it, or answers 409 on a duplicate licence number. ADMIN only.

Lines 32-34 | updateDriver

```
32 | updateDriver(driverId: number, driver: Driver): Observable<Driver> {
33 |   return this.http.put<Driver>(`${this.apiUrl}/api/driver/${driverId}`, driver, { headers: this.getHeaders() });
34 | }
```

`PUT /api/driver/{id}` with the changed driver; `DriverController.updateDriver`. ADMIN only. PUT means "replace/update this existing resource".

Lines 36-39 | deleteDriver

```
36 | deleteDriver(driverId: number): Observable<void> {
37 |   return this.http.delete<void>(`${this.apiUrl}/api/driver/${driverId}`, { headers: this.getHeaders() });
38 | }
39 | }
```

`DELETE /api/driver/{id}`; `DriverController.deleteDriver`. Returns `Observable<void>` because no body is expected. If the driver has requests the server refuses (a `DriverDeletionException` handled by `GlobalExceptionHandler`). ADMIN only. Line 39 closes the class.

Workflow, next: Driver calls are defined. The same pattern is used for trip requests in `src/app/services/driver-request.service.ts`.

Step 68: DriverRequestService - REST calls for trip requests [Arjun - Driver & Booking]

FRONTEND

angularapp/src/app/services/driver-request.service.ts

What this file is	The HTTP client for booking requests. It has seven methods, one for each endpoint of <code>DriverRequestController</code> . Its structure is identical to <code>DriverService</code> (injectable singleton, <code>getHeaders()</code> with the JWT, one Observable per call).
Who / what triggers it	Injected into the customer request form, the customer's request list and the admin request page.
What it uses / calls	<code>HttpClient</code> , <code>HttpHeaders</code> , <code>localStorage</code> , <code>apiUrl</code> and the <code>DriverRequest</code> model.
Size	47 lines

Lines 1-5 | Imports

```
1 | import { Injectable } from '@angular/core';
2 | import { HttpClient, HttpHeaders } from '@angular/common/http';
3 | import { Observable } from 'rxjs';
4 | import { apiUrl } from '../../apiconfig';
5 | import { DriverRequest } from '../models/driver-request.model';
```

The same set as `DriverService`, with the `DriverRequest` model.

Lines 7-18 | Class, constructor and getHeaders

```
7 | @Injectable({
8 |   providedIn: 'root'
9 | })
10 | export class DriverRequestService {
11 |   public apiUrl: string = apiUrl;
12 |
13 |   constructor(private http: HttpClient) {}
14 |
15 |   private getHeaders(): HttpHeaders {
16 |     const token = localStorage.getItem('token');
17 |     return new HttpHeaders({ Authorization: `Bearer ${token}` });
18 |   }
```

Single shared service (`providedIn: 'root'`), the base URL property, the injected `HttpClient`, and the private `getHeaders()` that returns `Authorization: Bearer <token>` read from `localStorage` (explained in `driver.service.ts`).

Lines 20-22 | getAllDriverRequests

```
20 |   getAllDriverRequests(): Observable<DriverRequest[]> {
21 |     return this.http.get<DriverRequest[]>(`${this.apiUrl}/api/driverRequest`, { headers: this.getHeaders() });
22 |   }
```

`GET /api/driverRequest; DriverRequestController.viewAllDriverRequests` (204 if there are none). ADMIN only: the admin request page lists every request.

Lines 24-26 | getDriverRequestById

```
24 |   getDriverRequestById(driverRequestId: number): Observable<DriverRequest> {
25 |     return this.http.get<DriverRequest>(`${this.apiUrl}/api/driverRequest/${driverRequestId}`, { headers: this.getHeaders() });
26 |   }
```

`GET /api/driverRequest/{id}; viewDriverRequestById`. Any logged-in user. The request form uses it to fill the fields in edit mode.

Lines 28-30 | getDriverRequestsByUserId

```
28 |   getDriverRequestsByUserId(userId: number): Observable<DriverRequest[]> {
29 |     return this.http.get<DriverRequest[]>(`${this.apiUrl}/api/driverRequest/user/${userId}`, { headers: this.getHeaders() });
30 |   }
```

`GET /api/driverRequest/user/{userId}; viewDriverRequestsByUserId` (404 if the user has none). CUSTOMER only: "my requests" page.

Lines 32-34 | `getDriverRequestsByDriverId`

```
32 |   getDriverRequestsByDriverId(driverId: number): Observable<DriverRequest[]> {  
33 |     return this.http.get<DriverRequest[]>(`${this.apiUrl}/api/driverRequest/driver/${driverId}`, { headers: this.getHeaders() });  
34 |   }
```

`GET /api/driverRequest/driver/{driverId}`; `viewDriverRequestsByDriverId`. ADMIN only. It is defined for completeness; the pages mainly use the first three.

Lines 36-38 | `addDriverRequest`

```
36 |   addDriverRequest(driverRequest: DriverRequest): Observable<DriverRequest> {  
37 |     return this.http.post<DriverRequest>(`${this.apiUrl}/api/driverRequest`, driverRequest, { headers: this.getHeaders() });  
38 |   }
```

`POST /api/driverRequest` with the new request as JSON (`{ user: { userId }, driver: { driverId }, ... }`); `addDriverRequest` in the controller saves it and sets `requestDate` if empty. CUSTOMER only.

Lines 40-42 | `updateDriverRequest`

```
40 |   updateDriverRequest(driverRequestId: number, driverRequest: DriverRequest): Observable<DriverRequest> {  
41 |     return this.http.put<DriverRequest>(`${this.apiUrl}/api/driverRequest/${driverRequestId}`, driverRequest, { headers: this.getHeaders() });  
42 |   }
```

`PUT /api/driverRequest/{id}`; `updateDriverRequest` merges only the non-null fields sent. Used by the customer (edit while Pending, Trip End with the calculated fare) and by the admin (Approve, Reject, Close). Any logged-in user.

Lines 44-47 | `deleteDriverRequest`

```
44 |   deleteDriverRequest(driverRequestId: number): Observable<void> {  
45 |     return this.http.delete<void>(`${this.apiUrl}/api/driverRequest/${driverRequestId}`, { headers: this.getHeaders() });  
46 |   }  
47 | }
```

`DELETE /api/driverRequest/{id}`; `deleteDriverRequest`. CUSTOMER only, and the UI allows it only while the status is Pending. Line 47 closes the class.

Workflow, next: Request calls are defined. Feedback calls follow in `src/app/services/feedback.service.ts`.

Step 69: FeedbackService - REST calls for feedback

[Saumya - Feedback & AI]

`angularapp/src/app/services/feedback.service.ts`

FRONTEND

What this file is	The HTTP client for customer feedback. It follows the same pattern as the other services and has four methods for <code>FeedbackController</code> .
Who / what triggers it	Injected into <code>customerpostfeedback</code> , <code>customerviewfeedback</code> and <code>adminviewfeedback</code> .
What it uses / calls	<code>HttpClient</code> , <code>HttpHeaders</code> , <code>localStorage</code> , <code>apiUrl</code> and the <code>Feedback</code> model.
Size	35 lines

Lines 1-5 | Imports

```
1 | import { Injectable } from '@angular/core';  
2 | import { HttpClient, HttpHeaders } from '@angular/common/http';  
3 | import { Observable } from 'rxjs';  
4 | import { apiUrl } from '../../apiconfig';  
5 | import { Feedback } from '../models/feedback.model';
```

The same set as the other services, with the `Feedback` model.

Lines 7-18 | Class, constructor and getHeaders

```
7 | @Injectable({
8 |   providedIn: 'root'
9 | })
10 | export class FeedbackService {
11 |   public apiUrl: string = apiUrl;
12 |
13 |   constructor(private http: HttpClient) {}
14 |
15 |   private getHeaders(): HttpHeaders {
16 |     const token = localStorage.getItem('token');
17 |     return new HttpHeaders({ Authorization: `Bearer ${token}` });
18 |   }
```

Singleton service, base URL, the injected `HttpClient` and the private `getHeaders()` that adds the Bearer token.

Lines 20-22 | sendFeedback

```
20 |   sendFeedback(feedback: Feedback): Observable<Feedback> {
21 |     return this.http.post<Feedback>(`${this.apiUrl}/api/feedback`, feedback, { headers: this.getHeaders() });
22 |   }
```

`POST /api/feedback` with the feedback as JSON (`{ feedbackText, rating, category, date, user: { userId }, driver: { driverId } }`); `FeedbackController.createFeedback` stores it and calls `AIService.applyAnalysis`, which adds sentiment, score and tags. The returned `Feedback` already contains the AI fields. CUSTOMER only.

Lines 24-26 | getAllFeedbacksByUserId

```
24 |   getAllFeedbacksByUserId(userId: number): Observable<Feedback[]> {
25 |     return this.http.get<Feedback[]>(`${this.apiUrl}/api/feedback/user/${userId}`, { headers: this.getHeaders() });
26 |   }
```

`GET /api/feedback/user/{userId}`; `viewFeedbacksByUser` (404 when the customer has no feedback yet). CUSTOMER only: the "my feedback" page.

Lines 28-30 | deleteFeedback

```
28 |   deleteFeedback(feedbackId: number): Observable<void> {
29 |     return this.http.delete<void>(`${this.apiUrl}/api/feedback/${feedbackId}`, { headers: this.getHeaders() });
30 |   }
```

`DELETE /api/feedback/{id}`; `deleteFeedback`. CUSTOMER only. Returns nothing (`void`).

Lines 32-35 | getFeedbacks

```
32 |   getFeedbacks(): Observable<Feedback[]> {
33 |     return this.http.get<Feedback[]>(`${this.apiUrl}/api/feedback`, { headers: this.getHeaders() });
34 |   }
35 | }
```

`GET /api/feedback`; `viewAllFeedbacks`. Public in `SecurityConfig`. The admin feedback page loads all feedback with this call. Line 35 closes the class. (The back end also has `GET /api/feedback/{id}`, but no front-end method calls it.)

Workflow, next: Feedback calls are defined. The last service is the AI one: `src/app/services/ai.service.ts`.

Step 70: AIService - AI driver search and feedback summary [Saumya - Feedback & AI]

angularapp/src/app/services/ai.service.ts

FRONTEND

What this file is	The HTTP client for the two AI features on the back end (<code>AiController</code>): searching drivers with a natural-language sentence, and getting an AI-written summary of a driver's feedback. The back end uses Google Gemini when a key is configured and otherwise a local keyword method, so the Angular code is identical in both cases.
Who / what triggers it	Injected into <code>customerviewdriver</code> (search box) and <code>adminviewfeedback</code> (summary pop-up).
What it uses / calls	<code>HttpClient</code> , <code>HttpHeaders</code> , <code>localStorage</code> , <code>apiUrl</code> and the models <code>Driver</code> and <code>DriverFeedbackSummary</code> .
Size	33 lines

Lines 1-6 | Imports

```
1 | import { Injectable } from '@angular/core';
2 | import { HttpClient, HttpHeaders } from '@angular/common/http';
3 | import { Observable } from 'rxjs';
4 | import { apiUrl } from '../..//apiconfig';
5 | import { Driver } from '../models/driver.model';
6 | import { DriverFeedbackSummary } from '../models/ai.model';
```

The standard service imports plus two models: **Driver** (the search result type) and **DriverFeedbackSummary** (the summary type from **ai.model.ts**).

Lines 8-19 | Class, constructor and getHeaders

```
8 | @Injectable({
9 |   providedIn: 'root'
10 | })
11 | export class AiService {
12 |   public apiUrl: string = apiUrl;
13 |
14 |   constructor(private http: HttpClient) {}
15 |
16 |   private getHeaders(): HttpHeaders {
17 |     const token = localStorage.getItem('token');
18 |     return new HttpHeaders({ Authorization: `Bearer ${token}` });
19 |   }
```

A singleton service (**providedIn: 'root'**) with the base URL, the injected **HttpClient** and the private **getHeaders()** that reads the JWT from **localStorage** and builds **Authorization: Bearer <token>**.

Lines 21-25 | searchDrivers - the query as plain text

```
21 | /** Semantic driver search: the natural-language query is sent as the request body. */
22 | searchDrivers(query: string): Observable<Driver[]> {
23 |   const headers = this.getHeaders().set('Content-Type', 'text/plain');
24 |   return this.http.post<Driver[]>(`${this.apiUrl}/api/ai/driver-search`, query, { headers });
25 | }
```

POST /api/ai/driver-search; **AiController.searchDrivers** takes **@RequestBody String query**, i.e. the **raw text** and not JSON. Therefore **this.getHeaders().set('Content-Type', 'text/plain')** adds a **Content-Type** header (**HttpHeaders** is immutable, so **set** returns a new object), and the query string is passed directly as the body. Angular would otherwise send a text body as **text/plain** anyway, but the explicit header makes it certain. **{ headers }** is shorthand for **{ headers: headers }**. The call is public in **SecurityConfig**, returns **Driver[]** ranked by similarity, and the interceptor ignores its errors (it skips **/api/ai/** URLs) so the page can fall back to normal search.

Lines 27-33 | getDriverFeedbackSummary

```
27 | getDriverFeedbackSummary(driverId: number): Observable<DriverFeedbackSummary> {
28 |   return this.http.get<DriverFeedbackSummary>(
29 |     `${this.apiUrl}/api/ai/driver/${driverId}/feedback-summary`,
30 |     { headers: this.getHeaders() }
31 |   );
32 | }
33 | }
```

GET /api/ai/driver/{id}/feedback-summary; **AiController.getFeedbackSummary** calls **AiService.summarizeDriverFeedback**. Allowed for **ADMIN** or **CUSTOMER**. The **Observable** delivers a **DriverFeedbackSummary** with summary text, strengths, improvements and scores. The call is written on several lines only for readability. Line 33 closes the class.

Workflow, next: All five services are in place. The last piece of the shared infrastructure is the route guard that uses **AuthService: src/app/components/authguard/authguard.guard.ts**.

Step 71: AuthGuard - protecting routes by login and role [Vanshika - Security]

FRONTEND

angularapp/src/app/components/authguard/authguard.guard.ts

What this file is	A route guard is a class the router asks "may this user open this route?" before it creates the page component. AuthGuard implements CanActivate . It rejects visitors who are not logged in (or whose token has expired) and logged-in users whose role does not match the route's data.role (for example a customer typing /adminviewrequests in the address bar). In both cases it redirects to the login page. This is only a convenience for the user: the real security is on the server, where SecurityConfig checks the JWT and role on every API call.
Who / what triggers it	The router, whenever it is about to activate a route that lists canActivate: [AuthGuard] in app-routing.module.ts : all routes except login , signup , error , the empty path and the wildcard.
What it uses / calls	AuthService.isLoggedIn() and AuthService.getUserRole() , Router.createUrlTree , and the route's data object.
Size	28 lines

Lines 1-3 | Imports

```
1 | import { Injectable } from '@angular/core';
2 | import { ActivatedRouteSnapshot, CanActivate, Router, UrlTree } from '@angular/router';
3 | import { AuthService } from '../../services/auth.service';
```

Injectable for dependency injection. **ActivatedRouteSnapshot** is a read-only description of the route being opened (its **data**, parameters and URL), **CanActivate** is the guard interface, **Router** creates redirect targets, and **UrlTree** is the type that represents a URL to navigate to. **AuthService** supplies the login information.

Lines 5-12 | Description, decorator and class

```
5 | /**
6 |  * Blocks manual navigation to pages the user may not see: anyone who is not logged in (or who
7 |  * is logged in with the wrong role for the page) is redirected to the login page.
8 |  */
9 | @Injectable({
10 |   providedIn: 'root'
11 | })
12 | export class AuthGuard implements CanActivate {
```

The comment summarises the rule. **@Injectable({ providedIn: 'root' })** creates one global instance so that the router can use it without a module entry. **implements CanActivate** requires a method **canActivate**. The class is called **AuthGuard**.

Line 13 | Constructor

```
13 |   constructor(private authService: AuthService, private router: Router) {}
```

Dependency injection supplies the **AuthService** singleton and the **Router**.

Lines 15-18 | canActivate - the login check

```
15 |   canActivate(route: ActivatedRouteSnapshot): boolean | UrlTree {
16 |     if (!this.authService.isLoggedIn()) {
17 |       return this.router.createUrlTree(['/login']);
18 |     }
19 |   }
```

The router calls **canActivate(route)** and uses its return value: **true** means "go ahead", a **UrlTree** means "cancel this navigation and go to this URL instead", and **false** would simply cancel. The return type **boolean | UrlTree** says exactly that. First **!this.authService.isLoggedIn()**: no token or an expired one. Then **this.router.createUrlTree(['/login'])** builds the target URL **/login** and returns it, so the router redirects there. Returning a **UrlTree** is the modern replacement for calling **router.navigate** inside a guard.

Lines 19-23 | canActivate - the role check

```
19 |     const requiredRole: string | undefined = route.data ? route.data['role'] : undefined;
20 |     const userRole = (this.authService.getUserRole() || '').toLowerCase();
21 |     if (requiredRole && userRole !== requiredRole.toLowerCase()) {
22 |       return this.router.createUrlTree(['/login']);
23 |     }
24 |   }
```

route.data is the **data: { role: 'Admin' }** object from the route table; **route.data ? route.data['role'] : undefined** reads it safely (the **home** route has no **data**, so **requiredRole** is **undefined**). **userRole** is the stored role in lower case (**|| ''** protects against **null**). If the route demands a role and it differs from the user's role (compared

in lower case, so 'Admin' equals 'admin'), the user is sent to `/login`. For example a Customer opening `/driver-management` fails this test.

Lines 24-26 | Allow the navigation

```
24 |     return true;
25 |   }
26 | }
```

If both checks passed, `return true` lets the router create the component. Lines 25-26 close the method and the class.

Line 28 | Alias export

```
28 | export { AuthGuard as AuthguardGuard };
```

`export { AuthGuard as AuthguardGuard };` also exports the class under a second name, `AuthguardGuard`, which is the name Angular's generator would have chosen for a file called `authguard.guard.ts`. No file in the app uses the alias; it is a harmless leftover that keeps either spelling working.

Workflow, next: The guard completes the shared infrastructure of the Angular app: the app shell is ready, the models and services exist and the routes are defined. The router now looks at the browser address, which is the empty path `/`. The matching route is the redirect `{ path: '', redirectTo: 'login', pathMatch: 'full' }`, so the router changes the URL to `/login`. The `AuthGuard` is **not involved** there, because the `login` route has no `canActivate`. The `LoginComponent` is therefore created and shown inside the `router-outlet`, and since nobody is logged in, no navigation bar is displayed. The first page the user sees is the login page, whose code is in `angularapp/src/app/components/login/login.component.ts` (the first file of the next part).

Part 3 - The user journey: page by page

Now a real person uses the app. We follow the pages in the order they are met: login and signup, the home page and navigation bars, then the Admin driver pages, the Customer driver and booking pages, the request approval and trip-end pages, and finally the feedback pages and the error page.

For every page you see three files: the **.ts** (the logic), the **.html** (the structure) and the **.css** (the look). Whenever the logic calls the backend, the text names the exact URL and the backend method that answers it, so you can jump back to Part 1.

Route	Page (component)	Who may open it	Backend calls
/login	LoginComponent	everyone	POST /api/login
/signup	SignupComponent	everyone	POST /api/register
/home	HomePageComponent	any logged-in user	-
/driver-management[:id]	DriverManagementComponent	Admin	POST/PUT /api/driver, GET /api/driver/{id}
/admin-view-drivers	AdminViewDriversComponent	Admin	GET/PUT/DELETE /api/driver
/adminviewrequests	AdminviewrequestsComponent	Admin	GET /api/driverRequest, PUT /api/driverRequest/{id}
/adminviewfeedback	AdminviewfeedbackComponent	Admin	GET /api/feedback, GET /api/ai/driver/{id}/feedback-summary
/customerviewdriver	CustomerviewdriverComponent	Customer	GET /api/driver, GET /api/driverRequest/user/{id}, POST /api/ai/driver-search
/customer-request/:driverId and /edit/:id	CustomerRequestComponent	Customer	POST /api/driverRequest, GET/PUT /api/driverRequest/{id}
/customerviewrequested	CustomerviewrequestedComponent	Customer	GET /api/driverRequest/user/{id}, PUT, DELETE
/customerpostfeedback	CustomerpostfeedbackComponent	Customer	POST /api/feedback
/customerviewfeedback	CustomerviewfeedbackComponent	Customer	GET /api/feedback/user/{id}, DELETE /api/feedback/{id}
/error and anything else	ErrorComponent	everyone	-

Step 72: LoginComponent (TypeScript) - the login page logic [Aleena - Authentication]

FRONTEND

angularapp/src/app/components/login/login.component.ts

What this file is

The browser has just landed on `/login` (the route table redirects the empty path to it, and `AuthGuard` sends anybody who is not logged in here). Angular therefore creates one object of the class `LoginComponent` and shows its template next to it.

A **component** is Angular's basic building block of a screen: one TypeScript class (the data and the behaviour), one HTML template (what is drawn) and one CSS file (how it looks). This file is the class. It owns a **reactive form** (a form whose fields, rules and values live in TypeScript code instead of being hidden inside the HTML), reads what the user typed, validates it, and sends it to the backend through `AuthService`.

Who / what triggers it

The Angular router. When the URL is `/login` the router looks up `{ path: 'login', component: LoginComponent }` in `app-routing.module.ts`, creates the component (calling its constructor) and puts its HTML where `<router-outlet>` sits in `app.component.html`. After that, user actions (typing, clicking the eye icon, pressing the Log In button) call the methods below.

What it uses / calls	<code>FormBuilder</code> , <code>FormGroup</code> and <code>Validators</code> from <code>@angular/forms</code> (reactive forms), <code>Router</code> (to change page), <code>AuthService</code> (does the HTTP call and stores the session), the <code>Login</code> model class, and the <code>EMAIL_REGEX</code> constant from <code>utils/constants.ts</code> . On the server side the call finally reaches <code>AuthController</code> , <code>SecurityConfig</code> , <code>MyUserDetailsService</code> , <code>UserPrinciple</code> and <code>JwtUtils</code> .
Size	60 lines

Lines 1-6 | Imports

```

1 | import { Component } from '@angular/core';
2 | import { FormBuilder, FormGroup, Validators } from '@angular/forms';
3 | import { Router } from '@angular/router';
4 | import { AuthService } from '../services/auth.service';
5 | import { Login } from '../models/login.model';
6 | import { EMAIL_REGEX } from '../utils/constants';

```

Every `import` makes a name from another file available here.

- `Component` is the decorator (a special label, written with `@`) that turns a plain class into an Angular component.
- `FormBuilder` is a helper that builds forms with short code, `FormGroup` is the type of a form object (a group of fields), and `Validators` is a toolbox of ready-made rules such as `required`, `minLength` and `pattern`.
- `Router` lets code navigate to another page, e.g. `this.router.navigate(['/home'])`.
- `AuthService` is the service that talks to the backend (it is a shared object created once by Angular).
- `Login` is the small model class with `email` and `password`; it is used here only as a type, so the compiler can check the shape of the credentials.
- `EMAIL_REGEX` is the regular expression `^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$` defined in `utils/constants.ts`: some text, an `@`, some text, a dot and more text, with no spaces. It is a quick sanity check, not a full e-mail standard.

Lines 8-12 | @Component decorator

```

8 | @Component({
9 |   selector: 'app-login',
10 |   templateUrl: './login.component.html',
11 |   styleUrls: ['./login.component.css']
12 | })

```

`@Component({...})` attaches settings to the class below it.

- `selector: 'app-login'` is the custom HTML tag name for this component. Here the router creates it itself, so the tag is never typed by hand, but the name still has to exist.
- `templateUrl` points to `login.component.html`, the page layout.
- `styleUrls` points to `login.component.css`. Angular **scopes** these styles to this component only (it adds a hidden attribute such as `_ngcontent-abc` to every element and to every CSS rule), so a rule written here cannot leak out and change another page.

Lines 13-17 | The class and its fields

```

13 | export class LoginComponent {
14 |   loginForm: FormGroup;
15 |   submitted: boolean = false;
16 |   showPassword: boolean = false;
17 |   errorMessage: string = '';

```

`export class LoginComponent` makes the class importable by `app.module.ts` (where it is declared) and by the routing module.

The four fields are the **state** of the page. The template reads them and re-draws itself whenever Angular notices they have changed.

- `loginForm: FormGroup` is the form object. It is only declared here; it gets its value in the constructor.
- `submitted` is `false` until the user presses Log In once. It lets the page show error messages for empty fields only after a first attempt.
- `showPassword` decides whether the password box shows dots or plain text (the eye button toggles it).
- `errorMessage` holds the red text under the form when the server rejects the login. An empty string means nothing is shown.

Lines 19-24 | Constructor and the reactive form

```

19 | constructor(private fb: FormBuilder, private authService: AuthService, private router: Router) {
20 |   this.loginForm = this.fb.group({
21 |     email: ['', [Validators.required, Validators.pattern(EMAIL_REGEX)]],
22 |     password: ['', [Validators.required, Validators.minLength(6)]]
23 |   });
24 | }

```

The constructor runs once, when the router creates the component. The three parameters use **dependency injection**: Angular sees the types `FormBuilder`, `AuthService` and `Router`, finds or creates the matching objects and passes them in. Writing `private` in front turns each parameter into a field (`this.fb`, `this.authService`, `this.router`) automatically.

`this.fb.group({...})` builds the form. Every key is a **form control** (one input). Each value is an array `[initialValue, [validators]]`.

- `email: ['', [Validators.required, Validators.pattern(EMAIL_REGEX)]]` starts empty and is valid only when it is not empty AND matches the e-mail pattern.
- `password: ['', [Validators.required, Validators.minLength(6)]]` starts empty and needs at least 6 characters.

A validator does not stop the user typing; it only marks the control **valid** or **invalid** and records which rule failed in `control.errors` (for example `{ required: true }` or `{ pattern: {...} }`). The group as a whole is invalid if any control is invalid, which is what `onSubmit` checks.

Lines 26-32 | Getters email and password

```
26 |   get email() {
27 |     return this.loginForm.get('email')!;
28 |   }
29 |
30 |   get password() {
31 |     return this.loginForm.get('password')!;
32 |   }
```

A **getter** (`get email()`) is a method that is used like a property: the template can write `email.errors` instead of `loginForm.get('email').errors`. `this.loginForm.get('email')` returns the control called `email`; it can return `null` for an unknown name, and the `!` after it is TypeScript's "trust me, it is not null" mark, which is safe because the name is always one of the two defined in the constructor.

Lines 34-37 | isValid - when to show an error

```
34 |   isValid(field: 'email' | 'password'): boolean {
35 |     const control = this.loginForm.get(field)!;
36 |     return control.invalid && (control.touched || this.submitted);
37 |   }
```

The template calls `isValid('email')` and `isValid('password')` to decide whether to draw the red message. The parameter type `'email' | 'password'` only allows those two words, so a typing mistake is caught by the compiler.

The message appears when the control is **invalid** AND the user has either **touched** it (clicked in and left again, Angular's `touched` flag) or already tried to submit (`this.submitted`). This avoids showing "Email address is required" on a freshly opened page before the user has done anything.

Lines 39-41 | togglePassword - the eye button

```
39 |   togglePassword(): void {
40 |     this.showPassword = !this.showPassword;
41 |   }
```

Flips `showPassword` between **false** and **true**. The template binds the input's **type** to this flag, so one click changes the field between `password` (dots) and `text` (readable), and the next click changes it back.

Lines 43-48 | onSubmit - validation first

```
43 |   onSubmit(): void {
44 |     this.submitted = true;
45 |     this.errorMessage = '';
46 |     if (this.loginForm.invalid) {
47 |       return;
48 |     }
```

`onSubmit()` is called when the form is submitted (the `(ngSubmit)` event in the HTML, triggered by the Log In button or by pressing Enter).

- `this.submitted = true` makes every invalid field show its error from now on.
- `this.errorMessage = ''` clears an old server error from a previous attempt.
- `if (this.loginForm.invalid) { return; }` stops here if any rule is broken. No request is sent, and the user sees the red messages next to the fields. This is the first of two checks: it is a convenience for the user, the server checks everything again.

Lines 49-51 | Send the credentials and follow them to the backend

```
49 |     const credentials: Login = { email: this.email.value.trim(), password: this.password.value };
50 |     this.authService.login(credentials).subscribe({
51 |       next: () => this.router.navigate(['/home']),
```

`const credentials: Login = {...}` builds the JSON body: the e-mail with leading and trailing spaces removed (`trim()`), and the password exactly as typed (never trimmed, because spaces can be part of a password). `.value` reads the current text of a control.

`this.authService.login(credentials)` returns an **Observable**: a description of an HTTP request that has not been sent yet. Nothing happens until somebody calls `.subscribe(...)`, which starts the request and registers what to do with the answer. `next` runs when a successful answer arrives; here it runs `this.router.navigate(['/home'])`.

What happens on the way (the whole login journey)

- `AuthService.login` sends `POST http://localhost:8080/api/login` with body `{"email": "...", "password": "..."}.` The browser first sends a CORS pre-check because port 8081 and port 8080 are different origins; `CorsConfig` answers that all origins, methods and headers are allowed.
- In Spring Security's filter chain `JwtAuthenticationFilter` runs first (registered in `SecurityConfig` line 160). This request has no `Authorization: Bearer` header yet, so the filter does nothing and passes it on. Then `SecurityConfig` line 127, `.requestMatchers("/api/register", "/api/login").permitAll()`, lets the anonymous request through. Without that line nobody could ever log in, because logging in requires no token.
- Spring MVC calls `AuthController.authenticateUser(@RequestBody User loginRequest)`. Jackson converts the JSON into a `User` object (only `email` and `password` are filled in).
- The controller calls `authenticationManager.authenticate(new UsernamePasswordAuthenticationToken(email, password))`. The `AuthenticationManager` bean is defined in `SecurityConfig` and hands the work to the `DaoAuthenticationProvider` bean, which was configured with our `MyUserDetailsService` and a `BCryptPasswordEncoder`.
- The provider calls `MyUserDetailsService.loadUserByUsername(email)`. That runs `UserRepo.findByEmail(email)` (a database SELECT on the `users` table) and, if a row exists, wraps it with `UserPrincipal.build(user)`. `UserPrincipal` is Spring Security's view of our user: it exposes the e-mail as the username, the stored password hash, and one authority named `ROLE_` plus the role in capitals (`ROLE_ADMIN` or `ROLE_CUSTOMER`). Those authority names are what `hasRole("ADMIN")` in `SecurityConfig` checks later. If no user has that e-mail an exception is thrown.
- The provider then compares the typed password with the stored hash using `BCrypt` (`passwordEncoder.matches`). `BCrypt` is a one-way hashing method; the database never holds the real password, so the check hashes the typed text with the same salt and compares the result. A mismatch throws `BadCredentialsException`.
- On success `JwtUtils.generateJwtToken(authentication)` builds a **JWT** (JSON Web Token): a signed text made of three parts (header, claims, signature). The claims hold the e-mail (subject), the issue time and an expiry 24 hours later; the signature uses HS512 and a secret key that only the server knows, so nobody can forge or alter a token.
- The controller reads the user back with `((UserPrincipal) authentication.getPrincipal()).getUser()` and builds a `LoginDTO(token, username, userRole, userId)` which is returned with status 200 OK. Any exception inside the `try` (unknown e-mail, wrong password) is caught and answered with `401 Unauthorized` and the text "Invalid credentials".

Back in the browser: `HttpErrorInterceptor` lets the answer pass (it ignores `/api/login` on purpose, so a wrong password stays on this page), then `AuthService`'s `tap` stores the session (see `auth.service.ts`: the four `localStorage` keys `token`, `userRole`, `userId`, `username` are written and the three `BehaviorSubjects` are updated). Only after that does `next` run and the router move to `/home`.

Lines 52-60 | Error handler

```
52 |         error: (err) => {
53 |             // 401 = wrong credentials. Server failures are routed to the error page by the interceptor.
54 |             if (err && (err.status === 401 || err.status === 403)) {
55 |                 this.errorMessage = 'Invalid email address or password';
56 |             }
57 |         }
58 |     });
59 | }
60 | }
```

`error:` runs when the server answers with an error status (or the request fails). `err.status` is the HTTP status code.

- If the status is `401` (what `AuthController` sends for wrong credentials) or `403`, the page sets `errorMessage = 'Invalid email address or password'`. The template then shows it in a red line under the form. The text is deliberately the same for "unknown e-mail" and "wrong password" so that an attacker cannot learn which e-mails exist.
- The comment on line 53 says server failures are sent to the error page by the interceptor. Be aware that `HttpErrorInterceptor` explicitly skips `/api/login` and `/api/register` (its `isAuthCall` test), so for this call a

server crash (500) or a stopped backend (status 0) is NOT redirected to `/error`: the user simply stays on the login page and no message is shown. The comment is therefore not exactly true for login; it describes the behaviour of the other pages.

The remaining lines close the `subscribe` call `{})`), the `onSubmit` method and the class.

Workflow, next: The class is ready, so Angular now needs the page that goes with it: the template `login.component.html`. It binds to `loginForm`, `isInvalid`, `togglePassword`, `onSubmit` and `errorMessage` that were just defined.

Step 73: LoginComponent (template) - what the user sees on `/login` [Aleena - Authentication]

FRONTEND

`angularapp/src/app/components/login/login.component.html`

What this file is	The HTML of the login page: a card split in two halves. The left half is a picture panel with the word "DriveU"; the right half holds the form with an e-mail box, a password box with a show/hide eye, a Log In button and a link to the Sign Up page. It is an Angular template : normal HTML plus special syntax that connects it to the fields and methods of <code>LoginComponent</code> .
Who / what triggers it	Angular renders it when <code>LoginComponent</code> is created for the route <code>/login</code> . It is re-rendered automatically each time a field in the class changes (for example <code>errorMessage</code> , <code>showPassword</code>) or a control becomes touched.
What it uses / calls	<code>LoginComponent</code> (fields <code>loginForm</code> , <code>showPassword</code> , <code>errorMessage</code> , getters <code>email</code> and <code>password</code> , methods <code>isInvalid</code> , <code>togglePassword</code> , <code>onSubmit</code>), <code>ReactiveFormsModule</code> (provides <code>[formGroup]</code> and <code>formControlName</code>), <code>CommonModule</code> (provides <code>*ngIf</code>), <code>RouterModule</code> (provides <code>routerLink</code>), and the global classes <code>auth-*</code> in <code>styles.css</code> .
Size	38 lines

Lines 1-5 | Outer page and the picture panel

```
1 | <div class="auth-page">
2 |   <div class="auth-card">
3 |     <div class="auth-image">
4 |       <h1 class="auth-heading">DriveU</h1>
5 |     </div>
```

`.auth-page` is the full-screen grey background that centres everything; `.auth-card` is the white rounded box with two columns; `.auth-image` is the left column whose background picture (`background.webp`) comes from the global CSS; `<h1 class="auth-heading">DriveU</h1>` is the white brand text over that picture. These classes are shared with the signup page and are defined once in `styles.css`.

Lines 7-9 | The form element

```
7 |   <div class="auth-panel">
8 |     <form class="auth-form" id="loginForm" [formGroup]="loginForm" (ngSubmit)="onSubmit()" novalidate>
9 |       <h2>Log In</h2>
```

`.auth-panel` is the right column. The `<form>` tag carries several Angular features:

- `[formGroup]="loginForm"`: square brackets mean **property binding**. The HTML form is tied to the `loginForm` object of the class, so every field inside is connected to a control of that group.
- `(ngSubmit)="onSubmit()"`: round brackets mean **event binding**. When the form is submitted (button of type submit, or Enter), Angular calls `onSubmit()`. Angular's `ngSubmit` is used instead of the browser's own submit because it does not reload the page.
- `novalidate` turns off the browser's built-in bubbles ("Please fill out this field"), so only our own red messages appear.
- `id="loginForm"` and the other `id` values below are fixed hooks (used by automated tests and for easy selection); they do not affect behaviour.

`<h2>Log In</h2>` is the card title.

Lines 11-17 | E-mail field and its errors

```
11 |         <div class="auth-field">
12 |             <input type="email" id="email" class="auth-input" placeholder="Email" formControlName="email" autocomplete="
username" />
13 |             <div class="auth-error" id="emailError" *ngIf="isInvalid('email')">
14 |                 <span *ngIf="email.errors?.['required']">Email address is required</span>
15 |                 <span *ngIf="!email.errors?.['required'] && email.errors?.['pattern']">Please enter a valid email address<
/ span>
16 |             </div>
17 |         </div>
```

`formControlName="email"` connects this `<input>` to the control named `email` in the group: whatever the user types is written into `loginForm.value.email`, and the control's validity is updated at each key press. `type="email"` gives a suitable mobile keyboard, `placeholder` is the grey hint, and `autocomplete="username"` tells the browser's password manager that this is the account name field.

Below it, `<div class="auth-error" *ngIf="isInvalid('email')">` is a **structural directive**: the `div` is added to the page only while the expression is true (after the field was touched or submit was tried, and the field is wrong). Inside it two `<span>` elements pick which sentence to show:

- `*ngIf="email.errors?.['required']"` shows "Email address is required" when the box is empty. The `?.` means "only read it if `errors` is not null" (valid controls have `errors` equal to `null`).
- `*ngIf="!email.errors?.['required'] && email.errors?.['pattern']"` shows "Please enter a valid email address" when it is not empty but fails the regular expression. The first half prevents both messages from showing at once.

Lines 19-23 | Password field with the show/hide button

```
19 |         <div class="auth-field">
20 |             <div class="password-wrap">
21 |                 <input [type]="showPassword ? 'text' : 'password'" id="password" class="auth-input" placeholder="Password"
formControlName="password" autocomplete="current-password" />
22 |                 <button type="button" class="eye-btn" id="togglePassword" (click)="togglePassword()" [attr.aria-label]="sh
owPassword ? 'Hide password' : 'Show password'">&#128065;</button>
23 |             </div>
```

`.password-wrap` is a positioned box so the eye button can sit on the right edge of the input (see `.password-wrap` and `.eye-btn` in `styles.css`).

- `[type]="showPassword ? 'text' : 'password'"` is a property binding with a conditional (ternary) expression: it sets the input's type to `text` when `showPassword` is true, otherwise `password`. This is the whole show/hide feature.
- `formControlName="password"` ties the box to the `password` control. `autocomplete="current-password"` tells the browser this is an existing password, so it can offer to fill it in.
- The `<button type="button" ... (click)="togglePassword()">` calls the method on each click. `type="button"` is important: without it a button inside a form would submit the form. `[attr.aria-label]` binds an HTML attribute (not a property) used by screen readers, and flips between "Hide password" and "Show password". `👁` is the HTML code for the eye symbol.

Lines 24-28 | Password errors

```
24 |         <div class="auth-error" id="passwordError" *ngIf="isInvalid('password')">
25 |             <span *ngIf="password.errors?.['required']">Password is required</span>
26 |             <span *ngIf="!password.errors?.['required'] && password.errors?.['minlength']">Password must be at least 6
characters long</span>
27 |         </div>
28 |     </div>
```

The same pattern as for the e-mail: the red block appears only when `isInvalid('password')` is true. "Password is required" is shown for an empty box; "Password must be at least 6 characters long" is shown when the box has text but the `minlength` rule failed. The `?.['minlength']` bracket form is needed because `errors` is an index-type object and strict TypeScript templates insist on this syntax.

Line 30 | Server error line

```
30 |         <div class="server-error" id="loginError" *ngIf="errorMessage">{{ errorMessage }}</div>
```

`*ngIf="errorMessage"` is true for any non-empty string, so the red `.server-error` line "Invalid email address or password" (set in `onSubmit`'s error handler) appears here and disappears again when `errorMessage` is cleared at the start of the next attempt. `{{ errorMessage }}` is **interpolation**: Angular prints the current value of the field as text.

Line 32 | Submit button

```
32 |         <button type="submit" class="auth-submit" id="loginButton">Log In</button>
```

The blue Log In button. Because it has `type="submit"` and sits inside the `<form>`, clicking it (or pressing Enter in an input) triggers the form's `ngSubmit` event and therefore `onSubmit()`. It has no `(click)` of its own.

Lines 34-38 | Sign Up link and closing tags

```
34 |         <p class="auth-link">Don't have an account? <a routerLink="/signup" id="signupLink">Sign Up</a></p>
35 |         </form>
36 |     </div>
37 | </div>
38 | </div>
```

`routerLink="/signup"` is the Angular way of making a link: it changes the route without reloading the page (a plain `href` would reload the whole app). Users without an account go to `SignupComponent`. The remaining lines close the form, `.auth-panel`, `.auth-card` and `.auth-page`.

Workflow, next: The template is complete. Its component-level stylesheet is next: `login.component.css` (only two lines, since nearly all the look comes from `styles.css`).

Step 74: LoginComponent (CSS) - component-level style

[Aleena - Authentication]

angularapp/src/app/components/login/login.component.css

FRONTEND

What this file is	The stylesheet attached to the login component through <code>styleUrls</code> . It is almost empty because both auth pages share one look that lives in the global <code>styles.css</code> .
Who / what triggers it	Angular loads it together with the component and scopes it to <code><app-login></code> only.
What it uses / calls	The global <code>styles.css</code> rules <code>.auth-page</code> , <code>.auth-card</code> , <code>.auth-image</code> , <code>.auth-heading</code> , <code>.auth-panel</code> , <code>.auth-form</code> , <code>.auth-field</code> , <code>.auth-input</code> , <code>.password-wrap</code> , <code>.eye-btn</code> , <code>.auth-error</code> , <code>.server-error</code> , <code>.auth-submit</code> , <code>.auth-link</code> .
Size	2 lines

Lines 1-2 | Comment and `:host` rule

```
1 | /* Layout of the two-panel auth card is shared with the signup page (see styles.css). */
2 | :host { display: block; }
```

Line 1 is a comment saying where the real styling is: the two-panel card (`.auth-page`, `.auth-card`, `.auth-image`, `.auth-panel`, `.auth-form`), the underlined input boxes (`.auth-input`), the red messages (`.auth-error`, `.server-error`), the blue button (`.auth-submit`) and the small media query that stacks the two halves vertically below 640 px width are all written in `src/styles.css`.

`:host { display: block; }` styles the component's own element, the `<app-login>` tag that Angular puts into the page. Custom tags are `display: inline` by default; making it `block` lets it fill the whole width so the `.auth-page` inside can centre the card properly.

Workflow, next: The login page is fully explained. A user without an account clicks the Sign Up link, so the next file is `signup.component.ts`. (A user who has an account types e-mail and password; after the journey described in `login.component.ts` the router goes to `/home`, `AuthGuard` lets the user in and `HomePageComponent` appears, which we reach right after the signup files.)

Step 75: SignupComponent (TypeScript) - registration page logic [Aleena - Authentication]

FRONTEND

angularapp/src/app/components/signup/signup.component.ts

What this file is	The class behind the <code>/signup</code> page. Like the login component it uses a reactive form, but with six fields (user name, e-mail, mobile number, password, confirm password and role) and one extra rule that compares two fields with each other. When the form is valid it sends a registration request, and on success it shows a small "Registration Successful" popup that leads back to the login page.
Who / what triggers it	The router, when the user opens <code>/signup</code> (for example by clicking the Sign Up link on the login page). The page needs no login, so the route has no guard.
What it uses / calls	<code>FormBuilder</code> , <code>FormGroup</code> , <code>Validators</code> , <code>AbstractControl</code> and <code>ValidationErrors</code> from <code>@angular/forms</code> ; <code>Router</code> ; <code>AuthService.register</code> ; the <code>User</code> model; <code>EMAIL_REGEX</code> and <code>MOBILE_REGEX</code> from <code>utils/constants.ts</code> . Backend: <code>AuthController.registerUser</code> and <code>UserServiceImpl.createUser</code> .
Size	82 lines

Lines 1-6 | Imports

```
1 | import { Component } from '@angular/core';
2 | import { AbstractControl, FormBuilder, FormGroup, ValidationErrors, Validators } from '@angular/forms';
3 | import { Router } from '@angular/router';
4 | import { AuthService } from '../services/auth.service';
5 | import { User } from '../models/user.model';
6 | import { EMAIL_REGEX, MOBILE_REGEX } from '../utils/constants';
```

Same idea as in the login component, plus two extra types: `AbstractControl` (the common parent type of a single control and of a whole group) and `ValidationErrors` (the shape of an error object, such as `{ mismatch: true }`), both needed to write our own validator. `User` is the model with `username`, `email`, `mobileNumber`, `password` and `userRole`. `MOBILE_REGEX` is `/^\d{10}$/`, which means exactly ten digits and nothing else.

Lines 8-12 | passwordsMatch - a custom group validator

```
8 | function passwordsMatch(group: AbstractControl): ValidationErrors | null {
9 |   const password = group.get('password')?.value;
10 |   const confirm = group.get('confirmPassword')?.value;
11 |   return confirm && password !== confirm ? { mismatch: true } : null;
12 | }
```

A **validator** is just a function that receives a control and returns either `null` (no problem) or an object that names the problem. The built-in `Validators.required` checks one field; this one is written by hand because it must look at **two** fields, so it is attached to the whole group, not to a single input.

- `group.get('password')?.value` and `group.get('confirmPassword')?.value` read the two texts.
- `confirm && password !== confirm ? { mismatch: true } : null` returns the error `{ mismatch: true }` only when the confirm box is not empty and the two texts differ. If the confirm box is empty, nothing is reported here because the separate `required` rule on that field already covers it.

The function is declared outside the class because it needs no data from the component; it is used on line 35.

Lines 14-18 | @Component decorator

```
14 | @Component({
15 |   selector: 'app-signup',
16 |   templateUrl: './signup.component.html',
17 |   styleUrls: ['./signup.component.css']
18 | })
```

Registers the class as the component `app-signup` with its template `signup.component.html` and its (almost empty) stylesheet `signup.component.css`. See the login component for the meaning of the three settings.

Lines 19-23 | Class and state fields

```
19 | export class SignupComponent {
20 |   signupForm: FormGroup;
21 |   submitted: boolean = false;
22 |   errorMessage: string = '';
23 |   showSuccessPopup: boolean = false;
```

- `signupForm` is the form object, built in the constructor.
- `submitted` becomes true after the first submit attempt so all errors can show.

- `errorMessage` is the red text for server answers such as "A user with this email already exists".
- `showSuccessPopup` is the switch for the success popup: the template draws the popup only while it is `true`.

Lines 25-33 | Constructor: injection and the six controls

```
25 | constructor(private fb: FormBuilder, private authService: AuthService, private router: Router) {
26 |     this.signupForm = this.fb.group(
27 |         {
28 |             username: ['', [Validators.required]],
29 |             email: ['', [Validators.required, Validators.pattern(EMAIL_REGEX)]],
30 |             mobileNumber: ['', [Validators.required, Validators.pattern(MOBILE_REGEX)]],
31 |             password: ['', [Validators.required, Validators.minLength(6)]],
32 |             confirmPassword: ['', [Validators.required]],
33 |             userRole: ['', [Validators.required]]
```

Angular injects `FormBuilder`, `AuthService` and `Router` (dependency injection, see the login component). `this.fb.group(controls, options)` takes two arguments; the first is the object of controls:

- `username` is only `required`, so any non-empty text is accepted.
- `email` must be present and match `EMAIL_REGEX`.
- `mobileNumber` must be present and match `MOBILE_REGEX` (exactly ten digits). Its error message is "Mobile number must be 10 digits".
- `password` is required with at least 6 characters (the same rule as on the login page).
- `confirmPassword` is required; its comparison with `password` is done by the group validator.
- `userRole` starts as an empty string (the "Select Role" option) and is `required`, so the user must pick `Admin` or `Customer`. This text becomes the role stored in the database and decides which navbar and which pages the user gets.

Lines 34-37 | Group-level validator

```
34 |         },
35 |         { validators: passwordsMatch }
36 |     );
37 | }
```

The second argument `{ validators: passwordsMatch }` attaches our function to the **group**. Angular runs it every time any control changes. If it returns `{ mismatch: true }`, then `signupForm.errors` contains `mismatch` and `signupForm.invalid` is true (even if every single field is individually valid), which blocks submit. The error is stored on the group, not on the confirm field, which is why `passwordMismatch` below reads `signupForm.errors`.

Lines 39-41 | Getter f - shortcut to the controls

```
39 | get f() {
40 |     return this.signupForm.controls;
41 | }
```

`this.signupForm.controls` is an object holding all six controls by name. The getter `f` lets the template write `f['email'].errors` instead of the long `signupForm.controls['email'].errors`. (The bracket form with a string is required by strict TypeScript templates.)

Lines 43-46 | isValid - when to show a field error

```
43 | isValid(field: string): boolean {
44 |     const control = this.signupForm.get(field)!;
45 |     return control.invalid && (control.touched || this.submitted);
46 | }
```

Same logic as on the login page: show the message when the control is invalid and the user either touched it or tried to submit. Here the parameter is a plain `string`, because there are six possible fields.

Lines 48-50 | passwordMismatch getter

```
48 | get passwordMismatch(): boolean {
49 |     return !!this.signupForm.errors?.['mismatch'] && !this.f['confirmPassword'].errors?.['required'];
50 | }
```

Returns `true` when two conditions hold: the group has the `mismatch` error (`!!` converts the value to a real true/false) AND the confirm field does not already show its own "required" error. The template uses it to print "Passwords do not match" only when the confirm box actually contains something different.

Lines 52-57 | onSubmit - validate first

```
52 | onSubmit(): void {
53 |     this.submitted = true;
54 |     this.errorMessage = '';
55 |     if (this.signupForm.invalid) {
56 |         return;
57 |     }
```

Sets `submitted = true` (so all hidden errors appear), clears the old `errorMessage` and stops if the form is invalid. No HTTP request is made in that case. The group validator is included in this check, so a password mismatch also stops here.

Lines 58-65 | Building the User object

```
58 |     const value = this.signupForm.value;
59 |     const user: User = {
60 |         username: value.username.trim(),
61 |         email: value.email.trim(),
62 |         mobileNumber: value.mobileNumber,
63 |         password: value.password,
64 |         userRole: value.userRole
65 |     };
```

`this.signupForm.value` is a plain object with the current text of the six controls. A new `User` is built from it:

- `username` and `email` are trimmed to remove accidental spaces at the ends.
- `mobileNumber`, `password` and `userRole` are copied unchanged.
- `confirmPassword` is **not** copied. It exists only for the front-end check and is never sent to the server.

The result is the JSON body `{"username": "...", "email": "...", "mobileNumber": "...", "password": "...", "userRole": "Admin" or "Customer"}`.

Lines 66-67 | Calling register and the success case

```
66 |     this.authService.register(user).subscribe({
67 |         next: () => (this.showSuccessPopup = true),
```

`this.authService.register(user)` returns an Observable for POST `http://localhost:8080/api/register`. `.subscribe({...})` sends it.

Backend journey. `SecurityConfig` line 127 (`permitAll` for `/api/register` and `/api/login`) lets the anonymous request through. Spring MVC calls `AuthController.registerUser(@RequestBody User user)`, which calls `UserServiceImpl.createUser(user)`. That method looks the e-mail up with `UserRepo.findByEmail`; if a user already exists it returns `null`, and the controller answers **409 Conflict** with the text "A user with this email already exists". Otherwise it replaces the plain password with `encoder.encode(...)` (a BCrypt hash, so the real password is never stored), saves the row with `userRepo.save` and returns the saved user. The controller answers **201 Created** with the user as JSON (the `password` field is marked write-only in the `User` entity, so the hash is never sent back).

`next: () => (this.showSuccessPopup = true)` runs on success (any 2xx answer) and simply flips the switch that makes the popup appear.

Lines 68-76 | Error handling

```
68 |         error: (err) => {
69 |             if (err && err.status === 409) {
70 |                 this.errorMessage = 'A user with this email already exists';
71 |             } else if (err && err.status >= 400 && err.status < 500) {
72 |                 this.errorMessage = typeof err.error === 'string' && err.error ? err.error : 'Registration failed. Please tr
73 |             y again.';
74 |             }
75 |         });
76 |     }
```

`error:` runs when the server answers with an error.

- Status **409** (duplicate e-mail from `createUser`) sets the message "A user with this email already exists".
- Any other 4xx status (400-499) shows the text the server sent if it is a non-empty string (`err.error`), otherwise the general "Registration failed. Please try again."
- A server crash (5xx) or stopped backend (status 0) sets no message at all, and, because `HttpErrorInterceptor` skips `/api/register`, does not redirect to the error page either (the same behaviour as the login page).

Line 76 closes the `onSubmit` method.

Lines 78-82 | goToLogin - the popup button

```
78 |     goToLogin(): void {
79 |         this.showSuccessPopup = false;
80 |         this.router.navigate(['/login']);
81 |     }
82 | }
```

Called by the popup's Log In button. It hides the popup (`showSuccessPopup = false`) and `this.router.navigate(['/login'])` moves to the login page, where the new user can sign in. The last line closes the class.

Workflow, next: The next file is the page layout that uses all of this: `signup.component.html`.

Step 76: SignupComponent (template) - the registration form [Aleena - Authentication]

FRONTEND

`angularapp/src/app/components/signup/signup.component.html`

What this file is	The HTML of the sign-up page. It has the same two-panel card as the login page, with a form that has six fields and, below the card, a hidden popup that appears after a successful registration.
Who / what triggers it	Rendered by Angular when <code>SignupComponent</code> is created for <code>/signup</code> , and re-rendered whenever its fields or form state change.
What it uses / calls	<code>SignupComponent</code> (<code>signupForm</code> , <code>f</code> , <code>isInvalid</code> , <code>passwordMismatch</code> , <code>onSubmit</code> , <code>goToLogin</code> , <code>errorMessage</code> , <code>showSuccessPopup</code>) and the global classes in <code>styles.css</code> : <code>auth-*</code> , <code>server-error</code> , <code>modal-backdrop</code> , <code>modal-box</code> , <code>success-box</code> , <code>modal-actions</code> and <code>btn btn-primary</code> .
Size	75 lines

Lines 1-5 | Page wrapper and picture panel

```
1 | <div class="auth-page">
2 |   <div class="auth-card">
3 |     <div class="auth-image">
4 |       <h1 class="auth-heading">DriveU</h1>
5 |     </div>
```

Identical to the login page: `.auth-page` (grey full-screen background), `.auth-card` (the two-column white card), `.auth-image` with the `DriveU` heading on the left.

Lines 7-10 | Form element and titles

```
7 |   <div class="auth-panel">
8 |     <form class="auth-form" id="signupForm" [formGroup]="signupForm" (ngSubmit)="onSubmit()" novalidate>
9 |       <h2>Sign Up</h2>
10 |     <div class="subtitle">JOIN DRIVEU TODAY</div>
```

`[formGroup]="signupForm"` connects the form to the reactive form object, `(ngSubmit)="onSubmit()"` calls the class method on submit, `novalidate` switches off the browser's own validation bubbles. `<h2>` is the title and `.subtitle` is the small blue line "JOIN DRIVEU TODAY" with an underline (style in `styles.css`).

Lines 12-15 | User name field

```
12 |     <div class="auth-field">
13 |       <input type="text" id="username" class="auth-input" placeholder="User name" formControlName="username" autocomplete="username" />
14 |       <div class="auth-error" id="usernameError" *ngIf="isInvalid('username')">Username is required</div>
15 |     </div>
```

`formControlName="username"` ties the input to the `username` control. The error line uses `*ngIf="isInvalid('username')"` and, since only one rule exists (`required`), shows one fixed sentence without inner `<span>` choices.

Lines 17-23 | E-mail field

```
17 |     <div class="auth-field">
18 |       <input type="email" id="email" class="auth-input" placeholder="Email" formControlName="email" autocomplete="email" />
19 |       <div class="auth-error" id="emailError" *ngIf="isInvalid('email')">
20 |         <span *ngIf="f['email'].errors?.['required']">Email is required</span>
21 |         <span *ngIf="!f['email'].errors?.['required'] && f['email'].errors?.['pattern']">Please enter a valid email address</span>
22 |       </div>
23 |     </div>
```

`type="email"` and `autocomplete="email"` help the browser. The error block uses the getter `f['email']` to pick between "Email is required" (empty box) and "Please enter a valid email address" (text that fails `EMAIL_REGEX`). The second `<span>` first checks that `required` is not failing so only one sentence shows.

Lines 25-31 | Mobile number field

```
25 |         <div class="auth-field">
26 |             <input type="text" id="mobileNumber" class="auth-input" placeholder="Mobile" formControlName="mobileNumber"
           autocomplete="tel" />
27 |             <div class="auth-error" id="mobileNumberError" *ngIf="isInvalid('mobileNumber')">
28 |                 <span *ngIf="f['mobileNumber'].errors?.['required']">Mobile number is required</span>
29 |                 <span *ngIf="!f['mobileNumber'].errors?.['required'] && f['mobileNumber'].errors?.['pattern']">Mobile num
           ber must be 10 digits</span>
30 |             </div>
31 |         </div>
```

A plain `type="text"` box (not `number`, which would allow `e` and `+` and drop leading zeros) tied to `mobileNumber`, with `autocomplete="tel"`. Errors: "Mobile number is required" for an empty box and "Mobile number must be 10 digits" when `MOBILE_REGEX` fails (letters or the wrong length).

Lines 33-39 | Password field

```
33 |         <div class="auth-field">
34 |             <input type="password" id="password" class="auth-input" placeholder="Password" formControlName="password" au
           tocomplete="new-password" />
35 |             <div class="auth-error" id="passwordError" *ngIf="isInvalid('password')">
36 |                 <span *ngIf="f['password'].errors?.['required']">Password is required</span>
37 |                 <span *ngIf="!f['password'].errors?.['required'] && f['password'].errors?.['minlength']">Password must be
           at least 6 characters long</span>
38 |             </div>
39 |         </div>
```

`type="password"` shows dots (there is no eye button on this page). `autocomplete="new-password"` tells the browser this is a new password, so it may suggest a strong one instead of filling in an old one. Errors: required, or "Password must be at least 6 characters long" from the `minlength` rule.

Lines 41-47 | Confirm password field

```
41 |         <div class="auth-field">
42 |             <input type="password" id="confirmPassword" class="auth-input" placeholder="Confirm password" formControlNam
           e="confirmPassword" autocomplete="new-password" />
43 |             <div class="auth-error" id="confirmPasswordError" *ngIf="isInvalid('confirmPassword') || (passwordMismatch &
           & (f['confirmPassword'].touched || submitted))">
44 |                 <span *ngIf="f['confirmPassword'].errors?.['required']">Confirm password is required</span>
45 |                 <span *ngIf="passwordMismatch">Passwords do not match</span>
46 |             </div>
47 |         </div>
```

The error block condition is `isInvalid('confirmPassword') || (passwordMismatch && (f['confirmPassword'].touched || submitted))`. It is longer than the others because a mismatch is an error of the **group**, not of the field: the control itself may be valid (not empty) while the group is invalid. So the block must also open when `passwordMismatch` is true and the user has left the box or tried to submit. Inside, "Confirm password is required" appears for an empty box and "Passwords do not match" appears when `passwordMismatch` is true.

Lines 49-56 | Role drop-down

```
49 |         <div class="auth-field">
50 |             <select id="userRole" class="auth-select" formControlName="userRole">
51 |                 <option value="">Select Role</option>
52 |                 <option value="Admin">Admin</option>
53 |                 <option value="Customer">Customer</option>
54 |             </select>
55 |             <div class="auth-error" id="userRoleError" *ngIf="isInvalid('userRole')">User role is required</div>
56 |         </div>
```

`<select formControlName="userRole">` is a drop-down tied to the `userRole` control. The control's value is the `value` attribute of the chosen option: the first option has `value=""` (the placeholder "Select Role", which fails `required`), then `Admin` and `Customer`. These exact words are stored in the database and later checked, case-insensitively, by `AuthGuard`, `AppComponent` and `UserPrincipal` to decide the navbar and permissions. The error line reads "User role is required".

Lines 58-66 | Server error, submit button, Login link

```
58 |         <div class="server-error" id="signupError" *ngIf="errorMessage">{{ errorMessage }}</div>
59 |
60 |         <button type="submit" class="auth-submit" id="signupButton">Sign Up</button>
61 |
62 |         <p class="auth-link">Already have an account? <a routerLink="/login" id="loginLink">Log In</a></p>
63 |     </form>
64 | </div>
65 | </div>
66 | </div>
```

`*ngIf="errorMessage"` shows the red `.server-error` line ("A user with this email already exists" etc.). The `type="submit"` button triggers `ngSubmit`. The link `routerLink="/login"` lets users who already have an account go

back without reloading. Line 63 closes the form and the following lines close the panel, card and page.

Lines 68-75 | Registration success popup

```
68 | <div class="modal-backdrop" *ngIf="showSuccessPopup">
69 |   <div class="modal-box success-box" id="registrationSuccessPopup">
70 |     <p>Registration Successful!</p>
71 |     <div class="modal-actions">
72 |       <button type="button" class="btn btn-primary" id="goToLogin" (click)="goToLogin()">Log In</button>
73 |     </div>
74 |   </div>
75 | </div>
```

This block lives outside the card. `*ngIf="showSuccessPopup"` means it exists in the page only after a successful registration. `.modal-backdrop` is a fixed, full-screen dark blurred layer that sits over everything, `.modal-box.success-box` is the white box with a blue border, and `<p>Registration Successful!</p>` is the message. The button `(click)="goToLogin()"` calls the class method which hides the popup and navigates to `/login`. The popup uses the global rules for `.modal-backdrop`, `.modal-box`, `.modal-actions` and `.btn-primary` in `styles.css`.

Workflow, next: Next is the stylesheet of this component: `signup.component.css`.

Step 77: SignupComponent (CSS) - component-level style [Aleena - Authentication]

FRONTEND

angularapp/src/app/components/signup/signup.component.css

What this file is	The stylesheet of the sign-up component. Like the login one it is only two lines: the page shares the global <code>auth-*</code> look.
Who / what triggers it	Loaded and scoped to <code><app-signup></code> by Angular because of <code>styleUrls</code> .
What it uses / calls	Global rules in <code>styles.css</code> : the auth rules (<code>.auth-page</code> to <code>.server-error</code>) and the modal rules (<code>.modal-backdrop</code> , <code>.modal-box</code> , <code>.modal-box.success-box</code> , <code>.modal-actions</code>) that style the success popup.
Size	2 lines

Lines 1-2 | Comment and `:host` rule

```
1 | /* Layout of the two-panel auth card is shared with the login page (see styles.css). */
2 | :host { display: block; }
```

The comment tells the reader that the card layout is shared with the login page and lives in `styles.css`. `:host { display: block; }` makes the `<app-signup>` element a block box (custom elements are inline by default) so it spans the full width and the `.auth-page` wrapper can centre the card vertically and horizontally. The extra things this page needs, such as `.auth-select` for the role drop-down and the popup's `.modal-*` classes, are also global.

Workflow, next: After the success popup the user is sent back to the login page, signs in, and the journey described in `login.component.ts` ends at `/home`. The route `/home` is protected by `AuthGuard (canActivate)`, which confirms that a token exists and has not expired and then lets `HomePageComponent` appear. So the next file is `home-page.component.ts`.

Step 78: HomePageComponent (TypeScript) - the landing page after login [Vaishalini - Customer Journey & Shared Infra]

FRONTEND

angularapp/src/app/components/home-page/home-page.component.ts

What this file is	The class for the <code>/home</code> page, the first page an Admin or Customer sees after logging in. It is the simplest component in the project: it only keeps two pieces of text (a phone number and an e-mail) that the template prints in the "Contact Us" card. It makes no HTTP calls and has no logic.
Who / what triggers it	The router, for the URL <code>/home</code> after <code>AuthGuard.canActivate</code> returned <code>true</code> . The guard calls <code>AuthService.isLoggedIn()</code> (token present and not expired); the <code>/home</code> route has no <code>data: { role }</code> entry, so both roles may enter. If the guard had returned a <code>UrlTree</code> for <code>/login</code> , this component would never have been created.
What it uses / calls	Only <code>Component</code> from Angular core. Its template reads <code>phone</code> and <code>email</code> . The navbar above the page is not part of this component: <code>AppComponent</code> draws <code>AdminnavComponent</code> or <code>CustomernavComponent</code> in <code>app.component.html</code> depending on the stored role.
Size	11 lines

Line 1 | Import

```
1 | import { Component } from '@angular/core';
```

`Component` is the decorator that makes the class an Angular component. Nothing else is needed, because this component injects no services.

Lines 3-7 | @Component decorator

```
3 | @Component({
4 |   selector: 'app-home-page',
5 |   templateUrl: './home-page.component.html',
6 |   styleUrls: ['./home-page.component.css']
7 | })
```

Registers the class under the tag `app-home-page` with `home-page.component.html` as layout and `home-page.component.css` as its scoped styles. The selector is not used in any template; the router creates the component itself.

Lines 8-11 | Class with the two contact values

```
8 | export class HomePageComponent {
9 |   phone: string = '987-654-3210';
10 |   email: string = 'support@driveu.com';
11 | }
```

`phone` and `email` are the support details shown in the Contact Us card (`987-654-3210` and `support@driveu.com`). They are plain strings written into the class, not loaded from a server; to change the contact information you change these two lines. The template prints them with interpolation `{{ phone }}` and `{{ email }}`. The explicit `: string` is a type annotation, which tells TypeScript the variable can only hold text.

Workflow, next: Angular now renders the page layout: `home-page.component.html`.

Step 79: HomePageComponent (template) - hero, welcome text, contact card [Vaishalini - Customer Journey & Shared Infra]

FRONTEND

angularapp/src/app/components/home-page/home-page.component.html

What this file is	The static content of the home page, from top to bottom: a large hero picture with the DriveU name and slogan, a white welcome card that overlaps the bottom of the picture, a thin divider line, and a dark Contact Us card. There are no forms or buttons here; navigation is done through the navbar that <code>AppComponent</code> shows above this page.
Who / what triggers it	Rendered inside <code><router-outlet></code> when the route <code>/home</code> is activated.
What it uses / calls	<code>HomePageComponent</code> for <code>phone</code> and <code>email</code> , and the classes defined in <code>home-page.component.css</code> (<code>.home</code> , <code>.hero</code> , <code>.hero-overlay</code> , <code>.welcome-card</code> , <code>.divider</code> , <code>.contact-card</code> , <code>.icon</code> , <code>.phone</code> , <code>.mail</code>).
Size	25 lines

Lines 1-7 | Hero section

```
1 | <div class="home">
2 |   <section class="hero" id="hero">
3 |     <div class="hero-overlay">
4 |       <h1>DriveU</h1>
5 |       <p>Your trusted platform for connecting with professional drivers on demand</p>
6 |     </div>
7 |   </section>
```

`<div class="home">` is the outer wrapper (it adds space at the bottom). `<section class="hero" id="hero">` is the big picture band; its background image is set in the CSS. `.hero-overlay` is the dark translucent box in the middle holding the `<h1>DriveU</h1>` heading and the slogan paragraph, so the white text stays readable on top of a photo. `id="hero"` is a fixed hook for tests.

Lines 9-16 | Welcome card

```
9 |   <section class="welcome-card" id="welcome">
10 |     <h2>Welcome to DriveU</h2>
11 |     <p>
12 |       Experience seamless connectivity with our professional drivers. DriveU is your one-stop solution
13 |       for short-term or long-term driving needs. Whether it's a daily commute, a vacation, or a
14 |       one-time trip, DriveU ensures safety, trust, and convenience.
15 |     </p>
16 |   </section>
```

`<section class="welcome-card" id="welcome">` is the white card with the heading "Welcome to DriveU" and a short paragraph describing the service (short-term and long-term drivers, daily commute, vacation or one-time trip). It is plain text; the CSS pulls the card upward so it overlaps the hero picture.

Line 18 | Divider

```
18 |   <hr class="divider" />
```

`<hr class="divider" />` draws a horizontal rule between the welcome text and the contact card. It is self-closing; its look (thin grey line, centred, partial width) comes from `.divider` in the CSS.

Lines 20-25 | Contact card

```
20 |   <section class="contact-card" id="contact">
21 |     <h3>Contact Us</h3>
22 |     <p><span class="icon phone">&#9742;</span> Phone: {{ phone }}</p>
23 |     <p><span class="icon mail">&#9993;</span> Email: {{ email }}</p>
24 |   </section>
25 | </div>
```

`<section class="contact-card" id="contact">` is the dark purple box. `<h3>Contact Us</h3>` is its title. Each `<p>` has a `<span class="icon ...">` that holds an HTML character (`☎` is the telephone symbol, `✉` is the envelope) coloured by `.phone` (red) or `.mail` (light blue), followed by a label and the interpolation `{{ phone }}` or `{{ email }}`, which Angular replaces with the class values. Line 25 closes the `.home` wrapper.

Workflow, next: Next: the stylesheet `home-page.component.css`, which positions these sections.

Step 80: HomePageComponent (CSS) - layout of the landing page [Vaishalini - Customer Journey & Shared Infra]

FRONTEND

angularapp/src/app/components/home-page/home-page.component.css

What this file is	All the visual rules for the home page: the picture band with its dark overlay, the welcome card that floats over the bottom of the picture, the divider line, and the contact card. Unlike the login and signup pages, this page does not use shared global rules; everything is defined here and scoped to this component.
Who / what triggers it	Loaded with the component and applied only to its elements through Angular's scoped attributes.
What it uses / calls	Classes used by <code>home-page.component.html</code> ; the image <code>src/assets/images/homepage.webp</code> (<code>angular.json</code> copies the whole <code>src/assets</code> folder to the web root, so the URL <code>/assets/images/homepage.webp</code> works).
Size	53 lines

Line 1 | .home - bottom space

```
1 | .home { padding-bottom: 44px; }
```

`.home { padding-bottom: 44px; }` adds 44 pixels of empty space under the contact card so the page does not end hard at the card's bottom edge.

Lines 3-9 | .hero - picture band

```
3 | .hero {
4 |   display: flex;
5 |   align-items: center;
6 |   justify-content: center;
7 |   height: 400px;
8 |   background: #cfe3ee url('/assets/images/homepage.webp') center / cover no-repeat;
9 | }
```

The hero is a flex container (`display: flex`) with `align-items: center` and `justify-content: center`, which centres the overlay box both vertically and horizontally. `height: 400px` fixes its height. The `background` shorthand sets four things in one line: a fallback colour `#cfe3ee` (light blue, visible if the image is missing or still loading), the image `url('/assets/images/homepage.webp')`, `center / cover` (position the image in the centre and scale it to cover the whole band, cropping if needed) and `no-repeat` (do not tile it).

Lines 10-18 | .hero-overlay - the dark text box

```
10 | .hero-overlay {
11 |   max-width: 430px;
12 |   margin-top: -24px;
13 |   padding: 20px 44px;
14 |   text-align: center;
15 |   color: #fff;
16 |   background: rgba(40, 40, 40, 0.78);
17 |   border-radius: 10px;
18 | }
```

`max-width: 430px` limits the width of the box. `margin-top: -24px` nudges it 24 px upward so it sits slightly above the exact centre (the welcome card will overlap the bottom of the picture). `padding: 20px 44px` is the inner space (20 px top and bottom, 44 px left and right). `text-align: center` and `color: #fff` centre the text and make it white. `background: rgba(40, 40, 40, 0.78)` is dark grey at 78 percent opacity, so the photo shows through a little. `border-radius: 10px` rounds the corners.

Line 19 | .hero-overlay h1

```
19 | .hero-overlay h1 { margin: 0 0 8px; font-size: 38px; }
```

The DriveU heading: `margin: 0 0 8px` removes the default top margin and leaves 8 px below; `font-size: 38px` makes it big.

Line 20 | .hero-overlay p

```
20 | .hero-overlay p { margin: 0; font-size: 14px; line-height: 1.4; }
```

The slogan: `margin: 0` removes the default paragraph spacing, `font-size: 14px` is small, `line-height: 1.4` sets the line spacing to 1.4 times the font size so a wrapped second line is easy to read.

Lines 22-31 | .welcome-card - overlapping white card

```
22 | .welcome-card {  
23 |   position: relative;  
24 |   width: min(440px, 90%);  
25 |   margin: -64px auto 0;  
26 |   padding: 18px 26px 24px;  
27 |   text-align: center;  
28 |   background: rgba(255, 255, 255, 0.97);  
29 |   border-radius: 10px;  
30 |   box-shadow: 0 2px 12px rgba(0, 0, 0, 0.16);  
31 | }
```

position: relative lets the card take part in stacking (it is drawn above the hero). **width: min(440px, 90%)** means 440 px, but never more than 90 percent of the screen, so it shrinks on a phone. **margin: -64px auto 0** has a negative top margin that pulls the card 64 px upward over the bottom of the hero picture, **auto** on the sides centres it, and **0** is the bottom margin. **padding: 18px 26px 24px** is the inner space, **text-align: center** centres text. **background: rgba(255, 255, 255, 0.97)** is almost solid white, **border-radius: 10px** rounds the corners, and **box-shadow: 0 2px 12px rgba(0, 0, 0, 0.16)** adds a soft shadow below so the card looks raised.

Line 32 | .welcome-card h2

```
32 | .welcome-card h2 { margin: 4px 0 10px; font-size: 24px; }
```

The heading: **margin: 4px 0 10px** small gap above and larger gap below, **font-size: 24px**.

Line 33 | .welcome-card p

```
33 | .welcome-card p { margin: 0; font-size: 13.5px; line-height: 1.6; color: #444; }
```

The paragraph: **margin: 0**, **font-size: 13.5px**, generous **line-height: 1.6** for readability and **color: #444** (dark grey, softer than pure black).

Line 35 | .divider - the horizontal line

```
35 | .divider { width: min(760px, 82%); margin: 34px auto 28px; border: none; border-top: 1px solid #777; }
```

An `<hr>` has a default 3-D border, so **border: none** removes it and **border-top: 1px solid #777** draws a single thin grey line. **width: min(760px, 82%)** makes it 760 px wide at most and 82 percent of the screen on small screens. **margin: 34px auto 28px** centres it (**auto**) with 34 px space above and 28 px below.

Lines 37-46 | .contact-card - dark purple box

```
37 | .contact-card {  
38 |   width: 240px;  
39 |   margin: 0 auto;  
40 |   padding: 18px 12px 20px;  
41 |   text-align: center;  
42 |   color: #fff;  
43 |   background: #2b0f2e;  
44 |   border-radius: 8px;  
45 |   box-shadow: 0 2px 10px rgba(0, 0, 0, 0.3);  
46 | }
```

width: 240px is a narrow fixed width, **margin: 0 auto** centres it, **padding: 18px 12px 20px** is the inner space, **text-align: center** and **color: #fff** give centred white text. **background: #2b0f2e** is the very dark purple. **border-radius: 8px** rounds the corners and **box-shadow: 0 2px 10px rgba(0, 0, 0, 0.3)** adds a darker shadow than the welcome card.

Line 47 | .contact-card h3

```
47 | .contact-card h3 { margin: 0 0 10px; font-size: 20px; }
```

Title: no top margin, 10 px below, **font-size: 20px**.

Line 48 | .contact-card p

```
48 | .contact-card p { margin: 5px 0; font-size: 14px; }
```

The two contact lines: 5 px space above and below each (**margin: 5px 0**), **font-size: 14px**.

Line 49 | .contact-card .icon

```
49 | .contact-card .icon { margin-right: 4px; }
```

margin-right: 4px leaves a small gap between the symbol and the text after it.

Lines 50-51 | Icon colours

```
50 | .contact-card .phone { color: #ff5a5a; }  
51 | .contact-card .mail { color: #cfe3ff; }
```

.phone colours the telephone symbol red (**#ff5a5a**) and **.mail** colours the envelope light blue (**#cfe3ff**), so each stands out on the dark purple.

Line 53 | Small-screen media query

```
53 | @media (max-width: 600px) { .hero { height: 320px; } .hero-overlay { padding: 16px 24px; } }
```

`@media (max-width: 600px)` applies only when the screen is 600 px wide or narrower (phones). Inside, the hero is made shorter (`height: 320px` instead of 400) and the overlay gets less side padding (`16px 24px` instead of `20px 44px`) so the text fits in a narrow screen.

Workflow, next: The home page itself is now fully explained, but the bar across the top of it is a separate component chosen by `AppComponent`: `app.component.html` contains `<app-adminnav *ngIf="isAdmin$ | async">` and `<app-customernav *ngIf="isCustomer$ | async">`. `AppComponent` turns `AuthService.userRole$` into those two true/false streams (it compares the role in lower case with `admin` or `customer`), so exactly one navbar appears, as soon as the login's `tap` pushed the role. The next file is the Admin navbar, `adminnav.component.ts`.

Step 81: AdminnavComponent (TypeScript) - admin navigation bar logic [Vaishalini - Customer Journey & Shared Infra]

FRONTEND

angularapp/src/app/components/adminnav/adminnav.component.ts

What this file is	The class behind the top navigation bar that an Admin sees on every page after logging in. It keeps three things: the signed-in user name, the role, and whether the "Are you sure you want to logout?" popup is open. It also performs the logout. The menu links themselves are plain <code>routerLinks</code> in the template and need no code.
Who / what triggers it	<code>AppComponent</code> creates it. In <code>app.component.html</code> , <code><app-adminnav *ngIf="isAdmin\$ async"></code> is true whenever <code>AuthService.userRole\$</code> currently holds <code>Admin</code> . Angular creates the component when the role becomes admin (right after login, or at page load when a token is already stored) and destroys it when the role is cleared (logout).
What it uses / calls	<code>AuthService</code> (<code>username\$</code> , <code>userRole\$</code> streams and <code>logout()</code>), <code>Router</code> , and <code>Subscription</code> from RxJS. The template is <code>adminnav.component.html</code> .
Size	42 lines

Lines 1-4 | Imports

```
1 | import { Component, OnDestroy, OnInit } from '@angular/core';
2 | import { Router } from '@angular/router';
3 | import { Subscription } from 'rxjs';
4 | import { AuthService } from '../services/auth.service';
```

- `Component` is the decorator; `OnInit` and `OnDestroy` are **lifecycle interfaces**. Angular calls the methods `ngOnInit` (once, after the component is created) and `ngOnDestroy` (once, just before the component is removed) automatically.
- `Router` is used to navigate to `/login` after logout.
- `Subscription` (from RxJS, the library that provides Observables) is a handle that holds running subscriptions so that they can all be cancelled together.
- `AuthService` provides the user data and the logout action.

Lines 6-10 | @Component decorator

```
6 | @Component({
7 |   selector: 'app-adminnav',
8 |   templateUrl: './adminnav.component.html',
9 |   styleUrls: ['./adminnav.component.css']
10 | })
```

Registers the class with the tag `app-adminnav` (the tag used in `app.component.html`), the template `adminnav.component.html` and the style file `adminnav.component.css`.

Lines 11-14 | Class and fields

```
11 | export class AdminnavComponent implements OnInit, OnDestroy {
12 |   username: string = '';
13 |   userRole: string = '';
14 |   showLogoutPopup: boolean = false;
```

`implements OnInit, OnDestroy` promises Angular that the class has those two lifecycle methods. `username` and `userRole` are the texts shown in the white chip in the navbar (for example "Asha / Admin"); they start empty and are filled from `AuthService` in `ngOnInit`. `showLogoutPopup` is the switch for the confirmation popup (closed at the start).

Lines 16-18 | Subscription holder and constructor

```
16 | private subscriptions = new Subscription();
17 |
18 | constructor(private authService: AuthService, private router: Router) {}
```

`private subscriptions = new Subscription()` creates an empty bag. Every subscription made later is added to it, so one call to `unsubscribe()` stops them all. The constructor receives `AuthService` and `Router` by dependency injection (`private` makes them fields).

Lines 20-23 | ngOnInit - subscribe to the session streams

```
20 | ngOnInit(): void {
21 |     this.subscriptions.add(this.authService.username$.subscribe((name) => (this.username = name || '')));
22 |     this.subscriptions.add(this.authService.userRole$.subscribe((role) => (this.userRole = role || '')));
23 | }
```

`ngOnInit` runs once after creation. `this.authService.username$` and `userRole$` are Observables built on `BehaviorSubjects` (see `auth.service.ts`). A **BehaviorSubject** always remembers its latest value and hands it immediately to each new subscriber, and then sends every later change. That is why the navbar shows the right name at once even though login happened a moment before the navbar was created.

`.subscribe((name) => (this.username = name || ''))` registers a function that copies each new value into the field; `|| ''` turns `null` (no one logged in) into an empty string. `this.subscriptions.add(...)` stores each subscription in the bag. Because the fields are updated by the subscription, the `{{ username }}` / `{{ userRole }}` text in the template always follows the stored session.

Lines 25-27 | ngOnDestroy - clean up

```
25 | ngOnDestroy(): void {
26 |     this.subscriptions.unsubscribe();
27 | }
```

Angular calls this when `*ngIf` in `AppComponent` removes the navbar (after logout).

`this.subscriptions.unsubscribe()` cancels both subscriptions. Without this, the old, destroyed component would keep receiving values and stay in memory (a **memory leak**).

Lines 29-35 | Opening and cancelling the popup

```
29 | openLogoutPopup(): void {
30 |     this.showLogoutPopup = true;
31 | }
32 |
33 | cancelLogout(): void {
34 |     this.showLogoutPopup = false;
35 | }
```

`openLogoutPopup()` sets `showLogoutPopup = true`; the template's `*ngIf` then draws the popup. It is called by the red Logout button. `cancelLogout()` sets it back to `false`, which removes the popup and leaves the user logged in. It is called by the popup's Cancel button.

Lines 37-42 | confirmLogout - the actual logout

```
37 | confirmLogout(): void {
38 |     this.showLogoutPopup = false;
39 |     this.authService.logout();
40 |     this.router.navigate(['/login']);
41 | }
42 | }
```

Called by the popup's "Yes, Logout" button. In order: (1) close the popup; (2) `this.authService.logout()` removes the four `localStorage` entries `token`, `userRole`, `userId` and `username`, and pushes `null` into the three `BehaviorSubjects`, which makes `AppComponent`'s `isAdmin$` and `isCustomer$` false so both navbars disappear (and this component's `ngOnDestroy` runs); (3) `this.router.navigate(['/login'])` returns to the login page. There is no call to the server: the JWT is stateless, so "logging out" just means the browser forgets the token. (The old token stays technically valid until its 24 hours expire, but no one holds it any more.) The last line closes the class.

Workflow, next: Next: the navbar's template, `adminnav.component.html`.

Step 82: AdminnavComponent (template) - admin menu and logout popup [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/components/adminnav/adminnav.component.html

FRONTEND

What this file is	The HTML of the Admin navbar: brand name on the left; on the right the user chip, the menu (Home, Drivers with a drop-down, Requests with a drop-down, Feedback with a drop-down) and the red Logout button. Below the navbar, outside it, is the hidden confirmation popup.
Who / what triggers it	Rendered whenever <code>AppComponent</code> shows <code><app-adminnav></code> , on every page for a logged-in Admin.
What it uses / calls	<code>AdminnavComponent</code> (<code>username</code> , <code>userRole</code> , <code>showLogoutPopup</code> , <code>openLogoutPopup</code> , <code>confirmLogout</code> , <code>cancelLogout</code>), <code>RouterModule</code> directives <code>routerLink</code> and <code>routerLinkActive</code> , and the global <code>.navbar</code> and <code>.modal-*</code> rules in <code>styles.css</code> . Routes linked: <code>/home</code> , <code>/driver-management</code> , <code>/admin-view-drivers</code> , <code>/adminviewrequests</code> , <code>/adminviewfeedback</code> (all with <code>data: { role: 'Admin' }</code>) and <code>AuthGuard</code> except <code>/home</code>).
Size	45 lines

Lines 1-2 | Nav element and brand link

```
1 | <nav class="navbar" id="adminnav">
2 |   <a class="brand" routerLink="/home">DriveU</a>
```

`<nav class="navbar" id="adminnav">` is the bar (the `.navbar` rule gives it the blue-to-gold gradient, fixed 54 px height and shadow). `<a class="brand" routerLink="/home">DriveU</a>` is the logo text; `routerLink="/home"` changes the page without reloading, so clicking the brand always returns home.

Lines 4-5 | Right-hand group and user chip

```
4 |   <div class="nav-right">
5 |     <span class="user-chip" id="userInfo">{{ username }} / {{ userRole }}</span>
```

`.nav-right` groups the chip, the menu and the button in one flex row. `{{ username }} / {{ userRole }}` is interpolation that prints the two fields from the class, e.g. "Asha / Admin", inside the bordered `.user-chip`.

Lines 7-8 | Menu list and Home link

```
7 |   <ul class="nav-links">
8 |     <li><a routerLink="/home" routerLinkActive="active" id="navHome">Home</a></li>
```

`<ul class="nav-links">` is the menu. The Home link uses `routerLink="/home"` and `routerLinkActive="active"`: Angular adds the CSS class `active` to this link whenever the current URL matches, and the global rule `.navbar .nav-links > li > a.active` then gives the link a lighter highlight.

Lines 10-16 | Drivers drop-down

```
10 |   <li class="dropdown">
11 |     <span class="nav-label" id="navDrivers" tabindex="0">Drivers</span>
12 |     <div class="dropdown-menu">
13 |       <a routerLink="/driver-management" id="navAddDriver">Add Driver</a>
14 |       <a routerLink="/admin-view-drivers" id="navViewDrivers">View Drivers</a>
15 |     </div>
16 |   </li>
```

`<li class="dropdown">` is a menu item with a hidden sub-menu. The `<span class="nav-label" tabindex="0">` is the visible word "Drivers"; it is a span, not a link, because it goes nowhere itself, and `tabindex="0"` makes it reachable with the Tab key so the menu opens for keyboard users too. `.dropdown-menu` contains two links: Add Driver (`/driver-management`) and View Drivers (`/admin-view-drivers`). The sub-menu is hidden by default and shown by CSS only (`.dropdown:hover .dropdown-menu` or `:focus-within`); no JavaScript is involved.

Lines 18-23 | Requests drop-down

```
18 |   <li class="dropdown">
19 |     <span class="nav-label" id="navRequests" tabindex="0">Requests</span>
20 |     <div class="dropdown-menu">
21 |       <a routerLink="/adminviewrequests" id="navViewRequests">View Requests</a>
22 |     </div>
23 |   </li>
```

The same structure with one entry: View Requests (`/adminviewrequests`), where the admin approves or rejects customer requests and closes finished trips.

Lines 25-30 | Feedback drop-down

```
25 |     <li class="dropdown">
26 |         <span class="nav-label" id="navFeedback" tabindex="0">Feedback</span>
27 |         <div class="dropdown-menu">
28 |             <a routerLink="/adminviewfeedback" id="navViewFeedback">View Feedback</a>
29 |         </div>
30 |     </li>
```

The same structure with one entry: View Feedback (`/adminviewfeedback`).

Lines 31-35 | End of menu, Logout button, end of nav

```
31 |     </ul>
32 |
33 |     <button type="button" class="logout-btn" id="logoutButton" (click)="openLogoutPopup()">Logout</button>
34 | </div>
35 | </nav>
```

Line 31 closes the list. The `<button type="button" class="logout-btn" ... (click)="openLogoutPopup()">` is the red Logout button; clicking it only calls `openLogoutPopup()`, which sets the flag that opens the popup, so a mis-click never logs the user out directly. Lines 34-35 close `.nav-right` and the `<nav>`.

Lines 37-39 | Logout popup wrapper

```
37 | <div class="modal-backdrop" *ngIf="showLogoutPopup">
38 |   <div class="modal-box">
39 |     <p>Are you sure you want to logout?</p>
```

`<div class="modal-backdrop" *ngIf="showLogoutPopup">` exists in the page only while the flag is true. It is a fixed, full-screen translucent layer with a blur that sits over everything (and blocks clicks on the page behind it). `.modal-box` is the white centred dialog and the paragraph is the question "Are you sure you want to logout?".

Lines 40-45 | Popup buttons

```
40 |     <div class="modal-actions row">
41 |       <button type="button" class="btn btn-success" id="confirmLogout" (click)="confirmLogout()">Yes, Logout</button>
42 |       <button type="button" class="btn btn-danger" id="cancelLogout" (click)="cancelLogout()">Cancel</button>
43 |     </div>
44 |   </div>
45 | </div>
```

`.modal-actions.row` lays the two buttons next to each other. "Yes, Logout" (green, `btn-success`, `(click)="confirmLogout()"`) clears the session and goes to the login page. "Cancel" (red, `btn-danger`, `(click)="cancelLogout()"`) just closes the dialog. The remaining lines close the wrappers.

Workflow, next: Next: the navbar's own stylesheet `adminnav.component.css`.

Step 83: AdminnavComponent (CSS) - component-level style [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/components/adminnav/adminnav.component.css

FRONTEND

What this file is	A very small stylesheet. The whole navbar look is global, written once in <code>styles.css</code> under <code>.navbar</code> so that the admin and the customer navbars are identical in style. Only two details are local to this component.
Who / what triggers it	Angular loads it with <code>AdminnavComponent</code> and scopes it to <code><app-adminnav></code> .
What it uses / calls	Global rules in <code>styles.css</code> : <code>.navbar</code> , <code>.navbar .brand</code> , <code>.navbar .nav-right</code> , <code>.navbar .user-chip</code> , <code>.navbar .nav-links</code> , <code>.navbar .dropdown-menu</code> , <code>.navbar .logout-btn</code> , the <code>@media (max-width: 720px)</code> navbar rules, and <code>.modal-backdrop</code> , <code>.modal-box</code> , <code>.modal-actions</code> , <code>.btn</code> , <code>.btn-success</code> , <code>.btn-danger</code> .
Size	3 lines

Lines 1-2 | Comment and :host rule

```
1 | /* Navbar look is shared with the customer navbar and lives in the global styles.css. */
2 | :host { display: block; }
```

The comment says the navbar look is shared and global. Here is what the global rules do: `.navbar` is a flex row (`space-between`) with a 54 px height and a horizontal gradient from blue (`--nav-from`) through grey to gold (`--nav-to`) defined as CSS variables at the top of `styles.css`; `.brand` is the large bold logo text; `.user-chip` is the white-bordered rounded box; `.nav-links` is the horizontal list without bullets; links get a lighter background on

hover or when `.active`; `.dropdown-menu` is absolutely positioned under its item, hidden until hover or focus; `.logout-btn` is red and darker on hover; below 720 px the bar wraps onto several lines.

`:host { display: block; }` makes the `<app-adminnav>` element a block so the bar spans the whole width instead of being squeezed inline.

Line 3 | Popup button size override

```
3 | .modal-actions.row .btn { min-width: 100px; border-radius: 4px; padding: 8px 14px; }
```

`.modal-actions.row .btn { min-width: 100px; border-radius: 4px; padding: 8px 14px; }` changes the two popup buttons, "Yes, Logout" and "Cancel". The global rule `.modal-actions .btn` makes modal buttons wide pills (`min-width: 140px, border-radius: 999px`). This local rule is more specific (and Angular's scoping attribute makes it stronger still), so it wins: the two buttons become smaller (`min-width: 100px`), with lightly rounded corners (`4px`) and less padding (`8px 14px`), so that both fit side by side in the popup.

Workflow, next: Admin navbar done. The Customer navbar follows the same pattern with different links, so the next file is `customernav.component.ts`.

Step 84: CustomernavComponent (TypeScript) - customer navigation bar logic [Vaishalini - Customer Journey & Shared Infra]

FRONTEND

angularapp/src/app/components/customernav/customernav.component.ts

What this file is	The class behind the navigation bar that a Customer sees. It is line for line the same as <code>AdminnavComponent</code> : it keeps the user name, the role and the logout-popup flag, subscribes to <code>AuthService</code> , and does the logout. Only the selector and the template differ. (The code was copied rather than shared, so any change would have to be made twice.)
Who / what triggers it	<code>AppComponent: <app-customernav *ngIf="isCustomer\$ async"></code> is true when the stored role is <code>Customer</code> (compared in lower case). Angular creates it after a customer's login and destroys it at logout.
What it uses / calls	<code>AuthService (username\$, userRole\$, logout())</code> , <code>Router</code> , <code>Subscription</code> from RxJS.
Size	42 lines

Lines 1-4 | Imports

```
1 | import { Component, OnDestroy, OnInit } from '@angular/core';
2 | import { Router } from '@angular/router';
3 | import { Subscription } from 'rxjs';
4 | import { AuthService } from '../services/auth.service';
```

Same as in the admin navbar: `Component`, the lifecycle interfaces `OnInit` and `OnDestroy`, `Router` for navigating to the login page, `Subscription` to hold the subscriptions, and `AuthService` for the data and the logout.

Lines 6-10 | @Component decorator

```
6 | @Component({
7 |   selector: 'app-customernav',
8 |   templateUrl: './customernav.component.html',
9 |   styleUrls: ['./customernav.component.css']
10 | })
```

Tag `app-customernav`, template `customernav.component.html`, style `customernav.component.css`.

Lines 11-14 | Class and fields

```
11 | export class CustomernavComponent implements OnInit, OnDestroy {
12 |   username: string = '';
13 |   userRole: string = '';
14 |   showLogoutPopup: boolean = false;
```

`username` and `userRole` feed the chip text (for example "Ravi / Customer"); `showLogoutPopup` controls the confirmation dialog.

Lines 16-18 | Subscription holder and constructor

```
16 | private subscriptions = new Subscription();
17 |
18 | constructor(private authService: AuthService, private router: Router) {}
```

An empty **Subscription** bag, plus injection of **AuthService** and **Router**.

Lines 20-23 | **ngOnInit** - subscribe to username and role

```
20 |   ngOnInit(): void {  
21 |     this.subscriptions.add(this.authService.username$.subscribe((name) => (this.username = name || '')));  
22 |     this.subscriptions.add(this.authService.userRole$.subscribe((role) => (this.userRole = role || '')));  
23 |   }
```

Runs once at creation. **username\$** and **userRole\$** are BehaviorSubject-based Observables, so each subscription gets the currently stored value immediately (the name set by **AuthService.login**'s **tap** or read back from **localStorage** after a page refresh) and every later change. The callbacks write the values into the fields, turning **null** into **''**. The subscriptions are added to the bag so they can be cancelled together.

Lines 25-27 | **ngOnDestroy** - cancel subscriptions

```
25 |   ngOnDestroy(): void {  
26 |     this.subscriptions.unsubscribe();  
27 |   }
```

Called by Angular when **AppComponent** removes the navbar (logout). **unsubscribe()** stops both subscriptions so nothing keeps running or stays in memory.

Lines 29-35 | **Open and cancel the logout popup**

```
29 |   openLogoutPopup(): void {  
30 |     this.showLogoutPopup = true;  
31 |   }  
32 |  
33 |   cancelLogout(): void {  
34 |     this.showLogoutPopup = false;  
35 |   }
```

openLogoutPopup() (red Logout button) shows the dialog by setting the flag; **cancelLogout()** (Cancel button) hides it and keeps the user logged in.

Lines 37-42 | **confirmLogout**

```
37 |   confirmLogout(): void {  
38 |     this.showLogoutPopup = false;  
39 |     this.authService.logout();  
40 |     this.router.navigate(['/login']);  
41 |   }  
42 | }
```

Closes the popup, calls **AuthService.logout()** (which removes **token**, **userRole**, **userId** and **username** from **localStorage** and pushes **null** into the three BehaviorSubjects, so both navbars vanish and any component that reads the session sees no user), and then navigates to **/login**. No request is sent to the server, because a JWT is stateless: the browser simply forgets it. After this, **AuthGuard** blocks every protected page until the user logs in again.

Workflow, next: Next: **customernav.component.html**.

Step 85: CustomernavComponent (template) - customer menu and logout popup [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/components/customernav/customernav.component.html

FRONTEND

What this file is	The HTML of the customer navbar: brand, user chip, a flat four-link menu (Home, Available Drivers, My Requests, Feedback) and the red Logout button, plus the hidden logout confirmation popup. Unlike the admin bar there are no drop-downs.
Who / what triggers it	Rendered whenever AppComponent shows <app-customernav> for a logged-in Customer.
What it uses / calls	CustomernavComponent fields and methods, routerLink and routerLinkActive , the global .navbar and .modal-* rules. Routes linked: /home , /customerviewdriver , /customerviewrequested , /customerviewfeedback .
Size	26 lines

Lines 1-5 | Nav, brand and user chip

```
1 | <nav class="navbar" id="customernav">
2 |   <a class="brand" routerLink="/home">DriveU</a>
3 |
4 |   <div class="nav-right">
5 |     <span class="user-chip" id="userInfo">{{ username }} / {{ userRole }}</span>
```

`<nav class="navbar" id="customernav">` is the gradient bar (global `.navbar` rule). The brand link goes to `/home`. `.nav-right` holds the chip, links and button; the chip prints `{{ username }} / {{ userRole }}` from the class.

Lines 7-12 | Menu links

```
7 |   <ul class="nav-links">
8 |     <li><a routerLink="/home" routerLinkActive="active" id="navHome">Home</a></li>
9 |     <li><a routerLink="/customerviewdriver" routerLinkActive="active" id="navAvailableDrivers">Available Drivers</a>
10 |   </li>
11 |     <li><a routerLink="/customerviewrequested" routerLinkActive="active" id="navMyRequests">My Requests</a></li>
12 |     <li><a routerLink="/customerviewfeedback" routerLinkActive="active" id="navFeedback">Feedback</a></li>
13 |   </ul>
```

Four plain links, each with `routerLink` and `routerLinkActive="active"` (Angular adds the class `active` to the link whose route matches the current URL, which highlights where the user is): Home (`/home`), Available Drivers (`/customerviewdriver`, where drivers are listed and booked), My Requests (`/customerviewrequested`, the customer's bookings and their status) and Feedback (`/customerviewfeedback`, where feedback is viewed and created). The `id` values such as `navAvailableDrivers` are fixed hooks for tests.

Lines 14-16 | Logout button and end of nav

```
14 |     <button type="button" class="logout-btn" id="logoutButton" (click)="openLogoutPopup()">Logout</button>
15 |   </div>
16 | </nav>
```

The red Logout button calls `openLogoutPopup()` on click (it does not log out directly). Lines 15-16 close `.nav-right` and the `<nav>`.

Lines 18-26 | Logout confirmation popup

```
18 | <div class="modal-backdrop" *ngIf="showLogoutPopup">
19 |   <div class="modal-box">
20 |     <p>Are you sure you want to logout?</p>
21 |     <div class="modal-actions row">
22 |       <button type="button" class="btn btn-success" id="confirmLogout" (click)="confirmLogout()">Yes, Logout</button>
23 |       <button type="button" class="btn btn-danger" id="cancelLogout" (click)="cancelLogout()">Cancel</button>
24 |     </div>
25 |   </div>
26 | </div>
```

Identical to the admin navbar. `*ngIf="showLogoutPopup"` draws the full-screen `.modal-backdrop` only while the flag is true. Inside the `.modal-box` the question "Are you sure you want to logout?" and two buttons in `.modal-actions.row`: "Yes, Logout" (green, `confirmLogout()`) and "Cancel" (red, `cancelLogout()`).

Workflow, next: Next: `customernav.component.css`.

Step 86: CustomernavComponent (CSS) - component-level style [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/components/customernav/customernav.component.css

FRONTEND

What this file is	The same tiny stylesheet as the admin navbar's: the real navbar rules are global in <code>styles.css</code> , and only a block display and a popup-button size override are local.
Who / what triggers it	Loaded with <code>CustomernavComponent</code> and scoped to <code><app-customernav></code> .
What it uses / calls	Global <code>.navbar</code> rules (gradient bar, brand, user chip, links, active highlight, logout button, the 720 px responsive rule) and the global <code>.modal-*</code> and <code>.btn-*</code> rules in <code>styles.css</code> .
Size	3 lines

Lines 1-2 | Comment and :host rule

```
1 | /* Navbar look is shared with the customer navbar and lives in the global styles.css. */
2 | :host { display: block; }
```

The comment says the navbar look is shared; it is a copy of the admin comment, so it says "the customer navbar" although this file belongs to the customer navbar itself (the global rules in `styles.css` are shared by

both). `:host { display: block; }` turns the `<app-customernav>` tag into a block-level box so the bar fills the full width.

Line 3 | Popup button size override

```
3 | .modal-actions.row .btn { min-width: 100px; border-radius: 4px; padding: 8px 14px; }
```

Makes the "Yes, Logout" and "Cancel" buttons smaller than the global pill-shaped modal buttons: `min-width: 100px, border-radius: 4px, padding: 8px 14px`. It is more specific than the global `.modal-actions .btn` rule, so it wins.

Workflow, next: This finishes the login, signup, home and navbar files. At this point a Customer has the home page and the customer navbar, and can continue with the Available Drivers page (`customerviewdriver`). An **Admin** continues with the **Add Driver page**, `angularapp/src/app/components/driver-management/driver-management.component.ts`, which is the first file of the next part.

Step 87: DriverManagementComponent (TypeScript) - the Add / Edit Driver form logic [Arjun - Driver & Booking]

FRONTEND

`angularapp/src/app/components/driver-management/driver-management.component.ts`

What this file is

This is the **brain of the Add Driver / Edit Driver page**. One component serves two jobs: when the route is `/driver-management` it is an empty "Add New Driver" form, and when the route is `/driver-management/5` (the `:driverId` part is a **route parameter**) it loads driver 5 from the server, pre-fills the form and becomes an "Edit Driver" form.

A few words you will meet in this file for the first time:

- **Component:** an Angular building block made of three files, a `.ts` class (logic and data), a `.html` template (what is drawn) and a `.css` file (how it looks). The class fields are what the template can show.
- **Reactive form:** a form whose structure (fields and validation rules) is defined in TypeScript code, not in the HTML. The class owns a `FormGroup` object, and the template just attaches inputs to it.
- **Validator:** a small rule attached to one form field, for example "must not be empty" or "must match this pattern". A field with a failing validator makes the field, and so the whole form, `invalid`.
- **Observable / subscribe:** an HTTP call in Angular does not return the answer directly. It returns an `*observable*`, which is like a promise of a future answer. Nothing is sent until someone calls `.subscribe({ next, error });` `next` runs when the answer arrives, `error` runs if the server replies with an error status.
- **Dependency injection (DI):** Angular creates the helper objects a class needs (a service, the router, ...) and hands them to the constructor. You only declare them as constructor parameters.

Who / what triggers it

The Admin clicks **Add Driver** in the admin navbar (`routerLink="/driver-management"`), or clicks **Edit** on a driver card in the View Drivers page (that navigates to `/driver-management/<driverId>`). The router (`app-routing.module.ts`) first runs `AuthGuard`, which only lets users with role `Admin` through (`data: { role: 'Admin' }`), then creates this component.

What it uses / calls

`FormBuilder`, `FormGroup`, `Validators` (Angular reactive forms), `ActivatedRoute` (to read `:driverId`), `Router` (to navigate), `DriverService` (HTTP calls to `/api/driver`), the `Driver` interface, `VEHICLE_TYPES` from `utils/constants.ts`, the browser `FileReader` API, and its own template `driver-management.component.html`.

Size

183 lines

Lines 1-6 | Imports

```
1 | import { Component, ElementRef, OnInit, ViewChild } from '@angular/core';
2 | import { FormBuilder, FormGroup, Validators } from '@angular/forms';
3 | import { ActivatedRoute, Router } from '@angular/router';
4 | import { DriverService } from '../services/driver.service';
5 | import { Driver } from '../models/driver.model';
6 | import { VEHICLE_TYPES } from '../utils/constants';
```

- `Component`, `ElementRef`, `OnInit`, `ViewChild` from `@angular/core`: `Component` is the decorator that makes a class an Angular component; `OnInit` is the interface for the `ngOnInit` start-up method; `ViewChild` and `ElementRef` let the class grab a real HTML element (the file input) from its template so it can clear it later.
- `FormBuilder`, `FormGroup`, `Validators` from `@angular/forms`: `FormBuilder` is a shortcut factory for building a form, `FormGroup` is the type of the finished form, `Validators` holds the ready-made rules (`required`, `min`, `max`, `pattern`).
- `ActivatedRoute`, `Router` from `@angular/router`: `ActivatedRoute` describes the route that opened this page (we read `driverId` from it); `Router` lets code move to another page.
- `DriverService` is the project's HTTP service for drivers; `Driver` is the TypeScript shape of a driver (`models/driver.model.ts`); `VEHICLE_TYPES` is the list `['Sedan', 'SUV', 'Hatchback', 'Truck', 'Van', 'Bike']` used to fill the vehicle drop-down.

Line 8 | Image size limit constant

```
8 | const MAX_IMAGE_BYTES = 2 * 1024 * 1024; // 2 MB
```

`MAX_IMAGE_BYTES = 2 * 1024 * 1024` is 2 megabytes written in bytes ($2 \times 1024 \times 1024 = 2,097,152$). It lives outside the class because it never changes. `onFileChange` compares the chosen file's `size` with it.

Lines 10-14 | @Component decorator

```
10 | @Component({
11 |   selector: 'app-driver-management',
12 |   templateUrl: './driver-management.component.html',
13 |   styleUrls: ['./driver-management.component.css']
14 | })
```

A **decorator** (the `@Something(...)` line) attaches information to the class right below it. `@Component` tells Angular: this class is a component, its HTML tag name is `app-driver-management` (`selector`), its template is `driver-management.component.html` and its styles are `driver-management.component.css`. The selector is not used as a tag anywhere in this app, because the router creates the component directly when the URL matches (it is declared in `AppModule` and in the routing table).

Lines 15-16 | Class declaration and @ViewChild

```
15 | export class DriverManagementComponent implements OnInit {
16 |   @ViewChild('imageInput') imageInput?: ElementRef<HTMLInputElement>;
```

`export class DriverManagementComponent implements OnInit` declares the class and promises Angular that it has an `ngOnInit()` method.

`@ViewChild('imageInput') imageInput?: ElementRef<HTMLInputElement>` asks Angular to look in the template for the element marked with the **template reference** `#imageInput` (the file `<input>`) and put a wrapper around it in this field. `nativeElement` gives the real browser element. The `?` means it may be undefined until the view exists. It is only used in `resetForm()` to empty the file chooser, because a file input cannot be cleared through the form model.

Lines 18-32 | Fields (the data the template can show)

```
18 | driverForm: FormGroup;
19 | vehicleTypes: string[] = VEHICLE_TYPES;
20 |
21 | isEditMode: boolean = false;
22 | driverId?: number;
23 | availabilityStatus: string = 'Active'; // Initially "Active"
24 | existingImage: string = '';
25 | imageBase64: string = '';
26 | imageError: string = '';
27 |
28 | submitted: boolean = false;
29 | formError: string = '';
30 | serverError: string = '';
31 | successMessage: string = '';
32 | showSuccessPopup: boolean = false;
```

- `driverForm`: `FormGroup`: the reactive form object, built in the constructor.
- `vehicleTypes`: copy of the constant list, used by `*ngFor` to build the drop-down options.
- `isEditMode` (false by default) switches the page between add and edit behaviour; `driverId` holds the id when editing.
- `availabilityStatus = 'Active'`: there is **no status field in the form**. A new driver is always created as Active; in edit mode the value loaded from the server is kept and sent back unchanged (status is changed from the Action menu on the View Drivers page instead).
- `existingImage`: the image string the server already has for this driver (edit mode only).
- `imageBase64`: the newly chosen picture converted to a Base64 text string (see `onFileChange`). `imageError` holds the message for a wrong file.

- **submitted**: becomes true on the first Submit click so that untouched fields also show their errors.
- **formError** (general "fill everything" message), **serverError** (message after a failed save), **successMessage** and **showSuccessPopup** (control the green "Driver Added Successfully!" dialog).

Lines 34-39 | Constructor (dependency injection)

```
34 | constructor(
35 |     private fb: FormBuilder,
36 |     private driverService: DriverService,
37 |     private route: ActivatedRoute,
38 |     private router: Router
39 | ) {
```

The four **private** parameters are injected by Angular: **fb** (FormBuilder), **driverService**, **route** (ActivatedRoute) and **router**. Writing **private** in front automatically creates a class field of the same name, so the rest of the class can use **this.driverService** and so on.

Lines 40-49 | The form definition and every validator

```
40 | this.driverForm = this.fb.group({
41 |     driverName: ['', [Validators.required, Validators.pattern(/\S/)]],
42 |     licenseNumber: ['', [Validators.required, Validators.pattern(/^[A-Za-z0-9 \-\/]{5,25}$/)]],
43 |     experienceYears: [null, [Validators.required, Validators.min(0), Validators.max(60), Validators.pattern(/^\d+$/)]],
44 |     contactNumber: ['', [Validators.required, Validators.pattern(/^\+?\d{10,13}$/)]],
45 |     vehicleType: ['', [Validators.required]],
46 |     hourlyRate: [null, [Validators.required, Validators.min(1)]],
47 |     address: ['', [Validators.required, Validators.pattern(/\S/)]],
48 | });
49 | }
```

this.fb.group({...}) builds the form. Each entry is **fieldName: [initialValue, [validators...]]**. The field names must match the **formControlName** attributes in the template. Rules, field by field:

- **driverName**: starts **''**. **Validators.required** fails on empty text. But **required** accepts a string of only spaces, so **Validators.pattern(/\S/)** is added: the regular expression **\S** means "at least one character that is not whitespace". (When you pass a RegExp object, Angular does not add **^** and **\$** itself, so this pattern just searches for one visible character.)
- **licenseNumber**: required plus **/^[A-Za-z0-9 \-\/]{5,25}\$/**. Read it as: from start **^** to end **\$**, between 5 and 25 characters **{5,25}**, each one a letter A-Z / a-z, a digit 0-9, a space, a hyphen **\-** or a slash **\/**. (The backslashes just escape characters that have a special meaning.)
- **experienceYears**: starts **null** (so the number box is empty). **required**, **Validators.min(0)**, **Validators.max(60)** and **/^\d+\$/**, which means digits only: a whole number with no decimal point, no minus sign and no letters. So the accepted range is the whole numbers 0 to 60.
- **contactNumber**: required plus **/^\+?\d{10,13}\$/**: an optional leading plus sign **\+?**, then 10 to 13 digits, nothing else (no spaces or dashes).
- **vehicleType**: just **required**. The first option of the drop-down has the empty value **''**, which counts as "not chosen".
- **hourlyRate**: starts **null**, **required** and **Validators.min(1)**, so the smallest accepted rate is 1. (The error text in the template says "greater than 0", but because the rule is **min(1)** a value such as 0.5 is also rejected.)
- **address**: required plus the same non-blank **/\S/** rule as the name.

Nothing is checked until the user types or submits; the checks run again every time a value changes, and each control keeps an **errors** object such as **{ required: true }** or **{ pattern: {...} }** that the template reads.

Lines 51-73 | `ngOnInit` - decide between add mode and edit mode

```
51 | ngOnInit(): void {
52 |   const id = this.route.snapshot.paramMap.get('driverId');
53 |   if (id) {
54 |     this.isEditMode = true;
55 |     this.driverId = Number(id);
56 |     this.driverService.getDriverById(this.driverId).subscribe({
57 |       next: (driver) => {
58 |         this.availabilityStatus = driver.availabilityStatus || 'Active';
59 |         this.existingImage = driver.image || '';
60 |         this.driverForm.patchValue({
61 |           driverName: driver.driverName,
62 |           licenseNumber: driver.licenseNumber,
63 |           experienceYears: driver.experienceYears,
64 |           contactNumber: driver.contactNumber,
65 |           vehicleType: driver.vehicleType,
66 |           hourlyRate: driver.hourlyRate,
67 |           address: driver.address
68 |         });
69 |       },
70 |       error: () => this.router.navigate(['/admin-view-drivers'])
71 |     });
72 |   }
73 | }
```

`ngOnInit` is a **lifecycle hook**: Angular calls it once, right after the component is created. It runs first when the page opens.

- `this.route.snapshot.paramMap.get('driverId')` reads the `:driverId` piece of the URL as it was when the page opened. On `/driver-management` it is `null` (add mode, nothing else happens and the form stays empty). On `/driver-management/5` it is the text `'5'`.
- In edit mode `isEditMode` becomes true, `driverId = Number(id)` converts the text to a number, and `driverService.getDriverById(...)` sends **GET** `/api/driver/5` with the JWT. On the backend that is `DriverController.viewDriverById`, which asks `DriverServiceImpl.getDriverById` -> `driverRepo.findById` and replies 200 with the driver, or 404 if it does not exist (this URL is ADMIN-only in `SecurityConfig`).
- `next`: runs when the driver arrives. It keeps the current status (`|| 'Active'` is a safety default), remembers the old image string, and calls `this.driverForm.patchValue({...})`. **patchValue** copies the listed values into the matching form controls so the inputs show them; unlike `setValue` it does not insist that every control is listed. The image is deliberately not part of the form: a file input cannot be pre-filled, so the old picture is kept in `existingImage` instead.
- `error`: runs for any failure (for example a driver id that does not exist, 404) and simply sends the Admin back to the list `/admin-view-drivers`.

Lines 75-86 | Small helpers used by the template

```
75 | get f() {
76 |   return this.driverForm.controls;
77 | }
78 |
79 | showError(field: string): boolean {
80 |   const control = this.driverForm.get(field)!;
81 |   return control.invalid && (control.touched || this.submitted);
82 | }
83 |
84 | get imageMissing(): boolean {
85 |   return this.submitted && !this.isEditMode && !this.imageBase64;
86 | }
```

- `get f()` is a **getter**: reading `f` returns `this.driverForm.controls`, the object that holds every control. The template uses it as `f['licenseNumber'].errors` to read the error object of a field.
- `showError(field)` returns true when that field is invalid **and** the user has either touched it (clicked in and out) or already pressed Submit. This is why a fresh empty form shows no red text until the user interacts. The `!` after `get(field)` tells TypeScript the control exists.
- `get imageMissing()` is true only when the form was submitted, we are in add mode and no image has been converted. The template uses it to show "Driver image is required". In edit mode the old picture is acceptable.

Lines 88-109 | onFileChange - reading the picture with FileReader (Base64)

```
88 | onFileChange(event: Event): void {
89 |     const input = event.target as HTMLInputElement;
90 |     const file = input.files && input.files.length ? input.files[0] : null;
91 |     this.imageError = '';
92 |     this.imageBase64 = '';
93 |     if (!file) {
94 |         return;
95 |     }
96 |     if (!file.type.startsWith('image/')) {
97 |         this.imageError = 'Please choose an image file';
98 |         input.value = '';
99 |         return;
100 |     }
101 |     if (file.size > MAX_IMAGE_BYTES) {
102 |         this.imageError = 'Image must be smaller than 2 MB';
103 |         input.value = '';
104 |         return;
105 |     }
106 |     const reader = new FileReader();
107 |     reader.onload = () => (this.imageBase64 = String(reader.result));
108 |     reader.readAsDataURL(file);
109 | }
```

This is the (**change**) handler of the file input; it runs each time the Admin picks (or cancels) a file.

- **event.target as HTMLInputElement** is the input element that fired the event; **input.files** is the list of chosen files. We take the first file, or **null** if nothing was chosen. Both **imageError** and **imageBase64** are reset first, so a new choice always discards the previous picture.
- **Type check:** **file.type** is the MIME type the browser reports (**image/png**, **image/jpeg**, ...). If it does not start with **image/**, the message "Please choose an image file" is stored, **input.value = ''** empties the chooser so the bad file is forgotten, and the function stops.
- **Size check:** **file.size** is in bytes; above **MAX_IMAGE_BYTES** (2 MB) the message "Image must be smaller than 2 MB" is shown and the chooser is cleared.
- **Conversion:** **new FileReader()** is a browser object that reads files asynchronously. **reader.readAsDataURL(file)** starts reading and, when finished, the **reader.onload** callback runs with **reader.result** holding a **data URL** such as **data:image/png;base64,iVBORw0KGgo...** **Base64** is a way of writing binary bytes as plain text characters, so a picture can travel inside JSON; it is about one third bigger than the original file. That text is saved in **imageBase64** and later sent to the server as the driver's **image** field, where the **Driver** entity stores it in a large text column. Because reading is asynchronous, **imageBase64** is filled a moment after the file is chosen.

Lines 111-119 | onSubmit part 1 - validation gate

```
111 | onSubmit(): void {
112 |     this.submitted = true;
113 |     this.formError = '';
114 |     this.serverError = '';
115 |     const imageOk = this.isEditMode || !!this.imageBase64;
116 |     if (this.driverForm.invalid || !imageOk || this.imageError) {
117 |         this.formError = 'All fields are required';
118 |         return;
119 |     }
```

onSubmit() runs when the form is submitted (the template's (**ngSubmit**)). First it sets **submitted = true** (so all errors become visible) and clears the old messages. **imageOk** is true in edit mode (old image is fine) or when a new picture was converted. If the form is invalid, the image is not ok or there is an image error, **formError = 'All fields are required'** is set and the function returns **before any HTTP call**. So nothing is sent while the data is wrong.

Lines 121-132 | onSubmit part 2 - building the Driver object

```
121 |     const value = this.driverForm.value;
122 |     const driver: Driver = {
123 |         driverName: String(value.driverName).trim(),
124 |         licenseNumber: String(value.licenseNumber).trim(),
125 |         experienceYears: Number(value.experienceYears),
126 |         contactNumber: String(value.contactNumber).trim(),
127 |         availabilityStatus: this.availabilityStatus,
128 |         address: String(value.address).trim(),
129 |         vehicleType: value.vehicleType,
130 |         hourlyRate: Number(value.hourlyRate),
131 |         image: this.imageBase64 || this.existingImage
132 |     };
```

this.driverForm.value is a plain object with the current values of all controls. A **Driver** object (the interface from **driver.model.ts**) is built from it: text fields are converted with **String(...).trim()** to remove accidental spaces at both ends, numbers with **Number(...)** (number inputs can hand back text), the status comes from the field

`availabilityStatus`, and `image` is the new Base64 string if one was chosen, otherwise the old `existingImage`. There is no `driverId` in the object; in edit mode the id travels in the URL instead.

Lines 134-145 | onSubmit part 3 - POST (add) or PUT (edit)

```
134 |     if (this.isEditMode && this.driverId !== undefined) {
135 |         this.driverService.updateDriver(this.driverId, driver).subscribe({
136 |             next: () => this.showSuccess('Driver Updated Successfully!'),
137 |             error: (err) => this.handleError(err)
138 |         });
139 |     } else {
140 |         this.driverService.addDriver(driver).subscribe({
141 |             next: () => this.showSuccess('Driver Added Successfully!'),
142 |             error: (err) => this.handleError(err)
143 |         });
144 |     }
145 | }
```

Edit mode (and the id is known): `updateDriver(id, driver)` sends **PUT** `/api/driver/{id}` with the whole driver as JSON. It reaches `DriverController.updateDriver`, which calls `DriverServiceImpl.updateDriver`: it loads the existing row with `findById`, overwrites all nine fields (name, license, experience, contact, status, address, vehicle type, hourly rate, image) with the received values and saves; it returns null (-> 404) if the id does not exist. Note that the update path has **no duplicate-license check** (only add has one) and the `licenseNumber` column has no unique constraint, so editing a driver to another driver's license number is accepted.

Add mode: `addDriver(driver)` sends **POST** `/api/driver`. It reaches `DriverController.addDriver`, which calls `DriverServiceImpl.addDriver`. That method runs `driverRepo.findByLicenseNumber(...)`; if a driver with the same license already exists it throws `DuplicateDriverException`, otherwise `driverRepo.save` inserts the row and the controller answers **201 Created**. In `addDriver` the controller wraps the call in `try { } catch (Exception e)` and turns **any** exception into **409 Conflict with an empty body**. So for a duplicate the 409 is produced by the controller itself, and `GlobalExceptionHandler` (which can also map `DuplicateDriverException` to 409) is never reached on this path.

Both calls use the observable pattern: `next` shows the success popup with a message ("Driver Updated Successfully!" or "Driver Added Successfully!") and `error` passes the failure to `handleError`.

Lines 147-158 | showSuccess and handleError

```
147 | private showSuccess(message: string): void {
148 |     this.successMessage = message;
149 |     this.showSuccessPopup = true;
150 | }
151 |
152 | private handleError(err: any): void {
153 |     if (err && err.status === 409) {
154 |         this.serverError = 'A driver with this license number already exists';
155 |     } else if (err && err.status >= 400 && err.status < 500) {
156 |         this.serverError = 'Unable to save the driver. Please check the details and try again.';
157 |     }
158 | }
```

`showSuccess(message)` stores the text and sets `showSuccessPopup = true`, which makes the template's popup appear.

`handleError(err)` receives the `HttpErrorResponse`. If `err.status === 409` the message becomes **"A driver with this license number already exists"** (this is the duplicate case traced above). Any other 4xx status (for example 400 bad data or 403 not an Admin) gives the generic "Unable to save the driver. Please check the details and try again.". For status 0 (server down) or 5xx nothing is set here, because the global `HttpErrorInterceptor` has already redirected the browser to the `/error` page. The message is shown by the `#serverError` block in the template.

Lines 160-167 | closeSuccessPopup

```
160 | closeSuccessPopup(): void {
161 |     this.showSuccessPopup = false;
162 |     if (this.isEditMode) {
163 |         this.router.navigate(['/admin-view-drivers']);
164 |     } else {
165 |         this.resetForm();
166 |     }
167 | }
```

Runs when the Admin clicks **Ok** in the popup. The popup is hidden. After an edit the Admin is sent back to `/admin-view-drivers`. After an add, `resetForm()` empties the form so another driver can be entered straight away (the Admin stays on the page).

Lines 169-178 | resetForm

```
169 | private resetForm(): void {
170 |     this.driverForm.reset({ driverName: '', licenseNumber: '', experienceYears: null, contactNumber: '', vehicleType:
    |     '', hourlyRate: null, address: '' });
171 |     this.submitted = false;
172 |     this.formError = '';
173 |     this.imageBase64 = '';
174 |     this.imageError = '';
175 |     if (this.imageInput) {
176 |         this.imageInput.nativeElement.value = '';
177 |     }
178 | }
```

`driverForm.reset({...})` sets every control back to its starting value (empty text, `null` for numbers) and also clears the touched/dirty flags, so no red messages remain. Then `submitted`, `formError`, `imageBase64` and `imageError` are cleared, and, through the `@ViewChild` reference, `nativeElement.value = ''` empties the actual file chooser so the old file name disappears. (The `if` guards against the element not existing.)

Lines 180-183 | goBack

```
180 | goBack(): void {
181 |     this.router.navigate(['/admin-view-drivers']);
182 | }
183 | }
```

Called by the **Back** button, which exists only in edit mode. It navigates to `/admin-view-drivers` without saving. The final `}` closes the class.

Workflow, next: The class is now ready: the form object exists, the validators are attached and, in edit mode, the data has been patched in. Angular next renders the template that binds these fields to the screen, `driver-management.component.html`.

Step 88: DriverManagementComponent (HTML) - the driver form template [Arjun - Driver & Booking]

FRONTEND

angularapp/src/app/components/driver-management/driver-management.component.html

What this file is

The **template** of the Add / Edit Driver page: one card-shaped form with seven fields plus a picture chooser, red error texts under each field, and a success popup. It contains no logic of its own; every `{{ }}`, `*ngIf` and (event) points at a field or method of `DriverManagementComponent`. The `id` attributes (such as `driverName`, `submitButton`, `successOk`) give every element a fixed name, which the project's automated UI tests use to find the elements. Template terms used below: **interpolation** `{{ expression }}` prints a value inside text; **structural directives** `*ngIf` (show an element only when a condition is true) and `*ngFor` (repeat an element for each item of a list); **property binding** `[x]="expr"`; **event binding** `(click)="method()"`; `[formGroup]` and `formControlName` connect HTML to a reactive form.

Who / what triggers it

Angular renders this template as soon as `DriverManagementComponent` has been created for the route `/driver-management` or `/driver-management/:driverId`.

What it uses / calls

The component class (`driverForm`, `isEditMode`, `showError()`, `f`, `vehicleTypes`, `onFileChange()`, `onSubmit()`, `goBack()`, `closeSuccessPopup()`, `imageMissing`, `imageError`, `formError`, `serverError`, `successMessage`, `showSuccessPopup()`); the global classes of `styles.css` (`form-card`, `form-control`, `error-text`, `btn`, `modal-backdrop`, ...) and the local rules of `driver-management.component.css`.

Size

70 lines

Lines 1-3 | Form element, Back button and heading

```
1 | <form class="form-card" id="driverForm" [formGroup]="driverForm" (ngSubmit)="onSubmit()" novalidate>
2 |   <button type="button" class="back-btn" id="backButton" *ngIf="isEditMode" (click)="goBack()">Back</button>
3 |   <h2 class="form-title" id="formHeading">{{ isEditMode ? 'Edit Driver' : 'Add New Driver' }}</h2>
```

`<form ... [formGroup]="driverForm" (ngSubmit)="onSubmit()" novalidate>` connects this HTML form to the `driverForm` object of the class. `(ngSubmit)` fires when the user presses the submit button or Enter and calls `onSubmit()` (it also stops the browser from reloading the page). `novalidate` switches off the browser's own pop-up validation bubbles so only our messages appear. Classes `form-card` (white card, 430px wide) come from `styles.css`.

The **Back** button has `type="button"` (so it does not submit the form) and `*ngIf="isEditMode"`, so it exists only when editing; `(click)="goBack()"` returns to the list.

The `<h2>` uses interpolation with a conditional expression: `{{ isEditMode ? 'Edit Driver' : 'Add New Driver' }}` prints one of the two headings.

Lines 5-7 | Name field

```
5 | <label class="form-label" for="driverName">Name<span class="req">*</span></label>
6 | <input type="text" id="driverName" class="form-control" formControlName="driverName" />
7 | <div class="error-text" id="driverNameError" *ngIf="showError('driverName')">*Name is required</div>
```

The `<label for="driverName">` is linked to the input with the same `id`, so clicking the label focuses it; the `<span class="req">*</span>` is a visible required mark. `formControlName="driverName"` binds the input to the control of that name inside `driverForm` (typing updates the control, `patchValue` updates the input).

The error line has `*ngIf="showError('driverName')"`: it is drawn only when the control is invalid and was touched or the form was submitted. The text `*Name is required` is the message (the leading asterisk is part of the visible text).

Lines 9-14 | License number field

```
9 | <label class="form-label" for="licenseNumber">License Number<span class="req">*</span></label>
10 | <input type="text" id="licenseNumber" class="form-control" formControlName="licenseNumber" />
11 | <div class="error-text" id="licenseNumberError" *ngIf="showError('licenseNumber')">
12 |   <span *ngIf="f['licenseNumber'].errors?.['required']">*License Number is required</span>
13 |   <span *ngIf="!f['licenseNumber'].errors?.['required']">*License Number must be 5-25 letters or digits</span>
14 | </div>
```

Same binding pattern. The error block shows one of two messages: `f['licenseNumber'].errors?.['required']` is true when the box is empty, giving `*License Number is required`; the second span, `!f['licenseNumber'].errors?.['required']`, shows when the field is invalid for another reason, which here means the pattern failed, giving `*License Number must be 5-25 letters or digits`. The `?.` (safe navigation) prevents an error when `errors` is null. (The message is a little shorter than the real rule, which also allows spaces, hyphens and slashes.)

Lines 16-21 | Experience years field

```
16 | <label class="form-label" for="experienceYears">Experience Years<span class="req">*</span></label>
17 | <input type="number" id="experienceYears" class="form-control" formControlName="experienceYears" min="0" />
18 | <div class="error-text" id="experienceYearsError" *ngIf="showError('experienceYears')">
19 |   <span *ngIf="f['experienceYears'].errors?.['required']">*Experience is required</span>
20 |   <span *ngIf="!f['experienceYears'].errors?.['required']">*Experience must be a whole number between 0 and 60</span>
21 | </div>
```

`type="number"` shows a numeric input (spinner arrows, only numbers accepted by the browser); `min="0"` is only a browser hint for the spinner, the real rules are the validators in the class. Messages: empty -> `*Experience is required`; any other failure (negative, above 60, decimal) -> `*Experience must be a whole number between 0 and 60`.

Lines 23-28 | Contact number field

```
23 | <label class="form-label" for="contactNumber">Contact Number<span class="req">*</span></label>
24 | <input type="text" id="contactNumber" class="form-control" formControlName="contactNumber" />
25 | <div class="error-text" id="contactNumberError" *ngIf="showError('contactNumber')">
26 |   <span *ngIf="f['contactNumber'].errors?.['required']">*Contact Number is required</span>
27 |   <span *ngIf="!f['contactNumber'].errors?.['required']">*Enter a valid contact number (10-13 digits)</span>
28 | </div>
```

A text input (not number, so a leading `+` is allowed). Empty gives `*Contact Number is required`; a pattern failure gives `*Enter a valid contact number (10-13 digits)`.

Lines 30-35 | Vehicle type drop-down

```
30 | <label class="form-label" for="vehicleType">Vehicle Type<span class="req">*</span></label>
31 | <select id="vehicleType" class="form-control" formControlName="vehicleType">
32 |   <option value="">Choose Vehicle Type</option>
33 |   <option *ngFor="let type of vehicleTypes" [value]="type">{{ type }}</option>
34 | </select>
35 | <div class="error-text" id="vehicleTypeError" *ngIf="showError('vehicleType')">*Vehicle type is required</div>
```

`<select formControlName="vehicleType">` is bound like an input. The first `<option value="">` is the placeholder "Choose Vehicle Type" with the empty value (which fails `required`). `<option *ngFor="let type of vehicleTypes" [value]="type">{{ type }}</option>` repeats an option for each entry of the list (Sedan, SUV, Hatchback, Truck, Van, Bike): `let type` is the loop variable, `[value]="type"` sets the option's value, `{{ type }}` prints its label. In edit mode `patchValue` selects the matching option automatically.

Lines 37-42 | Hourly rate field

```
37 | <label class="form-label" for="hourlyRate">Hourly Rate<span class="req">*</span></label>
38 | <input type="number" id="hourlyRate" class="form-control" formControlName="hourlyRate" min="1" />
39 | <div class="error-text" id="hourlyRateError" *ngIf="showError('hourlyRate')">
40 |   <span *ngIf="f['hourlyRate'].errors?.['required']">*</span>Hourly Rate is required</div>
41 |   <span *ngIf="!f['hourlyRate'].errors?.['required']">*</span>Hourly Rate must be greater than 0</div>
42 | </div>
```

A number input with `min="1"` as hint. Empty gives `"*Hourly Rate is required"`, otherwise (rate below 1) `"*Hourly Rate must be greater than 0"`.

Lines 44-46 | Address field

```
44 | <label class="form-label" for="address">Address<span class="req">*</span></label>
45 | <input type="text" id="address" class="form-control" formControlName="address" />
46 | <div class="error-text" id="addressError" *ngIf="showError('address')">*</div>
```

Plain text input; its only rules are required and not blank, so one message `"*Address is required"` is enough.

Lines 48-53 | Driver image chooser and its errors

```
48 | <label class="form-label" for="image">Driver Image<span class="req" *ngIf="!isEditMode">*</span></label>
49 | <div class="file-box">
50 |   <input type="file" id="image" #imageInput accept="image/*" (change)="onFileChange($event)" />
51 | </div>
52 | <div class="error-text" id="imageError" *ngIf="imageError">*<{{ imageError }}</div>
53 | <div class="error-text" id="imageRequiredError" *ngIf="imageMissing && !imageError">*</div>
```

`<span class="req" *ngIf="!isEditMode">*</span>`: the required star appears only in add mode (in edit mode the old picture is kept if none is chosen).

The `<input type="file" id="image" #imageInput accept="image/*" (change)="onFileChange($event)">` is **not** bound with `formControlName`, because Angular forms cannot hold a file. Instead `(change)` calls `onFileChange($event)` where `$event` is the browser event. `accept="image/*"` makes the file picker show only image files (this is only a convenience; the real check is in the class). `#imageInput` is the **template reference variable** that `@ViewChild('imageInput')` in the class picks up. The `file-box` wrapper div is styled in the component CSS.

Two error lines: `*ngIf="imageError"` prints the type/size message (the `*{{ imageError }}` adds a star in front), and `imageMissing && !imageError` prints `"*Driver image is required"` when the form was submitted in add mode without a picture.

Lines 55-56 | General and server error messages

```
55 | <div class="form-error" id="formError" *ngIf="formError">{{ formError }}</div>
56 | <div class="form-error" id="serverError" *ngIf="serverError">{{ serverError }}</div>
```

`formError` ("All fields are required") appears when the submit gate failed; `serverError` ("A driver with this license number already exists" or the generic 4xx text) appears after a failed save. Each is wrapped in `*ngIf` so it exists only when the string is not empty. Style `form-error` is red small text from `styles.css`.

Lines 58-61 | Submit button and end of form

```
58 | <div class="submit-row">
59 |   <button type="submit" class="btn btn-primary" id="submitButton">{{ isEditMode ? 'Update Driver' : 'Add Driver' }}<
60 |   /button>
61 | </div>
61 | </form>
```

`type="submit"` makes the click trigger (`ngSubmit`). The label switches with `{{ isEditMode ? 'Update Driver' : 'Add Driver' }}`. `btn btn-primary` is the blue button style. `</form>` closes the form.

Lines 63-70 | Success popup (modal)

```
63 | <div class="modal-backdrop" *ngIf="showSuccessPopup">
64 |   <div class="modal-box success-box" id="successPopup">
65 |     <p>{{ successMessage }}</p>
66 |     <div class="modal-actions">
67 |       <button type="button" class="btn btn-primary" id="successOk" (click)="closeSuccessPopup()">Ok</button>
68 |     </div>
69 |   </div>
70 | </div>
```

This block sits outside the form. `*ngIf="showSuccessPopup"` keeps it out of the page until a save succeeded. `modal-backdrop` is a full-screen dim, slightly blurred layer (`position: fixed`, defined in `styles.css`) that centres `modal-box`, the white dialog. `{{ successMessage }}` prints "Driver Added Successfully!" or "Driver Updated Successfully!". The **Ok** button calls `closeSuccessPopup()` (reset the form in add mode, go to the list in edit mode).

Workflow, next: The page is now drawn. Its one component-specific style sheet is `driver-management.component.css`, which only restyles the file chooser. After that, the Admin's next stop is the list where the new driver can be seen.

Step 89: DriverManagementComponent (CSS) - file chooser styling [Arjun - Driver & Booking]

FRONTEND

angularapp/src/app/components/driver-management/driver-management.component.css

What this file is	A very small style sheet. Almost everything on this page (card, labels, inputs, buttons, popup) is styled by the shared rules in <code>src/styles.css</code> . This file only adds the look of the picture chooser box . Angular scopes component CSS: each rule here applies only to elements of this component's own template, so these class names cannot leak to other pages.
Who / what triggers it	Loaded through <code>styleUrls</code> in the <code>@Component</code> decorator of <code>driver-management.component.ts</code> ; the rules apply to the <code>file-box</code> div in the template.
What it uses / calls	The elements <code>div.file-box</code> and the <code>input[type=file]</code> inside it from <code>driver-management.component.html</code> .
Size	13 lines

Lines 1-2 | The box around the file input

```
1 | .file-box { padding: 6px 8px; background: #efefef; border-radius: 4px; }
2 | .file-box input[type='file'] { width: 100%; font-size: 13px; }
```

`.file-box` gives the wrapper a small padding (6px top/bottom, 8px sides), a light grey background `#efefef` (the same grey as the text inputs, so it looks like part of the form) and slightly rounded corners. `.file-box input[type='file']` makes the file input fill the box (`width: 100%`) with a small 13px font for the "No file chosen" text.

Lines 3-13 | The "Choose file" button

```
3 | .file-box input[type='file']::file-selector-button {
4 |   margin-right: 8px;
5 |   padding: 6px 12px;
6 |   border: none;
7 |   border-radius: 3px;
8 |   background: #0a6fe8;
9 |   color: #fff;
10 |   font-size: 12px;
11 |   font-weight: 700;
12 |   cursor: pointer;
13 | }
```

`::file-selector-button` is a **pseudo-element** that targets the browser's built-in "Choose file" button inside a file input, which normally cannot be styled. The rule turns it into a blue button: `margin-right` leaves a gap before the file name, `padding` makes it roomy, `border: none` removes the grey border, `border-radius: 3px` rounds the corners, `background: #0a6fe8` is the project blue, `color: #fff` white text, `font-size: 12px`, `font-weight: 700` bold and `cursor: pointer` shows the hand cursor.

Workflow, next: The Add Driver screen is complete (HTML, class and CSS). Once the success popup is confirmed, the Admin opens **View Drivers** in the navbar to see the new card; that screen is `admin-view-drivers.component.ts`.

Step 90: AdminViewDriversComponent (TypeScript) - driver list, filters, edit, delete, status change [Arjun - Driver & Booking]

FRONTEND

angularapp/src/app/components/admin-view-drivers/admin-view-drivers.component.ts

What this file is	The logic of the Admin's driver list page . It loads all drivers, shows them as cards, lets the Admin search and filter by status, open a driver in the edit form, delete a driver (after a confirmation popup) and change a driver's availability status from an Action menu. Drivers whose status is Inactive are locked : their Edit and Delete buttons are disabled (the Action menu stays usable so the Admin can re-activate them).
Who / what triggers it	The Admin clicks View Drivers in the navbar (<code>/admin-view-drivers</code>) or arrives from the edit form (Back, or Ok after an update). <code>AuthGuard</code> requires role <code>Admin</code> .

What it uses / calls	<code>DriverService</code> (GET all, PUT, DELETE), <code>Router</code> , the <code>Driver</code> model, <code>DRIVER_STATUSES</code> (<code>Active</code> , <code>Inactive</code> , <code>On Leave</code>) from <code>utils/constants.ts</code> , and the image helpers <code>getDriverImage</code> and <code>useDefaultAvatar</code> from <code>utils/driver-image.ts</code> .
Size	129 lines

Lines 1-6 | Imports

```
1 | import { Component, OnInit } from '@angular/core';
2 | import { Router } from '@angular/router';
3 | import { DriverService } from '../services/driver.service';
4 | import { Driver } from '../models/driver.model';
5 | import { DRIVER_STATUSES } from '../utils/constants';
6 | import { getDriverImage, useDefaultAvatar } from '../utils/driver-image';
```

`Component`, `OnInit` (decorator and start-up hook); `Router` to navigate to the edit form; `DriverService` for the HTTP calls; `Driver` for typing; `DRIVER_STATUSES` for the filter drop-down and the Action menu; `getDriverImage` (turns the stored image string into something usable in `<img src>`, with a default silhouette when empty) and `useDefaultAvatar` (an `(error)` handler that swaps in the silhouette if an image fails to load).

Lines 8-12 | @Component decorator

```
8 | @Component({
9 |   selector: 'app-admin-view-drivers',
10 |   templateUrl: './admin-view-drivers.component.html',
11 |   styleUrls: ['./admin-view-drivers.component.css']
12 | })
```

Registers the class as a component with selector `app-admin-view-drivers`, template `admin-view-drivers.component.html` and style sheet `admin-view-drivers.component.css`. The router creates it for the URL `/admin-view-drivers`.

Lines 13-18 | Class and list data

```
13 | export class AdminViewDriversComponent implements OnInit {
14 |   drivers: Driver[] = [];
15 |   filteredDrivers: Driver[] = [];
16 |   searchText: string = '';
17 |   statusFilter: string = 'All';
18 |   statuses: string[] = DRIVER_STATUSES;
```

- `drivers`: the complete list returned by the server (the master copy).
- `filteredDrivers`: the part of that list that matches the current search text and status filter; **the template loops over this one.**
- `searchText` (text typed in the search box) and `statusFilter` ('All' or one status); `statuses` is the list for the drop-down.

Lines 20-26 | Popup/menu state and template helpers

```
20 | openActionDriverId: number | null = null;
21 | showDeletePopup: boolean = false;
22 | driverToDelete: Driver | null = null;
23 | deleteError: string = '';
24 |
25 | getDriverImage = getDriverImage;
26 | useDefaultAvatar = useDefaultAvatar;
```

- `openActionDriverId`: id of the driver whose Action menu is currently open (`null` = none), so only one menu is open at a time.
- `showDeletePopup`, `driverToDelete`, `deleteError`: control the confirmation dialog, remember which driver is about to be deleted and hold an error text if the server refuses.
- `getDriverImage = getDriverImage`; and `useDefaultAvatar = useDefaultAvatar`; copy the imported functions into class fields. The reason is that a template can only call members of its own class, not imported functions.

Lines 28-32 | Constructor and ngOnInit

```
28 | constructor(private driverService: DriverService, private router: Router) {}
29 |
30 | ngOnInit(): void {
31 |   this.loadDrivers();
32 | }
```

Angular injects `DriverService` and `Router`. `ngOnInit()` runs once when the page is created and its only job is to call `loadDrivers()`. This is the first thing that happens on the page.

Lines 34-46 | loadDrivers - GET /api/driver

```
34 | loadDrivers(): void {
35 |   this.driverService.getAllDrivers().subscribe({
36 |     // The API answers 204 (empty body) when there are no drivers.
37 |     next: (drivers) => {
38 |       this.drivers = drivers || [];
39 |       this.applyFilters();
40 |     },
41 |     error: () => {
42 |       this.drivers = [];
43 |       this.applyFilters();
44 |     }
45 |   });
46 | }
```

`driverService.getAllDrivers()` sends **GET /api/driver** with **Authorization: Bearer <token>**. In **SecurityConfig** this URL is **permitAll**, so anyone may call it (the Customer page uses the same call). The backend method is **DriverController.viewAllDrivers**, which calls **DriverServiceImpl.getAllDrivers()** -> **driverRepo.findAll()**. If the table is empty the controller answers **204 No Content** (no body), which Angular delivers as **null**; that is why the code writes **drivers || []**.

On success the master list is stored and **applyFilters()** fills **filteredDrivers**. On any error the list is emptied (so the page shows "Oops! No data found.") and filters are applied to the empty list; for server-down or 5xx errors the global interceptor also redirects to **/error**.

Lines 48-58 | applyFilters - search and status filter

```
48 | applyFilters(): void {
49 |   const query = this.searchText.trim().toLowerCase();
50 |   this.filteredDrivers = this.drivers.filter((driver) => {
51 |     const matchesStatus = this.statusFilter === 'All' || driver.availabilityStatus === this.statusFilter;
52 |     const matchesText =
53 |       !query ||
54 |       (driver.driverName || '').toLowerCase().includes(query) ||
55 |       (driver.vehicleType || '').toLowerCase().includes(query);
56 |     return matchesStatus && matchesText;
57 |   });
58 | }
```

Builds **filteredDrivers** from **drivers** with **Array.filter**, keeping a driver only if **both** conditions hold:

- **matchesStatus**: the filter is **'All'** or the driver's **availabilityStatus** equals the chosen status.
- **matchesText**: the search box is empty, or the lower-cased driver name **or** vehicle type contains the lower-cased, trimmed query (**includes**). The **|| ''** guards against missing values. Note that on this admin page the license number is not searched.

It is called after loading, whenever the search text changes and whenever the status filter changes (the template calls it from (**ngModelChange**)), and after a status update.

Lines 60-63 | isLocked

```
60 | /** Inactive drivers cannot be edited or deleted. */
61 | isLocked(driver: Driver): boolean {
62 |   return driver.availabilityStatus === 'Inactive';
63 | }
```

Returns true when the driver's status is exactly **Inactive**. The template uses it to disable Edit and Delete, and the methods below use it as a second safety net. This lock exists only in the Angular app; the backend would accept an update or delete of an Inactive driver if called directly.

Lines 65-70 | editDriver - go to the edit form

```
65 | editDriver(driver: Driver): void {
66 |   if (this.isLocked(driver)) {
67 |     return;
68 |   }
69 |   this.router.navigate(['/driver-management', driver.driverId]);
70 | }
```

Does nothing for a locked driver. Otherwise **router.navigate(['/driver-management', driver.driverId])** builds the URL **/driver-management/<id>**. That matches the route with **:driverId**, so **DriverManagementComponent** opens in edit mode and loads the driver.

Lines 72-85 | openDeletePopup and cancelDelete

```
72 | openDeletePopup(driver: Driver): void {
73 |     if (this.isLocked(driver)) {
74 |         return;
75 |     }
76 |     this.driverToDelete = driver;
77 |     this.deleteError = '';
78 |     this.showDeletePopup = true;
79 | }
80 |
81 | cancelDelete(): void {
82 |     this.showDeletePopup = false;
83 |     this.driverToDelete = null;
84 |     this.deleteError = '';
85 | }
```

`openDeletePopup(driver)` (Delete button): ignores locked drivers, remembers the driver in `driverToDelete`, clears any old error and shows the popup. `cancelDelete()` (Cancel button, and also reused after a successful delete) hides the popup and clears the remembered driver and the error.

Lines 87-105 | confirmDelete - DELETE /api/driver/{id}

```
87 | confirmDelete(): void {
88 |     if (!this.driverToDelete || this.driverToDelete.driverId === undefined) {
89 |         return;
90 |     }
91 |     this.driverService.deleteDriver(this.driverToDelete.driverId).subscribe({
92 |         next: () => {
93 |             this.cancelDelete();
94 |             this.loadDrivers();
95 |         },
96 |         error: (err) => {
97 |             if (err && err.status === 409) {
98 |                 this.deleteError = 'This driver has requests or feedback and cannot be deleted.';
99 |             } else if (err && err.status === 404) {
100 |                 this.cancelDelete();
101 |                 this.loadDrivers();
102 |             }
103 |         }
104 |     });
105 | }
```

The "Yes, Delete" button. A guard returns if there is no driver or no id. Otherwise `deleteDriver(id)` sends **DELETE** `/api/driver/{id}` (ADMIN only). Backend chain: `DriverController.deleteDriver` -> `DriverServiceImpl.deleteDriver(id)`. The service does `findById`; if the driver does not exist it returns null and the controller answers **404**. Otherwise it calls `driverRepo.delete(...)` inside a `try`. If the database refuses (the driver is still referenced by rows in the DriverRequest or Feedback tables through foreign keys), the `catch` throws `DriverDeletionException`. The controller does not catch it, so it travels to `GlobalExceptionHandler.handleConflict`, which stores an `ErrorLog` row and answers **409 Conflict** with a JSON message. On success the controller answers 200 with the deleted driver.

In Angular: `next` closes the popup (`cancelDelete()`) and reloads the list. In `error`: status **409** puts "This driver has requests or feedback and cannot be deleted." into `deleteError`, shown inside the still-open popup; status **404** (someone else already deleted it) closes the popup and reloads the list; other statuses are handled by the interceptor (5xx -> /error page).

Lines 107-113 | otherStatuses and toggleActionMenu

```
107 | otherStatuses(driver: Driver): string[] {
108 |     return this.statuses.filter((status) => status !== driver.availabilityStatus);
109 | }
110 |
111 | toggleActionMenu(driver: Driver): void {
112 |     this.openActionDriverId = this.openActionDriverId === driver.driverId ? null : (driver.driverId ?? null);
113 | }
```

`otherStatuses(driver)` returns the statuses **except the current one**, so the Action menu for an Active driver offers **Inactive** and **On Leave**. `toggleActionMenu(driver)` opens the menu for that driver, or closes it if it is already the open one (`condition ? null : (id ?? null)`; `??` gives `null` when the id is undefined).

Lines 115-129 | `changeStatus` - PUT with the new `availabilityStatus`

```
115 | changeStatus(driver: Driver, status: string): void {
116 |   if (driver.driverId === undefined) {
117 |     return;
118 |   }
119 |   const updated: Driver = { ...driver, availabilityStatus: status };
120 |   this.driverService.updateDriver(driver.driverId, updated).subscribe({
121 |     next: () => {
122 |       driver.availabilityStatus = status;
123 |       this.openActionDriverId = null;
124 |       this.applyFilters();
125 |     },
126 |     error: () => (this.openActionDriverId = null)
127 |   });
128 | }
129 | }
```

Called when the Admin picks a status in the Action menu. `{ ...driver, availabilityStatus: status }` uses the **spread operator** to copy every field of the driver (including the big Base64 image) into a new object and replace only `availabilityStatus`. `updateDriver(id, updated)` then sends **PUT** `/api/driver/{id}` with that whole driver, handled by `DriverController.updateDriver` -> `DriverServiceImpl.updateDriver` (overwrites all fields and saves, as in the edit form).

On `next`, the card in memory gets the new status, the menu is closed and `applyFilters()` runs, so with a status filter active a driver that no longer matches vanishes immediately. On `error` the menu is just closed and nothing changes. The last `}` closes the class.

Workflow, next: After `loadDrivers()` returns, `filteredDrivers` holds the cards to draw and Angular renders the template `admin-view-drivers.component.html`.

Step 91: AdminViewDriversComponent (HTML) - driver cards, toolbar and delete popup [Arjun - Driver & Booking]

FRONTEND

`angularapp/src/app/components/admin-view-drivers/admin-view-drivers.component.html`

What this file is	The template of the Admin's driver list: a heading, a toolbar with a text search box and a status drop-down, a responsive grid of driver cards (photo, details, Edit / Delete / Action buttons) and a "Are you sure you want to delete this driver?" popup. It shows "Oops! No data found." when nothing matches.
Who / what triggers it	Rendered by Angular when <code>AdminViewDriversComponent</code> is created; it re-renders automatically whenever a class field used here changes.
What it uses / calls	Fields and methods of <code>AdminViewDriversComponent</code> ; directives <code>ngModel</code> (from <code>FormsModule</code>), <code>*ngIf</code> , <code>*ngFor</code> ; global classes from <code>styles.css</code> (<code>page-container</code> , <code>page-card</code> , <code>toolbar</code> , <code>btn</code> , <code>modal-box</code>) and local ones from <code>admin-view-drivers.component.css</code> .
Size	63 lines

Lines 1-4 | Page container and heading

```
1 | <div class="page-container">
2 |   <div class="page-card" id="driversCard">
3 |     <h2 class="page-title left" id="driversHeading">Drivers</h2>
4 |     <div class="title-line"></div>
```

`page-container` centres the content (max width 1040px) and `page-card` is the white rounded card (both global). The heading `Drivers` has classes `page-title left` (the local `.left` aligns it to the left); `title-line` is an empty div that draws the thin blue line under the title.

Lines 6-18 | Toolbar: search box and status filter

```
6 | <div class="toolbar">
7 |   <label>
8 |     Search by:
9 |     <input type="text" id="searchInput" placeholder="Driver Name or Vehicle Type" [(ngModel)]="searchText" (ngModelChange)="applyFilters()" />
10 |   </label>
11 |   <label>
12 |     Filter by Status:
13 |     <select id="statusFilter" [(ngModel)]="statusFilter" (ngModelChange)="applyFilters()">
14 |       <option value="All">All Status</option>
15 |       <option *ngFor="let status of statuses" [value]="status">{{ status }}</option>
16 |     </select>
17 |   </label>
18 | </div>
```

`[(ngModel)]="searchText"` is **two-way binding** (the "banana in a box"): the input shows the field `searchText` and every keystroke writes back to it. `(ngModelChange)="applyFilters()"` additionally calls `applyFilters()` on each change, so the list narrows while typing. The select is bound to `statusFilter` the same way. Its options are the fixed `All Status` (value `All`) plus `<option *ngFor="let status of statuses" [value]="status">` repeating once per status (`Active`, `Inactive`, `On Leave`). `ngModel` works here because `FormsModule` is imported in `AppModule`.

Line 20 | Empty message

```
20 | <div class="empty-text" id="noDataMessage" *ngIf="filteredDrivers.length === 0">Oops! No data found.</div>
```

`*ngIf="filteredDrivers.length === 0"` shows "Oops! No data found." (id `noDataMessage`) when the filtered list is empty: no drivers exist, or none match. `empty-text` is bold centred text from the global CSS.

Lines 22-26 | Cards container, ngFor and the photo

```
22 | <div class="cards-area" *ngIf="filteredDrivers.length > 0">
23 |   <div class="driver-card" *ngFor="let driver of filteredDrivers" [attr.id]="driver.driverId">
24 |     <div class="card-header">
25 |       <img class="avatar" [src]="getDriverImage(driver.image)" (error)="useDefaultAvatar($event)" alt="Driver" />
26 |     </div>
```

The `cards-area` div is drawn only if there is at least one driver (`*ngIf="filteredDrivers.length > 0"`). Inside, `*ngFor="let driver of filteredDrivers"` creates one `driver-card` per driver. `[attr.id]="driver.driverId"` sets a real HTML id such as `driver-3` (the `attr.` prefix is needed to bind an attribute, not a DOM property). In the header, `<img [src]="getDriverImage(driver.image)" (error)="useDefaultAvatar($event)">` binds the picture source to the helper's result (a data URL for stored images, the silhouette when empty), and if the image cannot be displayed the browser's `error` event calls `useDefaultAvatar`.

Lines 28-37 | Card body: driver details

```
28 | <div class="card-body">
29 |   <h3>{{ driver.driverName }}</h3>
30 |   <p><strong>Vehicle Type:</strong> {{ driver.vehicleType }}</p>
31 |   <p><strong>License:</strong> {{ driver.licenseNumber }}</p>
32 |   <p><strong>Phone:</strong> {{ driver.contactNumber }}</p>
33 |   <p><strong>Address:</strong> {{ driver.address }}</p>
34 |   <p><strong>Experience:</strong> {{ driver.experienceYears }} years</p>
35 |   <p><strong>Hourly Rate:</strong> ₹{{ driver.hourlyRate }}</p>
36 |   <p><strong>Status:</strong> {{ driver.availabilityStatus }}</p>
37 | </div>
```

`{{ driver.driverName }}` and the other interpolations print the fields name, vehicle type, license number, phone, address, experience (followed by the word "years"), hourly rate (preceded by `₹`, the HTML code for the rupee sign) and the current status. `<strong>` makes the labels bold.

Lines 39-41 | Card footer: Edit and Delete buttons

```
39 | <div class="card-footer">
40 |   <button type="button" class="btn btn-success" [id]="editId" [disabled]="isLocked(driver)"
41 |     (click)="editDriver(driver)">Edit</button>
41 |   <button type="button" class="btn btn-danger" [id]="deleteId" [disabled]="isLocked(driver)"
41 |     (click)="openDeletePopup(driver)">Delete</button>
```

Both buttons use `[id]="editId" / "deleteId" + ...` to get unique ids, `[disabled]="isLocked(driver)"` to be greyed out and unclickable for Inactive drivers, and `(click)` to call `editDriver(driver)` or `openDeletePopup(driver)`, passing the driver of the current loop round. Classes `btn btn-success` (green) and `btn btn-danger` (red) are global.

Lines 42-48 | Action menu

```
42 |         <div class="action-wrap">
43 |             <button type="button" class="btn btn-warning btn-pill" [id]='action-' + driver.driverId" (click)="toggleA
         ctionMenu(driver)">Action</button>
44 |             <div class="action-menu" *ngIf="openActionDriverId === driver.driverId">
45 |                 <button type="button" *ngFor="let status of otherStatuses(driver)" (click)="changeStatus(driver, status)
         ">{{ status }}</button>
46 |             </div>
47 |         </div>
48 |     </div>
```

The orange pill-shaped **Action** button calls `toggleActionMenu(driver)`. The dropdown div is rendered only when `openActionDriverId === driver.driverId` (`*ngIf`), so just the clicked driver's menu appears. Inside, `*ngFor="let status of otherStatuses(driver)"` builds one button per alternative status, and `(click)="changeStatus(driver, status)"` starts the PUT call. `action-wrap` and `action-menu` are positioned by the local CSS so the menu floats above the button.

Lines 49-52 | Closing tags

```
49 |         </div>
50 |     </div>
51 | </div>
52 | </div>
```

Lines 49-52 close the `driver-card`, `cards-area`, `page-card` and `page-container` divs opened above.

Lines 54-63 | Delete confirmation popup

```
54 | <div class="modal-backdrop" *ngIf="showDeletePopup">
55 |     <div class="modal-box" id="deletePopup">
56 |         <p>Are you sure you want to delete this driver?</p>
57 |         <div class="form-error center" *ngIf="deleteError">{{ deleteError }}</div>
58 |         <div class="modal-actions">
59 |             <button type="button" class="btn btn-danger" id="confirmDelete" (click)="confirmDelete()">Yes, Delete</button>
60 |             <button type="button" class="btn btn-secondary" id="cancelDelete" (click)="cancelDelete()">Cancel</button>
61 |         </div>
62 |     </div>
63 | </div>
```

Outside the page card, so it can cover the whole screen. `*ngIf="showDeletePopup"` shows it only after Delete was clicked. The sentence asks for confirmation; the red `form-error center` div appears only when `deleteError` has text (the 409 message). **Yes, Delete** calls `confirmDelete()` and **Cancel** calls `cancelDelete()`. `modal-actions` stacks the buttons in the centre.

Workflow, next: The cards are visible. All visual layout that is specific to this list (grid of cards, round avatar, floating action menu) comes from `admin-view-drivers.component.css`.

Step 92: AdminViewDriversComponent (CSS) - card grid and action menu [Arjun - Driver & Booking]

FRONTEND

angularapp/src/app/components/admin-view-drivers/admin-view-drivers.component.css

What this file is	Styles that make the Admin driver list look like a two-column grid of cards with a blue header, a round photo, a grey footer of buttons and a floating Action menu. Basic building blocks (buttons, popup, toolbar) come from the global <code>styles.css</code> .
Who / what triggers it	Loaded through <code>styleUrls</code> of <code>admin-view-drivers.component.ts</code> and applied only inside this component's template (Angular scopes the rules).
What it uses / calls	The class names used in <code>admin-view-drivers.component.html</code> : <code>page-title</code> , <code>title-line</code> , <code>cards-area</code> , <code>driver-card</code> , <code>card-header</code> , <code>avatar</code> , <code>card-body</code> , <code>card-footer</code> , <code>action-wrap</code> , <code>action-menu</code> , <code>form-error</code> .
Size	59 lines

Lines 1-2 | Title alignment

```
1 | .page-title.left { text-align: left; padding-left: 8px; }
2 | .title-line { width: 82%; margin-left: auto; margin-right: 24px; }
```

`.page-title.left` (an element that has both classes) aligns the heading to the left with 8px padding. `.title-line` makes the blue rule 82% wide and pushes it to the right with `margin-left: auto`, `margin-right: 24px`, so it starts after the heading text.

Lines 4-12 | The grid area

```
4 | .cards-area {
5 |   display: flex;
6 |   flex-wrap: wrap;
7 |   justify-content: center;
8 |   gap: 16px;
9 |   padding: 14px;
10 |  background: #f7f7f7;
11 |  border-radius: 6px;
12 | }
```

`.cards-area` is a **flexbox** container: `display: flex` lays the cards in a row, `flex-wrap: wrap` moves cards to the next line when there is no room, `justify-content: center` centres each line, `gap: 16px` leaves space between cards. `padding`, light grey `background: #f7f7f7` and `border-radius` give it a tray look.

Lines 13-22 | A driver card

```
13 | .driver-card {
14 |   width: calc(50% - 8px);
15 |   min-width: 280px;
16 |   display: flex;
17 |   flex-direction: column;
18 |   background: #fff;
19 |   border-radius: 6px;
20 |   box-shadow: 0 2px 8px rgba(0, 0, 0, 0.18);
21 |   overflow: visible;
22 | }
```

`width: calc(50% - 8px)` makes two cards per row (half the width minus half the gap), `min-width: 280px` prevents cards from becoming too narrow on small screens (they then wrap to one per row). `display: flex`; `flex-direction: column` stacks header, body and footer vertically. White background, 6px rounded corners and a soft `box-shadow` lift the card; `overflow: visible` is essential so the Action menu can stick out beyond the card edge.

Lines 23-31 | Card header and avatar

```
23 | .card-header {
24 |   display: flex;
25 |   justify-content: center;
26 |   align-items: center;
27 |   height: 150px;
28 |   background: linear-gradient(135deg, #2f78b7, #3d93e6);
29 |   border-radius: 6px 6px 0 0;
30 | }
31 | .avatar { width: 96px; height: 96px; object-fit: cover; border-radius: 50%; border: 3px solid #1d5f64; background: #fff; }
```

The header is a 150px tall flex box that centres its content and has a diagonal blue `linear-gradient(135deg, #2f78b7, #3d93e6)` background with rounded top corners. `.avatar` is the photo: 96 x 96 px, `object-fit: cover` crops the picture to fill the square without stretching, `border-radius: 50%` turns it into a circle, plus a dark teal 3px border and a white background (visible for the transparent default silhouette).

Lines 32-35 | Card body text

```
32 | .card-body { padding: 16px 18px 10px; }
33 | .card-body h3 { margin: 0 0 12px; font-size: 18px; color: #444; }
34 | .card-body p { margin: 7px 0; font-size: 13px; color: #555; }
35 | .card-body strong { color: #222; }
```

Padding around the details; the driver name `h3` is 18px dark grey with a bottom margin; each detail line `p` is 13px grey with 7px vertical spacing; the bold labels (`strong`) are nearly black so they stand out.

Lines 36-44 | Card footer

```
36 | .card-footer {
37 |   display: flex;
38 |   justify-content: center;
39 |   gap: 8px;
40 |   padding: 12px;
41 |   margin-top: auto;
42 |   background: #f3f3f3;
43 |   border-radius: 0 0 6px 6px;
44 | }
```

A centred flex row of buttons with an 8px gap, light grey `#f3f3f3` background and rounded bottom corners. `margin-top: auto` pushes the footer to the bottom of the card so cards of different height keep their buttons aligned.

Lines 45-58 | Action menu

```
45 | .action-wrap { position: relative; }
46 | .action-menu {
47 |   position: absolute;
48 |   bottom: 110%;
49 |   right: 0;
50 |   z-index: 20;
51 |   min-width: 120px;
52 |   background: #fff;
53 |   border-radius: 4px;
54 |   box-shadow: 0 4px 14px rgba(0, 0, 0, 0.28);
55 |   overflow: hidden;
56 | }
57 | .action-menu button { display: block; width: 100%; padding: 9px 14px; border: none; background: #fff; text-align: left
58 |   ; font-size: 13px; cursor: pointer; }
59 | .action-menu button:hover { background: #e8f0fb; }
```

`.action-wrap { position: relative }` makes the wrapper the anchor for the absolutely positioned menu. `.action-menu` uses `position: absolute; bottom: 110%; right: 0` to float just **above** the Action button, right-aligned, with `z-index: 20` to stay in front of other content, a minimum width, white background, rounded corners and a stronger shadow; `overflow: hidden` clips the hover colour to the rounded corners. The menu's buttons are full-width blocks (`display: block; width: 100%`), borderless, left-aligned, with a hand cursor, and turn light blue (`#e8f0fb`) on hover.

Line 59 | Delete error message

```
59 | .form-error.center { text-align: center; margin: -6px 0 12px; }
```

`.form-error.center` re-uses the red error style but centres the text and uses a negative top margin so the 409 message sits snugly below the confirmation question in the popup.

Workflow, next: That completes the Admin's driver pages (add, edit, delete, status). The walk now switches to the **Customer** role. After logging in, the Customer opens **Available Drivers** in the customer navbar, which is `customerviewdriver.component.ts`.

Step 93: CustomerviewdriverComponent (TypeScript) - Available Drivers, plain search and AI search [Arjun - Driver & Booking + Saumya (AI search)]

FRONTEND

angularapp/src/app/components/customerviewdriver/customerviewdriver.component.ts

What this file is	The logic of the Customer's Available Drivers page. It shows every driver as a card, lets the Customer filter them with a plain text box, offers an AI Search box that accepts a natural sentence ("experienced SUV driver for a night outstation trip") and marks drivers the customer has already requested with a disabled Requested button. Pressing Request on an available driver moves on to the booking form.
Who / what triggers it	The Customer clicks Available Drivers in the customer navbar (<code>/customerviewdriver</code>). <code>AuthGuard</code> requires role <code>Customer</code> .
What it uses / calls	<code>DriverService.getAllDrivers</code> , <code>DriverRequestService.getDriverRequestsByUserId</code> , <code>AiService.searchDrivers</code> , <code>AuthService.getUserId</code> (reads <code>userId</code> from <code>localStorage</code>), <code>Router</code> , the <code>Driver</code> model and the image helpers.
Size	126 lines

Lines 1-8 | Imports

```
1 | import { Component, OnInit } from '@angular/core';
2 | import { Router } from '@angular/router';
3 | import { DriverService } from '../services/driver.service';
4 | import { DriverRequestService } from '../services/driver-request.service';
5 | import { AiService } from '../services/ai.service';
6 | import { AuthService } from '../services/auth.service';
7 | import { Driver } from '../models/driver.model';
8 | import { getDriverImage, useDefaultAvatar } from '../utils/driver-image';
```

`Component`, `OnInit`; `Router` (navigate to the booking form); three services: `DriverService` (list of drivers), `DriverRequestService` (this customer's requests) and `AiService` (AI search); `AuthService` (to know who is logged in); the `Driver` type; and the image helpers `getDriverImage` / `useDefaultAvatar`, as on the admin page.

Lines 10-11 | OPEN_STATUSES

```
10 | /** Request statuses that mean "this driver is currently requested by me". */
11 | const OPEN_STATUSES = ['Pending', 'Approved', 'Trip End'];
```

A constant array of the request statuses that mean "I currently have this driver": **Pending**, **Approved** and **Trip End**. A driver whose only requests are **Rejected** or **Closed** does **not** count, so the customer may request that driver again.

Lines 13-17 | @Component decorator

```
13 | @Component({
14 |   selector: 'app-customerviewdriver',
15 |   templateUrl: './customerviewdriver.component.html',
16 |   styleUrls: ['./customerviewdriver.component.css']
17 | })
```

Selector **app-customerviewdriver**, template **customerviewdriver.component.html**, styles **customerviewdriver.component.css**. The router creates it for **/customerviewdriver**.

Lines 18-20 | Class, driver list and plain search text

```
18 | export class CustomerviewdriverComponent implements OnInit {
19 |   drivers: Driver[] = [];
20 |   searchText: string = '';
```

drivers is the master list from the server; **searchText** is bound to the plain search box.

Lines 22-29 | AI search state and requestedDriverIds

```
22 | // AI (semantic) search
23 | aiQuery: string = '';
24 | aiActive: boolean = false;
25 | aiLoading: boolean = false;
26 | aiResults: Driver[] = [];
27 | aiMessage: string = '';
28 |
29 | requestedDriverIds: Set<number> = new Set<number>();
```

- **aiQuery**: the sentence typed in the AI box.
- **aiActive**: true while AI results are being displayed instead of the full list.
- **aiLoading**: true during the HTTP call (disables the button and changes its label).
- **aiResults**: the drivers returned by the AI search; **aiMessage**: a warning text.
- **requestedDriverIds**: a **Set<number>** (a collection with no duplicates and a fast **has()** check) of the ids of drivers this customer already has an open request for.

Lines 31-32 | Helper references for the template

```
31 | getDriverImage = getDriverImage;
32 | useDefaultAvatar = useDefaultAvatar;
```

Same trick as on the admin page: the imported functions are stored in fields so the template can call **getDriverImage(...)** and **useDefaultAvatar(\$event)**.

Lines 34-40 | Constructor

```
34 | constructor(
35 |   private driverService: DriverService,
36 |   private driverRequestService: DriverRequestService,
37 |   private aiService: AiService,
38 |   private authService: AuthService,
39 |   private router: Router
40 | ) {}
```

Angular injects the five collaborators: the three services, **AuthService** and the **Router**.

Lines 42-49 | ngOnInit - two calls on start-up

```
42 | ngOnInit(): void {
43 |   this.driverService.getAllDrivers().subscribe({
44 |     // The API answers 204 (empty body) when there are no drivers.
45 |     next: (drivers) => (this.drivers = drivers || []),
46 |     error: () => (this.drivers = [])
47 |   });
48 |   this.loadMyRequests();
49 | }
```

getAllDrivers() sends **GET /api/driver** (**permitAll** in **SecurityConfig**) which is answered by **DriverController.viewAllDrivers -> DriverServiceImpl.getAllDrivers**. A 204 reply (no drivers) arrives as **null**, hence **drivers || []**; on error the list is empty. Then **loadMyRequests()** is called so the page can mark drivers that are already requested. The two calls run in parallel.

Lines 51-67 | loadMyRequests - which drivers did I already request?

```
51 | private loadMyRequests(): void {
52 |   const userId = this.authService.getUserId();
53 |   if (userId === null) {
54 |     return;
55 |   }
56 |   this.driverRequestService.getDriverRequestsByUserId(userId).subscribe({
57 |     next: (requests) => {
58 |       this.requestedDriverIds = new Set<number>(
59 |         (requests || [])
60 |           .filter((request) => OPEN_STATUSES.includes(request.status))
61 |           .map((request) => request.driver?.driverId)
62 |           .filter((id): id is number => id !== undefined && id !== null)
63 |       );
64 |     },
65 |     error: () => (this.requestedDriverIds = new Set<number>()) // 404 = no requests yet
66 |   });
67 | }
```

`authService.getUserId()` returns the number stored in `localStorage` at login (or `null`, in which case nothing is done). `getDriverRequestsByUserId(userId)` sends **GET** `/api/driverRequest/user/{userId}`, allowed for role `CUSTOMER`. Backend: `DriverRequestController.viewDriverRequestsByUserId` ->

`DriverRequestServiceImpl.findDriverRequestsByUserId` -> `driverRequestRepo.findByUserUserId` (a Spring Data derived query: requests whose `user.userId` equals the id). If the list is empty the controller answers **404**, which Angular treats as an error.

On success the chain is: keep only requests whose `status` is in `OPEN_STATUSES` (`filter`), take each request's `request.driver?.driverId` (`map`; `?.` avoids a crash when `driver` is null), drop missing ids with a second `filter` whose type guard `(id): id is number` tells TypeScript that only numbers remain, and wrap the ids in a new `Set`. On `error` (typically the 404 meaning "no requests yet") the set is simply empty. The 404 does not trigger a redirect because the interceptor ignores 4xx errors other than 401.

Lines 69-82 | displayedDrivers - the list the page draws

```
69 | /** Drivers shown on the page: the AI matches (when active) narrowed down by the text search. */
70 | get displayedDrivers(): Driver[] {
71 |   const base = this.aiActive ? this.aiResults : this.drivers;
72 |   const query = this.searchText.trim().toLowerCase();
73 |   if (!query) {
74 |     return base;
75 |   }
76 |   return base.filter(
77 |     (driver) =>
78 |       (driver.driverName || '').toLowerCase().includes(query) ||
79 |       (driver.vehicleType || '').toLowerCase().includes(query) ||
80 |       (driver.licenseNumber || '').toLowerCase().includes(query)
81 |   );
82 | }
```

A **getter** (`get ...()`): the template reads `displayedDrivers` like a field and the code runs again whenever Angular re-checks the page, so the list always reflects the current state. Step 1: the base list is `aiResults` when `aiActive` is true, otherwise all `drivers`. Step 2: if the plain search box is empty, return the base list unchanged. Step 3: otherwise keep drivers whose name, vehicle type **or license number** contains the lower-cased query. Because the plain search is applied on top of the base list, it also narrows down AI results.

Lines 84-90 | isRequested and canRequest

```
84 | isRequested(driver: Driver): boolean {
85 |   return driver.driverId !== undefined && this.requestedDriverIds.has(driver.driverId);
86 | }
87 |
88 | canRequest(driver: Driver): boolean {
89 |   return driver.availabilityStatus === 'Active' && !this.isRequested(driver);
90 | }
```

`isRequested(driver)`: true if the driver has an id and that id is in `requestedDriverIds`. `canRequest(driver)`: true only when the driver's status is **Active** **and** the customer has no open request for them. So **Inactive** and **On Leave** drivers can be seen but not requested. These rules are enforced only here in the screen; the server does not check them.

Lines 92-96 | requestDriver

```
92 | requestDriver(driver: Driver): void {
93 |   if (this.canRequest(driver)) {
94 |     this.router.navigate(['/customer-request', driver.driverId]);
95 |   }
96 | }
```

Called by the **Request** button. If `canRequest` still holds, it navigates to `/customer-request/<driverId>`, the booking form for that driver (route `customer-request/:driverId`).

Lines 98-118 | runAiSearch - POST /api/ai/driver-search

```
98 | runAiSearch(): void {
99 |     const query = this.aiQuery.trim();
100 |     if (!query) {
101 |         return;
102 |     }
103 |     this.aiLoading = true;
104 |     this.aiMessage = '';
105 |     this.aiService.searchDrivers(query).subscribe({
106 |         next: (results) => {
107 |             this.aiResults = results || [];
108 |             this.aiActive = true;
109 |             this.aiLoading = false;
110 |         },
111 |         error: () => {
112 |             // AI search is optional: fall back to the normal list.
113 |             this.aiActive = false;
114 |             this.aiLoading = false;
115 |             this.aiMessage = 'AI search is currently unavailable. Showing all drivers.';
116 |         }
117 |     });
118 | }
```

Runs on submit of the AI form. The text is trimmed and an empty query is ignored. `aiLoading = true` and the old message is cleared. `aiService.searchDrivers(query)` sends **POST /api/ai/driver-search** with the sentence as a **plain text body** (Content-Type: text/plain, see `ai.service.ts`; the URL is `permitAll`).

Backend chain: `AiController.searchDrivers(@RequestBody String query)` receives the raw text and returns `aiService.searchDrivers(query)` wrapped in 200 OK. In `AiService.searchDrivers`: if the query is blank or there are no drivers, all drivers are returned. Otherwise it tries `semanticSearch`: if a Gemini key is configured, the query and each driver's profile text (name, vehicle, experience, rate, status, address) are turned into **embeddings** (number vectors that capture meaning), compared with **cosine similarity**, and the best 10 are returned. If Gemini is not configured or any embedding call fails, `semanticSearch` returns null and the **keyword fallback** `keywordSearch` is used instead: it splits the sentence into words, removes stop words, adds points for each word found in the driver's profile, adds bonuses for words such as "experienced" or "cheap", gives Active drivers a small extra, and returns the matching drivers sorted by score. So the endpoint normally answers 200 even without an AI key.

`next`: the result array (or `[]`) is stored in `aiResults`, `aiActive = true` switches the page to AI mode and the loading flag is cleared. `error` (the call itself failed, for example the server is unreachable): AI search is treated as optional, so `aiActive` becomes false (the normal full list stays visible), loading stops and the friendly text "**AI search is currently unavailable. Showing all drivers.**" is shown. The interceptor deliberately ignores `/api/ai/` calls, so a failed AI search never redirects to the error page.

Lines 120-126 | clearAiSearch

```
120 | clearAiSearch(): void {
121 |     this.aiQuery = '';
122 |     this.aiActive = false;
123 |     this.aiResults = [];
124 |     this.aiMessage = '';
125 | }
126 | }
```

The **Clear** button: empties the AI text box, switches `aiActive` off, drops the stored results and message. The page then shows the full list again (still narrowed by the plain search box, if any). The last `}` closes the class.

Workflow, next: Once the driver list and the set of requested ids have arrived, Angular renders `customerviewdriver.component.html`, which loops over `displayedDrivers`.

Step 94: CustomerviewdriverComponent (HTML) - search boxes and driver cards [Arjun - Driver & Booking + Saumya (AI search)]

FRONTEND

angularapp/src/app/components/customerviewdriver/customerviewdriver.component.html

What this file is	Template of the Customer's Available Drivers page: a plain search box, a purple AI Search row, small status notes, "Oops! No records found" when empty, and a grid of driver cards that end with either a Request button or a disabled blue Requested button.
Who / what triggers it	Rendered by Angular after <code>CustomerviewdriverComponent</code> is created; re-rendered on every change of the class fields.
What it uses / calls	Fields/methods of <code>CustomerviewdriverComponent</code> (<code>searchText</code> , <code>aiQuery</code> , <code>aiLoading</code> , <code>aiActive</code> , <code>aiMessage</code> , <code>displayedDrivers</code> , <code>isRequested</code> , <code>canRequest</code> , <code>requestDriver</code> , <code>runAiSearch</code> , <code>clearAiSearch</code> , <code>getDriverImage</code> , <code>useDefaultAvatar</code>), global classes from <code>styles.css</code> and local ones from <code>customerviewdriver.component.css</code> .
Size	45 lines

Lines 1-4 | Container and heading

```
1 | <div class="page-container narrow">
2 |   <div class="page-card" id="availableDriversCard">
3 |     <h2 class="page-title steel" id="availableDriversHeading">Available Drivers</h2>
4 |     <div class="title-line"></div>
```

`page-container narrow` is the global centred container narrowed to 920px by the local CSS; `page-card` is the white card; the heading "Available Drivers" has the `steel` colour class; `title-line` draws the blue rule.

Lines 6-8 | Plain search box

```
6 | <div class="search-row">
7 |   <input type="text" id="searchInput" class="search-input" placeholder="Search by Driver Name, Vehicle Type, or Li
8 |   cense..." [(ngModel)]="searchText" />
9 | </div>
```

`[(ngModel)]="searchText"` is two-way binding to the field. Because `displayedDrivers` is a getter that reads `searchText`, the cards narrow instantly while the customer types, with no button or method call needed. `placeholder` explains what is searched (name, vehicle type or license).

Lines 10-15 | AI search form

```
10 | <form class="ai-row" id="aiSearchForm" (ngSubmit)="runAiSearch()">
11 |   <span class="ai-label">&#10024; AI Search</span>
12 |   <input type="text" id="aiSearchInput" name="aiQuery" placeholder='Try: "experienced SUV driver for a night outst
13 |   ation trip"' [(ngModel)]="aiQuery" />
14 |   <button type="submit" class="btn ai-btn" id="aiSearchButton" [disabled]="aiLoading || !aiQuery.trim()">{{ aiLoad
15 |   ing ? 'Searching...' : 'Find Drivers' }}</button>
16 |   <button type="button" class="btn clear-btn" id="aiClearButton" (click)="clearAiSearch()">Clear</button>
17 | </form>
```

A real `<form>` with `(ngSubmit)="runAiSearch()"`, so both the button and the Enter key start the search. `✨` is the sparkles symbol. The input is bound with `[(ngModel)]="aiQuery"`; because it sits inside a `<form>`, `ngModel` requires a `name` attribute (`name="aiQuery"`). **Find Drivers** is `type="submit"`, disabled while loading or when the text is blank (`[disabled]="aiLoading || !aiQuery.trim()"`), and its label changes with `{{ aiLoading ? 'Searching...' : 'Find Drivers' }}`. **Clear** is `type="button"` (so it does not submit) and calls `clearAiSearch()`.

Lines 16-17 | Notes under the AI row

```
16 | <div class="ai-note" id="aiNote" *ngIf="aiActive">Showing AI best matches for: "{{ aiQuery }}"</div>
17 | <div class="ai-note warn" *ngIf="aiMessage">{{ aiMessage }}</div>
```

`*ngIf="aiActive"` shows "Showing AI best matches for: ..." with the query interpolated. The second note, with the extra class `warn` (orange), appears only when `aiMessage` has text, i.e. after a failed AI call ("AI search is currently unavailable. Showing all drivers.").

Line 19 | Empty message

```
19 | <div class="empty-text" id="noDataMessage" *ngIf="displayedDrivers.length === 0">Oops! No records found</div>
```

`*ngIf="displayedDrivers.length === 0"` prints "Oops! No records found" when the (AI and/or text filtered) list is empty.

Lines 21-24 | Cards container, ngFor and photo

```
21 | <div class="cards" *ngIf="displayedDrivers.length > 0">
22 |   <div class="driver-card" *ngFor="let driver of displayedDrivers" [attr.id]='driver-' + driver.driverId">
23 |     <div class="card-main">
24 |       <img class="avatar" [src]="getDriverImage(driver.image)" (error)="useDefaultAvatar($event)" alt="Driver" />
```

The `cards` div exists only when there is something to show. `*ngFor="let driver of displayedDrivers"` makes one `driver-card` per driver, `[attr.id]='driver-' + driver.driverId` gives it an id, and the avatar `<img>` uses the same `getDriverImage` / `(error)="useDefaultAvatar($event)"` pair as the admin page.

Lines 25-35 | Driver information

```
25 | <div class="info">
26 |   <h3>{{ driver.driverName }}</h3>
27 |   <p><strong>Vehicle Type:</strong> {{ driver.vehicleType }}</p>
28 |   <p><strong>License:</strong> {{ driver.licenseNumber }}</p>
29 |   <p><strong>Phone:</strong> {{ driver.contactNumber }}</p>
30 |   <p><strong>Address:</strong> {{ driver.address }}</p>
31 |   <p><strong>Experience:</strong> {{ driver.experienceYears }} years</p>
32 |   <p><strong>Hourly Rate:</strong> &#8377;{{ driver.hourlyRate }}</p>
33 |   <p><strong>Status:</strong> {{ driver.availabilityStatus }}</p>
34 | </div>
35 | </div>
```

The `info` div prints the name and the labelled details (vehicle type, license, phone, address, experience in years, hourly rate with `₹` the rupee sign, and the status) with `{{ driver.xxx }}`. Line 35 closes the `card-main` row that holds the photo and the info.

Lines 36-41 | Requested / Request button with ng-template else

```
36 | <div class="card-footer">
37 |   <button type="button" class="btn btn-primary requested" [id]='requested-' + driver.driverId" *ngIf="isRequested(driver); else requestButton" disabled>Requested</button>
38 |   <ng-template #requestButton>
39 |     <button type="button" class="btn btn-success" [id]='request-' + driver.driverId" [disabled]="!canRequest(driver)" (click)="requestDriver(driver)">Request</button>
40 |   </ng-template>
41 | </div>
```

`*ngIf="isRequested(driver); else requestButton"` is the **if-else** form: when the customer already has an open request the blue **Requested** button (class `requested`, `disabled`, id `requested-<id>`) is drawn. Otherwise Angular renders the named template `#requestButton`. `<ng-template #requestButton>` is a block that stays invisible until some `else` refers to it by its reference name. Inside it, the green **Request** button has `[disabled]="!canRequest(driver)"` (disabled for Inactive and On Leave drivers) and `(click)="requestDriver(driver)"` which navigates to the booking form.

Lines 42-45 | Closing tags

```
42 | </div>
43 | </div>
44 | </div>
45 | </div>
```

Closes `card-footer`, `driver-card`, the `cards` container, `page-card` and `page-container`.

Workflow, next: The page is complete in the browser. Its layout rules are in `customerviewdriver.component.css`.

Step 95: CustomerviewdriverComponent (CSS) - search rows and three-column cards [Arjun - Driver & Booking + Saumya (AI search)]

angularapp/src/app/components/customerviewdriver/customerviewdriver.component.css

FRONTEND

What this file is	Component styles for the Available Drivers page: narrower page width, centred search box, purple AI row, a flexible three-per-row card grid, round small avatars and the look of the disabled "Requested" button.
Who / what triggers it	Loaded through <code>styleUrls</code> in <code>customerviewdriver.component.ts</code> ; applies only to this component's template.
What it uses / calls	Class names from <code>customerviewdriver.component.html</code> ; base styles (<code>.btn</code> , <code>.page-card</code> , <code>.search-input</code>) from the global <code>styles.css</code> .
Size	24 lines

Lines 1-3 | Page width and plain search box

```
1 | .page-container.narrow { max-width: 920px; }
2 | .search-row { display: flex; justify-content: center; margin-bottom: 14px; }
3 | .search-row .search-input { width: min(520px, 100%); }
```

`.page-container.narrow` limits the container to 920px. `.search-row` is a flex row that centres the box and leaves 14px below; `.search-row .search-input` sets its width to `min(520px, 100%)` (520px on wide screens, never wider than the screen).

Lines 5-11 | AI search row and notes

```
5 | .ai-row { display: flex; align-items: center; gap: 8px; margin: 0 0 6px; padding: 8px 10px; background: #efe9fb; border-radius: 6px; }
6 | .ai-label { font-size: 13px; font-weight: 700; color: #5a45c8; white-space: nowrap; }
7 | .ai-row input { flex: 1; min-width: 0; padding: 8px 10px; border: 1px solid #b8aee6; border-radius: 4px; font-size: 13px; background: #fff; }
8 | .ai-btn { background: #5a45c8; }
9 | .clear-btn { background: #fff; color: #444; border: 1px solid #ccc; }
10 | .ai-note { margin: 0 0 10px; text-align: center; font-size: 12px; font-style: italic; color: #555; }
11 | .ai-note.warn { color: #b4540a; }
```

- `.ai-row`: flex row with centred items, 8px gap and a pale purple `#efe9fb` rounded background, so the AI area is visually separate.
- `.ai-label`: bold purple "AI Search" label, `white-space: nowrap` keeps it on one line.
- `.ai-row input`: `flex: 1` lets the text box take all remaining width; `min-width: 0` allows it to shrink inside the flex row; light purple border, white background.
- `.ai-btn` gives the **Find Drivers** button the purple background; `.clear-btn` is white with grey text and a thin border (a "secondary" look).
- `.ai-note`: small italic centred grey text for the two notes; `.ai-note.warn` turns it orange for the "unavailable" message.

Lines 13-22 | Card grid and card parts

```
13 | .cards { display: flex; flex-wrap: wrap; justify-content: center; gap: 16px; margin-top: 10px; }
14 | .driver-card { display: flex; flex-direction: column; width: calc(33.33% - 11px); min-width: 250px; background: #fff; border-radius: 6px; box-shadow: 0 2px 8px rgba(0, 0, 0, 0.2); }
15 | .card-main { display: flex; gap: 12px; padding: 16px 14px 10px; flex: 1; }
16 | .avatar { width: 58px; height: 58px; flex: 0 0 58px; object-fit: cover; border-radius: 50%; border: 3px solid #1d8a8a; background: #fff; }
17 | .info h3 { margin: 0 0 8px; font-size: 18px; color: #333; }
18 | .info p { margin: 4px 0; font-size: 12.5px; color: #555; word-break: break-word; }
19 | .info strong { color: #222; }
20 | .card-footer { padding: 10px; text-align: center; background: #f3f5f7; border-radius: 0 0 6px 6px; }
21 | .card-footer .btn { min-width: 84px; }
22 | .btn.requested:disabled { background: #0a6fe8; color: #fff; opacity: 1; cursor: default; }
```

- `.cards`: flex, wrapping, centred, 16px gap (same technique as the admin page).
- `.driver-card`: a flex column with `width: calc(33.33% - 11px)` for **three cards per row**, `min-width: 250px` so it wraps earlier on narrow screens, white background, rounded corners, soft shadow.
- `.card-main`: a flex row (photo left, text right) with `flex: 1` so the footer stays at the bottom.
- `.avatar`: 58px circle (`flex: 0 0 58px` forbids shrinking), `object-fit: cover`, teal border.
- `.info h3` / `.info p` / `.info strong`: heading and detail line sizes and colours; `word-break: break-word` stops long license numbers or addresses from overflowing.
- `.card-footer`: grey strip with the centred button; `.card-footer .btn` sets a minimum button width of 84px so Request and Requested are the same size.
- `.btn.requested:disabled`: normally disabled buttons are faded (global rule). This keeps the Requested button solid blue with white text, `opacity: 1` and a normal cursor, so it reads as a status, not as a broken button.

Line 24 | Responsive rule

```
24 | @media (max-width: 760px) { .ai-row { flex-wrap: wrap; } .driver-card { width: 100%; } }
```

`@media (max-width: 760px)` applies only on screens up to 760px wide (phones): the AI row may wrap onto several lines (`flex-wrap: wrap`) and each card takes the full width.

Workflow, next: The Customer has found a suitable Active driver and presses **Request**. `requestDriver()` navigates to `/customer-request/<driverId>`, which opens `customer-request.component.ts`.

Step 96: CustomerRequestComponent (TypeScript) - booking a driver (create) and editing a request [Arjun - Driver & Booking (create) + Vaishalini (edit mode)]

FRONTEND

angularapp/src/app/components/customer-request/customer-request.component.ts

What this file is	The logic of the booking form. With the URL <code>/customer-request/<driverId></code> it is a "New Driver Request" form (create mode). With <code>/customer-request/edit/<driverRequestId></code> it is "Edit Request" (edit mode): the existing request is loaded, the form is filled in and the save becomes an update. The form collects pickup and drop locations, trip date, time slot, estimated duration and optional comments, and refuses a trip date in the past. Honest note about the "Booking Engine": any rules about concurrency or double-booking of a driver (two customers requesting the same driver for the same time, locking a driver while a request is pending, and so on) are not implemented in the code. <code>DriverRequestServiceImpl.addDriverRequest</code> only fills in <code>requestDate</code> when it is empty and saves; it has no overlap check, no availability check, no transaction or locking logic, and <code>DriverRequestRepo</code> has only the two finder methods <code>findByUserUserId</code> and <code>findByDriverDriverId</code>. The only guards are in the Angular screens: the Request button is enabled only for Active drivers the customer has not already requested.
Who / what triggers it	The Customer clicks Request on a driver card (create mode, route <code>customer-request/:driverId</code>) or Edit on a Pending request in My Requests (edit mode, route <code>customer-request/edit/:driverRequestId</code>). <code>AuthGuard</code> requires role Customer.
What it uses / calls	<code>FormBuilder</code> and <code>Validators</code>, a custom validator, <code>ActivatedRoute</code>, <code>Router</code>, <code>DriverRequestService</code> (GET one, POST, PUT), <code>AuthService.getUserId</code>, the <code>DriverRequest</code> model and date helpers <code>todayString</code>, <code>toDateString</code>, <code>toTimeString</code> from <code>utils/format.ts</code>.
Size	138 lines

Lines 1-7 | Imports

```
1 | import { Component, OnInit } from '@angular/core';
2 | import { AbstractControl, FormBuilder, FormGroup, ValidationErrors, Validators } from '@angular/forms';
3 | import { ActivatedRoute, Router } from '@angular/router';
4 | import { DriverRequestService } from '../../../services/driver-request.service';
5 | import { AuthService } from '../../../services/auth.service';
6 | import { DriverRequest } from '../../../models/driver-request.model';
7 | import { todayString, toDateString, toTimeString } from '../../../utils/format';
```

`AbstractControl` (a single form control, the argument of a validator function) and `ValidationErrors` (the type of an error object like `{ pastDate: true }`) join the usual `FormBuilder`, `FormGroup`, `Validators`. `ActivatedRoute/Router` read the URL and navigate. `DriverRequestService` makes the HTTP calls, `AuthService` provides the logged-in user's id, `DriverRequest` is the data type, and `todayString` (today as YYYY-MM-DD), `toDateString` and `toTimeString` (convert whatever the server sent into the text format that `<input type="date">` and `<input type="time">` accept) come from `utils/format.ts`.

Lines 9-13 | @Component decorator

```
9 | @Component({
10 |   selector: 'app-customer-request',
11 |   templateUrl: './customer-request.component.html',
12 |   styleUrls: ['./customer-request.component.css']
13 | })
```

Selector `app-customer-request`, template `customer-request.component.html`, style sheet `customer-request.component.css`.

Lines 14-21 | Class fields: form, minimum date and mode

```
14 | export class CustomerRequestComponent implements OnInit {
15 |   requestForm: FormGroup;
16 |   minDate: string = todayString();
17 |
18 |   isEditMode: boolean = false;
19 |   driverId?: number;
20 |   driverRequestId?: number;
21 |   private originalTripDate: string = '';
```

`requestForm` is the reactive form; `minDate` is today's date as text, used by the date input's `[min]`. `isEditMode` selects create or edit behaviour. `driverId` (the driver being booked, or in edit mode the driver of the loaded

request) and `driverRequestId` (edit mode) are filled in `ngOnInit`. The `private originalTripDate` remembers the trip date of the request being edited, so the date validator can accept an unchanged date that is now in the past.

Lines 23-26 | UI state

```
23 | submitted: boolean = false;
24 | formError: string = '';
25 | successMessage: string = '';
26 | showSuccessPopup: boolean = false;
```

`submitted` makes untouched fields show errors after a Submit click; `formError` is the general message; `successMessage` and `showSuccessPopup` control the confirmation dialog.

Lines 28-34 | Constructor (dependency injection)

```
28 | constructor(
29 |   private fb: FormBuilder,
30 |   private driverRequestService: DriverRequestService,
31 |   private authService: AuthService,
32 |   private route: ActivatedRoute,
33 |   private router: Router
34 | ) {
```

Angular injects `FormBuilder`, `DriverRequestService`, `AuthService`, `ActivatedRoute` and `Router` as private fields.

Lines 35-43 | The form definition and its validators

```
35 | this.requestForm = this.fb.group({
36 |   pickupLocation: ['', [Validators.required, Validators.pattern(/\/S/)]],
37 |   dropLocation: ['', [Validators.required, Validators.pattern(/\/S/)]],
38 |   tripDate: ['', [Validators.required, (control: AbstractControl) => this.notInThePast(control)]],
39 |   timeSlot: ['', [Validators.required]],
40 |   estimatedDuration: ['', [Validators.required, Validators.pattern(/\/S/)]],
41 |   comments: ['']
42 | });
43 | }
```

- `pickupLocation` and `dropLocation`: `required` plus `/\/S/` (at least one non-space character, because `required` alone accepts only-spaces text).
- `tripDate`: `required` and a **custom validator**. Because the validator needs `this` (to read `originalTripDate`), it is written as an arrow function `(control: AbstractControl) => this.notInThePast(control)`; Angular calls it with the control and expects `null` (valid) or an error object.
- `timeSlot`: just `required` (the browser's time input gives `HH:mm`).
- `estimatedDuration`: `required` plus `/\/S/`; it is free text such as "3 hours" (the fare calculation at Trip End later parses it).
- `comments`: no validators, optional.

Lines 45-69 | ngOnInit - create mode or edit mode

```
45 | ngOnInit(): void {
46 |   const requestId = this.route.snapshot.paramMap.get('driverRequestId');
47 |   const driverId = this.route.snapshot.paramMap.get('driverId');
48 |   if (requestId) {
49 |     this.isEditMode = true;
50 |     this.driverRequestId = Number(requestId);
51 |     this.driverRequestService.getDriverRequestById(this.driverRequestId).subscribe({
52 |       next: (request) => {
53 |         this.driverId = request.driver?.driverId;
54 |         this.originalTripDate = toDateString(request.tripDate);
55 |         this.requestForm.patchValue({
56 |           pickupLocation: request.pickupLocation,
57 |           dropLocation: request.dropLocation,
58 |           tripDate: this.originalTripDate,
59 |           timeSlot: toTimeString(request.timeSlot),
60 |           estimatedDuration: request.estimatedDuration,
61 |           comments: request.comments || ''
62 |         });
63 |       },
64 |       error: () => this.router.navigate(['/customerviewrequested'])
65 |     });
66 |   } else if (driverId) {
67 |     this.driverId = Number(driverId);
68 |   }
69 | }
```

The page was opened by one of two routes, so both parameters are read from `route.snapshot.paramMap`: `driverRequestId` (edit route) and `driverId` (create route). Only one of them exists.

- **Edit mode**: `isEditMode = true`, the id is converted to a number and `getDriverRequestById(id)` sends **GET** `/api/driverRequest/{id}` (any logged-in user is allowed). Backend: `DriverRequestController.viewDriverRequestById -> DriverRequestServiceImpl.getDriverRequestById -> findById` (200 with the request, 404 if missing). On `next`, `driverId` is taken from the nested `request.driver`,

`originalTripDate` is remembered (via `toDateString`, because Jackson may send dates as arrays `[2025, 3, 7]` or as strings), and `patchValue` fills the form; `toTimeString` turns the time into `HH:mm`, and `comments` falls back to an empty string. On `error` (e.g. 404) the customer is sent to `/customerviewrequested`.

- **Create mode:** only `this.driverId = Number(driverId)` is stored; the form stays empty.

The server does not check that a request being edited is still Pending; the Edit button is only offered for Pending requests on the My Requests page, so that rule lives in the screen.

Lines 71-78 | `notInThePast` - custom validator

```
71 | /** A trip cannot be booked in the past (an unchanged date of an existing request is allowed). */
72 | private notInThePast(control: AbstractControl): ValidationErrors | null {
73 |     const value: string = control.value;
74 |     if (!value || value === this.originalTripDate) {
75 |         return null;
76 |     }
77 |     return value < todayString() ? { pastDate: true } : null;
78 | }
```

Returns `null` (valid) when the field is empty (the `required` rule reports that case) or when the value equals `originalTripDate` (so an old request can be edited without having to change its date). Otherwise it compares the chosen date with `todayString()` using `<`. Both are `YYYY-MM-DD` texts, and for that format alphabetical order equals date order, so the text comparison works. A past date returns `{ pastDate: true }`, which the template turns into `"*Trip Date cannot be in the past"`.

Lines 80-87 | `f` getter and `showError`

```
80 | get f() {
81 |     return this.requestForm.controls;
82 | }
83 |
84 | showError(field: string): boolean {
85 |     const control = this.requestForm.get(field)!;
86 |     return control.invalid && (control.touched || this.submitted);
87 | }
```

Same helpers as in the driver form: `f` exposes the controls to the template, and `showError(field)` is true when the control is invalid and was touched or the form was submitted.

Lines 89-95 | `onSubmit` part 1 - validation gate

```
89 | onSubmit(): void {
90 |     this.submitted = true;
91 |     this.formError = '';
92 |     if (this.requestForm.invalid) {
93 |         this.formError = 'All required fields must be filled out';
94 |         return;
95 |     }
```

Sets `submitted = true`, clears the message and, if the form is invalid, sets "All required fields must be filled out" and returns without calling the server.

Lines 96-104 | `onSubmit` part 2 - the common trip details

```
96 | const value = this.requestForm.value;
97 | const details = {
98 |     tripDate: value.tripDate,
99 |     timeSlot: value.timeSlot,
100 |     pickupLocation: String(value.pickupLocation).trim(),
101 |     dropLocation: String(value.dropLocation).trim(),
102 |     estimatedDuration: String(value.estimatedDuration).trim(),
103 |     comments: value.comments || ''
104 | };
```

`details` collects the values the customer typed: `tripDate` (`YYYY-MM-DD`) and `timeSlot` (`HH:mm`) are sent as they are (the backend entity uses `LocalDate` and `LocalTime`, which Jackson can read from those texts), and the locations, duration and comments are trimmed. `comments: value.comments || ''` makes an empty comment an empty text instead of null.

Lines 106-109 | Edit mode: `PUT /api/driverRequest/{id}`

```
106 | if (this.isEditMode && this.driverRequestId !== undefined) {
107 |     this.driverRequestService.updateDriverRequest(this.driverRequestId, details as unknown as DriverRequest).subscribe(
108 |         be({
109 |             next: () => this.showSuccess('Request Updated Successfully!')
```

`updateDriverRequest(id, details)` sends `PUT /api/driverRequest/{id}` with **only** the trip details (no user, driver or status). The `as unknown as DriverRequest` cast tells TypeScript to accept this partial object. Backend: `DriverRequestController.updateDriverRequest -> DriverRequestServiceImpl.updateDriverRequest`, which loads the stored request and copies a field **only when the received value is not null** (status, trip date, time slot, pickup,

drop, estimated duration, payment amount, comments, actual drop time/date/duration and driver). Because the form sends no status, the status stays as it was. (The same merge logic is what lets the admin and the Trip End screens send partial updates.) `next` opens the popup "Request Updated Successfully!". There is no `error` handler here, so a failure shows nothing on this page.

Lines 110-123 | Create mode: POST /api/driverRequest

```
110 |     } else {
111 |         const userId = this.authService.getUserId();
112 |         const request = {
113 |             ...details,
114 |             user: { userId },
115 |             driver: { driverId: this.driverId },
116 |             requestDate: todayString(),
117 |             status: 'Pending'
118 |         };
119 |         this.driverRequestService.addDriverRequest(request as unknown as DriverRequest).subscribe({
120 |             next: () => this.showSuccess('Request Submitted Successfully!')
121 |         });
122 |     }
123 | }
```

The customer's id comes from `authService.getUserId()` (the `userId` saved in `localStorage` at login). The body is the trip details plus the relations written as **nested objects**: `user: { userId }` and `driver: { driverId: this.driverId }`, plus `requestDate: todayString()` and `status: 'Pending'`. Nested objects are what the backend entity expects: `DriverRequest` has `@ManyToOne User user` and `@ManyToOne Driver driver`, and only the `userId` / `driverId` inside them are used to fill the foreign-key columns.

`addDriverRequest(request)` sends **POST /api/driverRequest** (role CUSTOMER). Backend: `DriverRequestController.addDriverRequest -> DriverRequestServiceImpl.addDriverRequest`, which sets `requestDate` to today only if it is null (here Angular already sent it) and calls `driverRequestRepo.save`; the controller answers **201 Created**. The initial status `Pending` is supplied by this Angular code, the server does not set or validate it. The controller wraps the call in `try/catch (Exception)` and answers 409 for any failure, and this component has no `error` handler, so the customer would see nothing in that case. `next` opens the popup "Request Submitted Successfully!".

Lines 125-128 | showSuccess

```
125 | private showSuccess(message: string): void {
126 |     this.successMessage = message;
127 |     this.showSuccessPopup = true;
128 | }
```

Stores the message and shows the popup by setting `showSuccessPopup = true`.

Lines 130-138 | closeSuccessPopup and goBack

```
130 | closeSuccessPopup(): void {
131 |     this.showSuccessPopup = false;
132 |     this.router.navigate([this.isEditMode ? '/customerviewrequested' : '/customerviewdriver']);
133 | }
134 |
135 | goBack(): void {
136 |     this.router.navigate([this.isEditMode ? '/customerviewrequested' : '/customerviewdriver']);
137 | }
138 | }
```

Both end the page the same way: after an edit (or Back in edit mode) the customer goes to `/customerviewrequested` (My Requests), after creating a request (or Back in create mode) to `/customerviewdriver` (Available Drivers), where the driver now shows the blue **Requested** button because `loadMyRequests()` runs again. The final `}` closes the class.

Workflow, next: With the form object and the mode known, Angular draws the booking form from `customer-request.component.html`.

Step 97: CustomerRequestComponent (HTML) - the booking form template [Arjun - Driver & Booking (create) + Vaishalini (edit mode)]

FRONTEND

angularapp/src/app/components/customer-request/customer-request.component.html

What this file is	The template of the booking / edit-request form: pickup, drop, date, time, duration and comments inputs with their error texts, a Back button, a submit button whose label depends on the mode and a success popup. Its structure is identical to the driver form template.
Who / what triggers it	Rendered after <code>CustomerRequestComponent</code> is created for either booking route.
What it uses / calls	Fields and methods of <code>CustomerRequestComponent</code> (<code>requestForm</code> , <code>isEditMode</code> , <code>minDate</code> , <code>f</code> , <code>showError()</code> , <code>onSubmit()</code> , <code>goBack()</code> , <code>formError</code> , <code>successMessage</code> , <code>showSuccessPopup</code> , <code>closeSuccessPopup()</code>); global form and modal classes from <code>styles.css</code> .
Size	45 lines

Lines 1-3 | Form, Back button and heading

```
1 | <form class="form-card" id="requestForm" [formGroup]="requestForm" (ngSubmit)="onSubmit()" novalidate>
2 |   <button type="button" class="back-btn" id="backButton" (click)="goBack()">Back</button>
3 |   <h2 class="form-title" id="formHeading">{{ isEditMode ? 'Edit Request' : 'New Driver Request' }}</h2>
```

[`formGroup`]="`requestForm`" connects the form to the class, (`ngSubmit`)="`onSubmit()`" handles the submit and `novalidate` disables the browser's own validation bubbles. The **Back** button (`type="button"`, always visible here) calls `goBack()`. The heading is `{{ isEditMode ? 'Edit Request' : 'New Driver Request' }}`.

Lines 5-11 | Pickup and drop location

```
5 | <label class="form-label" for="pickupLocation">Pickup Location<span class="req">*</span></label>
6 | <input type="text" id="pickupLocation" class="form-control" formControlName="pickupLocation" />
7 | <div class="error-text" id="pickupLocationError" *ngIf="showError('pickupLocation')">*Pickup Location is required</div>
8 |
9 | <label class="form-label" for="dropLocation">Drop Location<span class="req">*</span></label>
10 | <input type="text" id="dropLocation" class="form-control" formControlName="dropLocation" />
11 | <div class="error-text" id="dropLocationError" *ngIf="showError('dropLocation')">*Drop Location is required</div>
```

Two text inputs bound with `formControlName="pickupLocation"` and `"dropLocation"`. Each error div uses an `*ngIf` that calls `showError` with the field name, and shows one message ("`*Pickup Location is required`", "`*Drop Location is required`").

Lines 13-18 | Trip date

```
13 | <label class="form-label" for="tripDate">Trip Date<span class="req">*</span></label>
14 | <input type="date" id="tripDate" class="form-control" formControlName="tripDate" [min]="isEditMode ? '' : minDate" />
15 | <div class="error-text" id="tripDateError" *ngIf="showError('tripDate')">
16 |   <span *ngIf="f['tripDate'].errors?.['required']">*Trip Date is required</span>
17 |   <span *ngIf="f['tripDate'].errors?.['pastDate']">*Trip Date cannot be in the past</span>
18 | </div>
```

`type="date"` gives the browser's date picker. [`min`]="`isEditMode ? '' : minDate`" is a **property binding** that sets the earliest selectable date to today in create mode (past days are greyed out in the picker) and no limit in edit mode (the old date of the request may be in the past). The error block can show either "`*Trip Date is required`" (`errors?.['required']`) or "`*Trip Date cannot be in the past`" (`errors?.['pastDate']`), the key returned by the custom validator).

Lines 20-22 | Time slot

```
20 | <label class="form-label" for="timeSlot">Time Slot<span class="req">*</span></label>
21 | <input type="time" id="timeSlot" class="form-control" formControlName="timeSlot" />
22 | <div class="error-text" id="timeSlotError" *ngIf="showError('timeSlot')">*Time Slot is required</div>
```

`type="time"` gives a time picker returning `HH:mm`; the only rule is required.

Lines 24-26 | Estimated duration

```
24 | <label class="form-label" for="estimatedDuration">Estimated Duration<span class="req">*</span></label>
25 | <input type="text" id="estimatedDuration" class="form-control" formControlName="estimatedDuration" placeholder="e.g. 3 hours" />
26 | <div class="error-text" id="estimatedDurationError" *ngIf="showError('estimatedDuration')">*Estimated Duration is required</div>
```

Free text with the hint `placeholder="e.g. 3 hours"`. The duration text is later parsed to minutes for the fare, so a number followed by "hours" or "minutes" works best.

Lines 28-29 | Comments

```
28 | <label class="form-label" for="comments">Comments</label>
29 | <textarea id="comments" class="form-control" rows="3" formControlName="comments"></textarea>
```

A `<textarea rows="3">` bound to `comments`. It has no validators and no error message because it is optional.

Lines 31-36 | General error, submit button and end of form

```
31 | <div class="form-error" id="formError" *ngIf="formError">{{ formError }}</div>
32 |
33 | <div class="submit-row">
34 |   <button type="submit" class="btn btn-primary" id="submitButton">{{ isEditMode ? 'Update Request' : 'Submit Request' }}</button>
35 | </div>
36 | </form>
```

`formError` ("All required fields must be filled out") appears only when it has text. The submit button label is `{{ isEditMode ? 'Update Request' : 'Submit Request' }}`. `</form>` closes the form.

Lines 38-45 | Success popup

```
38 | <div class="modal-backdrop" *ngIf="showSuccessPopup">
39 |   <div class="modal-box success-box" id="successPopup">
40 |     <p>{{ successMessage }}</p>
41 |     <div class="modal-actions">
42 |       <button type="button" class="btn btn-primary" id="successOk" (click)="closeSuccessPopup()">Ok</button>
43 |     </div>
44 |   </div>
45 | </div>
```

Same pattern as the driver form: shown only when `showSuccessPopup` is true, prints `successMessage` ("Request Submitted Successfully!" or "Request Updated Successfully!") and its **Ok** button calls `closeSuccessPopup()`, which navigates away.

Workflow, next: The form is complete. The tiny style sheet that belongs to it is `customer-request.component.css`.

Step 98: CustomerRequestComponent (CSS) - date and time input height [Arjun - Driver & Booking (create) + Vaishalini (edit mode)]

`angularapp/src/app/components/customer-request/customer-request.component.css`

FRONTEND

What this file is	Almost empty on purpose: the card, labels, inputs, error texts and popup of this form all come from the shared rules in <code>src/styles.css</code> (<code>form-card</code> , <code>form-label</code> , <code>form-control</code> , <code>error-text</code> , <code>modal-box</code> , ...). The file only contains a comment and a single fix for the date and time inputs.
Who / what triggers it	Loaded through <code>styleUrls</code> in <code>customer-request.component.ts</code> , applied only to this component's template.
What it uses / calls	The two <code>form-control</code> inputs of type <code>date</code> and <code>time</code> in <code>customer-request.component.html</code> .
Size	2 lines

Lines 1-2 | Comment and date/time height rule

```
1 | /* The form card, labels, inputs and modal are styled by the shared styles in styles.css. */
2 | input[type='date'].form-control, input[type='time'].form-control { min-height: 40px; }
```

Line 1 is only a comment saying where the real styling lives. Line 2, `input[type='date'].form-control, input[type='time'].form-control { min-height: 40px; }`, selects exactly the date and time inputs that also carry the `form-control` class and gives them a minimum height of 40px. Browsers draw date and time pickers a little shorter than text inputs, so without this the Trip Date and Time Slot boxes would look smaller than the neighbouring text fields.

Workflow, next: That ends the customer's booking walk. The request has been saved with the status **Pending**, so the Customer sees it under My Requests, and the Admin sees the same record in the request-approval screen, where it can be approved or rejected. That screen is [angularapp/src/app/components/adminviewrequests/adminviewrequests.component.ts](#), the first file of the next part.

Step 99: AdminviewrequestsComponent - admin list, approve / reject, stages [Aarti - Admin Requests]

FRONTEND

[angularapp/src/app/components/adminviewrequests/adminviewrequests.component.ts](#)

What this file is	The admin "Driver Requests for Approval" page . It downloads every driver request made by every customer, shows them in a table, lets the admin search and filter them, approve or reject them, look at the driver's details in a pop-up (modal) and see a request's progression (Pending -> Approved -> Trip End -> Closed) in a second pop-up. It is also the only place where an admin can finish a request by pressing Close Request . A component in Angular is a class (this file, the logic) joined with an HTML template and a CSS file. The class holds the data and the methods; the template shows them. Everything the template can read or click must be a property or method of this class.
Who / what triggers it	The Angular router. In <code>app-routing.module.ts</code> the path <code>adminviewrequests</code> is mapped to this component and protected by <code>AuthGuard</code> with <code>data: { role: 'Admin' }</code> , so only a logged-in admin can open it. The admin gets here from the admin navbar link. When the route is activated Angular creates the class and calls <code>ngOnInit()</code> .
What it uses / calls	<code>DriverRequestService</code> (injected) for the HTTP calls <code>getAllDriverRequests()</code> and <code>updateDriverRequest()</code>. - The <code>DriverRequest</code> model (TypeScript interface describing the JSON shape). <code>REQUEST_STAGES</code> and <code>REQUEST_STATUSES</code> from <code>utils/constants.ts</code>. <code>toDateString</code> from <code>utils/format.ts</code> and <code>getDriverImage / useDefaultAvatar</code> from <code>utils/driver-image.ts</code>, handed to the template. - Backend: <code>GET /api/driverRequest</code> answered by <code>DriverRequestController.viewAllDriverRequests</code> and <code>PUT /api/driverRequest/{id}</code> answered by <code>DriverRequestController.updateDriverRequest</code>.
Size	135 lines

Lines 1-6 | Imports

```
1 | import { Component, OnInit } from '@angular/core';
2 | import { DriverRequestService } from '../services/driver-request.service';
3 | import { DriverRequest } from '../models/driver-request.model';
4 | import { REQUEST_STAGES, REQUEST_STATUSES } from '../utils/constants';
5 | import { toDateString } from '../utils/format';
6 | import { getDriverImage, useDefaultAvatar } from '../utils/driver-image';
```

Imports make other code available in this file.

- `Component`, `OnInit` come from Angular core. `Component` is the decorator that turns the class into a component; `OnInit` is an **interface** that promises the class has an `ngOnInit()` method, which Angular calls once after the component is created.
- `DriverRequestService` is the service that talks to the backend (it wraps Angular's `HttpClient` and adds the JWT header).
- `DriverRequest` is the TypeScript interface for one request object (id, status, pickupLocation, nested `user` and `driver`, ...). It exists only at compile time and gives auto-complete and type checking.
- `REQUEST_STAGES` (Pending, Approved, Trip End, Closed) and `REQUEST_STATUSES` (those plus Rejected) are constant arrays shared by several pages.
- `toDateString` converts whatever date shape the backend sent into YYYY-MM-DD.
- `getDriverImage` turns a stored image string into something an `<img src>` can show and `useDefaultAvatar` is the fallback used when the image fails to load.

Lines 8-12 | @Component decorator

```
8 | @Component({
9 |   selector: 'app-adminviewrequests',
10 |   templateUrl: './adminviewrequests.component.html',
11 |   styleUrls: ['./adminviewrequests.component.css']
12 | })
```

A **decorator** (written with `@`) attaches metadata to the class right under it. `selector: 'app-adminviewrequests'` is the HTML tag name this component could be written as; here it is never written by hand because the router inserts it into `<router-outlet>`. `templateUrl` points at the HTML file and `styleUrls` at the CSS file. Angular's default view encapsulation adds a hidden attribute to every element of this template and rewrites the CSS so these styles only affect this component.

Lines 13-19 | Class declaration and list / filter state

```
13 | export class AdminviewrequestsComponent implements OnInit {
14 |   requests: DriverRequest[] = [];
15 |   filteredRequests: DriverRequest[] = [];
16 |   searchText: string = '';
17 |   statusFilter: string = 'All';
18 |   statuses: string[] = REQUEST_STATUSES;
19 |   stages: string[] = REQUEST_STAGES;
```

`export class AdminviewrequestsComponent implements OnInit` declares the class and exports it so `AppModule` and the router can import it.

The fields are the **state** of the page:

- `requests` holds the full list received from the server and never shrinks because of filtering.
- `filteredRequests` is what the table actually loops over: the part of `requests` that matches the search box and the status drop-down.
- `searchText` is bound to the search input with `[(ngModel)]`.
- `statusFilter` is bound to the drop-down; 'All' means no status filtering.
- `statuses` (all five statuses) fills the drop-down options and `stages` (four stages) drives the progression modal.

Lines 21-23 | Modal state

```
21 |   selectedRequest: DriverRequest | null = null;
22 |   showDriverModal: boolean = false;
23 |   showStageModal: boolean = false;
```

`selectedRequest` remembers which table row the admin clicked (it is `null` when nothing is chosen).

`showDriverModal` and `showStageModal` are plain booleans; the template uses them in `*ngIf` to decide whether each pop-up exists in the page. This is the usual Angular way to do a modal: no library, just a flag.

Lines 25-27 | Helpers exposed to the template

```
25 |   toDateString = toDateString;
26 |   getDriverImage = getDriverImage;
27 |   useDefaultAvatar = useDefaultAvatar;
```

An Angular template can only call things that are members of its component class. `toDateString = toDateString;` copies the imported function into a property of the same name so the HTML can write `toDateString(request.tripDate)`. The same is done for `getDriverImage` and `useDefaultAvatar`.

Lines 29-33 | Constructor and ngOnInit

```
29 |   constructor(private driverRequestService: DriverRequestService) {}
30 |
31 |   ngOnInit(): void {
32 |     this.loadRequests();
33 |   }
```

The constructor uses **dependency injection**: writing `private driverRequestService: DriverRequestService` as a constructor parameter makes Angular create (or reuse) the service and hand it to the class, and the `private` keyword also stores it as `this.driverRequestService`. The constructor body is empty on purpose; real start-up work belongs in `ngOnInit`.

`ngOnInit()` runs once when the page opens and simply calls `loadRequests()`, so the table is filled as soon as the page appears.

Lines 35-47 | loadRequests - GET /api/driverRequest

```
35 |   loadRequests(): void {
36 |     this.driverRequestService.getAllDriverRequests().subscribe({
37 |       // The API answers 204 (empty body) when there are no requests.
38 |       next: (requests) => {
39 |         this.requests = requests || [];
40 |         this.applyFilters();
41 |       },
42 |       error: () => {
43 |         this.requests = [];
44 |         this.applyFilters();
45 |       }
46 |     });
47 |   }
```

`getAllDriverRequests()` returns an **Observable** (a stream that will deliver the HTTP response later). Nothing is sent until `.subscribe(...)` is called. The service sends `GET http://localhost:8080/api/driverRequest` with `Authorization: Bearer <token>`.

On the server Spring Security's rule `GET /api/driverRequest -> hasRole("ADMIN")` lets only admins through, then `DriverRequestController.viewAllDriverRequests` calls `DriverRequestServiceImpl.getAllDriverRequests()` (which is `driverRequestRepo.findAll()`). The controller answers **204 No Content** with an empty body when the list is empty, otherwise **200** with a JSON array. Each element contains the nested `user` and `driver` objects because the entity has `@ManyToOne` links to them.

- `next` runs for a successful answer. The comment on line 37 explains why `requests || []` is needed: on a 204 Angular gives `null` as the body, and `null` would break `.filter` later.
- `error` runs for a failure (for example a 401 or the server being down; the global `HttpErrorInterceptor` may also redirect). The list is simply emptied.
- Both paths call `applyFilters()` so `filteredRequests` is rebuilt.

Lines 49-59 | `applyFilters` - search box and status drop-down

```
49 |   applyFilters(): void {
50 |     const query = this.searchText.trim().toLowerCase();
51 |     this.filteredRequests = this.requests.filter((request) => {
52 |       const matchesStatus = this.statusFilter === 'All' || request.status === this.statusFilter;
53 |       const matchesText =
54 |         !query ||
55 |         (request.driver?.driverName || '').toLowerCase().includes(query) ||
56 |         (request.pickupLocation || '').toLowerCase().includes(query);
57 |       return matchesStatus && matchesText;
58 |     });
59 |   }
```

Called after loading and every time the admin types or changes the drop-down (the template wires `(ngModelChange)` to it).

- Line 50 trims the search text and lower-cases it so searching is case-insensitive.
- `this.requests.filter(...)` creates a new array keeping only the requests for which the callback returns `true`. The original array is untouched, so clearing the filter restores everything.
- `matchesStatus` is true when the drop-down is `'All'` or the request status equals the chosen one.
- `matchesText` is true when the search box is empty, or the driver's name contains the text, or the pickup location contains the text. `request.driver?.driverName` uses **optional chaining**: if `driver` is null the whole expression becomes `undefined` instead of crashing, and `|| ''` turns that into an empty string.
- A row is kept only if both conditions are true (`&&`).

Lines 61-67 | `canApprove` / `canReject` - which buttons may be shown

```
61 |   canApprove(request: DriverRequest): boolean {
62 |     return request.status === 'Pending' || request.status === 'Rejected';
63 |   }
64 |
65 |   canReject(request: DriverRequest): boolean {
66 |     return request.status === 'Pending' || request.status === 'Approved';
67 |   }
```

These two small methods encode the admin rules of the status chain and the template uses them in `*ngIf` on the buttons.

- **Approve** is available when the status is `Pending` or `Rejected` (an admin may change their mind about a rejected request).
- **Reject** is available when the status is `Pending` or `Approved` (an approved request can still be rejected before the trip).
- Once a request is `Trip End` or `Closed` neither button appears, so the admin cannot reverse a finished trip.

Note this is only a **front-end** rule. The backend does not check transitions; it would accept any status text sent in a PUT.

Lines 69-75 | `approve` and `reject`

```
69 |   approve(request: DriverRequest): void {
70 |     this.changeStatus(request, 'Approved');
71 |   }
72 |
73 |   reject(request: DriverRequest): void {
74 |     this.changeStatus(request, 'Rejected');
75 |   }
```

Two one-line wrappers. The buttons call `approve(request)` or `reject(request)` and each simply forwards to `changeStatus` with the new status text `'Approved'` or `'Rejected'`.

Lines 77-91 | changeStatus - PUT /api/driverRequest/{id} with only the status

```
77 | private changeStatus(request: DriverRequest, status: string, onDone?: () => void): void {
78 |     if (request.driverRequestId === undefined) {
79 |         return;
80 |     }
81 |     const update = { status } as unknown as DriverRequest;
82 |     this.driverRequestService.updateDriverRequest(request.driverRequestId, update).subscribe({
83 |         next: () => {
84 |             request.status = status;
85 |             this.applyFilters();
86 |             if (onDone) {
87 |                 onDone();
88 |             }
89 |         }
90 |     });
91 | }
```

This is the core of the admin workflow; approve, reject and close all go through it.

- Lines 78-80: a guard. If the request somehow has no id there is nothing to update, so the method stops (`return`).
- Line 81: builds the body `{ status }` (shorthand for `{ status: status }`). `as unknown as DriverRequest` is a TypeScript trick that tells the compiler to treat this tiny object as a `DriverRequest` even though required fields such as `pickupLocation` are missing; at run time it is just `{"status": "Approved"}`.
- Line 82: `updateDriverRequest(id, update)` sends PUT `http://localhost:8080/api/driverRequest/{id}`. Security rule: `PUT /api/driverRequest/** -> authenticated()` (any logged-in user, which is why the customer page can use the same endpoint).
- On the server `DriverRequestController.updateDriverRequest` calls `DriverRequestServiceImpl.updateDriverRequest`. That method loads the stored row with `findById`, then copies a field from the incoming object **only if it is not null** (`if (driverRequest.getStatus() != null) existingReq.setStatus(...)` and the same for each other field) and saves. **That is exactly why sending only the status is safe**: Jackson leaves every missing field as `null`, so pickup, drop, dates, driver and payment stay as they were and only the status changes. If the id does not exist the service returns null and the controller answers 404.
- `next` (lines 83-89) runs after the server confirmed. It updates the local object `request.status = status` so the table cell changes without reloading, calls `applyFilters()` (if the admin is filtering by `Pending`, an approved row now disappears from the list), and finally calls the optional callback `onDone` if one was given. Only the Close Request button passes a callback.
- There is no `error` handler: on failure nothing happens on screen. That is a small robustness gap in the code.

Lines 93-111 | Opening and closing the two modals

```
93 | openDriverDetails(request: DriverRequest): void {
94 |     this.selectedRequest = request;
95 |     this.showDriverModal = true;
96 | }
97 |
98 | closeDriverModal(): void {
99 |     this.showDriverModal = false;
100 |     this.selectedRequest = null;
101 | }
102 |
103 | openStages(request: DriverRequest): void {
104 |     this.selectedRequest = request;
105 |     this.showStageModal = true;
106 | }
107 |
108 | closeStageModal(): void {
109 |     this.showStageModal = false;
110 |     this.selectedRequest = null;
111 | }
```

- `openDriverDetails(request)` remembers the clicked row and sets `showDriverModal = true`; `closeDriverModal()` hides it and sets `selectedRequest` back to `null`.
- `openStages` and `closeStageModal` do the same for the progression modal.

Both modals share the single `selectedRequest` variable, which is safe because only one is open at a time. No HTTP call happens here; the driver data is already inside the request object (the nested `driver` came with the first GET).

Lines 113-124 | visibleStages and stageClass - progression logic

```
113 | /** Stages shown in the progression modal: a rejected request stops at "Rejected". */
114 | get visibleStages(): string[] {
115 |     return this.selectedRequest?.status === 'Rejected' ? ['Pending', 'Rejected'] : this.stages;
116 | }
117 |
118 | stageClass(stage: string): string {
119 |     const status = this.selectedRequest?.status || '';
120 |     if (status === 'Rejected') {
121 |         return stage === 'Rejected' ? 'rejected' : 'idle';
122 |     }
123 |     return this.stages.indexOf(stage) <= this.stages.indexOf(status) ? 'done' : 'idle';
124 | }
```

`get visibleStages()` is a **getter**: it is read like a property (`visibleStages`) but runs code each time. If the selected request is **Rejected**, only two boxes are shown, **Pending** and **Rejected**, because a rejected request never reaches **Approved** or **Trip End**; otherwise all four stages from `REQUEST_STAGES` are shown.

`stageClass(stage)` returns the CSS class name for one box of the progress bar:

- For a rejected request: the box labelled **Rejected** gets `'rejected'` (red) and everything else, including **Pending**, gets `'idle'` (grey).
- Otherwise each stage is compared by position. `this.stages.indexOf(stage) <= this.stages.indexOf(status)` means "this stage is at or before the current status", so it is `'done'` (green) and later stages are `'idle'`. Example: status **Approved** has index 1, so **Pending** and **Approved** are green while **Trip End** and **Closed** are grey. A status that is not in the list gives index -1, making every stage grey.

The template puts the returned word into `[ngClass]`, and `adminviewrequests.component.css` defines `.stage.done`, `.stage.idle` and `.stage.rejected`.

Lines 126-135 | canCloseRequest and closeRequest - the final admin step

```
126 | get canCloseRequest(): boolean {
127 |     return this.selectedRequest?.status === 'Trip End';
128 | }
129 |
130 | closeRequest(): void {
131 |     if (this.selectedRequest && this.canCloseRequest) {
132 |         this.changeStatus(this.selectedRequest, 'Closed', () => this.closeStageModal());
133 |     }
134 | }
135 | }
```

`canCloseRequest` is another getter, true only when the open request has status **Trip End**; the template shows the **Close Request** button only then. This is the last link of the chain: the customer ended the trip, now the admin confirms and closes it.

`closeRequest()` double-checks the same rule, then calls `changeStatus(selectedRequest, 'Closed', () => this.closeStageModal())`. The third argument is an **arrow function** passed as the `onDone` callback: after the server confirms the PUT, the modal closes itself. Line 135 closes the class.

Workflow, next: The admin's list is on screen and every button is wired to a method in this file. The visible markup that uses all these properties and methods is the template, so control moves to `adminviewrequests.component.html`.

Step 100: adminviewrequests.component.html - admin requests table and modals [Aarti - Admin Requests]

angularapp/src/app/components/adminviewrequests/adminviewrequests.component.html

FRONTEND

What this file is	The template of the admin requests page. It draws a search box and a status filter, a table with one row per request (with Approve, Reject, Show Driver Details and View Stage buttons) and two modals: Driver Details and Request Progression . It contains no logic of its own; every decision comes from <code>adminviewrequests.component.ts</code> .
Who / what triggers it	Angular renders it when the component is created by the router (route <code>adminviewrequests</code>), and re-renders the changed parts whenever a property of the class changes (change detection).
What it uses / calls	Properties and methods of <code>AdminviewrequestsComponent</code> . <code>ngModel</code> needs <code>FormsModule</code> (imported in <code>AppModule</code>) and <code>*ngIf</code> / <code>*ngFor</code> / <code>[ngClass]</code> come from <code>CommonModule</code> . The classes <code>page-container</code> , <code>page-card</code> , <code>btn</code> , <code>data-table</code> , <code>modal-backdrop</code> and <code>modal-box</code> are defined globally in <code>src/styles.css</code> ; the rest are in the component CSS.

Lines 1-4 | Page container, card and title

```
1 | <div class="page-container">
2 |   <div class="page-card" id="requestsCard">
3 |     <h2 class="page-title" id="requestsHeading">Driver Requests for Approval</h2>
4 |     <div class="title-line"></div>
```

Plain HTML wrappers. `page-container` centres the content with a maximum width, `page-card` is the white rounded card, `page-title` the big heading and `title-line` the thin blue underline (all in `styles.css`). The `id` attributes (`requestsCard`, `requestsHeading`) are hooks for automated tests.

Lines 6-15 | Toolbar: search box and status filter

```
6 |   <div class="toolbar">
7 |     <input type="text" id="searchInput" class="wide" placeholder="Search by Driver Name or Pickup Location..." [(ngM
8 |       <label>
9 |         Filter by Status:
10 |         <select id="statusFilter" [(ngModel)]="statusFilter" (ngModelChange)="applyFilters()">
11 |           <option value="All">All</option>
12 |           <option *ngFor="let status of statuses" [value]="status">{{ status }}</option>
13 |         </select>
14 |       </label>
15 |     </div>
```

- Line 7: an `<input>` with `[(ngModel)]="searchText"`. This is **two-way binding**: typing updates `searchText` in the class and changing `searchText` in code would update the box. `(ngModelChange)="applyFilters()"` is an event binding that runs `applyFilters()` on every keystroke, so the table narrows as you type. `class="wide"` is styled by the component CSS.
- Lines 8-14: a `<label>` wrapping a `<select id="statusFilter">`, also two-way bound to `statusFilter` with `(ngModelChange)="applyFilters()"`.
- Line 11 is a fixed option `All`. Line 12 uses `*ngFor="let status of statuses"` to repeat an `<option>` for each of the five statuses; `[value]="status"` sets the option's value property and `{{ status }}` (**interpolation**) prints the text.

Lines 17-29 | Table and header row

```
17 |   <div class="table-wrap">
18 |     <table class="data-table" id="requestsTable">
19 |       <thead>
20 |         <tr>
21 |           <th>S.NO</th>
22 |           <th>USERNAME</th>
23 |           <th>PICKUP LOCATION</th>
24 |           <th>DROP LOCATION</th>
25 |           <th>TRIP DATE</th>
26 |           <th>STATUS</th>
27 |           <th>ACTION</th>
28 |         </tr>
29 |       </thead>
```

`table-wrap` allows horizontal scrolling on small screens. `data-table` (global CSS) styles the table. The header lists seven columns: S.NO, USERNAME, PICKUP LOCATION, DROP LOCATION, TRIP DATE, STATUS, ACTION. It is static HTML.

Lines 30-33 | No-data row

```
30 |   <tbody>
31 |     <tr class="empty-row" id="noDataMessage" *ngIf="filteredRequests.length === 0">
32 |       <td colspan="7">Oops! No records found</td>
33 |     </tr>
```

`*ngIf="filteredRequests.length === 0"` is a **structural directive**: the `<tr>` is added to the page only when the filtered list is empty. `colspan="7"` makes the message span all columns. This message ("Oops! No records found") is shown both when there are really no requests (HTTP 204) and when the filters hide everything.

Lines 34-40 | Row loop and data cells

```
34 |     <tr *ngFor="let request of filteredRequests; let i = index" [attr.id]="request.driverRequestId"
35 |       >
36 |       <td>{{ i + 1 }}</td>
37 |       <td>{{ request.user?.username }}</td>
38 |       <td>{{ request.pickupLocation }}</td>
39 |       <td>{{ request.dropLocation }}</td>
40 |       <td>{{ toDateString(request.tripDate) }}</td>
41 |       <td class="status-cell" [class.trip-end]="request.status === 'Trip End'" [class.blink]="request.status ===
42 |         'Trip End'">{{ request.status }}</td>
```

- Line 34: `*ngFor="let request of filteredRequests; let i = index"` repeats the `<tr>` for every request and also gives the zero-based position as `i`. `[attr.id]='request-' + request.driverRequestId` sets the HTML attribute `id` to e.g. `request-3` (the `attr.` prefix is needed because `id` is an attribute here and the value is built by an expression).
- `{{ i + 1 }}` prints the serial number starting at 1.
- `request.user?.username` and the pickup/drop texts are plain interpolation; `?.` avoids an error if `user` is missing.
- `toDateString(request.tripDate)` calls the helper copied into the class, giving `YYYY-MM-DD`.
- Line 40, the status cell: `[class.trip-end]='request.status === 'Trip End'` adds the CSS class `trip-end` only when the condition is true, and the second `[class.blink]` adds `blink` under the same condition. `blink` is an animation defined **globally** in `styles.css` (`@keyframes blink` makes the opacity pulse between 1 and 0.25), so a request waiting to be closed flashes to catch the admin's attention. `trip-end` gives it a pink background through the component CSS.

Lines 41-47 | Action buttons

```
41 |         <td class="actions">
42 |             <button type="button" class="btn btn-primary btn-pill" (click)="openDriverDetails(request)">Show Driver
         Details</button>
43 |             <button type="button" class="btn btn-success btn-pill" *ngIf="canApprove(request)" (click)="approve(requ
         est)">Approve</button>
44 |             <button type="button" class="btn btn-danger btn-pill" *ngIf="canReject(request)" (click)="reject(request
         )">Reject</button>
45 |             <button type="button" class="btn btn-primary btn-pill" [class.stage-alert]='request.status === 'Trip End
         ' " (click)="openStages(request)">View Stage</button>
46 |         </td>
47 |     </tr>
```

Four buttons per row, all `type="button"` so they never submit a form.

- Show Driver Details** always appears and calls `openDriverDetails(request)`.
- Approve** has `*ngIf="canApprove(request)"` so it exists only for Pending or Rejected rows, and `(click)="approve(request)"` triggers the PUT.
- Reject** is shown by `canReject(request)` (Pending or Approved rows) and calls `reject(request)`.
- View Stage** opens the progression modal. `[class.stage-alert]` adds an attention style (blinking light-red text) on rows whose status is Trip End.
- `btn`, `btn-primary`, `btn-success`, `btn-danger` and `btn-pill` are global button styles (blue, green, red, rounded).

Lines 48-52 | Closing the table and card

```
48 |         </tbody>
49 |     </table>
50 | </div>
51 | </div>
52 | </div>
```

Closes `tbody`, `table`, the `table-wrap`, the card and the container opened earlier.

Lines 54-68 | Driver Details modal

```
54 | <!-- Driver details -->
55 | <div class="modal-backdrop" *ngIf="showDriverModal && selectedRequest">
56 |     <div class="modal-box detail-box" id="driverDetailsModal">
57 |         <button type="button" class="modal-close" id="closeDriverDetails" (click)="closeDriverModal()">X</button>
58 |         <h3>Driver Details</h3>
59 |         <img class="modal-avatar" [src]="getDriverImage(selectedRequest.driver?.image)" (error)="useDefaultAvatar($event)"
         alt="Driver" />
60 |         <p><strong>Driver Name:</strong> {{ selectedRequest.driver?.driverName }}</p>
61 |         <p><strong>License Number:</strong> {{ selectedRequest.driver?.licenseNumber }}</p>
62 |         <p><strong>Experience Years:</strong> {{ selectedRequest.driver?.experienceYears }}</p>
63 |         <p><strong>Contact Number:</strong> {{ selectedRequest.driver?.contactNumber }}</p>
64 |         <p><strong>Vehicle Type:</strong> {{ selectedRequest.driver?.vehicleType }}</p>
65 |         <p><strong>Hourly Rate:</strong> &#8377;{{ selectedRequest.driver?.hourlyRate }}</p>
66 |         <p><strong>Address:</strong> {{ selectedRequest.driver?.address }}</p>
67 |     </div>
68 | </div>
```

The comment on line 54 labels the block. The outer `<div class="modal-backdrop" *ngIf="showDriverModal && selectedRequest">` covers the whole screen with a dim blurred layer and exists only while the flag is true and a request is selected (the second check also lets TypeScript know `selectedRequest` is not null inside). The inner `modal-box` is the white dialog.

- The `X` button has `(click)="closeDriverModal()"`.
- The `<img>` uses `[src]="getDriverImage(selectedRequest.driver?.image)"` (property binding, so the value is computed) and `(error)="useDefaultAvatar($event)"`: if the picture cannot be displayed, the browser's error event calls the helper, which swaps in the default silhouette. `$event` is the DOM event object.

- The seven `<p>` lines print driver name, licence, experience, contact, vehicle, hourly rate (with `₹`, the rupee sign) and address from `selectedRequest.driver`, using `?.` throughout.

Lines 70-74 | Request Progression modal: frame

```
70 | <!-- Request progression -->
71 | <div class="modal-backdrop" *ngIf="showStageModal && selectedRequest">
72 |   <div class="modal-box stage-box" id="stageModal">
73 |     <button type="button" class="modal-close" id="closeStageModal" (click)="closeStageModal()">X</button>
74 |     <h3>Request Progression</h3>
```

Same modal pattern as above, shown while `showStageModal` is true. `closeStageModal()` is wired to the X button.

Lines 75-80 | Stage boxes and arrows

```
75 |   <div class="stages">
76 |     <ng-container *ngFor="let stage of visibleStages; let last = last">
77 |       <span class="stage" [ngClass]="stageClass(stage)">{{ stage }}</span>
78 |       <span class="arrow" *ngIf="!last" [class.on]="stageClass(visibleStages[visibleStages.indexOf(stage) + 1]) !==
79 |         'idle'">&#9654;</span>
80 |     </ng-container>
80 |   </div>
```

`<ng-container>` is an invisible wrapper that exists only so an `*ngFor` can be placed without adding an extra HTML element. `*ngFor="let stage of visibleStages; let last = last"` loops over the getter (two boxes for a rejected request, four otherwise) and `last` is true on the final item.

- `<span class="stage" [ngClass]="stageClass(stage)">` always has class `stage` and `[ngClass]` adds one more class (`done`, `idle` or `rejected`) returned by the method, which decides the colour.
- The arrow `▶` (a triangle) is drawn only if `*ngIf="!last"`, so there is no arrow after the last box. `[class.on]` turns it green when the **next** stage is not idle, i.e. the progress has already passed this point.

Lines 81-83 | Close Request button

```
81 |   <button type="button" class="btn btn-primary close-request" id="closeRequestButton" *ngIf="canCloseRequest" (click
82 |     )="closeRequest()">Close Request</button>
83 | </div>
```

`*ngIf="canCloseRequest"` shows the button only for a **Trip End** request. Clicking it runs `closeRequest()`, which sends the PUT with status **Closed**. Lines 82-83 close the modal box and backdrop.

Workflow, next: The structure is in place; the colours, spacing and the look of the modals and stage bar come from the stylesheet, so control moves to `adminviewrequests.component.css`.

Step 101: adminviewrequests.component.css - table actions, modals, stage bar [Aarti - Admin Requests]

angularapp/src/app/components/adminviewrequests/adminviewrequests.component.css

FRONTEND

What this file is	Styles that only apply to the admin requests page (Angular rewrites each selector so it only matches elements of this template). Common looks such as buttons, cards, tables and the modal backdrop are already provided by the global <code>src/styles.css</code> ; this file adds the page-specific parts.
Who / what triggers it	Loaded automatically through <code>styleUrls</code> in the component decorator when the component is used.
What it uses / calls	The CSS animation <code>blink</code> defined globally in <code>src/styles.css</code> (a component stylesheet can use a keyframe name defined in the global file).
Size	21 lines

Lines 1-4 | Toolbar input, table wrapper and action buttons

```
1 | .toolbar input.wide { flex: 1; min-width: 240px; max-width: 460px; font-weight: 400; }
2 | .table-wrap { margin-top: 4px; }
3 | .actions { white-space: nowrap; }
4 | .actions .btn { margin: 2px 2px; font-size: 12px; }
```

- `.toolbar input.wide` lets the search box grow (`flex: 1`) with a minimum 240 px and maximum 460 px width, and normal font weight (the global toolbar font is bold).
- `.table-wrap` adds a small top margin.
- `.actions { white-space: nowrap }` keeps the four buttons of a row on one line.

- `.actions .btn` gives each button a small margin and a smaller 12 px font so four buttons fit in one cell.

Lines 5-6 | Trip End highlight and blinking View Stage button

```
5 | .status-cell.trip-end { background: #fbd9dc; }
6 | .stage-alert { color: #ffd4d4; animation: blink 1.1s ease-in-out infinite; }
```

`.status-cell.trip-end` gives the status cell of a Trip End row a pink background (`#fbd9dc`). The cell also carries the global `blink` class, so the text pulses. `.stage-alert` makes the View Stage button's text light red and applies `animation: blink 1.1s ease-in-out infinite`; `blink` is defined in `styles.css` as `@keyframes blink { 0%,100% {opacity:1} 50% {opacity:0.25} }`, so the button fades in and out forever, 1.1 seconds per cycle.

Lines 8-11 | Driver details modal

```
8 | .detail-box { min-width: 340px; }
9 | .detail-box h3 { margin-bottom: 10px; }
10 | .detail-box p { margin: 6px 0; font-size: 13px; }
11 | .modal-avatar { width: 84px; height: 84px; margin: 0 auto 14px; display: block; object-fit: cover; border-radius: 50%;
    | border: 3px solid #1d8a8a; background: #fff; }
```

`.detail-box` makes the dialog at least 340 px wide; its heading gets a bottom margin and each `<p>` a tighter 13 px size. `.modal-avatar` makes the driver photo an 84 by 84 px circle (`border-radius: 50%`), centred (`display:block; margin: 0 auto`), cropped to fill (`object-fit: cover`), with a teal 3 px ring and a white background in case the image is transparent.

Lines 13-15 | Stage modal and stage pill

```
13 | .stage-box { min-width: 420px; padding-bottom: 24px; }
14 | .stages { display: flex; align-items: center; justify-content: center; gap: 4px; margin: 6px 0 4px; flex-wrap: wrap; }
15 | .stage { padding: 6px 18px; border-radius: 999px; font-size: 12px; font-weight: 700; }
```

`.stage-box` widens the dialog to 420 px minimum. `.stages` is a flex row that centres the pills and arrows, with a small gap and `flex-wrap` so they wrap on narrow screens. `.stage` makes each stage a pill (`border-radius: 999px`) with padding and bold 12 px text.

Lines 16-21 | Stage colours, arrows and the Close button

```
16 | .stage.done { background: #2fa84f; color: #fff; }
17 | .stage.idle { background: #ecec; color: #9a9a9a; }
18 | .stage.rejected { background: #f0443a; color: #fff; }
19 | .arrow { font-size: 10px; color: #bbb; }
20 | .arrow.on { color: #2fa84f; }
21 | .close-request { margin-top: 18px; padding: 8px 20px; }
```

These are the classes returned by `stageClass()`: `.stage.done` is green with white text, `.stage.idle` is light grey with grey text and `.stage.rejected` is red with white text. `.arrow` is a small grey triangle and `.arrow.on` turns it green when progress has passed it. `.close-request` adds space above and padding to the Close Request button.

Workflow, next: The admin side of the lifecycle is complete: the admin can approve or reject, and later close, the requests that customers create. Now we look at the same data from the customer's point of view, the "My Requests" page, starting with `customerviewrequested.component.ts`.

Step 102: CustomerviewrequestedComponent - My Requests, trip end and fare [Vaishalini - Customer Journey & Shared Infra + Prajan (Trip End / Pay Amount)]

FRONTEND

`angularapp/src/app/components/customerviewrequested/customerviewrequested.component.ts`

What this file is

The customer's **"My Requests"** page. It lists only the logged-in customer's own requests as cards, with a search by driver name, and offers actions that depend on the request status:

- **Show More** opens a details modal (any status).
- **Edit** and **Delete** are allowed only while the request is **Pending**.
- **Trip End** is allowed only when the admin has set the request to **Approved**.
- **Fetch Pay Amount** is allowed after the trip has ended (**Trip End** or **Closed**) and shows a trip summary, then leads to the review page.

The most important logic is `endTrip()`, which works out how long the trip lasted and calculates the fare in the browser.

Honest note about payment: there is no payment gateway, card form or bank integration anywhere in this project. The "payment" is only a calculation (hourly rate × time) saved in the database field `paymentAmount`, followed by an on-screen summary. Nobody is actually charged.

Who / what triggers it	The router: path <code>customerviewrequested</code> with <code>AuthGuard</code> and <code>data: { role: 'Customer' }</code> . The customer arrives from the customer navbar, or after submitting a new request in <code>CustomerRequestComponent</code> . Angular constructs the class and calls <code>ngOnInit()</code> ; when the user leaves the page it calls <code>ngOnDestroy()</code> .
What it uses / calls	<code>DriverRequestService</code>: <code>getDriverRequestsByUserId</code>, <code>deleteDriverRequest</code>, <code>updateDriverRequest</code>. <code>AuthService</code>.<code>getUserId()</code> (reads <code>userId</code> from <code>localStorage</code>). <code>Router</code> for navigation. - Helpers in <code>utils/format.ts</code>: <code>combineDateTime</code>, <code>formatDuration</code>, <code>parseDurationToMinutes</code>, <code>toDateString</code>, <code>toDisplayDate</code>, <code>toTimeString</code>; and <code>utils/driver-image.ts</code>. - Backend: <code>GET /api/driverRequest/user/{id} -> DriverRequestController.viewDriverRequestsByUserId</code>; <code>PUT /api/driverRequest/{id} -> updateDriverRequest</code>; <code>DELETE /api/driverRequest/{id} -> deleteDriverRequest</code>.
Size	184 lines

Lines 1-14 | Imports

```

1 | import { Component, OnDestroy, OnInit } from '@angular/core';
2 | import { Router } from '@angular/router';
3 | import { DriverRequestService } from '../services/driver-request.service';
4 | import { AuthService } from '../services/auth.service';
5 | import { DriverRequest } from '../models/driver-request.model';
6 | import {
7 |   combineDateTime,
8 |   formatDuration,
9 |   parseDurationToMinutes,
10 |   toDateString,
11 |   toDisplayDate,
12 |   toTimeString
13 | } from '../utils/format';
14 | import { getDriverImage, useDefaultAvatar } from '../utils/driver-image';

```

- `Component`, `OnDestroy`, `OnInit` from Angular core: the decorator and two **lifecycle hook** interfaces (`OnInit` = run code after creation, `OnDestroy` = run code just before the component is thrown away).
- `Router` lets the code navigate to another page without a link click.
- `DriverRequestService` and `AuthService` are the injected services; `DriverRequest` is the model interface.
- The multi-line import (lines 6-13) pulls six helpers from `utils/format.ts`: `combineDateTime` (date + time -> JavaScript `Date`), `formatDuration` (minutes -> "H hours M minutes"), `parseDurationToMinutes` ("3 hours" -> 180), and three formatters for dates and times.
- `getDriverImage` and `useDefaultAvatar` handle the driver photo and its fallback, as on the admin page.

Lines 16-20 | @Component decorator

```

16 | @Component({
17 |   selector: 'app-customerviewrequested',
18 |   templateUrl: './customerviewrequested.component.html',
19 |   styleUrls: ['./customerviewrequested.component.css']
20 | })

```

Registers the class as a component with the tag name `app-customerviewrequested`, its HTML template and its CSS file. As on every page, the router places it into the `<router-outlet>`.

Lines 21-30 | Class and state fields

```

21 | export class CustomerviewrequestedComponent implements OnInit, OnDestroy {
22 |   requests: DriverRequest[] = [];
23 |   searchText: string = '';
24 |
25 |   selectedRequest: DriverRequest | null = null;
26 |   showDetailsModal: boolean = false;
27 |   showDeleteModal: boolean = false;
28 |   showPayModal: boolean = false;
29 |   payLoading: boolean = false;
30 |   private payTimer: any;

```

`implements OnInit, OnDestroy` promises the two hook methods below.

- `requests` is the list of this customer's requests; `searchText` is the text typed in the search box.
- `selectedRequest` is the request chosen for whichever modal is open.
- `showDetailsModal`, `showDeleteModal`, `showPayModal` are the three modal flags.
- `payLoading` is true during the artificial "Calculating your fare..." wait.
- `payTimer` stores the handle returned by `setTimeout` so the timer can be cancelled later; `private` means only this class can touch it, and `any` is used because the type differs between browser and Node typings.

Lines 32-36 | Helpers exposed to the template

```
32 |   toDateString = toDateString;
33 |   toTimeString = toTimeString;
34 |   toDisplayDate = toDisplayDate;
35 |   getDriverImage = getDriverImage;
36 |   useDefaultAvatar = useDefaultAvatar;
```

The same trick as on the admin page: functions imported at the top are assigned to class properties so the HTML can call `toDisplayDate(...)`, `toTimeString(...)`, `getDriverImage(...)` and so on.

Lines 38-42 | Constructor (dependency injection)

```
38 |   constructor(
39 |     private driverRequestService: DriverRequestService,
40 |     private authService: AuthService,
41 |     private router: Router
42 |   ) {}
```

Angular injects three objects: the request service, the auth service (to know who is logged in) and the router. The `private` keyword turns each parameter into a field.

Lines 44-50 | `ngOnInit` and `ngOnDestroy`

```
44 |   ngOnInit(): void {
45 |     this.loadRequests();
46 |   }
47 |
48 |   ngOnDestroy(): void {
49 |     clearTimeout(this.payTimer);
50 |   }
```

`ngOnInit()` calls `loadRequests()` so the cards appear when the page opens.

`ngOnDestroy()` runs when the customer leaves the page. It calls `clearTimeout(this.payTimer)` to cancel the 1.5-second fare timer if it is still waiting; otherwise the callback would later change a component that no longer exists.

Lines 52-62 | `loadRequests` - GET `/api/driverRequest/user/{id}`

```
52 |   loadRequests(): void {
53 |     const userId = this.authService.getUserId();
54 |     if (userId === null) {
55 |       return;
56 |     }
57 |     this.driverRequestService.getDriverRequestsByUserId(userId).subscribe({
58 |       // The API answers 404 / 204 when the customer has no requests yet.
59 |       next: (requests) => (this.requests = requests || []),
60 |       error: () => (this.requests = [])
61 |     });
62 |   }
```

- Line 53: `getUserId()` reads the `userId` stored in `localStorage` at login (it returns `null` if missing). If it is null the method stops.
- Line 57: sends GET `http://localhost:8080/api/driverRequest/user/{userId}` with the Bearer token. Security rule: GET `/api/driverRequest/user/**` -> `hasRole("CUSTOMER")`.
- On the server `DriverRequestController.viewDriverRequestsByUserId` calls `DriverRequestServiceImpl.findDriverRequestsByUserId` which runs the derived query `driverRequestRepo.findByUserUserId(userId)` ("find requests whose user's `userId` equals this"). If the list is empty the controller answers **404 Not Found** (not 204), otherwise 200 with the JSON array.
- `next` stores the array (`requests || []` also protects against a null body). The comment on line 58 explains the `error` branch: a customer with no requests gets a 404, which Angular treats as an error, so the code just sets an empty list and the page shows "No Data Found". The arrow function without braces (`(requests) => (this.requests = ...)`) is a one-expression shortcut.

Lines 64-70 | `filteredRequests` getter - the search

```
64 |   get filteredRequests(): DriverRequest[] {
65 |     const query = this.searchText.trim().toLowerCase();
66 |     if (!query) {
67 |       return this.requests;
68 |     }
69 |     return this.requests.filter((request) => (request.driver?.driverName || '').toLowerCase().includes(query));
70 |   }
```

A **getter**, so the template can write `filteredRequests` as if it were a variable. It trims and lower-cases `searchText`; if empty it returns the full list; otherwise it returns only requests whose driver name contains the text. It is recalculated every time Angular checks the page, which is fine for a short list. Because the search box is bound with `[(ngModel)]`, typing changes `searchText` and the cards update instantly with no button.

Lines 72-82 | Status rules for the buttons

```
72 |   isPending(request: DriverRequest): boolean {
73 |     return request.status === 'Pending';
74 |   }
75 |
76 |   canEndTrip(request: DriverRequest): boolean {
77 |     return request.status === 'Approved';
78 |   }
79 |
80 |   canFetchPayAmount(request: DriverRequest): boolean {
81 |     return request.status === 'Trip End' || request.status === 'Closed';
82 |   }
```

Three tiny predicates decide which buttons are enabled (the template binds them to `[disabled]`):

- `isPending` is true only for **Pending**, gating **Edit** and **Delete**: once the admin has approved or rejected a request the customer can no longer change or remove it.
- `canEndTrip` is true only for **Approved**: a trip can only be ended after approval.
- `canFetchPayAmount` is true for **Trip End** or **Closed**: the fare exists only after the trip ended; it stays visible after the admin closes the request.

Lines 84-93 | Show More modal

```
84 |   // ----- details -----
85 |   showMore(request: DriverRequest): void {
86 |     this.selectedRequest = request;
87 |     this.showDetailsModal = true;
88 |   }
89 |
90 |   closeDetails(): void {
91 |     this.showDetailsModal = false;
92 |     this.selectedRequest = null;
93 |   }
```

`showMore` stores the clicked request and opens the details modal; `closeDetails` closes it and clears the selection. No server call is needed because all fields were already downloaded in the list.

Lines 95-100 | editRequest - go to the edit form

```
95 |   // ----- edit / delete -----
96 |   editRequest(request: DriverRequest): void {
97 |     if (this.isPending(request)) {
98 |       this.router.navigate(['/customer-request/edit', request.driverRequestId]);
99 |     }
100 |   }
```

If the request is still pending, `this.router.navigate(['/customer-request/edit', request.driverRequestId])` changes the URL to e.g. `/customer-request/edit/5`. The routing table maps that path to `CustomerRequestComponent`, which sees the `driverRequestId` route parameter, loads the request and shows the form in edit mode. The `isPending` check repeats the disabled-button rule as a safety net.

Lines 102-112 | askDelete and cancelDelete

```
102 |   askDelete(request: DriverRequest): void {
103 |     if (this.isPending(request)) {
104 |       this.selectedRequest = request;
105 |       this.showDeleteModal = true;
106 |     }
107 |   }
108 |
109 |   cancelDelete(): void {
110 |     this.showDeleteModal = false;
111 |     this.selectedRequest = null;
112 |   }
```

`askDelete` (pending requests only) remembers the request and opens the confirmation modal; the delete is not sent yet. `cancelDelete` hides the modal and clears the selection. This two-step flow prevents accidental deletes.

Lines 114-126 | confirmDelete - DELETE /api/driverRequest/{id}

```
114 |   confirmDelete(): void {
115 |     const id = this.selectedRequest?.driverRequestId;
116 |     if (id === undefined) {
117 |       return;
118 |     }
119 |     this.driverRequestService.deleteDriverRequest(id).subscribe({
120 |       next: () => {
121 |         this.requests = this.requests.filter((request) => request.driverRequestId !== id);
122 |         this.cancelDelete();
123 |       },
124 |       error: () => this.cancelDelete()
125 |     });
126 |   }
```

Reads the id from the selected request (?. because it may be null). If undefined it stops. Otherwise it sends `DELETE http://localhost:8080/api/driverRequest/{id}` (security: `hasRole("CUSTOMER")`).

On the server `DriverRequestController.deleteDriverRequest` calls `DriverRequestServiceImpl.deleteDriverRequest`, which finds the row, deletes it and returns it (200), or returns null (404) if it does not exist.

On success `this.requests.filter(...)` builds a new array without the deleted id so the card vanishes immediately, then `cancelDelete()` closes the modal. On error the modal also just closes.

Lines 128-141 | endTrip - measuring the duration and computing the fare

```
128 | // ----- trip end -----
129 | endTrip(request: DriverRequest): void {
130 |   if (!this.canEndTrip(request) || request.driverRequestId === undefined) {
131 |     return;
132 |   }
133 |   const now = new Date();
134 |   const start = combineDateTime(request.tripDate, request.timeSlot);
135 |   let minutes = start ? Math.round((now.getTime() - start.getTime()) / 60000) : 0;
136 |   if (minutes <= 0) {
137 |     // The trip "started" in the future (e.g. demo data): fall back to the estimated duration.
138 |     minutes = parseDurationToMinutes(request.estimatedDuration);
139 |   }
140 |   const hourlyRate = Number(request.driver?.hourlyRate || 0);
141 |   const paymentAmount = Math.round(hourlyRate * (minutes / 60) * 100) / 100;
```

Triggered by the **Trip End** button. The guard on line 130 stops the method unless the status is `Approved` and an id exists.

1. `now = new Date()` is the moment of the click (the time the customer ends the trip).
2. `start = combineDateTime(request.tripDate, request.timeSlot)` merges the booked trip date and time slot into one `Date` (local time); it returns `null` if either is missing.
3. `minutes = whole minutes from start to now: Math.round((now.getTime() - start.getTime()) / 60000). getTime() gives milliseconds since 1970 and 60,000 ms is one minute. If there is no start the value is 0.`
4. If `minutes <= 0` (the trip is booked for the **future**, or the time data is missing, which is common with demo data) the code falls back to `parseDurationToMinutes(request.estimatedDuration)`, i.e. the duration the customer typed when booking, such as "3 hours" -> 180.
5. `hourlyRate` is the driver's rate as a number (`|| 0` avoids NaN).
6. `paymentAmount = Math.round(hourlyRate * (minutes / 60) * 100) / 100`: hours = minutes / 60, fare = rate × hours, multiplied by 100, rounded to a whole number, divided by 100 again, which **rounds to 2 decimals**.

Worked example 1 (trip already happened): hourly rate 500, booked 10:00, customer presses Trip End at 12:30 the same day. Minutes = 150. Fare = 500 × 150 / 60 = 1250.00. Duration text will be "2 hours 30 minutes".

Worked example 2 (rounding): rate 350 and 100 minutes. 350 × 100 / 60 = 583.3333...; times 100 = 58333.33; rounded 58333; divided by 100 = **583.33**. Duration text "1 hours 40 minutes".

Worked example 3 (future trip fallback): trip booked for tomorrow, estimated duration "3 hours", rate 400. The difference is negative so minutes = 180 and fare = 400 × 180 / 60 = 1200.00.

Lines 143-149 | The update body

```
143 | const update = {
144 |   status: 'Trip End',
145 |   actualDropDate: toDateString(now),
146 |   actualDropTime: toTimeString(now, true),
147 |   actualDuration: formatDuration(minutes),
148 |   paymentAmount
149 | };
```

Builds the object to send:

- `status: 'Trip End'` moves the request along the chain (Approved -> Trip End).
- `actualDropDate: toDateString(now)` gives YYYY-MM-DD.
- `actualDropTime: toTimeString(now, true)` gives HH:mm:ss (the `true` asks for seconds). The backend fields are `LocalDate` and `LocalTime`, which Jackson can read from these strings.
- `actualDuration: formatDuration(minutes)` gives text such as "2 hours 30 minutes" (it always writes the word "hours", even for 1).
- `paymentAmount` is the number computed above (the backend field is a `Double`).

Lines 150-161 | Sending the PUT and updating the card

```
150 |     this.driverRequestService
151 |       .updateDriverRequest(request.driverRequestId, update as unknown as DriverRequest)
152 |       .subscribe({
153 |         next: () => {
154 |           request.status = update.status;
155 |           request.actualDropDate = update.actualDropDate as unknown as Date;
156 |           request.actualDropTime = update.actualDropTime as unknown as Date;
157 |           request.actualDuration = update.actualDuration;
158 |           request.paymentAmount = update.paymentAmount;
159 |         }
160 |       });
161 | }
```

`updateDriverRequest(id, update)` sends PUT `http://localhost:8080/api/driverRequest/{id}`. The cast `as unknown as DriverRequest` is again just to satisfy TypeScript. The server method

`DriverRequestServiceImpl.updateDriverRequest` copies each non-null field (status, actualDropDate, actualDropTime, actualDuration, paymentAmount) onto the stored row and leaves everything else alone.

In `next` the same values are copied onto the local `request` object so the card refreshes at once: its status text turns into "Trip End", the Trip End button becomes disabled and Fetch Pay Amount becomes enabled. The `as unknown as Date` casts exist because the model declares those fields as `Date` but we store strings. There is no `error` handler, so a failure is silent.

Be aware that the amount is calculated by the **browser** and trusted by the server; the backend does not recompute or validate it.

Lines 163-170 | fetchPayAmount - the fake "calculating" wait

```
163 | // ----- payment -----
164 | fetchPayAmount(request: DriverRequest): void {
165 |   this.selectedRequest = request;
166 |   this.showPayModal = true;
167 |   this.payLoading = true;
168 |   clearTimeout(this.payTimer);
169 |   this.payTimer = setTimeout(() => (this.payLoading = false), 1500);
170 | }
```

Triggered by **Fetch Pay Amount**. It remembers the request, opens the pay modal (`showPayModal = true`) and sets `payLoading = true`, which makes the template show a spinner and the text "Calculating your fare...".

`clearTimeout(this.payTimer)` cancels any earlier timer (for example after a double click), then `setTimeout(() => (this.payLoading = false), 1500)` schedules a callback after 1500 ms (1.5 seconds) that turns the spinner off, so the **Trip Summary** appears. No server call is made: the amount, duration and drop-off data were already saved in the request, so the delay is only a visual effect.

Lines 172-177 | closePayModal

```
172 | closePayModal(): void {
173 |   clearTimeout(this.payTimer);
174 |   this.showPayModal = false;
175 |   this.payLoading = false;
176 |   this.selectedRequest = null;
177 | }
```

Cancels the timer (so it cannot fire after the modal is closed), hides the modal, resets `payLoading` and clears `selectedRequest`. It is used by the X button, the **Skip Now** button and by `writeReview`.

Lines 179-184 | writeReview - go to the feedback page

```
179 | writeReview(): void {
180 |   const driverId = this.selectedRequest?.driver?.driverId;
181 |   this.closePayModal();
182 |   this.router.navigate(['/customerpostfeedback'], { queryParams: driverId !== undefined ? { driverId } : {} });
183 | }
184 | }
```

Triggered by **Write a Review** in the Trip Summary.

- Line 180 captures the driver's id **before** the modal is closed, because `closePayModal()` sets `selectedRequest` to null.
- `this.router.navigate(['/customerpostfeedback'], { queryParams: ... })` changes the URL to `/customerpostfeedback?driverId=7` (if the id exists). A **query parameter** is the `?key=value` part of a URL; the feedback page can read it to pre-select the driver. If no id is known an empty object is passed and the URL has no query string.

Line 184 closes the class.

Workflow, next: The component class is complete. Its visible part, including the cards, the three modals and the `[disabled]` bindings that use these methods, is `customerviewrequested.component.html`.

Step 103: customerviewrequested.component.html - request cards and modals [Vaishalini - Customer Journey & Shared Infra + Prajan (Trip End / Pay Amount)]

FRONTEND

angularapp/src/app/components/customerviewrequested/customerviewrequested.component.html

What this file is	The template of the "My Requests" page: a search box, a set of cards (one per request) with a coloured status and five buttons, and three modals: Request Details , Delete confirmation and Payment / Trip Summary (with the loading spinner).
Who / what triggers it	Rendered by Angular when the component is created; updated automatically whenever the class properties change.
What it uses / calls	Everything exposed by <code>CustomerviewrequestedComponent</code> . Directives need <code>CommonModule</code> and <code>FormsModule</code> . Layout classes (<code>page-container</code> , <code>page-card</code> , <code>btn</code> , <code>modal-*</code> , <code>spinner</code>) are in the global <code>styles.css</code> ; the card styles are in the component CSS.
Size	86 lines

Lines 1-4 | Container, card and heading

```
1 | <div class="page-container narrow">
2 |   <div class="page-card" id="myRequestsCard">
3 |     <h2 class="page-title steel" id="myRequestsHeading">My Requests</h2>
4 |     <div class="title-line"></div>
```

`page-container narrow` is the global container with a smaller 920 px maximum width (set in the component CSS). `page-title steel` colours the heading steel blue. `title-line` is the blue underline. The `id` attributes are test hooks.

Lines 6-8 | Search box

```
6 |   <div class="search-row">
7 |     <input type="text" id="searchInput" class="search-input" placeholder="Search by Driver Name..." [(ngModel)]="searchText" />
8 |   </div>
```

`[(ngModel)]="searchText"` is two-way binding: each keystroke updates `searchText`. There is no `(ngModelChange)` because `filteredRequests` is a getter that is re-evaluated automatically, so the cards filter live.

Line 10 | Empty message

```
10 |   <div class="empty-text" id="noDataMessage" *ngIf="filteredRequests.length === 0">No Data Found</div>
```

`*ngIf="filteredRequests.length === 0"` shows "No Data Found" when the customer has no requests (server answered 404/204) or when the search matches nothing.

Lines 12-13 | Card list and the repeating card

```
12 |   <div class="cards" *ngIf="filteredRequests.length > 0">
13 |     <div class="request-card" *ngFor="let request of filteredRequests" [attr.id]="request.driverRequestId">
```

`*ngIf="filteredRequests.length > 0"` creates the `.cards` container only when there is something to show.

`*ngFor="let request of filteredRequests"` makes one `.request-card` per request, and `[attr.id]` sets an id such as `request-5`.

Lines 14-24 | Card body: photo and information

```
14 |     <div class="card-main">
15 |       <img class="avatar" [src]="getDriverImage(request.driver?.image)" (error)="useDefaultAvatar($event)" alt="Driver" />
16 |       <div class="info">
17 |         <h3>{{ request.driver?.driverName }}</h3>
18 |         <p><strong>Vehicle Type:</strong> {{ request.driver?.vehicleType }}</p>
19 |         <p><strong>Pickup:</strong> {{ request.pickupLocation }}</p>
20 |         <p><strong>Drop:</strong> {{ request.dropLocation }}</p>
21 |         <p><strong>Trip Date:</strong> {{ toDisplayDate(request.tripDate) }}</p>
22 |         <p><strong>Status:</strong> <span class="status" [ngClass]="statusClass" >{{ request.status }}</span></p>
23 |       </div>
24 |     </div>
```

- `<img class="avatar" [src]="getDriverImage(request.driver?.image)" (error)="useDefaultAvatar($event)">`: the driver's photo, with the default silhouette if loading fails.
- `{{ request.driver?.driverName }}`, vehicle type, pickup, drop and the trip date (`toDisplayDate` shows DD/MM/YYYY).
- Line 22 is the **status label**: `<span class="status" [ngClass]='s-' + request.status.split(' ').join(' ').toLowerCase()'>`. The expression builds a class name from the status text in four steps. `split(' ')` cuts the string at spaces, `join(' ')` glues the pieces back with no spaces, `toLowerCase()` makes it lower case and `'s-' +` puts a prefix in front. So `Pending` becomes `s-pending`, `Approved` -> `s-approved`, `Rejected` -> `s-rejected`, `Closed` -> `s-closed` and `Trip End` -> `TripEnd` -> `tripend` -> `s-tripend` (the space has to be removed because a CSS class name cannot contain one). The component CSS defines a colour for each of these classes.

Lines 25-31 | Card footer: the five buttons

```

25 |         <div class="card-footer">
26 |             <button type="button" class="btn btn-primary btn-pill" [id]='showMore-' + request.driverRequestId" (click)=
|             "showMore(request)">Show More</button>
27 |             <button type="button" class="btn btn-success btn-pill" [id]='edit-' + request.driverRequestId" [disabled]="
|             !isPending(request)" (click)="editRequest(request)">Edit</button>
28 |             <button type="button" class="btn btn-danger btn-pill" [id]='delete-' + request.driverRequestId" [disabled]="
|             !isPending(request)" (click)="askDelete(request)">Delete</button>
29 |             <button type="button" class="btn btn-warning btn-pill" [id]='tripEnd-' + request.driverRequestId" [disabled
|             ]=!canEndTrip(request)" (click)="endTrip(request)">Trip End</button>
30 |             <button type="button" class="btn btn-primary btn-pill" [id]='fetchPay-' + request.driverRequestId" [disable
|             d]=!canFetchPayAmount(request)" (click)="fetchPayAmount(request)">Fetch Pay Amount</button>
31 |         </div>

```

Every button gets an id built with `[id]='edit-' + request.driverRequestId`, so tests can find a specific one, and `(click)` calls the matching method.

- **Show More** is always enabled and calls `showMore(request)`.
- **Edit** and **Delete** have `[disabled]='!isPending(request)'`: the HTML `disabled` property is true unless the status is Pending, so the buttons are greyed out and unclickable. They call `editRequest` and `askDelete`.
- **Trip End** is disabled unless `canEndTrip(request)` (status Approved) and calls `endTrip(request)`.
- **Fetch Pay Amount** is disabled unless `canFetchPayAmount(request)` (Trip End or Closed) and calls `fetchPayAmount(request)`.

The style of a disabled button is described in the CSS file.

Lines 32-35 | Closing the loop and container

```

32 |         </div>
33 |     </div>
34 | </div>
35 | </div>

```

Closes the card, the cards list, the page card and the container.

Lines 37-53 | Request Details modal

```

37 | <!-- Request details -->
38 | <div class="modal-backdrop" *ngIf="showDetailsModal && selectedRequest">
39 |     <div class="modal-box detail-box" id="detailsModal">
40 |         <button type="button" class="modal-close" id="closeDetails" (click)="closeDetails()">X</button>
41 |         <h3>Request Details</h3>
42 |         <p><strong>Driver Name:</strong> {{ selectedRequest.driver?.driverName }}</p>
43 |         <p><strong>Contact Number:</strong> {{ selectedRequest.driver?.contactNumber }}</p>
44 |         <p><strong>Pickup Location:</strong> {{ selectedRequest.pickupLocation }}</p>
45 |         <p><strong>Drop Location:</strong> {{ selectedRequest.dropLocation }}</p>
46 |         <p><strong>Request Date:</strong> {{ toDisplayDate(selectedRequest.requestDate) }}</p>
47 |         <p><strong>Trip Date:</strong> {{ toDisplayDate(selectedRequest.tripDate) }}</p>
48 |         <p><strong>Time Slot:</strong> {{ toTimeString(selectedRequest.timeSlot) }}</p>
49 |         <p><strong>Estimated Duration:</strong> {{ selectedRequest.estimatedDuration }}</p>
50 |         <p><strong>Status:</strong> {{ selectedRequest.status }}</p>
51 |         <p><strong>Comments:</strong> {{ selectedRequest.comments || '-' }}</p>
52 |     </div>
53 | </div>

```

The comment names the block. `*ngIf="showDetailsModal && selectedRequest"` creates the dim backdrop and dialog only while the flag is on. The X button runs `closeDetails()`.

The ten `<p>` lines print: driver name and contact, pickup and drop locations, request date and trip date (`toDisplayDate`), the time slot (`toTimeString`, giving HH:mm), the estimated duration, the status, and the comments. `{{ selectedRequest.comments || '-' }}` shows a dash if the customer wrote no comment.

Lines 55-64 | Delete confirmation modal

```
55 | <!-- Delete confirmation -->
56 | <div class="modal-backdrop" *ngIf="showDeleteModal">
57 |   <div class="modal-box" id="deleteModal">
58 |     <p>Are you sure you want to delete this request?</p>
59 |     <div class="modal-actions">
60 |       <button type="button" class="btn btn-danger" id="confirmDelete" (click)="confirmDelete()">Yes, Delete</button>
61 |       <button type="button" class="btn btn-secondary" id="cancelDelete" (click)="cancelDelete()">Cancel</button>
62 |     </div>
63 |   </div>
64 | </div>
```

Shown while `showDeleteModal` is true. It asks "Are you sure you want to delete this request?" with two buttons: **Yes, Delete** calls `confirmDelete()` (which sends the DELETE) and **Cancel** calls `cancelDelete()`. `modal-actions` is the global container that stacks the buttons.

Lines 66-72 | Payment modal: frame and loading state

```
66 | <!-- Payment details -->
67 | <div class="modal-backdrop" *ngIf="showPayModal && selectedRequest">
68 |   <div class="modal-box detail-box" id="payModal">
69 |     <button type="button" class="modal-close" id="closePayModal" (click)="closePayModal()">X</button>
70 |     <div class="pay-loading" *ngIf="payLoading">
71 |       <div class="spinner"></div>
72 |       <p>Calculating your fare...</p>
```

The modal exists while `showPayModal && selectedRequest`. Inside, the `.pay-loading` block has `*ngIf="payLoading"`, so for 1.5 seconds it shows an empty `<div class="spinner">` (the rotating ring, drawn by the global `.spinner` rule and the `spin` keyframes) and the text "Calculating your fare...".

Lines 73-84 | Payment modal: Trip Summary and the two buttons

```
73 |   </div>
74 |   <ng-container *ngIf="!payLoading">
75 |     <h3>Trip Summary</h3>
76 |     <p><strong>Travel Charge:</strong> &#8377;{{ selectedRequest.paymentAmount }}</p>
77 |     <p><strong>Trip Duration:</strong> {{ selectedRequest.actualDuration }}</p>
78 |     <p><strong>Dropped At:</strong> {{ toDisplayDate(selectedRequest.actualDropDate) }} {{ toTimeString(selectedRequest.actualDropTime) }}</p>
79 |     <p><strong>Drop Location:</strong> {{ selectedRequest.dropLocation }}</p>
80 |     <div class="modal-actions">
81 |       <button type="button" class="btn btn-primary" id="writeReview" (click)="writeReview()">Write a Review</button>
82 |       <button type="button" class="btn btn-secondary" id="skipNow" (click)="closePayModal()">Skip Now</button>
83 |     </div>
84 |   </ng-container>
```

`<ng-container *ngIf="!payLoading">` is an invisible wrapper that appears once the timer has ended, so it replaces the spinner. It shows **Trip Summary**: travel charge (`₹` is the rupee symbol followed by `selectedRequest.paymentAmount`), trip duration (the `actualDuration` saved by `endTrip`), drop-off date and time and the drop location.

There is no card form or payment gateway; this is just a read-out of the numbers computed earlier.

Write a Review (`id="writeReview"`) calls `writeReview()`, which navigates to `/customerpostfeedback?driverId=...`. **Skip Now** just calls `closePayModal()`.

Lines 85-86 | Closing tags

```
85 |   </div>
86 | </div>
```

Closes the modal box and the backdrop of the payment modal.

Workflow, next: All the markup is described. Cards, the status colours, disabled buttons and the loader are styled in `customerreviewrequested.component.css`.

Step 104: customerviewrequested.component.css - cards and status colours [Vaishalini - Customer Journey & Shared Infra + Prajan (Trip End / Pay Amount)]

angularapp/src/app/components/customerviewrequested/customerviewrequested.component.css

FRONTEND

What this file is	Page-specific styles for the My Requests cards, status colours, disabled buttons and the loading text. Generic pieces such as the modal, the buttons and the spinner come from the global <code>styles.css</code> .
Who / what triggers it	Loaded through <code>styleUrls</code> and automatically scoped to this component.
What it uses / calls	Global classes <code>.btn</code> and <code>.page-container</code> (extended or overridden here). The page does not use the blinking animation.
Size	17 lines

Lines 1-4 | Container width, search row and card grid

```
1 | .page-container.narrow { max-width: 920px; }
2 | .search-row { display: flex; justify-content: center; margin-bottom: 14px; }
3 | .search-row .search-input { width: min(520px, 100%); }
4 | .cards { display: flex; flex-wrap: wrap; justify-content: center; gap: 16px; }
```

- `.page-container.narrow` limits the container to 920 px.
- `.search-row` is a flex row that centres the search box; `.search-row .search-input` makes it 520 px wide but never wider than its parent (`min(520px, 100%)`).
- `.cards` is a flex container with wrapping, centred items and a 16 px gap, so cards sit side by side where there is room.

Lines 5-9 | The request card

```
5 | .request-card { display: flex; flex-direction: column; width: calc(50% - 8px); min-width: 300px; background: #fff; border-radius: 6px; box-shadow: 0 2px 8px rgba(0, 0, 0, 0.2); }
6 | .card-main { display: flex; gap: 12px; padding: 14px; flex: 1; }
7 | .avatar { width: 56px; height: 56px; flex: 0 0 56px; object-fit: cover; border-radius: 50%; border: 3px solid #1d8a8a; background: #fff; }
8 | .info h3 { margin: 0 0 8px; font-size: 17px; color: #333; }
9 | .info p { margin: 4px 0; font-size: 12.5px; color: #555; word-break: break-word; }
```

- `.request-card` is a column flex box taking half the width minus half the gap (`calc(50% - 8px)`), two cards per row), at least 300 px wide, white, with slightly rounded corners and a soft shadow.
- `.card-main` places the photo and text side by side with a 12 px gap and `flex: 1` so it fills the card above the footer.
- `.avatar` is a 56 px circle (it will not shrink, `flex: 0 0 56px`), cropped with `object-fit: cover` and with a teal ring.
- `.info h3` styles the driver name; `.info p` the small grey lines, with `word-break: break-word` so long locations wrap.

Lines 10-12 | Status colours

```
10 | .status { font-weight: 700; }
11 | .s-pending { color: #b4540a; } .s-approved { color: #1b8a3a; } .s-rejected { color: #c62828; }
12 | .s-tripend { color: #0a6fe8; } .s-closed { color: #666; }
```

`.status` makes the status text bold. The five rules after it match the class generated by the template expression `'s-' + status.split(' ').join(' ').toLowerCase()`:

- `.s-pending` orange-brown (`#b4540a`)
- `.s-approved` green (`#1b8a3a`)
- `.s-rejected` red (`#c62828`)
- `.s-tripend` blue (`#0a6fe8`), which is the class produced from `Trip End` after the space is removed
- `.s-closed` grey (`#666`)

Lines 13-14 | Card footer and disabled buttons

```
13 | .card-footer { display: flex; flex-wrap: wrap; gap: 6px; justify-content: center; padding: 10px; background: #f3f5f7; border-radius: 0 0 6px 6px; }
14 | .btn:disabled { opacity: 0.45; cursor: not-allowed; }
```

`.card-footer` is a wrapping, centred row of buttons on a light grey strip whose bottom corners are rounded to match the card. `.btn:disabled` adds `opacity: 0.45` and the `not-allowed` cursor on top of the global grey disabled look, so locked Edit, Delete, Trip End and Fetch Pay Amount buttons are visibly faded.

Lines 15-16 | Fare loading text

```
15 | .pay-loading { text-align: center; padding: 10px 0; }
16 | .pay-loading p { margin: 12px 0 0; color: #555; font-size: 13px; }
```

`.pay-loading` centres the spinner and message in the payment modal; its paragraph gets a top margin, grey colour and 13 px size.

Line 17 | Phone layout

```
17 | @media (max-width: 760px) { .request-card { width: 100%; } }
```

`@media (max-width: 760px)` is a **media query**: on screens narrower than 760 px each card becomes 100% wide instead of half, so cards stack in one column on phones.

Workflow, next: That finishes the "My Requests" page and the whole request lifecycle: the customer creates a request, the admin approves it, the customer ends the trip and sees the fare, and the admin closes it. The last button of the page, **Write a Review**, navigates (with `?driverId=...`) to the route `/customerpostfeedback`. The customer clicks 'Write a Review' and lands on `angularapp/src/app/components/customerpostfeedback/customerpostfeedback.component.ts`, the first file of the next part.

Step 105: CustomerpostfeedbackComponent - the "Submit Feedback" form logic [Saumya - Feedback & AI]

FRONTEND

`angularapp/src/app/components/customerpostfeedback/customerpostfeedback.component.ts`

What this file is	This is the TypeScript class behind the page where a customer writes feedback about a driver. It builds a reactive form (a form whose fields and rules are created in code and held in a <code>FormGroup</code> object, instead of being only described in the HTML) with three controls: <code>category</code> , <code>rating</code> and <code>feedbackText</code> . When the customer presses Submit it checks the rules, builds the JSON payload the backend expects and sends it with <code>FeedbackService</code> . The page is reached from the <code>writeReview()</code> method of the My Requests page (the "write a review" step after a trip), which navigates to <code>/customerpostfeedback?driverId=3</code> (the number is the id of the driver the trip was with).
Who / what triggers it	Angular's router. In <code>app-routing.module.ts</code> the path <code>customerpostfeedback</code> is mapped to this component and protected by <code>AuthGuard</code> with <code>data: { role: 'Customer' }</code> , so only a logged-in customer can open it. Angular creates the component, runs the constructor, then calls <code>ngOnInit</code> . After that, user actions (clicking a star, typing, pressing Submit or Back) call the methods below.
What it uses / calls	<code>FormBuilder</code> , <code>Validators</code> (Angular forms), <code>ActivatedRoute</code> (reads the URL query string), <code>Router</code> (navigation), <code>FeedbackService.sendFeedback</code> (HTTP), <code>AuthService.getUserId</code> (reads <code>userId</code> from <code>localStorage</code>), the constant <code>FEEDBACK_CATEGORIES</code> from <code>utils/constants.ts</code> and the helper <code>todayString</code> from <code>utils/format.ts</code> . The template is <code>customerpostfeedback.component.html</code> .
Size	91 lines

Lines 1-8 | Imports

```
1 | import { Component, OnInit } from '@angular/core';
2 | import { FormBuilder, FormGroup, Validators } from '@angular/forms';
3 | import { ActivatedRoute, Router } from '@angular/router';
4 | import { FeedbackService } from '../services/feedback.service';
5 | import { AuthService } from '../services/auth.service';
6 | import { Feedback } from '../models/feedback.model';
7 | import { FEEDBACK_CATEGORIES } from '../utils/constants';
8 | import { todayString } from '../utils/format';
```

Each import brings in something used later in the file:

- `Component`, `OnInit` (line 1): `Component` is the decorator that turns the class into a page piece; `OnInit` is the interface for the `ngOnInit` method that Angular calls once after creating the component.
- `FormBuilder`, `FormGroup`, `Validators` (line 2): `FormBuilder` is a helper to create forms in a short way, `FormGroup` is the type of the form object, `Validators` holds ready-made rules such as `required`.

- **ActivatedRoute, Router** (line 3): **ActivatedRoute** gives access to the current URL (we need its `?driverId=` part); **Router** lets the code jump to another page.
- **FeedbackService** (line 4): the service that makes the HTTP call to the backend.
- **AuthService** (line 5): used only to read the logged-in user's id.
- **Feedback** (line 6): the model class describing a feedback object; it is used for a type cast when sending.
- **FEEDBACK_CATEGORIES** (line 7): the list of category names shown in the dropdown (Driver Performance, Service Experience, Punctuality, Vehicle Condition, Other).
- **todayString** (line 8): returns today's date as `YYYY-MM-DD`.

Lines 10-14 | @Component decorator

```
10 | @Component({
11 |   selector: 'app-customerpostfeedback',
12 |   templateUrl: './customerpostfeedback.component.html',
13 |   styleUrls: ['./customerpostfeedback.component.css']
14 | })
```

A **decorator** is an annotation placed above a class that gives Angular information about it. **selector**: `'app-customerpostfeedback'` is the custom HTML tag name for this component (the router does not use the tag, it creates the component itself, but the name must exist). **templateUrl** points to the HTML file and **styleUrls** to the CSS file; the CSS in that file is applied only to this component (Angular's view encapsulation).

Lines 15-23 | Class and its fields

```
15 | export class CustomerpostfeedbackComponent implements OnInit {
16 |   feedbackForm: FormGroup;
17 |   categories: string[] = FEEDBACK_CATEGORIES;
18 |   stars: number[] = [1, 2, 3, 4, 5];
19 |   driverId?: number;
20 |
21 |   submitted: boolean = false;
22 |   formError: string = '';
23 |   showSuccessPopup: boolean = false;
```

`export class CustomerpostfeedbackComponent implements OnInit` declares the class and promises that it has an `ngOnInit` method.

- **feedbackForm: FormGroup** (16): the form object. It is only declared here and is filled in inside the constructor.
- **categories** (17): copy of the constant list, so the template can loop over it with `*ngFor` to produce the dropdown options.
- **stars: number[] = [1, 2, 3, 4, 5]** (18): the template loops over this array to draw five star buttons.
- **driverId?: number** (19): the optional `?` means it may stay undefined. It is filled in `ngOnInit` from the URL.
- **submitted** (21): turns true when Submit is pressed once, so that error messages for untouched fields can appear.
- **formError** (22): the general message shown above the Submit button ("All fields are required").
- **showSuccessPopup** (23): true while the "Thank you for your feedback!" box is visible.

Lines 25-37 | Constructor and the form definition

```
25 |   constructor(
26 |     private fb: FormBuilder,
27 |     private feedbackService: FeedbackService,
28 |     private authService: AuthService,
29 |     private route: ActivatedRoute,
30 |     private router: Router
31 |   ) {
32 |     this.feedbackForm = this.fb.group({
33 |       category: ['', [Validators.required]],
34 |       rating: [0, [Validators.required, Validators.min(1), Validators.max(5)]],
35 |       feedbackText: ['', [Validators.required, Validators.pattern(/\S/)]]
36 |     });
37 |   }
```

The constructor receives the five helpers through **dependency injection** (Angular sees the parameter types and hands over the single shared instance of each; **private** also turns each parameter into a class property such as `this.fb`).

Inside the body `this.fb.group({...})` creates the form. Each entry is `controlName: [initialValue, [validators]]`:

- **category: ['', [Validators.required]]** starts empty and must be chosen.
- **rating: [0, [Validators.required, Validators.min(1), Validators.max(5)]]** starts at 0. Because `min(1)` fails for 0, the form is invalid until a star is clicked. The value is later kept between 1 and 5 by the star buttons.
- **feedbackText: ['', [Validators.required, Validators.pattern(/\S/)]]**: `required` rejects an empty string and the regular expression `/\S/` means "at least one character that is not whitespace", so a text made only of spaces is rejected too.

A control is **invalid** when any of its validators fails, and the whole **FormGroup** is invalid when any control is invalid.

Lines 39-44 | ngOnInit - reading driverId from the URL

```
39 |   ngOnInit(): void {
40 |     const driverId = this.route.snapshot.queryParamMap?.get('driverId');
41 |     if (driverId) {
42 |       this.driverId = Number(driverId);
43 |     }
44 |   }
```

`ngOnInit` runs once after the component is created. `this.route.snapshot.queryParamMap?.get('driverId')` reads the query string of the address, for `/customerpostfeedback?driverId=3` it returns the text `'3'` (or `null` when the parameter is missing; the `?` protects against `queryParamMap` itself being undefined, which happens in unit tests with a fake route). Query parameters are always text, so `Number(driverId)` converts it before saving it in `this.driverId`. If the customer opened the page without a driver, `driverId` stays **undefined** and the feedback is saved with no driver.

Lines 46-48 | Getter f - shortcut to the controls

```
46 |   get f() {
47 |     return this.feedbackForm.controls;
48 |   }
```

`get f()` is a getter: the template can write `f['rating'].value` as if `f` were a property. It simply returns `this.feedbackForm.controls`, the object that holds the three controls. The star buttons use it to know the current rating.

Lines 50-53 | setRating - the star buttons

```
50 |   setRating(rating: number): void {
51 |     this.feedbackForm.patchValue({ rating });
52 |     this.feedbackForm.get('rating')!.markAsTouched();
53 |   }
```

The star buttons are not form fields by themselves, so each click calls `setRating(star)` with 1 to 5. `patchValue({ rating })` updates only the `rating` control of the form (the short property form `{ rating }` means `{ rating: rating }`). Then `markAsTouched()` tells Angular the user has interacted with this control, which is what lets the error message "Rating is required" appear/disappear correctly. The `!` after `get('rating')` tells TypeScript the control is certainly not null.

Lines 55-58 | showError - when to display a red message

```
55 |   showError(field: string): boolean {
56 |     const control = this.feedbackForm.get(field)!;
57 |     return control.invalid && (control.touched || this.submitted);
58 |   }
```

The template calls `showError('category')` and so on. It returns true when the control is invalid **and** either the user has already touched it or Submit has been pressed (`this.submitted`). That way the page does not show red errors on a fresh, untouched form.

Lines 60-66 | onSubmit - validation step

```
60 |   onSubmit(): void {
61 |     this.submitted = true;
62 |     this.formError = '';
63 |     if (this.feedbackForm.invalid) {
64 |       this.formError = 'All fields are required';
65 |       return;
66 |     }
```

Runs when the form is submitted (Submit button or Enter key, via `ngSubmit` in the template). It first sets `submitted = true` and clears any older `formError`. If `this.feedbackForm.invalid` it sets the message `'All fields are required'` and returns immediately, so nothing is sent to the server.

Lines 67-77 | onSubmit - building the payload

```
67 |     const userId = this.authService.getUserId();
68 |     const value = this.feedbackForm.value;
69 |     const feedback = {
70 |       feedbackText: String(value.feedbackText).trim(),
71 |       category: value.category,
72 |       rating: Number(value.rating),
73 |       date: todayString(),
74 |       userId,
75 |       user: { userId },
76 |       driver: this.driverId !== undefined ? { driverId: this.driverId } : undefined
77 |     };
```

`this.authService.getUserId()` reads the logged-in user's id from local storage (saved at login). `this.feedbackForm.value` is a plain object with the three field values.

The object `feedback` is the JSON body for the backend:

- `feedbackText`: the text with leading/trailing spaces removed by `trim()`.
- `category`: the chosen category.
- `rating`: converted with `Number(...)` (select/inputs can give strings).
- `date`: `todayString()`: today's date as `YYYY-MM-DD`; Spring converts it into a `LocalDate`.
- `userId` and `user`: `{ userId }`: the id is sent twice, once flat and once nested. The Java entity `Feedback` only has a nested `user` field, so the nested form is the one that really matters: Spring (Jackson) builds a `User` object containing just that id, and Hibernate uses it as the foreign key. The flat `userId` is ignored by the backend (Spring Boot does not fail on unknown JSON properties); it is kept because the Angular `Feedback` model declares `userId`.
- `driver`: `{ driverId }` when a driver id was in the URL, otherwise `undefined` (JSON leaves undefined properties out). The Java column `driverId` is `nullable = true`, so feedback without a driver can be saved.

Lines 78-81 | Sending the request

```
78 |     this.feedbackService.sendFeedback(feedback as unknown as Feedback).subscribe({
79 |       next: () => (this.showSuccessPopup = true)
80 |     });
81 | }
```

`sendFeedback(...)` returns an **Observable** (a stream that does nothing until someone subscribes). `as unknown as Feedback` is a double cast that silences TypeScript because the object literal does not exactly match the `Feedback` class. `.subscribe({ next: ... })` actually sends the HTTP POST to `http://localhost:8080/api/feedback` and, when the server answers with success (201 Created), runs `next`, which sets `showSuccessPopup = true`.

There is no `error` handler here. If the server refuses (for example 403 or the 409 that the controller returns on any failure) nothing visible happens; for status 0 or 5xx the global `HttpErrorInterceptor` sends the user to the Error page instead.

Lines 83-86 | closeSuccessPopup

```
83 |     closeSuccessPopup(): void {
84 |       this.showSuccessPopup = false;
85 |       this.router.navigate(['/customerviewfeedback']);
86 |     }
```

Called by the Ok button of the popup. It hides the popup and navigates to `/customerviewfeedback`, the page that lists the customer's own feedback, so the new item can be seen immediately.

Lines 88-91 | goBack

```
88 |     goBack(): void {
89 |       this.router.navigate(['/customerviewrequested']);
90 |     }
91 | }
```

Called by the Back button: navigates to `/customerviewrequested` (My Requests), where the customer came from. The last line closes the class.

Workflow, next: The template `customerpostfeedback.component.html` is what the customer actually sees and what calls these methods, so we read it next. Remember the backend part of this page: when Submit succeeds the request goes to `POST /api/feedback`, answered by `FeedbackController.createFeedback`; we trace that in detail inside the HTML entry's flow.

Step 106: customerpostfeedback.component.html - the feedback form page [Saumya - Feedback & AI]

`angularapp/src/app/components/customerpostfeedback/customerpostfeedback.component.html`

FRONTEND

What this file is	The visible form: a category dropdown, five clickable stars, a text area, an error line, a Submit button, and a hidden "thank you" popup. Everything with <code>id="..."</code> is there so automated tests (and the Jasmine spec files) can find the element.
Who / what triggers it	Angular renders it when the router creates <code>CustomerpostfeedbackComponent</code> . Its bindings read the component's fields and its events call the component's methods.
What it uses / calls	The component class (<code>feedbackForm</code> , <code>categories</code> , <code>stars</code> , <code>f</code> , <code>showError</code> , <code>setRating</code> , <code>onSubmit</code> , <code>goBack</code> , <code>closeSuccessPopup</code> , <code>formError</code> , <code>showSuccessPopup</code>) and the global classes <code>form-card</code> , <code>form-control</code> , <code>btn</code> , <code>modal-backdrop</code> , <code>modal-box</code> from <code>src/styles.css</code> .

Lines 1-3 | The form tag, Back button and heading

```
1 | <form class="form-card" id="feedbackForm" [formGroup]="feedbackForm" (ngSubmit)="onSubmit()" novalidate>
2 |   <button type="button" class="back-btn" id="backButton" (click)="goBack()">Back</button>
3 |   <h2 class="form-title" id="formHeading">Submit Feedback</h2>
```

```
<form ... [formGroup]="feedbackForm" (ngSubmit)="onSubmit()" novalidate>:
```

- `[formGroup]="feedbackForm"` connects this HTML form to the `FormGroup` object of the class (square brackets = "bind this attribute to a TypeScript value").
- `(ngSubmit)="onSubmit()"` (round brackets = "listen to an event") calls `onSubmit()` when the form is submitted.
- `novalidate` switches off the browser's own pop-up validation so only Angular's messages are used.
- `class="form-card"` is a global card style; `id="feedbackForm"` is a test hook.

The Back button has `type="button"` so that it does not submit the form, and `(click)="goBack()"` navigates back to My Requests. The `<h2>` is the page title.

Lines 5-10 | Category dropdown

```
5 | <label class="form-label" for="category">Category<span class="req">*</span></label>
6 | <select id="category" class="form-control" formControlName="category">
7 |   <option value="">Select a category</option>
8 |   <option *ngFor="let category of categories" [value]="category">{{ category }}</option>
9 | </select>
10 | <div class="error-text" id="categoryError" *ngIf="showError('category')">*Category is required</div>
```

The `<label for="category">` is linked to the select with the same id; the `<span class="req">*</span>` draws the red star meaning "required".

`<select formControlName="category">` binds this dropdown to the control named `category` inside the form group. The first option has the empty value `"` and the text "Select a category", which matches the control's starting value and fails `required`. `*ngFor="let category of categories"` is a **structural directive** that repeats the `<option>` element once per item of the array; `[value]="category"` sets each option's value and `{{ category }}` (interpolation, prints a value as text) shows its name.

Line 10: `*ngIf="showError('category')"` adds the red message `"*Category is required"` to the page only while that method returns true.

Lines 12-16 | Star rating buttons

```
12 | <label class="form-label">Rating<span class="req">*</span></label>
13 | <div class="stars" id="rating">
14 |   <button type="button" class="star" *ngFor="let star of stars" [id]='star-' + star" [class.on]="star <= f['rating']
15 |     ].value" (click)="setRating(star)" [attr.aria-label]="star + ' star'">&#9733;</button>
16 | </div>
16 | <div class="error-text" id="ratingError" *ngIf="showError('rating')">*Rating is required</div>
```

There is no input control named `rating` in the HTML; the stars are ordinary buttons that change the control through `setRating`.

`*ngFor="let star of stars"` creates five buttons (1 to 5). `[id]='star-' + star"` gives them ids `star-1 ... star-5`. `[class.on]="star <= f['rating'].value"` adds the CSS class `on` when the star's number is not greater than the current rating, so for rating 3 the first three stars turn yellow. `(click)="setRating(star)"` stores the clicked number in the form. `[attr.aria-label]` sets an accessibility label such as "4 star" for screen readers (`attr.` is needed because `aria-label` is an HTML attribute, not a DOM property). `★` is the HTML code for the solid star character. `type="button"` prevents form submission.

Line 16 shows `"*Rating is required"` via the `showError('rating')` check.

Lines 18-20 | Feedback text area

```
18 | <label class="form-label" for="feedbackText">Feedback<span class="req">*</span></label>
19 | <textarea id="feedbackText" class="form-control" rows="4" formControlName="feedbackText" placeholder="Tell us about
20 |   your trip..."></textarea>
20 | <div class="error-text" id="feedbackTextError" *ngIf="showError('feedbackText')">*Feedback is required</div>
```

`<textarea formControlName="feedbackText" rows="4" placeholder="...">` is bound to the `feedbackText` control; `rows="4"` sets its height, the placeholder is the grey hint text. Line 20 is the red message `"*Feedback is required"`, shown when the control is invalid and touched (or Submit was pressed). Note that a text containing only spaces also triggers it because of the `\S` pattern validator.

Line 22 | General form error

```
22 | <div class="form-error" id="formError" *ngIf="formError">{{ formError }}</div>
```

`*ngIf="formError"` shows the text "All fields are required" (set by `onSubmit` when the form was invalid). `{{ formError }}` prints it.

Lines 24-27 | Submit button and end of form

```
24 | <div class="submit-row">
25 |   <button type="submit" class="btn btn-primary" id="submitButton">Submit</button>
26 | </div>
27 | </form>
```

The button has `type="submit"`, which makes the browser fire the form's submit event and therefore `(ngSubmit)`. `btn btn-primary` are global button styles. Line 27 closes the form.

Lines 29-36 | Success popup

```
29 | <div class="modal-backdrop" *ngIf="showSuccessPopup">
30 |   <div class="modal-box success-box" id="successPopup">
31 |     <p>Thank you for your feedback!</p>
32 |     <div class="modal-actions">
33 |       <button type="button" class="btn btn-primary" id="successOk" (click)="closeSuccessPopup()">Ok</button>
34 |     </div>
35 |   </div>
36 | </div>
```

A modal is a box shown over a dimmed background. The outer `div.modal-backdrop` is the dark full-screen layer and exists only when `showSuccessPopup` is true (`*ngIf`). Inside, `modal-box success-box` is the white message box with the text "Thank you for your feedback!" and an Ok button whose `(click)="closeSuccessPopup()"` closes it and navigates to My Feedback.

What happens after the customer presses Submit (backend trace).

1. `onSubmit()` validates and builds the JSON, then `FeedbackService.sendFeedback` posts it to `POST http://localhost:8080/api/feedback` with the header `Authorization: Bearer <token>`.
2. On the server, the JWT filter reads the token and `SecurityConfig` checks the rule `.requestMatchers(HttpMethod.POST, "/api/feedback").hasRole("CUSTOMER")`; an admin token would get 403.
3. `FeedbackController.createFeedback(@RequestBody Feedback feedback)` receives the JSON converted into a `Feedback` entity (with a nested `User` holding only the id and a nested `Driver` holding only the id). It calls `feedbackService.createFeedback(feedback)`; any exception is turned into a 409 Conflict with an empty body.
4. `FeedbackServiceImpl.createFeedback` sets the date to today if it is null, then calls `aiService.applyAnalysis(feedback)` inside a try/catch so that feedback is still stored even if the analysis crashes.
5. `AiService.applyAnalysis` calls `analyzeFeedback(text, category, rating)`. **The sentiment, sentiment score and tags are computed on the server at this moment, not in Angular.** If a `GEMINI_API_KEY` is configured, `analyzeWithGemini` sends the text, category and rating to Google Gemini and asks for JSON like `{"sentiment":"Positive","sentimentScore":0.9,"tags":["punctuality"]}`. If there is no key, or Gemini fails or answers something unusable, `analyzeLocally` runs a local word-list method: it counts words found in a `POSITIVE_WORDS` set and a `NEGATIVE_WORDS` set, turns that into a text score (0.5 plus 0.15 per net positive word, kept between 0 and 1), averages it with the star rating converted to 0..1, and calls the result Positive (score 0.6 or more), Negative (0.4 or less) or Neutral. Tags come from a keyword table, for example "polite" gives the tag `behaviour`.
6. Example, local method: text "The driver was polite and punctual, very safe", rating 5. Words polite, punctual, safe are positive, so the text score is $0.5 + 0.45 = 0.95$; the rating gives 1.0; the average is about 0.98. Result: sentiment **Positive**, score 0.98, tags `punctuality,safety,behaviour`. A text like "rude and late, car was dirty" with rating 1 gives a score near 0.03 and a red **Negative** badge with tags `punctuality,cleanliness,behaviour,vehicle condition`.
7. `applyAnalysis` writes those three values into the entity with `setSentiment`, `setSentimentScore`, `setAiTags`; back in the service `feedbackRepo.save(feedback)` inserts the row into the H2 `feedback` table, and the saved entity returns to the controller, which answers 201 Created.
8. Back in the browser the `next` callback shows the success popup; pressing Ok navigates to `/customerreviewfeedback`.

The stars' yellow colour and hover effect come from the small CSS file, so we read `customerpostfeedback.component.css` next.

Step 107: customerpostfeedback.component.css - star styles [Saumya - Feedback & AI]

FRONTEND

angularapp/src/app/components/customerpostfeedback/customerpostfeedback.component.css

What this file is	Four CSS rules that style only the star-rating row. Everything else on the page (form card, inputs, buttons, popup) comes from the global <code>src/styles.css</code> .
Who / what triggers it	Angular loads it because of <code>styleUrls</code> in the component and scopes the rules to this component's elements.
What it uses / calls	The classes <code>stars</code> , <code>star</code> and <code>on</code> written in <code>customerpostfeedback.component.html</code> .
Size	4 lines

Lines 1-2 | .stars row and .star button

```
1 | .stars { display: flex; gap: 4px; margin-bottom: 4px; }
2 | .star { background: none; border: none; padding: 0 2px; font-size: 28px; line-height: 1; color: #c9ced3; cursor: pointer; }
```

`.stars` is the container of the five buttons: `display: flex` puts them side by side, `gap: 4px` leaves a small space between them and `margin-bottom: 4px` separates the row from the error text below.

`.star` turns a normal button into a plain star: `background: none`; `border: none` removes the grey button box, `padding: 0 2px` keeps a tiny click area, `font-size: 28px` makes the star big, `line-height: 1` removes extra height, `color: #c9ced3` is the default light-grey (empty) colour and `cursor: pointer` shows the hand cursor.

Lines 3-4 | Selected and hover colours

```
3 | .star.on { color: #f5b301; }
4 | .star:hover { color: #e0a000; }
```

`.star.on` (a star that also has the class `on`, which the template adds when the star number is less than or equal to the rating) is coloured gold `#f5b301`. `.star:hover` makes a star darker gold `#e0a000` when the mouse is over it. Because `:hover` is declared after `.star.on` with the same specificity, hovering also changes already-selected stars slightly.

Workflow, next: The form page is complete. After pressing Ok in the popup the customer lands on `/customerviewfeedback`, which shows the saved feedback with its AI badge and tags. Next file: `customerviewfeedback.component.ts`.

Step 108: CustomerviewfeedbackComponent - "My Feedback" list logic [Saumya - Feedback & AI]

FRONTEND

angularapp/src/app/components/customerviewfeedback/customerviewfeedback.component.ts

What this file is	The class behind the page that lists every feedback the logged-in customer has written. It loads the list when the page opens, turns the numeric rating into star characters, splits the AI tags into pieces, picks the CSS class for the Positive/Neutral/Negative badge, opens a modal with the driver's details, and deletes a feedback after confirmation.
Who / what triggers it	The router, for the path <code>customerviewfeedback</code> (customer-only through <code>AuthGuard</code>). It is also the page the customer is sent to after the success popup of the feedback form, and it is reachable from the customer navbar.
What it uses / calls	<code>FeedbackService.getAllFeedbacksByUserId</code> and <code>deleteFeedback</code> (HTTP), <code>AuthService.getUserId</code> , the models <code>Feedback</code> and <code>Driver</code> , and the helpers <code>toDisplayDate</code> (<code>format.ts</code>) and <code>getDriverImage</code> , <code>useDefaultAvatar</code> (<code>driver-image.ts</code>).
Size	94 lines

Lines 1-7 | Imports

```
1 | import { Component, OnInit } from '@angular/core';
2 | import { FeedbackService } from '../services/feedback.service';
3 | import { AuthService } from '../services/auth.service';
4 | import { Feedback } from '../models/feedback.model';
5 | import { Driver } from '../models/driver.model';
6 | import { toDisplayDate } from '../utils/format';
7 | import { getDriverImage, useDefaultAvatar } from '../utils/driver-image';
```

`Component`, `OnInit` for the decorator and lifecycle method; `FeedbackService` and `AuthService` for the HTTP call and the user id; `Feedback` and `Driver` are model types used for the fields; `toDisplayDate` converts `2026-10-06` into `06/10/2026`; `getDriverImage` returns the picture URL for a driver (or a default one) and `useDefaultAvatar` is the handler that swaps in the default picture if an image fails to load.

Lines 9-15 | Decorator, class and the feedback list

```
9 | @Component({
10 |   selector: 'app-customerviewfeedback',
11 |   templateUrl: './customerviewfeedback.component.html',
12 |   styleUrls: ['./customerviewfeedback.component.css']
13 | })
14 | export class CustomerviewfeedbackComponent implements OnInit {
15 |   feedbacks: Feedback[] = [];
```

`@Component` gives the selector `app-customerviewfeedback`, the HTML file and the CSS file. The class has one main field, `feedbacks: Feedback[] = []`, the array that the template loops over. It starts empty, so the page first shows "No Data Found" until the server answers.

Lines 17-21 | Modal state fields

```
17 |   showDriverModal: boolean = false;
18 |   selectedDriver: Driver | null = null;
19 |
20 |   showDeleteModal: boolean = false;
21 |   feedbackToDelete: Feedback | null = null;
```

Two modals exist on this page, each controlled by a boolean plus the object it shows. `showDriverModal` and `selectedDriver` control the driver details box; `showDeleteModal` and `feedbackToDelete` control the "Are you sure?" box. `Driver | null` is a union type: the value is either a driver or null.

Lines 23-25 | Helper functions exposed to the template

```
23 |   toDisplayDate = toDisplayDate;
24 |   getDriverImage = getDriverImage;
25 |   useDefaultAvatar = useDefaultAvatar;
```

An Angular template can only call things that are members of its component. These three lines copy the imported functions into class properties (`toDisplayDate = toDisplayDate;`) so that the HTML can write `toDisplayDate(feedback.date)` and `getDriverImage(...)`.

Lines 27-31 | Constructor and ngOnInit

```
27 |   constructor(private feedbackService: FeedbackService, private authService: AuthService) {}
28 |
29 |   ngOnInit(): void {
30 |     this.loadFeedbacks();
31 |   }
```

The constructor injects the two services. `ngOnInit` runs once when the page is created and simply calls `loadFeedbacks()`, so loading starts automatically.

Lines 33-43 | loadFeedbacks - GET /api/feedback/user/{id}

```
33 |   loadFeedbacks(): void {
34 |     const userId = this.authService.getUserId();
35 |     if (userId === null) {
36 |       return;
37 |     }
38 |     this.feedbackService.getAllFeedbacksByUserId(userId).subscribe({
39 |       // The API answers 404 / 204 when the customer has not posted any feedback yet.
40 |       next: (feedbacks) => (this.feedbacks = feedbacks || []),
41 |       error: () => (this.feedbacks = [])
42 |     });
43 |   }
```

First `getUserId()` returns the number saved at login (or null if nothing is stored); in that case the method stops. Otherwise it subscribes to `getAllFeedbacksByUserId(userId)`, which sends `GET /api/feedback/user/{userId}` with the bearer token. On the server `SecurityConfig` requires the CUSTOMER role and `FeedbackController.viewFeedbacksByUser` runs `feedbackService.getFeedbacksByUserId(userId)`, which calls the Spring Data method `FeedbackRepo.findByUserUserId` (Spring writes the SQL "where user.userId = ?" from the method name).

- **next**: the list arrives, stored in `this.feedbacks` (`feedbacks || []` guards against an empty body).
- **error**: the controller answers **404 Not Found when the customer has no feedback yet**; that arrives here as an error, and the code treats it as "empty list", so the page shows "No Data Found" instead of crashing. The error interceptor leaves 4xx errors alone, so it does not redirect.

Lines 45-48 | starsOf - number to stars

```
45 | starsOf(rating: number): string {
46 |     const value = Math.max(0, Math.min(5, Math.round(Number(rating) || 0)));
47 |     return '★'.repeat(value) + '☆'.repeat(5 - value);
48 | }
```

Turns a rating such as 4 into the text `★★★★☆`. `Number(rating) || 0` converts to a number (and uses 0 for bad values), `Math.round` removes decimals, `Math.min(5, ...)` and `Math.max(0, ...)` clamp the value between 0 and 5. Then `'★'.repeat(value)` writes that many filled stars and `'☆'.repeat(5 - value)` fills the rest with empty ones.

Lines 50-55 | tagsOf - AI tags to a list

```
50 | tagsOf(feedback: Feedback): string[] {
51 |     return (feedback.aiTags || '')
52 |         .split(',')
53 |         .map((tag) => tag.trim())
54 |         .filter((tag) => !!tag);
55 | }
```

The server stores the tags as one comma-separated string such as `punctuality,safety,behaviour` (field `aiTags`). This method makes it a list for the template: `(feedback.aiTags || '')` avoids an error when it is missing, `.split(',')` cuts at the commas, `.map(tag => tag.trim())` removes spaces and `.filter(tag => !!tag)` drops empty pieces (`!!` converts to true/false). Each tag will be displayed as a small purple chip.

Lines 57-59 | sentimentClass - badge colour name

```
57 | sentimentClass(sentiment?: string): string {
58 |     return (sentiment || 'none').toLowerCase();
59 | }
```

Returns the sentiment in lower case (**Positive** becomes **positive**) or **'none'** when absent. The template puts it in `[ngClass]`, so the CSS rules `.badge.positive`, `.badge.neutral` and `.badge.negative` choose green, grey or red.

Lines 61-69 | Driver info modal methods

```
61 | viewDriverInfo(feedback: Feedback): void {
62 |     this.selectedDriver = feedback.driver || null;
63 |     this.showDriverModal = true;
64 | }
65 |
66 | closeDriverModal(): void {
67 |     this.showDriverModal = false;
68 |     this.selectedDriver = null;
69 | }
```

`viewDriverInfo(feedback)` stores the driver that is already nested inside the feedback object (the backend returns the full `driver` because the `@ManyToOne` relation is loaded with the feedback) in `selectedDriver`, and sets `showDriverModal = true`. If the feedback has no driver, `selectedDriver` is null and the modal explains that. `closeDriverModal()` hides it and clears the selection. No extra HTTP call is needed.

Lines 71-79 | Delete confirmation methods

```
71 | askDelete(feedback: Feedback): void {
72 |     this.feedbackToDelete = feedback;
73 |     this.showDeleteModal = true;
74 | }
75 |
76 | cancelDelete(): void {
77 |     this.showDeleteModal = false;
78 |     this.feedbackToDelete = null;
79 | }
```

`askDelete(feedback)` remembers which feedback the customer clicked and opens the confirmation box. `cancelDelete()` closes the box and forgets the feedback; nothing is deleted.

Lines 81-94 | confirmDelete - DELETE /api/feedback/{id}

```
81 |   confirmDelete(): void {
82 |     const id = this.feedbackToDelete?.feedbackId;
83 |     if (id === undefined) {
84 |       return;
85 |     }
86 |     this.feedbackService.deleteFeedback(id).subscribe({
87 |       next: () => {
88 |         this.feedbacks = this.feedbacks.filter((feedback) => feedback.feedbackId !== id);
89 |         this.cancelDelete();
90 |       },
91 |       error: () => this.cancelDelete()
92 |     });
93 |   }
94 | }
```

`this.feedbackToDelete?.feedbackId` reads the id (the `?` yields undefined when nothing is selected, and then the method returns). `deleteFeedback(id)` sends **DELETE** `/api/feedback/{id}`; `SecurityConfig` allows it for `CUSTOMER` and `FeedbackController.deleteFeedback` calls `FeedbackServiceImpl.deleteFeedback`, which finds the row with `findById`, deletes it with `feedbackRepo.delete(...)` and returns it (or null, which becomes 404).

On success the component removes the item from the local array with `filter(...)` (so the card disappears without reloading) and closes the box. On error it just closes the box. Line 94 closes the class.

Workflow, next: The template `customerviewfeedback.component.html` shows the list and calls these methods. Next file: `customerviewfeedback.component.html`.

Step 109: customerviewfeedback.component.html - the My Feedback cards [Saumya - Feedback & AI]

FRONTEND

`angularapp/src/app/components/customerviewfeedback/customerviewfeedback.component.html`

What this file is	Shows one card per feedback: category, date, stars, text, the AI sentiment badge with tags, and two buttons (View Driver Info and Delete). It also contains two hidden modals, one for the driver details and one for delete confirmation.
Who / what triggers it	Rendered by Angular when <code>CustomerviewfeedbackComponent</code> is created; it re-renders automatically whenever <code>feedbacks</code> or a modal flag changes.
What it uses / calls	The component's fields and methods (<code>feedbacks</code> , <code>starsOf</code> , <code>tagsOf</code> , <code>sentimentClass</code> , <code>toDisplayDate</code> , <code>getDriverImage</code> , <code>useDefaultAvatar</code> , <code>viewDriverInfo</code> , <code>askDelete</code> , <code>confirmDelete</code> , <code>cancelDelete</code> , <code>closeDriverModal</code>), global classes from <code>styles.css</code> (<code>page-container</code> , <code>page-card</code> , <code>btn</code> , <code>modal-box</code> , ...) and the local CSS file.
Size	56 lines

Lines 1-4 | Page container, card and heading

```
1 | <div class="page-container narrow">
2 |   <div class="page-card" id="myFeedbackCard">
3 |     <h2 class="page-title steel" id="myFeedbackHeading">My Feedback</h2>
4 |     <div class="title-line"></div>
```

`page-container narrow` and `page-card` are layout classes (the `narrow` modifier is styled in the local CSS to limit the width to 920px). The `<h2 class="page-title steel">` is the heading "My Feedback" and `<div class="title-line">` draws a thin decorative line under it.

Line 6 | Empty message

```
6 |   <div class="empty-text" id="noDataMessage" *ngIf="feedbacks.length === 0">No Data Found</div>
```

`*ngIf="feedbacks.length === 0"` shows "No Data Found" only when the list is empty, which includes the case of the 404 answer.

Lines 8-9 | The list of cards

```
8 |   <div class="cards" *ngIf="feedbacks.length > 0">
9 |     <div class="feedback-card" *ngFor="let feedback of feedbacks" [attr.id]='feedback-' + feedback.feedbackId">
```

The container `div.cards` exists only when there is at least one feedback (`*ngIf="feedbacks.length > 0"`). Line 9 uses `*ngFor="let feedback of feedbacks"` to create one `feedback-card` per item, and `[attr.id]='feedback-' + feedback.feedbackId` gives each card a unique id like `feedback-12`.

Lines 10-15 | Card header, stars and text

```
10 |         <div class="card-head">
11 |             <span class="category">{{ feedback.category }}</span>
12 |             <span class="date">{{ toDisplayDate(feedback.date) }}</span>
13 |         </div>
14 |         <div class="rating" [attr.aria-label]="feedback.rating + ' out of 5'">{{ starsOf(feedback.rating) }}</div>
15 |         <p class="text">{{ feedback.feedbackText }}</p>
```

`{{ feedback.category }}` and `{{ toDisplayDate(feedback.date) }}` print the category and the date as DD/MM/YYYY. The rating line prints the star characters from `starsOf(feedback.rating)` and `[attr.aria-label]` gives screen readers a sentence like "4 out of 5". `<p class="text">` prints the written feedback.

Lines 16-19 | AI row: sentiment badge and tags

```
16 |         <div class="ai-row" *ngIf="feedback.sentiment">
17 |             <span class="badge" [ngClass]="sentimentClass(feedback.sentiment)">{{ feedback.sentiment }}</span>
18 |             <span class="tag" *ngFor="let tag of tagsOf(feedback)">{{ tag }}</span>
19 |         </div>
```

This row only appears when the feedback has a sentiment (`*ngIf="feedback.sentiment"`), which is true for every feedback created after the AI feature was added; the sentiment was calculated on the server when the feedback was saved. `<span class="badge" [ngClass]="sentimentClass(feedback.sentiment)">` combines the fixed class `badge` with a dynamic class (`positive`, `neutral` or `negative`) so the pill is green, grey or red, and prints the word. The second `span` repeats with `*ngFor` for every tag returned by `tagsOf(feedback)`, so a Positive feedback may show `Positive` then chips such as `punctuality`, `safety`.

Lines 20-27 | Card footer buttons and closing tags

```
20 |         <div class="card-footer">
21 |             <button type="button" class="btn btn-primary btn-pill" [id]="viewDriver-' + feedback.feedbackId" (click)="viewDriverInfo(feedback)">View Driver Info</button>
22 |             <button type="button" class="btn btn-danger btn-pill" [id]="delete-' + feedback.feedbackId" (click)="askDelete(feedback)">Delete</button>
23 |         </div>
24 |     </div>
25 | </div>
26 | </div>
27 | </div>
```

Two pill-shaped buttons with `type="button"`: View Driver Info calls `viewDriverInfo(feedback)` and Delete calls `askDelete(feedback)`. `[id]` binds unique ids (`viewDriver-5`, `delete-5`). Lines 24-27 close the card, the list, the page card and the container.

Lines 29-33 | Driver modal frame

```
29 | <!-- Driver info -->
30 | <div class="modal-backdrop" *ngIf="showDriverModal">
31 |     <div class="modal-box detail-box" id="driverModal">
32 |         <button type="button" class="modal-close" id="closeDriverModal" (click)="closeDriverModal()">X</button>
33 |         <h3>Driver Details</h3>
```

The HTML comment on line 29 names the section. The backdrop appears only while `showDriverModal` is true. The X button (`id="closeDriverModal"`) calls `closeDriverModal()`, and the heading says "Driver Details".

Lines 34-45 | Driver modal content

```
34 |         <ng-container *ngIf="selectedDriver; else noDriver">
35 |             <img class="modal-avatar" [src]="getDriverImage(selectedDriver.image)" (error)="useDefaultAvatar($event)" alt="Driver" />
36 |             <p><strong>Driver Name:</strong> {{ selectedDriver.driverName }}</p>
37 |             <p><strong>License Number:</strong> {{ selectedDriver.licenseNumber }}</p>
38 |             <p><strong>Experience Years:</strong> {{ selectedDriver.experienceYears }}</p>
39 |             <p><strong>Contact Number:</strong> {{ selectedDriver.contactNumber }}</p>
40 |             <p><strong>Vehicle Type:</strong> {{ selectedDriver.vehicleType }}</p>
41 |             <p><strong>Hourly Rate:</strong> ₹{{ selectedDriver.hourlyRate }}</p>
42 |         </ng-container>
43 |         <ng-template #noDriver><p>No driver is linked to this feedback.</p></ng-template>
44 |     </div>
45 | </div>
```

`<ng-container *ngIf="selectedDriver; else noDriver">` is an invisible grouping element: it shows its content when a driver exists, otherwise it shows the template named `noDriver`. Inside, the `<img>` uses `[src]="getDriverImage(selectedDriver.image)"` and `(error)="useDefaultAvatar($event)"` (the event runs if the picture cannot be loaded). The `<p>` lines print name, licence number, experience years, contact number, vehicle type and hourly rate; `₹` is the Indian rupee sign. `<ng-template #noDriver>` defines the fallback text "No driver is linked to this feedback."; `#noDriver` is a **template reference name** that `else noDriver` points to. The next two lines close the box and the backdrop.

Lines 47-56 | Delete confirmation modal

```
47 | <!-- Delete confirmation -->
48 | <div class="modal-backdrop" *ngIf="showDeleteModal">
49 |   <div class="modal-box" id="deleteModal">
50 |     <p>Are you sure you want to delete this feedback?</p>
51 |     <div class="modal-actions">
52 |       <button type="button" class="btn btn-danger" id="confirmDelete" (click)="confirmDelete()">Yes, Delete</button>
53 |       <button type="button" class="btn btn-secondary" id="cancelDelete" (click)="cancelDelete()">Cancel</button>
54 |     </div>
55 |   </div>
56 | </div>
```

Shown while `showDeleteModal` is true. The text asks "Are you sure you want to delete this feedback?". "Yes, Delete" (`id="confirmDelete"`) calls `confirmDelete()`, which sends the DELETE request; "Cancel" calls `cancelDelete()`.

Backend trace for this page. When the page opens it calls **GET** `/api/feedback/user/{userId}` (404 means no feedback yet), answered by `FeedbackController.viewFeedbacksByUser`, which uses `FeedbackServiceImpl.getFeedbacksByUserId` and `FeedbackRepo.findByUserUserId`. Pressing Yes, Delete calls **DELETE** `/api/feedback/{id}`, answered by `FeedbackController.deleteFeedback` and `FeedbackServiceImpl.deleteFeedback`. The AI fields on each card were produced earlier by `AIService.applyAnalysis` when the feedback was created. Now the look of the cards: `customerviewfeedback.component.css`.

Step 110: customerviewfeedback.component.css - card layout and badges [Saumya - Feedback & AI]

angularapp/src/app/components/customerviewfeedback/customerviewfeedback.component.css

FRONTEND

What this file is	Styles the feedback cards, the footer bar, the stars line and the sentiment badge and tag chips. General elements such as buttons and modals come from <code>styles.css</code> .
Who / what triggers it	Loaded with the component through <code>styleUrls</code> .
What it uses / calls	The classes in <code>customerviewfeedback.component.html</code> .
Size	13 lines

Lines 1-2 | Page width and card container

```
1 | .page-container.narrow { max-width: 920px; }
2 | .cards { display: flex; flex-wrap: wrap; gap: 16px; }
```

`.page-container.narrow` (an element having both classes) gets `max-width: 920px`, so the list does not stretch too wide. `.cards` is a flex container with `flex-wrap: wrap` (cards move to the next line when there is no room) and `gap: 16px` between them.

Line 3 | Feedback card

```
3 | .feedback-card { display: flex; flex-direction: column; width: calc(50% - 8px); min-width: 280px; padding: 14px 14px 0
   | ; background: #fff; border-radius: 6px; box-shadow: 0 2px 8px rgba(0, 0, 0, 0.2); }
```

Each card is a vertical flex box (`flex-direction: column`) so the footer can stay at the bottom. `width: calc(50% - 8px)` makes two cards per row (half the width minus half the gap), `min-width: 280px` stops them from getting too small, `padding: 14px 14px 0` pads the top and sides but not the bottom (the footer bar goes to the very bottom), and `background`, `border-radius` and `box-shadow` give a white rounded card with a soft shadow.

Lines 4-7 | Header, category, stars and text

```
4 | .card-head { display: flex; justify-content: space-between; align-items: center; font-size: 12px; color: #666; }
5 | .category { font-weight: 700; color: #1d5f8a; }
6 | .rating { margin: 6px 0; font-size: 18px; letter-spacing: 2px; color: #f5b301; }
7 | .text { flex: 1; margin: 0 0 10px; font-size: 13.5px; color: #333; word-break: break-word; }
```

`.card-head` puts category and date on one line, one at each end (`justify-content: space-between`), small and grey. `.category` is bold blue. `.rating` has margins, an 18px gold colour (`#f5b301`) and `letter-spacing: 2px` to spread the stars. `.text` has `flex: 1`, so it takes all remaining height and keeps footers aligned between cards, plus `word-break: break-word` so long words wrap instead of overflowing.

Lines 8-10 | AI row, footer and phone layout

```
8 | .ai-row { display: flex; flex-wrap: wrap; gap: 6px; margin-bottom: 10px; }
9 | .card-footer { display: flex; justify-content: center; gap: 8px; margin: 0 -14px; padding: 10px; background: #f3f5f7;
  |   border-radius: 0 0 6px 6px; }
10 | @media (max-width: 760px) { .feedback-card { width: 100%; } }
```

`.ai-row` lays the badge and tags in a wrapping row with 6px gaps. `.card-footer` centres the two buttons on a light-grey bar; the negative margin `0 -14px` cancels the card's side padding so the bar spans the full card width, and `border-radius: 0 0 6px 6px` rounds only the bottom corners. The media query on line 10 says: on screens up to 760px wide each card becomes `width: 100%` (one card per row).

Lines 11-12 | Sentiment badge colours

```
11 | .badge { display: inline-block; padding: 3px 12px; border-radius: 999px; font-size: 11px; font-weight: 700; color: #fff; }
12 | .badge.positive { background: #2fa84f; } .badge.neutral { background: #8a8f98; } .badge.negative { background: #d9363e; }
```

`.badge` is a small pill (`border-radius: 999px`), white bold 11px text. The three modifier rules give the colour: `.badge.positive` green `#2fa84f`, `.badge.neutral` grey `#8a8f98`, `.badge.negative` red `#d9363e`. The class names match the output of `sentimentClass()`.

Line 13 | Tag chips

```
13 | .tag { padding: 2px 8px; border-radius: 999px; background: #ece8fb; color: #5a45c8; font-size: 10px; }
```

`.tag` makes each AI tag a small lavender pill with purple text and a 10px font.

Workflow, next: This completes the customer's feedback pair (post, then view). The admin sees the same data from the other side, with filters. Next file: `adminviewfeedback.component.ts`.

Step 111: AdminviewfeedbackComponent - all feedback with filters and AI summary [Saumya - Feedback & AI (best fit)]

FRONTEND

`angularapp/src/app/components/adminviewfeedback/adminviewfeedback.component.ts`

What this file is	The admin's "Feedback Received" page. It loads every feedback of every customer, filters them in the browser by category and sentiment, shows the writer's profile in one modal and, in another modal, the driver's details together with an AI Review Summary that is fetched from the backend for that driver.
Who / what triggers it	The router (<code>adminviewfeedback</code> , protected by <code>AuthGuard</code> with role Admin); it is linked from the admin navbar.
What it uses / calls	<code>FeedbackService.getFeedbacks</code> , <code>AiService.getDriverFeedbackSummary</code> , the models <code>Feedback</code> , <code>Driver</code> , <code>User</code> , <code>DriverFeedbackSummary</code> , constants <code>FEEDBACK_CATEGORIES</code> and <code>SENTIMENTS</code> , and <code>toDisplayDate</code> .
Size	121 lines

Lines 1-9 | Imports

```
1 | import { Component, OnInit } from '@angular/core';
2 | import { FeedbackService } from '../services/feedback.service';
3 | import { AiService } from '../services/ai.service';
4 | import { Feedback } from '../models/feedback.model';
5 | import { Driver } from '../models/driver.model';
6 | import { User } from '../models/user.model';
7 | import { DriverFeedbackSummary } from '../models/ai.model';
8 | import { FEEDBACK_CATEGORIES, SENTIMENTS } from '../utils/constants';
9 | import { toDisplayDate } from '../utils/format';
```

`FeedbackService` loads the list; `AiService` loads the AI summary; `Feedback`, `Driver`, `User` and `DriverFeedbackSummary` (an interface with `summary`, `strengths`, `improvements`, `aiScore`, `averageRating`, `feedbackCount`) type the data; `FEEDBACK_CATEGORIES` and `SENTIMENTS` (`Positive`, `Neutral`, `Negative`) fill the two dropdowns; `toDisplayDate` formats dates.

Lines 11-15 | @Component decorator

```
11 | @Component({
12 |   selector: 'app-adminviewfeedback',
13 |   templateUrl: './adminviewfeedback.component.html',
14 |   styleUrls: ['./adminviewfeedback.component.css']
15 | })
```

Registers the selector `app-adminviewfeedback` and its template and style files.

Lines 16-22 | Class and list/filter fields

```
16 | export class AdminviewfeedbackComponent implements OnInit {
17 |   feedbacks: Feedback[] = [];
18 |   filteredFeedbacks: Feedback[] = [];
19 |   categories: string[] = FEEDBACK_CATEGORIES;
20 |   sentiments: string[] = SENTIMENTS;
21 |   categoryFilter: string = 'All';
22 |   sentimentFilter: string = 'All';
```

`feedbacks` holds everything the server sent and `filteredFeedbacks` holds what is currently shown (the table loops over this one). `categories` and `sentiments` are the dropdown options. `categoryFilter` and `sentimentFilter` hold the current selection; the text 'All' means "do not filter".

Lines 24-30 | Modal state fields

```
24 | showUserModal: boolean = false;
25 | selectedUser: User | null = null;
26 |
27 | showDriverModal: boolean = false;
28 | selectedDriver: Driver | null = null;
29 | summary: DriverFeedbackSummary | null = null;
30 | summaryLoading: boolean = false;
```

`showUserModal` and `selectedUser` control the "User Details" modal. `showDriverModal` and `selectedDriver` control the driver modal. `summary` holds the AI summary object (or null) and `summaryLoading` is true while the request is in progress; the template uses it to show a spinner (a small rotating circle).

Lines 32-34 | Helper and constructor

```
32 | toDisplayDate = toDisplayDate;
33 |
34 | constructor(private feedbackService: FeedbackService, private aiService: AiService) {}
```

`toDisplayDate` is copied to a property so the template can call it. The constructor injects the two services.

Lines 36-38 | ngOnInit

```
36 | ngOnInit(): void {
37 |   this.loadFeedbacks();
38 | }
```

Runs when the page is created and calls `loadFeedbacks()`.

Lines 40-52 | loadFeedbacks - GET /api/feedback

```
40 | loadFeedbacks(): void {
41 |   this.feedbackService.getFeedbacks().subscribe({
42 |     // The API answers 204 (empty body) when there is no feedback.
43 |     next: (feedbacks) => {
44 |       this.feedbacks = feedbacks || [];
45 |       this.applyFilters();
46 |     },
47 |     error: () => {
48 |       this.feedbacks = [];
49 |       this.applyFilters();
50 |     }
51 |   });
52 | }
```

`getFeedbacks()` sends **GET /api/feedback**. In `SecurityConfig` this URL is `permitAll()` (anyone can call it, even without a token), and `FeedbackController.viewAllFeedbacks` returns `feedbackService.getAllFeedbacks()` (`feedbackRepo.findAll()`), or **204 No Content** with an empty body when there is nothing. An empty body arrives as `null`, which is why the code writes `feedbacks || []`. After storing the list, `applyFilters()` fills `filteredFeedbacks`. On any error the list is emptied and filtered too, so the table shows "No Data Found".

Lines 54-60 | applyFilters - client-side filtering

```
54 | applyFilters(): void {
55 |   this.filteredFeedbacks = this.feedbacks.filter(
56 |     (feedback) =>
57 |       (this.categoryFilter === 'All' || feedback.category === this.categoryFilter) &&
58 |       (this.sentimentFilter === 'All' || feedback.sentiment === this.sentimentFilter)
59 |   );
60 | }
```

`Array.filter` keeps a feedback only when both conditions are true: the category filter is 'All' or equals the feedback's category, **and** the sentiment filter is 'All' or equals the feedback's sentiment. Everything happens in the browser; no new request is sent. A feedback with no sentiment (an old row) matches only when the sentiment filter is All.

Lines 62-71 | tagsOf and sentimentClass

```
62 | tagsOf(feedback: Feedback): string[] {
63 |     return (feedback.aiTags || '')
64 |         .split(',')
65 |         .map((tag) => tag.trim())
66 |         .filter((tag) => !!tag);
67 | }
68 |
69 | sentimentClass(sentiment?: string): string {
70 |     return (sentiment || 'none').toLowerCase();
71 | }
```

Same two helpers as on the customer page: `tagsOf` splits the comma-separated `aiTags` string into a clean list of tag chips, and `sentimentClass` returns the lower-case sentiment word used as the CSS class for the badge colour.

Lines 73-81 | Show Profile modal methods

```
73 | showProfile(feedback: Feedback): void {
74 |     this.selectedUser = feedback.user || null;
75 |     this.showUserModal = true;
76 | }
77 |
78 | closeUserModal(): void {
79 |     this.showUserModal = false;
80 |     this.selectedUser = null;
81 | }
```

`showProfile(feedback)` takes the nested `user` object from the feedback (the server includes the writer's `User` with email, username and mobile number; the password is marked write-only in the Java entity, so it is never sent) and opens the modal. `closeUserModal()` hides it and clears the selection.

Lines 83-102 | viewDriverInfo - open modal and fetch AI summary

```
83 | viewDriverInfo(feedback: Feedback): void {
84 |     this.selectedDriver = feedback.driver || null;
85 |     this.summary = null;
86 |     this.showDriverModal = true;
87 |     const driverId = this.selectedDriver?.driverId;
88 |     if (driverId !== undefined) {
89 |         this.summaryLoading = true;
90 |         this.aiService.getDriverFeedbackSummary(driverId).subscribe({
91 |             next: (summary) => {
92 |                 this.summary = this.normalise(summary);
93 |                 this.summaryLoading = false;
94 |             },
95 |             error: () => {
96 |                 // The AI summary is optional: the driver details stay available without it.
97 |                 this.summary = null;
98 |                 this.summaryLoading = false;
99 |             }
100 |         });
101 |     }
102 | }
```

First the driver nested in the feedback is stored, the old `summary` is cleared and the modal is opened immediately, so the driver details appear at once. If the feedback really has a driver (`driverId !== undefined`), `summaryLoading` becomes true and `getDriverFeedbackSummary(driverId)` sends **GET** `/api/ai/driver/{driverId}/feedback-summary`. The server side is described in the flow of the HTML entry.

- On `next`, the answer is passed through `normalise` (below) and stored; loading ends.
- On `error`, the summary stays null and loading ends. The comment explains why: the AI summary is optional, so the driver details remain usable. The `HttpErrorInterceptor` also skips URLs containing `/api/ai/`, so even a server failure here does not redirect to the error page.

Lines 104-109 | closeDriverModal

```
104 | closeDriverModal(): void {
105 |     this.showDriverModal = false;
106 |     this.selectedDriver = null;
107 |     this.summary = null;
108 |     this.summaryLoading = false;
109 | }
```

Hides the driver modal and resets driver, summary and the loading flag so nothing stale appears the next time.

Lines 111-121 | normalise - make strengths and improvements real arrays

```
111 | private normalise(summary: any): DriverFeedbackSummary {
112 |     const toList = (value: any): string[] =>
113 |         Array.isArray(value)
114 |         ? value
115 |         : String(value || '')
116 |             .split('\n')
117 |             .map((item) => item.trim())
118 |             .filter((item) => !!item);
119 |     return { ...summary, strengths: toList(summary?.strengths), improvements: toList(summary?.improvements) };
120 | }
121 | }
```

The template calls `.length` and loops over `summary.strengths` and `summary.improvements`, so they must be arrays. `toList` handles both shapes: if the value is already an array it is used as is; otherwise it is converted to text (`String(value || '')`), split into lines at `\n`, trimmed, and empty lines are dropped. This protects the page if the server (or Gemini) ever answers a newline-separated string, or leaves the field out. `{ ...summary, strengths: ..., improvements: ... }` copies all other fields (`summary`, `aiScore`, `averageRating`, `feedbackCount`) and replaces the two lists. The `any` type means "no type checking". With the current Java code the answer already contains lists, so the string branch is a defensive extra.

Workflow, next: The template `adminviewfeedback.component.html` shows the table, the two filters and the two modals. Next file: `adminviewfeedback.component.html`.

Step 112: adminviewfeedback.component.html - feedback table, filters and modals [Saumya - Feedback & AI (best fit)]

angularapp/src/app/components/adminviewfeedback/adminviewfeedback.component.html

FRONTEND

What this file is	A table of all feedback (number, user name, text, AI sentiment with tags, date, actions), two filter dropdowns above it, a "User Details" modal and a "Driver Details" modal that contains the AI Review Summary panel.
Who / what triggers it	Rendered by Angular for <code>AdminviewfeedbackComponent</code> .
What it uses / calls	Component fields and methods (<code>categoryFilter</code> , <code>sentimentFilter</code> , <code>applyFilters</code> , <code>filteredFeedbacks</code> , <code>showProfile</code> , <code>viewDriverInfo</code> , <code>summary</code> , <code>summaryLoading</code> , ...), the <code>FormsModule</code> (needed for <code>ngModel</code>), and classes such as <code>data-table</code> , <code>table-wrap</code> , <code>toolbar</code> , <code>spinner</code> from <code>styles.css</code> .
Size	102 lines

Lines 1-4 | Page frame and heading

```
1 | <div class="page-container">
2 |   <div class="page-card" id="feedbackCard">
3 |     <h2 class="page-title" id="feedbackHeading">Feedback Received</h2>
4 |     <div class="title-line"></div>
```

Page container and card, the title "Feedback Received" and the decorative `title-line`.

Lines 6-21 | Filter toolbar

```
6 | <div class="toolbar centered">
7 |   <label>
8 |     Filter by Category:
9 |     <select id="categoryFilter" [(ngModel)]="categoryFilter" (ngModelChange)="applyFilters()">
10 |       <option value="All">All Categories</option>
11 |       <option *ngFor="let category of categories" [value]="category">{{ category }}</option>
12 |     </select>
13 |   </label>
14 |   <label>
15 |     Filter by Sentiment:
16 |     <select id="sentimentFilter" [(ngModel)]="sentimentFilter" (ngModelChange)="applyFilters()">
17 |       <option value="All">All Sentiments</option>
18 |       <option *ngFor="let sentiment of sentiments" [value]="sentiment">{{ sentiment }}</option>
19 |     </select>
20 |   </label>
21 | </div>
```

Each `<label>` wraps a text and a `<select>`.

`[(ngModel)]="categoryFilter"` is **two-way binding** (the "banana in a box" syntax): the selected option is written into `categoryFilter`, and if the field changes in code the select updates. `ngModel` comes from `FormsModule`, which `AppModule` imports. `(ngModelChange)="applyFilters()"` runs the filter every time the selection changes. The first option has value `All`, the rest are generated with `*ngFor` from `categories` (or `sentiments`) with `[value]` and `{{ }}`.

Lines 23-38 | Table header and "No Data Found" row

```
23 | <div class="table-wrap">
24 |   <table class="data-table" id="feedbackTable">
25 |     <thead>
26 |       <tr>
27 |         <th>S.NO</th>
28 |         <th>USER NAME</th>
29 |         <th>FEEDBACK</th>
30 |         <th>AI SENTIMENT</th>
31 |         <th>POSTED DATE</th>
32 |         <th>ACTION</th>
33 |       </tr>
34 |     </thead>
35 |     <tbody>
36 |       <tr class="empty-row" id="noDataMessage" *ngIf="filteredFeedbacks.length === 0">
37 |         <td colspan="6">No Data Found</td>
38 |       </tr>
```

`table-wrap` allows horizontal scrolling on small screens. The `<thead>` defines six column titles: S.NO, USER NAME, FEEDBACK, AI SENTIMENT, POSTED DATE, ACTION. The first body row is an `empty-row` with `colspan="6"` (one cell across all six columns) shown only when `filteredFeedbacks.length === 0`, so a filter without matches also shows it.

Lines 39-42 | One row per feedback

```
39 | <tr *ngFor="let feedback of filteredFeedbacks; let i = index" [attr.id]='feedback-' + feedback.feedbackId">
40 |   <td>{{ i + 1 }}</td>
41 |   <td>{{ feedback.user?.username }}</td>
42 |   <td>{{ feedback.feedbackText }}</td>
```

`*ngFor="let feedback of filteredFeedbacks; let i = index"` repeats the row and gives the counter `i` (starting at 0), so `{{ i + 1 }}` prints 1, 2, 3 as S.NO. `[attr.id]` gives ids like `feedback-4`. `{{ feedback.user?.username }}` prints the writer's name; the `?.` safe-navigation operator avoids an error if `user` is missing. `{{ feedback.feedbackText }}` prints the text.

Lines 43-49 | Sentiment cell

```
43 | <td>
44 |   <span class="badge" [ngClass]="sentimentClass(feedback.sentiment)" *ngIf="feedback.sentiment; else noSen
45 |   timent">{{ feedback.sentiment }}</span>
46 |   <ng-template #noSentiment><span class="muted">&mdash;</span></ng-template>
47 |   <div class="tags" *ngIf="tagsOf(feedback).length">
48 |     <span class="tag" *ngFor="let tag of tagsOf(feedback)">{{ tag }}</span>
49 |   </div>
49 | </td>
```

The badge `span` uses `[ngClass]="sentimentClass(feedback.sentiment)"` for its colour and `*ngIf="feedback.sentiment; else noSentiment"` shows it only when a sentiment exists; otherwise the template named `noSentiment` (`<ng-template #noSentiment>`, a dash `—`) is shown instead. Below it, a `div.tags` appears when `tagsOf(feedback).length` is non-zero (0 counts as false) and loops over the tags to draw chips.

Lines 50-60 | Date, action buttons and closing tags

```
50 | <td>{{ toDisplayDate(feedback.date) }}</td>
51 | <td class="actions">
52 |   <button type="button" class="btn btn-primary btn-pill" (click)="showProfile(feedback)">Show Profile</but
53 |   ton>
54 |   <button type="button" class="btn btn-primary btn-pill" (click)="viewDriverInfo(feedback)">View Driver In
55 |   fo</button>
56 | </td>
57 | </tr>
58 | </tbody>
59 | </table>
60 | </div>
60 | </div>
60 | </div>
```

The date is formatted by `toDisplayDate`. The actions cell holds **Show Profile** (`showProfile(feedback)`) and **View Driver Info** (`viewDriverInfo(feedback)`). The remaining lines close row, body, table, wrapper, card and container.

Lines 62-71 | User Details modal

```
62 | <!-- User details -->
63 | <div class="modal-backdrop" *ngIf="showUserModal">
64 |   <div class="modal-box left-box" id="userModal">
65 |     <h3 class="small">User Details:</h3>
66 |     <p><strong>Email:</strong> {{ selectedUser?.email }}</p>
67 |     <p><strong>Username:</strong> {{ selectedUser?.username }}</p>
68 |     <p><strong>Mobile Number:</strong> {{ selectedUser?.mobileNumber }}</p>
69 |     <div class="modal-actions"><button type="button" class="btn btn-primary" id="closeUserModal" (click)="closeUserMod
    al()">Close</button></div>
70 |   </div>
71 | </div>
```

Shown when `showUserModal` is true. It prints `selectedUser?.email`, `username` and `mobileNumber` of the person who wrote the feedback (`?.` because `selectedUser` can be null). The Close button (`id="closeUserModal"`) calls `closeUserModal()`.

Lines 73-83 | Driver Details modal (driver data)

```
73 | <!-- Driver details + AI review summary -->
74 | <div class="modal-backdrop" *ngIf="showDriverModal">
75 |   <div class="modal-box left-box driver-box" id="driverModal">
76 |     <button type="button" class="modal-close" id="closeDriverModal" (click)="closeDriverModal()">&times;</button>
77 |     <h3 class="small">Driver Details</h3>
78 |     <ng-container *ngIf="selectedDriver; else noDriver">
79 |       <p><strong>Driver Name:</strong> {{ selectedDriver.driverName }}</p>
80 |       <p><strong>License Number:</strong> {{ selectedDriver.licenseNumber }}</p>
81 |       <p><strong>Experience Years:</strong> {{ selectedDriver.experienceYears }}</p>
82 |       <p><strong>Vehicle Type:</strong> {{ selectedDriver.vehicleType }}</p>
83 |       <p><strong>Hourly Rate:</strong> &#8377;{{ selectedDriver.hourlyRate }}</p>
```

Shown while `showDriverModal` is true. The `×` button closes it. `<ng-container *ngIf="selectedDriver; else noDriver">` prints name, licence number, experience years, vehicle type and hourly rate (with the rupee sign `₹`) of the driver nested in the feedback.

Lines 85-98 | AI Review Summary panel

```
85 |   <div class="ai-panel" id="aiReviewSummary" *ngIf="summaryLoading || summary">
86 |     <div class="ai-title">&#10024; AI Review Summary</div>
87 |     <div *ngIf="summaryLoading" class="spinner small"></div>
88 |     <ng-container *ngIf="summary">
89 |       <div class="ai-meta">{{ summary.feedbackCount }} review(s) &middot; Avg rating {{ summary.averageRating }}/5
      &middot; AI score {{ summary.aiScore }}/100</div>
90 |       <p class="ai-text">{{ summary.summary }}</p>
91 |       <div *ngIf="summary.strengths.length"><strong>Strengths</strong>
92 |         <ul><li *ngFor="let item of summary.strengths">{{ item }}</li></ul>
93 |       </div>
94 |       <div *ngIf="summary.improvements.length"><strong>Areas to improve</strong>
95 |         <ul><li *ngFor="let item of summary.improvements">{{ item }}</li></ul>
96 |       </div>
97 |     </ng-container>
98 |   </div>
```

`div#aiReviewSummary` appears when `summaryLoading || summary` is truthy, so it is visible while loading and after a successful answer, and **disappears completely when the request failed** (graceful failure: the admin simply sees the driver details). Inside:

- The title with the sparkles symbol `✨`.
- `<div *ngIf="summaryLoading" class="spinner small">` is the loading spinner (rotating circle from the global CSS, resized by the local CSS).
- `<ng-container *ngIf="summary">` shows the result: a line "N review(s) · Avg rating X/5 · AI score Y/100" (`·` is a dot separator), the paragraph `summary.summary`, then the **Strengths** list and the **Areas to improve** list. Each list block has `*ngIf="summary.strengths.length"` so an empty list shows nothing, and `<li *ngFor="let item of summary.strengths">` prints one bullet per string.

Lines 99-102 | Fallback template and closing tags

```
99 |   </ng-container>
100 |   <ng-template #noDriver><p>No driver is linked to this feedback.</p></ng-template>
101 | </div>
102 | </div>
```

`<ng-template #noDriver>` shows "No driver is linked to this feedback." when the feedback has no driver. The rest closes the modal box and backdrop.

Backend trace for this page.

- Loading the table: **GET** `/api/feedback` (`permitAll` in `SecurityConfig`, so no role is needed) is answered by `FeedbackController.viewAllFeedbacks`, which returns all rows or 204. A side effect worth knowing: because the endpoint is public, the nested user data (email, mobile number) is readable by anyone who calls the URL; only the password is hidden.
- Filters: no request; `applyFilters` works on the array already in memory.
- View Driver Info: **GET** `/api/ai/driver/{driverId}/feedback-summary` requires ADMIN or CUSTOMER (rule `/api/ai/driver/**`) and is answered by `AiController.getFeedbackSummary`, which returns `ResponseEntity.ok(aiService.summarizeDriverFeedback(driverId))`.
- `AiService.summarizeDriverFeedback(driverId)` loads that driver's feedback with `feedbackRepo.findByDriverDriverId`, then computes: `count`, the average rating rounded to one decimal, and the number of Positive feedbacks. The **aiScore** (0 to 100) is `round(average / 5 * 70 + positive / count * 30)`: ratings count for up to 70 points and the share of Positive feedback for up to 30. Example: average 4.5 with 3 of 4 positive gives $63 + 22.5 = 85.5$, rounded to 86. With zero feedback the score is 0 and the summary text is "No feedback has been posted for this driver yet."
- The text parts: `summarizeWithGemini` sends up to 40 reviews to Gemini and expects JSON with `summary`, `strengths` and `improvements` (at most 4 short items each); if Gemini is disabled, fails, or returns an empty summary it returns false and `summarizeLocally` is used. The local method counts the AI tags of good feedback (Positive or rating 4+) and bad feedback (Negative or rating 2-), keeps the top 3 of each and writes lines such as "Customers praise the safety" and "Customers raised concerns about punctuality" (or "No recurring issues reported"); the summary sentence is "Based on N review(s) with an average rating of X/5, P of them were positive."
- The result map `summary`, `strengths`, `improvements`, `aiScore`, `averageRating`, `feedbackCount` returns as JSON; Angular runs it through `normalise` and displays it.

Styling for this page is in `adminviewfeedback.component.css`.

Step 113: adminviewfeedback.component.css - toolbar, badges, modals, AI panel [Saumya - Feedback & AI (best fit)]

FRONTEND

angularapp/src/app/components/adminviewfeedback/adminviewfeedback.component.css

What this file is	Local styling for the admin feedback page: the filter bar, action buttons, sentiment badges and tag chips, the left-aligned modals and the lilac AI summary panel.
Who / what triggers it	Loaded with the component through <code>styleUrls</code> .
What it uses / calls	Classes from <code>adminviewfeedback.component.html</code> ; <code>.spinner</code> itself is defined in <code>src/styles.css</code> and only resized here.
Size	23 lines

Lines 1-3 | Toolbar and action buttons

```
1 | .toolbar.centered { justify-content: center; gap: 22px; }
2 | .actions { white-space: nowrap; }
3 | .actions .btn { margin: 2px 2px; font-size: 12px; }
```

`.toolbar.centered` centres the two filters (`justify-content: center`) with a 22px gap. `.actions` has `white-space: nowrap`, so the two buttons never wrap onto separate lines. `.actions .btn` gives each button a small margin and a 12px font.

Lines 5-10 | Badges, tags

```
5 | .badge { display: inline-block; padding: 3px 12px; border-radius: 999px; font-size: 11px; font-weight: 700; color: #ff
  | f; }
6 | .badge.positive { background: #2fa84f; }
7 | .badge.neutral { background: #8a8f98; }
8 | .badge.negative { background: #d9363e; }
9 | .tags { margin-top: 4px; }
10 | .tag { display: inline-block; margin: 1px 2px; padding: 1px 8px; border-radius: 999px; background: #ece8fb; color: #5a
  | 45c8; font-size: 10px; }
```

`.badge` is a white-on-colour pill; `.positive`, `.neutral`, `.negative` pick green, grey, red (the same colours as the customer page). `.tags` adds space above the chip row. `.tag` is an inline-block lavender chip with purple 10px text and small margins so chips sit side by side.

Lines 12-16 | Left-aligned modals

```
12 | .left-box { text-align: left; min-width: 330px; max-width: 420px; }
13 | .left-box h3.small { margin-bottom: 12px; font-size: 15px; }
14 | .left-box p { margin: 6px 0; font-size: 13px; }
15 | .left-box .modal-actions { margin-top: 14px; }
16 | .driver-box { max-width: 440px; }
```

`.left-box` overrides the default centred modal: `text-align: left`, `min-width: 330px`, `max-width: 420px`. The following rules tune the heading (`h3.small`: 15px with bottom margin), paragraphs (6px margins, 13px font) and the action area. `.driver-box` allows a slightly wider driver modal (440px), which has to hold the AI panel.

Lines 18-21 | AI summary panel text

```
18 | .ai-panel { margin-top: 14px; padding: 12px 14px; background: #f1ecfd; border-radius: 6px; font-size: 13px; }
19 | .ai-title { margin-bottom: 4px; font-weight: 700; color: #5a45c8; }
20 | .ai-meta { font-size: 12px; color: #444; }
21 | .ai-text { margin: 8px 0 !important; line-height: 1.5; }
```

`.ai-panel` is the lilac box (`#f1ecfd`) with padding, rounded corners and 13px text. `.ai-title` is bold purple. `.ai-meta` is a small grey-dark line for count, average and score. `.ai-text` gets 8px vertical margin with `!important`, which is needed to beat the more specific `.left-box p` margin rule defined above, and a comfortable `line-height: 1.5`.

Lines 22-23 | Lists and spinner size

```
22 | .ai-panel ul { margin: 4px 0 8px 18px; padding: 0; }
23 | .spinner.small { width: 20px; height: 20px; margin: 6px 0; }
```

`.ai-panel ul` sets the list margins and removes default padding so bullets align inside the box. `.spinner.small` (an element with both classes) reduces the global 30px spinner to 20 by 20 pixels.

Workflow, next: The admin feedback page is finished. Last of all pages is the one shown when something goes wrong. Next file: `error.component.ts`.

Step 114: ErrorComponent - the "Something Went Wrong" page [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/components/error/error.component.ts

FRONTEND

What this file is	The smallest component of the app. It only holds two fixed texts, a title and a message, that the template prints. It has no logic and no services.
Who / what triggers it	The router, in two situations. First, <code>HttpErrorInterceptor</code> calls <code>this.router.navigate(['/error'])</code> whenever a request (other than login/register/AI calls) fails with status 0 (server unreachable) or 5xx (server error). Second, in <code>app-routing.module.ts</code> the last route <code>{ path: '**', component: ErrorComponent }</code> is a wildcard that matches any address nobody else matched, such as a mistyped URL. It has no <code>AuthGuard</code> , so it is reachable even when not logged in.
What it uses / calls	Only <code>Component</code> from Angular. Its template is <code>error.component.html</code> .
Size	11 lines

Line 1 | Import

```
1 | import { Component } from '@angular/core';
```

Brings in the `Component` decorator.

Lines 3-7 | @Component decorator

```
3 | @Component({
4 |   selector: 'app-error',
5 |   templateUrl: './error.component.html',
6 |   styleUrls: ['./error.component.css']
7 | })
```

Selector `app-error`, plus the template and style files.

Lines 8-11 | Class and texts

```
8 | export class ErrorComponent {
9 |   title: string = 'Something Went Wrong';
10 |   message: string = "We're sorry, but an error occurred. Please try again later.";
11 | }
```

`title` is "Something Went Wrong" and `message` is "We're sorry, but an error occurred. Please try again later." Both are typed as `string` and are used in the template with `{{ }}`. The message is written with double quotes because it contains an apostrophe.

Workflow, next: The template prints these two fields. Next file: `error.component.html`.

Step 115: error.component.html - error message markup [Vaishalini - Customer Journey & Shared Infra]

FRONTEND

angularapp/src/app/components/error/error.component.html

What this file is	One box with a heading and a paragraph.
Who / what triggers it	Rendered inside <code><router-outlet></code> of <code>AppComponent</code> when the route is <code>/error</code> or an unknown URL.
What it uses / calls	<code>title</code> and <code>message</code> from <code>ErrorComponent</code> ; the CSS class <code>error-page</code> .
Size	4 lines

Lines 1-4 | Error box

```
1 | <div class="error-page" id="errorPage">
2 |   <h1>{{ title }}</h1>
3 |   <p>{{ message }}</p>
4 | </div>
```

`<div class="error-page" id="errorPage">` is the wrapper (the id is a test hook). `<h1>{{ title }}</h1>` prints "Something Went Wrong" and `<p>{{ message }}</p>` prints the explanation. `{{ }}` is interpolation: Angular reads the class field and inserts it as text.

Workflow, next: Next is the style file `error.component.css`.

Step 116: error.component.css - error page look [Vaishalini - Customer Journey & Shared Infra]

FRONTEND

angularapp/src/app/components/error/error.component.css

What this file is	Three rules that centre the text, make the title red and keep the page simple.
Who / what triggers it	Loaded with <code>ErrorComponent</code> .
What it uses / calls	The class <code>error-page</code> in <code>error.component.html</code> .
Size	3 lines

Lines 1-3 | Wrapper, title and message

```
1 | .error-page { padding-top: 46px; text-align: center; font-family: 'Times New Roman', Times, serif; }
2 | .error-page h1 { margin: 0 0 12px; font-size: 30px; font-weight: 700; color: red; }
3 | .error-page p { margin: 0; font-size: 16px; color: #111; }
```

`.error-page` adds 46px of space above, centres the text and sets the font to Times New Roman with generic serif fallbacks. `.error-page h1` removes the default margin except 12px below, uses a 30px bold font and the colour red. `.error-page p` removes its margin, uses a 16px font and almost-black `#111`.

That completes the user journey. Every runtime file of the project has now been followed, from `pom.xml` and `main()` on the backend, through the Angular bootstrap, to the last page of the app, with each click traced to the controller, service and repository that answer it. The appendix explains the test files that verify all of this, starting with `angularapp/src/app/app.component.spec.ts`.

Part 4 - Appendix: tests, harness and project files

These files are not part of the request flow, but they prove the project works and show how it is built, tested and checked. They are explained after the journey because nobody "runs" them while using the app: Karma (frontend) and JUnit (backend) run them on demand.

Important facts, verified by comparing the files:

- The **official** tests are the 8 files in `karma/` (4 models + 4 services, all using `fit`) and the JUnit class in `junit/`. They are byte-identical to the copies in the project.
- The component, guard, AppComponent and AIService specs were written by us to check behaviour. They use `it`, not `fit`, so the official Karma run (which only runs focused specs) skips them.

Step 117: app.component.spec.ts - tests for the root component [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/app.component.spec.ts

TEST

What this file is

The user journey is finished: a person can register, log in, browse drivers, request a trip, end the trip, pay and leave feedback. This appendix covers the files that **prove** that journey works and the files that **build and check** the project. None of them is part of the running request flow: when you start `mvn spring-boot:run` and `ng serve`, not one line of this appendix executes. A **spec file** (`*.spec.ts`) is a small program that creates a piece of the app in a fake environment, pokes it, and checks the result. Angular uses the **Jasmine** testing library to write the checks and **Karma** to run them inside a real (headless, meaning no visible window) Chrome browser. A few words you will see all through this appendix:

- `describe('Name', () => { ... })` groups related tests under a heading. It does not test anything by itself.
- `it('does something', () => { ... })` is **one test**. If every `expect` inside it passes, the test passes.
- `fit(...)` is a **focused it**. As soon as Jasmine sees even one `fit` anywhere in the whole test run, it runs **only** the focused tests and **skips every plain it**. The official graded tests in this project all use `fit`; the tests our group wrote use `it`. We come back to the consequence in the karma entries.
- `beforeEach(() => { ... })` is code Jasmine runs again before every single test in the group, so each test starts from a clean state. `afterEach` runs after every test (we use it to wipe `localStorage`).
- `expect(value).toBe(x), .toEqual(x), .toBeTruthy(), .toContain(x), .toBeTrue()` are **matchers**: each states what the value must be, and the test fails if it is not.
- `TestBed` is Angular's test workbench. `TestBed.configureTestingModule({...})` builds a tiny throw-away module (like a mini `AppModule`) that declares the component under test and imports only what it needs. `TestBed.createComponent(X)` builds the component and returns a **fixture** (a wrapper); `fixture.componentInstance` is the real TypeScript object, `fixture.nativeElement` is its real HTML, and `fixture.detectChanges()` tells Angular to refresh the HTML after the data changed (in tests nothing refreshes automatically).

This first spec protects the very first thing the browser renders: `AppComponent`, the shell that shows the correct navigation bar for the logged-in role above the `<router-outlet>`. It is a file we wrote ourselves (the grading folder only contains the stock Angular placeholder spec for this component).

Who / what triggers it

Only when someone runs `ng test` (or the grading script `karma/karma.sh`). Karma bundles every `*.spec.ts` file found by `tsconfig.spec.json`, opens headless Chrome and runs them. Nothing in the running application imports a spec file.

What it uses / calls

`AppComponent` (`app.component.ts/html`), `AuthService` (to push a fake role into its role stream), `RouterTestingModule` and `HttpClientTestingModule` (fake router and fake HTTP), Angular `NO_ERRORS_SCHEMA`.

Size

50 lines

Lines 1-6 | Imports

```
1 | import { TestBed } from '@angular/core/testing';
2 | import { NO_ERRORS_SCHEMA } from '@angular/core';
3 | import { RouterTestingModule } from '@angular/router/testing';
4 | import { HttpClientTestingModule } from '@angular/common/http/testing';
5 | import { AppComponent } from './app.component';
6 | import { AuthService } from './services/auth.service';
```

`TestBed` is the test workbench described above. `NO_ERRORS_SCHEMA` tells Angular's template compiler "do not complain about HTML tags you do not know". It is needed because `app.component.html` contains `<app-adminnav>` and `<app-customernav>`, and this tiny test module does not declare those two components. `RouterTestingModule` supplies a **fake router** so `<router-outlet>` and `routerLink` work without real navigation. `HttpClientTestingModule` supplies a **fake HttpClient** that never touches the network, because `AuthService` (injected into `AppComponent`) needs an `HttpClient`. `AppComponent` is the class under test and `AuthService` is imported so the test can reach the same singleton instance the component uses.

Lines 8-16 | describe + beforeEach: building the test module

```
8 | describe('AppComponent', () => {
9 |   beforeEach(() => {
10 |     localStorage.clear();
11 |     TestBed.configureTestingModule({
12 |       imports: [RouterTestingModule, HttpClientTestingModule],
13 |       declarations: [AppComponent],
14 |       schemas: [NO_ERRORS_SCHEMA]
15 |     });
16 |   });
```

`describe('AppComponent', ...)` opens the group. Before every test, `localStorage.clear()` removes any `token`, `userRole`, `userId` or `username` left by earlier tests, because `AuthService` reads those keys when it is created and would otherwise start "logged in". `configureTestingModule` then declares only `AppComponent`, imports the two fake modules and sets `schemas: [NO_ERRORS_SCHEMA]`. Note that the component is **not** created here; each test creates its own fixture so it can choose what to set up first.

Line 18 | afterEach: clean up

```
18 |   afterEach(() => localStorage.clear());
```

After every test the browser storage is wiped again, so a test that "logs in" cannot leak into the next test or into a different spec file running in the same browser tab.

Lines 20-23 | Test: should create the app

```
20 |   it('should create the app', () => {
21 |     const fixture = TestBed.createComponent(AppComponent);
22 |     expect(fixture.componentInstance).toBeTruthy();
23 |   });
```

Builds the component with `TestBed.createComponent(AppComponent)` and asserts `fixture.componentInstance` is truthy. It protects against a broken constructor or a missing dependency (for example if `AuthService` could no longer be injected, this fails first).

Lines 25-28 | Test: title is "angularapp"

```
25 |   it('should have as title 'angularapp'', () => {
26 |     const fixture = TestBed.createComponent(AppComponent);
27 |     expect(fixture.componentInstance.title).toEqual('angularapp');
28 |   });
```

Asserts the public field `title` of `AppComponent` equals `'angularapp'`. This is the stock test every Angular project starts with; it protects the one public field the class declares.

Lines 30-37 | Test: router outlet and no navbar while logged out

```
30 |   it('should render a router outlet and no navbar while logged out', () => {
31 |     const fixture = TestBed.createComponent(AppComponent);
32 |     fixture.detectChanges();
33 |     const el = fixture.nativeElement as HTMLElement;
34 |     expect(el.querySelector('router-outlet')).toBeTruthy();
35 |     expect(el.querySelector('app-adminnav')).toBeNull();
36 |     expect(el.querySelector('app-customernav')).toBeNull();
37 |   });
```

With empty `localStorage` the user is a stranger. `fixture.detectChanges()` renders `app.component.html`. The test asserts three things in the real rendered HTML (`nativeElement`): a `router-outlet` element exists (so pages can load), and **no** `app-adminnav` and **no** `app-customernav` element exists. This protects the rule that the login and signup pages show no menu: `*ngIf="isAdmin$ | async"` and `*ngIf="isCustomer$ | async"` must both be false when no role is stored.

Lines 39-50 | Test: matching navbar for the logged-in role

```
39 | it('should render the matching navbar for the logged in role', () => {
40 |     const fixture = TestBed.createComponent(AppComponent);
41 |     const auth = TestBed.inject(AuthService);
42 |     fixture.detectChanges();
43 |     (auth as any).userRoleSubject.next('Admin');
44 |     fixture.detectChanges();
45 |     expect((fixture.nativeElement as HTMLElement).querySelector('app-adminnav')).toBeTruthy();
46 |     (auth as any).userRoleSubject.next('Customer');
47 |     fixture.detectChanges();
48 |     expect((fixture.nativeElement as HTMLElement).querySelector('app-customernav')).toBeTruthy();
49 | });
50 | });
```

The service keeps the role in a private `BehaviorSubject` called `userRoleSubject` (an observable that remembers its latest value). The test reaches it with `(auth as any)` (the cast switches off TypeScript's privacy check) and calls `.next('Admin')`, which is exactly what `AuthService.login` does after a successful login. After `detectChanges()` the async pipe sees the new role and the test expects `app-adminnav` to be present. It then pushes `'Customer'` and expects `app-customernav`. It protects the live switching of the navigation bar without reloading the page, and the case-insensitive role check inside `AppComponent`. Line 50 closes the `describe`.

Workflow, next: `AppComponent` is verified. The next specs follow the page order of the journey, starting with the first page a visitor sees: `login.component.spec.ts`.

Step 118: login.component.spec.ts - tests for the login page [Aleena - Authentication]

angularapp/src/app/components/login/login.component.spec.ts

TEST

What this file is	Checks the login form from the outside, the way a user experiences it: what the page says, which validation messages appear, what happens when the server rejects the password and what happens when the server accepts it. The tests call the component's <code>onSubmit()</code> method directly instead of clicking the button; that is the same code the <code>(ngSubmit)</code> event runs.
Who / what triggers it	Run by Karma during <code>ng test</code> . It is not part of the running app.
What it uses / calls	<code>LoginComponent</code> , <code>ReactiveFormsModule</code> (the page uses <code>[formGroup]</code>), <code>RouterTestingModule</code> , <code>HttpClientTestingModule</code> plus its controller <code>HttpTestingController</code> , and the real <code>Router</code> (to spy on <code>navigate</code>). The real <code>AuthService</code> is used underneath, so the test also proves that the token is saved to <code>localStorage</code> .
Size	72 lines

Lines 1-7 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { ReactiveFormsModule } from '@angular/forms';
3 | import { RouterTestingModule } from '@angular/router/testing';
4 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
5 | import { Router } from '@angular/router';
6 |
7 | import { LoginComponent } from './login.component';
```

`ComponentFixture` is the type of the fixture wrapper. `ReactiveFormsModule` must be imported or the `[formGroup]` and `formControlName` attributes in the login template would be unknown. `HttpTestingController` is the remote control for the fake HTTP backend: it lets a test see which request the component sent and then answer it by hand. `Router` is imported so the test can replace its `navigate` method with a spy. `LoginComponent` is the class under test.

Lines 9-15 | describe, shared variables and the text() helper

```
9 | describe('LoginComponent', () => {
10 |     let component: LoginComponent;
11 |     let fixture: ComponentFixture<LoginComponent>;
12 |     let http: HttpTestingController;
13 |     let router: Router;
14 |
15 |     const text = () => (fixture.nativeElement as HTMLElement).textContent || '';
```

Four variables are shared by all tests: the component, its fixture, the HTTP controller and the router. `text` is a small helper function that returns all text currently visible in the component's HTML (`textContent`); tests use `expect(text()).toContain(...)` to check what the user can read on the page.

Lines 17-28 | beforeEach: fresh module, fresh component

```
17 | beforeEach(() => {
18 |     localStorage.clear();
19 |     TestBed.configureTestingModule({
20 |         imports: [ReactiveFormsModule, RouterTestingModule, HttpClientTestingModule],
21 |         declarations: [LoginComponent]
22 |     });
23 |     fixture = TestBed.createComponent(LoginComponent);
24 |     component = fixture.componentInstance;
25 |     http = TestBed.inject(HttpTestingController);
26 |     router = TestBed.inject(Router);
27 |     fixture.detectChanges();
28 | });
```

Clears `localStorage`, builds a test module with the three imports and declares `LoginComponent`. It creates the component, grabs the `HttpTestingController` and `Router` from the module with `TestBed.inject`, then calls `detectChanges()` once, which runs `ngOnInit` and draws the first version of the page. Every test below therefore starts with a freshly drawn, empty login form and no stored session.

Line 30 | afterEach

```
30 | afterEach(() => localStorage.clear());
```

Wipes `localStorage` so the token stored by the success test cannot affect other tests.

Lines 32-34 | Test: should create

```
32 | it('should create', () => {
33 |     expect(component).toBeTruthy();
34 | });
```

The minimum sanity check: the component can be constructed.

Lines 36-39 | Test: heading and sign-up link

```
36 | it('shows the DriveU heading and a sign up link', () => {
37 |     expect(text()).toContain('DriveU');
38 |     expect(text()).toContain("Don't have an account?");
39 | });
```

Reads the rendered text and expects the brand name `DriveU` and the sentence `Don't have an account?` (the line that leads to the signup page). It protects the page against someone accidentally removing the heading or the link to registration.

Lines 41-46 | Test: required messages on an empty submit

```
41 | it('shows required messages when submitted empty', () => {
42 |     component.onSubmit();
43 |     fixture.detectChanges();
44 |     expect(text()).toContain('Email address is required');
45 |     expect(text()).toContain('Password is required');
46 | });
```

Calls `onSubmit()` on the untouched form. The component sets its `submitted` flag to true, which makes the template show the error text of every invalid field. After `detectChanges()` the test expects `Email address is required` and `Password is required`. It protects the rule that the form must show why it is invalid instead of silently doing nothing, and that no login request is sent for an invalid form.

Lines 48-54 | Test: email format and minimum password length

```
48 | it('validates the email format and password length', () => {
49 |     component.loginForm.setValue({ email: 'qq', password: 'a' });
50 |     component.onSubmit();
51 |     fixture.detectChanges();
52 |     expect(text()).toContain('Please enter a valid email address');
53 |     expect(text()).toContain('Password must be at least 6 characters long');
54 | });
```

`component.loginForm.setValue({ email: 'qq', password: 'a' })` fills the form with a malformed email and a one-character password, then submits. The test expects `Please enter a valid email address` and `Password must be at least 6 characters long`. This protects the email pattern rule (`Validators.pattern(EMAIL_REGEX)`) and the `Validators.minLength(6)` rule.

Lines 56-62 | Test: wrong credentials show an error

```
56 | it('shows an error for invalid credentials', () => {
57 |     component.loginForm.setValue({ email: 'admin@gmail.com', password: 'wrongpass' });
58 |     component.onSubmit();
59 |     http.expectOne('http://localhost:8080/api/login').flush('Invalid credentials', { status: 401, statusText: 'Unautho
60 | rized' });
61 |     fixture.detectChanges();
62 |     expect(text()).toContain('Invalid email address or password');
```

Fills the form with a valid-looking but wrong password and submits. The component calls `AuthService.login`, which does `POST http://localhost:8080/api/login` (answered in the real app by `AuthController.authenticateUser`). `http.expectOne(url)` asserts exactly one such request was made and `.flush(body, { status: 401, ... })` answers it by hand with **401 Unauthorized**, exactly what the backend returns for bad credentials. After `detectChanges()` the page must show **Invalid email address or password**. It protects the error branch of the subscribe callback.

Lines 64-72 | Test: success stores the token and goes home

```
64 | it('stores the token and goes to the home page on success', () => {
65 |     const navigate = spyOn(router, 'navigate').and.returnValue(Promise.resolve(true));
66 |     component.loginForm.setValue({ email: 'admin@gmail.com', password: 'secret123' });
67 |     component.onSubmit();
68 |     http.expectOne('http://localhost:8080/api/login').flush({ token: 'jwt', username: 'admin', userRole: 'Admin', user
    |     Id: 1 });
69 |     expect(localStorage.getItem('token')).toBe('jwt');
70 |     expect(navigate).toHaveBeenCalledOnceWith(['/home']);
71 | });
72 | });
```

`spyOn(router, 'navigate').and.returnValue(Promise.resolve(true))` replaces the real `navigate` with a **spy**, a fake that records its calls and does nothing else (so no real page change happens). After submitting valid values the test answers the login request with the JSON the backend sends: `{ token, username, userRole, userId }`. It then checks two things: `localStorage` now holds `token` equal to `jwt` (written by `AuthService.login`), and `navigate` was called with `['/home']`. It protects the happy path: save the session, then go to the home page.

Workflow, next: Login is verified. Registration comes next in the journey: `signup.component.spec.ts`.

Step 119: signup.component.spec.ts - tests for the registration page [Aleena - Authentication]

angularapp/src/app/components/signup/signup.component.spec.ts

TEST

What this file is	Verifies the registration form: required-field messages, format rules, the password-match rule, the duplicate-email conflict from the server and the success popup. As in the login spec, the tests call <code>onSubmit()</code> and answer the HTTP request by hand.
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>SignupComponent</code> , <code>ReactiveFormsModule</code> , <code>RouterTestingModule</code> , <code>HttpClientTestingModule</code> and <code>HttpTestingController</code> . The request under test is <code>POST /api/register</code> , answered in the real app by <code>AuthController.registerUser</code> .
Size	71 lines

Lines 1-6 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { ReactiveFormsModule } from '@angular/forms';
3 | import { RouterTestingModule } from '@angular/router/testing';
4 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
5 |
6 | import { SignupComponent } from './signup.component';
```

Same set as the login spec: fixture type, reactive forms, fake router, fake HTTP and its controller, plus `SignupComponent`. There is no `Router` import because no test here needs to spy on navigation.

Lines 8-21 | describe, helpers and a valid sample form

```
8 | describe('SignupComponent', () => {
9 |     let component: SignupComponent;
10 |     let fixture: ComponentFixture<SignupComponent>;
11 |     let http: HttpTestingController;
12 |
13 |     const text = () => (fixture.nativeElement as HTMLElement).textContent || '';
14 |     const validValues = {
15 |         username: 'customer2name',
16 |         email: 'cus2@gmail.com',
17 |         mobileNumber: '7888499919',
18 |         password: 'secret12',
19 |         confirmPassword: 'secret12',
20 |         userRole: 'Customer'
21 |     };
```

`text()` returns the visible text. `validValues` is a complete, valid registration (username, a Gmail-style email, a 10-digit mobile number, matching 8-character passwords and role `Customer`) that two tests reuse. Keeping it in one constant avoids repeating the object.

Lines 23-32 | `beforeEach`

```
23 | beforeEach(() => {
24 |     TestBed.configureTestingModule({
25 |         imports: [ReactiveFormsModule, RouterTestingModule, HttpClientTestingModule],
26 |         declarations: [SignupComponent]
27 |     });
28 |     fixture = TestBed.createComponent(SignupComponent);
29 |     component = fixture.componentInstance;
30 |     http = TestBed.inject(HttpTestingController);
31 |     fixture.detectChanges();
32 | });
```

Builds the test module and the component, fetches the HTTP controller and runs the first `detectChanges()`. There is no `localStorage.clear()` here because signup never reads the stored session.

Lines 34-36 | Test: `should create`

```
34 | it('should create', () => {
35 |     expect(component).toBeTruthy();
36 | });
```

The component can be constructed.

Lines 38-43 | Test: `every required message on an empty submit`

```
38 | it('shows required messages on an empty submit', () => {
39 |     component.onSubmit();
40 |     fixture.detectChanges();
41 |     ['Username is required', 'Email is required', 'Mobile number is required', 'Password is required',
42 |      'Confirm password is required', 'User role is required'].forEach((m) => expect(text()).toContain(m));
43 | });
```

Submits the empty form and loops over six messages with `forEach`: `Username is required`, `Email is required`, `Mobile number is required`, `Password is required`, `Confirm password is required`, `User role is required`. Each must be visible in the text. It protects the `Validators.required` rules on all six fields.

Lines 45-51 | Test: `format rules and password match`

```
45 | it('validates email, mobile, password length and matching passwords', () => {
46 |     component.signupForm.setValue({ username: 'a', email: 'aa', mobileNumber: '9', password: 'a', confirmPassword: 'b'
47 |     , userRole: '' });
48 |     component.onSubmit();
49 |     fixture.detectChanges();
50 |     ['Please enter a valid email address', 'Mobile number must be 10 digits',
51 |      'Password must be at least 6 characters long', 'Passwords do not match'].forEach((m) => expect(text()).toContain
52 |      (m));
53 | });
```

Fills the form with deliberately bad data: email `aa`, mobile `9`, password `a` that differs from confirm `b`, and an empty role. After a submit it expects `Please enter a valid email address`, `Mobile number must be 10 digits`, `Password must be at least 6 characters long` and `Passwords do not match`. The last one comes from a **form-level validator** that compares the two password fields, so this test protects the cross-field rule.

Lines 53-59 | Test: `duplicate email (HTTP 409)`

```
53 | it('shows the duplicate email message on a 409', () => {
54 |     component.signupForm.setValue(validValues);
55 |     component.onSubmit();
56 |     http.expectOne('http://localhost:8080/api/register').flush('A user with this email already exists', { status: 409,
57 |     statusText: 'Conflict' });
58 |     fixture.detectChanges();
59 |     expect(text()).toContain('A user with this email already exists');
```

Submits a valid form, then answers `POST http://localhost:8080/api/register` with status **409 Conflict** and the text `A user with this email already exists`. That is what `AuthController.registerUser` returns when `UserServiceImpl.createUser` finds the email already in the database. The test expects the same sentence on the page, protecting the error handler that turns the server's reply into a visible message.

Lines 61-71 | Test: success popup and the exact request body

```
61 | it('shows the success popup after registering', () => {
62 |     component.signupForm.setValue(validValues);
63 |     component.onSubmit();
64 |     const req = http.expectOne('http://localhost:8080/api/register');
65 |     expect(req.request.body.userRole).toBe('Customer');
66 |     expect(req.request.body.confirmPassword).toBeUndefined();
67 |     req.flush({ userId: 5 }, { status: 201, statusText: 'Created' });
68 |     fixture.detectChanges();
69 |     expect(text()).toContain('Registration Successful!');
70 | });
71 | });
```

Submits valid values and keeps the intercepted request in `req`. Before answering it checks the body that would be sent: `userRole` is `Customer` and `confirmPassword` is `undefined`. The second check matters: the confirm field exists only for the browser-side comparison and the component removes it before posting, because the backend `User` entity has no such field. Then `req.flush({ userId: 5 }, { status: 201 ... })` plays the backend's **201 Created** answer and the test expects the popup text `Registration Successful!`.

Workflow, next: Registration is verified. After login the user lands on the home page, whose test is next: `home-page.component.spec.ts`.

Step 120: home-page.component.spec.ts - tests for the home page [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/components/home-page/home-page.component.spec.ts

TEST

What this file is	The simplest component spec in the project. The home page is static information (about text and contact details), has no form, no service and no routing, so the test module needs almost nothing.
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>HomePageComponent</code> only. No <code>HttpClientTestingModule</code> , no router, no forms, because the page injects nothing.
Size	28 lines

Lines 1-3 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 |
3 | import { HomeComponent } from './home-page.component';
```

Fixture type and `TestBed`, plus the component under test. Nothing else is needed.

Lines 5-14 | describe + beforeEach

```
5 | describe('HomePageComponent', () => {
6 |     let component: HomeComponent;
7 |     let fixture: ComponentFixture<HomePageComponent>;
8 |
9 |     beforeEach(() => {
10 |         TestBed.configureTestingModule({ declarations: [HomePageComponent] });
11 |         fixture = TestBed.createComponent(HomePageComponent);
12 |         component = fixture.componentInstance;
13 |         fixture.detectChanges();
14 |     });
```

Declares only `HomePageComponent`, creates it and calls `detectChanges()`. If someone later adds a service or a `routerLink` to the page, this test will start failing with a "No provider" or "unknown element" error, which is a useful early warning that the test module must grow.

Lines 16-18 | Test: should create

```
16 | it('should create', () => {
17 |     expect(component).toBeTruthy();
18 | });
```

The component can be constructed.

Lines 20-28 | Test: information and contact details are shown

```
20 | it('shows the application information and contact details', () => {
21 |   const text = (fixture.nativeElement as HTMLElement).textContent || '';
22 |   expect(text).toContain('DriveU');
23 |   expect(text).toContain('Welcome to DriveU');
24 |   expect(text).toContain('Contact Us');
25 |   expect(text).toContain('987-654-3210');
26 |   expect(text).toContain('support@driveu.com');
27 | });
28 | });
```

Takes the whole visible text of the page and expects five strings: **DriveU**, **Welcome to DriveU**, **Contact Us**, the phone number **987-654-3210** and the email **support@driveu.com**. It protects the content the SRS requires on the home page (application information plus contact details).

Workflow, next: Home page verified. Both navigation bars appear on every logged-in page, so we test them next, admin first: **adminnav.component.spec.ts**.

Step 121: adminnav.component.spec.ts - tests for the admin navigation bar [Vaishalini - Customer Journey & Shared Infra]

angularapp/src/app/components/adminnav/adminnav.component.spec.ts

TEST

What this file is	Verifies the admin menu: it shows who is logged in, lists all admin links and uses a confirmation popup before logging out.
Who / what triggers it	Run by Karma during ng test .
What it uses / calls	AdminnavComponent , AuthService (to push fake username and role), RouterTestingModule (for routerLink) and HttpClientTestingModule .
Size	49 lines

Lines 1-6 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { RouterTestingModule } from '@angular/router/testing';
3 | import { HttpClientTestingModule } from '@angular/common/http/testing';
4 |
5 | import { AdminnavComponent } from './adminnav.component';
6 | import { AuthService } from '../services/auth.service';
```

Fake router (the nav uses **routerLink**), fake HTTP (because **AuthService** needs **HttpClient**), the component and the service whose streams the test will drive.

Lines 8-21 | describe + beforeEach

```
8 | describe('AdminnavComponent', () => {
9 |   let component: AdminnavComponent;
10 |   let fixture: ComponentFixture<AdminnavComponent>;
11 |
12 |   beforeEach(() => {
13 |     localStorage.clear();
14 |     TestBed.configureTestingModule({
15 |       imports: [RouterTestingModule, HttpClientTestingModule],
16 |       declarations: [AdminnavComponent]
17 |     });
18 |     fixture = TestBed.createComponent(AdminnavComponent);
19 |     component = fixture.componentInstance;
20 |     fixture.detectChanges();
21 |   });
```

Clears **localStorage**, declares **AdminnavComponent**, creates it and runs the first **detectChanges()**. The navbar starts with nobody logged in.

Line 23 | afterEach

```
23 |   afterEach(() => localStorage.clear());
```

Clears **localStorage** so nothing leaks.

Lines 25-27 | Test: should create

```
25 | it('should create', () => {
26 |   expect(component).toBeTruthy();
27 | });
```

The component can be constructed.

Lines 29-39 | Test: username, role and the admin menu

```
29 | it('shows the username and role and the admin menu', () => {
30 |   const auth = TestBed.inject(AuthService);
31 |   (auth as any).usernameSubject.next('adminname');
32 |   (auth as any).userRoleSubject.next('Admin');
33 |   fixture.detectChanges();
34 |   const el = fixture.nativeElement as HTMLElement;
35 |   expect(el.querySelector('#userInfo')?.textContent).toContain('adminname / Admin');
36 |   const text = el.textContent || '';
37 |   ['Home', 'Drivers', 'Add Driver', 'View Drivers', 'Requests', 'View Requests', 'Feedback', 'View Feedback', 'Logout',
38 |     t']
39 |     .forEach((label) => expect(text).toContain(label));
39 | });
```

Gets the shared `AuthService` and pushes 'adminname' into `usernameSubject` and 'Admin' into `userRoleSubject`, the same streams `AuthService.login` fills after a login. After `detectChanges()` the test finds the element with id `userInfo` (`el.querySelector('#userInfo')`) and expects its text to contain `adminname / Admin`. It then loops over nine labels: Home, Drivers, Add Driver, View Drivers, Requests, View Requests, Feedback, View Feedback and Logout, and expects each to appear in the page text. It protects the admin menu structure required by the SRS.

Lines 41-49 | Test: logout asks for confirmation

```
41 | it('asks for confirmation before logging out', () => {
42 |   component.openLogoutPopup();
43 |   fixture.detectChanges();
44 |   expect((fixture.nativeElement as HTMLElement).textContent).toContain('Are you sure you want to logout?');
45 |   component.cancelLogout();
46 |   fixture.detectChanges();
47 |   expect((fixture.nativeElement as HTMLElement).textContent).not.toContain('Are you sure you want to logout?');
48 | });
49 | });
```

`openLogoutPopup()` is what the Logout link calls. After `detectChanges()` the page must contain `Are you sure you want to logout?`. `cancelLogout()` (the No button) then closes it, and after another `detectChanges()` the text must be **gone** (`.not.toContain`). It protects the two-step logout so a stray click does not end the session.

Workflow, next: Admin navbar verified. Its customer twin is next: `customernav.component.spec.ts`.

Step 122: customernav.component.spec.ts - tests for the customer navigation bar [Vaishalini - Customer Journey & Shared Infra]

TEST

angularapp/src/app/components/customernav/customernav.component.spec.ts

What this file is	The same checks as the admin navbar spec, with the customer menu labels. The two files are intentionally almost identical because the two components are twins.
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>CustomernavComponent</code> , <code>AuthService</code> , <code>RouterTestingModule</code> , <code>HttpClientTestingModule</code> .
Size	49 lines

Lines 1-6 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { RouterTestingModule } from '@angular/router/testing';
3 | import { HttpClientTestingModule } from '@angular/common/http/testing';
4 |
5 | import { CustomernavComponent } from './customernav.component';
6 | import { AuthService } from '../services/auth.service';
```

Same pattern as the admin spec: fake router, fake HTTP, `CustomernavComponent` and `AuthService`.

Lines 8-21 | describe + beforeEach

```
8 | describe('CustomernavComponent', () => {
9 |   let component: CustomernavComponent;
10 |   let fixture: ComponentFixture<CustomernavComponent>;
11 |
12 |   beforeEach(() => {
13 |     localStorage.clear();
14 |     TestBed.configureTestingModule({
15 |       imports: [RouterTestingModule, HttpClientTestingModule],
16 |       declarations: [CustomernavComponent]
17 |     });
18 |     fixture = TestBed.createComponent(CustomernavComponent);
19 |     component = fixture.componentInstance;
20 |     fixture.detectChanges();
21 |   });
```

Clear storage, declare `CustomernavComponent`, create it and draw it once with nobody logged in.

Line 23 | afterEach

```
23 |   afterEach(() => localStorage.clear());
```

Clear storage again.

Lines 25-27 | Test: should create

```
25 |   it('should create', () => {
26 |     expect(component).toBeTruthy();
27 |   });
```

The component can be constructed.

Lines 29-39 | Test: username, role and the customer menu

```
29 |   it('shows the username and role and the customer menu', () => {
30 |     const auth = TestBed.inject(AuthService);
31 |     (auth as any).usernameSubject.next('demo');
32 |     (auth as any).userRoleSubject.next('Customer');
33 |     fixture.detectChanges();
34 |     const el = fixture.nativeElement as HTMLElement;
35 |     expect(el.querySelector('#userInfo')?.textContent).toContain('demo / Customer');
36 |     const text = el.textContent || '';
37 |     ['Home', 'Available Drivers', 'My Requests', 'Feedback', 'Logout']
38 |       .forEach((label) => expect(text).toContain(label));
39 |   });
```

Pushes 'demo' and 'Customer' into the service streams and expects the `#userInfo` element to read `demo / Customer`. It then expects five labels: Home, Available Drivers, My Requests, Feedback and Logout. The customer menu is shorter than the admin one because customers cannot add drivers or see all requests.

Lines 41-49 | Test: logout asks for confirmation

```
41 |   it('asks for confirmation before logging out', () => {
42 |     component.openLogoutPopup();
43 |     fixture.detectChanges();
44 |     expect((fixture.nativeElement as HTMLElement).textContent).toContain('Are you sure you want to logout?');
45 |     component.cancelLogout();
46 |     fixture.detectChanges();
47 |     expect((fixture.nativeElement as HTMLElement).textContent).not.toContain('Are you sure you want to logout?');
48 |   });
49 | });
```

Opens the logout popup, expects `Are you sure you want to logout?`, cancels it and expects the text to disappear. Same behaviour as the admin bar.

Workflow, next: Both navbars verified. Next the pages with real data: the admin's add/edit driver form, `driver-management.component.spec.ts`.

Step 123: driver-management.component.spec.ts - tests for the Add / Edit Driver form [Arjun - Driver & Booking]

TEST

angularapp/src/app/components/driver-management/driver-management.component.spec.ts

What this file is	Verifies the admin form that creates and edits drivers. The same component serves two modes depending on whether the URL contains a <code>driverId</code> , so the spec uses a <code>setup()</code> helper that builds the component either way. It checks validation, the exact HTTP calls (<code>POST /api/driver</code> for adding, <code>GET</code> then <code>PUT /api/driver/{id}</code> for editing) and the popups.
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>DriverManagementComponent</code> , <code>ReactiveFormsModule</code> , <code>RouterTestingModule</code> , <code>HttpClientTestingModule</code> with <code>HttpTestingController</code> , and a hand-made <code>ActivatedRoute</code> so the test can decide which URL parameter the page sees. In the real app the calls are answered by <code>DriverController.addDriver</code> , <code>viewDriverById</code> and <code>updateDriver</code> .
Size	78 lines

Lines 1-7 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { ReactiveFormsModule } from '@angular/forms';
3 | import { RouterTestingModule } from '@angular/router/testing';
4 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
5 | import { ActivatedRoute, convertToParamMap } from '@angular/router';
6 |
7 | import { DriverManagementComponent } from './driver-management.component';
```

Same testing imports as before, plus `ActivatedRoute` and `convertToParamMap`. `ActivatedRoute` is the Angular object that tells a page which route parameters it was opened with (for example `/driver-management/7` gives `driverId = 7`). `convertToParamMap` wraps a plain object in the `ParamMap` type the component expects when it calls `paramMap.get('driverId')`.

Lines 9-14 | describe, shared variables and text()

```
9 | describe('DriverManagementComponent', () => {
10 |   let component: DriverManagementComponent;
11 |   let fixture: ComponentFixture<DriverManagementComponent>;
12 |   let http: HttpTestingController;
13 |
14 |   const text = () => (fixture.nativeElement as HTMLElement).textContent || '';
```

Component, fixture and HTTP controller are shared. `text()` returns the visible page text. Notice there is **no** `beforeEach` in this file: the test module depends on the optional `driverId`, so each test builds its own module through `setup()`.

Lines 16-26 | The setup() helper

```
16 | const setup = (driverId?: string) => {
17 |   TestBed.configureTestingModule({
18 |     imports: [ReactiveFormsModule, RouterTestingModule, HttpClientTestingModule],
19 |     declarations: [DriverManagementComponent],
20 |     providers: [{ provide: ActivatedRoute, useValue: { snapshot: { paramMap: convertToParamMap(driverId ? { driverId
21 |       } : {}) } } } ]
22 |   });
23 |   fixture = TestBed.createComponent(DriverManagementComponent);
24 |   component = fixture.componentInstance;
25 |   http = TestBed.inject(HttpTestingController);
26 |   fixture.detectChanges();
27 | }
```

Takes an optional route parameter. It configures the test module and, in `providers`, **replaces** the real `ActivatedRoute` with a fake object `{ snapshot: { paramMap: ... } }` containing `{ driverId }` or an empty map. That one parameter decides the mode: with an id the component is in edit mode and loads the driver on start-up; without it, it shows the empty Add form. The helper then creates the component, grabs the HTTP controller and calls `detectChanges()` (which runs `ngOnInit`).

Lines 28-33 | Test: create and show the Add New Driver form

```
28 | it('should create and show the Add New Driver form', () => {
29 |     setup();
30 |     expect(component).toBeTruthy();
31 |     expect(text()).toContain('Add New Driver');
32 |     expect(text()).toContain('Add Driver');
33 | });
```

`setup()` with no id. Expects the component exists and that the page contains the heading **Add New Driver** and the button text **Add Driver**. This is add mode.

Lines 35-42 | Test: empty submit shows all messages

```
35 | it('shows "All fields are required" and field messages on an empty submit', () => {
36 |     setup();
37 |     component.onSubmit();
38 |     fixture.detectChanges();
39 |     expect(text()).toContain('All fields are required');
40 |     ['*Name is required', '*License Number is required', '*Experience is required', '*Contact Number is required',
41 |      '*Vehicle type is required', '*Hourly Rate is required', '*Address is required'].forEach((m) => expect(text()).t
42 | oContain(m));
43 | });
```

Calls `onSubmit()` on the empty form. The page must show the banner **All fields are required** and one message per field: ***Name is required**, ***License Number is required**, ***Experience is required**, ***Contact Number is required**, ***Vehicle type is required**, ***Hourly Rate is required**, ***Address is required**. It protects the seven **Validators.required** rules; the asterisk is part of the real message text on the page.

Lines 44-60 | Test: add a driver (POST) and the success popup

```
44 | it('adds a driver with the status "Active" and shows the success popup', () => {
45 |     setup();
46 |     component.driverForm.setValue({
47 |       driverName: 'demo name', licenseNumber: '1234567890987654', experienceYears: 3, contactNumber: '9123456789',
48 |       vehicleType: 'SUV', hourlyRate: 450, address: 'demo address'
49 |     });
50 |     component.imageBase64 = 'data:image/png;base64,AAAA';
51 |     component.onSubmit();
52 |     const req = http.expectOne('http://localhost:8080/api/driver');
53 |     expect(req.request.method).toBe('POST');
54 |     expect(req.request.body.availabilityStatus).toBe('Active');
55 |     req.flush({ driverId: 1 }, { status: 201, statusText: 'Created' });
56 |     fixture.detectChanges();
57 |     expect(text()).toContain('Driver Added Successfully!');
58 |     component.closeSuccessPopup();
59 |     expect(component.driverForm.value.driverName).toBe('');
60 | });
```

`driverForm.setValue({...})` fills all seven fields with valid data (the license is 16 characters, the contact number 10 digits). `component.imageBase64` is set by hand, standing in for the picture the user would pick in the file chooser (the image travels as a base64 text string). After `onSubmit()` the test expects exactly one request to **http://localhost:8080/api/driver** with method **POST**, whose body has `availabilityStatus` equal to **Active**. The form has no status field, so the component adds **Active** to every new driver. `req.flush({ driverId: 1 }, { status: 201 })` plays the backend's **201 Created**. The popup text **Driver Added Successfully!** must appear. Finally `closeSuccessPopup()` (the OK button) must reset the form, shown by `driverName` becoming an empty string, so the admin can add another driver straight away.

Lines 62-78 | Test: edit mode loads the driver and saves with PUT

```
62 | it('loads the driver into the form in edit mode', () => {
63 |     setup('7');
64 |     http.expectOne('http://localhost:8080/api/driver/7').flush({
65 |       driverId: 7, driverName: 'demo name three', licenseNumber: '2234567890987651', experienceYears: 7,
66 |       contactNumber: '9123456789', availabilityStatus: 'On Leave', address: 'demo address', vehicleType: 'Truck', hour
67 | lyRate: 330, image: ''
68 |     });
69 |     fixture.detectChanges();
70 |     expect(text()).toContain('Edit Driver');
71 |     expect(text()).toContain('Update Driver');
72 |     expect(component.driverForm.value.driverName).toBe('demo name three');
73 |     component.onSubmit();
74 |     const req = http.expectOne('http://localhost:8080/api/driver/7');
75 |     expect(req.request.method).toBe('PUT');
76 |     expect(req.request.body.availabilityStatus).toBe('On Leave');
77 |     req.flush({});
78 | });
```

`setup('7')` puts `driverId = 7` in the fake route, so on start-up the component calls **GET /api/driver/7**. The test answers it with a full driver (status **On Leave**). After `detectChanges()` the heading must say **Edit Driver**, the button **Update Driver**, and the form field `driverName` must be pre-filled with **demo name three**. Then `onSubmit()` must send **PUT** to **/api/driver/7** and the body must still carry `availabilityStatus On Leave`: the form has no status field, so

the component keeps the loaded status in its own field (`availabilityStatus`, default `Active`) and puts it back into the body; otherwise an edit would silently reset the status. This protects edit mode and the rule that editing never changes availability.

Workflow, next: Add/Edit Driver verified. The list that links to this form is next:

`admin-view-drivers.component.spec.ts`.

Step 124: admin-view-drivers.component.spec.ts - tests for the admin driver list [Arjun - Driver & Booking]

TEST

`angularapp/src/app/components/admin-view-drivers/admin-view-drivers.component.spec.ts`

What this file is	Verifies the admin page that lists driver cards: empty state, one card per driver, locked Edit/Delete buttons for inactive drivers, filtering, the Action menu that changes availability and the confirmed delete. The page loads its list with <code>GET /api/driver</code> (public endpoint, <code>DriverController.viewAllDrivers</code>).
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>AdminViewDriversComponent</code> , <code>FormsModule</code> (the page uses <code>[(ngModel)]</code> for the search box), <code>RouterTestingModule</code> , <code>HttpClientTestingModule</code> and <code>HttpTestingController</code> .
Size	86 lines

Lines 1-6 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { FormsModule } from '@angular/forms';
3 | import { RouterTestingModule } from '@angular/router/testing';
4 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
5 |
6 | import { AdminViewDriversComponent } from './admin-view-drivers.component';
```

`FormsModule` (template-driven forms) is imported because the template binds `[(ngModel)]`. The rest is the usual fake router and fake HTTP.

Lines 8-18 | describe, URL constant and sample drivers

```
8 | describe('AdminViewDriversComponent', () => {
9 |   let component: AdminViewDriversComponent;
10 |   let fixture: ComponentFixture<AdminViewDriversComponent>;
11 |   let http: HttpTestingController;
12 |
13 |   const url = 'http://localhost:8080/api/driver';
14 |   const text = () => (fixture.nativeElement as HTMLElement).textContent || '';
15 |   const drivers = [
16 |     { driverId: 1, driverName: 'demo name one', licenseNumber: '1234567890987654', experienceYears: 3, contactNumber:
17 |       '91234567899', availabilityStatus: 'Inactive', address: 'demo address', vehicleType: 'SUV', hourlyRate: 450, image: ''
18 |     },
19 |     { driverId: 2, driverName: 'demo name two', licenseNumber: '1234567890987655', experienceYears: 2, contactNumber:
20 |       '91234567893', availabilityStatus: 'Active', address: 'demo address', vehicleType: 'Sedan', hourlyRate: 230, image: ''
21 |     }
22 |   ];
23 | }
```

`url` is the list endpoint. `text()` returns the visible text. `drivers` is a hand-written list of two drivers: id 1 is `Inactive` (an SUV at 450 per hour) and id 2 is `Active` (a Sedan at 230). Having one of each status lets the tests check the lock rule. Their `image` is an empty string, so the card shows the placeholder picture.

Lines 20-29 | beforeEach: create the component and trigger the load

```
20 | beforeEach(() => {
21 |   TestBed.configureTestingModule({
22 |     imports: [FormsModule, RouterTestingModule, HttpClientTestingModule],
23 |     declarations: [AdminViewDriversComponent]
24 |   });
25 |   fixture = TestBed.createComponent(AdminViewDriversComponent);
26 |   component = fixture.componentInstance;
27 |   http = TestBed.inject(HttpTestingController);
28 |   fixture.detectChanges();
29 | });
```

Builds the module and the component and calls `detectChanges()`. That runs `ngOnInit`, which immediately sends `GET /api/driver`. The request is left **unanswered** here; each test answers it itself with the data it wants, using `http.expectOne(url).flush(...)`.

Lines 31-34 | Test: should create

```
31 | it('should create', () => {
32 |   http.expectOne(url).flush(null);
33 |   expect(component).toBeTruthy();
34 | });
```

Answers the pending request with **null** (an empty list) and expects the component to exist. Answering is necessary so no request stays open.

Lines 36-41 | Test: empty state (HTTP 204)

```
36 | it('shows "Oops! No data found." when there are no drivers', () => {
37 |   http.expectOne(url).flush(null, { status: 204, statusText: 'No Content' });
38 |   fixture.detectChanges();
39 |   expect(text()).toContain('Drivers');
40 |   expect(text()).toContain('Oops! No data found. ');
41 | });
```

Answers with **null** and status **204 No Content**, exactly what `viewAllDrivers` returns when there are no drivers. The page must still show its heading **Drivers** and the message **Oops! No data found.**

Lines 43-50 | Test: one card per driver, lock for inactive

```
43 | it('renders a card per driver and locks Edit/Delete for inactive drivers', () => {
44 |   http.expectOne(url).flush(drivers);
45 |   fixture.detectChanges();
46 |   const el = fixture.nativeElement as HTMLElement;
47 |   expect(el.querySelectorAll('.driver-card').length).toBe(2);
48 |   expect((el.querySelector('#edit-1') as HTMLButtonElement).disabled).toBeTrue();
49 |   expect((el.querySelector('#edit-2') as HTMLButtonElement).disabled).toBeFalse();
50 | });
```

Answers with both drivers. `querySelectorAll('.driver-card').length` must be 2. The Edit button of driver 1 (`#edit-1`) must be **disabled** (it is Inactive) while `#edit-2` is enabled. This protects the rule that inactive drivers cannot be edited or deleted. `toBeTrue()` / `toBeFalse()` are strict boolean matchers.

Lines 52-61 | Test: filters by name and by status

```
52 | it('filters by name and by status', () => {
53 |   http.expectOne(url).flush(drivers);
54 |   component.searchText = 'two';
55 |   component.applyFilters();
56 |   expect(component.filteredDrivers.length).toBe(1);
57 |   component.searchText = '';
58 |   component.statusFilter = 'Inactive';
59 |   component.applyFilters();
60 |   expect(component.filteredDrivers[0].driverName).toBe('demo name one');
61 | });
```

Sets `searchText = 'two'` and calls `applyFilters()`; only one driver (**demo name two**) must remain in `filteredDrivers`. Then it clears the text, sets `statusFilter = 'Inactive'` and expects the first filtered driver to be **demo name one**. This protects the search box and the status dropdown.

Lines 63-72 | Test: Action menu and status change (PUT)

```
63 | it('lists the other statuses in the Action menu and updates the driver', () => {
64 |   http.expectOne(url).flush(drivers);
65 |   expect(component.otherStatuses(drivers[1] as any)).toEqual(['Inactive', 'On Leave']);
66 |   component.changeStatus(component.drivers[1], 'On Leave');
67 |   const req = http.expectOne(`${url}/2`);
68 |   expect(req.request.method).toBe('PUT');
69 |   expect(req.request.body.availabilityStatus).toBe('On Leave');
70 |   req.flush({});
71 |   expect(component.drivers[1].availabilityStatus).toBe('On Leave');
72 | });
```

`otherStatuses(driver2)` must return the statuses the driver does **not** currently have: `['Inactive', 'On Leave']` (the full list is Active, Inactive, On Leave). `changeStatus(driver, 'On Leave')` must send **PUT** `/api/driver/2` with the new `availabilityStatus` in the body (answered by `DriverController.updateDriver`). After the fake 200 reply the in-memory driver must show **On Leave** without reloading the list.

Lines 74-86 | Test: confirmed delete

```
74 | it('asks for confirmation and deletes a driver', () => {
75 |     http.expectOne(url).flush(drivers);
76 |     component.openDeletePopup(component.drivers[1]);
77 |     fixture.detectChanges();
78 |     expect(text()).toContain('Are you sure you want to delete this driver?');
79 |     component.confirmDelete();
80 |     const req = http.expectOne(`${url}/2`);
81 |     expect(req.request.method).toBe('DELETE');
82 |     req.flush({});
83 |     http.expectOne(url).flush([drivers[0]]);
84 |     expect(component.drivers.length).toBe(1);
85 | });
86 | });
```

`openDeletePopup(driver2)` shows `Are you sure you want to delete this driver?`. `confirmDelete()` must send **DELETE** `/api/driver/2` (`DriverController.deleteDriver`). After the fake reply the component reloads the list, so the test expects a **second GET** `/api/driver` and answers it with only driver 1; the component must then hold exactly 1 driver. It protects the confirm-then-delete-then-refresh sequence.

Workflow, next: Admin driver list verified. The customer's version of the list, with request buttons and AI search, is next: `customerviewdriver.component.spec.ts`.

Step 125: customerviewdriver.component.spec.ts - tests for the customer driver list [Arjun - Driver & Booking + Saumya (AI search)]

TEST

angularapp/src/app/components/customerviewdriver/customerviewdriver.component.spec.ts

What this file is	Verifies the customer page "Available Drivers": the list, the empty state, the rule for which drivers can be requested, the redirect to the request form, text search and the AI search box. On start-up the page makes two calls: GET <code>/api/driver</code> and GET <code>/api/driverRequest/user/{userId}</code> (the customer's own requests, so it can mark already requested drivers).
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>CustomerviewdriverComponent</code> , <code>FormsModule</code> , <code>RouterTestingModule</code> , <code>HttpClientTestingModule</code> , <code>HttpTestingController</code> and the real <code>Router</code> (spied). Backend answers: <code>DriverController.viewAllDrivers</code> , <code>DriverRequestController.viewDriverRequestsByUserId</code> (404 when there are none) and <code>AiController.searchDrivers</code> .
Size	101 lines

Lines 1-7 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { FormsModule } from '@angular/forms';
3 | import { RouterTestingModule } from '@angular/router/testing';
4 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
5 | import { Router } from '@angular/router';
6 |
7 | import { CustomerviewdriverComponent } from './customerviewdriver.component';
```

Fixture, forms, fake router, fake HTTP with its controller, `Router` (for the spy) and the component.

Lines 9-20 | describe, base URL and three sample drivers

```
9 | describe('CustomerviewdriverComponent', () => {
10 |     let component: CustomerviewdriverComponent;
11 |     let fixture: ComponentFixture<CustomerviewdriverComponent>;
12 |     let http: HttpTestingController;
13 |
14 |     const base = 'http://localhost:8080/api';
15 |     const text = () => (fixture.nativeElement as HTMLElement).textContent || '';
16 |     const drivers = [
17 |         { driverId: 1, driverName: 'demo name one', licenseNumber: '123', experienceYears: 3, contactNumber: '91', availabilityStatus: 'Inactive', address: 'a', vehicleType: 'SUV', hourlyRate: 450, image: '' },
18 |         { driverId: 2, driverName: 'demo name two', licenseNumber: '124', experienceYears: 2, contactNumber: '92', availabilityStatus: 'Active', address: 'a', vehicleType: 'Sedan', hourlyRate: 230, image: '' },
19 |         { driverId: 3, driverName: 'demo name three', licenseNumber: '125', experienceYears: 7, contactNumber: '93', availabilityStatus: 'Active', address: 'a', vehicleType: 'Truck', hourlyRate: 330, image: '' }
20 |     ];
```

`base` is the API root. The three drivers: id 1 **Inactive** SUV, id 2 **Active** Sedan, id 3 **Active** Truck, with simple placeholders for the other fields. Their mix drives the "who can be requested" test.

Lines 22-36 | The `init()` helper

```
22 | const init = (requests: any[] | null, status = 200) => {
23 |   localStorage.setItem('userId', '2');
24 |   fixture = TestBed.createComponent(CustomerviewdriverComponent);
25 |   component = fixture.componentInstance;
26 |   http = TestBed.inject(HttpTestingController);
27 |   fixture.detectChanges();
28 |   http.expectOne(`${base}/driver`).flush(drivers);
29 |   const req = http.expectOne(`${base}/driverRequest/user/2`);
30 |   if (status === 404) {
31 |     req.flush('', { status: 404, statusText: 'Not Found' });
32 |   } else {
33 |     req.flush(requests);
34 |   }
35 |   fixture.detectChanges();
36 | };
```

Takes the customer's existing requests (or `null`) and a status. It stores `userId` 2 in `localStorage` (the page reads it to build the second URL), creates the component and runs `detectChanges()`, which fires both start-up requests. It answers `GET /api/driver` with the three drivers, then picks up `GET /api/driverRequest/user/2`. If `status` is 404 it answers with an HTTP 404 error, simulating a customer who has never requested anything; otherwise it answers with the supplied list. A final `detectChanges()` draws the cards. Using one helper keeps every test short.

Lines 38-44 | `beforeEach`

```
38 | beforeEach(() => {
39 |   localStorage.clear();
40 |   TestBed.configureTestingModule({
41 |     imports: [FormsModule, RouterTestingModule, HttpClientTestingModule],
42 |     declarations: [CustomerviewdriverComponent]
43 |   });
44 | });
```

Clears `localStorage` and configures the test module. The component is not created here because `init()` must first store the user id.

Line 46 | `afterEach`

```
46 | afterEach(() => localStorage.clear());
```

Clears `localStorage`.

Lines 48-53 | Test: create and list drivers

```
48 | it('should create and list drivers', () => {
49 |   init([]);
50 |   expect(component).toBeTruthy();
51 |   expect(text()).toContain('Available Drivers');
52 |   expect(fixture.nativeElement.querySelectorAll('.driver-card').length).toBe(3);
53 | });
```

`init([])` (no requests). Expects the heading **Available Drivers** and exactly three `.driver-card` elements.

Lines 55-64 | Test: no drivers (204) shows "Oops! No records found"

```
55 | it('shows "Oops! No records found" when there are no drivers', () => {
56 |   localStorage.setItem('userId', '2');
57 |   fixture = TestBed.createComponent(CustomerviewdriverComponent);
58 |   http = TestBed.inject(HttpTestingController);
59 |   fixture.detectChanges();
60 |   http.expectOne(`${base}/driver`).flush(null, { status: 204, statusText: 'No Content' });
61 |   http.expectOne(`${base}/driverRequest/user/2`).flush('', { status: 404, statusText: 'Not Found' });
62 |   fixture.detectChanges();
63 |   expect(text()).toContain('Oops! No records found');
64 | });
```

This test builds the component itself instead of using `init()`, because it needs to answer the driver list with **204 No Content** (`null` body) and the request list with a 404. The page must then show **Oops! No records found**.

Lines 66-73 | Test: who can be requested

```
66 | it('only enables Request for active drivers that are not already requested', () => {
67 |   init([ { driverRequestId: 9, status: 'Pending', driver: { driverId: 3 } } ]);
68 |   const el = fixture.nativeElement as HTMLElement;
69 |   expect((el.querySelector('#request-1') as HTMLButtonElement).disabled).toBeTrue();
70 |   expect((el.querySelector('#request-2') as HTMLButtonElement).disabled).toBeFalse();
71 |   expect(el.querySelector('#request-3')).toBeNull();
72 |   expect(el.querySelector('#requested-3')?.textContent).toContain('Requested');
73 | });
```

The customer already has a **Pending** request for driver 3. The expectations: `#request-1` is disabled (driver 1 is Inactive), `#request-2` is enabled (Active, not yet requested), `#request-3` does **not exist** (`toBeNull`) and instead an

element `#requested-3` containing the word `Requested` is shown. This protects the rule that a driver can only be requested when Active and not already requested by this customer.

Lines 75-80 | Test: 404 means "no requests yet", and the request button navigates

```
75 | it('handles a 404 (no requests yet) and navigates to the request form', () => {
76 |   init(null, 404);
77 |   const navigate = spyOn(TestBed.inject(Router), 'navigate').and.returnValue(Promise.resolve(true));
78 |   component.requestDriver(component.drivers[1]);
79 |   expect(navigate).toHaveBeenCalledOnceWith(['/customer-request', 2]);
80 | });
```

`init(null, 404)` proves a 404 on the request list does not break the page (a customer with no requests gets 404 from `viewDriverRequestsByUserId`, which is a normal situation here). A spy replaces `Router.navigate`; `requestDriver(driver2)` must call `navigate(['/customer-request', 2])`, that is the request form for driver id 2.

Lines 82-86 | Test: text search

```
82 | it('filters by the search text', () => {
83 |   init([]);
84 |   component.searchText = 'truck';
85 |   expect(component.displayedDrivers.length).toBe(1);
86 | });
```

Sets `searchText = 'truck'`; the getter `displayedDrivers` must then contain exactly 1 driver (the Truck). The search box matches the name, the vehicle type and the license number.

Lines 88-101 | Test: AI search and clear

```
88 | it('runs the AI search and can clear it', () => {
89 |   init([]);
90 |   component.aiQuery = 'big truck for moving';
91 |   component.runAiSearch();
92 |   const req = http.expectOne(`${base}/ai/driver-search`);
93 |   expect(req.request.body).toBe('big truck for moving');
94 |   req.flush([drivers[2]]);
95 |   fixture.detectChanges();
96 |   expect(component.displayedDrivers.length).toBe(1);
97 |   expect(text()).toContain('Showing AI best matches for');
98 |   component.clearAiSearch();
99 |   expect(component.displayedDrivers.length).toBe(3);
100 | });
101 | });
```

Sets `aiQuery` to `big truck for moving` and calls `runAiSearch()`. The test expects one request to `/api/ai/driver-search` whose **body is the plain query string** (that is how `AiController.searchDrivers` receives it). It answers with `[driver3]`; `displayedDrivers` must then be 1 and the page must say `Showing AI best matches for`. `clearAiSearch()` must bring back all 3 drivers. It protects both the request format and the switch between normal and AI-ranked lists.

Workflow, next: Customer driver list verified. The form it leads to is next: `customer-request.component.spec.ts`.

Step 126: customer-request.component.spec.ts - tests for the trip request form [Arjun - Driver & Booking (create) + Vaishalini (edit mode)]

`angularapp/src/app/components/customer-request/customer-request.component.spec.ts`

TEST

What this file is	Verifies the form where a customer requests a driver. Like the driver form it has two modes: new (route parameter <code>driverId</code>) and edit (route parameter <code>driverRequestId</code>). It checks validation, the "no trip date in the past" rule, the exact JSON sent to <code>POST /api/driverRequest</code> , and loading + saving through <code>GET/PUT /api/driverRequest/{id}</code> .
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>CustomerRequestComponent</code> , <code>ReactiveFormsModule</code> , fake router and HTTP, and a hand-made <code>ActivatedRoute</code> . Backend answers: <code>DriverRequestController.addDriverRequest</code> , <code>viewDriverRequestById</code> and <code>updateDriverRequest</code> .
Size	91 lines

Lines 1-7 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { ReactiveFormsModule } from '@angular/forms';
3 | import { RouterTestingModule } from '@angular/router/testing';
4 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
5 | import { ActivatedRoute, convertToParamMap } from '@angular/router';
6 |
7 | import { CustomerRequestComponent } from './customer-request.component';
```

The usual testing imports plus **ActivatedRoute** and **convertToParamMap** for the fake route.

Lines 9-20 | describe, URL, text() and the future() helper

```
9 | describe('CustomerRequestComponent', () => {
10 |   let component: CustomerRequestComponent;
11 |   let fixture: ComponentFixture<CustomerRequestComponent>;
12 |   let http: HttpTestingController;
13 |
14 |   const url = 'http://localhost:8080/api/driverRequest';
15 |   const text = () => (fixture.nativeElement as HTMLElement).textContent || '';
16 |   const future = () => {
17 |     const d = new Date();
18 |     d.setDate(d.getDate() + 5);
19 |     return d.toISOString().substring(0, 10);
20 |   };
```

url is **/api/driverRequest**. **future()** returns today plus five days as a **YYYY-MM-DD** string (**toISOString().substring(0, 10)**). It is computed at run time so the test never breaks as the calendar moves on; a hard-coded date would eventually become "in the past" and fail the validator.

Lines 22-33 | The setup() helper

```
22 | const setup = (params: { [key: string]: string }) => {
23 |   localStorage.setItem('userId', '2');
24 |   TestBed.configureTestingModule({
25 |     imports: [ReactiveFormsModule, RouterTestingModule, HttpClientTestingModule],
26 |     declarations: [CustomerRequestComponent],
27 |     providers: [{ provide: ActivatedRoute, useValue: { snapshot: { paramMap: convertToParamMap(params) } } } ]
28 |   });
29 |   fixture = TestBed.createComponent(CustomerRequestComponent);
30 |   component = fixture.componentInstance;
31 |   http = TestBed.inject(HttpTestingController);
32 |   fixture.detectChanges();
33 | };
```

Takes a parameter object such as **{ driverId: '3' }** or **{ driverRequestId: '9' }**. It stores **userId** 2 in **localStorage** (the form needs it to say who is requesting), builds the module with the fake **ActivatedRoute** whose **paramMap** contains those parameters, creates the component and calls **detectChanges()**.

Line 35 | afterEach

```
35 | afterEach(() => localStorage.clear());
```

Clears **localStorage**.

Lines 37-42 | Test: create and show the New Driver Request form

```
37 | it('should create and show the New Driver Request form', () => {
38 |   setup({ driverId: '3' });
39 |   expect(component).toBeTruthy();
40 |   expect(text()).toContain('New Driver Request');
41 |   expect(text()).toContain('Submit Request');
42 | });
```

New mode. The page must contain the heading **New Driver Request** and the button **Submit Request**.

Lines 44-50 | Test: validation messages on an empty submit

```
44 | it('shows validation messages on an empty submit', () => {
45 |   setup({ driverId: '3' });
46 |   component.onSubmit();
47 |   fixture.detectChanges();
48 |   ['*Pickup Location is required', '*Drop Location is required', '*Trip Date is required', '*Time Slot is required',
49 |     '*Estimated Duration is required', 'All required fields must be filled out'].forEach((m) => expect(text()).toContain(m));
50 | });
```

Submits the empty form and expects five field messages (***Pickup Location is required**, ***Drop Location is required**, ***Trip Date is required**, ***Time Slot is required**, ***Estimated Duration is required**) and the banner **All required fields must be filled out**. The optional comments field has no message, as expected.

Lines 52-56 | Test: past trip dates are rejected

```
52 | it('rejects a trip date in the past', () => {
53 |   setup({ driverId: '3' });
54 |   component.requestForm.patchValue({ tripDate: '2000-01-01' });
55 |   expect(component.f['tripDate'].errors?.['pastDate']).toBeTrue();
56 | });
```

`patchValue` sets `tripDate` to `2000-01-01`. `component.f` is a shortcut getter for `requestForm.controls`; the test expects the control to carry the error key `pastDate`, produced by the custom validator that compares the chosen date with today. (In edit mode a date that was already saved is exempt, otherwise old requests could never be edited.)

Lines 58-70 | Test: submit builds the nested request (POST)

```
58 | it('submits a nested request for the selected driver', () => {
59 |   setup({ driverId: '3' });
60 |   component.requestForm.setValue({ pickupLocation: 'demo pick 2', dropLocation: 'demo drop 2', tripDate: future(), t
61 |     imeSlot: '22:56', estimatedDuration: '3 hours', comments: 'demo comments' });
62 |   component.onSubmit();
63 |   const req = http.expectOne(url);
64 |   expect(req.request.method).toBe('POST');
65 |   expect(req.request.body.user).toEqual({ userId: 2 });
66 |   expect(req.request.body.driver).toEqual({ driverId: 3 });
67 |   expect(req.request.body.status).toBe('Pending');
68 |   req.flush({ status: 201, statusText: 'Created' });
69 |   fixture.detectChanges();
70 |   expect(text()).toContain('Request Submitted Successfully!');
```

Fills all six fields with valid values, using `future()` for the date, then submits. One **POST** to `/api/driverRequest` must be sent. The body must contain `user` equal to `{ userId: 2 }` and `driver` equal to `{ driverId: 3 }`; the backend `DriverRequest` entity has `@ManyToOne` links to `User` and `Driver`, so JSON has to carry them as nested objects (not as flat `userId/driverId`). The body must also say `status: 'Pending'`. After a fake **201 Created** the popup `Request Submitted Successfully!` must show.

Lines 72-91 | Test: edit mode loads and updates (GET then PUT)

```
72 | it('loads and updates an existing request in edit mode', () => {
73 |   setup({ driverRequestId: '9' });
74 |   http.expectOne(`${url}/9`).flush({
75 |     driverRequestId: 9, status: 'Pending', pickupLocation: 'demo pick 4', dropLocation: 'demo drop 4', tripDate: '20
76 |     25-01-25',
77 |     timeSlot: '14:35:00', estimatedDuration: '2 hours', comments: 'demo comments', driver: { driverId: 3 }
78 |   });
79 |   fixture.detectChanges();
80 |   expect(text()).toContain('Edit Request');
81 |   expect(text()).toContain('Update Request');
82 |   expect(component.requestForm.value.timeSlot).toBe('14:35');
83 |   component.requestForm.patchValue({ comments: 'demo comments updated' });
84 |   component.onSubmit();
85 |   const req = http.expectOne(`${url}/9`);
86 |   expect(req.request.method).toBe('PUT');
87 |   expect(req.request.body.comments).toBe('demo comments updated');
88 |   req.flush({});
89 |   fixture.detectChanges();
90 |   expect(text()).toContain('Request Updated Successfully!');
91 | });
```

`setup({ driverRequestId: '9' })` makes the form call `GET /api/driverRequest/9`; the test answers it with an existing request whose `timeSlot` is `14:35:00`. After drawing, the heading must be `Edit Request` and the button `Update Request`, and the time field must hold `14:35` (the browser time input wants hours and minutes only, so the component trims the seconds). The test then patches `comments`, submits, and expects **PUT** `/api/driverRequest/9` with the new comment in the body. After a fake 200 the popup `Request Updated Successfully!` must show.

Workflow, next: The customer can now create requests; the admin handles them in `adminviewrequests.component.spec.ts`.

Step 127: adminviewrequests.component.spec.ts - tests for the admin request table [Aarti - Admin Requests]

TEST

angularapp/src/app/components/adminviewrequests/adminviewrequests.component.spec.ts

What this file is	Verifies the admin page "Driver Requests for Approval": the table, date formatting, which actions are allowed per status (Pending / Approved / Rejected / Trip End / Closed), approve, filters and the "Request Progression" popup that also lets the admin close a request. The page loads with <code>GET /api/driverRequest</code> (admin only, <code>DriverRequestController.viewAllDriverRequests</code>).
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>AdminviewrequestsComponent</code> , <code>FormsModule</code> , <code>HttpClientTestingModule</code> with its controller. This component has no router links, so <code>RouterTestingModule</code> is not imported.
Size	100 lines

Lines 1-5 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { FormsModule } from '@angular/forms';
3 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
4 |
5 | import { AdminviewrequestsComponent } from '../adminviewrequests.component';
```

Fixture and `TestBed`, `FormsModule` for the search box, the fake HTTP pair and the component.

Lines 7-18 | describe, URL, a driver and a request factory

```
7 | describe('AdminviewrequestsComponent', () => {
8 |   let component: AdminviewrequestsComponent;
9 |   let fixture: ComponentFixture<AdminviewrequestsComponent>;
10 |   let http: HttpTestingController;
11 |
12 |   const url = 'http://localhost:8080/api/driverRequest';
13 |   const text = () => (fixture.nativeElement as HTMLElement).textContent || '';
14 |   const driver = { driverId: 3, driverName: 'demo name three', licenseNumber: '22', experienceYears: 7, contactNumber:
15 |     '91', vehicleType: 'Truck', hourlyRate: 330, address: 'demo address' };
16 |   const requests = () => [
17 |     { driverRequestId: 1, user: { username: 'demo' }, driver, status: 'Pending', pickupLocation: 'demo pick 3', dropLo
18 |       cation: 'demo drop 3', tripDate: '2025-01-26' },
19 |     { driverRequestId: 2, user: { username: 'demo' }, driver, status: 'Trip End', pickupLocation: 'demo pick 2', dropL
20 |       ocation: 'demo drop 2', tripDate: [2025, 1, 25] }
21 |   ];
```

`driver` is a minimal driver object. `requests` is declared as a **function** that returns two fresh requests each time: id 1 is `Pending` with `tripDate` as the string `2025-01-26`; id 2 is `Trip End` with `tripDate` as the **array** `[2025, 1, 25]`. The array form is what Spring/Jackson produces for a `LocalDate`, so the test covers both shapes. Returning new objects each call means one test changing a status cannot affect another test.

Lines 20-29 | beforeEach

```
20 | beforeEach(() => {
21 |   TestBed.configureTestingModule({
22 |     imports: [FormsModule, HttpClientTestingModule],
23 |     declarations: [AdminviewrequestsComponent]
24 |   });
25 |   fixture = TestBed.createComponent(AdminviewrequestsComponent);
26 |   component = fixture.componentInstance;
27 |   http = TestBed.inject(HttpTestingController);
28 |   fixture.detectChanges();
29 | });
```

Builds the module and component. `detectChanges()` runs `ngOnInit`, which sends `GET /api/driverRequest`; each test answers it.

Lines 31-34 | Test: should create

```
31 | it('should create', () => {
32 |   http.expectOne(url).flush(null);
33 |   expect(component).toBeTruthy();
34 | });
```

Answers with `null` and expects the component to exist.

Lines 36-41 | Test: empty state (204)

```
36 | it('shows "Oops! No records found" when there is no data', () => {
37 |   http.expectOne(url).flush(null, { status: 204, statusText: 'No Content' });
38 |   fixture.detectChanges();
39 |   expect(text()).toContain('Driver Requests for Approval');
40 |   expect(text()).toContain('Oops! No records found');
41 | });
```

Answers 204. The heading **Driver Requests for Approval** must be there together with **Oops! No records found**.

Lines 43-48 | Test: table rows and formatted dates

```
43 | it('lists the requests with formatted trip dates', () => {
44 |   http.expectOne(url).flush(requests());
45 |   fixture.detectChanges();
46 |   expect(fixture.nativeElement.querySelectorAll('tbody tr').length).toBe(2);
47 |   expect(text()).toContain('2025-01-25');
48 | });
```

Answers with the two requests. The table body must have 2 **tr** rows, and the text must contain **2025-01-25**: the array **[2025, 1, 25]** is converted to a **YYYY-MM-DD** string by **toDateString** from **utils/format.ts**, which the template calls.

Lines 50-55 | Test: which actions each status allows

```
50 | it('offers Approve/Reject depending on the status', () => {
51 |   http.expectOne(url).flush(requests());
52 |   const [pending, tripEnd] = component.requests;
53 |   expect(component.canApprove(pending) && component.canReject(pending)).toBeTrue();
54 |   expect(component.canApprove(tripEnd) || component.canReject(tripEnd)).toBeFalse();
55 | });
```

For the Pending request both **canApprove** and **canReject** must be true. For the Trip End request both must be false. This is the status rule: Approve/Reject only while Pending (and Approved/Rejected can be switched), nothing after Trip End.

Lines 57-65 | Test: approve sends a PUT with only the status

```
57 | it('approves a request', () => {
58 |   http.expectOne(url).flush(requests());
59 |   component.approve(component.requests[0]);
60 |   const req = http.expectOne(`${url}/1`);
61 |   expect(req.request.method).toBe('PUT');
62 |   expect(req.request.body).toEqual({ status: 'Approved' });
63 |   req.flush({});
64 |   expect(component.requests[0].status).toBe('Approved');
65 | });
```

approve(request) must send **PUT /api/driverRequest/1** and the body must equal exactly **{ status: 'Approved' }**. Sending only that field works because **DriverRequestServiceImpl.updateDriverRequest** merges only non-null fields. After the fake reply the request in memory must show **Approved**.

Lines 67-76 | Test: search and status filters

```
67 | it('filters by driver name / pickup location and by status', () => {
68 |   http.expectOne(url).flush(requests());
69 |   component.searchText = 'pick 2';
70 |   component.applyFilters();
71 |   expect(component.filteredRequests.length).toBe(1);
72 |   component.searchText = '';
73 |   component.statusFilter = 'Pending';
74 |   component.applyFilters();
75 |   expect(component.filteredRequests[0].driverRequestId).toBe(1);
76 | });
```

Searching **pick 2** (a pickup location) with **applyFilters()** must leave 1 request. Clearing it and filtering by status **Pending** must leave request id 1.

Lines 78-91 | Test: progression popup and closing a request

```
78 | it('shows the progression and closes a request at "Trip End"', () => {
79 |   http.expectOne(url).flush(requests());
80 |   const tripEnd = component.requests[1];
81 |   component.openStages(tripEnd);
82 |   fixture.detectChanges();
83 |   expect(text()).toContain('Request Progression');
84 |   expect(component.stageClass('Trip End')).toBe('done');
85 |   expect(component.stageClass('Closed')).toBe('idle');
86 |   expect(fixture.nativeElement.querySelector('#closeRequestButton')).toBeTruthy();
87 |   component.closeRequest();
88 |   http.expectOne(`${url}/2`).flush({});
89 |   expect(tripEnd.status).toBe('Closed');
90 |   expect(component.showStageModal).toBeFalse();
91 | });
```

`openStages(tripEnd)` opens the modal titled **Request Progression**. `stageClass('Trip End')` must be **done** and `stageClass('Closed')` must be **idle**, the CSS classes that colour the timeline. For a request at Trip End the button with id `closeRequestButton` must exist. `closeRequest()` must send a PUT to `/api/driverRequest/2`; after the fake reply the status must be **Closed** and the modal flag `showStageModal` must be false. This protects the last link of the chain: only the admin closes, and only from Trip End.

Lines 93-100 | Test: a rejected request shows only two stages

```
93 | it('shows Pending and Rejected stages for a rejected request', () => {
94 |   http.expectOne(url).flush(requests());
95 |   component.requests[0].status = 'Rejected';
96 |   component.openStages(component.requests[0]);
97 |   expect(component.visibleStages).toEqual(['Pending', 'Rejected']);
98 |   expect(component.stageClass('Rejected')).toBe('rejected');
99 | });
100 | });
```

After setting the first request to **Rejected** and opening its stages, `visibleStages` must be `['Pending', 'Rejected']` and `stageClass('Rejected')` must be **rejected** (drawn red). A rejected request never reaches Approved or Trip End, so the timeline shortens.

Workflow, next: Admin side verified. The customer's own request list, where the trip is ended and paid, is next: `customerviewrequested.component.spec.ts`.

Step 128: customerviewrequested.component.spec.ts - tests for "My Requests" [Vaishalini - Customer Journey & Shared Infra + Prajan (Trip End / Pay Amount)]

`angularapp/src/app/components/customerviewrequested/customerviewrequested.component.spec.ts`

TEST

What this file is	Verifies the customer's request cards and the late part of the status chain: Edit/Delete only while Pending, Trip End only while Approved, the fare calculation at Trip End, the payment popup (Pay Amount) and the jump to the review form. The page loads with GET <code>/api/driverRequest/user/{userId}</code> (<code>DriverRequestController.viewDriverRequestsByUserId</code>).
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>CustomerviewrequestedComponent</code> , <code>FormsModule</code> , fake router and HTTP, the spied Router , and Angular's <code>fakeAsync</code> and <code>tick</code> for the timer test.
Size	103 lines

Lines 1-7 | Imports

```
1 | import { ComponentFixture, TestBed, fakeAsync, tick } from '@angular/core/testing';
2 | import { FormsModule } from '@angular/forms';
3 | import { RouterTestingModule } from '@angular/router/testing';
4 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
5 | import { Router } from '@angular/router';
6 |
7 | import { CustomerviewrequestedComponent } from './customerviewrequested.component';
```

Besides the usual items this file imports `fakeAsync` and `tick`. `fakeAsync` runs a test inside a controlled clock; `tick(ms)` moves that clock forward instantly, so a `setTimeout` of 1.5 seconds inside the component can be tested without really waiting.

Lines 9-21 | describe, helpers and three sample requests

```
9 | describe('CustomerviewrequestedComponent', () => {
10 |   let component: CustomerviewrequestedComponent;
11 |   let fixture: ComponentFixture<CustomerviewrequestedComponent>;
12 |   let http: HttpTestingController;
13 |
14 |   const url = 'http://localhost:8080/api/driverRequest';
15 |   const el = () => fixture.nativeElement as HTMLElement;
16 |   const driver = { driverId: 3, driverName: 'demo name three', vehicleType: 'Truck', hourlyRate: 300, contactNumber: '93' };
17 |   const requests = () => [
18 |     { driverRequestId: 1, status: 'Pending', pickupLocation: 'a', dropLocation: 'b', tripDate: '2025-01-25', timeSlot: '10:00:00', estimatedDuration: '2 hours', driver },
19 |     { driverRequestId: 2, status: 'Approved', pickupLocation: 'c', dropLocation: 'd', tripDate: '2025-01-26', timeSlot: '10:00:00', estimatedDuration: '2 hours', driver },
20 |     { driverRequestId: 3, status: 'Trip End', pickupLocation: 'e', dropLocation: 'f', tripDate: '2025-01-27', timeSlot: '10:00:00', estimatedDuration: '2 hours', paymentAmount: 600, actualDuration: '2 hours 0 minutes', driver }
21 |   ];
```

`el()` returns the component's HTML element. `driver` is a Truck driver at an hourly rate of 300. The `requests()` factory returns three requests at the three interesting stages: id 1 **Pending**, id 2 **Approved**, id 3 **Trip End** (which already has `paymentAmount` 600 and `actualDuration` 2 hours 0 minutes). Because each carries the nested `driver` object, the cards can show the driver name and rate.

Lines 23-36 | The init() helper

```
23 | const init = (body: any, status = 200) => {
24 |   localStorage.setItem('userId', '2');
25 |   TestBed.configureTestingModule({
26 |     imports: [FormsModule, RouterTestingModule, HttpClientTestingModule],
27 |     declarations: [CustomerviewrequestedComponent]
28 |   });
29 |   fixture = TestBed.createComponent(CustomerviewrequestedComponent);
30 |   component = fixture.componentInstance;
31 |   http = TestBed.inject(HttpTestingController);
32 |   fixture.detectChanges();
33 |   const req = http.expectOne(`${url}/user/2`);
34 |   status === 200 ? req.flush(body) : req.flush('', { status, statusText: 'err' });
35 |   fixture.detectChanges();
36 | };
```

Takes the response body and an optional status. Stores `userId` 2, configures the module, creates the component and calls `detectChanges()`, which sends `GET /api/driverRequest/user/2`. The helper then answers it: with the body when `status === 200`, otherwise with an error of that status (the call `req.flush('', { status, ... })`). One final `detectChanges()` draws the cards.

Line 38 | afterEach

```
38 | afterEach(() => localStorage.clear());
```

Clears `localStorage` after each test.

Lines 40-44 | Test: create and list cards

```
40 | it('should create and list the requests as cards', () => {
41 |   init(requests());
42 |   expect(component).toBeTruthy();
43 |   expect(el().querySelectorAll('.request-card').length).toBe(3);
44 | });
```

Three `.request-card` elements must be drawn, one per request.

Lines 46-49 | Test: no requests (404)

```
46 | it('shows "No Data Found" when the customer has no requests (404)', () => {
47 |   init(null, 404);
48 |   expect(el().textContent).toContain('No Data Found');
49 | });
```

`init(null, 404)` answers with 404, what `viewDriverRequestsByUserId` returns when the customer has no requests. The page must display **No Data Found**.

Lines 51-61 | Test: which buttons are enabled

```
51 | it('enables Edit/Delete only for Pending and Trip End only for Approved', () => {
52 |   init(requests());
53 |   const disabled = (id: string) => (el().querySelector(id) as HTMLButtonElement).disabled;
54 |   expect(disabled('#edit-1')).toBeFalse();
55 |   expect(disabled('#delete-1')).toBeFalse();
56 |   expect(disabled('#edit-2')).toBeTrue();
57 |   expect(disabled('#tripEnd-1')).toBeTrue();
58 |   expect(disabled('#tripEnd-2')).toBeFalse();
59 |   expect(disabled('#fetchPay-3')).toBeFalse();
60 |   expect(disabled('#fetchPay-2')).toBeTrue();
61 | });
```

A small helper `disabled(selector)` reads the `disabled` property of a button. The test asserts, per request: `#edit-1` and `#delete-1` are enabled (Pending), `#edit-2` is disabled (Approved), `#tripEnd-1` is disabled (not Approved yet), `#tripEnd-2` is enabled (Approved), `#fetchPay-3` is enabled (Trip End) and `#fetchPay-2` is disabled. This single test protects the whole status rule set for customers.

Lines 63-71 | Test: delete after confirmation

```
63 | it('deletes a pending request after confirmation', () => {
64 |   init(requests());
65 |   component.askDelete(component.requests[0]);
66 |   fixture.detectChanges();
67 |   expect(el().textContent).toContain('Yes, Delete');
68 |   component.confirmDelete();
69 |   http.expectOne(`${url}/1`).flush({});
70 |   expect(component.requests.length).toBe(2);
71 | });
```

`askDelete(request1)` opens the popup whose confirm button reads `Yes, Delete`. `confirmDelete()` must send `DELETE /api/driverRequest/1` (`DriverRequestController.deleteDriverRequest`, customer only). After the fake reply the list must shrink from 3 to 2 cards without reloading.

Lines 73-83 | Test: ending a trip calculates and saves the fare

```
73 | it('ends a trip and sends the actual drop details and payment', () => {
74 |   init(requests());
75 |   component.endTrip(component.requests[1]);
76 |   const req = http.expectOne(`${url}/2`);
77 |   expect(req.request.method).toBe('PUT');
78 |   expect(req.request.body.status).toBe('Trip End');
79 |   expect(req.request.body.paymentAmount).toBeGreaterThan(0);
80 |   expect(req.request.body.actualDuration).toContain('hours');
81 |   req.flush({});
82 |   expect(component.requests[1].status).toBe('Trip End');
83 | });
```

`endTrip(approvedRequest)` must send `PUT /api/driverRequest/2` with `status Trip End`, a `paymentAmount` greater than 0 and an `actualDuration` that contains the word `hours`. The component computes the fare itself: elapsed time since the booked start (or the estimated duration if the booked start is in the future) multiplied by the driver's hourly rate. The backend does no calculation; it just stores what is sent. After the fake reply the in-memory request must show `Trip End`.

Lines 85-96 | Test: payment popup, loading phase and review link

```
85 | it('shows the payment modal after a short loading phase and navigates to the review page', fakeAsync(() => {
86 |   init(requests());
87 |   const navigate = spyOn(TestBed.inject(Router), 'navigate').and.returnValue(Promise.resolve(true));
88 |   component.fetchPayAmount(component.requests[2]);
89 |   fixture.detectChanges();
90 |   expect(el().querySelector('.spinner')).not.toBeNull();
91 |   tick(1600);
92 |   fixture.detectChanges();
93 |   expect(el().textContent).toContain('Travel Charge');
94 |   component.writeReview();
95 |   expect(navigate).toHaveBeenCalledWith(['/customerpostfeedback'], { queryParams: { driverId: 3 } });
96 | });
```

Wrapped in `fakeAsync`. A spy replaces `Router.navigate`. `fetchPayAmount(request3)` opens the payment popup and starts the loading phase; after `detectChanges()` an element with class `spinner` must exist. `tick(1600)` fast-forwards past the component's 1500 ms timer; after another `detectChanges()` the spinner is replaced by the amount and the text `Travel Charge` must be visible. `writeReview()` must navigate to `['/customerpostfeedback']` with `queryParams: { driverId: 3 }`, so the feedback form knows which driver is being reviewed.

Lines 98-103 | Test: filter by driver name

```
98 | it('filters by driver name', () => {
99 |   init(requests());
100 |   component.searchText = 'nobody';
101 |   expect(component.filteredRequests.length).toBe(0);
102 | });
103 | });
```

Sets `searchText = 'nobody'`; `filteredRequests` must be empty.

Workflow, next: The trip is paid and the customer chooses to write a review. That form is tested next: `customerpostfeedback.component.spec.ts`.

Step 129: customerpostfeedback.component.spec.ts - tests for the Submit Feedback form [Saumya - Feedback & AI]

angularapp/src/app/components/customerpostfeedback/customerpostfeedback.component.spec.ts

TEST

What this file is	Verifies the feedback form: required fields, the star rating and the request sent to <code>POST /api/feedback</code> (<code>FeedbackController.createFeedback</code> , customer only), including the nested <code>user</code> and <code>driver</code> objects.
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>CustomerpostfeedbackComponent</code> , <code>ReactiveFormsModule</code> , fake router and HTTP, and a fake <code>ActivatedRoute</code> that supplies the query parameter <code>driverId</code> (the <code>?driverId=3</code> in the URL).
Size	58 lines

Lines 1-7 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { ReactiveFormsModule } from '@angular/forms';
3 | import { RouterTestingModule } from '@angular/router/testing';
4 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
5 | import { ActivatedRoute, convertToParamMap } from '@angular/router';
6 |
7 | import { CustomerpostfeedbackComponent } from './customerpostfeedback.component';
```

The usual set plus `ActivatedRoute` and `convertToParamMap`.

Lines 9-14 | describe and shared variables

```
9 | describe('CustomerpostfeedbackComponent', () => {
10 |   let component: CustomerpostfeedbackComponent;
11 |   let fixture: ComponentFixture<CustomerpostfeedbackComponent>;
12 |   let http: HttpTestingController;
13 |
14 |   const text = () => (fixture.nativeElement as HTMLElement).textContent || '';
```

Component, fixture, HTTP controller and the `text()` helper.

Lines 16-27 | beforeEach

```
16 |   beforeEach(() => {
17 |     localStorage.setItem('userId', '2');
18 |     TestBed.configureTestingModule({
19 |       imports: [ReactiveFormsModule, RouterTestingModule, HttpClientTestingModule],
20 |       declarations: [CustomerpostfeedbackComponent],
21 |       providers: [{ provide: ActivatedRoute, useValue: { snapshot: { queryParams: convertToParamMap({ driverId: '3'
22 |     }) } } ]
23 |     });
24 |     fixture = TestBed.createComponent(CustomerpostfeedbackComponent);
25 |     component = fixture.componentInstance;
26 |     http = TestBed.inject(HttpTestingController);
27 |     fixture.detectChanges();
28 |   });
```

Stores `userId` 2 in `localStorage`, then builds the module with a fake route whose `snapshot.queryParams` contains `driverId` 3. This simulates arriving from "My Requests" through the `writeReview()` navigation of the previous page. The component is created and drawn once.

Line 29 | afterEach

```
29 |   afterEach(() => localStorage.clear());
```

Clears `localStorage`.

Lines 31-34 | Test: create and heading

```
31 |   it('should create and show the Submit Feedback heading', () => {
32 |     expect(component).toBeTruthy();
33 |     expect(text()).toContain('Submit Feedback');
34 |   });
```

The page must show `Submit Feedback`.

Lines 36-43 | Test: validation on an empty submit

```
36 | it('shows validation messages on an empty submit', () => {
37 |     component.onSubmit();
38 |     fixture.detectChanges();
39 |     expect(text()).toContain('*Category is required');
40 |     expect(text()).toContain('*Rating is required');
41 |     expect(text()).toContain('*Feedback is required');
42 |     expect(text()).toContain('All fields are required');
43 | });
```

Expects ***Category is required**, ***Rating is required**, ***Feedback is required** and the banner **All fields are required**. It protects the three required inputs: category dropdown, star rating and feedback text.

Lines 45-58 | Test: successful post and thank-you popup

```
45 | it('posts the feedback for the driver and shows the thank-you popup', () => {
46 |     component.setRating(4);
47 |     component.feedbackForm.patchValue({ category: 'Punctuality', feedbackText: 'On time and polite' });
48 |     component.onSubmit();
49 |     const req = http.expectOne('http://localhost:8080/api/feedback');
50 |     expect(req.request.method).toBe('POST');
51 |     expect(req.request.body.user).toEqual({ userId: 2 });
52 |     expect(req.request.body.driver).toEqual({ driverId: 3 });
53 |     expect(req.request.body.rating).toBe(4);
54 |     req.flush({}, { status: 201, statusText: 'Created' });
55 |     fixture.detectChanges();
56 |     expect(text()).toContain('Thank you for your feedback!');
57 | });
58 | });
```

setRating(4) is what clicking the fourth star does. **patchValue** sets the category **Punctuality** and the text. After **onSubmit()** exactly one **POST** to **/api/feedback** must be sent. The body must contain **user** equal to **{ userId: 2 }**, **driver** equal to **{ driverId: 3 }** (taken from the query parameter) and **rating** 4. The backend then runs **AIService.applyAnalysis** to fill sentiment, score and tags before saving; the browser does not send those. After a fake **201 Created** the popup **Thank you for your feedback!** must appear.

Workflow, next: Feedback can be posted; the customer's list of their own feedback is next:
customerviewfeedback.component.spec.ts.

Step 130: customerviewfeedback.component.spec.ts - tests for "My Feedback" [Saumya - Feedback & AI]

TEST

angularapp/src/app/components/customerviewfeedback/customerviewfeedback.component.spec.ts

What this file is	Verifies the page where a customer sees the feedback they posted: cards, empty state, a driver-information popup and a confirmed delete. It loads with GET /api/feedback/user/{userId} (FeedbackController , customer only, 404 when none).
Who / what triggers it	Run by Karma during ng test .
What it uses / calls	CustomerviewfeedbackComponent , fake router and HTTP. No forms module is imported.
Size	68 lines

Lines 1-5 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { RouterTestingModule } from '@angular/router/testing';
3 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
4 |
5 | import { CustomerviewfeedbackComponent } from './customerviewfeedback.component';
```

Fixture, fake router, fake HTTP with its controller, and the component.

Lines 7-17 | describe, URL and two feedback records

```
7 | describe('CustomerviewfeedbackComponent', () => {
8 |   let component: CustomerviewfeedbackComponent;
9 |   let fixture: ComponentFixture<CustomerviewfeedbackComponent>;
10 |   let http: HttpTestingController;
11 |
12 |   const url = 'http://localhost:8080/api/feedback';
13 |   const el = () => fixture.nativeElement as HTMLElement;
14 |   const data = () => [
15 |     { feedbackId: 1, feedbackText: 'Great ride', date: '2025-01-25', category: 'Punctuality', rating: 5, sentiment: 'Positive', aiTags: 'punctual, polite', driver: { driverId: 3, driverName: 'demo name three', hourlyRate: 300 } },
16 |     { feedbackId: 2, feedbackText: 'Okay', date: '2025-01-26', category: 'Other', rating: 3 }
17 |   ];
```

`data()` returns two records: id 1 is full (text, date, category, rating 5, **sentiment Positive**, **aiTags** and a nested driver) and id 2 is minimal (no driver, no sentiment). Having both shapes checks that the cards cope with missing optional fields.

Lines 19-32 | The init() helper

```
19 |   const init = (body: any, status = 200) => {
20 |     localStorage.setItem('userId', '2');
21 |     TestBed.configureTestingModule({
22 |       imports: [RouterTestingModule, HttpClientTestingModule],
23 |       declarations: [CustomerviewfeedbackComponent]
24 |     });
25 |     fixture = TestBed.createComponent(CustomerviewfeedbackComponent);
26 |     component = fixture.componentInstance;
27 |     http = TestBed.inject(HttpTestingController);
28 |     fixture.detectChanges();
29 |     const req = http.expectOne(`${url}/user/2`);
30 |     status === 200 ? req.flush(body) : req.flush('', { status, statusText: 'err' });
31 |     fixture.detectChanges();
32 |   };
```

Same pattern as in the other customer pages: stores **userId** 2, builds the component, calls `detectChanges()` (which sends **GET /api/feedback/user/2**), answers with the body or with an error status and redraws.

Line 34 | afterEach

```
34 |   afterEach(() => localStorage.clear());
```

Clears **localStorage**.

Lines 36-41 | Test: heading and cards

```
36 |   it('should create and show the My Feedback heading with the feedback cards', () => {
37 |     init(data());
38 |     expect(component).toBeTruthy();
39 |     expect(el().textContent).toContain('My Feedback');
40 |     expect(el().querySelectorAll('.feedback-card').length).toBe(2);
41 |   });
```

The page must show **My Feedback** and two **.feedback-card** elements.

Lines 43-46 | Test: 404 shows "No Data Found"

```
43 |   it('shows "No Data Found" when nothing has been posted (404)', () => {
44 |     init(null, 404);
45 |     expect(el().textContent).toContain('No Data Found');
46 |   });
```

A customer who has posted nothing gets 404 from the backend; the page must show **No Data Found** rather than breaking.

Lines 48-56 | Test: driver info popup

```
48 |   it('opens the driver info modal', () => {
49 |     init(data());
50 |     component.viewDriverInfo(component.feedbacks[0]);
51 |     fixture.detectChanges();
52 |     expect(el().textContent).toContain('demo name three');
53 |     component.closeDriverModal();
54 |     fixture.detectChanges();
55 |     expect(el().querySelector('#driverModal')).toBeNull();
56 |   });
```

`viewDriverInfo(feedback1)` opens a popup that must contain the driver name **demo name three**. `closeDriverModal()` must remove the element with id **driverModal** (`querySelector` returns **null**).

Lines 58-68 | Test: delete after confirmation

```
58 | it('deletes a feedback after confirmation', () => {
59 |     init(data());
60 |     component.askDelete(component.feedbacks[0]);
61 |     fixture.detectChanges();
62 |     expect(el().textContent).toContain('Yes, Delete');
63 |     component.confirmDelete();
64 |     http.expectOne(`${url}/1`).flush({});
65 |     fixture.detectChanges();
66 |     expect(component.feedbacks.length).toBe(1);
67 | });
68 | });
```

`askDelete(feedback1)` shows the confirm button **Yes, Delete**. `confirmDelete()` must send **DELETE** `/api/feedback/1` (`FeedbackController.deleteFeedback`). After the fake reply the list must contain 1 feedback.

Workflow, next: Customer feedback verified. The admin's counterpart, with filters and the AI summary, is next: `adminviewfeedback.component.spec.ts`.

Step 131: adminviewfeedback.component.spec.ts - tests for the admin feedback table [Saumya - Feedback & AI (best fit)]

TEST

`angularapp/src/app/components/adminviewfeedback/adminviewfeedback.component.spec.ts`

What this file is	Verifies the admin page "Feedback Received": table, date format, sentiment badge, AI tags, filters, the user-profile popup and the driver popup with the AI Review Summary . The list comes from GET <code>/api/feedback</code> (public in <code>SecurityConfig</code>), the summary from GET <code>/api/ai/driver/{id}/feedback-summary</code> (<code>AiController.getFeedbackSummary</code>).
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>AdminviewfeedbackComponent</code> , <code>FormsModule</code> , fake HTTP. There is no router in this component.
Size	83 lines

Lines 1-5 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 | import { FormsModule } from '@angular/forms';
3 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
4 |
5 | import { AdminviewfeedbackComponent } from './adminviewfeedback.component';
```

Fixture, forms, fake HTTP and the component.

Lines 7-19 | describe, URL and two feedback records

```
7 | describe('AdminviewfeedbackComponent', () => {
8 |     let component: AdminviewfeedbackComponent;
9 |     let fixture: ComponentFixture<AdminviewfeedbackComponent>;
10 |    let http: HttpTestingController;
11 |
12 |    const url = 'http://localhost:8080/api/feedback';
13 |    const text = () => (fixture.nativeElement as HTMLElement).textContent || '';
14 |    const feedbacks = () => [
15 |        { feedbackId: 1, feedbackText: 'excellent in punctuality', date: '2025-01-25', category: 'Punctuality', rating: 5,
16 |          sentiment: 'Positive', aiTags: 'punctuality,politeness',
17 |          user: { email: 'cus@gmail.com', username: 'demo', mobileNumber: '9123456789' },
18 |          driver: { driverId: 2, driverName: 'demo name two', licenseNumber: '1234567890987655', experienceYears: 2, vehicleType: 'Sedan', hourlyRate: 230 } },
19 |        { feedbackId: 2, feedbackText: 'late again', date: '2025-01-26', category: 'Driver Performance', rating: 1, sentiment: 'Negative', aiTags: '', user: { username: 'demo' }, driver: null }
20 |    ];
```

Record 1 is rich: a Punctuality rating of 5, sentiment **Positive**, tags, a nested user (email, name, mobile) and a full nested driver. Record 2 is **Negative**, has no tags and `driver: null`, which covers feedback that is not about a specific driver.

Lines 21-30 | beforeEach

```
21 | beforeEach(() => {
22 |     TestBed.configureTestingModule({
23 |         imports: [FormsModule, HttpClientTestingModule],
24 |         declarations: [AdminviewfeedbackComponent]
25 |     });
26 |     fixture = TestBed.createComponent(AdminviewfeedbackComponent);
27 |     component = fixture.componentInstance;
28 |     http = TestBed.inject(HttpTestingController);
29 |     fixture.detectChanges();
30 | });
```

Builds the module and component; `detectChanges()` sends `GET /api/feedback`, which each test answers.

Lines 32-35 | Test: should create

```
32 | it('should create', () => {
33 |     http.expectOne(url).flush(null);
34 |     expect(component).toBeTruthy();
35 | });
```

Answers with `null`; the component exists.

Lines 37-42 | Test: empty state (204)

```
37 | it('shows "No Data Found" when there is no feedback', () => {
38 |     http.expectOne(url).flush(null, { status: 204, statusText: 'No Content' });
39 |     fixture.detectChanges();
40 |     expect(text()).toContain('Feedback Received');
41 |     expect(text()).toContain('No Data Found');
42 | });
```

Answers 204. The page keeps the heading `Feedback Received` and shows `No Data Found`.

Lines 44-50 | Test: dd/MM/yyyy date, sentiment badge and tags

```
44 | it('lists feedback with dd/MM/yyyy dates, sentiment badge and tags', () => {
45 |     http.expectOne(url).flush(feedbacks());
46 |     fixture.detectChanges();
47 |     expect(text()).toContain('25/01/2025');
48 |     expect(text()).toContain('Positive');
49 |     expect(text()).toContain('punctuality');
50 | });
```

After answering with both records the text must contain `25/01/2025` (the stored `2025-01-25` is displayed day first), `Positive` and `punctuality` (one of the AI tags). It protects the display formatting that is not visible in the stored data.

Lines 52-61 | Test: category and sentiment filters

```
52 | it('filters by category and sentiment', () => {
53 |     http.expectOne(url).flush(feedbacks());
54 |     component.categoryFilter = 'Punctuality';
55 |     component.applyFilters();
56 |     expect(component.filteredFeedbacks.length).toBe(1);
57 |     component.categoryFilter = 'All';
58 |     component.sentimentFilter = 'Negative';
59 |     component.applyFilters();
60 |     expect(component.filteredFeedbacks[0].feedbackId).toBe(2);
61 | });
```

Setting `categoryFilter` to `Punctuality` and calling `applyFilters()` leaves 1 record. Resetting it to `All` and setting `sentimentFilter` to `Negative` leaves record 2.

Lines 63-69 | Test: user profile popup

```
63 | it('shows the user profile modal', () => {
64 |     http.expectOne(url).flush(feedbacks());
65 |     component.showProfile(component.feedbacks[0]);
66 |     fixture.detectChanges();
67 |     expect(text()).toContain('User Details:');
68 |     expect(text()).toContain('cus@gmail.com');
69 | });
```

`showProfile(record1)` opens the popup. The text must contain `User Details:` and the email `cus@gmail.com`.

Lines 71-83 | Test: driver popup with AI summary

```
71 | it('shows driver details with the AI review summary', () => {
72 |   http.expectOne(url).flush(feedbacks());
73 |   component.viewDriverInfo(component.feedbacks[0]);
74 |   http.expectOne('http://localhost:8080/api/ai/driver/2/feedback-summary').flush({
75 |     summary: 'Consistently punctual.', strengths: ['Good punctuality'], improvements: [], aiScore: 80, averageRating
76 |     : 4, feedbackCount: 1
77 |   });
78 |   fixture.detectChanges();
79 |   expect(text()).toContain('Driver Details');
80 |   expect(text()).toContain('demo name two');
81 |   expect(text()).toContain('AI Review Summary');
82 |   expect(text()).toContain('Good punctuality');
83 | });
```

`viewDriverInfo(record1)` must trigger **GET** `/api/ai/driver/2/feedback-summary`. The test answers with a summary object: `summary` text, `strengths` list, empty `improvements`, `aiScore` 80, `averageRating` 4 and `feedbackCount` 1. The page must show **Driver Details**, the driver name, the section title **AI Review Summary** and the strength **Good punctuality**. It protects the connection between this page and the AI endpoint.

Workflow, next: Admin feedback verified. The last page of the journey, the error page, is next: `error.component.spec.ts`.

Step 132: error.component.spec.ts - tests for the error page [Vaishali - Customer Journey & Shared Infra]

angularapp/src/app/components/error/error.component.spec.ts

TEST

What this file is	The error page is static text shown when the interceptor finds that the server is down or crashed (status 0 or 5xx). Its spec is as small as the home page spec.
Who / what triggers it	Run by Karma during <code>ng test</code> .
What it uses / calls	<code>ErrorComponent</code> only.
Size	25 lines

Lines 1-3 | Imports

```
1 | import { ComponentFixture, TestBed } from '@angular/core/testing';
2 |
3 | import { ErrorComponent } from './error.component';
```

Fixture type, `TestBed` and the component.

Lines 5-14 | describe + beforeEach

```
5 | describe('ErrorComponent', () => {
6 |   let component: ErrorComponent;
7 |   let fixture: ComponentFixture<ErrorComponent>;
8 |
9 |   beforeEach(() => {
10 |     TestBed.configureTestingModule({ declarations: [ErrorComponent] });
11 |     fixture = TestBed.createComponent(ErrorComponent);
12 |     component = fixture.componentInstance;
13 |     fixture.detectChanges();
14 |   });
```

Declares only `ErrorComponent`, creates it and draws it. No other providers are needed because the page has no dependencies.

Lines 16-18 | Test: should create

```
16 | it('should create', () => {
17 |   expect(component).toBeTruthy();
18 | });
```

The component can be constructed.

Lines 20-25 | Test: error message is displayed

```
20 | it('displays the error message', () => {
21 |     const text = (fixture.nativeElement as HTMLElement).textContent || '';
22 |     expect(text).toContain('Something Went Wrong');
23 |     expect(text).toContain("We're sorry, but an error occurred. Please try again later.");
24 | });
25 | });
```

The page text must contain the title **Something Went Wrong** and the sentence **We're sorry, but an error occurred. Please try again later.** This is the only thing a user sees when the backend is unreachable, so the test makes sure that message cannot disappear silently.

Workflow, next: All pages are covered. One more piece of front-end logic has a spec: the route guard that protects pages by role, `authguard.guard.spec.ts`.

Step 133: authguard.guard.spec.ts - tests for the role-based route guard [Vanshika - Security]

angularapp/src/app/components/authguard/authguard.guard.spec.ts

TEST

What this file is	Verifies AuthGuard , the class the router asks "may this user open this page?" before showing it. It must send strangers and users with the wrong role to /login and let the right role through.
Who / what triggers it	Run by Karma during ng test .
What it uses / calls	AuthGuard , AuthService (real, backed by localStorage), Router , RouterTestingModule and HttpClientTestingModule .
Size	48 lines

Lines 1-7 | Imports

```
1 | import { TestBed } from '@angular/core/testing';
2 | import { ActivatedRouteSnapshot, Router } from '@angular/router';
3 | import { RouterTestingModule } from '@angular/router/testing';
4 | import { HttpClientTestingModule } from '@angular/common/http/testing';
5 |
6 | import { AuthGuard } from '../authguard.guard';
7 | import { AuthService } from '../../services/auth.service';
```

ActivatedRouteSnapshot is the type of the route description the guard receives (its **data** holds the required role from `app-routing.module.ts`). **Router** is used to turn a returned URL object into text. The guard and the service are the real classes.

Lines 9-14 | describe, variables and the routeFor() helper

```
9 | describe('AuthGuard', () => {
10 |     let guard: AuthGuard;
11 |     let auth: AuthService;
12 |     let router: Router;
13 |
14 |     const routeFor = (role?: string) => ({ data: role ? { role } : {} } as unknown as ActivatedRouteSnapshot);
```

routeFor(role) builds a minimal fake route snapshot: `{ data: { role } }`, or an empty **data** when no role is given. The double cast **as unknown as ActivatedRouteSnapshot** tells TypeScript "trust me, this is a snapshot" because building a real one is awkward.

Lines 16-22 | beforeEach

```
16 | beforeEach(() => {
17 |     localStorage.clear();
18 |     TestBed.configureTestingModule({ imports: [RouterTestingModule, HttpClientTestingModule] });
19 |     guard = TestBed.inject(AuthGuard);
20 |     auth = TestBed.inject(AuthService);
21 |     router = TestBed.inject(Router);
22 | });
```

Clears **localStorage**, configures a module with the fake router and fake HTTP (the guard is **providedIn: 'root'**, so no **declarations** are needed) and fetches the guard, the service and the router with **TestBed.inject**.

Line 24 | afterEach

```
24 | afterEach(() => localStorage.clear());
```

Clears **localStorage**.

Lines 26-28 | Test: should be created

```
26 | it('should be created', () => {
27 |   expect(guard).toBeTruthy();
28 | });
```

The guard can be constructed (all its dependencies resolve).

Lines 30-33 | Test: nobody logged in goes to /login

```
30 | it('redirects to the login page when nobody is logged in', () => {
31 |   const result = guard.canActivate(routeFor('Admin'));
32 |   expect(router.serializeUrl(result as any)).toBe('/login');
33 | });
```

With empty storage, `canActivate(routeFor('Admin'))` returns a **UrlTree** (the router's way of saying "redirect there"). `router.serializeUrl(result)` turns it into the text `/login`, which must be exactly that.

Lines 35-40 | Test: wrong role goes to /login

```
35 | it('redirects to the login page when the role does not match', () => {
36 |   localStorage.setItem('token', 'abc');
37 |   localStorage.setItem('userRole', 'Customer');
38 |   const result = guard.canActivate(routeFor('Admin'));
39 |   expect(router.serializeUrl(result as any)).toBe('/login');
40 | });
```

Stores a token (`abc`) and the role `Customer`, then asks for a page that needs `Admin`. The token `abc` is not a real JWT, so `AuthService.isTokenExpired` cannot decode it and leaves the judgement to the server, which means the user counts as logged in; the guard then compares roles and must again return the `/login` URL.

Lines 42-48 | Test: right role is allowed

```
42 | it('allows a logged in user with the right role', () => {
43 |   localStorage.setItem('token', 'abc');
44 |   localStorage.setItem('userRole', 'Admin');
45 |   expect(guard.canActivate(routeFor('Admin'))).toBeTrue();
46 |   expect(auth.isLoggedIn()).toBeTrue();
47 | });
48 | });
```

Stores a token and the role `Admin` and asks for an Admin page; `canActivate` must return exactly `true`, and `auth.isLoggedIn()` must also be true. This protects the happy path so the guard cannot lock everybody out.

Workflow, next: Every component and the guard are covered. Next come the tiny specs for the data models and services, starting with the four **model specs**: `user.model.spec.ts`.

Step 134: user.model.spec.ts - official test for the User model [Aleena - Authentication]

angularapp/src/app/models/user.model.spec.ts

TEST

What this file is

One of the **predefined official tests** supplied by the assignment platform (the copy in `karma/user.model.spec.ts` is byte-identical, checked with `diff`). It builds a `User` object and checks that every property exists. Because `User` is a TypeScript type, the real value of the test is that the file **compiles**: if someone renamed or removed a field of `User`, the object literal below would no longer match the type and the whole test bundle would fail to build.

Who / what triggers it

Run by `ng test`, and by the grading script `karma/karma.sh`, which copies this exact file over the project copy before running.

What it uses / calls

`User` from `user.model.ts` (fields `userId`, `email`, `password`, `username`, `mobileNumber`, `userRole`).

Size

25 lines

Line 1 | Import

```
1 | import { User } from './user.model';
```

Brings in the `User` class. It is used only as a type for the sample object.

Lines 3-5 | describe and fit: a focused test

```
3 | describe('User Model', () => {
4 |
5 |   fit('Frontend_User_model_should_create_an_instance', () => {
```

The group is named `User Model`. The single test is declared with `fit`, not `it`. `fit` means "focused it": when Jasmine finds at least one `fit` in the run, it executes only the focused tests and marks all the plain `it` tests as skipped. The official tests use `fit` so that the grader can run exactly its own named tests. The test name `Frontend_User_model_should_create_an_instance` is the identifier the grader reads in the output (printed as `SUCCESS-Frontend_User_model_should_create_an_instance`).

Lines 6-14 | Sample User

```
6 | // Create a sample User object
7 | const user: User = {
8 |   userId: 1,
9 |   email: 'user@example.com',
10 |   password: 'password123',
11 |   username: 'user123',
12 |   mobileNumber: '1234567890',
13 |   userRole: 'Manager'
14 | };
```

A comment and an object literal typed as `User`. All six fields are given (even `userRole: 'Manager'`, which is not a role the app uses; the test only cares that the field exists, not about its value). Note that the object is a plain literal, not created with `new User()`; TypeScript only checks that its shape matches.

Lines 16-25 | Assertions

```
16 | expect(user).toBeTruthy();
17 | expect(user.userId).toBeDefined();
18 | expect(user.email).toBeDefined();
19 | expect(user.password).toBeDefined();
20 | expect(user.username).toBeDefined();
21 | expect(user.mobileNumber).toBeDefined();
22 | expect(user.userRole).toBeDefined();
23 | });
24 |
25 | };
```

`toBeTruthy()` checks the object is not null or undefined, and `toBeDefined()` for each of the six fields checks it is not `undefined`. Line 25 closes the `describe`. The test is deliberately weak (it cannot fail at run time unless the file fails to compile), but it is what the platform asks for.

Workflow, next: User model checked. Its sibling for drivers is next: `driver.model.spec.ts`.

Step 135: driver.model.spec.ts - official test for the Driver model [Arjun - Driver & Booking]

angularapp/src/app/models/driver.model.spec.ts

TEST

What this file is	Official test (identical to <code>karma/driver.model.spec.ts</code> , checked with <code>diff</code>). It creates a <code>Driver</code> and asserts each field holds the value it was given. Its main protection is compile-time: <code>Driver</code> is an <code>interface</code> , so the object literal must provide every required property with the right type.
Who / what triggers it	<code>ng test</code> and <code>karma/karma.sh</code> .
What it uses / calls	<code>Driver</code> from <code>driver.model.ts</code> .
Size	33 lines

Line 1 | Import

```
1 | import { Driver } from "../driver.model";
```

Imports the `Driver` interface.

Lines 3-5 | describe and fit

```
3 | describe('Driver Model', () => {
4 |
5 |   fit('Frontend_Driver_model_should_create_an_instance_with_defined_properties', () => {
```

Group `Driver Model`; the focused test `Frontend_Driver_model_should_create_an_instance_with_defined_properties` (the grader's test id). `fit` focuses the run as explained in the user model spec.

Lines 6-18 | Sample Driver

```
6 | // Create a sample Driver object
7 | const driver: Driver = {
8 |   driverId: 301,
9 |   driverName: 'John Doe',
10 |   licenseNumber: 'ABC12345',
11 |   experienceYears: 5,
12 |   contactNumber: '1234567890',
13 |   availabilityStatus: 'Available',
14 |   address: '123 Main Street, Springfield',
15 |   vehicleType: 'SUV',
16 |   hourlyRate: 15,
17 |   image: 'base64EncodedImageString'
18 | };
```

Builds a driver with id 301, name **John Doe**, license **ABC12345**, 5 years, contact **1234567890**, status **Available**, an address, vehicle **SUV**, hourly rate 15 and a made-up base64 string for **image**. The values are arbitrary test data and do not follow the app's own rules (the app's own statuses are Active, Inactive and On Leave); the model is only a type, so this is allowed.

Lines 20-33 | Assertions

```
20 | expect(driver).toBeTruthy();
21 | expect(driver.driverId).toBe(301);
22 | expect(driver.driverName).toBe('John Doe');
23 | expect(driver.licenseNumber).toBe('ABC12345');
24 | expect(driver.experienceYears).toBe(5);
25 | expect(driver.contactNumber).toBe('1234567890');
26 | expect(driver.availabilityStatus).toBe('Available');
27 | expect(driver.address).toBe('123 Main Street, Springfield');
28 | expect(driver.vehicleType).toBe('SUV');
29 | expect(driver.hourlyRate).toBe(15);
30 | expect(driver.image).toBe('base64EncodedImageString');
31 | });
32 |
33 | };
```

First **toBeTruthy()** on the object, then one **toBe(...)** per field checking the value is exactly what was assigned. Line 33 closes the **describe**.

Workflow, next: Next the model for trip requests: **driver-request.model.spec.ts**.

Step 136: driver-request.model.spec.ts - official test for the DriverRequest model [Arjun - Driver & Booking]

angularapp/src/app/models/driver-request.model.spec.ts

TEST

What this file is	Official test (identical to karma/driver-request.model.spec.ts). It builds a DriverRequest with all fifteen scalar fields, including the Trip End fields (paymentAmount , actualDropTime , actualDuration , actualDropDate), and checks each one.
Who / what triggers it	ng test and karma/karma.sh .
What it uses / calls	DriverRequest from driver-request.model.ts , an interface whose date fields are typed Date .
Size	43 lines

Line 1 | Import

```
1 | import { DriverRequest } from "../driver-request.model";
```

Imports the **DriverRequest** interface.

Lines 3-5 | describe and fit

```
3 | describe('DriverRequest Model', () => {
4 |
5 |   fit('Frontend_DriverRequest_model_should_create_an_instance_with_defined_properties', () => {
```

Group **DriverRequest Model**; focused test

Frontend_DriverRequest_model_should_create_an_instance_with_defined_properties.

Lines 6-23 | Sample DriverRequest

```
6 | // Create a sample DriverRequest object
7 | const driverRequest: DriverRequest = {
8 |   driverRequestId: 501,
9 |   userId: 1,
10 |   driverId: 301,
11 |   requestDate: new Date('2024-01-01'),
12 |   status: 'Pending',
13 |   tripDate: new Date('2024-01-10'),
14 |   timeSlot: new Date('2024-01-10T10:00:00'),
15 |   pickupLocation: '123 Main Street, Springfield',
16 |   dropLocation: '456 Elm Street, Shelbyville',
17 |   estimatedDuration: '3 hours',
18 |   paymentAmount: 50,
19 |   comments: 'This is a test comment.',
20 |   actualDropTime: new Date('2024-01-10T13:00:00'),
21 |   actualDuration: '3 hours',
22 |   actualDropDate: new Date('2024-01-10')
23 | };
```

Fields: ids **501**, **1** and **301**, **requestDate**, **status** **Pending**, **tripDate** and **timeSlot** as JavaScript **Date** objects, the two locations, **estimatedDuration** **3 hours**, **paymentAmount** **50**, a comment and the actual-drop fields. The model types the dates as **Date** while the real JSON from the backend arrives as strings or arrays; the app converts them with the helpers in **utils/format.ts**, so this spec only proves the type shape, not the runtime data.

Lines 25-43 | Assertions

```
25 | expect(driverRequest).toBeTruthy();
26 | expect(driverRequest.driverRequestId).toBe(501);
27 | expect(driverRequest.userId).toBe(1);
28 | expect(driverRequest.driverId).toBe(301);
29 | expect(driverRequest.requestDate).toEqual(new Date('2024-01-01'));
30 | expect(driverRequest.status).toBe('Pending');
31 | expect(driverRequest.tripDate).toEqual(new Date('2024-01-10'));
32 | expect(driverRequest.timeSlot).toEqual(new Date('2024-01-10T10:00:00'));
33 | expect(driverRequest.pickupLocation).toBe('123 Main Street, Springfield');
34 | expect(driverRequest.dropLocation).toBe('456 Elm Street, Shelbyville');
35 | expect(driverRequest.estimatedDuration).toBe('3 hours');
36 | expect(driverRequest.paymentAmount).toBe(50);
37 | expect(driverRequest.comments).toBe('This is a test comment. ');
38 | expect(driverRequest.actualDropTime).toEqual(new Date('2024-01-10T13:00:00'));
39 | expect(driverRequest.actualDuration).toBe('3 hours');
40 | expect(driverRequest.actualDropDate).toEqual(new Date('2024-01-10'));
41 | });
42 |
43 | };
```

toBeTruthy() first, then **toBe** for ids, texts and numbers and **toEqual** for the **Date** fields. **toEqual** compares dates by value, whereas **toBe** would compare object identity and fail for two separate **Date** objects. Line 43 closes the **describe**.

Workflow, next: Next the feedback model: **feedback.model.spec.ts**.

Step 137: feedback.model.spec.ts - official test for the Feedback model [Saumya - Feedback & AI]

TEST

angularapp/src/app/models/feedback.model.spec.ts

What this file is	Official test (identical to karma/feedback.model.spec.ts). It builds a Feedback and checks its main fields. Feedback is a class with default values, but the literal below still type-checks because it supplies every property that has no ? .
Who / what triggers it	ng test and karma/karma.sh .
What it uses / calls	Feedback from feedback.model.ts .
Size	27 lines

Line 1 | Import

```
1 | import { Feedback } from './feedback.model';
```

Imports the **Feedback** class.

Lines 3-5 | describe and fit

```
3 | describe('Feedback Model', () => {
4 |
5 |   fit('Frontend_Feedback_model_should_create_an_instance_with_defined_properties', () => {
```

Group **Feedback Model**; focused test

Frontend_Feedback_model_should_create_an_instance_with_defined_properties.

Lines 6-15 | Sample Feedback

```
6 |     // Create a sample Feedback object
7 |     const feedback: Feedback = {
8 |       feedbackId: 101,
9 |       userId: 1,
10 |       driverId: 202,
11 |       category: 'Support',
12 |       feedbackText: 'This is a feedback text.',
13 |       date: new Date('2024-07-02'),
14 |       rating : 2
15 |     };
```

Id 101, user id 1, driver id 202, category **Support**, a text, a **Date** and **rating: 2**. The optional AI fields (**sentiment**, **sentimentScore**, **aiTags**) and the nested **user** and **driver** are left out because the model marks them optional (?): the backend fills the AI fields itself when feedback is saved.

Lines 17-27 | Assertions

```
17 |     expect(feedback).toBeTruhy();
18 |     expect(feedback.feedbackId).toBe(101);
19 |     expect(feedback.userId).toBe(1);
20 |     expect(feedback.driverId).toBe(202);
21 |     expect(feedback.category).toBe('Support');
22 |     expect(feedback.feedbackText).toBe('This is a feedback text.');
```

```
23 |     expect(feedback.date).toEqual(new Date('2024-07-02'));
24 |   });
25 |
26 |
27 | };
```

toBeTruhy() then **toBe** for the id, user id, driver id, category and text, and **toEqual** for the date. The **rating** is assigned but never asserted (a small gap in the official test). Line 26 is blank and line 27 closes the **describe**.

Workflow, next: The four model specs are done. Now the five service specs, which are even shorter. First the one for login and registration: **auth.service.spec.ts**.

Step 138: auth.service.spec.ts - official test for AuthService [Aleena - Authentication]

angularapp/src/app/services/auth.service.spec.ts

TEST

What this file is	Official test (identical to karma/auth.service.spec.ts). It only checks that Angular can create AuthService . That already proves the class compiles and that its only dependency (HttpClient) can be supplied.
Who / what triggers it	ng test and karma/karma.sh (which first checks that auth.service.ts exists and prints Frontend_should_create_auth_service FAILED if it does not).
What it uses / calls	AuthService , HttpClientTestingModule (a fake HttpClient).
Size	18 lines

Lines 1-3 | Imports

```
1 | import { TestBed } from '@angular/core/testing';
2 | import { AuthService } from '../auth.service';
3 | import { HttpClientTestingModule } from '@angular/common/http/testing';
```

TestBed, the service and **HttpClientTestingModule**. **AuthService** is injected with an **HttpClient**, and without a provider for it, creating the service would throw "No provider for HttpClient".

Lines 5-13 | describe + beforeEach

```
5 | describe('AuthService', () => {
6 |   let service: AuthService;
7 |
8 |   beforeEach(() => {
9 |     TestBed.configureTestingModule({
10 |       imports: [HttpClientTestingModule],
11 |     });
12 |     service = TestBed.inject(AuthService);
13 |   });
```

Declares `let service`. Before each test, `TestBed.configureTestingModule({ imports: [HttpClientTestingModule] })` builds the fake module and `TestBed.inject(AuthService)` asks Angular's injector for the service. Because the service is `providedIn: 'root'`, no `providers` entry is needed.

Lines 15-18 | fit: the one test

```
15 |   fit('Frontend_should_create_auth_service', () => {
16 |     expect(service).toBeTruthy();
17 |   });
18 | });
```

`fit('Frontend_should_create_auth_service', ...)` is focused (see the user model spec) and asserts `expect(service).toBeTruthy()`. Line 18 closes the `describe`.

Workflow, next: Next the driver service spec: `driver.service.spec.ts`.

Step 139: driver.service.spec.ts - official test for DriverService [Arjun - Driver & Booking]

angularapp/src/app/services/driver.service.spec.ts

TEST

What this file is	Official test (identical to <code>karma/driver.service.spec.ts</code>). The same "can it be created?" check for <code>DriverService</code> .
Who / what triggers it	<code>ng test</code> and <code>karma/karma.sh</code> .
What it uses / calls	<code>DriverService</code> , <code>HttpClientTestingModule</code> .
Size	19 lines

Lines 1-4 | Imports

```
1 | import { TestBed } from '@angular/core/testing';
2 |
3 | import { DriverService } from './driver.service';
4 | import { HttpClientTestingModule } from '@angular/common/http/testing';
```

`TestBed`, `DriverService` and the fake HTTP module.

Lines 6-14 | describe + beforeEach

```
6 | describe('DriverService', () => {
7 |   let service: DriverService;
8 |
9 |   beforeEach(() => {
10 |     TestBed.configureTestingModule({
11 |       imports: [HttpClientTestingModule],
12 |     });
13 |     service = TestBed.inject(DriverService);
14 |   });
```

Builds the module with the fake `HttpClient` and injects the service into the shared variable.

Lines 16-19 | fit: the one test

```
16 |   fit('Frontend_should_create_drive_service', () => {
17 |     expect(service).toBeTruthy();
18 |   });
19 | });
```

Focused test `Frontend_should_create_drive_service` (the typo "drive" is in the official name and must not be changed, because the grader looks for that exact text). It expects the service to be truthy.

Workflow, next: Next: `driver-request.service.spec.ts`.

Step 140: driver-request.service.spec.ts - official test for DriverRequestService [Arjun - Driver & Booking]

TEST

angularapp/src/app/services/driver-request.service.spec.ts

What this file is	Official test (identical to <code>karma/driver-request.service.spec.ts</code>). Checks that <code>DriverRequestService</code> can be created.
Who / what triggers it	<code>ng test</code> and <code>karma/karma.sh</code> .
What it uses / calls	<code>DriverRequestService</code> , <code>HttpClientTestingModule</code> .
Size	19 lines

Lines 1-4 | Imports

```
1 | import { TestBed } from '@angular/core/testing';
2 |
3 | import { DriverRequestService } from './driver-request.service';
4 | import { HttpClientTestingModule } from '@angular/common/http/testing';
```

`TestBed`, the service and the fake HTTP module.

Lines 6-14 | describe + beforeEach

```
6 | describe('DriverRequestService', () => {
7 |   let service: DriverRequestService;
8 |
9 |   beforeEach(() => {
10 |     TestBed.configureTestingModule({
11 |       imports: [HttpClientTestingModule],
12 |     });
13 |     service = TestBed.inject(DriverRequestService);
14 |   });
```

Same module setup and injection as in the other service specs.

Lines 16-19 | fit: the one test

```
16 |   fit('Frontend should create driverRequest_service', () => {
17 |     expect(service).toBeTruthy();
18 |   });
19 | });
```

Focused test `Frontend_should_create_driverRequest_service`, asserting the service is truthy.

Workflow, next: Next: `feedback.service.spec.ts`.

Step 141: feedback.service.spec.ts - official test for FeedbackService [Saumya - Feedback & AI]

TEST

angularapp/src/app/services/feedback.service.spec.ts

What this file is	Official test (identical to <code>karma/feedback.service.spec.ts</code>). Checks that <code>FeedbackService</code> can be created.
Who / what triggers it	<code>ng test</code> and <code>karma/karma.sh</code> .
What it uses / calls	<code>FeedbackService</code> , <code>HttpClientTestingModule</code> .
Size	19 lines

Lines 1-4 | Imports

```
1 | import { TestBed } from '@angular/core/testing';
2 |
3 | import { FeedbackService } from './feedback.service';
4 | import { HttpClientTestingModule } from '@angular/common/http/testing';
```

`TestBed`, the service and the fake HTTP module.

Lines 6-14 | describe + beforeEach

```
6 | describe('FeedbackService', () => {
7 |   let service: FeedbackService;
8 |
9 |   beforeEach(() => {
10 |     TestBed.configureTestingModule({
11 |       imports: [HttpClientTestingModule],
12 |     });
13 |     service = TestBed.inject(FeedbackService);
14 |   });
```

Same setup as the other service specs.

Lines 16-19 | fit: the one test

```
16 |   fit('Frontend should create feedback service', () => {
17 |     expect(service).toBeTruthy();
18 |   });
19 | });
```

Focused test `Frontend_should_create_feedback_service`, asserting the service is truthy.

Workflow, next: The four official service specs are done. The fifth service spec is ours and does more than creation: `ai.service.spec.ts`.

Step 142: ai.service.spec.ts - our own test for AiService

[Saumya - Feedback & AI]

angularapp/src/app/services/ai.service.spec.ts

TEST

What this file is

A spec we wrote ourselves (there is no official version in `karma/`; the AI feature is an extra of this project). Unlike the creation-only official specs it checks the **exact HTTP calls** `AiService` makes: the method, URL, request body and the `Authorization` header. It uses `it`, not `fit`, so it is skipped whenever the focused official tests are present (see the karma entries).

Who / what triggers it

`ng test` after the `fit` markers are removed or changed to `it` (we did that in a scratch copy and all tests passed).

What it uses / calls

`AiService`, `HttpClientTestingModule` and `HttpTestingController` (the remote control that sees and answers the fake requests).

Size

37 lines

Lines 1-4 | Imports

```
1 | import { TestBed } from '@angular/core/testing';
2 | import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
3 |
4 | import { AiService } from './ai.service';
```

`TestBed`, the fake HTTP module with its controller and the service.

Lines 6-14 | describe + beforeEach

```
6 | describe('AiService', () => {
7 |   let service: AiService;
8 |   let http: HttpTestingController;
9 |
10 |   beforeEach(() => {
11 |     TestBed.configureTestingModule({ imports: [HttpClientTestingModule] });
12 |     service = TestBed.inject(AiService);
13 |     http = TestBed.inject(HttpTestingController);
14 |   });
```

Declares `service` and `http`, builds the module and injects both the service and the `HttpTestingController` (the same fake backend that the service's `HttpClient` talks to).

Line 16 | afterEach: http.verify()

```
16 |   afterEach(() => http.verify());
```

`http.verify()` fails the test if any request was sent that the test did not claim with `expectOne`. It catches accidental extra calls.

Lines 18-20 | Test: should be created

```
18 | it('should be created', () => {
19 |     expect(service).toBeTruthy();
20 | });
```

The service can be constructed.

Lines 22-29 | Test: searchDrivers posts the query

```
22 | it('should POST the natural language query to /api/ai/driver-search', () => {
23 |     service.searchDrivers('van driver').subscribe();
24 |     const req = http.expectOne(`${service.apiUrl}/api/ai/driver-search`);
25 |     expect(req.request.method).toBe('POST');
26 |     expect(req.request.body).toBe('van driver');
27 |     expect(req.request.headers.has('Authorization')).toBeTrue();
28 |     req.flush([]);
29 | });
```

Calls `service.searchDrivers('van driver').subscribe()` (nothing is sent until someone subscribes, so `subscribe()` is what triggers the request). `expectOne` asserts there was exactly one request to `{apiUrl}/api/ai/driver-search`, and the next lines check the method is **POST**, the body is the raw string `van driver` (sent as plain text, as `AiController.searchDrivers` expects) and an `Authorization` header is present (the `Bearer` token). `req.flush([])` answers with an empty result list to complete the call.

Lines 31-37 | Test: getDriverFeedbackSummary

```
31 | it('should GET the feedback summary of a driver', () => {
32 |     service.getDriverFeedbackSummary(4).subscribe();
33 |     const req = http.expectOne(`${service.apiUrl}/api/ai/driver/4/feedback-summary`);
34 |     expect(req.request.method).toBe('GET');
35 |     req.flush({});
36 | });
37 | });
```

Calls `service.getDriverFeedbackSummary(4)`, expects one request to `/api/ai/driver/4/feedback-summary` with method **GET** (answered by `AiController.getFeedbackSummary`), and answers with an empty object. Line 37 closes the `describe`.

Workflow, next: All specs are covered. The next three files configure how tests are found and run, starting with `angularapp/src/test.ts`.

Step 143: test.ts - old Angular test entry point (unused)

[Vaishalini - Customer Journey & Shared Infra (test setup)]

angularapp/src/test.ts

FRONTEND

What this file is

In older Angular versions this file was the entry point of the test bundle: it started the Angular test environment and loaded every spec file. **In this project it is a leftover and is not used.** We verified three things: `angular.json` has no `"main": "src/test.ts"` in its `test` target (the only `"main"` is the application's `src/main.ts`); `tsconfig.spec.json` includes only `*.spec.ts` and `*.d.ts` files, so Angular 16 does not even compile `test.ts`; and the line `import 'zone.js/dist/zone-testing'` points to a folder that no longer exists in the installed `zone.js 0.13` (the current form is `zone.js/testing`, configured in `angular.json` under `polyfills`). If the file were ever compiled it would fail.

Who / what triggers it

Nothing runs it. It is kept because the original project template contains it.

What it uses / calls

Would use `zone.js`, `@angular/core/testing` and `@angular/platform-browser-dynamic/testing`.

Size

25 lines

Line 1 | Comment

```
1 | // This file is required by karma.conf.js and loads recursively all the .spec and framework files
```

Says the file is required by `karma.conf.js` and loads the specs. That was true for old Angular; it is not true here.

Lines 3-8 | Imports

```
3 | import 'zone.js/dist/zone-testing';
4 | import { TestBed } from '@angular/core/testing';
5 | import {
6 |   BrowserDynamicTestingModule,
7 |   platformBrowserDynamicTesting
8 | } from '@angular/platform-browser-dynamic/testing';
```

`zone.js/dist/zone-testing` would patch the browser so Angular can track asynchronous work in tests (the old import path). `getTestBed` gives access to the global test bed. `BrowserDynamicTestingModule` and `platformBrowserDynamicTesting` describe the browser platform for tests.

Lines 10-15 | Type declaration for `require.context`

```
10 | declare const require: {
11 |   context(path: string, deep?: boolean, filter?: RegExp): {
12 |     keys(): string[];
13 |     <T>(id: string): T;
14 |   };
15 | };
```

`require.context` is a Webpack feature that returns a list of files matching a pattern. TypeScript does not know it, so this `declare const require` block describes its shape (`keys()` lists the matched paths and calling it loads one).

Lines 17-21 | Initialise the test environment

```
17 | // First, initialize the Angular testing environment.
18 | TestBed.configureTestingModule(
19 |   BrowserDynamicTestingModule,
20 |   platformBrowserDynamicTesting()
21 | );
```

`getTestBed().initTestEnvironment(...)` tells Angular to use the browser-based testing platform. This must happen once before any spec runs.

Lines 22-25 | Find and load all specs

```
22 | // Then we find all the tests.
23 | const context = require.context('./', true, /\.spec\.ts$/);
24 | // And load the modules.
25 | context.keys().map(context);
```

`require.context('./', true, /\.spec\.ts$/)` collects every file under `src` ending in `.spec.ts`, and `context.keys().map(context)` loads each one so its `describe` blocks register. In the current Angular builder this discovery is done by the `include` pattern in `tsconfig.spec.json` instead.

Workflow, next: Since `test.ts` is inert, the settings that really matter for running tests are in `angularapp/karma.conf.js`.

Step 144: `karma.conf.js` (angularapp) - Karma test-runner settings **[Shared - whole team]**

`angularapp/karma.conf.js`

CONFIG

What this file is	Configuration for Karma , the tool that launches a browser, loads the compiled specs into it and reports the results. <code>angular.json</code> points at this file in its <code>test</code> target (<code>"karmaConfig": "karma.conf.js"</code>), so <code>ng test</code> reads it. During grading it is overwritten by <code>karma/karma.conf.js</code> (see <code>karma.sh</code>); the two files differ only on line 3.
Who / what triggers it	<code>ng test</code> (the <code>npm test</code> script), directly or through <code>karma/karma.sh</code> .
What it uses / calls	<code>puppeteer</code> (required at the top), the Karma plugins <code>karma-jasmine</code> , <code>karma-chrome-launcher</code> , <code>karma-jasmine-html-reporter</code> , <code>karma-coverage</code> , <code>karma-spec-reporter</code> and the Angular plugin <code>@angular-devkit/build-angular/plugins/karma</code> ; a Chrome/Chromium binary.
Size	63 lines

Lines 1-3 | Puppeteer and the `CHROME_BIN` comment

```
1 | const puppeteer = require('puppeteer');
2 | // Uncomment the below line if you need to set the CHROME_BIN environment variable to Puppeteer's executable path
3 | // process.env.CHROME_BIN = puppeteer.executablePath();
```

`require('puppeteer')` loads the library that downloads its own Chrome; it is listed in `package.json` and must be installed or Karma crashes on this first line. The next two lines are **comments** explaining that you could point the `CHROME_BIN` environment variable at Puppeteer's Chrome. In this copy the assignment is commented out, so the browser must come from `CHROME_BIN` set outside (the grading script does that).

Lines 4-8 | Export and basic settings

```
4 | module.exports = function (config) {
5 |   config.set({
6 |     basePath: '',
7 |     colors: false, // Disable colored output
8 |     frameworks: ['jasmine', '@angular-devkit/build-angular'],
```

Karma loads this file as a function receiving `config`. `config.set({...})` applies the options. `basePath: ''` means paths are relative to this file. `colors: false` turns off coloured console output (easier to read in logs and for the grader). `frameworks` chooses Jasmine as the test language and the Angular builder to compile TypeScript.

Lines 9-16 | Plugins

```
9 |   plugins: [
10 |     require('karma-jasmine'),
11 |     require('karma-chrome-launcher'),
12 |     require('karma-jasmine-html-reporter'),
13 |     require('karma-coverage'),
14 |     require('@angular-devkit/build-angular/plugins/karma'),
15 |     require('karma-spec-reporter'),
16 |   ],
```

Lists the Node modules Karma may use: the Jasmine adapter, the Chrome launcher, an HTML reporter, coverage support, the Angular build plugin and the **spec reporter** that prints one readable line per test.

Lines 17-22 | Client and HTML reporter options

```
17 |   client: {
18 |     clearContext: false // Leave Jasmine Spec Runner output visible in browser
19 |   },
20 |   jasmineHtmlReporter: {
21 |     suppressAll: true // Removes the duplicated traces
22 |   },
```

`clearContext: false` keeps the Jasmine output visible in the browser page instead of clearing it. `suppressAll: true` for the HTML reporter hides duplicated stack traces.

Lines 23-37 | Spec reporter format

```
23 |   specReporter: {
24 |     maxLogLines: 5, // Limit number of lines logged per test
25 |     suppressErrorSummary: true, // Do not print error summary
26 |     suppressFailed: false, // Do not print information about failed tests
27 |     suppressPassed: false, // Do not print information about passed tests
28 |     suppressSkipped: true, // Do not print information about skipped tests
29 |     showSpecTiming: false, // Print the time elapsed for each spec
30 |     failFast: false, // Test would finish with error when a first fail occurs
31 |     suppressColor: true, // Ensure color is suppressed in specReporter
32 |     prefixes: {
33 |       success: 'SUCCESS-', // Override prefix for passed tests, default is '✓ '
34 |       failure: 'FAILED-', // Override prefix for failed tests, default is 'x '
35 |       skipped: 'SKIPPED-' // Override prefix for skipped tests, default is '- '
36 |     }
37 |   },
```

This block shapes the console output the grader reads. `maxLogLines: 5` limits error lines per test. Passed and failed tests are printed (`suppressPassed: false`, `suppressFailed: false`) while skipped tests are hidden (`suppressSkipped: true`, which is why unfocused specs never show in the log). `prefixes` replaces the default tick/cross with the words `SUCCESS-`, `FAILED-` and `SKIPPED-`, so each result line looks like `SUCCESS-Frontend_should_create_auth_service`. The grading system searches for these words.

Lines 38-42 | Reporters and progress

```
38 |   reporters: ['spec'],
39 |   progressReporter: {
40 |     showFailed: true,
41 |     showPassed: true // Corrected the typo here
42 |   },
```

`reporters: ['spec']` selects the spec reporter. `progressReporter` is a leftover option block (it belongs to a different reporter and is ignored here).

Lines 43-46 | Port, log level, watch and browser choice

```
43 |   port: 9876,
44 |   logLevel: config.LOG_INFO,
45 |   autoWatch: true,
46 |   browsers: ['CustomChromeHeadless'],
```

Karma serves the tests on port 9876. `logLevel: config.LOG_INFO` prints informational messages. `autoWatch: true` would re-run on file changes (irrelevant because `singleRun` is true). `browsers` selects the custom launcher defined next.

Lines 47-59 | Custom headless Chrome launcher

```
47 |     customLaunchers: {
48 |       CustomChromeHeadless: {
49 |         base: 'Chrome',
50 |         flags: [
51 |           '--headless',
52 |           '--disable-gpu',
53 |           '--remote-debugging-port=9222',
54 |           '--no-sandbox',
55 |           '--disable-setuid-sandbox'
56 |         ],
57 |         executablePath: '/usr/bin/chromium-browser'
58 |       },
59 |     },
```

Defines `CustomChromeHeadless`, based on the Chrome launcher, with flags: `--headless` (no window), `--disable-gpu`, a remote debugging port 9222, and `--no-sandbox` plus `--disable-setuid-sandbox` (needed when Chrome runs as root inside containers like the grading machine). The `executablePath` entry points at `/usr/bin/chromium-browser`. We tested this: `karma-chrome-launcher` does **not** read that option; it uses the `CHROME_BIN` environment variable. When we ran the tests without `CHROME_BIN` Karma failed with "No binary for Chrome browser" even though `executablePath` was set.

Lines 60-63 | Run once and exit

```
60 |     singleRun: true,
61 |     restartOnFileChange: false
62 |   });
63 | };
```

`singleRun: true` makes Karma run all tests once and exit with a result code (needed for a grading script). `restartOnFileChange: false` does not restart on edits. Lines 62-63 close `config.set` and the exported function.

Workflow, next: The other test-related config file is the TypeScript one, `angularapp/tsconfig.spec.json`, which decides which files are compiled for tests.

Step 145: tsconfig.spec.json - TypeScript settings for tests CONFIG

angularapp/tsconfig.spec.json

What this file is	TypeScript configuration used only when compiling tests (<code>angular.json</code> names it as " <code>tsConfig</code> " for the <code>test</code> target). It inherits the main <code>tsconfig.json</code> and adds the Jasmine type definitions. Its include list is what makes Angular find the spec files: every file matching <code>src/**/*.spec.ts</code> is compiled into the test bundle.
Who / what triggers it	<code>ng test</code> .
What it uses / calls	<code>tsconfig.json</code> (parent), the <code>@types/jasmine</code> package.
Size	14 lines

Line 1 | Comment

```
1 | /* To learn more about this file see: https://angular.io/config/tsconfig. */
```

A link to Angular's `tsconfig` documentation.

Lines 2-9 | Extend the main config and add Jasmine types

```
2 | {
3 |   "extends": "../tsconfig.json",
4 |   "compilerOptions": {
5 |     "outDir": "../out-tsc/spec",
6 |     "types": [
7 |       "jasmine"
8 |     ]
9 |   },
```

"extends": "./tsconfig.json" reuses every setting from the main file (target, module, decorators). `outDir` is where compiled test output would go (`./out-tsc/spec`). `"types": ["jasmine"]` makes the globals `describe`, `it`, `fit`, `expect`, `beforeEach` and friends known to TypeScript; without it every spec would show "cannot find name 'describe'".

Lines 10-14 | Which files to compile

```
10 |   "include": [
11 |     "src/**/*.spec.ts",
12 |     "src/**/*.d.ts"
13 |   ]
14 | }
```

`include` lists the glob patterns: all `*.spec.ts` files anywhere under `src`, plus `*.d.ts` declaration files. That means all 89 tests (8 official plus 81 of ours) are compiled, and also that `test.ts` is **not** included, which confirms it is unused.

Workflow, next: We now leave the Angular configuration and look at the official grading harness that runs these specs, starting with `karma/karma.sh`.

Step 146: karma.sh - the official frontend grading script

[Shared - whole team]

karma/karma.sh

TEST

What this file is	A Bash script that belongs to the assignment platform, not to the app. When the project is graded, this script prepares the Angular folder, drops the official spec files into it, installs libraries if needed and runs <code>npm test</code> . Its output (lines starting with <code>SUCCESS-</code> or <code>FAILED-</code>) is what the platform reads to award marks. The paths inside it (<code>/home/coder/project/workspace/...</code>) are the paths on the grading machine; on your own computer they do not exist, so running it locally does nothing useful.
Who / what triggers it	The grading system runs it. You can also read it to understand exactly how your frontend will be judged.
What it uses / calls	The folder <code>karma/</code> (official spec files, the Karma config and a skeleton project in <code>karma/angularapp/</code>), your <code>angularapp/</code> folder, <code>nvm</code> (Node version manager), Chromium and <code>npm</code> .
Size	79 lines

Lines 1-5 | Shebang, Node version and browser

```
1 | #!/bin/bash
2 | export NVM_DIR="/usr/local/nvm"
3 | [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
4 | nvm use 20
5 | export CHROME_BIN=/usr/bin/chromium
```

`#!/bin/bash` says run with Bash. The next two lines load `nvm` (if its script exists) so that `nvm use 20` can switch to Node.js 20. `export CHROME_BIN=/usr/bin/chromium` tells Karma's Chrome launcher which browser program to start. (Note that `karma/karma.conf.js` later overrides this variable with Puppeteer's own Chrome.)

Lines 7-9 | If there is no angularapp, copy the skeleton

```
7 | if [ ! -d "/home/coder/project/workspace/angularapp" ]; then
8 |   cp -r /home/coder/project/workspace/karma/angularapp /home/coder/project/workspace/;
9 | fi
```

If the workspace has no `angularapp` folder at all, copy the whole `karma/angularapp` skeleton there. That folder is the stock Angular project (with `package.json`, `angular.json`, a placeholder `AppComponent` and its stock spec) presumably so that a submission without a frontend still produces output instead of a crash; the missing-file checks below then print `FAILED`.

Lines 11-13 | Project present: copy the official karma.conf.js

```
11 | if [ -d "/home/coder/project/workspace/angularapp" ]; then
12 |   echo "project folder present"
13 |   cp /home/coder/project/workspace/karma/karma.conf.js /home/coder/project/workspace/angularapp/karma.conf.js;
```

When `angularapp` exists the script prints `project folder present` and **overwrites** `angularapp/karma.conf.js` with `karma/karma.conf.js`. So whatever Karma settings the student wrote are replaced by the platform's for grading.

Lines 14-18 | auth.service spec

```
14 | if [ -e "/home/coder/project/workspace/angularapp/src/app/services/auth.service.ts" ]; then
15 |     cp /home/coder/project/workspace/karma/auth.service.spec.ts /home/coder/project/workspace/angularapp/src/app/s
    services/auth.service.spec.ts;
16 | else
17 |     echo "Frontend_should_create_auth_service FAILED";
18 | fi
```

If `src/app/services/auth.service.ts` exists, copy `karma/auth.service.spec.ts` over `angularapp/src/app/services/auth.service.spec.ts`. If not, print `Frontend_should_create_auth_service FAILED` by itself, which is the exact test id the grader looks for. This is the pattern for every official spec: **file exists, then test it; file missing, then fail it by name**.

Lines 20-24 | driver-request model spec

```
20 | if [ -e "/home/coder/project/workspace/angularapp/src/app/models/driver-request.model.ts" ]; then
21 |     cp /home/coder/project/workspace/karma/driver-request.model.spec.ts /home/coder/project/workspace/angularapp/s
    rc/app/models/driver-request.model.spec.ts;
22 | else
23 |     echo "Frontend_DriverRequest_model_should_create_an_instance_with_defined_properties FAILED";
24 | fi
```

Same pattern for `models/driver-request.model.ts` and its spec; the failure text is `Frontend_DriverRequest_model_should_create_an_instance_with_defined_properties FAILED`.

Lines 26-30 | driver-request service spec

```
26 | if [ -e "/home/coder/project/workspace/angularapp/src/app/services/driver-request.service.ts" ]; then
27 |     cp /home/coder/project/workspace/karma/driver-request.service.spec.ts /home/coder/project/workspace/angularapp
    /src/app/services/driver-request.service.spec.ts;
28 | else
29 |     echo "Frontend_should_create_driverRequest_service FAILED";
30 | fi
```

Same pattern for `services/driver-request.service.ts` (test id `Frontend_should_create_driverRequest_service`).

Lines 32-36 | driver model spec

```
32 | if [ -e "/home/coder/project/workspace/angularapp/src/app/models/driver.model.ts" ]; then
33 |     cp /home/coder/project/workspace/karma/driver.model.spec.ts /home/coder/project/workspace/angularapp/src/app/m
    odels/driver.model.spec.ts;
34 | else
35 |     echo "Frontend_Driver_model_should_create_an_instance_with_defined_properties FAILED";
36 | fi
```

Same pattern for `models/driver.model.ts` (test id `Frontend_Driver_model_should_create_an_instance_with_defined_properties`).

Lines 38-42 | driver service spec

```
38 | if [ -e "/home/coder/project/workspace/angularapp/src/app/services/driver.service.ts" ]; then
39 |     cp /home/coder/project/workspace/karma/driver.service.spec.ts /home/coder/project/workspace/angularapp/src/app
    /services/driver.service.spec.ts;
40 | else
41 |     echo "Frontend_should_create_drive_service FAILED";
42 | fi
```

Same pattern for `services/driver.service.ts` (test id `Frontend_should_create_drive_service`).

Lines 44-48 | feedback model spec

```
44 | if [ -e "/home/coder/project/workspace/angularapp/src/app/models/feedback.model.ts" ]; then
45 |     cp /home/coder/project/workspace/karma/feedback.model.spec.ts /home/coder/project/workspace/angularapp/src/app
    /models/feedback.model.spec.ts;
46 | else
47 |     echo "Frontend_Feedback_model_should_create_an_instance_with_defined_properties FAILED";
48 | fi
```

Same pattern for `models/feedback.model.ts` (test id `Frontend_Feedback_model_should_create_an_instance_with_defined_properties`).

Lines 50-54 | feedback service spec

```
50 | if [ -e "/home/coder/project/workspace/angularapp/src/app/services/feedback.service.ts" ]; then
51 |     cp /home/coder/project/workspace/karma/feedback.service.spec.ts /home/coder/project/workspace/angularapp/src/a
    pp/services/feedback.service.spec.ts;
52 | else
53 |     echo "Frontend_should_create_feedback_service FAILED";
54 | fi
```

Same pattern for `services/feedback.service.ts` (test id `Frontend_should_create_feedback_service`).

Lines 56-60 | user model spec

```
56 |     if [ -e "/home/coder/project/workspace/angularapp/src/app/models/user.model.ts" ]; then
57 |         cp /home/coder/project/workspace/karma/user.model.spec.ts /home/coder/project/workspace/angularapp/src/app/mod
    |     els/user.model.spec.ts;
58 |     else
59 |         echo "Frontend_User_model_should_create_an_instance FAILED";
60 |     fi
```

Same pattern for `models/user.model.ts` (test id `Frontend_User_model_should_create_an_instance`). Together these eight blocks are the 8 official frontend tests: 4 models and 4 services. This is why the project must keep exactly these file names and class names.

Lines 62-69 | Install if needed, then run the tests

```
62 |     if [ -d "/home/coder/project/workspace/angularapp/node_modules" ]; then
63 |         cd /home/coder/project/workspace/angularapp/
64 |         npm test;
65 |     else
66 |         cd /home/coder/project/workspace/angularapp/
67 |         yes | npm install
68 |         npm test
69 |     fi
```

Go into `angularapp`. If `node_modules` already exists, run `npm test` (which is `ng test`, defined in `package.json`). Otherwise first run `yes | npm install` (the `yes` command answers any prompt with "y") and then `npm test`. Karma then compiles **all** specs, but because the eight official ones use `fit`, Jasmine runs only those and skips the rest. We confirmed this: with the project as it stands the run ends with **Executed 8 of 89 (skipped 81) SUCCESS**.

Lines 70-79 | No angularapp: fail everything

```
70 | else
71 |     echo "Frontend_should_create_auth_service FAILED";
72 |     echo "Frontend_DriverRequest_model_should_create_an_instance_with_defined_properties FAILED";
73 |     echo "Frontend_should_create_driverRequest_service FAILED";
74 |     echo "Frontend_Driver_model_should_create_an_instance_with_defined_properties FAILED";
75 |     echo "Frontend_should_create_drive_service FAILED";
76 |     echo "Frontend_Feedback_model_should_create_an_instance_with_defined_properties FAILED";
77 |     echo "Frontend_should_create_feedback_service FAILED";
78 |     echo "Frontend_User_model_should_create_an_instance FAILED";
79 | fi
```

If even after the first step `angularapp` is missing, print all eight **FAILED** lines directly, then `fi` closes the outer `if`.

Workflow, next: The config file this script copies over the project's own is next: `karma/karma.conf.js`.

Step 147: karma/karma.conf.js - the official Karma configuration [Shared - whole team]

karma/karma.conf.js

TEST

What this file is

The platform's version of the Karma settings. `karma.sh` copies it over `angularapp/karma.conf.js` before running the tests. We compared it with `angularapp/karma.conf.js` using `diff`: the only difference is **line 3**. Here `process.env.CHROME_BIN = puppeteer.executablePath();` is active; in the project copy it is commented out. So during grading Chrome comes from the Puppeteer library, overriding the `/usr/bin/chromium` that `karma.sh` exported.

Honest summary of what is in the `karma/` folder after `diff` checks: the eight spec files (`auth.service`, `driver.service`, `driver-request.service`, `feedback.service`, `user.model`, `driver.model`, `driver-request.model`, `feedback.model`) are **byte-identical** to the copies in `angularapp/src/app/`, so those are the predefined official tests. The specs for the 15 components, the guard, `AuthService` and `AppComponent` exist **only** in `angularapp` and are tests our group wrote to verify behaviour; the grader never runs them (they use `it`, and the official tests use `fit`). Our `AppComponent` spec differs from the stock one in `karma/angularapp/src/app/app.component.spec.ts`, which expects the text `angularapp app is running!` that our page does not contain; it is only used if the whole skeleton is copied, and it would be skipped anyway.

Who / what triggers it

Read by Karma when `ng test` starts under the grading script.

What it uses / calls

Same plugins and Chrome as the project copy; `puppeteer`.

Size

63 lines

Lines 1-2 | Puppeteer and a comment

```
1 | const puppeteer = require('puppeteer');
2 | // Uncomment the below line if you need to set the CHROME_BIN environment variable to Puppeteer's executable path
require('puppeteer') loads Puppeteer (installed through package.json). Line 2 is a comment about the next line.
```

Line 3 | Set CHROME_BIN from Puppeteer

```
3 | process.env.CHROME_BIN = puppeteer.executablePath();
```

The only line that differs from the project copy. `puppeteer.executablePath()` returns the path of the Chrome that Puppeteer downloaded, and assigning it to `process.env.CHROME_BIN` makes Karma's launcher start that browser.

Lines 4-22 | Export, frameworks, plugins and client options

```
4 | module.exports = function (config) {
5 |   config.set({
6 |     basePath: '',
7 |     colors: false, // Disable colored output
8 |     frameworks: ['jasmine', '@angular-devkit/build-angular'],
9 |     plugins: [
10 |       require('karma-jasmine'),
11 |       require('karma-chrome-launcher'),
12 |       require('karma-jasmine-html-reporter'),
13 |       require('karma-coverage'),
14 |       require('@angular-devkit/build-angular/plugins/karma'),
15 |       require('karma-spec-reporter'),
16 |     ],
17 |     client: {
18 |       clearContext: false // Leave Jasmine Spec Runner output visible in browser
19 |     },
20 |     jasmineHtmlReporter: {
21 |       suppressAll: true // Removes the duplicated traces
22 |     },
```

Same as in the project copy: `module.exports = function (config) { config.set({...}) }`, base path, no colours, Jasmine plus the Angular builder, the six plugins, `clearContext: false` and the HTML reporter option. See the [angularapp/karma.conf.js](#) entry for each option.

Lines 23-42 | Reporter settings

```
23 |     specReporter: {
24 |       maxLogLines: 5, // Limit number of lines logged per test
25 |       suppressErrorSummary: true, // Do not print error summary
26 |       suppressFailed: false, // Do not print information about failed tests
27 |       suppressPassed: false, // Do not print information about passed tests
28 |       suppressSkipped: true, // Do not print information about skipped tests
29 |       showSpecTiming: false, // Print the time elapsed for each spec
30 |       failFast: false, // Test would finish with error when a first fail occurs
31 |       suppressColor: true, // Ensure color is suppressed in specReporter
32 |       prefixes: {
33 |         success: 'SUCCESS-', // Override prefix for passed tests, default is '✓ '
34 |         failure: 'FAILED-', // Override prefix for failed tests, default is 'x '
35 |         skipped: 'SKIPPED-' // Override prefix for skipped tests, default is '- '
36 |       }
37 |     },
38 |     reporters: ['spec'],
39 |     progressReporter: {
40 |       showFailed: true,
41 |       showPassed: true // Corrected the typo here
42 |     },
```

The spec reporter prints **SUCCESS-** / **FAILED-** / **SKIPPED-** prefixes, hides skipped tests and limits log lines; `reporters: ['spec']`; the leftover `progressReporter` block.

Lines 43-63 | Port, browser, launcher and single run

```
43 |     port: 9876,  
44 |     logLevel: config.LOG_INFO,  
45 |     autoWatch: true,  
46 |     browsers: ['CustomChromeHeadless'],  
47 |     customLaunchers: {  
48 |       CustomChromeHeadless: {  
49 |         base: 'Chrome',  
50 |         flags: [  
51 |           '--headless',  
52 |           '--disable-gpu',  
53 |           '--remote-debugging-port=9222',  
54 |           '--no-sandbox',  
55 |           '--disable-setuid-sandbox'  
56 |         ],  
57 |         executablePath: '/usr/bin/chromium-browser'  
58 |       },  
59 |     },  
60 |     singleRun: true,  
61 |     restartOnFileChange: false  
62 |   });  
63 | };
```

Port 9876, info logging, the `CustomChromeHeadless` launcher with the headless and no-sandbox flags, and `singleRun: true`, which makes Karma exit after one run. The `executablePath` line is ignored by the launcher, as explained in the other copy.

Workflow, next: That is the whole official frontend harness. The backend has its own equivalent: first the test class itself, `SpringappApplicationTests.java`.

Step 148: SpringappApplicationTests - the official backend integration tests [Shared - whole team]

springapp/src/test/java/com/examly/springapp/SpringappApplicationTests.java

TEST

What this file is	The backend test class. It is an integration test: it starts the whole Spring Boot application (real security chain, real controllers and services, a real embedded Tomcat on a random port and a fresh in-memory H2 database), then talks to it over HTTP exactly like the Angular app would. We compared it with <code>junit/test/java/com/examly/springapp/SpringappApplicationTests.java</code> using <code>diff</code>: the two files are identical, so this is the predefined official test, not something we wrote. The file has 15 tests (<code>@Order(1)</code> to <code>@Order(15)</code>); a plain text search for <code>@Test</code> finds 17 matches only because <code>@TestMethodOrder</code> and <code>@TestInstance</code> also contain that text. The tests share state: the first four create two users and obtain two JWT tokens (an admin and a customer), and the later ones use those tokens to prove that URL security by role works (<code>SecurityConfig</code>) and that the main operations return the right HTTP status. We could not run them in this environment (Maven could not download dependencies, access to Maven Central was blocked), so the expected statuses below are read from the code and <code>SecurityConfig</code>, not from a live run.
Who / what triggers it	<code>mvn test</code> in <code>springapp</code> (run by <code>junit/junit.sh</code> during grading). Maven's Surefire plugin finds the class because its name ends in <code>Tests</code> .
What it uses / calls	Spring Boot Test (<code>@SpringBootTest</code> , <code>TestRestTemplate</code>), Jackson (<code>ObjectMapper</code> , <code>JsonNode</code>), JUnit 5 (<code>@Test</code> , <code>@Order</code> , <code>Assertions</code>), and indirectly every backend class: <code>AuthController</code> , <code>SecurityConfig</code> , <code>JwtUtils</code> , <code>DriverController</code> , <code>DriverRequestController</code> , <code>FeedbackController</code> and their services.
Size	488 lines

Line 1 | Package

```
1 | package com.examly.springapp;
```

The test lives in the same package as `SpringappApplication`, `com.examly.springapp`. That is why `@SpringBootTest` can automatically find the `@SpringBootApplication` class to start.

Lines 4-23 | Imports

```
4 | import com.fasterxml.jackson.databind.JsonNode;
5 | import com.fasterxml.jackson.databind.ObjectMapper;
6 | import org.junit.jupiter.api.Test;
7 | import org.junit.jupiter.api.TestInstance;
8 | import org.springframework.boot.test.context.SpringBootTest;
9 | import org.springframework.boot.test.web.client.TestRestTemplate;
10 | import java.net.URLEncoder;
11 | import java.nio.charset.StandardCharsets;
12 |
13 | import org.junit.jupiter.api.Assertions;
14 | import org.junit.jupiter.api.MethodOrderer;
15 | import org.junit.jupiter.api.Order;
16 | import org.junit.jupiter.api.TestMethodOrder;
17 | import org.springframework.beans.factory.annotation.Autowired;
18 | import org.springframework.http.HttpEntity;
19 | import org.springframework.http.HttpHeaders;
20 | import org.springframework.http.HttpMethod;
21 | import org.springframework.http.HttpStatus;
22 | import org.springframework.http.MediaType;
23 | import org.springframework.http.ResponseEntity;
```

`ObjectMapper` and `JsonNode` parse the JSON the server returns (to pull the token out of the login reply). `Test`, `TestInstance`, `TestMethodOrder`, `MethodOrderer` and `Order` are JUnit 5 annotations and helpers. `Assertions` provides `assertEquals`, `assertNotNull` and `assertTrue`. `SpringBootTest` starts the application. `TestRestTemplate` is a tiny HTTP client made for tests. `Autowired` asks Spring to inject objects. `HttpEntity`, `HttpHeaders`, `HttpMethod`, `HttpStatus`, `MediaType` and `ResponseEntity` describe requests and responses. `URLEncoder` and `StandardCharsets` are imported but never used (leftovers).

Lines 25-28 | Class-level annotations

```
25 | @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
26 | @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
27 | @TestInstance(TestInstance.Lifecycle.PER_CLASS)
28 | class SpringappApplicationTests {
```

`@SpringBootTest(webEnvironment = RANDOM_PORT)` starts the complete application once for this class, including real Tomcat on a **random free port** (so it never clashes with a server already running on 8080).

`@TestMethodOrder(OrderAnnotation.class)` makes JUnit run the methods in the order given by `@Order(n)` instead of an unpredictable order; the tests depend on each other, so the order is essential. `@TestInstance(PER_CLASS)` makes JUnit create **one** instance of the class for all tests instead of a new one per test. That is what lets the fields `userToken` and `adminToken` set in tests 3 and 4 still be there in test 5 onward. The class is package-private (no `public`), which JUnit 5 allows.

Lines 31-38 | Shared fields

```
31 |     private String userToken;
32 |     private String adminToken;
33 |
34 |     @Autowired
35 |     private TestRestTemplate restTemplate;
36 |
37 |     @Autowired
38 |     private ObjectMapper objectMapper;
```

`userToken` and `adminToken` hold the two JWTs. `@Autowired TestRestTemplate restTemplate` is the HTTP client, already configured by Spring to know the random port, so tests can use relative URLs like `/api/register`.

`@Autowired ObjectMapper objectMapper` is the application's own Jackson instance.

Lines 40-44 | createHeaders()

```
40 |     private HttpHeaders createHeaders() {
41 |         HttpHeaders headers = new HttpHeaders();
42 |         headers.setContentType(MediaType.APPLICATION_JSON);
43 |         return headers;
44 |     }
```

Returns a new `HttpHeaders` with `Content-Type: application/json`. Each test adds an `Authorization: Bearer ...` header on top when it needs one.

Lines 46-54 | Test 1: register the admin

```
46 |     @Test
47 |     @Order(1)
48 |     void backend_testRegisterAdmin() {
49 |         String requestBody = "{\"userId\": 1, \"email\": \"demoadmin@gmail.com\", \"password\": \"admin@1234\",
50 |         | \"username\": \"admin123\", \"userRole\": \"Admin\", \"mobileNumber\": \"9876543210\"}";
51 |         ResponseEntity<String> response = restTemplate.postForEntity("/api/register",
52 |         |         new HttpEntity<>(requestBody, createHeaders()), String.class);
53 |         Assertions.assertEquals(HttpStatus.CREATED, response.getStatusCode());
54 |     }
```

POST /api/register with no token (the endpoint is **permitAll**) and a JSON body: email **demoadmin@gmail.com**, password **admin@1234**, username **admin123**, role **Admin**, mobile **9876543210**. **postForEntity** sends it and returns a **ResponseEntity<String>**. Expected: **201 Created** (**AuthController.registerUser**). The primary key is generated by the database (**GenerationType.IDENTITY** in **User**), and the database starts empty, so the first user is meant to be id 1; the **userId** in the body is not what the later tests depend on.

Lines 56-64 | Test 2: register the customer

```
56 | @Test
57 | @Order(2)
58 | void backend_testRegisterCustomer() {
59 |     String requestBody = "{\"userId\": 2, \"email\": \"demouser@gmail.com\", \"password\": \"user@1234\", \"username\": \"user123\", \"userRole\": \"Customer\", \"mobileNumber\": \"1122334455\"}";
60 |     ResponseEntity<String> response = restTemplate.postForEntity("/api/register",
61 |         new HttpEntity<>(requestBody, createHeaders()), String.class);
62 |
63 |     Assertions.assertEquals(HttpStatus.CREATED, response.getStatusCode());
64 | }
```

Same call with **demouser@gmail.com**, password **user@1234**, role **Customer**. Expected: **201 Created**. This user gets id 2, an assumption later tests rely on (**/api/driverRequest/user/2**).

Lines 66-81 | Test 3: log in as admin and keep the token

```
66 | @Test
67 | @Order(3)
68 | void backend_testLoginAdmin() throws Exception {
69 |     String requestBody = "{\"email\": \"demoadmin@gmail.com\", \"password\": \"admin@1234\"}";
70 |
71 |     ResponseEntity<String> response = restTemplate.postForEntity("/api/login",
72 |         new HttpEntity<>(requestBody, createHeaders()), String.class);
73 |
74 |     Assertions.assertNotNull(response.getBody(), "Response body is null!");
75 |     JsonNode responseBody = objectMapper.readTree(response.getBody());
76 |     String token = responseBody.get("token").asText();
77 |     adminToken = token;
78 |
79 |     Assertions.assertEquals(HttpStatus.OK, response.getStatusCode());
80 |     Assertions.assertNotNull(token);
81 | }
```

POST /api/login with email and password only. Asserts the body is not null, parses it with **objectMapper.readTree**, reads the **token** field, stores it in **adminToken**, and asserts the status is **200 OK** and the token is not null. This is the same login reply (**token**, **username**, **userRole**, **userId**) the Angular login page uses.

Lines 83-97 | Test 4: log in as customer and keep the token

```
83 | @Test
84 | @Order(4)
85 | void backend_testLoginUser() throws Exception {
86 |     String requestBody = "{\"email\": \"demouser@gmail.com\", \"password\": \"user@1234\"}";
87 |
88 |     ResponseEntity<String> response = restTemplate.postForEntity("/api/login",
89 |         new HttpEntity<>(requestBody, createHeaders()), String.class);
90 |
91 |     JsonNode responseBody = objectMapper.readTree(response.getBody());
92 |     String token = responseBody.get("token").asText();
93 |     userToken = token;
94 |
95 |     Assertions.assertEquals(HttpStatus.OK, response.getStatusCode());
96 |     Assertions.assertNotNull(token);
97 | }
```

Same for the customer: expected **200 OK**, token stored in **userToken**.

Lines 99-114 | Test 5 setup: add a driver, admin versus customer

```
99 | @Test
100 | @Order(5)
101 | void backend_testAddDriverWithRoleValidation() throws Exception {
102 |     Assertions.assertNotNull(adminToken, "Admin token should not be null");
103 |     Assertions.assertNotNull(userToken, "User token should not be null");
104 |     String requestBody = "{"
105 |         + "\"driverName\": \"Rajesh Kumar\", \"
106 |         + "\"licenseNumber\": \"DL123456789\", \"
107 |         + "\"experienceYears\": 5, \"
108 |         + "\"contactNumber\": \"9876543210\", \"
109 |         + "\"availabilityStatus\": \"Available\", \"
110 |         + "\"address\": \"12, Gandhi Street, Chennai\", \"
111 |         + "\"vehicleType\": \"SUV\", \"
112 |         + "\"hourlyRate\": 350.0, \"
113 |         + "\"image\": \"base64encodedImageDataHere\" \"
114 |         + \"}";
```

First asserts both tokens exist (so a failure in tests 3 or 4 is reported clearly). Then builds the driver JSON: name **Rajesh Kumar**, license **DL123456789**, 5 years, contact number, status **Available**, address, vehicle **SUV**, hourly rate 350.0 and a fake image string.

Lines 117-129 | Test 5, admin part

```
117 | // Try with ADMIN (should be ALLOWED)
118 | HttpHeaders adminHeaders = createHeaders();
119 | adminHeaders.set("Authorization", "Bearer " + adminToken);
120 | HttpEntity<String> adminRequest = new HttpEntity<>(requestBody, adminHeaders);
121 |
122 | ResponseEntity<String> adminResponse = restTemplate.exchange(
123 |     "/api/driver", HttpMethod.POST, adminRequest, String.class);
124 |
125 | JsonNode adminJson = objectMapper.readTree(adminResponse.getBody());
126 |
127 | System.out.println(adminResponse.getStatusCode() + " Status code for Admin adding driver");
128 | Assertions.assertEquals(HttpStatus.CREATED, adminResponse.getStatusCode());
129 |
```

Adds **Authorization: Bearer <adminToken>** and sends **POST /api/driver** with **restTemplate.exchange(url, HttpMethod.POST, entity, String.class)** (**exchange** is the general method that allows any HTTP verb and headers). Parses the body (the variable **adminJson** is never used), prints the status and asserts **201 Created** (**DriverController.addDriver**). This is driver id 1.

Lines 131-141 | Test 5, customer part

```
131 | // Try with user (should be FORBIDDEN) //User
132 | HttpHeaders userHeaders = createHeaders();
133 | userHeaders.set("Authorization", "Bearer " + userToken);
134 | HttpEntity<String> userRequest = new HttpEntity<>(requestBody, userHeaders);
135 |
136 | ResponseEntity<String> userResponse = restTemplate.exchange(
137 |     "/api/driver", HttpMethod.POST, userRequest, String.class);
138 |
139 | System.out.println(userResponse.getStatusCode() + " Status code for User trying to add driver");
140 | Assertions.assertEquals(HttpStatus.FORBIDDEN, userResponse.getStatusCode());
141 | }
```

Sends the same body with the customer's token. **SecurityConfig** allows **POST /api/driver** for **ADMIN** only, so the request is stopped by Spring Security before reaching the controller. Asserts **403 Forbidden**.

Lines 143-146 | Test 6 start

```
143 | @Test
144 | @Order(6)
145 | void backend_testGetDriverById_AccessibleByAdmin() throws Exception {
146 |     Long driverId = 1L;
```

Test 6 checks **GET /api/driver/{id}** for driver id 1L.

Lines 148-161 | Test 6, admin part

```
148 | // trying with the admin( Should success)
149 | HttpHeaders adminHeaders = createHeaders();
150 | adminHeaders.set("Authorization", "Bearer " + adminToken);
151 | HttpEntity<Void> adminRequest = new HttpEntity<>(adminHeaders);
152 |
153 | ResponseEntity<String> adminResponse = restTemplate.exchange(
154 |     "/api/driver/" + driverId, HttpMethod.GET, adminRequest, String.class);
155 |
156 | JsonNode adminJson = objectMapper.readTree(adminResponse.getBody());
157 |
158 | System.out.println(
159 |     adminResponse.getStatusCode() + " Status code for admin fetching Driver");
160 | Assertions.assertEquals(HttpStatus.OK, adminResponse.getStatusCode());
161 | Assertions.assertEquals(driverId, adminJson.get("driverId").asLong());
```

With the admin token, **GET /api/driver/1** must return **200 OK** and a JSON whose **driverId** equals 1 (**DriverController.viewDriverById**).

Lines 163-178 | Test 6, customer part

```
163 | Assertions.assertNotNull(userToken, "User token should not be null");
164 | Assertions.assertNotNull(adminToken, "Admin token should not be null");
165 | HttpHeaders userHeaders = createHeaders();
166 | userHeaders.set("Authorization", "Bearer " + userToken);
167 | HttpEntity<Void> userRequest = new HttpEntity<>(userHeaders);
168 |
169 | ResponseEntity<String> userResponse = restTemplate.exchange(
170 |     "/api/driver/" + driverId, HttpMethod.GET, userRequest, String.class);
171 |
172 | // JsonNode userJson = objectMapper.readTree(userResponse.getBody());
173 |
174 | System.out.println(userResponse.getStatusCode() + " Status code for user fetching Driver");
175 | Assertions.assertEquals(HttpStatus.FORBIDDEN, userResponse.getStatusCode());
176 |
177 |
178 | }
```

With the customer token the same URL must return **403 Forbidden**, because `SecurityConfig` marks `GET /api/driver/**` as ADMIN only (only the list `GET /api/driver` is public). The line that would parse the user's body is commented out because there is no JSON in a 403.

Lines 180-197 | Test 7, customer lists all drivers

```
180 | @Test
181 | @Order(7)
182 | void backend_testGetAllDrivers_AccessibleByUserAndAdmin() throws Exception {
183 |     Assertions.assertNotNull(userToken, "User token should not be null");
184 |     Assertions.assertNotNull(adminToken, "Admin token should not be null");
185 |
186 |     // --- user tries to get all drivers ---
187 |     HttpHeaders userHeaders = createHeaders();
188 |     userHeaders.set("Authorization", "Bearer " + userToken);
189 |     HttpEntity<Void> userRequest = new HttpEntity<>(userHeaders);
190 |
191 |     ResponseEntity<String> userResponse = restTemplate.exchange(
192 |         "/api/driver", HttpMethod.GET, userRequest, String.class);
193 |
194 |     System.out.println(userResponse.getStatusCode() + " Status code for User fetching all drivers");
195 |     Assertions.assertEquals(HttpStatus.OK, userResponse.getStatusCode());
196 |     Assertions.assertTrue(userResponse.getBody().contains("driverId"),
197 |         "Response body should contain 'driverId'");
198 | }
```

`GET /api/driver` with the customer token. Expected **200 OK** and a body that contains the text `driverId` (so at least the driver from test 5 is listed, `DriverController.viewAllDrivers`). The list endpoint is public, so a customer is allowed.

Lines 199-211 | Test 7, admin lists all drivers

```
199 | // --- admin tries to get all drivers ---
200 | HttpHeaders adminHeaders = createHeaders();
201 | adminHeaders.set("Authorization", "Bearer " + adminToken);
202 | HttpEntity<Void> adminRequest = new HttpEntity<>(adminHeaders);
203 |
204 | ResponseEntity<String> adminResponse = restTemplate.exchange(
205 |     "/api/driver", HttpMethod.GET, adminRequest, String.class);
206 |
207 | System.out.println(adminResponse.getStatusCode() + " Status code for Admin fetching all driver");
208 | Assertions.assertEquals(HttpStatus.OK, adminResponse.getStatusCode());
209 | Assertions.assertTrue(adminResponse.getBody().contains("driverId"),
210 |     "Response body should contain 'driverId'");
211 | }
```

Same call with the admin token. Expected **200 OK** and the body contains `driverId`.

Lines 213-233 | Test 8 setup: update a driver

```
213 | @Test
214 | @Order(8)
215 | void backend_testUpdateDriver_AccessibleByAdminOnly() throws Exception {
216 |     Long driverId = 1L; // Use an existing Driver ID or create one before running the test
217 |
218 |     // Ensure tokens are available
219 |     Assertions.assertNotNull(adminToken, "Admin token should not be null");
220 |     Assertions.assertNotNull(userToken, "User token should not be null");
221 |
222 |     // Driver update request body // needs to change the request body
223 |     String requestBody = "{"
224 |         + "\"driverName\": \"Ravi Kumar updated\",",
225 |         + "\"licenseNumber\": \"DL04 2023 567890\",",
226 |         + "\"experienceYears\": 12,",
227 |         + "\"contactNumber\": \"9876543210\",",
228 |         + "\"availabilityStatus\": \"Active updated\",",
229 |         + "\"address\": \"123 MG Road, Bengaluru, Karnataka,updated\",",
230 |         + "\"vehicleType\": \"Sedan\",",
231 |         + "\"hourlyRate\": 350.50,",
232 |         + "\"image\": \"updatedsampleBase64EncodedImage==\""
233 |         + "}";
```

Targets driver id 1 and builds an update body: name **Ravi Kumar updated**, a new license, 12 years, new status text **Active updated** (the backend stores any text), new address, vehicle **Sedan**, rate 350.50 and a new image string.

Lines 235-248 | Test 8, admin part

```
235 | // --- Admin tries to update driver (should succeed) ---
236 | HttpHeaders adminHeaders = createHeaders();
237 | adminHeaders.set("Authorization", "Bearer " + adminToken);
238 | HttpEntity<String> adminRequest = new HttpEntity<>(requestBody, adminHeaders);
239 |
240 | ResponseEntity<String> adminResponse = restTemplate.exchange(
241 |     "/api/driver/" + driverId, HttpMethod.PUT, adminRequest, String.class);
242 |
243 | JsonNode adminJson = objectMapper.readTree(adminResponse.getBody());
244 |
245 | System.out.println(adminResponse.getStatusCode() + " Status code for Admin updating driver");
246 | Assertions.assertEquals(HttpStatus.OK, adminResponse.getStatusCode());
247 | Assertions.assertEquals("Ravi Kumar updated", adminJson.get("driverName").asText());
248 | Assertions.assertEquals(12, adminJson.get("experienceYears").asInt());
```

PUT /api/driver/1 with the admin token. Expected **200 OK**, and the returned JSON must have **driverName** equal to **Ravi Kumar updated** and **experienceYears** equal to 12 (**DriverController.updateDriver** returns the saved driver).

Lines 250-260 | Test 8, customer part

```
250 | // --- User tries to update driver (should be forbidden) ---
251 | HttpHeaders userHeaders = createHeaders();
252 | userHeaders.set("Authorization", "Bearer " + userToken);
253 | HttpEntity<String> userRequest = new HttpEntity<>(requestBody, userHeaders);
254 |
255 | ResponseEntity<String> userResponse = restTemplate.exchange(
256 |     "/api/driver/" + driverId, HttpMethod.PUT, userRequest, String.class);
257 |
258 | System.out.println(userResponse.getStatusCode() + " Status code for user trying to update driver");
259 | Assertions.assertEquals(HttpStatus.FORBIDDEN, userResponse.getStatusCode());
260 | }
```

Same PUT with the customer token: expected **403 Forbidden** (PUT on **/api/driver/**** is ADMIN only).

Lines 263-286 | Test 9 setup: create a driver request

```
263 | @Test
264 | @Order(9)
265 | void backend_testAddDriverRequests_AccessibleByuserOnly() throws Exception {
266 |     // Ensure tokens are available
267 |     Assertions.assertNotNull(userToken, "User token should not be null");
268 |     Assertions.assertNotNull(adminToken, "admin token should not be null");
269 |
270 |     // Updated driverRequest add request body
271 |     String requestBody = "{"
272 |         + "\"user\": { \"userId\": 2 },"
273 |         + "\"driver\": { \"driverId\": 1 },"
274 |         + "\"requestDate\": \"2024-05-13\","
275 |         + "\"status\": \"Pending\","
276 |         + "\"tripDate\": \"2024-05-15\","
277 |         + "\"timeSlot\": \"10:30:00\","
278 |         + "\"pickupLocation\": \"Indiranagar, Bengaluru\","
279 |         + "\"dropLocation\": \"Electronic City, Bengaluru\","
280 |         + "\"estimatedDuration\": \"2 hours\","
281 |         + "\"paymentAmount\": 450.0,"
282 |         + "\"comments\": \"Customer requested AC vehicle\","
283 |         + "\"actualDropTime\": \"12:45:00\","
284 |         + "\"actualDropDate\": \"2024-05-15\","
285 |         + "\"actualDuration\": \"2 hours 15 minutes\""
286 |         + "}";
```

Builds a request JSON with **nested** objects **user**: { **userId**: 2 } and **driver**: { **driverId**: 1 } plus request date, status **Pending**, trip date, time slot **10:30:00**, pickup and drop locations, estimated duration **2 hours**, payment amount, comments and actual-drop fields.

Lines 288-299 | Test 9, customer part

```
288 | // --- user tries to add driver request (should succeed) ---
289 | HttpHeaders userHeaders = createHeaders();
290 | userHeaders.set("Authorization", "Bearer " + userToken);
291 | HttpEntity<String> userRequest = new HttpEntity<>(requestBody, userHeaders);
292 |
293 | ResponseEntity<String> userResponse = restTemplate.exchange(
294 |     "/api/driverRequest", HttpMethod.POST, userRequest, String.class);
295 |
296 | JsonNode userJson = objectMapper.readTree(userResponse.getBody());
297 |
298 | System.out.println(userResponse.getStatusCode() + " Status code for user adding driverRequest");
299 | Assertions.assertEquals(HttpStatus.CREATED, userResponse.getStatusCode());
```

POST /api/driverRequest with the customer token. Expected **201 Created** (**DriverRequestController.addDriverRequest**). This is the request that later tests read.

Lines 301-313 | Test 9, admin part

```
301 | // --- Admin tries to add driver request (should be forbidden) ---
302 | HttpHeaders adminHeaders = createHeaders();
303 | adminHeaders.set("Authorization", "Bearer " + adminToken);
304 | HttpEntity<String> adminRequest = new HttpEntity<>(requestBody, adminHeaders);
305 |
306 | ResponseEntity<String> adminResponse = restTemplate.exchange(
307 |     "/api/driverRequest", HttpMethod.POST, adminRequest, String.class);
308 |
309 | System.out.println(
310 |     adminResponse.getStatusCode()
311 |     + " Status code for admin trying to add driverRequest");
312 | Assertions.assertEquals(HttpStatus.FORBIDDEN, adminResponse.getStatusCode());
313 | }
```

The same body with the admin token must give **403 Forbidden**: only a customer may create requests.

Lines 317-326 | Test 10 setup: requests of one user

```
317 | @Test
318 | @Order(10)
319 | void backend_testGetDriverRequestsByUserId_onlyCustomerAccess() throws Exception {
320 |     // Ensure tokens are available
321 |     Assertions.assertNotNull(userToken, "User token should not be null");
322 |     Assertions.assertNotNull(adminToken, "Admin token should not be null");
323 |
324 |     Long userId = 2L; // Assuming this user exists and is a Customer
325 |
326 |     String url = "/api/driverRequest/user/" + userId;
```

Checks **GET /api/driverRequest/user/2**. The URL is built from the user id **2L**. (This method is indented differently from its neighbours, which is only formatting.)

Lines 328-335 | Test 10, customer part

```
328 | // Test with Customer token (Expecting 200 OK)
329 | HttpHeaders userHeaders = createHeaders();
330 | userHeaders.set("Authorization", "Bearer " + userToken);
331 | HttpEntity<Void> userRequest = new HttpEntity<>(userHeaders);
332 |
333 | ResponseEntity<String> userResponse = restTemplate.exchange(url, HttpMethod.GET, userRequest, String.class);
334 | System.out.println(userResponse.getStatusCode() + " Status code for Customer fetching Driver Requests");
335 | Assertions.assertEquals(HttpStatus.OK, userResponse.getStatusCode());
```

With the customer token: **200 OK** (`viewDriverRequestsByUserId` finds the request created in test 9; had there been none, it would be 404).

Lines 337-345 | Test 10, admin part

```
337 | // Test with Admin token (Expecting 403 FORBIDDEN)
338 | HttpHeaders adminHeaders = createHeaders();
339 | adminHeaders.set("Authorization", "Bearer " + adminToken);
340 | HttpEntity<Void> adminRequest = new HttpEntity<>(adminHeaders);
341 |
342 | ResponseEntity<String> adminResponse = restTemplate.exchange(url, HttpMethod.GET, adminRequest, String.class);
343 | System.out.println(adminResponse.getStatusCode() + " Status code for Admin trying to fetch Driver Requests by User ID");
344 | Assertions.assertEquals(HttpStatus.FORBIDDEN, adminResponse.getStatusCode());
345 | }
```

With the admin token: **403 Forbidden**; `GET /api/driverRequest/user/**` is CUSTOMER only.

Lines 347-354 | Test 11 start: all requests, admin only

```
347 | @Test
348 | @Order(11)
349 | void backend_testGetAllDriverRequests_onlyAdminAccess() throws Exception {
350 |     // Ensure tokens are available
351 |     Assertions.assertNotNull(userToken, "User token should not be null");
352 |     Assertions.assertNotNull(adminToken, "Admin token should not be null");
353 |
354 |     String url = "/api/driverRequest";
```

Test 11 checks `GET /api/driverRequest` and stores the URL.

Lines 356-363 | Test 11, admin part

```
356 | // Test with Admin token (Expecting 200 OK)
357 | HttpHeaders adminHeaders = createHeaders();
358 | adminHeaders.set("Authorization", "Bearer " + adminToken);
359 | HttpEntity<Void> adminRequest = new HttpEntity<>(adminHeaders);
360 |
361 | ResponseEntity<String> adminResponse = restTemplate.exchange(url, HttpMethod.GET, adminRequest,
362 |     String.class);
363 | Assertions.assertEquals(HttpStatus.OK, adminResponse.getStatusCode());
```

With the admin token: **200 OK** (`viewAllDriverRequests`, which returns the request from test 9).

Lines 365-373 | Test 11, customer part

```
365 | // Test with user token (Expecting 403 FORBIDDEN)
366 | HttpHeaders userHeaders = createHeaders();
367 | userHeaders.set("Authorization", "Bearer " + userToken);
368 | HttpEntity<Void> userRequest = new HttpEntity<>(userHeaders);
369 |
370 | ResponseEntity<String> userResponse = restTemplate.exchange(url, HttpMethod.GET, userRequest,
371 |     String.class);
372 | Assertions.assertEquals(HttpStatus.FORBIDDEN, userResponse.getStatusCode());
373 | }
```

With the customer token: **403 Forbidden** (listing every request is ADMIN only).

Lines 375-388 | Test 12 setup: post feedback

```
375 | @Test
376 | @Order(12)
377 | void backend_testAddFeedback() throws Exception {
378 |     Assertions.assertNotNull(userToken, "User token should not be null");
379 |     Assertions.assertNotNull(adminToken, "Admin token should not be null");
380 |
381 |     String requestBody = "{"
382 |         + "\"feedbackText\": \"The driver was punctual and polite.\",\"
383 |         + "\"date\": \"2024-05-13\",\"
384 |         + "\"user\": { \"userId\": 2 },\"
385 |         + "\"driver\": { \"driverId\": 1 },\"
386 |         + "\"category\": \"Driver Performance\",\"
387 |         + "\"rating\": 5\"
388 |         + "}";
```

Builds a feedback body: text `The driver was punctual and polite.`, a date, nested `user` id 2 and `driver` id 1, category `Driver Performance` and rating 5. There is no sentiment in it: `FeedbackServiceImpl.createFeedback` calls `AIService.applyAnalysis` to add it (the local keyword logic, since no Gemini key is set).

Lines 392-400 | Test 12, customer part

```
392 | // user should be able to add feedback
393 | HttpHeaders userHeaders = createHeaders();
394 | userHeaders.set("Authorization", "Bearer " + userToken);
395 | HttpEntity<String> userRequest = new HttpEntity<>(requestBody, userHeaders);
396 | ResponseEntity<String> userResponse = restTemplate.exchange("/api/feedback", HttpMethod.POST,
397 |     userRequest,
398 |     String.class);
399 | Assertions.assertEquals(HttpStatus.CREATED, userResponse.getStatusCode(),
400 |     "User should be able to add feedback");
```

POST `/api/feedback` with the customer token. Expected **201 Created**, with the message `User should be able to add feedback` shown if the assertion fails.

Lines 402-410 | Test 12, admin part

```
402 | // admin should NOT be able to add feedback
403 | HttpHeaders adminHeaders = createHeaders();
404 | adminHeaders.set("Authorization", "Bearer " + adminToken);
405 | HttpEntity<String> adminRequest = new HttpEntity<>(requestBody, adminHeaders);
406 | ResponseEntity<String> adminResponse = restTemplate.exchange("/api/feedback", HttpMethod.POST,
407 |     adminRequest,
408 |     String.class);
409 | Assertions.assertEquals(HttpStatus.FORBIDDEN, adminResponse.getStatusCode());
410 | }
```

With the admin token: **403 Forbidden**. Only customers can post feedback. Since this second POST is blocked, exactly one feedback exists afterwards, with id 1.

Lines 412-416 | Test 13 start: list all feedback

```
412 | @Test
413 | @Order(13)
414 | void backend_testGetAllFeedback() throws Exception {
415 |     Assertions.assertNotNull(userToken, "User token should not be null");
416 |     Assertions.assertNotNull(adminToken, "Admin token should not be null");
```

Test 13 checks **GET** `/api/feedback`.

Lines 418-426 | Test 13, admin part

```
418 | // admin should be able to get all feedback
419 | HttpHeaders adminHeaders = createHeaders();
420 | adminHeaders.set("Authorization", "Bearer " + adminToken);
421 | HttpEntity<Void> adminRequest = new HttpEntity<>(adminHeaders);
422 | ResponseEntity<String> adminResponse = restTemplate.exchange("/api/feedback", HttpMethod.GET,
423 |     adminRequest,
424 |     String.class);
425 | Assertions.assertEquals(HttpStatus.OK, adminResponse.getStatusCode(),
426 |     "admin should be able to view all feedback");
```

With the admin token: **200 OK**.

Lines 428-437 | Test 13, customer part

```
428 | // user should be able to get all feedback
429 | HttpHeaders userHeaders = createHeaders();
430 | userHeaders.set("Authorization", "Bearer " + userToken);
431 | HttpEntity<Void> userRequest = new HttpEntity<>(userHeaders);
432 | ResponseEntity<String> userResponse = restTemplate.exchange("/api/feedback", HttpMethod.GET,
433 |     userRequest,
434 |     String.class);
435 | Assertions.assertEquals(HttpStatus.OK, userResponse.getStatusCode(),
436 |     "user should be able to view all feedback");
437 | }
```

With the customer token: **200 OK**. In `SecurityConfig` this endpoint is `permitAll`, so even a request without any token would succeed; the test checks only the two roles.

Lines 439-443 | Test 14 start: feedback of one user

```
439 | @Test
440 | @Order(14)
441 | void backend_testGetFeedbackById_onlyUserAccess() throws Exception {
442 |     Assertions.assertNotNull(userToken, "User token should not be null");
443 |     Assertions.assertNotNull(adminToken, "Admin token should not be null");
```

Test 14 checks **GET** `/api/feedback/user/2`.

Lines 445-452 | Test 14, customer part

```
445 | // user should be able to get their own feedback
446 | HttpHeaders userHeaders = createHeaders();
447 | userHeaders.set("Authorization", "Bearer " + userToken);
448 | HttpEntity<Void> userRequest = new HttpEntity<>(userHeaders);
449 | ResponseEntity<String> userResponse = restTemplate.exchange("/api/feedback/user/2", HttpMethod.GET,
450 |     userRequest,
451 |     String.class);
452 | Assertions.assertEquals(HttpStatus.OK, userResponse.getStatusCode());
```

With the customer token: **200 OK** (the feedback from test 12 belongs to user 2).

Lines 454-461 | Test 14, admin part

```
454 | // admin should NOT be able to get user feedback
455 | HttpHeaders adminHeaders = createHeaders();
456 | adminHeaders.set("Authorization", "Bearer " + adminToken);
457 | HttpEntity<Void> adminRequest = new HttpEntity<>(adminHeaders);
458 | ResponseEntity<String> adminResponse = restTemplate.exchange("/api/feedback/user/2", HttpMethod.GET,
459 |     adminRequest, String.class);
460 | Assertions.assertEquals(HttpStatus.FORBIDDEN, adminResponse.getStatusCode());
461 | }
```

With the admin token: **403 Forbidden** (this URL is CUSTOMER only).

Lines 463-467 | Test 15 start: feedback by id

```
463 | @Test
464 | @Order(15)
465 | void backend_testGetFeedbackById_userAndAdminAccess() throws Exception {
466 |     Assertions.assertNotNull(userToken, "User token should not be null");
467 |     Assertions.assertNotNull(adminToken, "Admin token should not be null");
```

Test 15 checks **GET /api/feedback/1**, the feedback created in test 12.

Lines 469-476 | Test 15, customer part

```
469 | // user should be able to get feedback by ID
470 | HttpHeaders userHeaders = createHeaders();
471 | userHeaders.set("Authorization", "Bearer " + userToken);
472 | HttpEntity<Void> userRequest = new HttpEntity<>(userHeaders);
473 | ResponseEntity<String> userResponse = restTemplate.exchange("/api/feedback/1", HttpMethod.GET,
474 |     userRequest,
475 |     String.class);
476 | Assertions.assertEquals(HttpStatus.OK, userResponse.getStatusCode());
```

With the customer token: **200 OK**. **SecurityConfig** requires only a logged-in user for **GET /api/feedback/****.

Lines 478-486 | Test 15, admin part

```
478 | // admin should be able to get feedback by ID
479 | HttpHeaders adminHeaders = createHeaders();
480 | adminHeaders.set("Authorization", "Bearer " + adminToken);
481 | HttpEntity<Void> adminRequest = new HttpEntity<>(adminHeaders);
482 | ResponseEntity<String> adminResponse = restTemplate.exchange("/api/feedback/1", HttpMethod.GET,
483 |     adminRequest,
484 |     String.class);
485 | Assertions.assertEquals(HttpStatus.OK, adminResponse.getStatusCode());
486 | }
```

With the admin token: **200 OK** (any authenticated user may read one feedback).

Line 488 | End of class

```
488 | }
```

Closes the class. When all 15 tests pass, **mvn test** reports 15 tests run, 0 failures.

Workflow, next: Next is the shell script that runs this class during grading, **junit/junit.sh**.

Step 149: junit.sh - the official backend grading script

[Shared - whole team]

junit/junit.sh

TEST

What this file is	The backend counterpart of <code>karma.sh</code> : it puts the official test class into <code>springapp</code> , runs Maven's tests and then prints a pass/fail summary. We compared <code>junit/test/java/com/examly/springapp/SpringappApplicationTests.java</code> with the one in <code>springapp/src/test/...</code> using <code>diff</code> and they are identical , so running this script changes nothing in the file. The folder <code>junit/Report</code> holds a small Node program (<code>customReport.js</code> and a <code>package.json</code> that depends on <code>xml2js</code>) which reads the XML files Maven writes in <code>springapp/target/surefire-reports</code> , prints one line <code>Passed <test name></code> or <code>Failed <test name></code> per test, and a final line with total, passed and failed counts.
Who / what triggers it	The grading system runs it. The paths belong to the grading machine (<code>/home/coder/project/workspace</code>).
What it uses / calls	Maven (<code>mvn -q test</code>), the folders <code>junit/test</code> and <code>junit/Report</code> , <code>nvm</code> with Node 14, <code>npm</code> .
Size	20 lines

Lines 1-2 | Shebang and cleaning old reports

```
1 | #!/bin/bash
2 | rm -rf /home/coder/project/workspace/springapp/target/surefire-reports;
```

Runs with Bash and deletes the previous `target/surefire-reports` folder so old results cannot be mistaken for new ones.

Lines 3-12 | Copy the official test and run Maven

```
3 | src_directory="/home/coder/project/workspace/springapp/src"
4 |
5 | if [ -d "$src_directory" ]; then
6 |     cp -r /home/coder/project/workspace/junit/test "$src_directory"
7 |     cd /home/coder/project/workspace/springapp;
8 |     mvn -q test
9 | else
10 |
11 |     echo "The 'src' folder does not exist in"
12 | fi
```

Stores the `src` path in a variable. If `springapp/src` exists, `cp -r junit/test src` copies the official `test` tree into it (creating or overwriting `src/test/java/com/examly/springapp/SpringappApplicationTests.java`), changes into `springapp` and runs `mvn -q test` (`-q` is quiet mode). Surefire starts the Spring application, runs the 15 tests and writes XML reports. If `src` does not exist, a message is printed instead.

Lines 14-15 | Copy the report tool

```
14 | cp -r /home/coder/project/workspace/junit/Report /home/coder/project/workspace;
15 | cd /home/coder/project/workspace/Report;
```

Copies the `Report` folder next to the project and enters it.

Lines 16-19 | Install and run the report

```
16 | source /usr/local/nvm/nvm.sh
17 | nvm use 14
18 | npm i
19 | node customReport.js
```

Loads `nvm`, switches to Node 14, installs `xml2js` with `npm i` and runs `node customReport.js`, which prints the pass/fail lines and the totals the grader reads.

Line 20 | Clean-up line (contains a typo)

```
20 | rm -rf /home/coder/project/workspace/Report;
```

`rm -rf ../Report;customReport.js` was probably meant to delete the folder and the script, but the semicolon makes the shell try to **run** a command named `customReport.js`, which does not exist, so Bash prints a harmless "command not found". Nothing breaks, because the report was already printed on the previous line.

Workflow, next: Both grading harnesses are explained. The remaining files are about continuous integration on GitHub, starting with `.github/workflows/build.yml`.

Step 150: build.yml - GitHub Actions workflow from the template [Shared - whole team]

CONFIG

`.github/workflows/build.yml`

What this file is	A GitHub Actions workflow. If this repository is pushed to GitHub, GitHub reads every file in <code>.github/workflows/</code> and runs it on its servers. This one builds the project and sends a code-quality report to a SonarQube server (a tool that scans source code for bugs and code smells). It came with the original project template of the assessment platform; it is not part of the DriveU application, and nothing in the app, in <code>mvn spring-boot:run</code> or in <code>ng serve</code> reads it. Important and honest: the file contains literal values called <code>GITHUB_TOKEN</code> and <code>SONAR_TOKEN</code> and a Sonar project key. They come from the original template, belong to the platform's own Sonar server (<code>sonar.server.examly.io</code>), and are not used by DriveU . Putting a token directly in a workflow file is bad practice (it should be a repository secret), so do not copy this pattern into a project of your own and do not publish those values elsewhere.
Who / what triggers it	GitHub, on every push to any branch and on pull requests that are opened, synchronized or reopened. It never runs on your computer.
What it uses / calls	GitHub-hosted <code>ubuntu-latest</code> runner, JDK 17, Maven, the Sonar scanner plugin, <code>.github/workflows/pom.xml</code> and repository secrets <code>SONAR_HOST_URL</code> , <code>API_URL</code> and <code>API_SECRET</code> (which are not defined in this repository).
Size	53 lines

Lines 1-10 | Name and triggers

```
1 | name: Build
2 | 'on':
3 |   push:
4 |     branches:
5 |       - '*'
6 |   pull_request:
7 |     types:
8 |       - opened
9 |       - synchronize
10 |      - reopened
```

`name: Build` is the label shown in GitHub. `'on':` (quoted because plain `on` can be read as the boolean `true` in YAML) lists the events: `push` on every branch (`'*'`) and `pull_request` when it is `opened`, `synchronize` (new commits pushed) or `reopened`.

Lines 11-18 | Job and checkout

```
11 | jobs:
12 |   build:
13 |     name: Build
14 |     runs-on: ubuntu-latest
15 |     steps:
16 |       - uses: actions/checkout@v2
17 |         with:
18 |           fetch-depth: 0
```

One job called `build` runs on a fresh Ubuntu machine. The first step `actions/checkout@v2` downloads the repository; `fetch-depth: 0` fetches the **full** git history, which Sonar needs to work out who changed what.

Lines 19-22 | Install JDK 17

```
19 |       - name: Set up JDK 17
20 |         uses: actions/setup-java@v1
21 |         with:
22 |           java-version: 17
```

`actions/setup-java@v1` installs Java 17 on the runner.

Lines 23-24 | Move the workflow pom to the root

```
23 |       - name: Move Pom.xml to Root
24 |         run: mv .github/workflows/pom.xml ./pom.xml
```

`mv .github/workflows/pom.xml ./pom.xml` moves the small POM stored in the workflows folder to the repository root, so that Maven has a `pom.xml` to work with (the real project POM is in `springapp/`). See the next entry for what

it contains.

Lines 25-35 | Build and analyze with Sonar

```
25 | - name: Build and analyze
26 |   env:
27 |     GITHUB_TOKEN: >-
28 |     <id-masked>-<id-masked>
29 |     SONAR_TOKEN: sqa_<secret-masked>
30 |     SONAR_HOST_URL: '${{ secrets.SONAR_HOST_URL }}'
31 |   run: >-
32 |     mvn -B verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar
33 |     -Dsonar.projectKey=iamneo-production-2_<id-masked>-<id-masked>
34 |     -Dsonar.login=sqa_<secret-masked>
35 |     -Dsonar.host.url=$SONAR_HOST_URL
```

Defines environment variables for this step: `GITHUB_TOKEN` and `SONAR_TOKEN` (literal values from the template, see above) and `SONAR_HOST_URL` (taken from a repository secret). The command `mvn -B verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar` runs Maven in batch mode (`-B`), then the Sonar plugin, with `-Dsonar.projectKey`, `-Dsonar.login` and `-Dsonar.host.url` passed as options. The `>-` after `run:` is YAML syntax for a long text folded into one line.

Lines 36-53 | Trigger API step

```
36 | - name: Trigger API
37 |   if: always()
38 |   env:
39 |     API_URL: '${{ secrets.API_URL }}'
40 |     API_SECRET: '${{ secrets.API_SECRET }}'
41 |   run: >
42 |     REPO_NAME=$(echo "${GITHUB_REPOSITORY}" | cut -d '/' -f 2)
43 |
44 |     ORG_NAME=$(echo "${GITHUB_REPOSITORY}" | cut -d '/' -f 1)
45 |
46 |     BRANCH_NAME=${GITHUB_REF#refs/heads/}
47 |
48 |     API_PAYLOAD="{ \"repo_id\": \"${REPO_NAME}\", \"branch_name\":
49 |     \"${BRANCH_NAME}\", \"api_secret\": \"${API_SECRET}\", \"org_name\":
50 |     \"${ORG_NAME}\" }"
51 |
52 |     curl -X POST -H "Content-Type: application/json" -d "$API_PAYLOAD"
53 |     $API_URL
```

`if: always()` makes this step run even when the build failed. It reads secrets `API_URL` and `API_SECRET`, derives the repository name, organisation and branch name from GitHub's built-in variables with `cut` and a `${GITHUB_REF#refs/heads/}` trick, builds a small JSON payload and sends it with `curl -X POST` to the platform's API, which is how the assessment platform learns that a build finished. The blank lines inside the folded `run: >` block make each shell command start on its own line.

Workflow, next: The workflow moves a POM to the root before building; that file is next: [.github/workflows/pom.xml](#).

Step 151: workflows/pom.xml - tiny Maven file for the Sonar run [Shared - whole team]

CONFIG

[.github/workflows/pom.xml](#)

What this file is	A minimal Maven project description that <code>build.yml</code> copies to the repository root. It is not the pom of the DriveU backend (that is <code>springapp/pom.xml</code>). Its job is to hold the Sonar settings: which folders to scan and where the Sonar server is. Its group id (<code>com.houari</code>) and artifact id (the long platform id) come from the original template and have nothing to do with DriveU.
Who / what triggers it	Moved to the root by the <code>Move Pom.xml to Root</code> step of <code>build.yml</code> , then read by <code>mvn -B verify ...sonar</code> . It only ever runs on GitHub.
What it uses / calls	The Sonar Maven plugin 3.7.0.1746 and JUnit Jupiter API 5.7.0 (test scope).
Size	34 lines

Lines 1-2 | XML header and project element

```
1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLoc
  | ation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

Standard XML declaration and the `<project>` element with the Maven schema.

Lines 3-6 | Coordinates

```
3 | <modelVersion>4.0.0</modelVersion>
4 | <groupId>com.houari</groupId>
5 | <artifactId>iamneo-production-2<id-masked>-<id-masked></artifactId>
6 | <version>1.0-SNAPSHOT</version>
```

`modelVersion` 4.0.0, group id `com.houari`, an artifact id that is the template's long project id and version `1.0-SNAPSHOT`.

Lines 7-14 | Properties

```
7 | <properties>
8 |   <maven.compiler.source>8</maven.compiler.source>
9 |   <maven.compiler.target>8</maven.compiler.target>
10 |   <sonar.organization>iamneo-production-2</sonar.organization>
11 |   <sonar.host.url>https://sonar.server.examly.io</sonar.host.url>
12 |   <sonar.sources>springapp/src/main,angularapp/src</sonar.sources>
13 |   <sonar.java.binaries>target</sonar.java.binaries>
14 | </properties>
```

The compiler is set to Java **8** (`maven.compiler.source` and `target`), which differs from the backend's Java 17, but since this root project contains no code to compile it does not matter. The `sonar.organization`, `sonar.host.url` (the platform's server) and `sonar.sources` (`springapp/src/main,angularapp/src`, the folders Sonar should analyse) tell the plugin what to scan. `sonar.java.binaries` says where compiled classes would be.

Lines 15-22 | Dependency

```
15 | <dependencies>
16 |   <dependency>
17 |     <groupId>org.junit.jupiter</groupId>
18 |     <artifactId>junit-jupiter-api</artifactId>
19 |     <version>5.7.0</version>
20 |     <scope>test</scope>
21 |   </dependency>
22 | </dependencies>
```

A single test-scope dependency on `junit-jupiter-api` 5.7.0. It is there so the build has something to resolve; no tests exist at the root.

Lines 23-34 | Build plugin version

```
23 | <build>
24 |   <pluginManagement>
25 |     <plugins>
26 |       <plugin>
27 |         <groupId>org.sonarsource.scanner.maven</groupId>
28 |         <artifactId>sonar-maven-plugin</artifactId>
29 |         <version>3.7.0.1746</version>
30 |       </plugin>
31 |     </plugins>
32 |   </pluginManagement>
33 | </build>
34 | </project>
```

`pluginManagement` fixes the version of `sonar-maven-plugin` to 3.7.0.1746 so the workflow always uses the same scanner. Lines 33-34 close `<build>` and `<project>`.

Workflow, next: The last two files are leftovers: `README.md` at the project root.

Step 152: README.md - platform stub, not a real README [Shared - whole team]

CONFIG

README.md

What this file is	A two-line file generated by the assessment platform. It is not documentation of DriveU: line 1 is the platform's project id as a Markdown heading and line 2 is a link to the platform's SonarQube dashboard for that id. It contains no instructions on how to run or use the app (this walk-through is the closest thing to one).
Who / what triggers it	Nothing runs it; GitHub shows it as the front page of the repository.
What it uses / calls	Nothing.

Size	2 lines
------	---------

Line 1 | Heading with the project id

```
1 | # <id-masked>-<id-masked>
```

A Markdown heading (#) that is just the template's unique project id.

Line 2 | Sonar dashboard link

```
2 | https://sonar.server.examly.io/dashboard?id=iamneo-production-2_<id-masked>-<id-masked>&amp;codeScope=overall
```

A URL to the platform's Sonar dashboard. The `&` is an HTML-escaped ampersand that the platform wrote into the link; it works as a separator in a browser. The link belongs to the template and is not part of the application.

Workflow, next: One more leftover file remains in the front-end folder: `angularapp/db.json`.

Step 153: db.json - leftover json-server file (not used by DriveU) [Shared - whole team]

CONFIG

`angularapp/db.json`

What this file is	A data file for json-server , a tool that turns a JSON file into a fake REST API. It comes from a different template, a train booking project: it holds three trains with names, numbers, operators, source and destination, type, coaches and pantry flag. DriveU does not use it . DriveU has no json-server, no trains data, and its Angular services call the Spring Boot backend on port 8080 instead (<code>apiconfig.ts</code>). It is safe to delete, but we left it unchanged. The same file is also present in the <code>karma/angularapp</code> skeleton.
Who / what triggers it	Nothing. No script in <code>package.json</code> starts json-server and no code reads this file.
What it uses / calls	Nothing in DriveU.
Size	38 lines

Lines 1-2 | Opening and the trains collection

```
1 | {
2 |   "trains": [
```

The outer `{` opens the JSON object and `"trains": [` starts an array. json-server would expose it as `/trains`.

Lines 3-13 | Train 1

```
3 |     {
4 |       "id": 1,
5 |       "trainName": "Express 101",
6 |       "trainNumber": "EX101",
7 |       "operator": "Indian Railways",
8 |       "source": "City A",
9 |       "destination": "City B",
10 |      "trainType": "Superfast",
11 |      "totalCoaches": 12,
12 |      "pantryAvailable": true
13 |    },
```

Id 1, **Express 101**, number **EX101**, operator **Indian Railways**, from **City A** to **City B**, type **Superfast**, 12 coaches, pantry available (**true**).

Lines 14-24 | Train 2

```
14 |     {
15 |       "id": 2,
16 |       "trainName": "Mail 202",
17 |       "trainNumber": "ML202",
18 |       "operator": "Southern Railways",
19 |       "source": "City X",
20 |       "destination": "City Y",
21 |       "trainType": "Mail",
22 |       "totalCoaches": 10,
23 |       "pantryAvailable": false
24 |    },
```

Id 2, **Mail 202**, **ML202**, operator **Southern Railways**, from **City X** to **City Y**, type **Mail**, 10 coaches, no pantry.

Lines 25-35 | Train 3

```
25 |     {  
26 |         "id": 3,  
27 |         "trainName": "Local 303",  
28 |         "trainNumber": "LC303",  
29 |         "operator": "Metro Rail",  
30 |         "source": "Station A",  
31 |         "destination": "Station Z",  
32 |         "trainType": "Local",  
33 |         "totalCoaches": 8,  
34 |         "pantryAvailable": false  
35 |     }
```

Id 3, Local 303, LC303, operator Metro Rail, Station A to Station Z, type Local, 8 coaches, no pantry.

Lines 37-38 | Closing

```
37 |     ]  
38 | }
```

Closes the array and the object.

Workflow, next: This is the end of the walk-through. To see how everything fits together: start the backend with `mvn spring-boot:run` in `springapp` (Maven reads `pom.xml` and `application.properties`, `SpringappApplication.main` boots Spring, Hibernate creates the tables for the entities, the security chain and controllers start answering on port 8080). Then start the frontend with `ng serve --port 8081` in `angularapp` (the build reads `package.json` and `angular.json`, `main.ts` boots `AppModule`, routing and the guard decide which page to show). Finally open the browser at port 8081: every page calls a service, the service sends an HTTP request with the JWT, `SecurityConfig` checks the role, a controller and service use the repository to read or write the H2 database, and the JSON travels back to the page. The tests in this appendix verify that chain from the browser side (Karma and Jasmine) and from the server side (JUnit), and the build files only matter to the grader. That is the whole project, and the walk-through ends here.

Part 5 - Observations and presentation tips

This part collects what we found while reading every line. None of it breaks the official tests, but a teacher or reviewer may ask about it, so it is better that the team knows.

A. Sprint document items that are NOT implemented

- **Booking concurrency / double-booking** (Arjun's advanced scope):
`DriverRequestServiceImpl.addDriverRequest` only sets `requestDate` when empty and saves. There is no overlap check, no locking, no queueing. The only guards are in Angular: the Request button needs an **Active** driver the customer has no open request for.
- **Payment gateway** (Prajan): the "payment" is a fare calculation in the browser (`endTrip`) plus a summary popup. There is no gateway and the server trusts the `paymentAmount` it receives.
- **Status transition rules:** Approved <-> Rejected, Trip End only after Approved, Closed only after Trip End are enforced by which **buttons** Angular shows, not by the server. A direct API call could set any status.

B. Security observations

- **Anyone can register as Admin:** `createUser` does not restrict `userRole`.
- **No ownership checks:** any logged-in user can GET or PUT any request by id; any customer can delete any feedback by id.
- **GET /api/feedback is public** (as the SRS says) and each item carries the nested `user`, so the writer's email and mobile number are visible. The password is hidden by `@JsonProperty(WRITE_ONLY)`.
- **JWT secret is hard-coded** in `JwtUtils`; catch blocks there are empty, so a rejected token leaves no log; logout only clears the browser's storage, the token stays valid on the server for 24 hours.
- **CORS** allows every origin with credentials (fine for a college project, too open for production).
- `UserPrinciple.build` would throw a `NullPointerException` for a user stored with no role.

C. Behaviour worth knowing

- Controllers `addDriver`, `addDriverRequest` and `createFeedback` catch every `Exception` and answer **409** with no body; so a duplicate license on add never reaches `GlobalExceptionHandler` (no `ErrorLog` row). Only the two deletion exceptions do.
- `updateDriver` overwrites **all** fields: a body without `image` erases the stored picture; the license number is not re-checked for uniqueness.
- Empty "all" lists answer **204** but "by user/driver" lists answer **404**; Angular handles both in its `error/next` branches.
- `HttpErrorInterceptor` ignores `/api/login`, `/api/register` and `/api/ai/`, so a backend that is down during login/signup shows no message (the user simply stays on the page).
- `parseDurationToMinutes` reads only the first number, so "2 hours 30 minutes" is read as 2 minutes (the text contains "min"); `formatDuration` always prints "hours" (even "1 hours").
- After a backend restart the in-memory H2 database is empty, so old browser tokens belong to users that no longer exist (401); log in again.
- `ErrorLogs` is write-only: nothing in the API reads it; use the H2 console to see rows.

D. Leftover / unused code

- `UserService.loginUser`, `analyze-existing` endpoint (no Angular page calls it), `AuthGuardGuard` alias export, `userId$`, `@angular/animations`, `puppeteer` in dependencies, `environment.production`.
- `polyfills.ts` and `test.ts` are not referenced by `angular.json`; `db.json` is a train-data file from another template; `SecurityConfig` has a misleading `// Public` comment above a rule that needs a login.
- `adminnav` and `customernav` are copies of each other.
- `junit.sh` line 20 has a typo (`...Report;customReport.js`) that only prints "command not found".

E. Test harness facts

- With the project as it is, `ng test` reports **8 of 89 specs executed** (the 8 official `fit` specs pass; the other 81 are skipped because `fit` focuses). With `fit` changed to `it` in a scratch copy, all 89 passed.

- The JUnit class has 15 tests. Maven could not be run in the preparation environment (the Maven Central repository was unreachable), so the backend was verified by reading the code, not by a live run.

F. A 2-minute demo script

1. Register an Admin and a Customer (</signup>), then log in as Admin.
2. Add a driver (photo, license, rate) - show the duplicate-license message by adding it twice.
3. Log out, log in as Customer. Open **Available Drivers**, try AI Search ("experienced SUV driver"), press **Request**, fill the form.
4. Log in as Admin: **View Requests** -> **Approve**; open **View Stage**.
5. As Customer: **My Requests** -> **Trip End** -> **Fetch Pay Amount** -> **Write a Review**.
6. As Admin: **Close Request** from the stage popup; open **View Feedback**, filter by sentiment, open **View Driver Info** for the AI summary.